Artificial Intelligence

ebook

Dr. Napoleon H. Reyes
n.h.reyes@massey.ac.nz

Reference: **MIT OpenCourseWare**

Table of Contents: ANY-PATH SEARCH

- State Space Search
- Depth-First Search
- Breadth-First Search
- Best-First Search
- Progressive Deepening Search

- Tutorial Questions
- Tutorial Solutions

Table of Contents: OPTIMAL SEARCH

- Uniform Cost Search (UC)
- A* Search
- A* and UC with Expanded List
- A* with Non-Strict Expanded List

- Tutorial Questions
- Tutorial Solutions

STATE-SPACE SEARCH

Motivation and Background

Uninformed Search, Informed Search, Any-Path Search, Optimal Search

n.h.reyes@massey.ac.nz

Reference: **MIT OpenCourseWare**

SEARCH

Background and Motivation

General Idea: Search allows exploring alternatives.

These algorithms provide the conceptual backbone of almost every approach to systematic exploration of alternatives.

Topics for Discussion:

- Background
- Uninformed vs. Informed Searches
- Any Path vs. Optimal Path
- Implementation and Performance

SEARCH

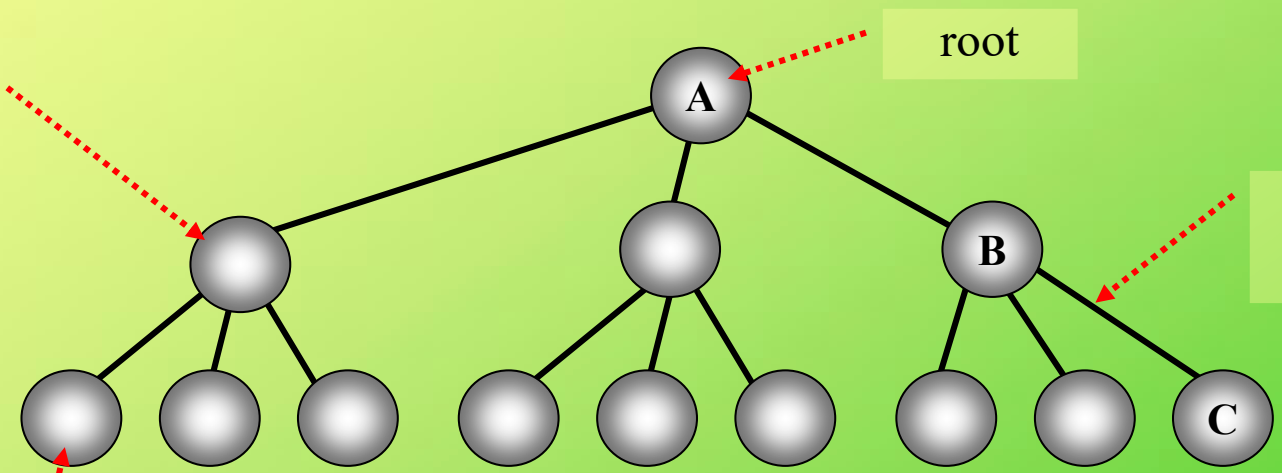
Topics for Discussion

- Basic principles and terminologies
- Graphs vs. Trees
- How to convert a graph into a tree
- Requirements for applying search
- Any-Path Search (Uninformed): Depth-First Search
- Any-Path Search (Uninformed): Breadth-First Search
- Any-Path Search (Informed): Best-First Search
- Hand-simulations

Background and Motivation

Trees and Graphs

No loops are allowed!

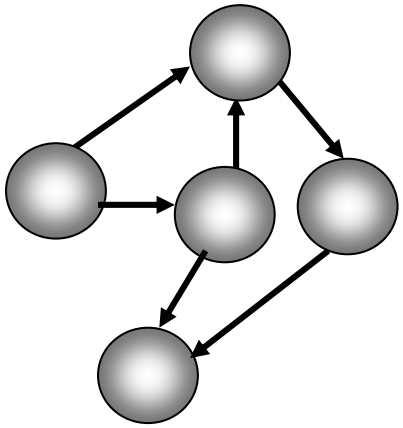


B is parent of C
C is a child of B
A is an ancestor of C
C is descendant of A

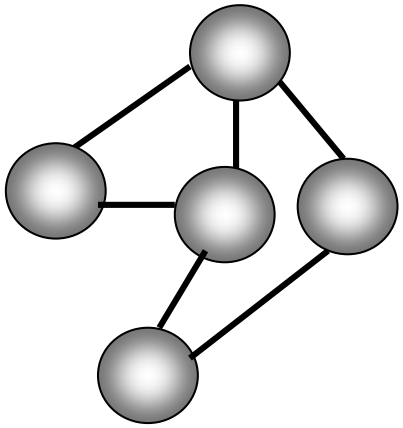
back

Background and Motivation

Directed Graphs
(one-way streets)



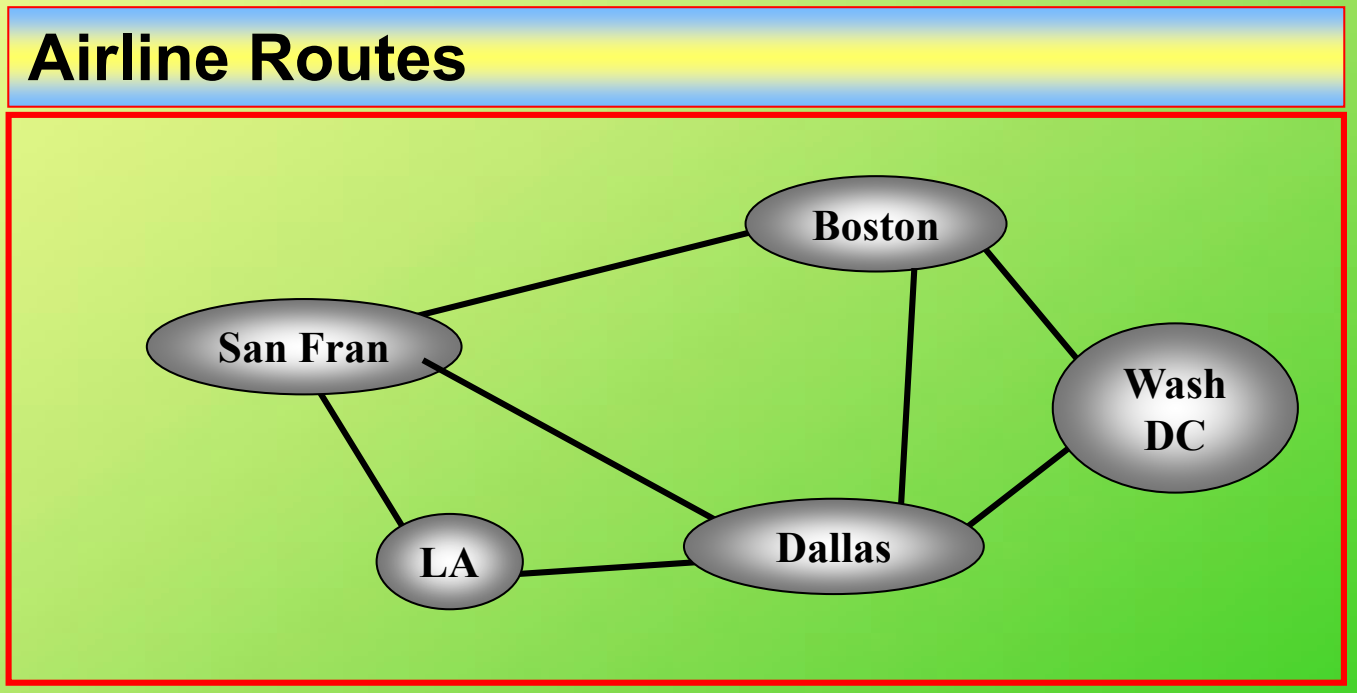
Undirected Graphs
(two-way streets)



Loops are allowed, node can have **multiple parents**

Examples of Graphs

Graphs are everywhere…



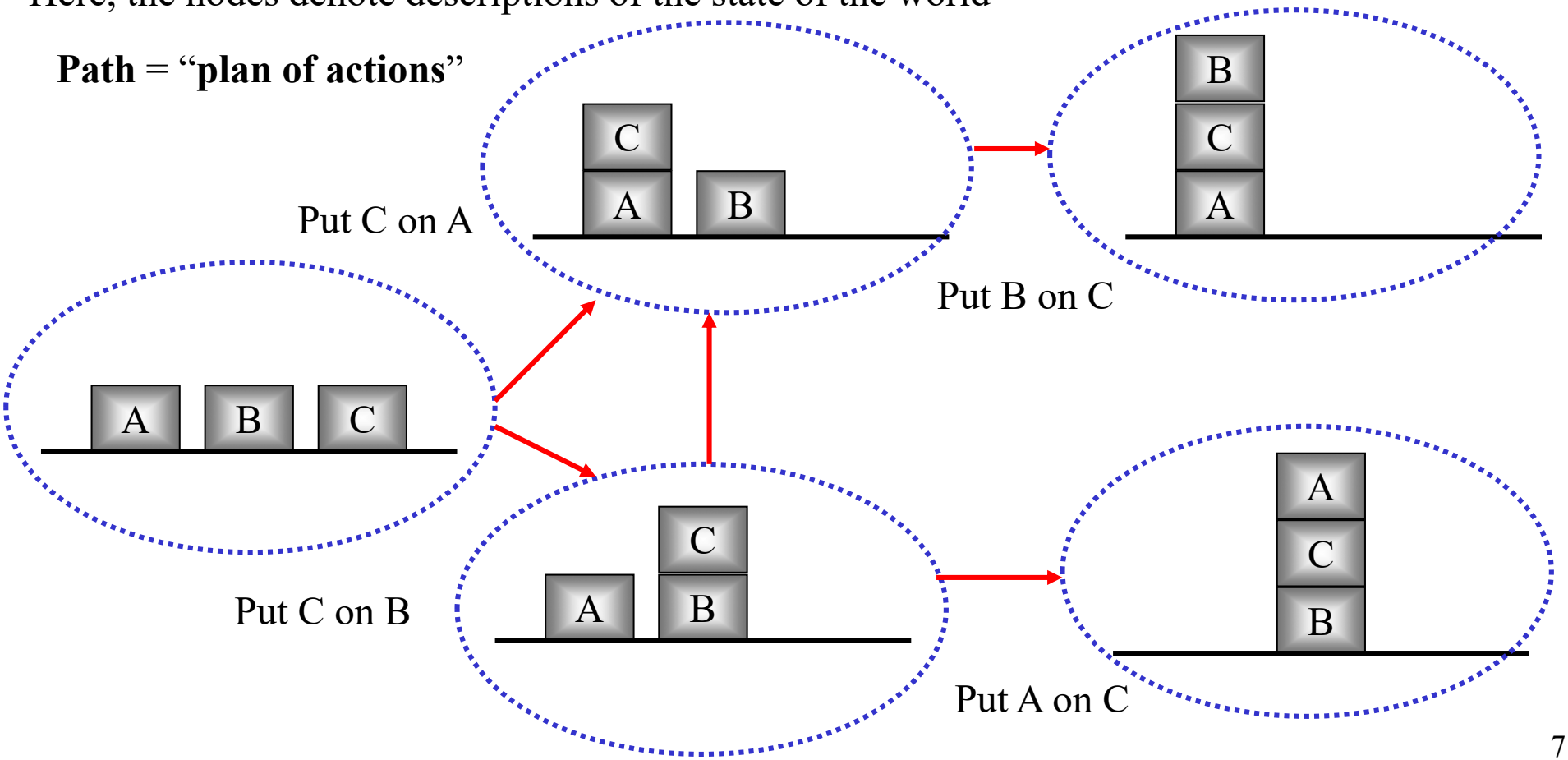
We could be interested in least cost paths, or paths with minimum hops

Examples of Graphs

Planning actions (graph of possible states of the world)

Here, the nodes denote descriptions of the state of the world

Path = "**plan of actions**"



SEARCH

Problem Solving Paradigm

1. **What are the states? (All relevant aspects of the problem)**

- arrangement of parts (to plan an assembly)
- Positions of trucks (to plan package distribution)
- City (to plan a trip)
- set of facts (to prove geometry theorem)

2. **What are the actions? (operators – Deterministic and Discrete)**

- assemble two parts
- move a truck to a new position
- fly to a new city
- apply a theorem to derive a new fact

Problem Solving Paradigm

3. What is the goal test? (Conditions for success)

- all parts in place (to plan an assembly)
- all packages delivered (to plan package distribution)
- reached final destination city (to plan a trip)
- derived goal fact (e.g. to prove geometry theorem)

Graph Search as Tree Search

- **Trees are directed graphs without cycles, and with nodes having <= 1 parent**

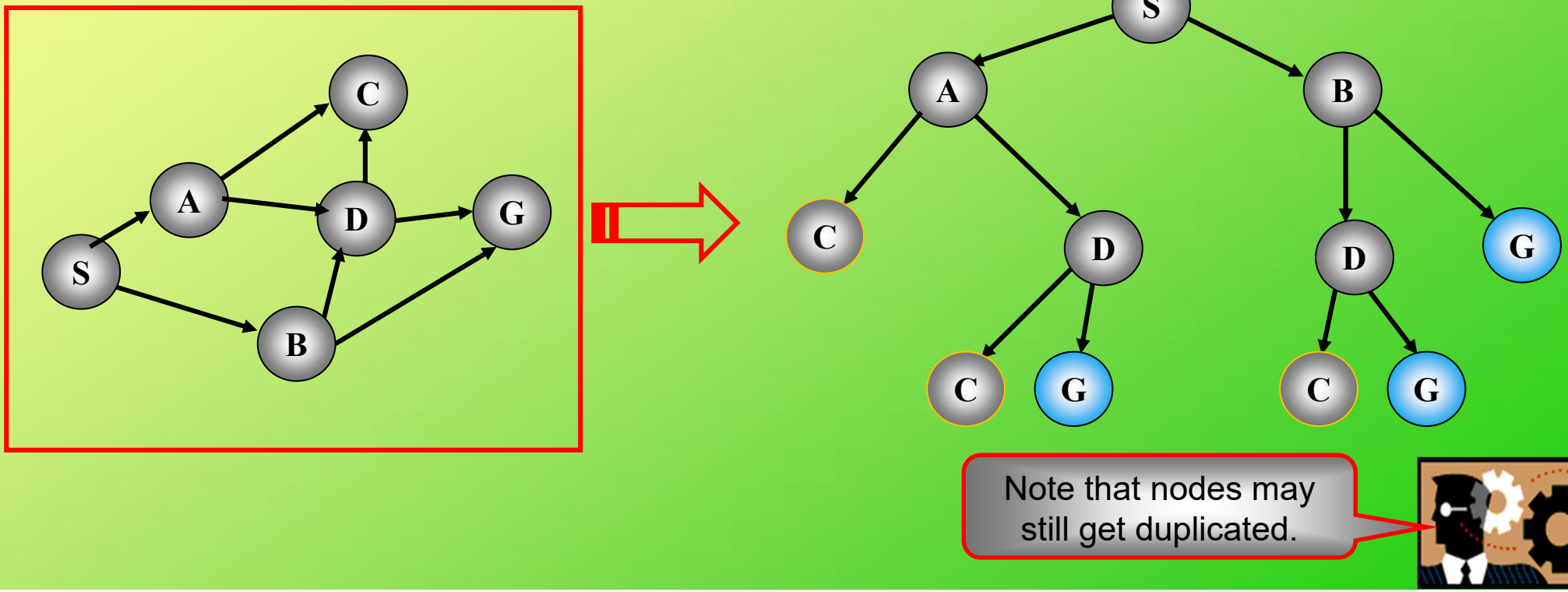
- **We can turn graph search problems into tree search problems by:**

1. Replacing undirected links by 2 directed links
2. Avoiding loops in path (or better yet, keeping track of visited nodes globally)

Graph Search as Tree Search

- **We can turn graph search problems (from S to G) into tree search problems by:**

1. Replacing undirected links by 2 directed links
2. Avoiding loops in path (or keeping track of visited nodes globally)



Terminologies

State – used to refer to the vertices of the underlying graph that is being searched; that is, states in the problem domain
(e.g. a city, an arrangement of blocks or the arrangement of parts in a puzzle)

Search Node – refers to the vertices of the search tree which is being generated by the search algorithm. Each node refers to a state of the world; Many nodes may refer to the same state. Importantly, a node implicitly represents a path (from the start state of the search to the state associated with the node). Because search nodes are part of a search tree, they have a unique ancestor node (except for the root node).

3

STATE-SPACE SEARCH

Any-Path Search

Depth-First Search, Breadth-First Search , Best-First Search

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	

Class	Name	Operation
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	

Class	Name	Operation
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)

Class	Name	Operation
Optimal Uninformed	**Uniform-Cost**	Uses path-length measure. Finds "shortest" path.

back

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)
Optimal Uninformed	**Uniform-Cost**	Uses path-length measure. Finds "shortest" path.
Optimal Informed	**A***	Uses path "length" measure and heuristic. Finds "shortest" path.

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

back

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

back

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

Implementing the Search Strategies

Depth-First

1. Pick **first** element of Q.

2. Add path extensions to **front** of Q.

Implementing the Search Strategies

Depth-First

1. Pick **first** element of Q.
2. Add path extensions to **front** of Q.

Breadth-First

1. Pick **first** element of Q.
2. Add path extensions to **end** of Q.

SEARCH

Implementing the Search Strategies

Depth-First (backtracking search)

1. Pick **first** element of Q.
2. Add path extensions to **front** of Q.

Breadth-First

1. Pick **first** element of Q.
2. Add path extensions to **end** of Q.

Heuristic functions are applied in the hope of completing the **search quicker** or **finding a relatively good goal state**.
It does not guarantee finding the "best" path though.

Best-First (greedy search)

1. Pick **best** (measured heuristic value of state) element of Q.
2. Add path extensions **anywhere** in Q (it may be efficient to keep the Q ordered in some way so as to make it easier to find the "best" element.

For our convention in class, add the path extensions to the **front** of the Queue.

Any-Path Algorithms

Simulations of the Algorithms

Let's look at some simulation (hand-computations) of the algorithms first before we characterize them more fully.

5

159.302 STATE-SPACE SEARCH

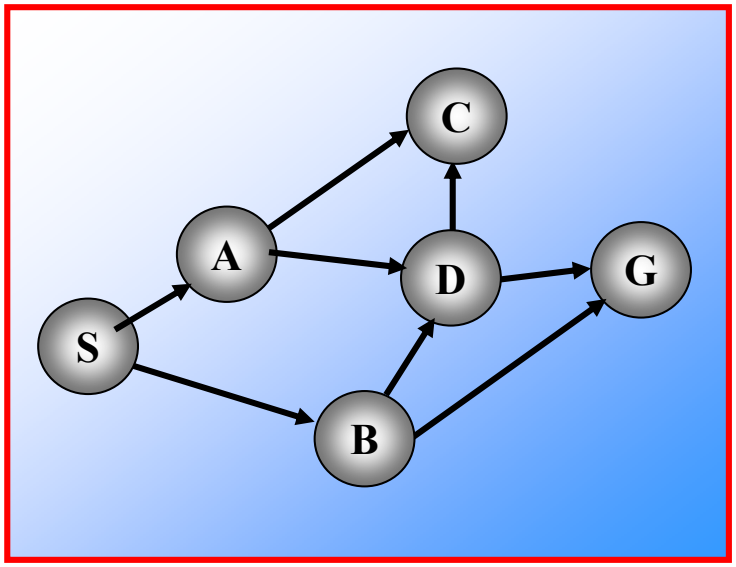
Any-Path Search Examples

Depth-First, Breadth-First, Best-First Search Algorithms

Simulations by hand

Using the following graph, simulate:

1. Depth-First with Visited List
2. Depth-First without Visited List
3. Breadth-First with Visited List
4. Breadth-First without Visited List
5. Best-First with Visited List



Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

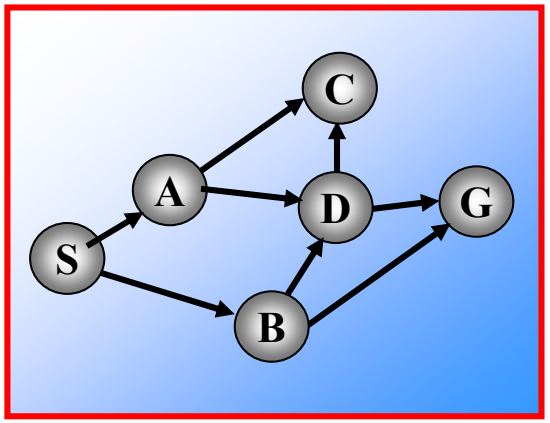
Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

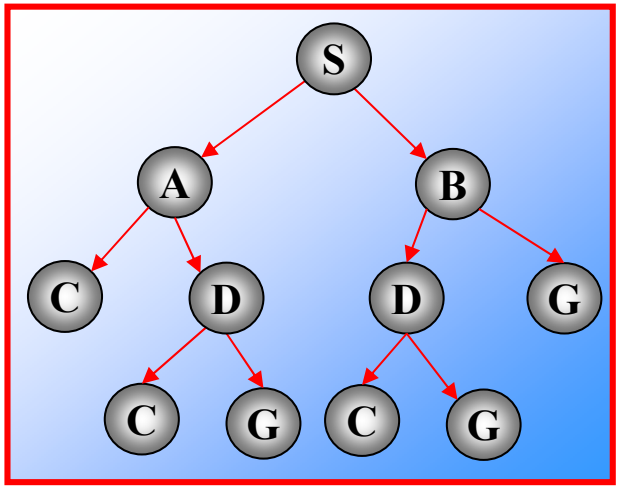
Depth-First with Visited List

Given the following graph, find a path from **S** to **G** using **Depth-First Search with the Visited List.**

Graph



Search tree equivalent

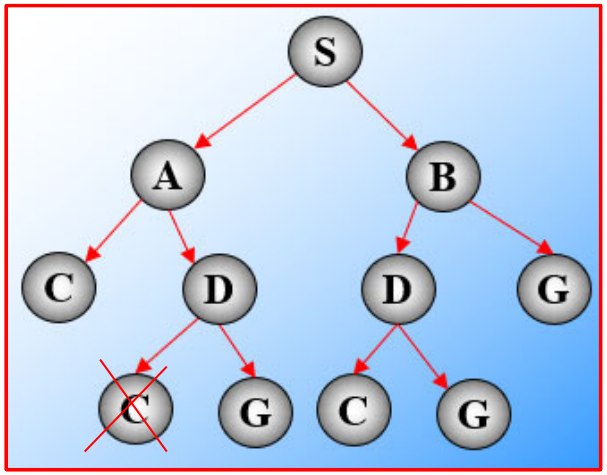


SEARCH

Depth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS)(BS)	S,A,B
2	(AS)	(CAS)(DAS)(BS)	S,A,B,C,D
3	(CAS)	(DAS)(BS)	S,A,B,C,D
4	(DAS)	(GDAS)(BS)	S,A,B,C,D,G
5	(GDAS)	(BS)	

Search Tree



Path: **S-A-D-G**
Path length: **3**
Max Q length: **3**
Number of state expansions: **5**

Sequence of State Expansions: S-A-C-D-G

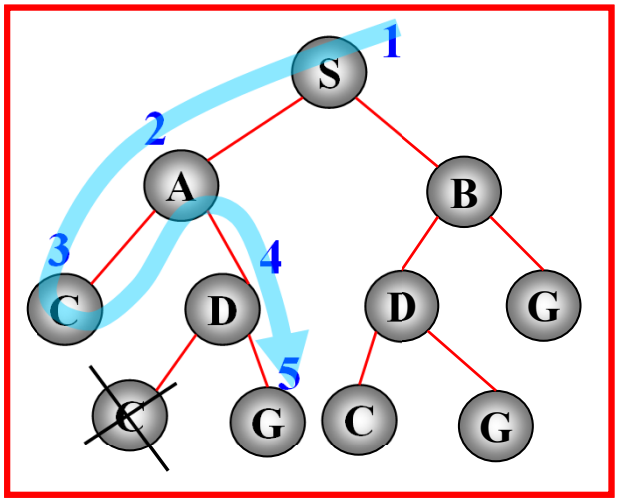
Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

Depth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS)(BS)	S,A,B
2	(AS)	(CAS)(DAS)(BS)	S,A,B,C,D
3	(CAS)	(DAS)(BS)	S,A,B,C,D
4	(DAS)	(GDAS)(BS)	S,A,B,C,D,G
5	(GDAS)	(BS)	

Search Tree



Path: **S-A-D-G**
Path length: **3**
Max Q length: **3**
Number of state expansions: **5**

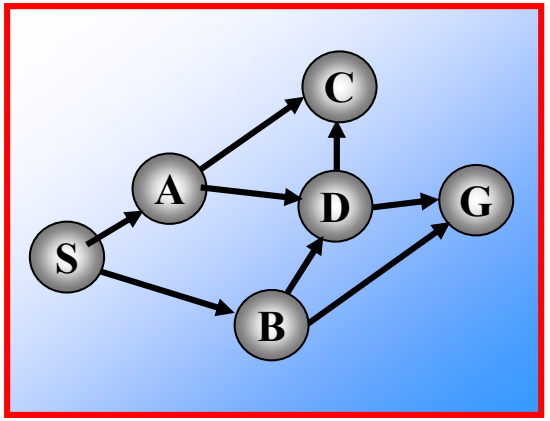
Sequence of State Expansions: S-A-C-D-G

Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

Depth-First without Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS)(BS)
2	(AS)	(CAS)(DAS)(BS)
3	(CAS)	(DAS)(BS)
4	(DAS)	(CDAS)(GDAS)(BS)
5	(CDAS)	(GDAS)(BS)
6	(GDAS)	



Path: **S-A-D-G**
Path length: **3**
Max Q length: **3**
Number of state expansions: **6**

Sequence of State Expansions: S-A-C-D-C-G

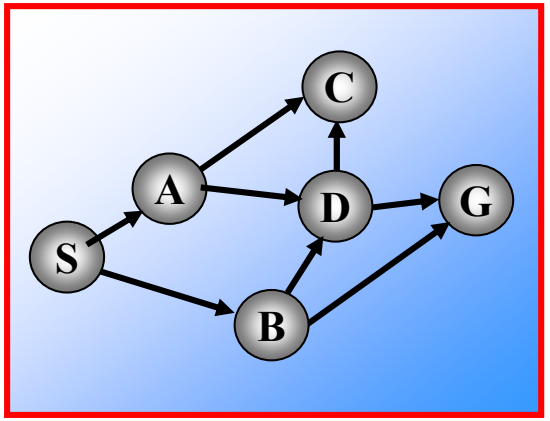
Pick first element of Q; Add path extensions to **front** of Q.

Do NOT extend a path to a state if the resulting path would have a loop.

If we have visited a state along a particular path, we do not re-visit that state again in any extensions of the path

Depth-First without Visited List

Step	Q
1	(S)
2	(AS)(BS)
3	(CAS)(DAS)(BS)
4	(DAS)(BS)
5	(CDAS)(GDAS)(BS)
6	(GDAS)(BS)



Sequence of State Expansions: S-A-C-D-C-G

Path: **S-A-D-G**
Path length: **3**
Max Q length: **3**
Number of state expansions: **6**

Pick first element of Q; Add path extensions to **front** of Q.

Do NOT extend a path to a state if the resulting path would have a loop.

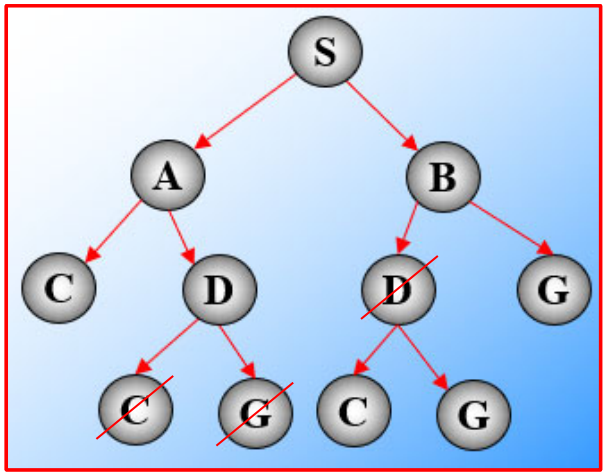
If we have visited a state along a particular path, we do not re-visit that state again in any extensions of the path

back

SEARCH

Breadth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS)(BS)	S,A,B
2	(AS)	(BS)(CAS)(DAS)	S,A,B,C,D
3	(BS)	(CAS)(DAS)(GBS)	S,A,B,C,D,G
4	(CAS)	(DAS)(GBS)	S,A,B,C,D,G
5	(DAS)	(GBS)	S,A,B,C,D,G
6	(GBS)		S,A,B,C,D,G



Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **6**

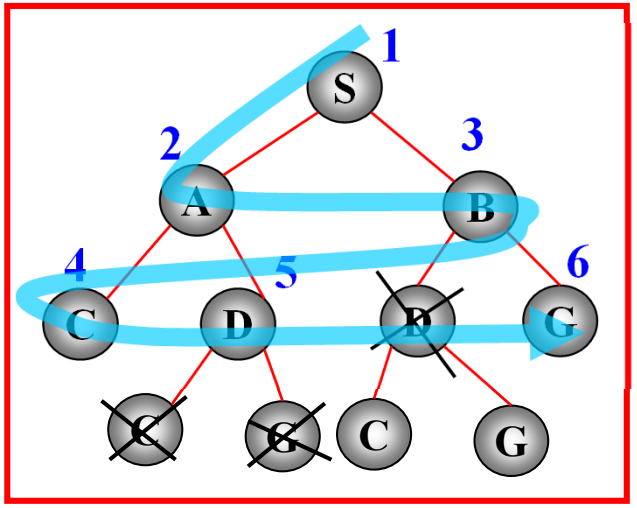
Sequence of State Expansions: S-A-B-C-D-G

Pick **first** element of Q; Add path extensions to **END** of Q.

SEARCH

Breadth-First with Visited List

Step	Dequeued	Q	Visited
0		~~(S)~~	S
1	(S)	~~(AS)~~(BS)	S,A,B
2	(AS)	~~(BS)~~(CAS)(DAS)	S,A,B,C,D
3	(BS)	~~(CAS)~~(DAS)(GBS)	S,A,B,C,D,G
4	(CAS)	~~(DAS)~~(GBS)	S,A,B,C,D,G
5	(DAS)	~~(GBS)~~	S,A,B,C,D,G
6	(GBS)		S,A,B,C,D,G



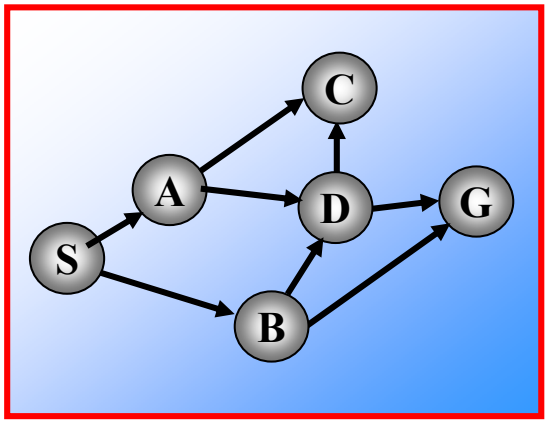
Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **6**

Sequence of State Expansions: S-A-B-C-D-G

Pick **first** element of Q; Add path extensions to **END** of Q.

Breadth-First without Visited List

Step		Q
0		(S)
1	(S)	(AS)(BS)
2	(AS)	(BS)(CAS)(DAS)
3	(BS)	(CAS)(DAS)(DBS)(GBS)
4	(CAS)	(DAS)(DBS)(GBS)
5	(DAS)	(DBS)(GBS)(CDAS)(GDAS)
6	(DBS)	(GBS)(CDAS)(GDAS)(CDBS)(GDBS)
7	(GBS)	



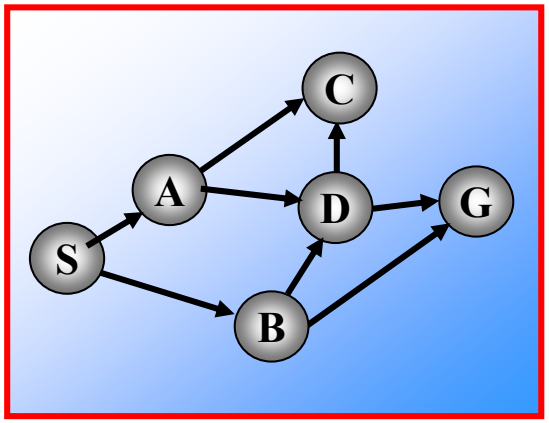
Path: **S-B-G**
Path length: **2**
Max Q length: **5**
Number of state expansions: **7**

Sequence of State Expansions: S-A-B-C-D-D-G

Pick **first** element of Q; Add path extensions to **END** of Q.

Breadth-First without Visited List

Step	Q
1	(S)
2	(AS)(BS)
3	(BS)(CAS)(DAS)
4	(CAS)(DAS)(DBS)(GBS)
5	(DAS)(DBS)(GBS)
6	(DBS)(GBS)(CDAS)(GDAS)
7	(GBS)(CDAS)(GDAS)(CDBS)(GDBS)



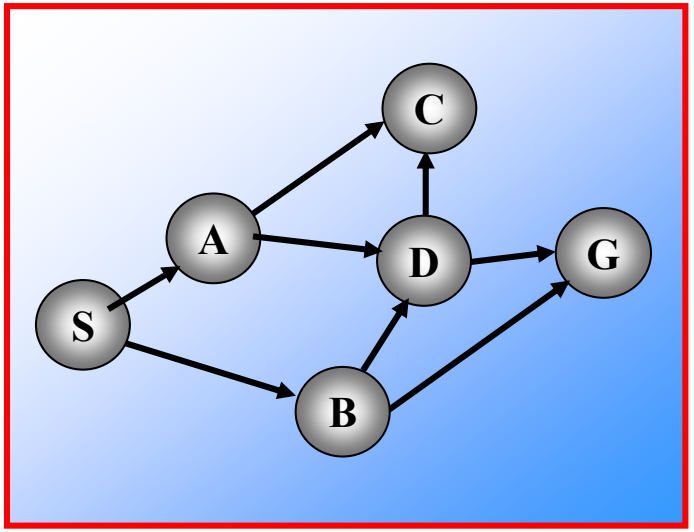
Path: **S-B-G**
Path length: **2**
Max Q length: **5**
Number of state expansions: **7**

Sequence of State Expansions: S-A-B-C-D-D-G

Pick **first** element of Q; Add path extensions to **END** of Q.

Best-First with Visited List

Step		Q	Visited
0		(10 S)	S
1	(10 S)	(2 AS)(3 BS)	A, B, S
2	(2 AS)	(1 CAS)(4 DAS)(3 BS)	C, D, B, A, S
3	(1 CAS)	(4 DAS) (3 BS)	C, D, B, A, S
4	(3 BS)	(0 GBS)(4 DAS)	G, C, D, B, A, S
5	(0 GBS)		



Heuristic Values:
S=10, A=2, B=3, C=1, D=4, G=0

Sequence of State Expansions: S-A-C-B-G

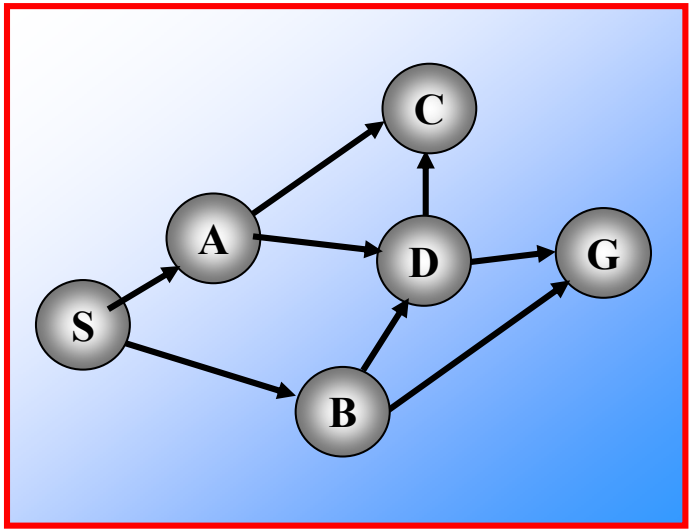
Pick "best" element of Q (by heuristic value); Add path extensions **anywhere** in Q.

Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **5**

For our convention in class, add the path extensions to the **front** of the Queue.

Best-First with Visited List

Step	Q	Visited
1	(10 S)	S
2	(2 AS)(3 BS)	A, B, S
3	(1 CAS)(4 DAS)(3 BS)	C, D, B, A, S
4	(4 DAS) (3 BS)	C, D, B, A, S
5	(0 GBS)(4 DAS)	G, C, D, B, A, S



Heuristic Values:
S=10, A=2, B=3, C=1, D=4, G=0

Sequence of State Expansions: S-A-C-B-G

Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **5**

For our convention in class, add the path extensions to the **front** of the Queue.

Pick **"best"** element of Q (by heuristic value); Add path extensions **anywhere** in Q.

SEARCH

Testing for the goal

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

This algorithm stops (in **Step 3**) when state (N) = G, or, in general, when state (N) satisfies the goal test.

We could have performed this test in **Step 6** as each extended path is added to **Q**. This would catch termination earlier and be perfectly correct for the searches we have covered so far.

However, performing the test in **Step 6** will be incorrect for the optimal searches. We have chosen to leave the test in **Step 3** to maintain uniformity with these future searches.

Terminology

Visited – a state M is first visited when a path to M first gets added to Q.
In general, a state is said to have been visited if it has ever shown up in a search node in Q. The intuition is that we have briefly **"visited"** them to place them on Q, but we have not yet examined them carefully.

Terminology

Visited – a state M is first visited when a path to M first gets added to Q.
In general, a state is said to have been visited if it has ever shown up in a search node in Q. The intuition is that we have briefly "visited" them to place them on Q, but we have not yet examined them carefully.

Expanded – a state M is expanded when it is the state of a search node that is pulled-off of **Q**. At that point, the descendants of M are visited and the path that led to M is extended to the eligible descendants. In principle, a state may be expanded multiple times. We sometimes refer to the search node that led to M (instead of M itself) as being expanded. However, once a node is expanded we are done with it; we will not need to expand it again. In fact, we discard it from **Q**.

Terminology

Visited – a state M is first visited when a path to M first gets added to Q. In general, a state is said to have been visited if it has ever shown up in a search node in Q. The intuition is that we have briefly "visited" them to place them on Q, but we have not yet examined them carefully.

Expanded – a state M is expanded when it is the state of a search node that is pulled-off of Q. At that point, the descendants of M are visited and the path that led to M is extended to the eligible descendants. In principle, a state may be expanded multiple times. We sometimes refer to the search node that led to M (instead of M itself) as being expanded. However, once a node is expanded we are done with it; we will not need to expand it again. In fact, we discard it from Q.

***This distinction plays a key role in our discussion of the various search algorithms; study it carefully.**

Visited States (for Any-Path search algorithms only)

PROS and CONS

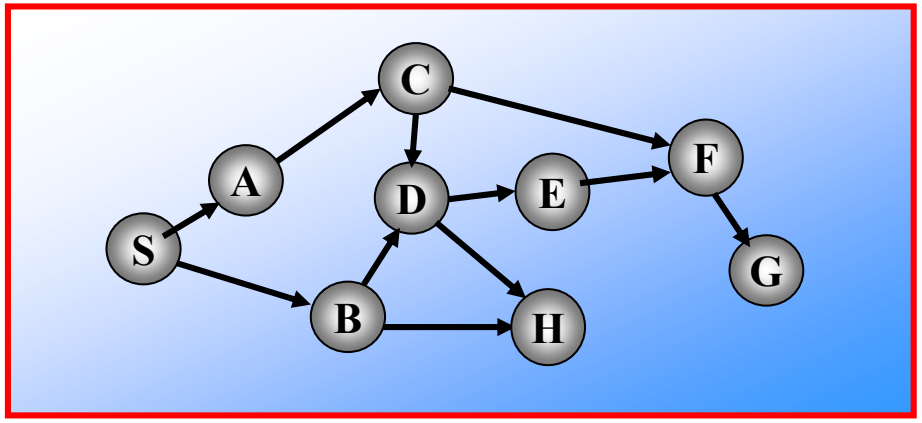
Keeping track of **visited states generally improves time efficiency** when searching graphs, without affecting correctness. Note, however, that **substantial additional space** may be required to keep track of visited states.

Why avoid revisiting states?

If all we want to do is find a path from the start to the goal, there is no advantage to adding a search node whose state is already the state of another search node.

Any state reachable from the node the second time would have been reachable from that node the first time.

e.g.



(DCAS) place in Q?

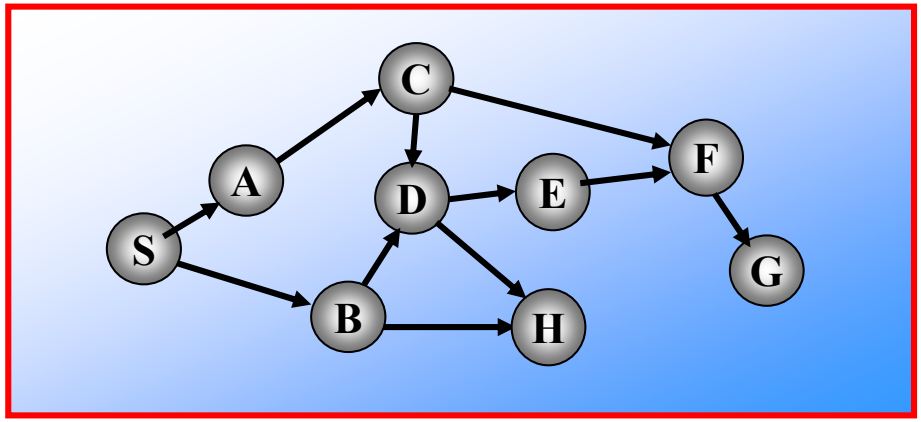
Q (DBS) …

Visited List S, A, B, D

back

Visited States (for Any-Path search algorithms only)

e.g.



~~(DCAS)~~ place in Q?

Q (DBS) …

Visited List S, A, B, D

Global check

Note that, when using the **Visited List**, each state will only ever have at most one path to it (search node) in **Q**.

We'll have to revisit this issue when we look at optimal searching

back

Visited States

Do we really need a Visited List to keep track of the Visited States?

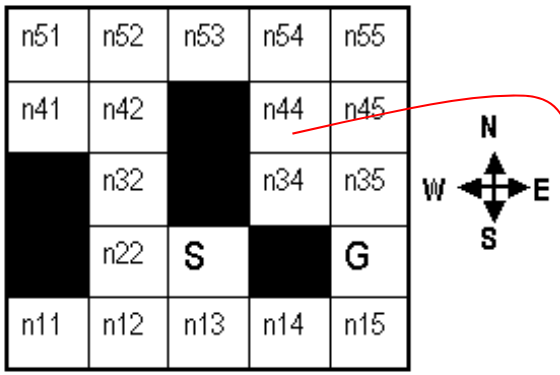
The purpose of using a **Visited list** is to identify and remember the states of all the nodes that have been placed in Q, as the algorithm runs, so as to avoid visiting them again, but how to do this efficiently?

Avoid declaring a list if possible

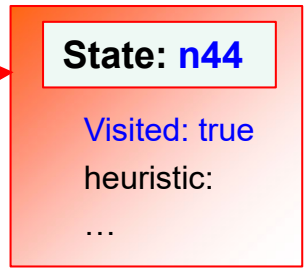
Although we speak of a **Visited list**, this is never the preferred implementation

If the **graph states** are known ahead of time as an explicit set, then space is allocated in the state itself to keep a mark; which makes both adding to Visited and checking if a state is Visited a constant time operation

e.g.



the graph states are the squares of the maze



Sample State

back

Visited States

How to speed up checking for visited states, if the states are generated on the fly?

Alternatively, as is more common in AI, if the states are generated on the fly, then a **hash table** may be used for efficient detection of previously visited states.

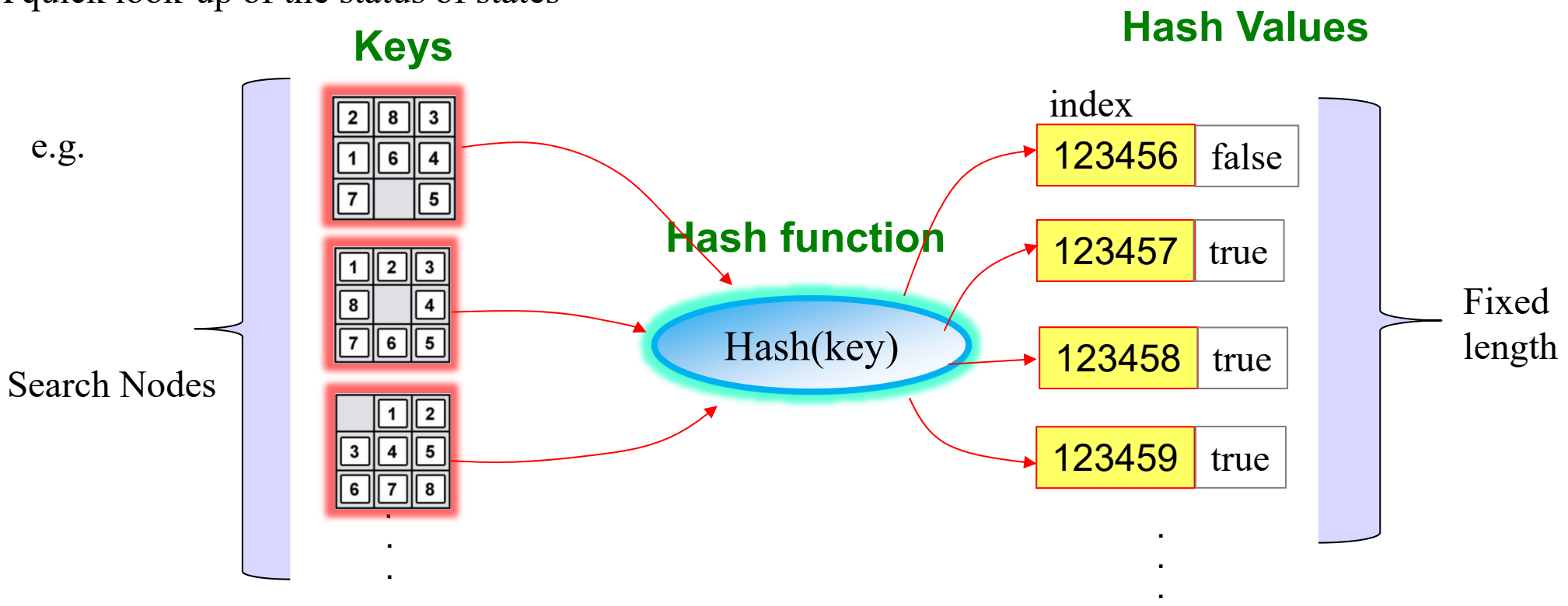
Inevitably, additional space cost is incurred

Note that, in any case, the incremental space cost of a **Visited list** will be **proportional to the number of states** – which can be very high in some problems.

SEARCH

Hash function

A quick look-up of the status of states

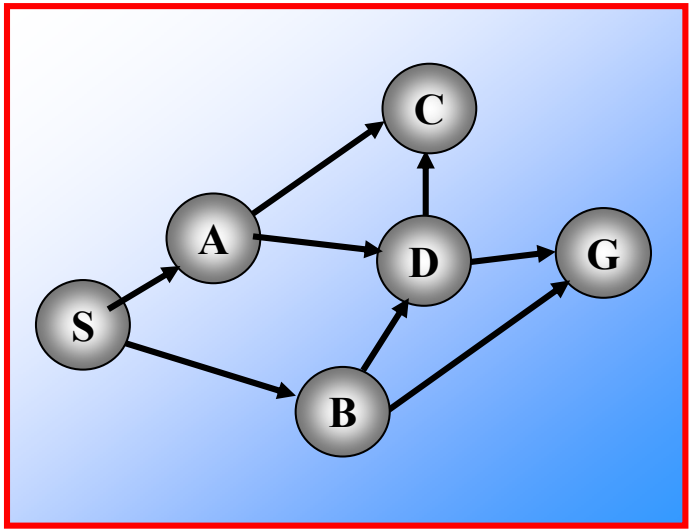


A **hash function** could be used to map large data sets of **variable length**, called **keys** to **hash values** that are of a **fixed length**. The hash values may serve as indexes to an associative array.

However, hash values may have duplicates; therefore, may cause collision between keys.

Best-First with Visited List

Step	Q	Visited
1	(10 S)	S
2	(2 AS)(3 BS)	A, B, S
3	(1 CAS)(4 DAS)(3 BS)	C, D, B, A, S
4	(4 DAS) (3 BS)	C, D, B, A, S
5	(0 GBS)(4 DAS)	G, C, D, B, A, S



Heuristic Values:
S=10, A=2, B=3, C=1, D=4, G=0

Sequence of State Expansions: S-A-C-B-G

Path: **S-B-G**
Path length: **2**
Max Q length: **3**
Number of state expansions: **5**

For our convention in class, add the path extensions to the **front** of the Queue.

Pick **"best"** element of Q (by heuristic value); Add path extensions **anywhere** in Q.

Heuristic

Heuristic

- the word generally refers to a "rule of thumb", something that may be helpful in some cases but not always.
- Generally held to be in contrast to "guaranteed" or "optimal"

Heuristic function

- in search terms, a function that **computes a value for a state** (but does not depend on any path to that state) that may be helpful in guiding the search.

e.g. **Estimated distance to goal**

- this type of heuristic function depends on the state of the goal. The classic example is straight-line distance used as an estimate for actual distance in a road network. This type of information can help increase the efficiency of a search.

Implementation Issues: Finding the best node in Q

There are many possible **approaches to finding the best node in Q**.

1. **Scanning Q** to find lowest value
2. **Sorting Q** and picking the first element
3. Keeping the **Q** sorted by doing "sorted" insertions
4. Keeping **Q** as a **Priority Queue**

Which of these is best will depend among other things on how many children the nodes have on average. We will look at this in more detail later.

http://www.cplusplus.com/reference/stl/priority_queue/

Tutorial #1

Let us do some practice using the Any-Path algorithms

back

STATE-SPACE SEARCH

Any-Path Search

Cost and Performance Properties

n.h.reyes@massey.ac.nz

Reference: **MIT OpenCourseWare**

Worst case scenarios

- Worst case running time
- Worst case space

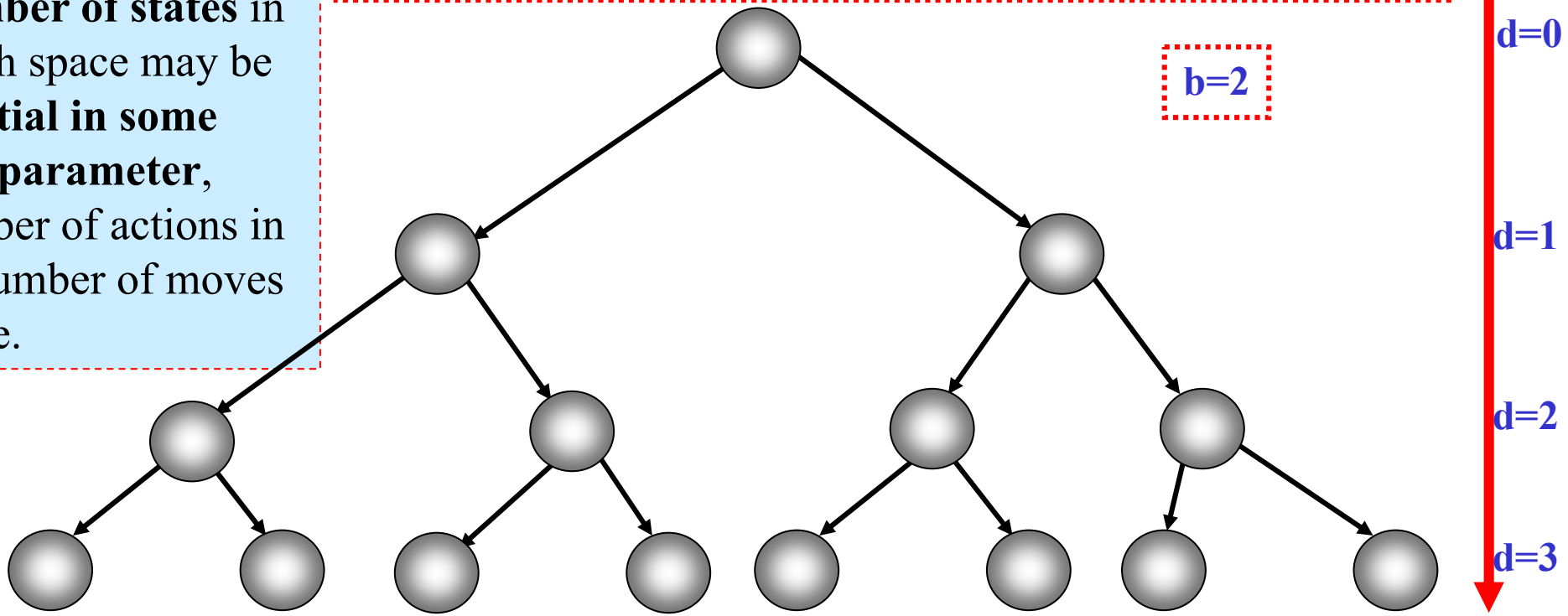
back

SEARCH

Worst Case Running Time for all Searches

The **number of states** in the search space may be **exponential in some** "**depth**" **parameter**, e.g. number of actions in a plan, number of moves in a game.

Max Time is proportional to the **Max. number of Nodes visited**.



Tree represents the search space
d is depth
b is branching factor
Number of States in the tree = $b^d < (b^{d+1} - 1)/(b-1) < b^{d+1}$

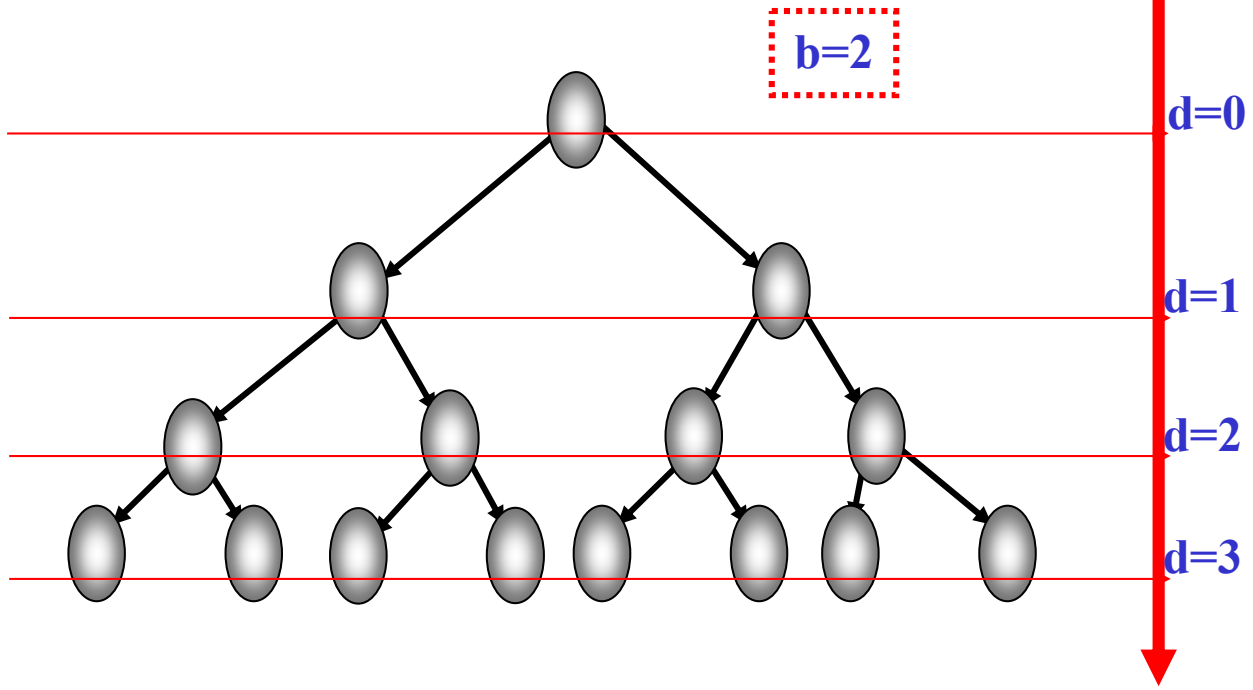
ck

Worst Case Running Time for all Searches

In the worst case, all the searches, **with or without visited list** may have to visit each state at least once.

So, all searches will have worst case running times that are at least proportional to the total number of states and therefore exponential in the "**depth**" parameter.

Max Time is proportional to the **Max. number of Nodes visited**.

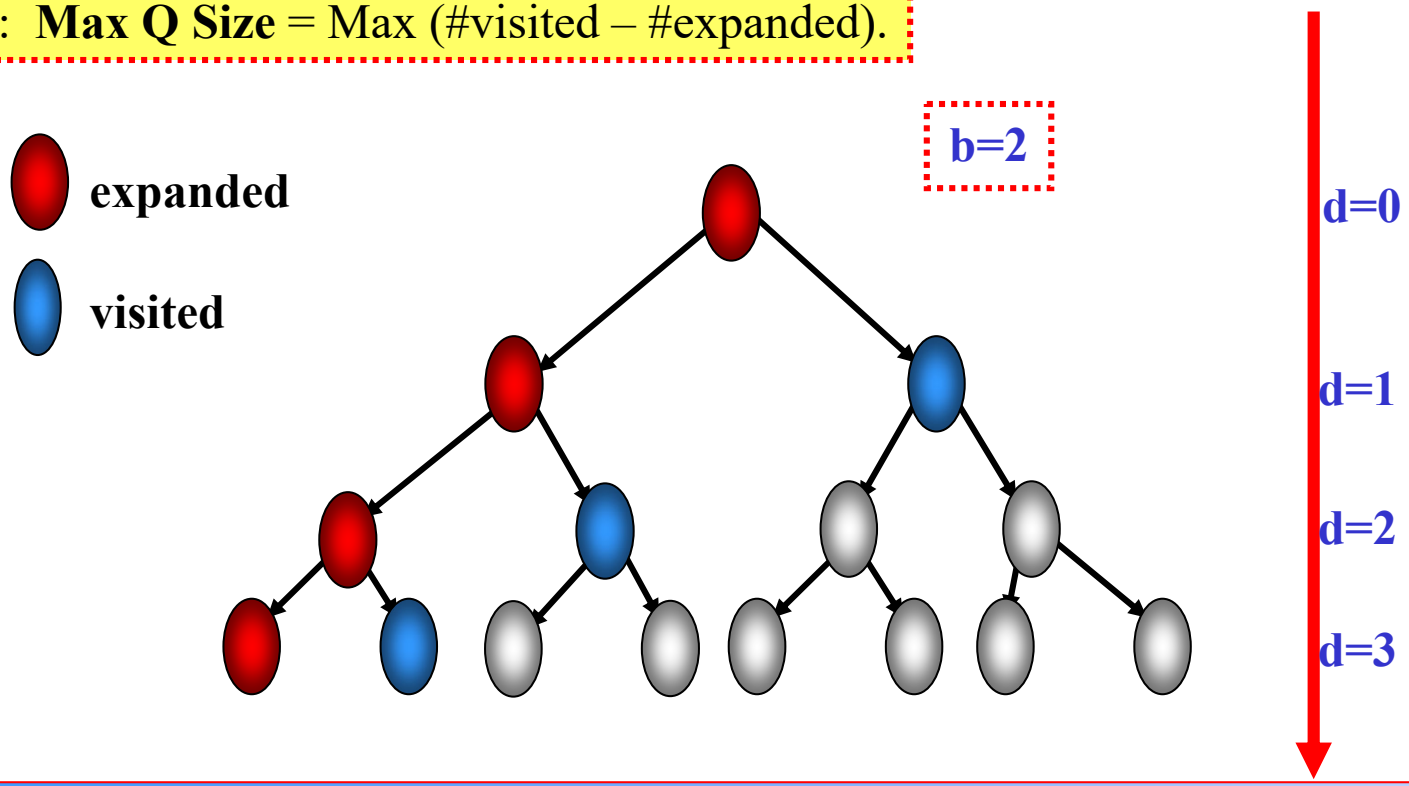


d is depth
b is branching factor
Number of States in the tree = $b^d < (b^{d+1} - 1)/(b-1) < b^{d+1}$

back

Worst Case Space for Depth-First Search

Dominant Factor: **Max Q Size** = Max (#visited – #expanded).



Depth-first Search: maximum Q size (always less than bd): $(b-1)d \approx bd$

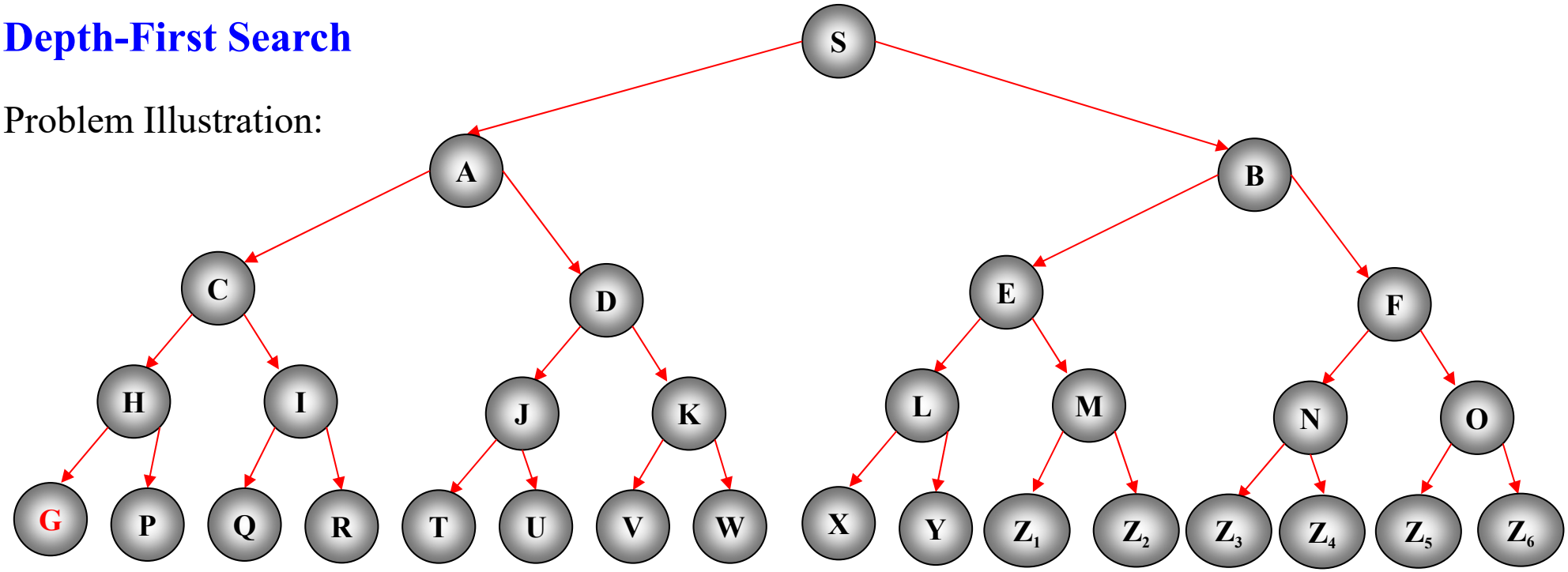
The **Q** holds the number of unexpanded "siblings" of the nodes along the path that we are currently considering.
The **space requirements** are **linear** in **d**.

Exercise: Worst Case Space for Depth-First Search

Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Depth-First Search

Problem Illustration:



In search for a path to **G**, what is the max Q length incurred, and when does it happen (at which depth)?

Exercise: Worst Case Space for Depth-First Search

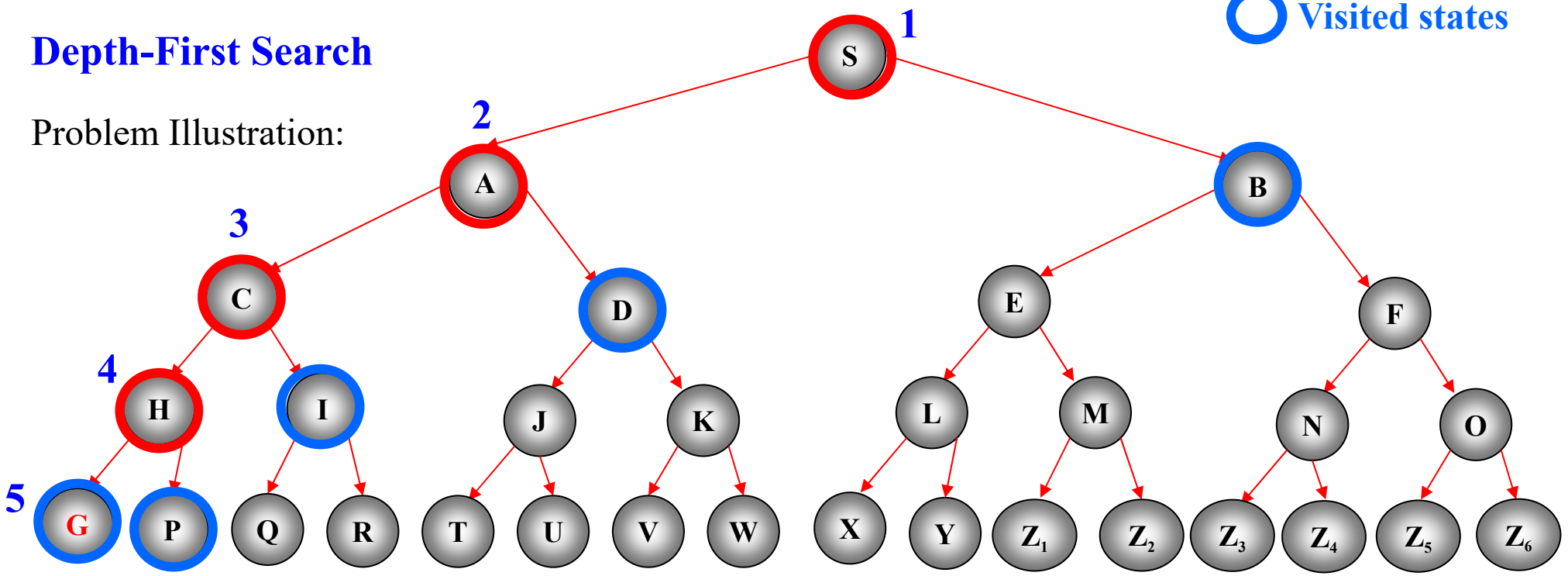
Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Expanded states

Visited states

Depth-First Search

Problem Illustration:

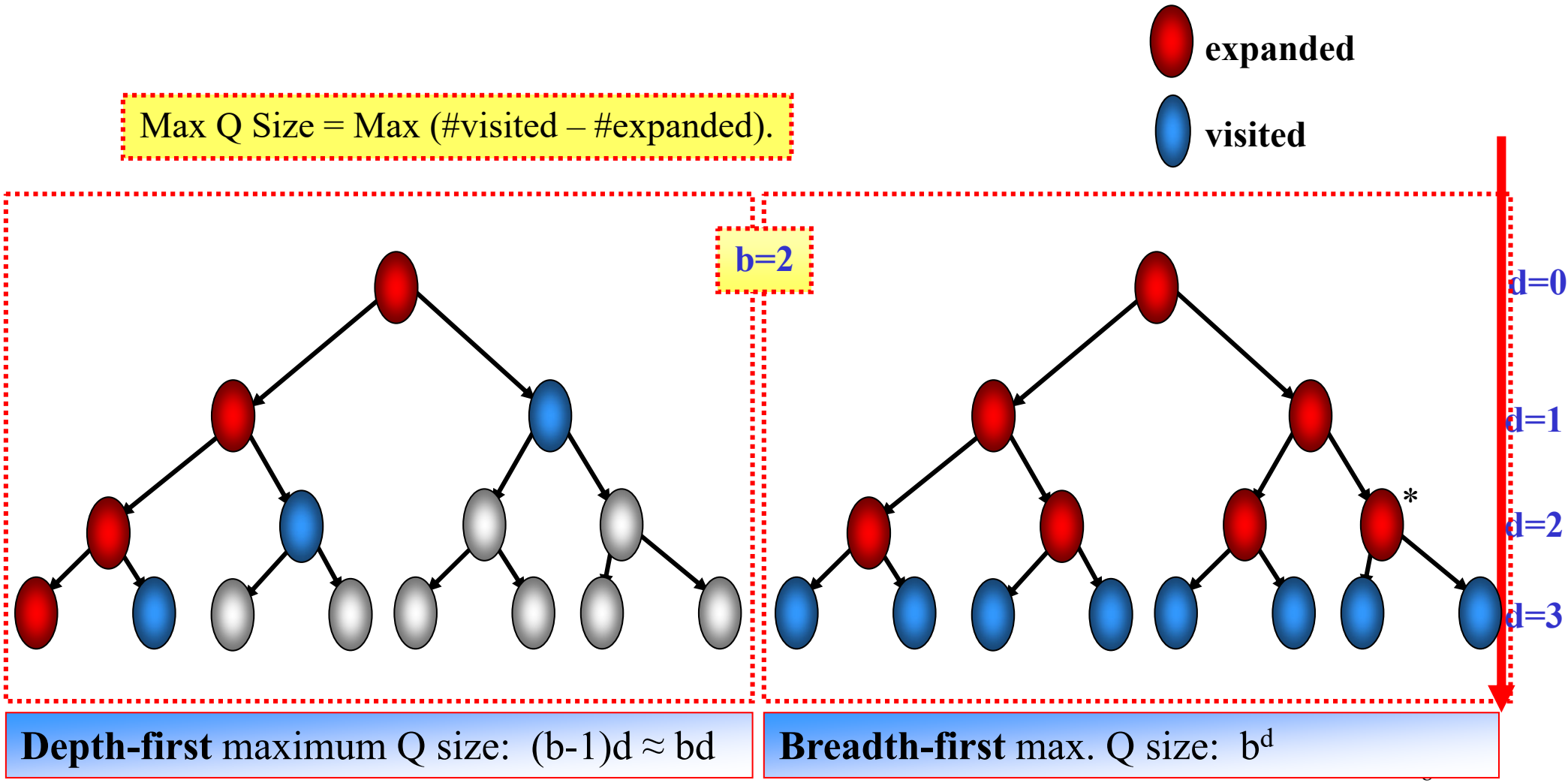


In search for a path to G, what is the max Q length incurred, and when does it happen (at which depth)?

Max Q Length = Max(#visited - #expanded) = 9 – 4 = 5

SEARCH

Worst Case Space (DFS vs. BFS)



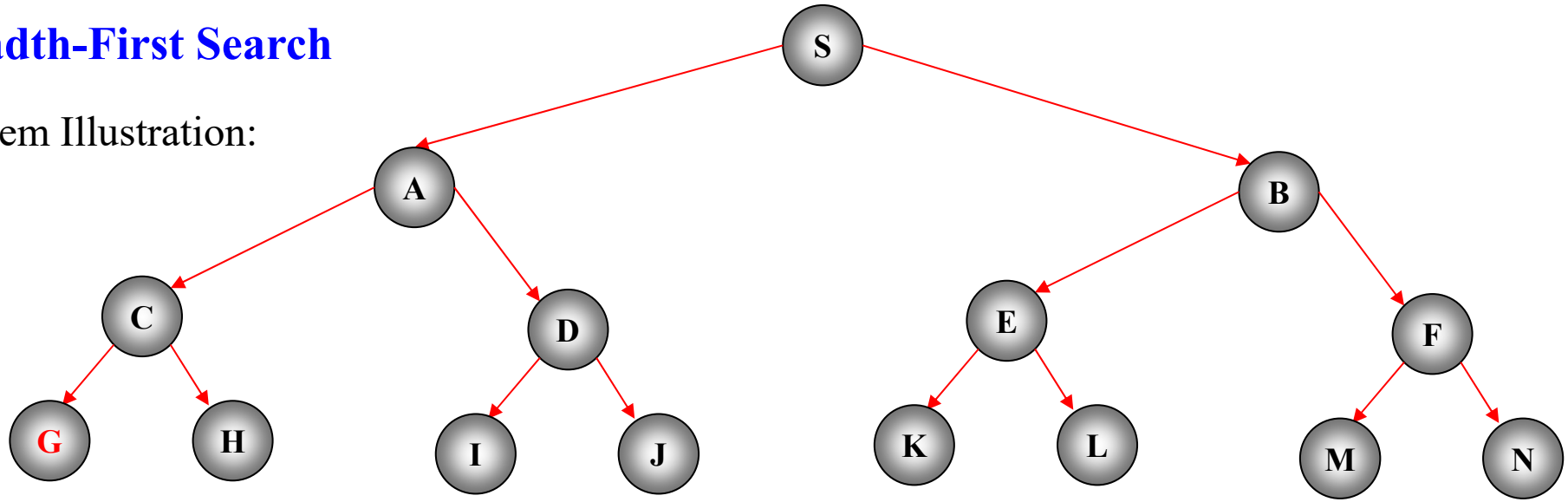
back

Exercise: Worst Case Space for Breadth-First Search

Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Breadth-First Search

Problem Illustration:



In search for a path to **G**, what is the max Q length incurred, and when does it happen (at which depth)?

Exercise: Worst Case Space for Breadth-First Search

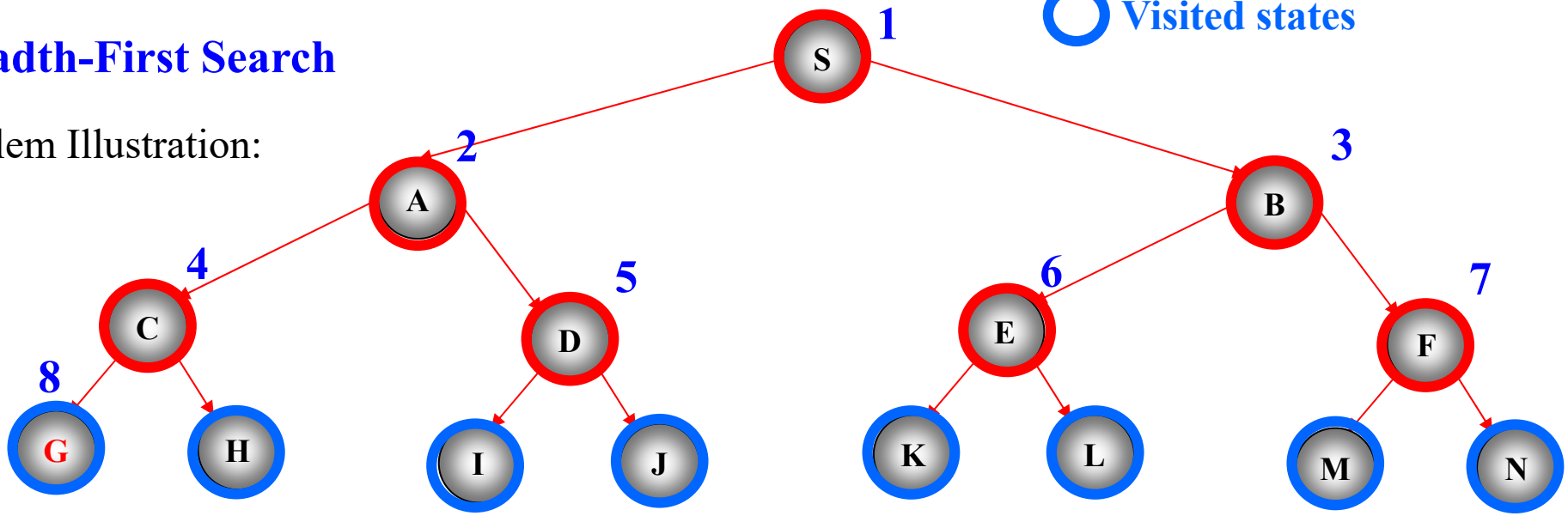
Dominant Factor: **Max Q Size** = Max (#visited – #expanded).

Expanded states

Visited states

Breadth-First Search

Problem Illustration:



In search for a path to **G**, what is the max Q length incurred, and when does it happen (at which depth)?

Max Q Length = Max(#visited - #expanded) = 15 – 7 = 8

SEARCH

Cost and Performance of Any-Path Methods

Searching a tree with branching factor **b** and depth **d** (**without using a Visited List**)

Search Method	Worst Time	Worst Space	Fewest states?	Guaranteed to find a path?
Depth-First	b^{d+1}	bd	No	Yes*
Breadth-First	b^{d+1}	b^d	Yes	Yes
Best-First	b^{d+1} **	b^d	No	Yes*

*If there are no indefinitely long paths in the search space
**Best-First needs more time to locate the best node in Q.

Worst case time is proportional to the number of nodes added to Q.
Worst case space is proportional to maximal length of Q.

Considering that all states are unique

Cost and Performance of Any-Path Methods

Searching a tree with branching factor **b** and depth **d** (**with Visited List**)

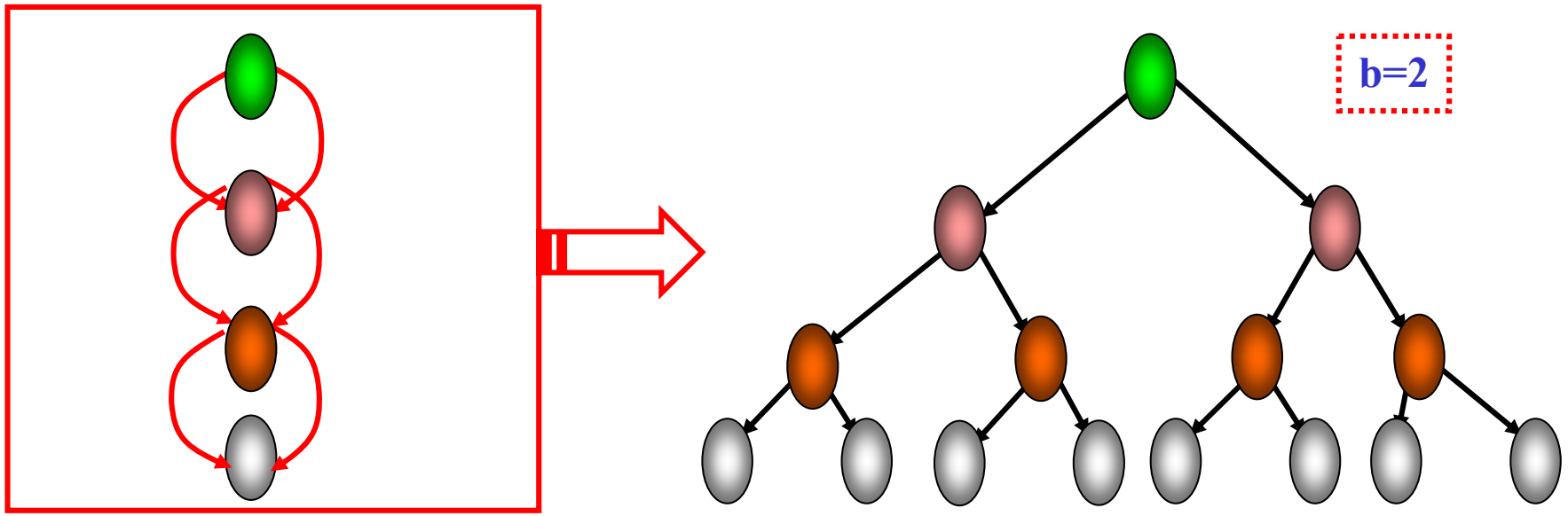
Search Method	Worst Time	Worst Space	Fewest states?	Guaranteed to find a path?
Depth-First	b^{d+1}	~~b^d~~ b^{d+1}	No	Yes*
Breadth-First	b^{d+1}	~~b^d~~ b^{d+1}	Yes	Yes
Best-First	b^{d+1} **	~~b^d~~ b^{d+1}	No	Yes*

*If there are no indefinitely long paths in the search space
**Best-First needs more time to locate the best node in Q.

Worst case time is proportional to the number of nodes added to Q.
Worst case space is proportional to maximal length of Q **and Visited list**.

States vs. Path

Using the Visited List is a way of spending space to limit the worst case running time.



Using a Visited list, the **worst-case time performance** is limited by the **number of states** in the search space **rather than the number of paths** through the nodes in the space (which may be exponentially larger than the number of states).

Using a Visited list helps prevent loops; that is, no path visits a state more than once. However, using the Visited list for very large spaces may not be appropriate as the space requirements would be prohibitive.

back

STATE-SPACE SEARCH

Any-Path Search

Progressive Deepening Search

n.h.reyes@massey.ac.nz

Reference: **MIT OpenCourseWare**

Space (the final frontier)

In large search problems, **memory** is often the **limiting factor**.

BFS

• Imagine searching a tree with **branching factor 8** and **depth 10**. Assume a **node** requires just **8 bytes of storage**. Then, Breadth-First search might require up to:

$(2^3)^{10} * 2^3 = 2^{33}$ bytes $= 8{,}000$ Mbytes $= 8$ Gbytes

Recall Worst case space for Breadth-First Search

Progressive Deepening Search

Depth-First Search (DFS) has **small space requirements** (linear in depth), but has major problems:

- DFS can run forever in search space with infinite length paths
- DFS does not guarantee finding the **shallowest** goal

Progressive Deepening Search

Best of Both Worlds

Depth-First Search (DFS) has small space requirements (linear in depth), but has major problems:

- DFS can run forever in search space with infinite length paths
- DFS does not guarantee finding the shallowest goal

Breadth-First Search (BFS) guarantees finding the **shallowest goal states**, even in the presence of infinite paths, but has one great problem:

- BFS requires a great deal of space (exponential in depth)

SEARCH

Space (the final frontier)

PDS

One strategy is to trade time for memory.
For example, we can emulate Breadth-First search by **repeated applications of Depth-First search**, each up to a preset depth limit. This is called **Progressive Deepening Search (PDS)**:

Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

Pick **first** element of Q; Add path extensions to **front** of Q.

Progressive Deepening Search

Best of Both Worlds

Depth-First Search (DFS) has small space requirements (linear in depth), but has major problems:

- DFS can run forever in search space with infinite length paths
- DFS does not guarantee finding the shallowest goal

Breadth-First Search (BFS) guarantees finding the shallowest goal states, even in the presence of infinite paths, but has one great problem:

- BFS requires a great deal of space (exponential in depth)

Progressive Deepening Search (PDS) has the advantages of both DFS and BFS.

- PDS has small space requirements (linear in depth)
- PDS guarantees finding the shallowest goal

back

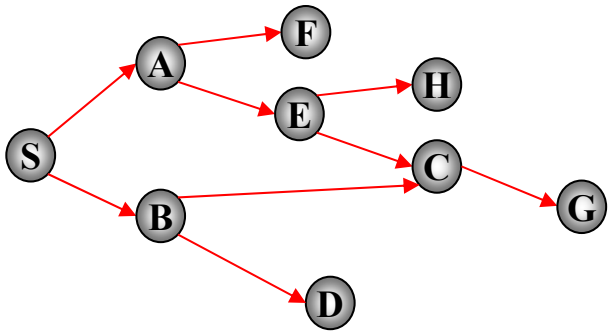
Progressive Deepening Search

Pseudocode

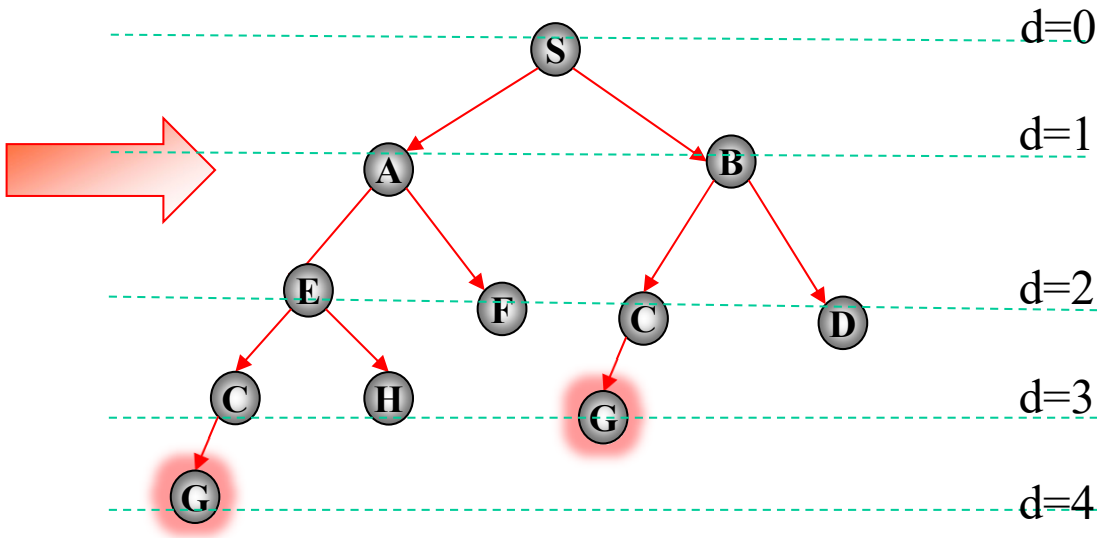
1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

Example: Apply PDS on the given graph.

GRAPH



SEARCH TREE



Progressive Deepening Search
without the **Visited List**

back

SEARCH

Progressive Deepening Search

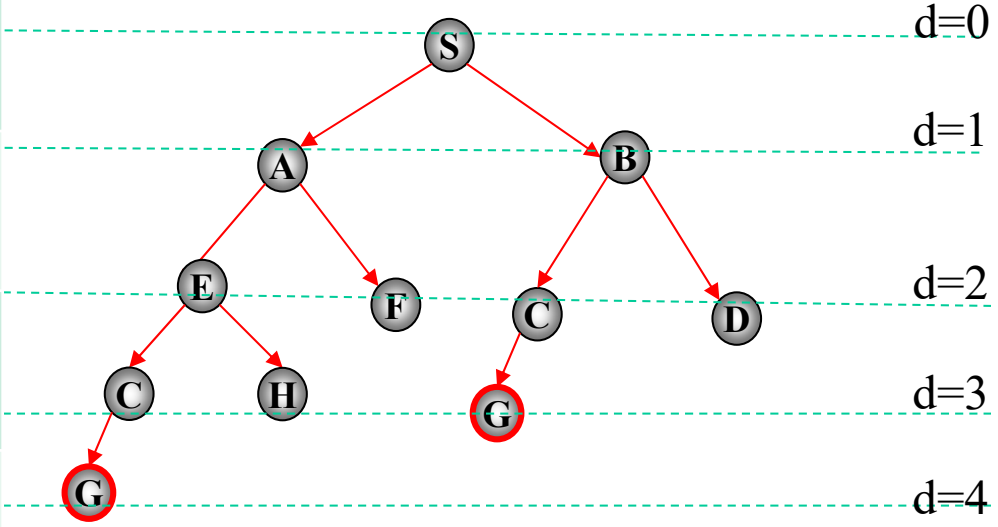
Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

C=1

Step	Dequeued	Enqueued
0		(S^0) *depth=0*

SEARCH TREE



Pick **first** element of Q; Add path extensions to **front** of Q.

back

Progressive Deepening Search

Pseudocode

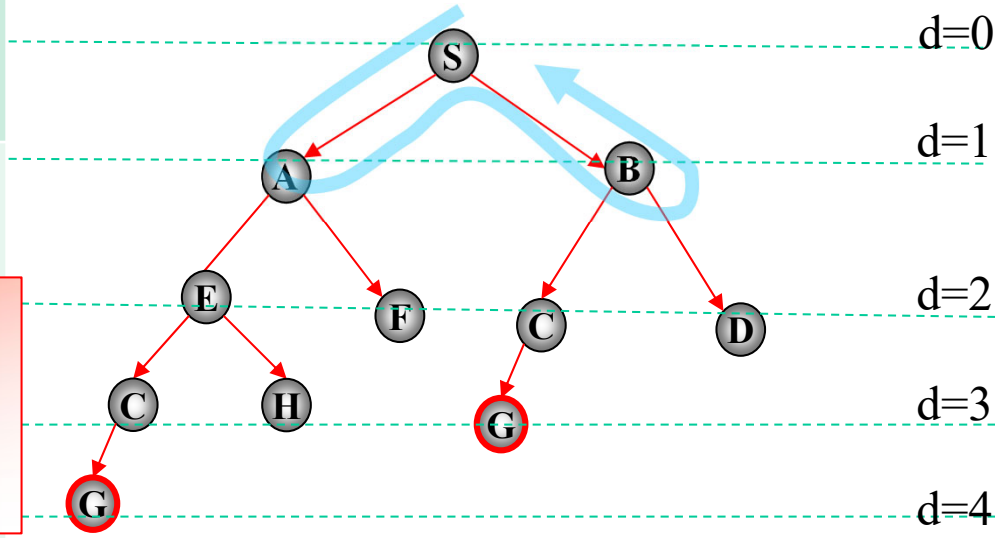
1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

C=1

Step	Dequeued	Enqueued
0		(S^0) *depth=0*
1	(S^0)	(AS^1) (BS^1)

S^0 has depth level 0.
Since it is less than the setting for parameter C (i.e. C=1), that means we will allow to expand it

SEARCH TREE



Notation: AS^1 *depth=1*

Pick **first** element of Q; Add path extensions to **front** of Q.

SEARCH

Progressive Deepening Search

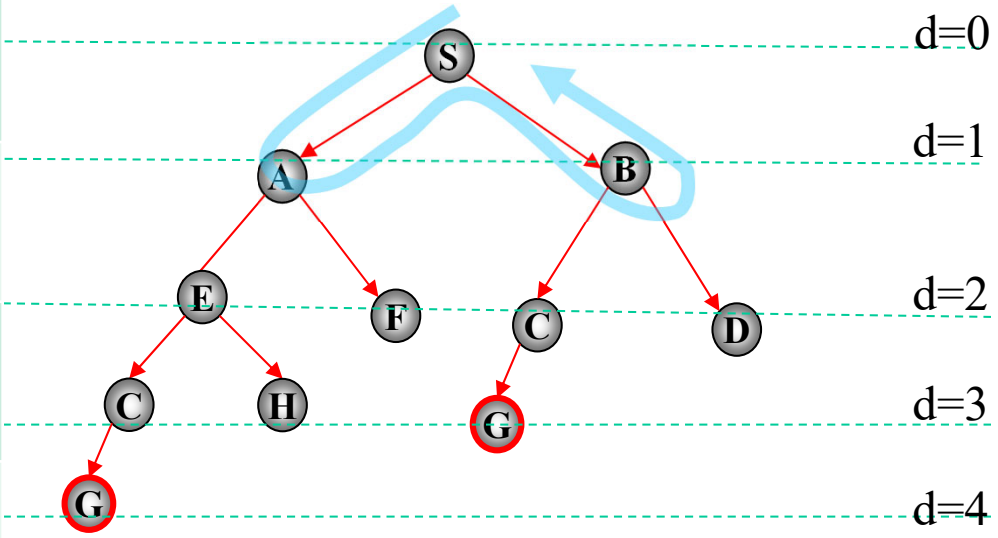
Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

C=1

Step	Dequeued	Enqueued
0		~~(S^0)~~ *depth=0*
1	(S^0)	~~(AS^1)~~ (BS^1)
2	(AS^1) . *already at depth=1*	~~(BS^1)~~
3	(BS^1) . *already at depth=1*	*Q is empty.* *No Solution.*

SEARCH TREE



depth=1

Notation: AS^1

Pick **first** element of Q; Add path extensions to **front** of Q.

back

SEARCH

Progressive Deepening Search

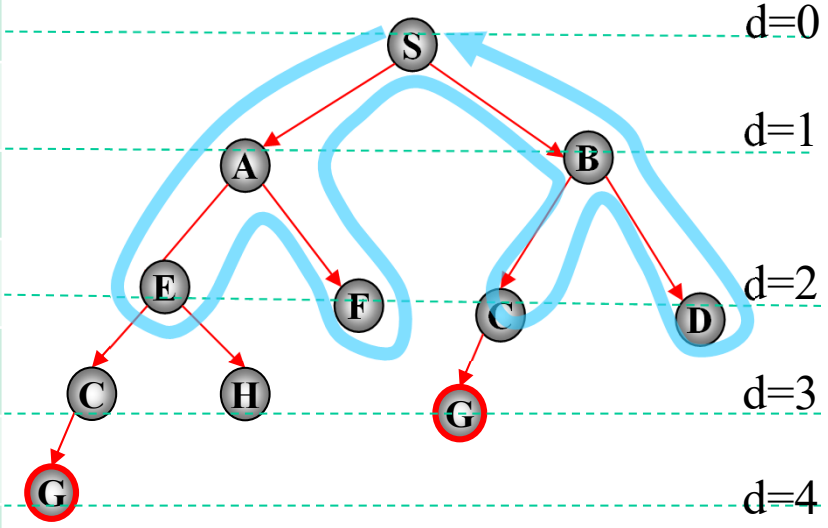
Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

➡ **C=2**

Step	Dequeued	Enqueued
0		(S^0)
1	(S^0)	(AS^1) (BS^1)
2	(AS^1)	(EAS^2) (FAS^2) (BS^1)
3	(EAS^2) .	(FAS^2) (BS^1)
4	(FAS^2) .	(BS^1)
5	(BS^1)	(CBS^2) (DBS^2)
6	(CBS^2) .	(DBS^2)
7	(DBS^2) .	*No Solution.*

SEARCH TREE



Pick **first** element of Q; Add path extensions to **front** of Q.

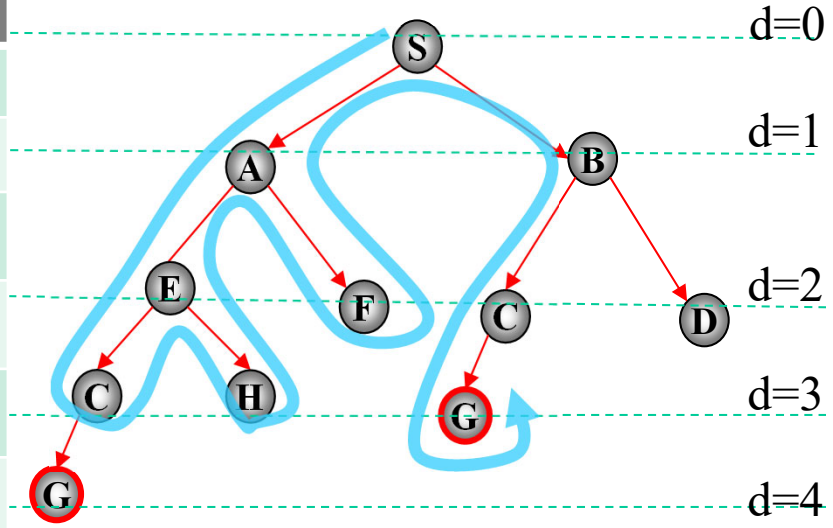
Progressive Deepening Search

C=3

Step	Dequeued	Enqueued
0		(S^0)
1	(S^0)	(AS^1) (BS^1)
2	(AS^1)	(EAS^2) (FAS^2) (BS^1)
3	(EAS^2)	($CEAS^3$)($HEAS^3$)(FAS^2) (BS^1)
4	($CEAS^3$) .	($HEAS^3$)(FAS^2) (BS^1)
5	($HEAS^3$) .	(FAS^2) (BS^1)
6	(FAS^2)	(BS^1)
7	(BS^1)	(CBS^2) (DBS^2)
8	(CBS^2)	($GCBS^3$) (DBS^2)
9	($\underline{G}CBS^3$)	(DBS^2)

Solution found!

SEARCH TREE



path = S-B-C-G

Is this the shallowest goal in the tree?

Notation: AS^1 → *depth=1*

Progressive Deepening Search

Best of Both Worlds

Isn't **Progressive Deepening Search** (PDS) too expensive?

- PDS looks at the same nodes over and over again…

Progressive Deepening Search

Best of Both Worlds

Isn't **Progressive Deepening Search** (PDS) too expensive?

- In small graphs?
- In exponential trees, time is dominated by deepest search

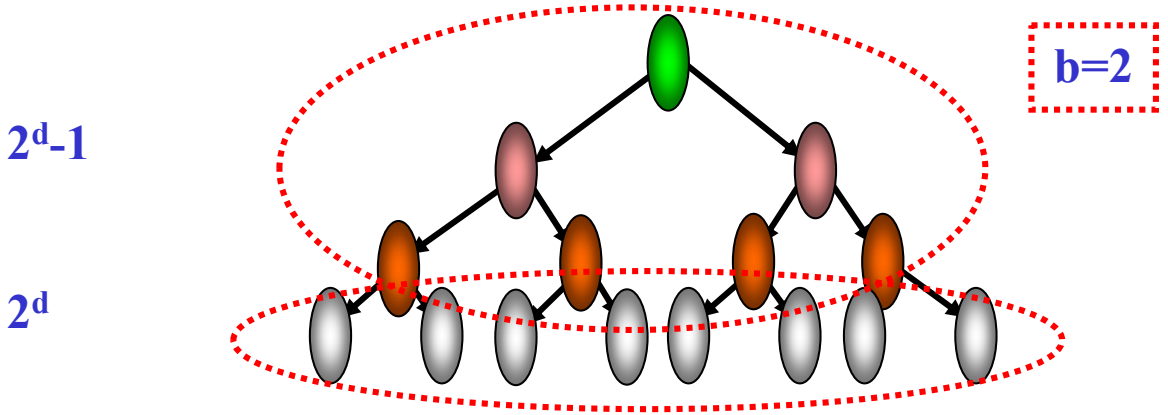
Progressive Deepening Search

Isn't **Progressive Deepening Search** (PDS) too expensive?

- In small graphs?
- In exponential trees, time is dominated by deepest search

For example, if branching factor is 2, then the number of nodes at depth d (also the **worst case running time at depth d**) is $\mathbf{2^d}$ while the total number of nodes in all previous levels is $(2^d - 1)$, so the difference between looking at the whole tree versus only the deepest level is at worst a **factor of 2** in performance.

At level **d**, compare the work done by **Breadth-First Search vs**. work done by **PDS**



Progressive Deepening Search

Ratio of the work done by PDS to that done by a single DFS

Compare the **ratio of average time** spent on **PDS** with average time spent on a **single DFS** with the full depth tree:

(Average time for PDS)/(Average time for DFS) ≈ **(b+1)/(b-1)**

b	ratio
2	3
3	2
5	1.5
25	1.08
100	1.02

Progressive Deepening Search

Ratio of the work done by PDS to that done by a single DFS

Compare the **ratio of average time** spent on **PDS** with average time spent on a **single DFS** with the full depth tree:

(Average time for PDS)/(Average time for DFS) ≈ **(b+1)/(b-1)**

Progressive Deepening is an effective strategy for difficult searches.

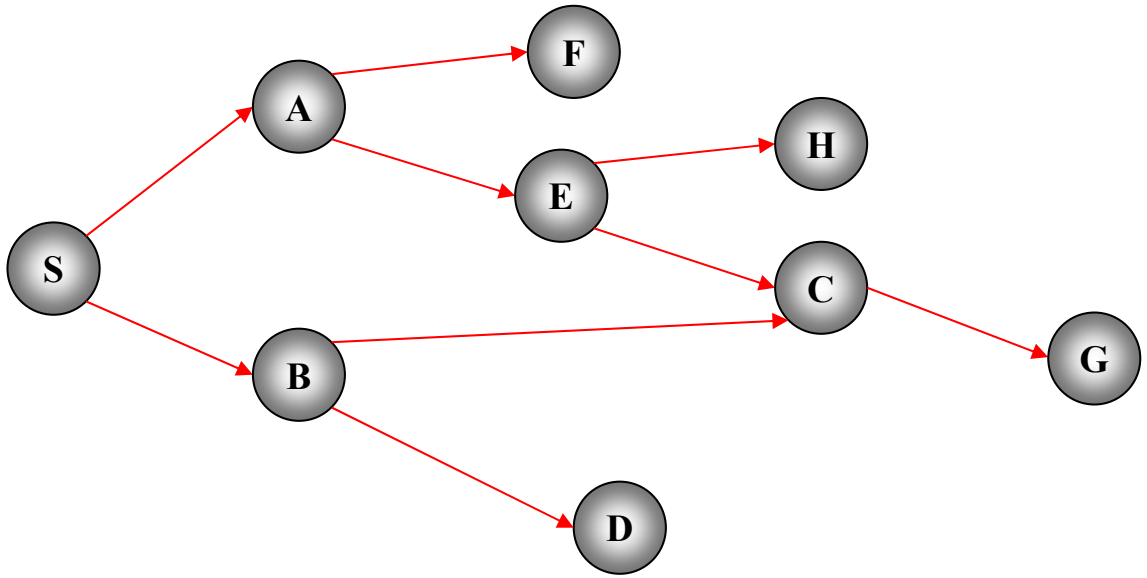
b	ratio
2	3
3	2
5	1.5
25	1.08
100	1.02

Progressive Deepening Search
with the **Visited List**

back

Progressive Deepening Search

Will the use of Visited List help the algorithm find the shallowest goal?



Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

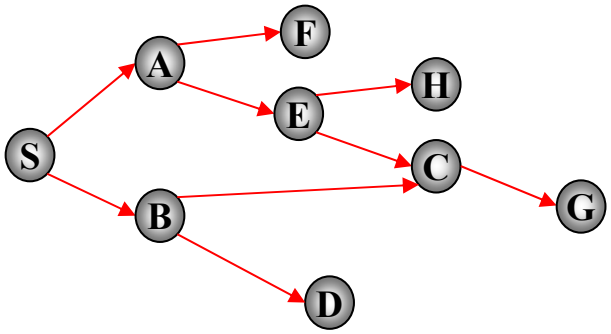
back

SEARCH

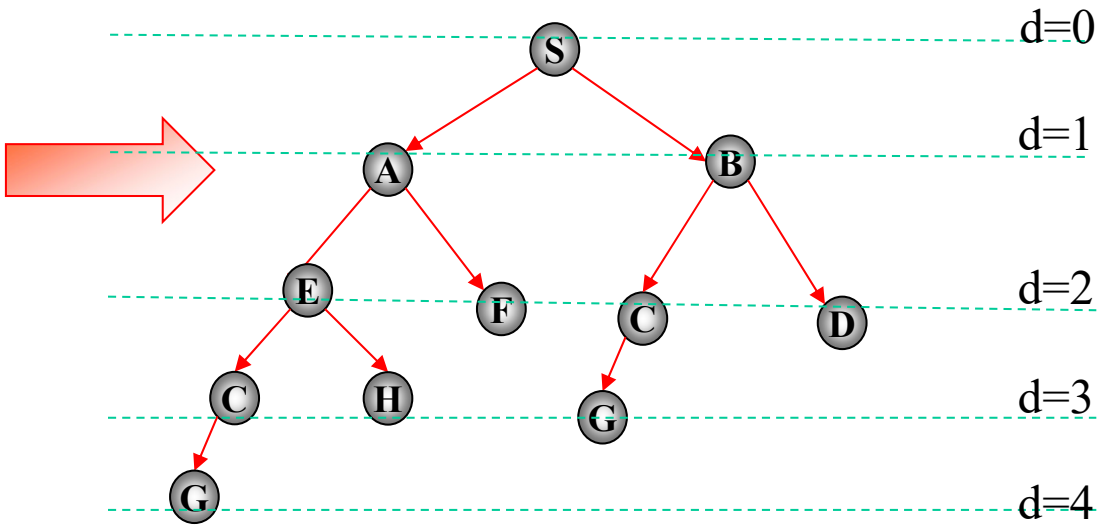
Progressive Deepening Search

Given the graph below, simulate PDS with and without using the Visited List.

GRAPH



SEARCH TREE



Pseudocode

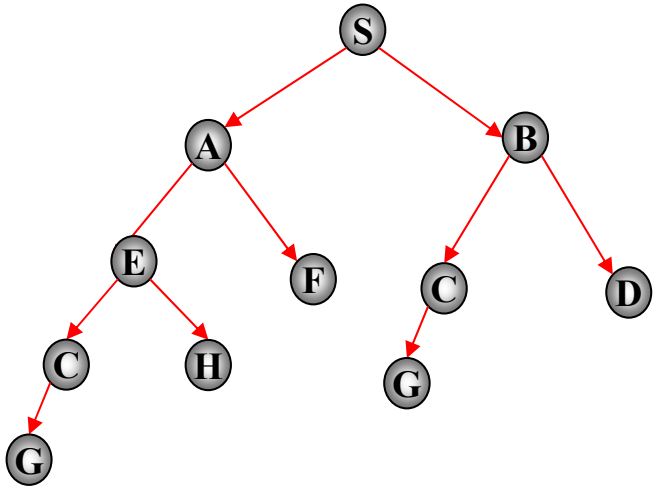
1. **C**=1
2. Do **Depth-limited DFS** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

back

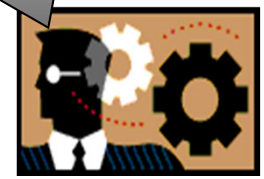
Progressive Deepening Search

Depth-Limited Depth-First Search with Visited List (max depth=3)

SEARCH TREE



Can we improve PDS by using the Visited List?



back

Progressive Deepening Search with the **Visited List**

Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** (**with Visited List)** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

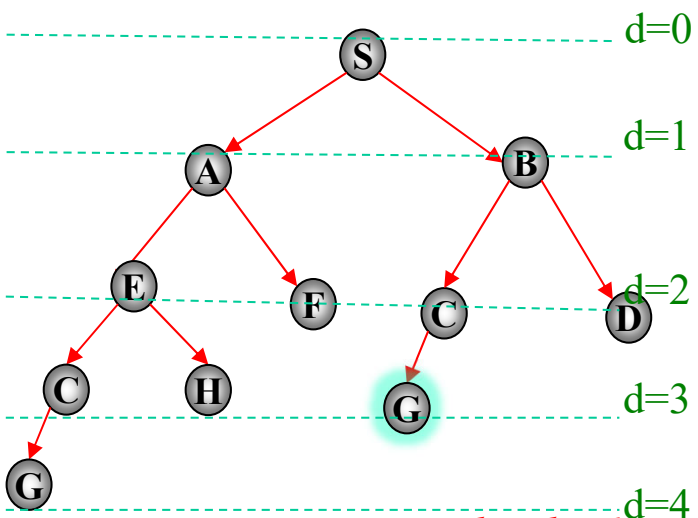
back

Progressive Deepening Search

Depth-Limited Depth-First Search with Visited List (max depth=3)

Recall previous PDS solution that does not use the Visited List

SEARCH TREE



Notation: AS^1

Step	Dequeued	Enqueued	Visited
0		(S^0)	S
1	(S^0)	$\underline{(AS^1)}(BS^1)$	S,A,B
2	(AS^1)	$\underline{(EAS^2)}(FAS^2)(BS^1)$	“ ,E,F
3	(EAS^2)	$\underline{(CEAS^3)}(HEAS^3)(FAS^2)(BS^1)$	“, C,H
4	$(CEAS^3)$. d=3	$\underline{(HEAS^3)}(FAS^2)(BS^1)$	“
5	$(HEAS^3)$. d=3	$\underline{(FAS^2)}(BS^1)$	“
6	(FAS^2)	$\underline{(BS^1)}$	“
7	(BS^1)	$\underline{(DBS^2)}$	“, D
8	(DBS^2)	Q is empty! No solution found.	

back

Progressive Deepening Search

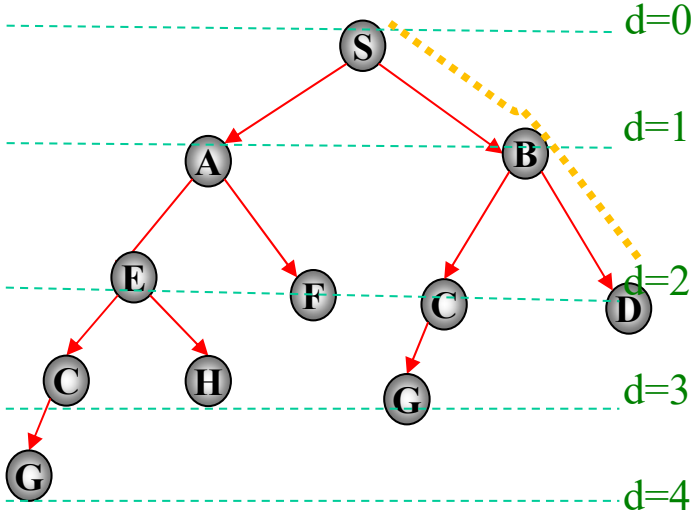
Depth-Limited Depth-

ited

The use of the Visited List caused the algorithm to miss finding a solution!

How can we remedy this?

ARCH TREE



Step	Dequeued	Enqueued	Visited
0		(S^0)	S
1	(S^0)	(AS^1)(BS^1)	S,A,B
2	(AS^1)	(EAS^2)(FAS^2)(BS^1)	“ ,E,F
3	(EAS^2)	($CEAS^3$)($HEAS^3$)(FAS^2)(BS^1)	“, C,H
4	($CEAS^3$). d=3	($HEAS^3$)(FAS^2)(BS^1)	“
5	($HEAS^3$). d=3	(FAS^2)(BS^1)	“
6	(FAS^2)	(BS^1)	“
7	(BS^1)	(DBS^2) Not extended to **C** as it is already in the Visited List	“, D
8	(DBS^2)	Q is empty! No solution found.	

back

Progressive Deepening Search with the Non-Strict Visited List

Pseudocode

1. **C**=1
2. Do **Depth-limited DFS** (**with Non-Strict Visited List)** to max. depth **C**. If path is found, return it.
3. Otherwise, increment **C** and go to Step 2.

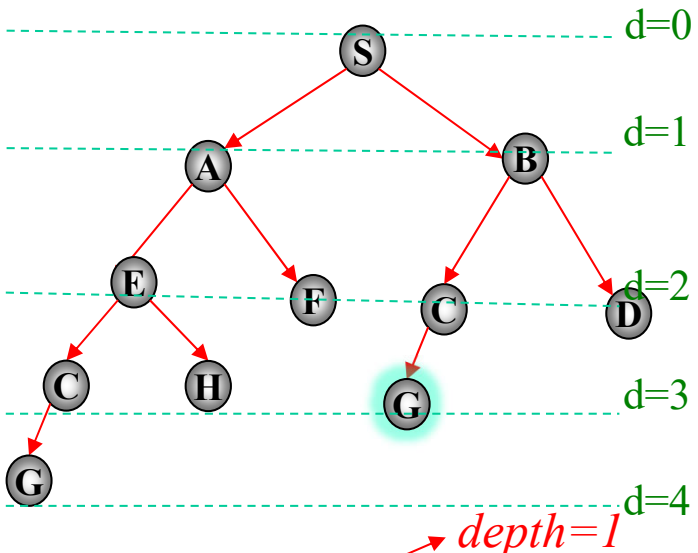
back

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0

SEARCH TREE



Notation: AS^1

We will keep track of the depth at which a state has been visited.

SEARCH

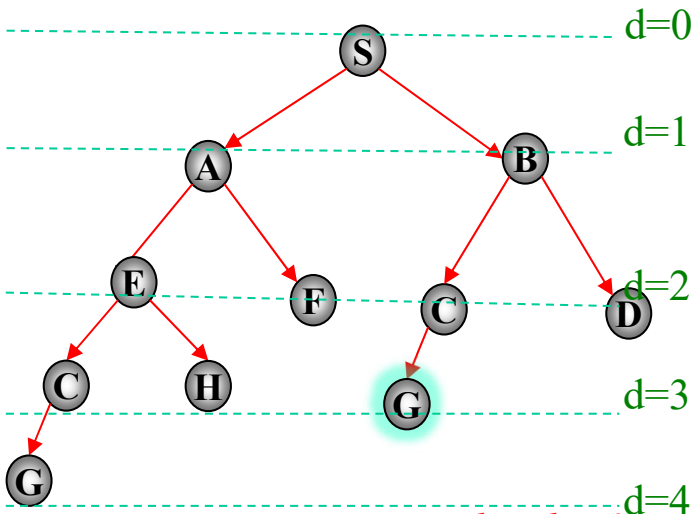
Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0
1	(S^0)	$\underline{(AS^1}\underline{)}(BS^1)$	S^0, A^1, B^1

We will keep track of the depth at which a state has been visited.

SEARCH TREE



Notation: AS^1 — *depth=1*

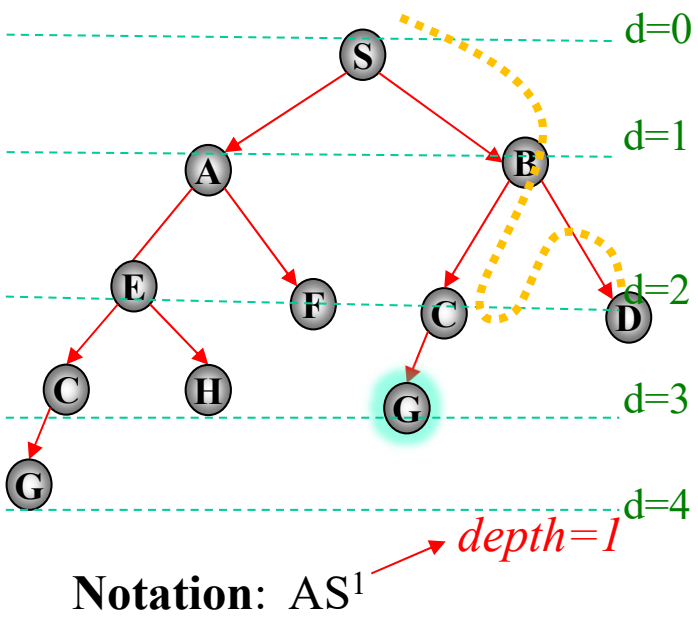
back

SEARCH

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

SEARCH TREE



Notation: AS^1

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0
1	(S^0)	$(AS^1)(BS^1)$	S^0, A^1, B^1
2	(AS^1)	$(EAS^2)(FAS^2)(BS^1)$	“ $, E^2, F^2$
3	(EAS^2)	$(CEAS^3)(HEAS^3)(FAS^2)(BS^1)$	“, ~~C^3~~, H^3
4	$(CEAS^3)$. d=3	$(HEAS^3)(FAS^2)(BS^1)$	“
5	$(HEAS^3)$. d=3	$(FAS^2)(BS^1)$	“
6	(FAS^2)	(BS^1)	“
7	(BS^1)	(CBS^2) (DBS^2)	“, C^2, D^2

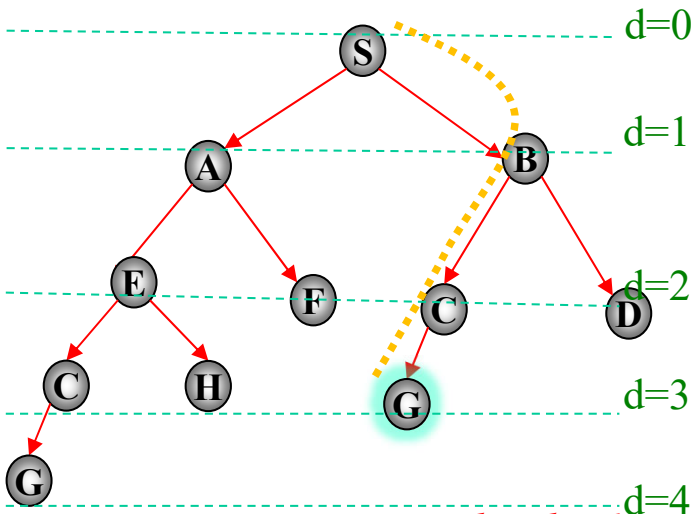
extended to C as even if it is already in the Visited List, since the path is at a lower depth-level

SEARCH

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

SEARCH TREE



Notation: AS^1

Step	Dequeued	Enqueued	Non-Strict Visited List
0		(S^0)	S^0
1	(S^0)	$(AS^1)(BS^1)$	S^0, A^1, B^1
2	(AS^1)	$(EAS^2)(FAS^2)(BS^1)$	" $, E^2, F^2$
3	(EAS^2)	$(CEAS^3)(HEAS^3)(FAS^2)(BS^1)$	", ~~C^3~~, H^3
4	$(CEAS^3)$. d=3	$(HEAS^3)(FAS^2)(BS^1)$	"
5	$(HEAS^3)$. d=3	$(FAS^2)(BS^1)$	"
6	(FAS^2)	(BS^1)	"
7	(BS^1)	(CBS^2) (DBS^2)	", C^2, D^2
8	(CBS^2)	$(GCBS^3)$ (DBS^2)	", G^3
9	$(GCBS^3)$	(DBS^2) *Solution found!*	

back

Progressive Deepening Search

Depth-Limited Depth-First Search with **Non-Strict Visited List** (**max depth=3**)

The non-strict visited list allows the PDS algorithm to add a node whose state already appears in the visited list, provided that the duplicate state has a lower depth-level than the previous state added into the Visited List. This prevents the algorithm from missing the shallowest goal in the tree.

The Farmer, Goose, Fox and Grain Problem

- Learn how to perform abstractions in order to make the problem solvable using Search
- States are generated on-the-fly; that is, neither the graph, nor the tree is given beforehand

n.h.reyes@massey.ac.nz

How to approach this problem using AI?

The farmer-fox-goose-grain problem.

A farmer has to cross a river with his fox, goose and grain. However, his little boat can only accommodate himself and one of his possessions. Therefore, in order to bring all of them across the river, he needs to cross it several times. During planning, he is cautious of two unwanted situations: an unguarded fox will eat the goose, and an unguarded goose will eat the grain.

Image: http://cairnarvon.rotahall.org/2010/02/28/search-trees-are-easy/

How can he cross the river without losing his possessions?

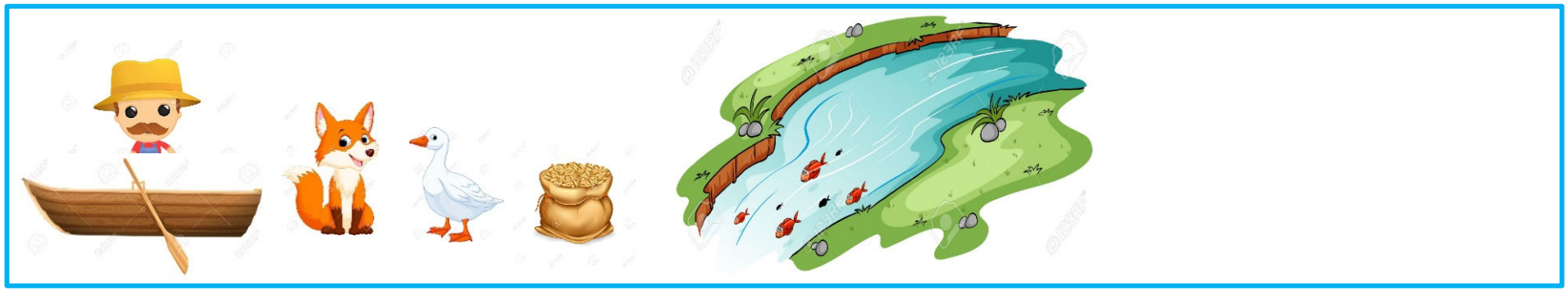
How to represent the problem as a Search problem?

What are the constraints to consider in this problem?

How many possible arrangements (states) are there? How many valid states?

Preconditions

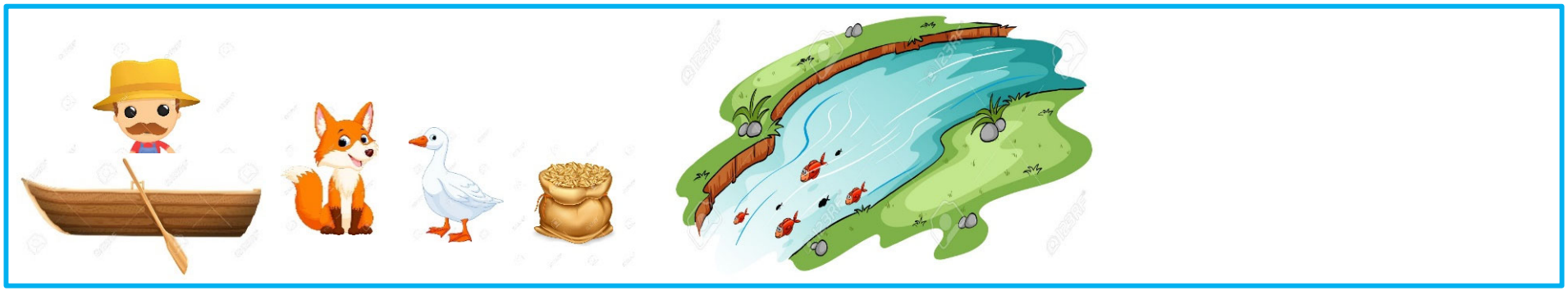
- The **fox** and the **goose** should not be in the same side of the river, not unless the farmer is with them.
- The **goose** and the **grain** should not be in the same side of the river, not unless the farmer is with them.



Left river bank river right river bank

Rules

1. Farmer can travel alone to the right river bank.
2. Farmer can travel alone to the left river bank.
3. Farmer can bring one of his possessions to the right river bank, provided that all preconditions are met.
4. Farmer can bring one of his possessions to the left river bank, provided that all preconditions are met.
5. The farmer should cross the river, every time he makes a move.



Left river bank river right river bank

Representation

How to represent the problem as a Search problem?

The objective is to move all of the farmer's possessions to the right side of the river bank.

What are the **States? Actions? Goal test?**

Initial state of the problem:

Left river bank

river

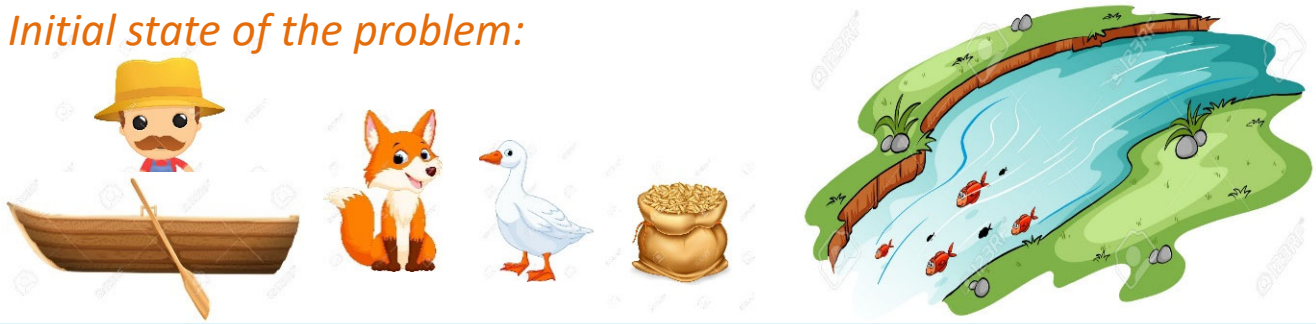
right river bank

States

The **state** is a tuple of **4 booleans**, representing the position of the farmer, goose, fox and grain, relative to the river.

Fa, Fo, Go, Gr

Initial state of the problem:



Left river bank river right river bank

Notation: When all 4 boolean variables are set to false, that means the farmer and all of his possessions are in the left side of the river bank.

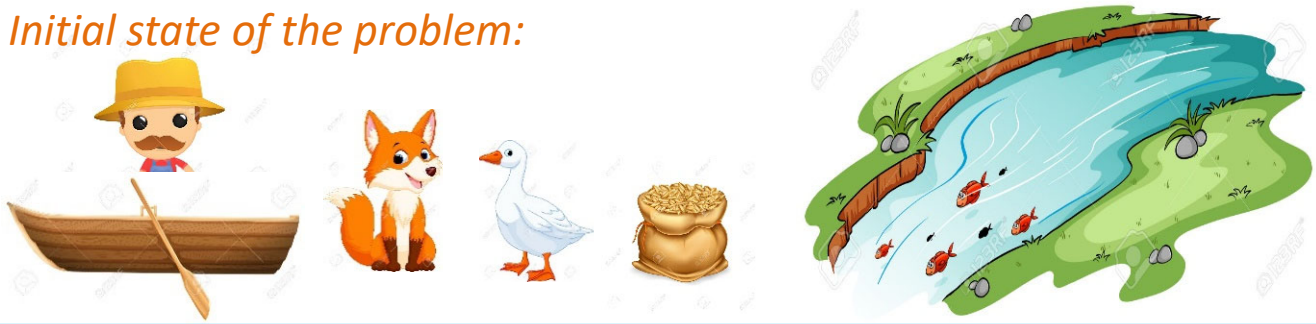
Fa=false, Fo=false, Go=false, Gr=false → 0, 0, 0, 0

States

The **state** is a tuple of **4 booleans**, representing the position of the farmer, goose, fox and grain, relative to the river.

Fa, Fo, Go, Gr

Initial state of the problem:



Left river bank river right river bank

Notation: When all 4 boolean variables are set to false, that means the farmer and all of his possessions are in the left side of the river bank.

Fa=false, Fo=false, Go=false, Gr=false → 0, 0, 0, 0

In this hand-simulation, to make the notations more intuitive, we can write the same state as follows:

< Fa, Fo, Go, Gr | >

The bar indicates the position of the river, relative to all characters.

Actions/Operators

There are **4 operators:** the farmer crosses alone, or with one of his three possessions. In each case all the preconditions must be met.

e.g. the farmer travels to the right river bank, the precondition is goose not equal fox and goose not equal grain.

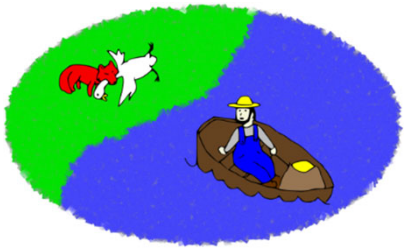
Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.

Invalid States

Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.



The following are invalid states that we should avoid:

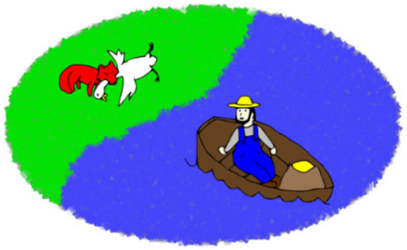
When the farmer travels alone to one of the river banks:

Fo, Go, Gr | **Fa**	The goose eats the grain, or the fox eats the goose first.
Fa | Fo, Go, Gr	The goose eats the grain, or the fox eats the goose first.

Invalid States

Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.



The following are invalid states that we should avoid:

When the farmer travels alone to one of the river banks:

Fo, Go, Gr | **Fa**	The goose eats the grain, or the fox eats the goose first.
Fa | Fo, Go, Gr	The goose eats the grain, or the fox eats the goose first.

When the farmer travels with one of his possessions to one of the river banks:

Fo, Go | **Fa**, Gr	the fox eats the goose.
Fa, Gr | Fo, Go	the fox eats the goose.
Go, Gr | **Fa**, Fo	the goose eats the grain.
Fa, Fo | Go, Gr	the goose eats the grain.

There are (2^4)=**16 states** in total, but **6 are invalid**. Therefore, there are only **10 valid** states.

Goal Test

How to check if the algorithm has succeeded already?

Goal test:

All Booleans are true. That means all possessions, including the farmer are already on the right river bank.

i.e. farmer (**Fa**) == fox (**Fo**) == goose (**Go**) == grain (**Gr**) == true

Using our notation for the hand-simulation:

| Fa, Fo, Go, Gr

*To keep track of the characters in the puzzle more easily, enumerate them in alphabetical order

Exercise

Solve the problem using Depth-First Search

In your solution, consider the following sequence of moves:

1. Farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.

Initial state of the problem:

Left river bank

river

right river bank

5

STATE-SPACE SEARCH

Optimal Search

Optimal Uninformed Search: **Uniform-Cost Search**

n.h.reyes@massey.ac.nz

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)
Optimal Uninformed	**Uniform-Cost**	Uses path **"length"** measure. Finds "shortest" path.

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

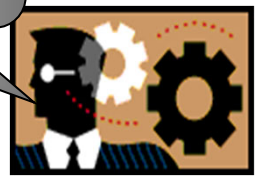
The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

This is what we have seen previously.



Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

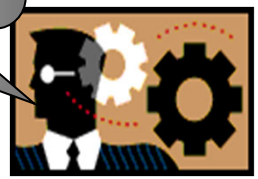
The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry; ~~set Visited = (S)~~.
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not ~~in Visited~~ and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; ~~add children of state (N) to Visited~~.
7. Go to Step 2.

Do NOT use Visited List for **Optimal** Searching!



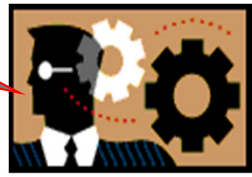
Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Optimal Search

Why not a Visited List?

Why can't we use a Visited List in connection with optimal searching? In the earlier searches, the use of the Visited List guaranteed that we would not do extra work by re-visiting or re-expanding states. It did not cause any failures then (except possibly of intuition)

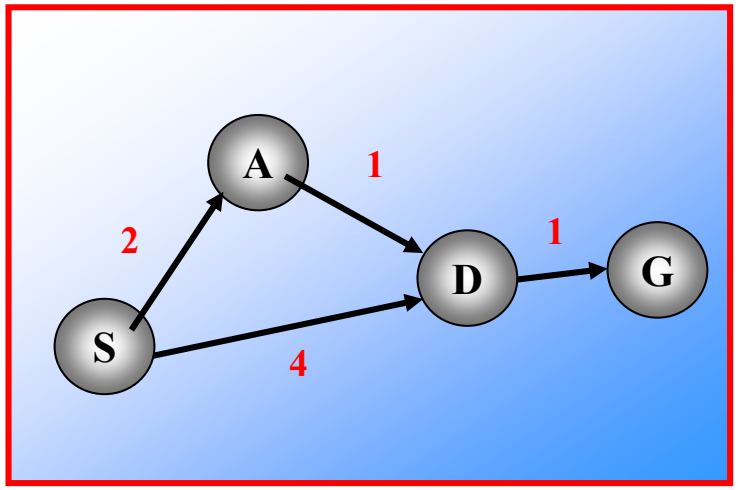


- For the **Any-Path algorithms**, the **Visited List** would not cause us to fail to find a path when one existed, since the path to a state did not matter.

- However, the **Visited List** in connection with **Optimal searches** can cause us to miss the best path.

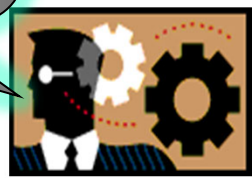
Why not a Visited List?

- For the **Any-Path algorithms**, the Visited List would not cause us to fail to find a path when one existed, since the path to a state did not matter.
- However, the Visited List in connection with **Optimal searches** can cause us to miss the best path.



- The shortest path from S to G is (S A D G).

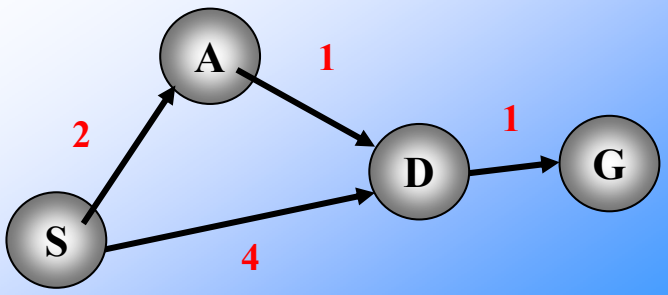
Clearly, the shortest path (as determined by sum of link costs) to G is (S A D G) and an optimal search had better find it.



SEARCH

Why not a Visited List?

- For the **Any-Path algorithms**, the Visited List would not cause us to fail to find a path when one existed, since the path to a state did not matter.
- However, the Visited List in connection with **Optimal searches** can cause us to miss the best path.



- But on extending (S), A and D would be added to the Visited List and so (S A) would not be extended to (S A D)

Step	Dequeued	Q	Visited
0		~~(S)~~	S
1	(S)	~~(AS)~~ (DS)	S,A,D
2	(AS)	~~(DS)~~	
3	(DS)	~~(GDS)~~	
4	(GDS)	This is not the shortest path.	

- The shortest path from S to G is (S A D G).

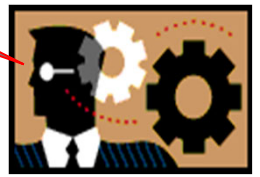
back

Implementing Optimal Search Strategies

Uniform Cost

- Pick the **best** (**measured by path length**) element of Q.
- Add path extensions **anywhere** in Q.

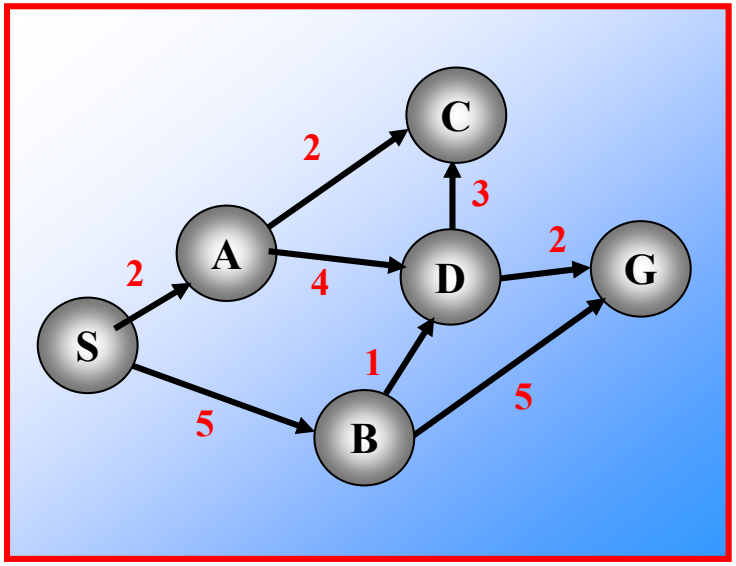
For our convention in class, add the path extensions to the **front** of the Queue.



Implementing Optimal Search Strategies

Uniform Cost

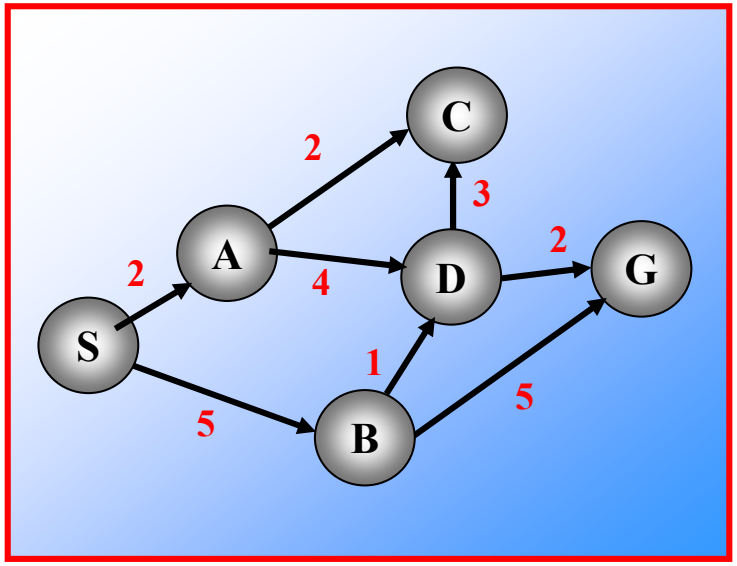
- Like **Best-First Search** except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



Implementing Optimal Search Strategies

Uniform Cost

- Like Best-First Search except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



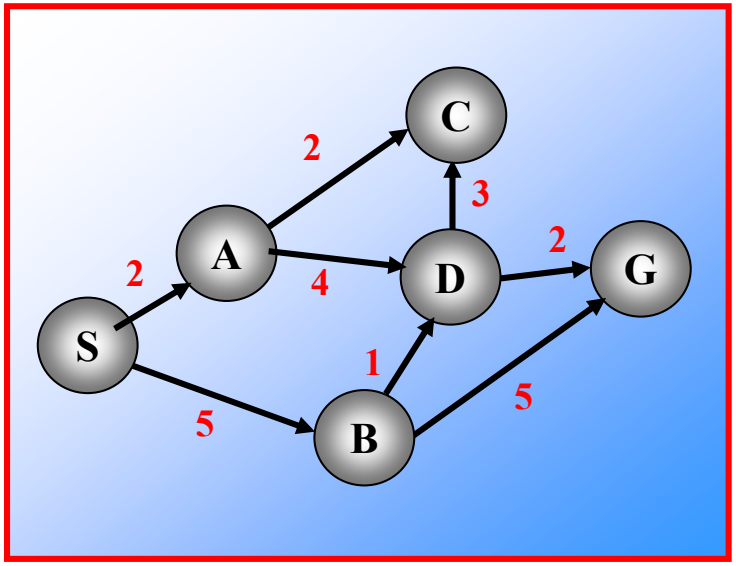
Total path cost:

S-A-C 4

Implementing Optimal Search Strategies

Uniform Cost

- Like Best-First Search except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



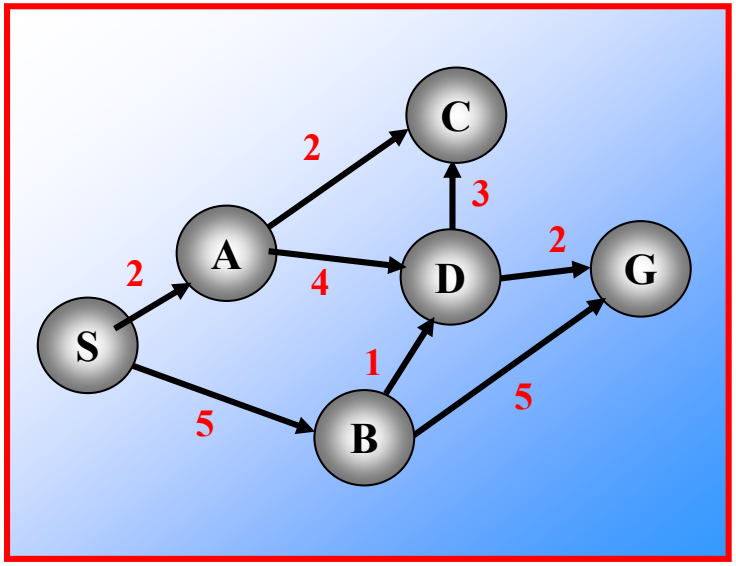
Total path cost:

S-A-C 4
S-B-D-G 8

Implementing Optimal Search Strategies

Uniform Cost

- Like Best-First Search except that it uses the **"total length (cost)"** of a path instead of a heuristic value for the state.
- Each link has a **"length"** or **"cost"** (which is always greater than 0)
- We want **"shortest"** or **"least cost"** path



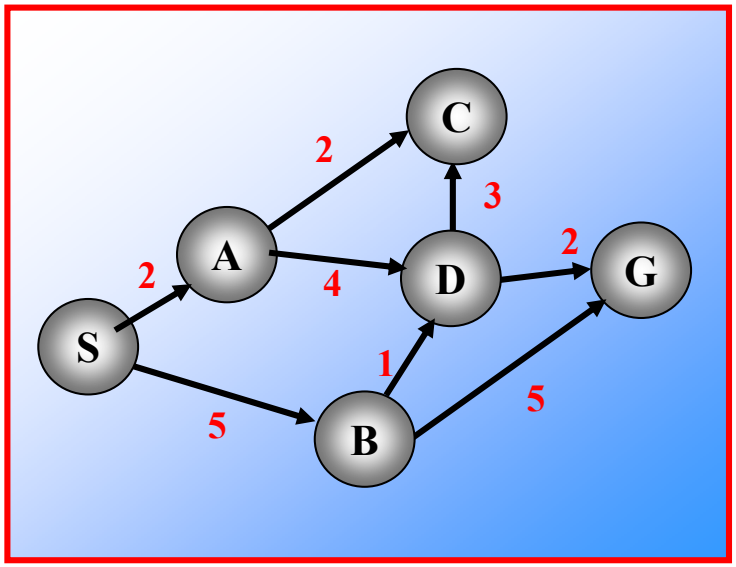
Total path cost:

S-A-C	4
S-B-D-G	8
S-A-D-C	9

SEARCH

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)



Note: the underlined paths are chosen for extension.

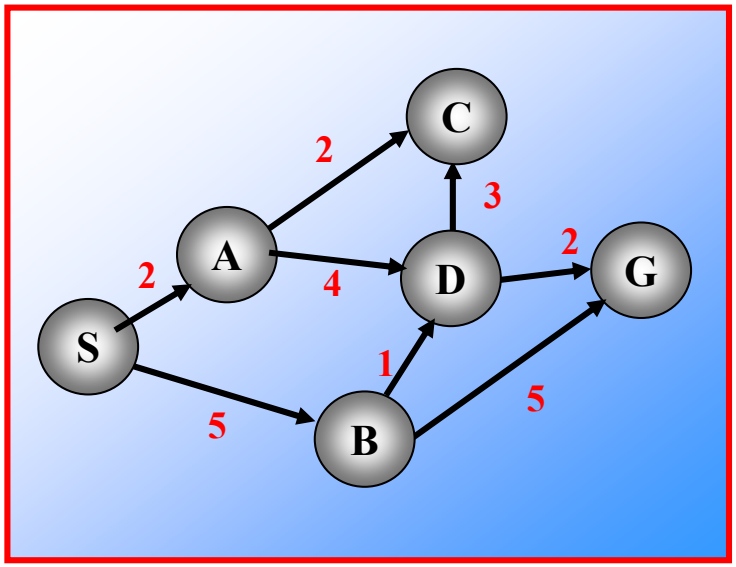
The most recent path is added at the **front of Q** (order is used to settle ties).

Pick the **best** (by path length) element of Q; Add path extensions to **front** of Q.

SEARCH

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS) (8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)



Max Q length = 5
Number of State Expansions = 7
Path = S-A-D-G
Path length = 8

Sequence of State Expansions:
S-A-C-B-D-D-G

Pick the **best** (by path length) element of Q; Add path extensions to **front** of Q.

Why not stop on first visiting a goal?

Uniform Cost

- When doing Uniform Cost, it is **not** correct to stop the search when the first path to a goal is generated; that is, when a node whose state is a goal is added to Q. (**look at Step#5 of the simulation earlier**)

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)

Why not stop on first visiting a goal?

- When doing Uniform Cost, it is **not** correct to stop the search when the first path to a goal is generated; that is, when a node whose state is a goal is added to Q. (**look at Step#5 of the simulation earlier**)

- We must **wait** until such a path is **pulled off the Q** and tested in Step 3. It is only at this point that we are sure it is the shortest path to a goal since there are no other shorter paths that remain unexpanded.

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)

Why not stop on first visiting a goal?

Uniform Cost

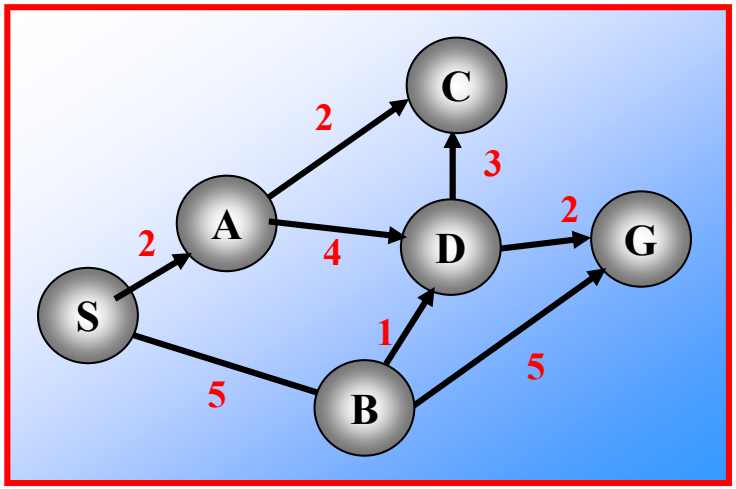
- When doing Uniform Cost, it is not correct to stop the search when the first path to a goal is generated; that is, when a node whose state is a goal is added to Q.

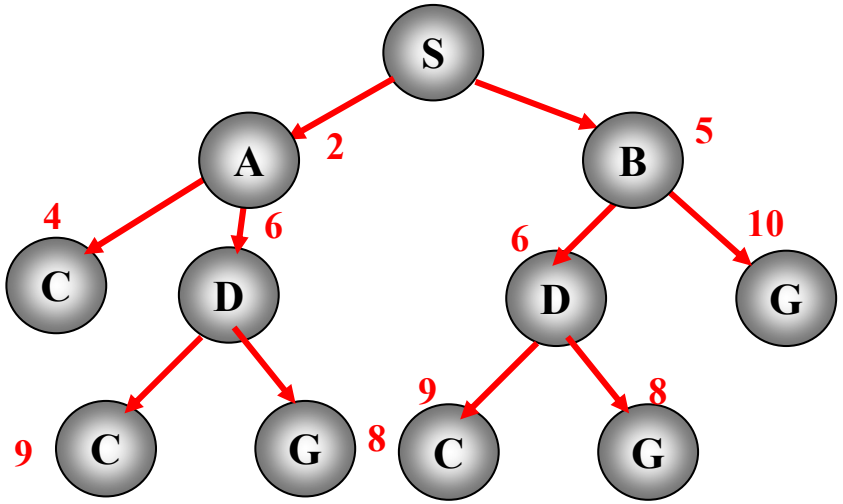
- We must wait until such a path is pulled off the Q and tested in Step 3. It is only at this point that we are sure it is the shortest path to a goal since there are no other shorter paths that remain unexpanded.

- This contrasts with the **non-optimal searches** where the choice of where to test for a goal was a matter of convenience and efficiency, not correctness.

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)

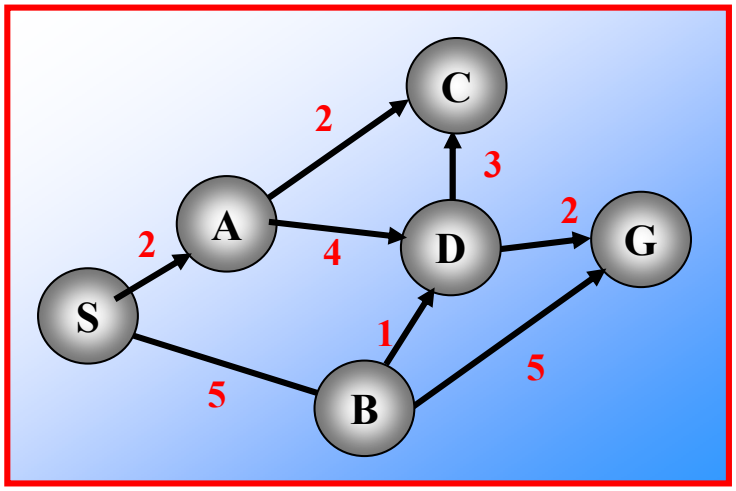


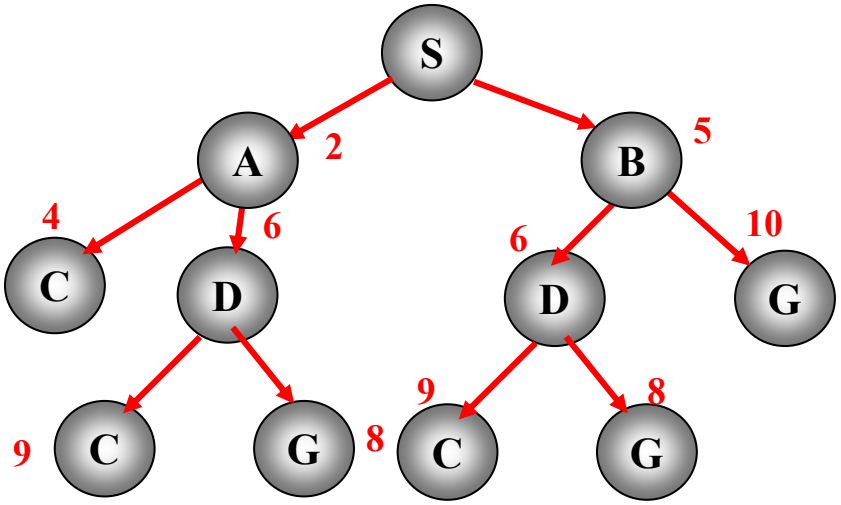


Uniform Cost enumerates paths in order of total path cost!

Uniform Cost

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(2 AS)(5 BS)
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)
3	(4 CAS)	(6 DAS)(5 BS)
4	(5 BS)	(6 DBS)(10 GBS)(6 DAS)
5	(6 DBS)	(9 CDBS) (8 GDBS) (10 GBS)(6 DAS)
6	(6 DAS)	(9 CDAS) (8 GDAS) (9 CDBS)(8 GDBS) (10 GBS)
7	(8 GDAS)	(9 CDAS)(9 CDBS)(8 GDBS) (10 GBS)





The sequence of path extensions corresponds precisely to path-length order, so it is not surprising we find the shortest path.

Sequence of State Expansions: S – A – C – B – D – D - G

0 2 4 5 6 6 8

Uniform Cost enumerates paths in order of total path cost!

5

STATE-SPACE SEARCH

Optimal Search

Optimal Informed Search: **A* Search**

n.h.reyes@massey.ac.nz

Classes of Search

Class	Name	Operation
Any Path Uninformed	**Depth-First**	Systematic exploration of the whole tree until a goal is found
	Breadth-First	
Any Path Informed	**Best-First**	Uses heuristic measure of goodness of a state (e.g. estimated distance to goal)
Optimal Uninformed	**Uniform-Cost**	Uses path **"length"** measure. Finds "shortest" path.
Optimal Informed	**A***	Uses path **"length"** measure and **heuristic**. Finds "shortest" path.

back

Goal Direction

- **UC** is really trying to identify the **shortest path** **to** **every** **state** in the graph in order. It has **no particular bias** to finding a path to a goal early in the search.

Goal Direction

- **UC** is really trying to identify the shortest path to every state in the graph in order. It has no particular bias to finding a path to a goal early in the search.

- We can introduce such a **bias** by means of heuristic function **h(state(N))**, which is an estimate (**h**) of the distance from a state to the goal.

Goal Direction

- **UC** is really trying to identify the shortest path to every state in the graph in order. It has no particular bias to finding a path to a goal early in the search.

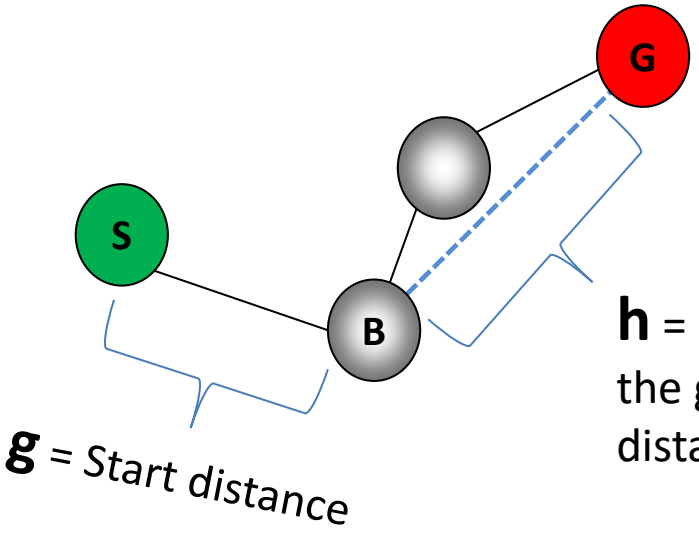
- We can introduce such a bias by means of heuristic function h(state(N)), which is an estimate (h) of the distance from a state to the goal.

- Instead of enumerating paths in order of just length (**g**), enumerate paths in terms of **f = estimated total path length = g + h.**

SEARCH

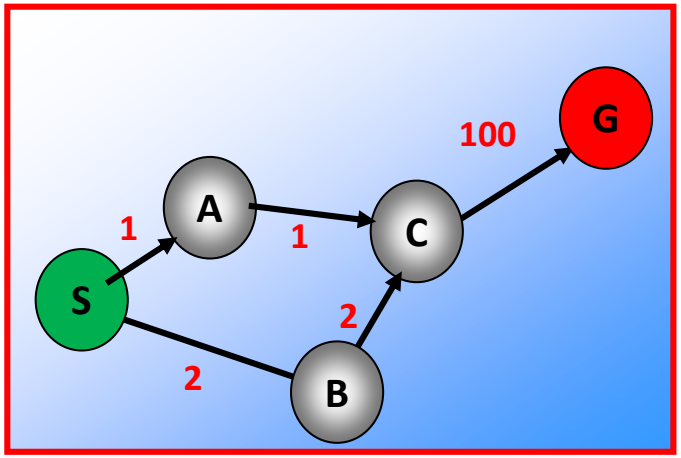
f= g + h

f(BS) = g(S→B) + h(state(BS))



h = Estimate of the goal distance

g = *Start distance*



Heuristic Values:
A=100, B=88, C=100, S=90, G=0

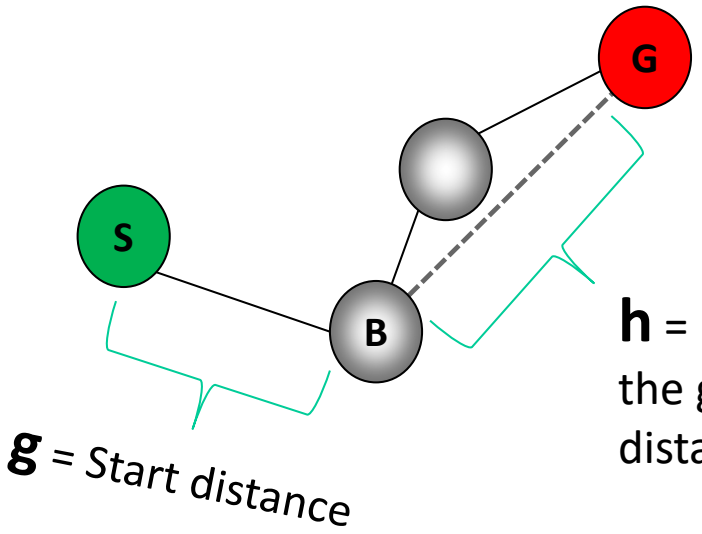
- Instead of enumerating paths in order of just length (**g**), enumerate paths in terms of **f = estimated total path length = g + h.**

- An estimate that always underestimates the real path length to the goal is called **admissible**. For example, an estimate of **0** is **admissible** (**but useless**). Straight line distance is admissible estimate for path length in Euclidean space.

Goal Direction

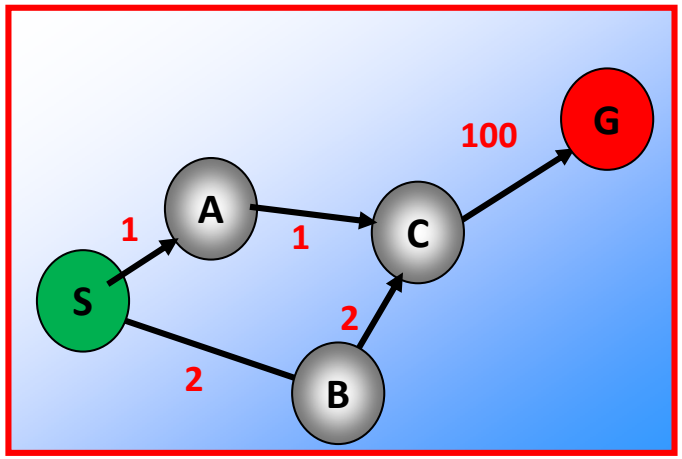
f= g + h

$$f(BS) = g(S \rightarrow B) + h(state(BS))$$



g = *Start distance*

h = Estimate of the goal distance



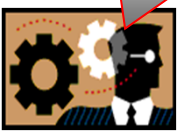
Heuristic Values:
A=100, B=88, C=100, S=90, G=0

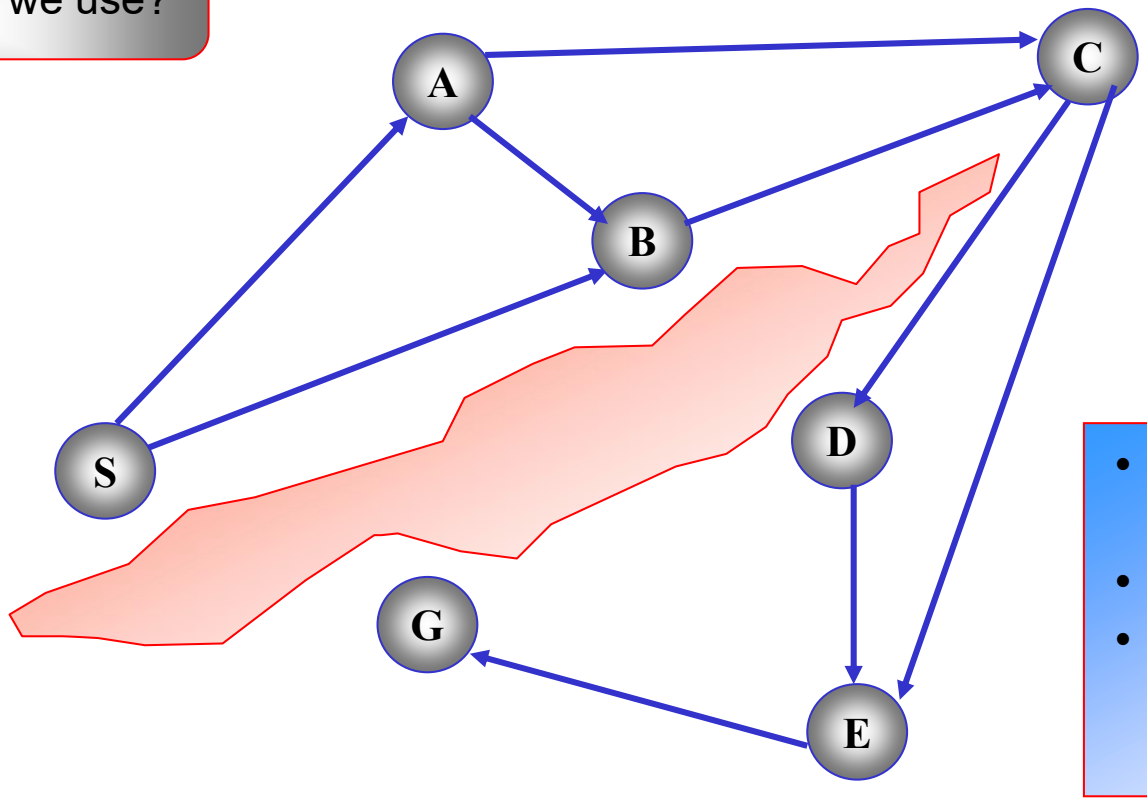
- Use of an **admissible estimate** guarantees that **UC** will still find the **shortest** path.

- **UC with an admissible estimate** is known as **A* Search**.

Straight-Line Estimate

Which heuristic function can we use?



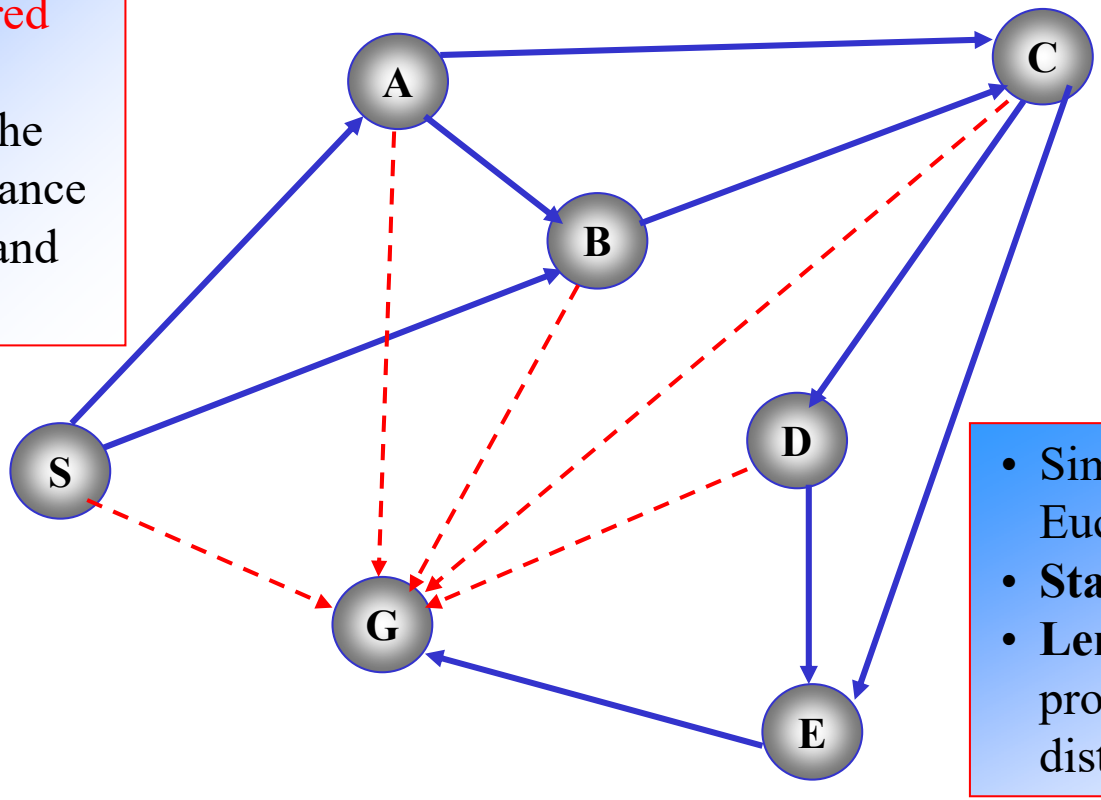


- Simple graph embedded in Euclidean space
- **States** as city locations
- **Length of the links** are proportional to the driving distances between cities

In Euclidean space

Straight-Line Estimate

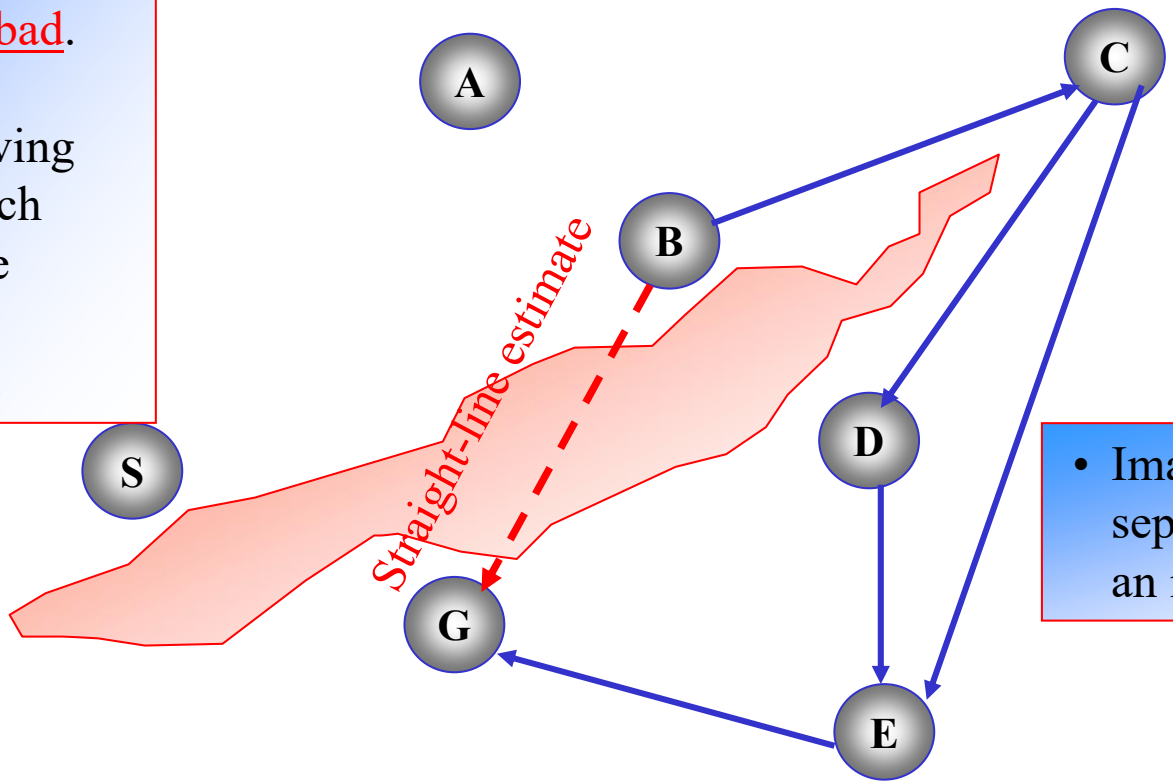
- We can use the straight-line (airline) distances (dotted red lines) as an underestimate of the actual driving distance between any city and the goal.



- Simple graph embedded in Euclidean space
- **States** as city locations
- **Length of the links** are proportional to the driving distances between cities

Straight-Line Estimate

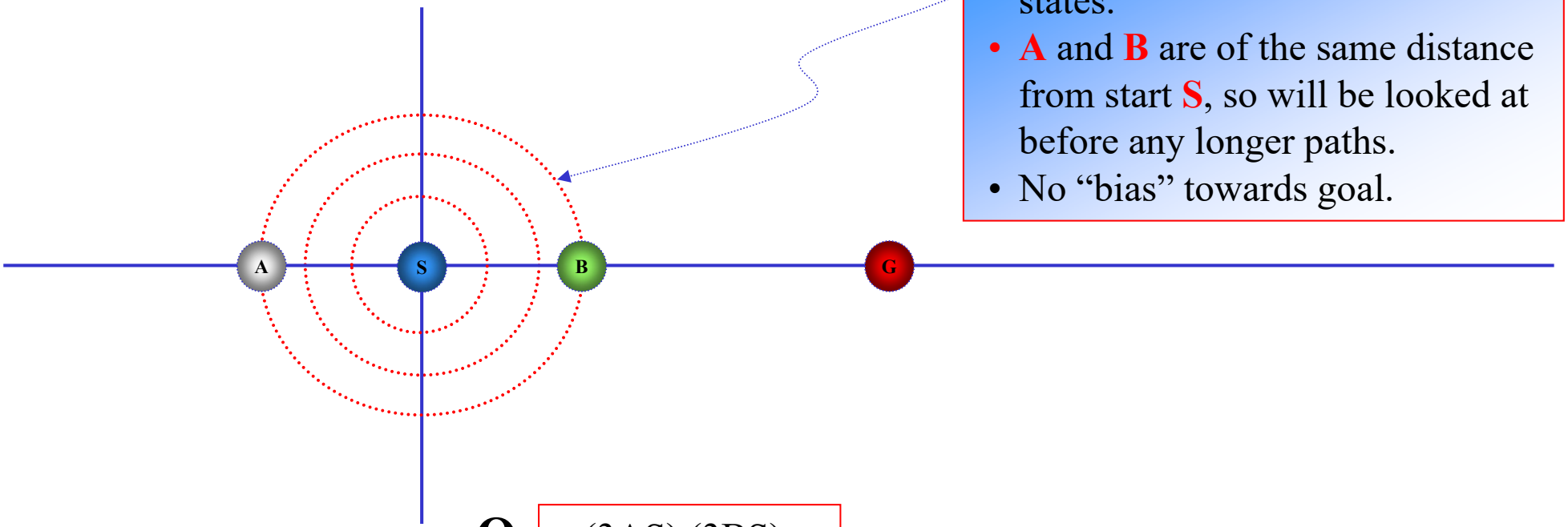
- Here we can see that the straight-line estimate between **B** and **G** is very bad.
- The actual driving distance is much longer than the straight-line underestimate.



- Imagine that **B** and **G** could be separated by a big mountain or an island

Why use estimate of goal distance?

- Order in which **UC** looks at states.
- **A** and **B** are of the same distance from start **S**, so will be looked at before any longer paths.
- No "bias" towards goal.



i.e. **Q** (3AS) (3BS)

- Assume **states** are **points** in the Euclidean space

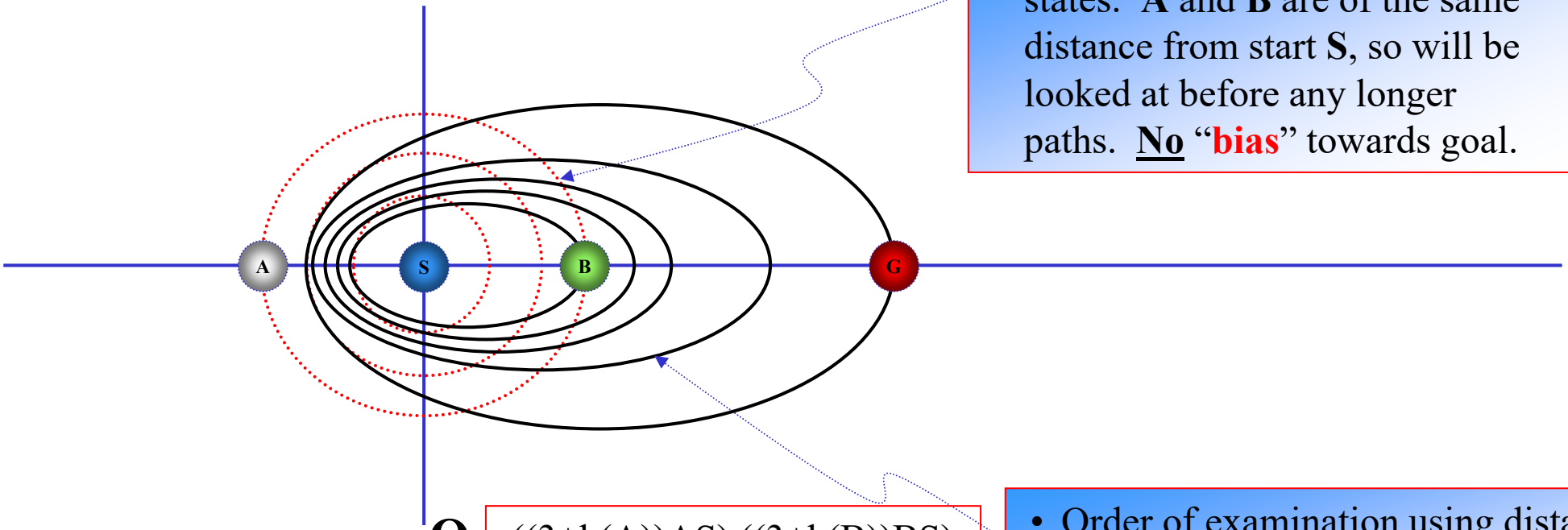
Based on the ordering, this node is will be expanded first

SEARCH

Why use estimate of goal distance?

- Assume **states** are **points** in the Euclidean space

- Order in which **UC** looks at states. **A** and **B** are of the same distance from start **S**, so will be looked at before any longer paths. **No** "**bias**" towards goal.



e.g. **Q** ((3+h(A))AS) ((3+h(B))BS)

12 8

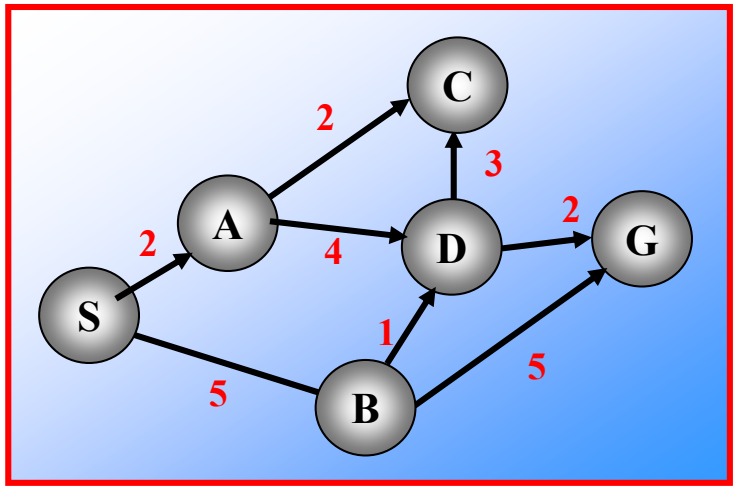
Q = (15AS)(11BS)

- Order of examination using distance from **S** + estimate of distance to **G**.
- Note: **"bias"** towards the goal;
- The points away from **G** look worse

f= g + h

A*

Given the following graph and heuristic values, find a path from **S** to **G** using the A* algorithm.



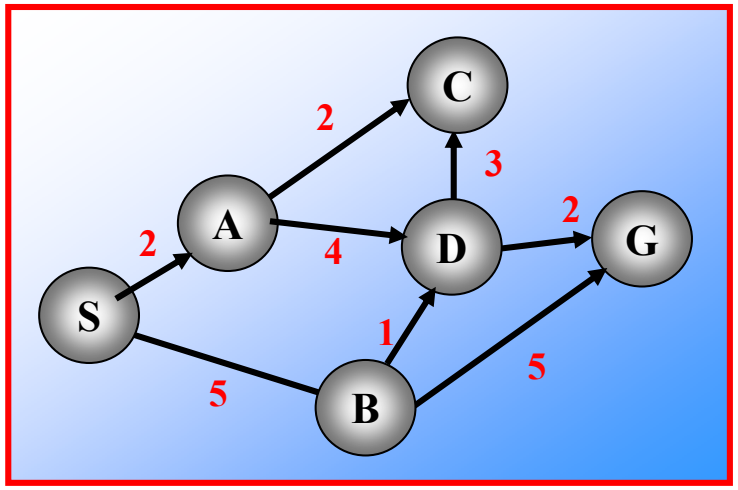
Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

For our convention in class, add the path extensions to the **front** of the Q.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

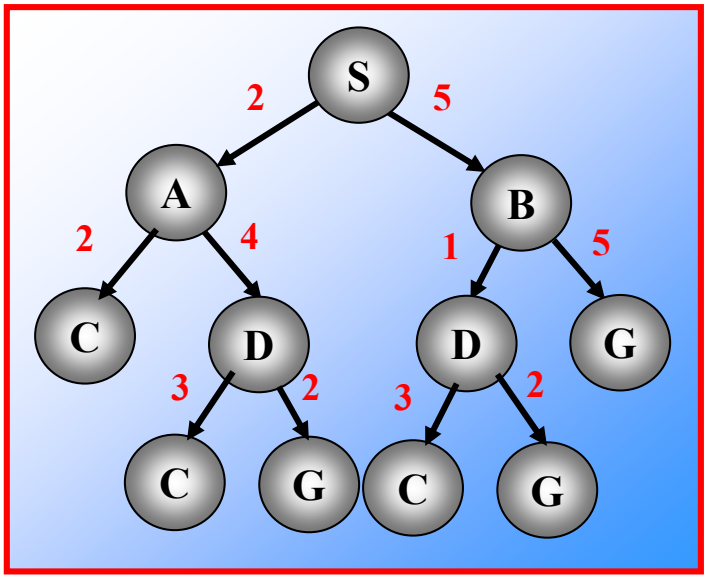
A*

Given the following graph and heuristic values, find a path from **S** to **G** using the A* algorithm.



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

Search Tree



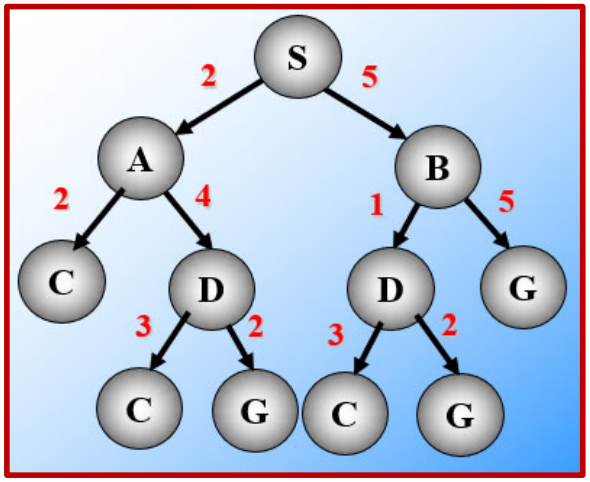
For our convention in class, add the path extensions to the front of the Q.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(4 AS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

f= g + h

$f(AS) = g(S \rightarrow A) + h(state(AS))$

$f(AS) = 2 + h(A)$

$= 2 + 2 = 4$

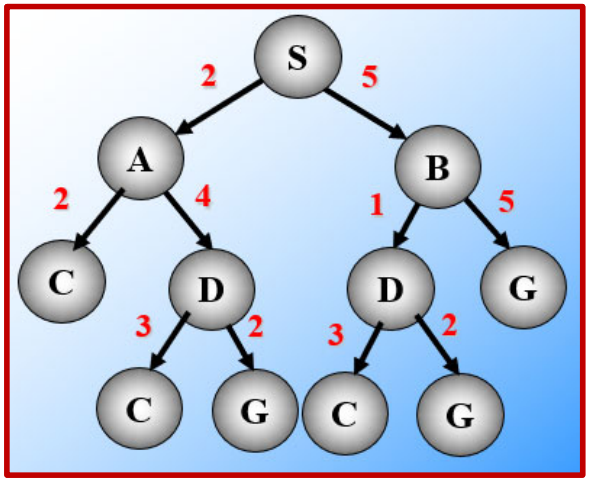
For our convention in class, add the path extensions to the **front** of the Q.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		(0 S)
1	(0 S)	(4 AS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

f= g + h

f(BS) = g(S→B) + h(state(BS))

f(AS) = 5 + h(B)

= 5 + 3 = 8

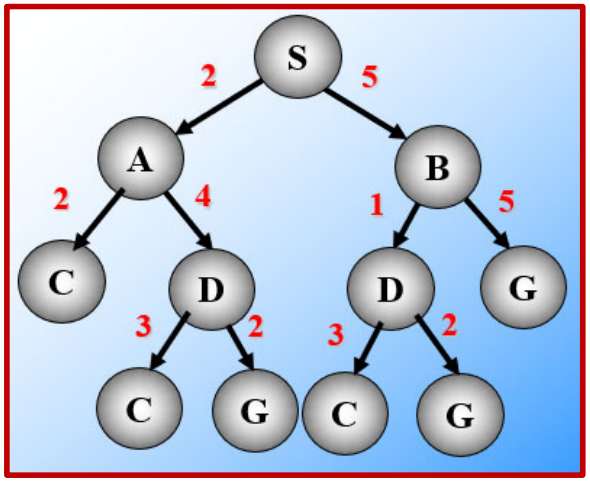
For our convention in class, add the path extensions to the **front** of the Q.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		~~(0 S)~~
1	(0 S)	~~(4 AS)~~(8 BS)
2	(4 AS)	(5 CAS)(7 DAS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

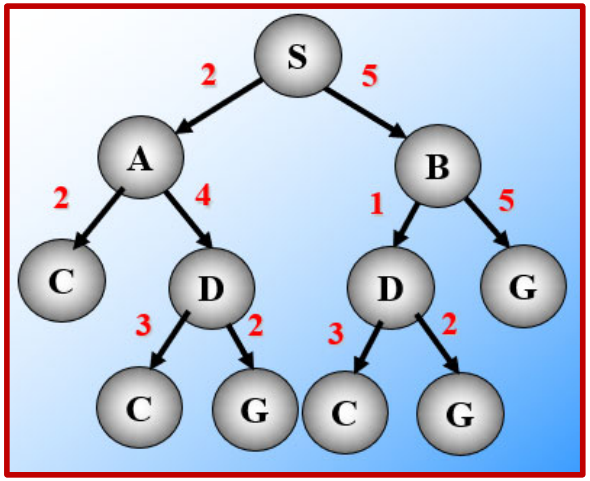
For our convention in class, add the path extensions to the front of the Queue.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		~~(0 S)~~
1	(0 S)	~~(4 AS)~~(8 BS)
2	(4 AS)	(5 CAS)(7 DAS)(8 BS)



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

f= g + h

$$f(CAS) = g(S \rightarrow A \rightarrow C) + h(state(CAS))$$

$$f(AS) = 4 + h(C)$$
$$= 4 + 1 = 5$$

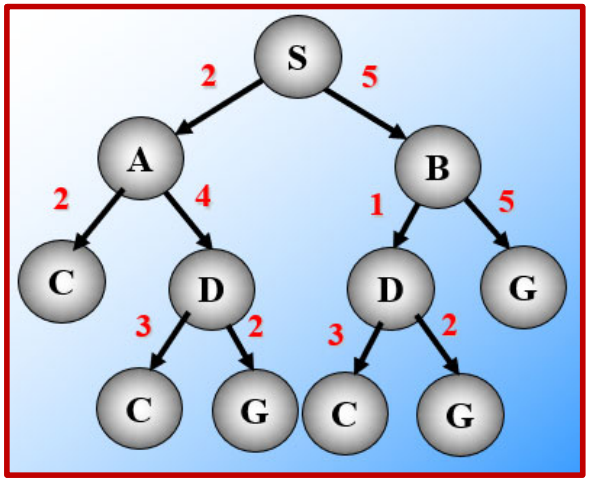
For our convention in class, add the path extensions to the **front** of the Queue.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

SEARCH

A*

Step	Dequeued	Q
0		~~(0 S)~~
1	(0 S)	(4 ~~AS~~)(8 BS)
2	(4 AS)	(5 ~~CAS~~)(7 DAS)(8 BS)
3	(5 CAS)	(7 ~~DAS~~)(8 BS)
4	(7 DAS)	(10 CDAS)(8 ~~GDAS~~) (8 BS)
5	(8 GDAS)	



Heuristic Values:
A=2, B=3, C=1, D=1, S=0, G=0

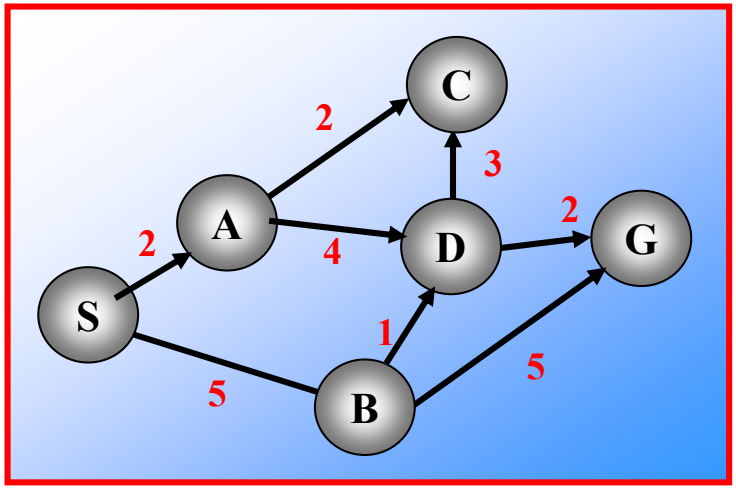
Sequence of State Expansions: S – A – C – D - G

For our convention in class, add the path extensions to the **front** of the Queue.

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions **anywhere** in Q.

Not all heuristics are admissible

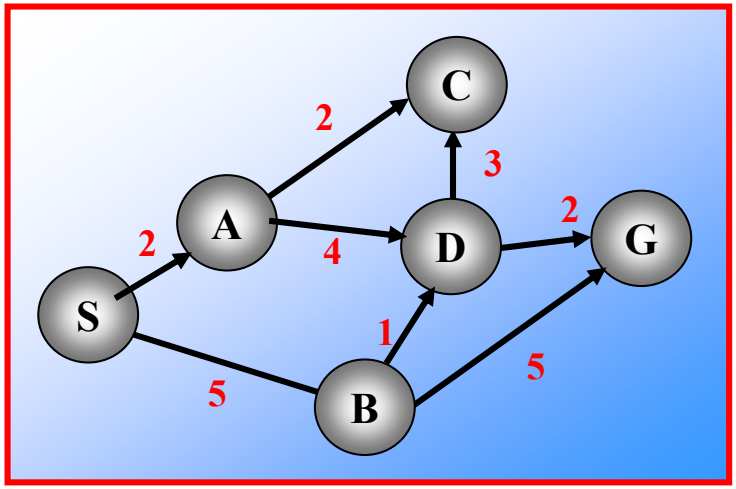
Given the link lengths in the figure, is the table of heuristic values that we used in our earlier Best-First example an **admissible heuristic**?



Heuristic values:
A=2, B=3, C=1, D=4, S=10, G=0

Not all heuristics are admissible

Given the link lengths in the figure, is the table of heuristic values that we used in our earlier Best-First example an **admissible heuristic**?



Heuristic values:
A=2, B=3, C=1, D=4, S=10, G=0

No!

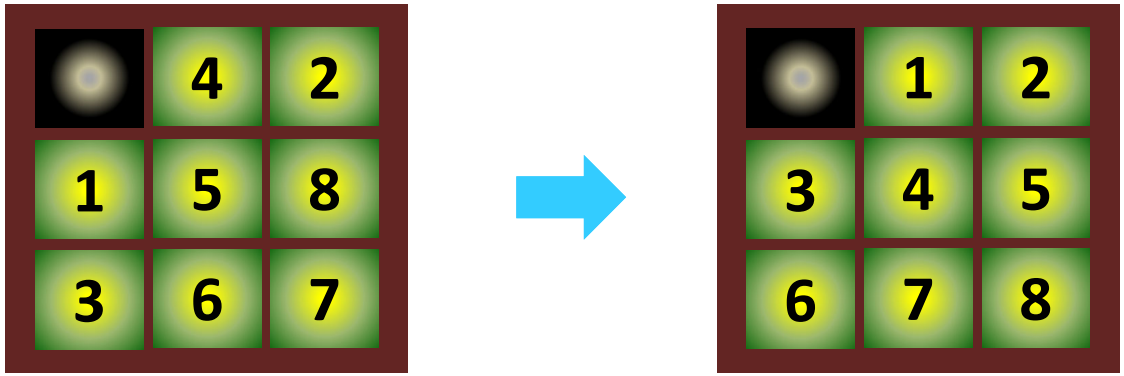
- A is ok
- B is ok
- C is ok
- **D** is **too big**, needs to be <= 2
- S is too big, can always use 0 for start

Solving the 8-Puzzle using Search

back

8 Puzzle Problem

Given the **start** state, find the sequence of moves that will arrange the tiles according to the **goal** state.



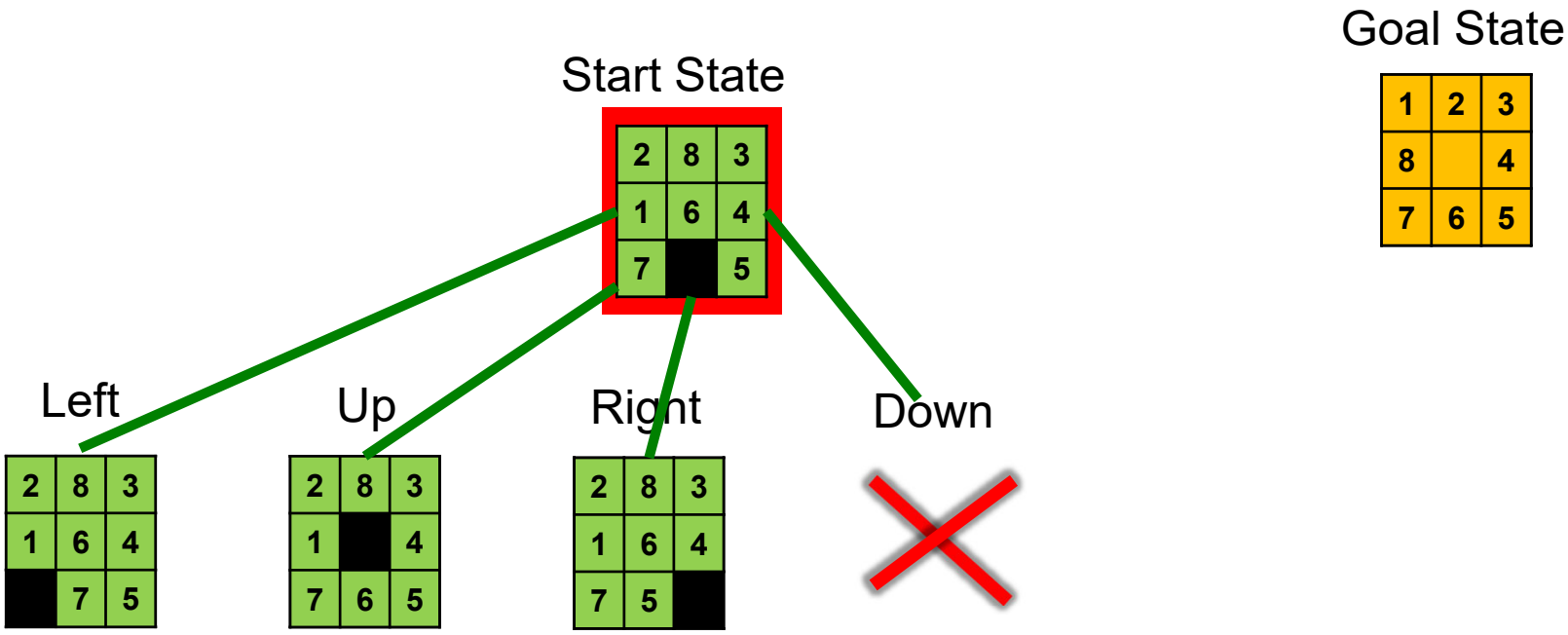
Start **Goal**

Tiles can be moved **up, down, left,** or **right**, provided that the following conditions are met:

- There is no other tile blocking the direction of the movement.
- Tiles are not moved outside of the boundaries.

- Let’s look at the details of how A* solves this problem.

Search

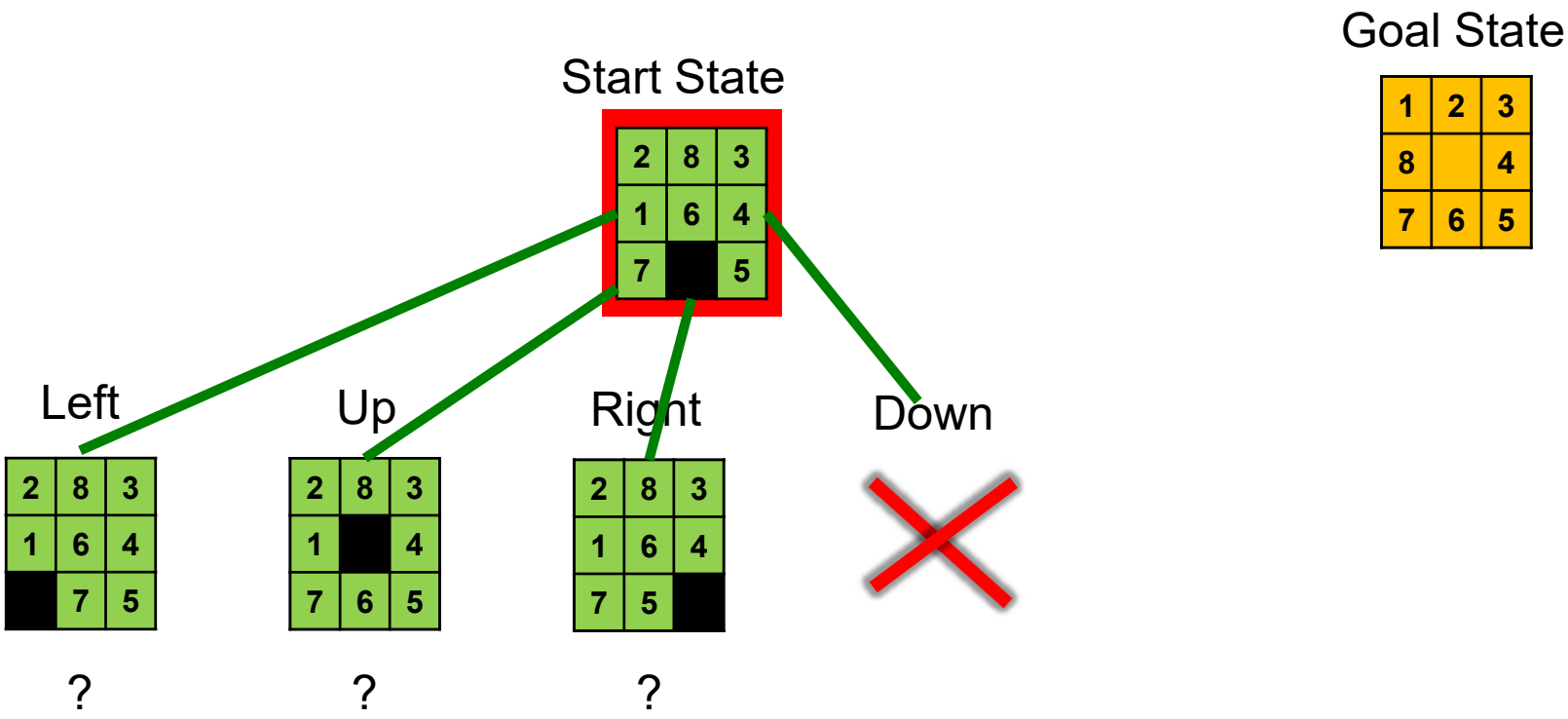


Given the **start state**, we can examine the different possible moves that can be carried out from that state.

Think of the moves as moving the **blank tile** in the following order: Left, Up, Right, Down

menu

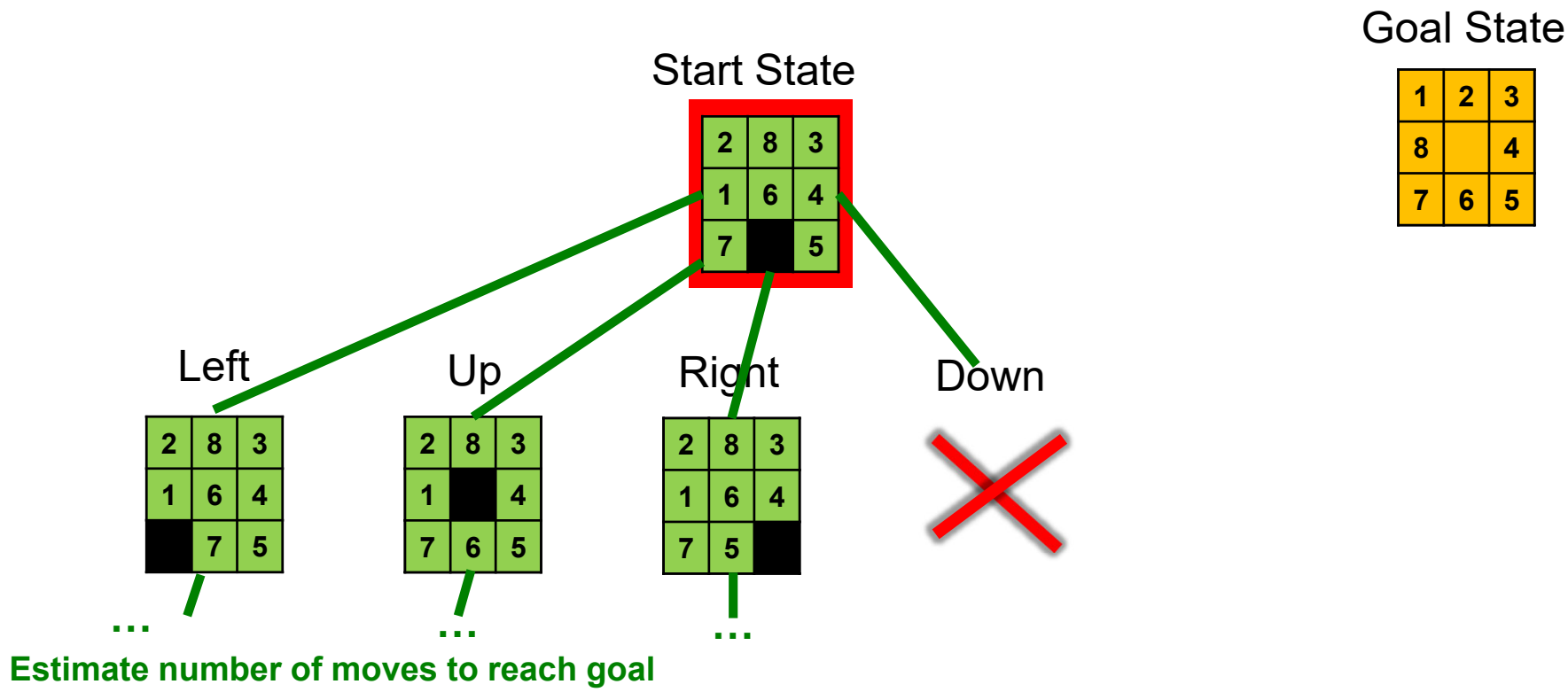
Search



But can we tell which among the configuration of tiles will likely lead us to the goal more quickly?

menu

Heuristic Search



But can we tell which among the config. of tiles will likely lead us to the goal more quickly?

We can use a heuristic function to find an estimate of the remaining number of moves to reach the goal.

Sum of Manhattan Distance of tile to its goal location

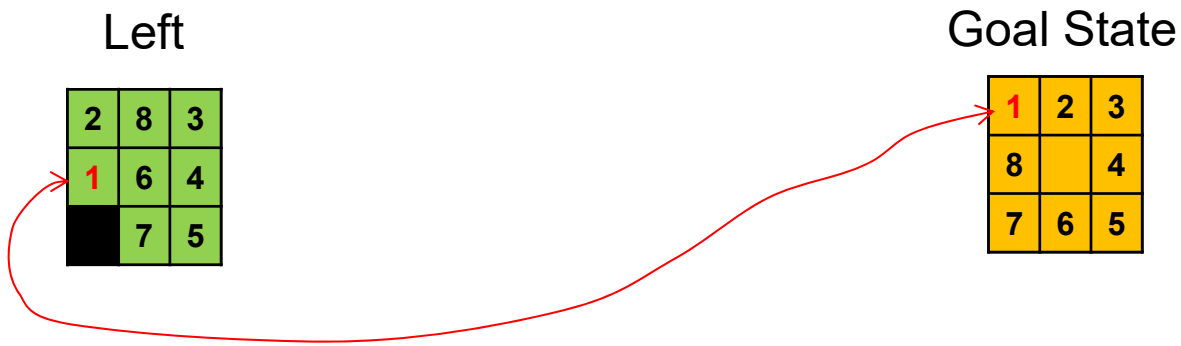
Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Heuristic Search

Let's apply the Manhattan Distance formula to the following puzzle configuration.



Tile 1 *is 1 distance unit away from its target position*

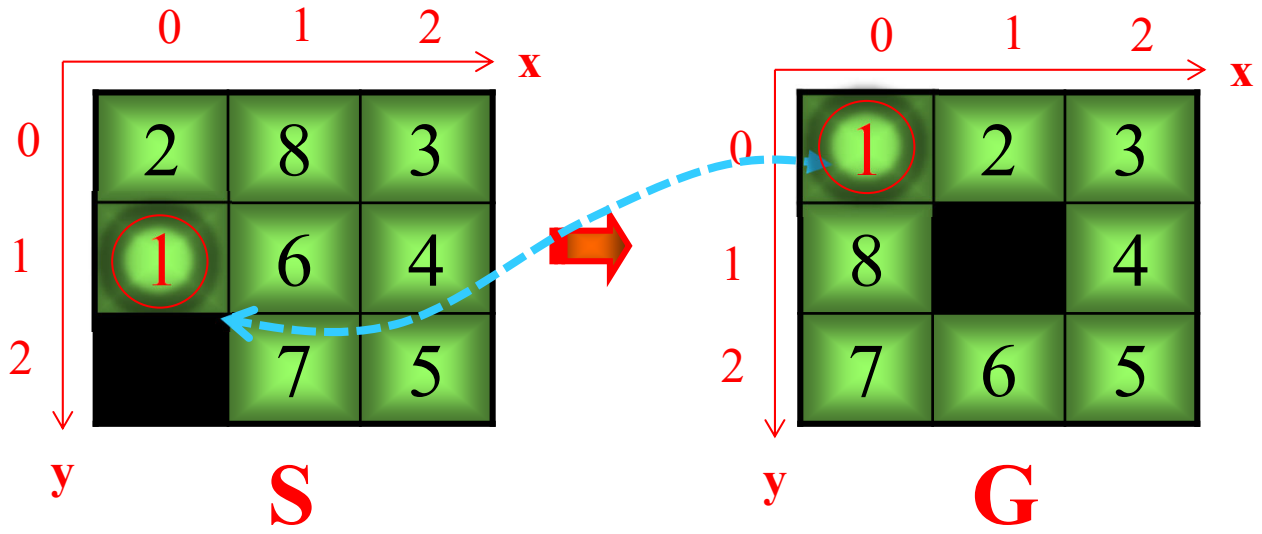
Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Admissible Heuristics



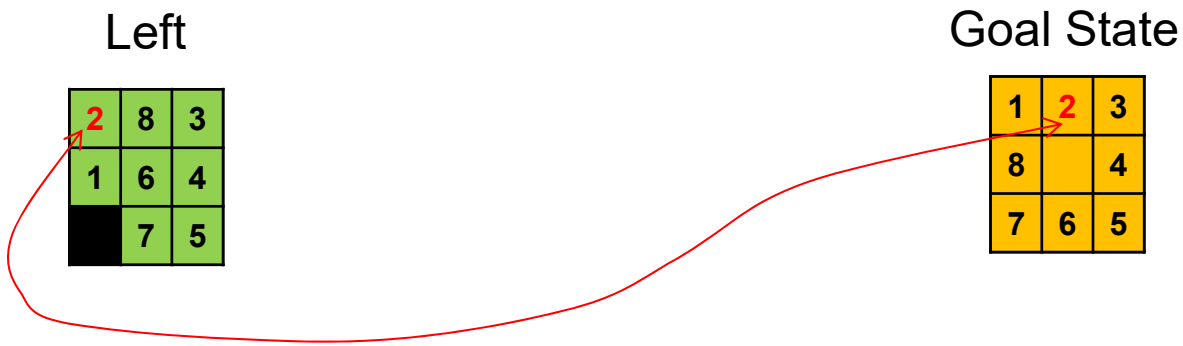
Sample Computations:

Tile 1: Coordinates of Tile 1 in State **S**: (x=0, y=1)
Coordinates of Tile 1 in State **G**: (x=0, y=0)
Manhattan distance = |0-0| + |1-0| = **1**

Manhattan Distance between (x_1, y_1) and $(x_2, y_2) = |x_1 - x_2| + |y_1 - y_2|$

Absolute value

Heuristic Search



Tile 1 *is 1 distance unit away from its target position*

Tile 2 *is 1 distance unit away from its target position*

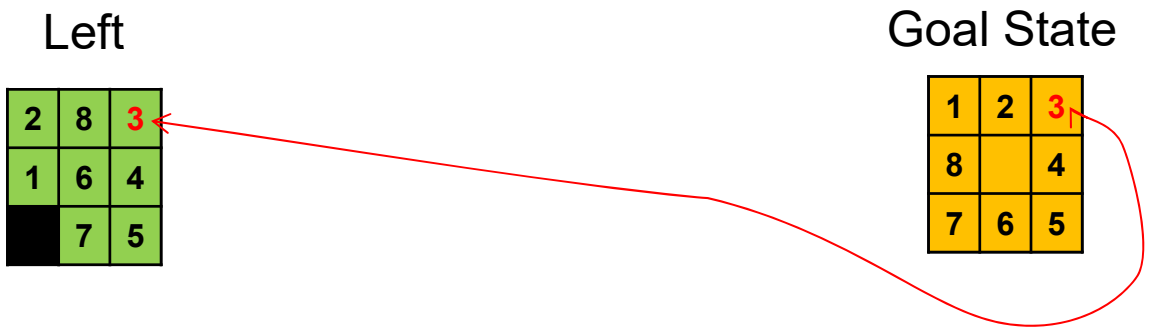
Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Heuristic Search



Tile 1 *is 1 distance unit away from its target position*

Tile 2 *is 1 distance unit away from its target position*

Tile 3 *is 0 distance unit away from its target position*

Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

menu

Heuristic Search

Left

2	8	3
1	6	4
	7	5

Goal State

1	2	3
8		4
7	6	5

6

Tile 1 *is 1 distance unit away from its target position*

Tile 2 *is 1 distance unit away from its target position*

Tile 3 *is 0 distance unit away from its target position*

Tile 4 *is 0 distance unit away from its target position*

Tile 5 *is 0 distance unit away from its target position*

Tile 6 *is 1 distance unit away from its target position*

Tile 7 *is 1 distance unit away from its target position*

Tile 8 *is 2 distance unit2 away from its target position*

Sum = 6

Sum of Manhattan Distance of tile to its goal location

Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2|$ and $|y_1 - y_2|$.

Each move can only decrease the distance of exactly one tile.

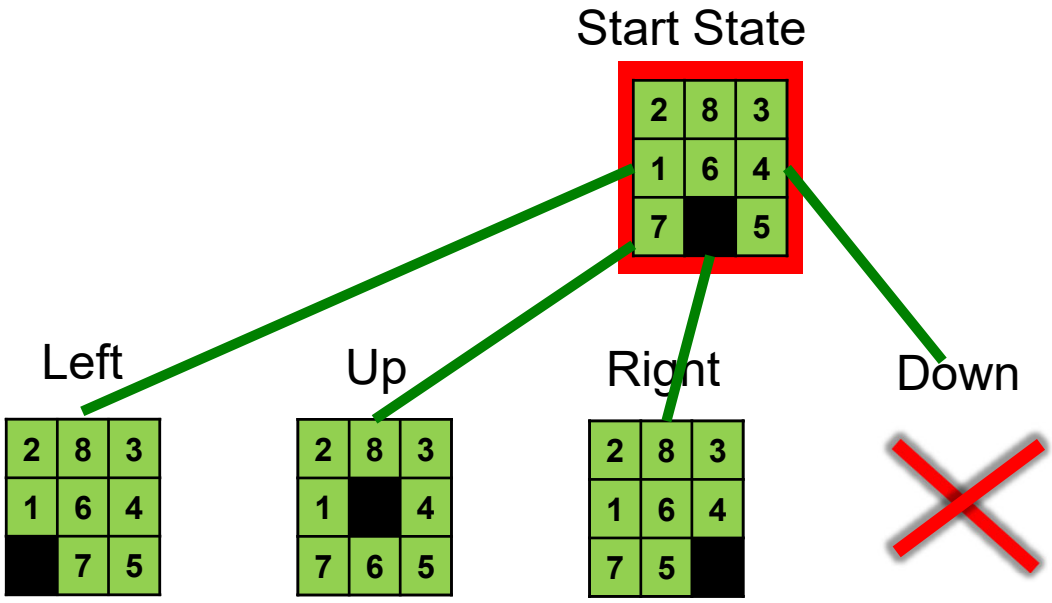
menu

Heuristic Search

Computing for the heuristic values of each of the children of the Start state:

Goal State

1	2	3
8		4
7	6	5



Heuristic value: 6 4 6 ——— **Sum of Manhattan Distance**

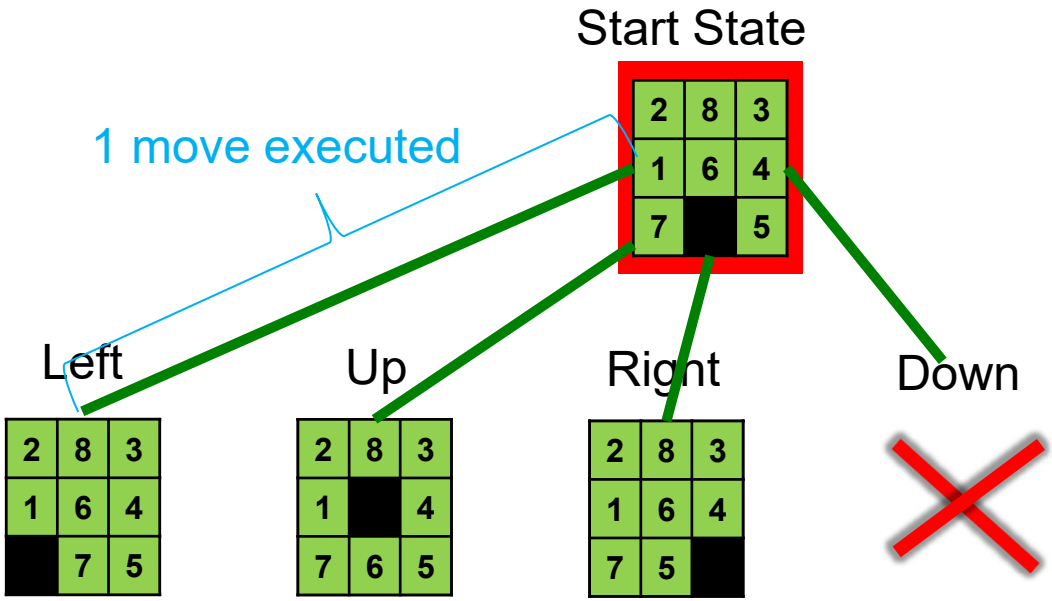
menu

Heuristic Search

Next, computing for the **F-cost** of each of the children of the Start state:

Goal State

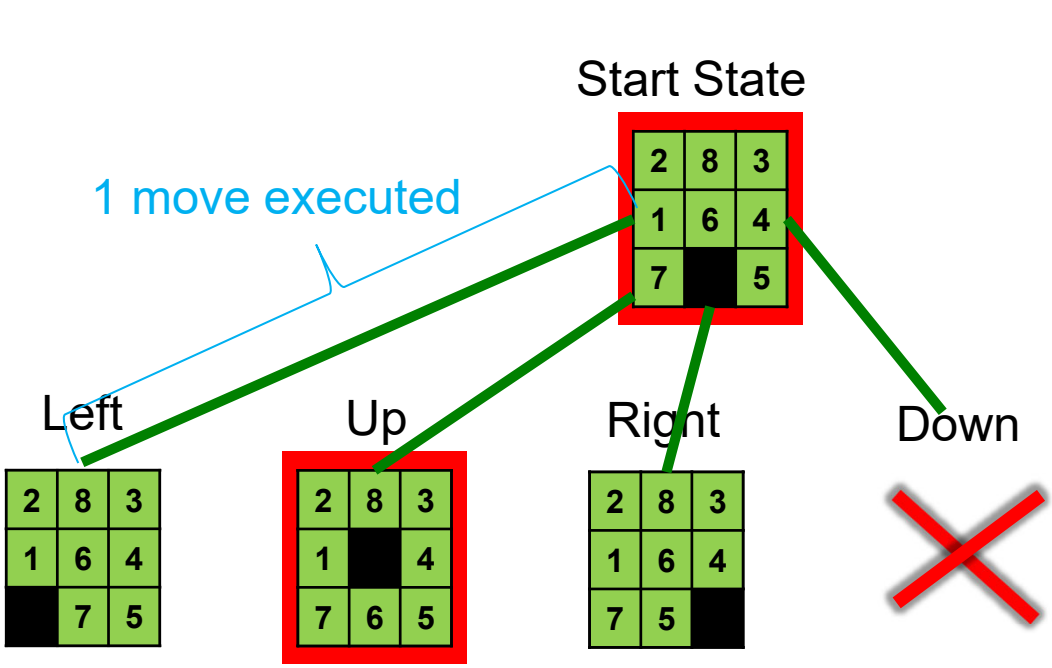
1	2	3
8		4
7	6	5



	Left	Up	Right	Down
Heuristic value:	6	4	6	Sum of Manhattan Distance
+	+	+	+	+
g-value:	1	1	1	Number of moves made
F-cost	**7**	**5**	**7**	Estimate of the total number of moves required to reach the goal state

menu

Heuristic Search



	Left	Up	Right	Down
	6	4	6	
+	1	1	1	
F-cost	**7**	**5**	**7**	

Sum of Manhattan Distance

\+

Number of moves made

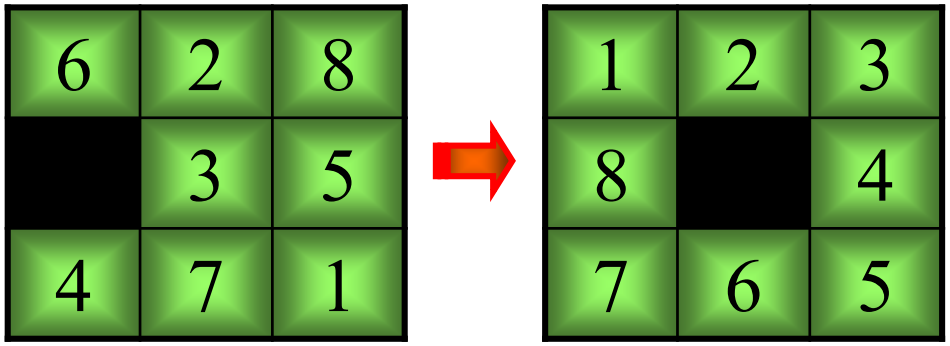
Estimate of the total number of moves required to reach the goal state (**f-cost**)

minimum number of moves!

menu

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as **moving an "empty" tile**.



S **G**

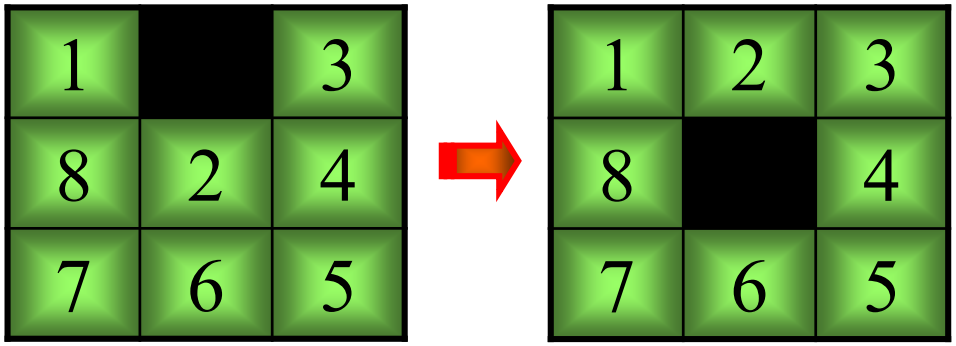
Alternative underestimates of "distance" (number of moves) to goal:

Number of misplaced tiles (7 in the example above).

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.

Number of **misplaced tiles**



S G

Are we supposed to include the blank tile when counting the number of misplaced tiles?

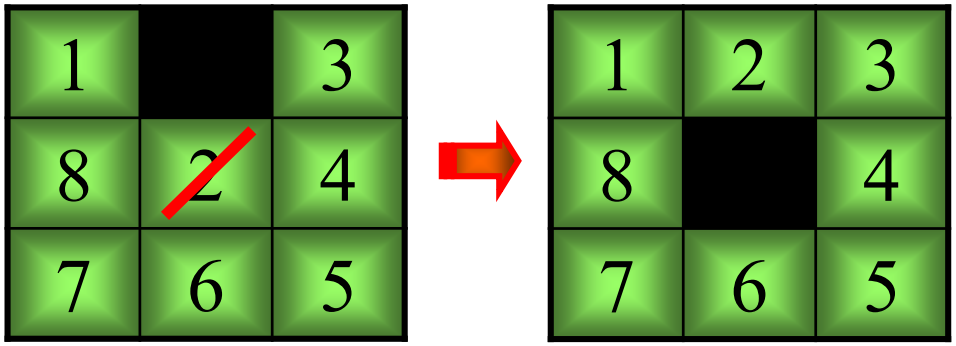
What will be the consequence if we include it? Examine the case above.

Number of misplaced tiles:

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.

Number of **misplaced tiles**



S **G**

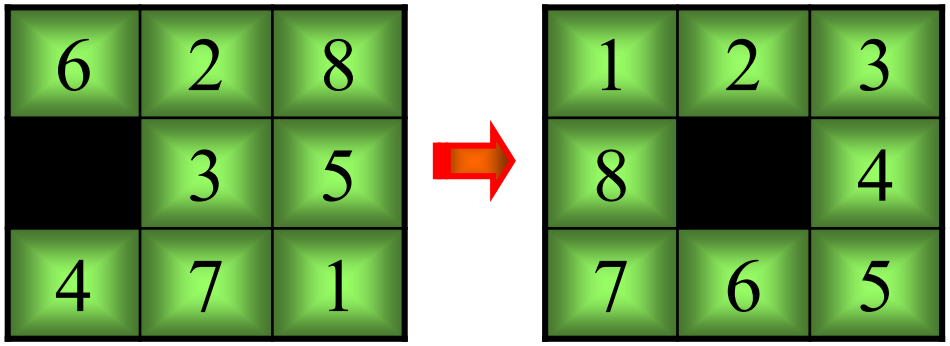
Are we supposed to include the blank tile when counting the number of misplaced tiles? **No.**

What will be the consequence if we include it? **the heuristic will not be admissible.**

Number of misplaced tiles: **1**

Admissible Heuristics

8 Puzzle: Move tiles to reach goal. Think of a move as moving an "empty" tile.



S G

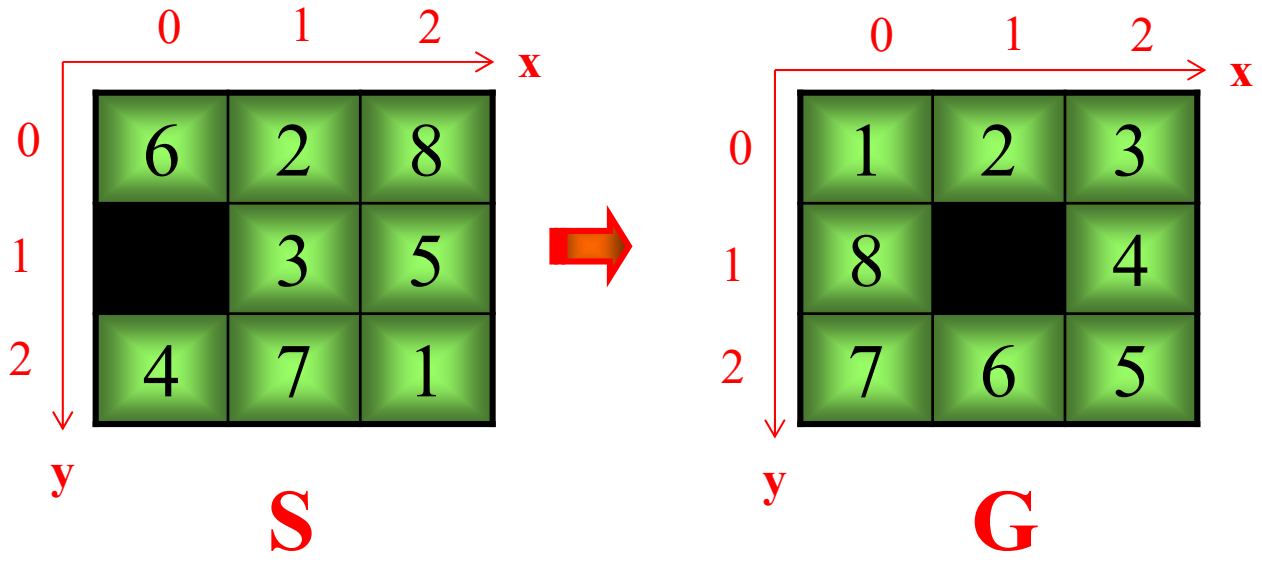
Alternative underestimates of "distance" (number of moves) to goal:

1. Number of **misplaced tiles** (7 in the example above).

2. Sum of **Manhattan Distance** of tile to its goal location (17 in example above).
   Manhattan Distance between (x_1, y_1) and (x_2, y_2) is $|x_1 - x_2| + |y_1 - y_2|$.
   Each move can only decrease the distance of exactly one tile.

The second of these is much better at predicting actual number of moves.

Admissible Heuristics



Note: Do not include the blank tile in the calculations.

Manhattan Distance between (x_1, y_1) and (x_2, y_2) = $|x_1 - x_2| + |y_1 - y_2|$

Absolute value

Number of Misplaced Tiles = **7**

Sum of Manhattan distance = **17**

5

STATE-SPACE SEARCH

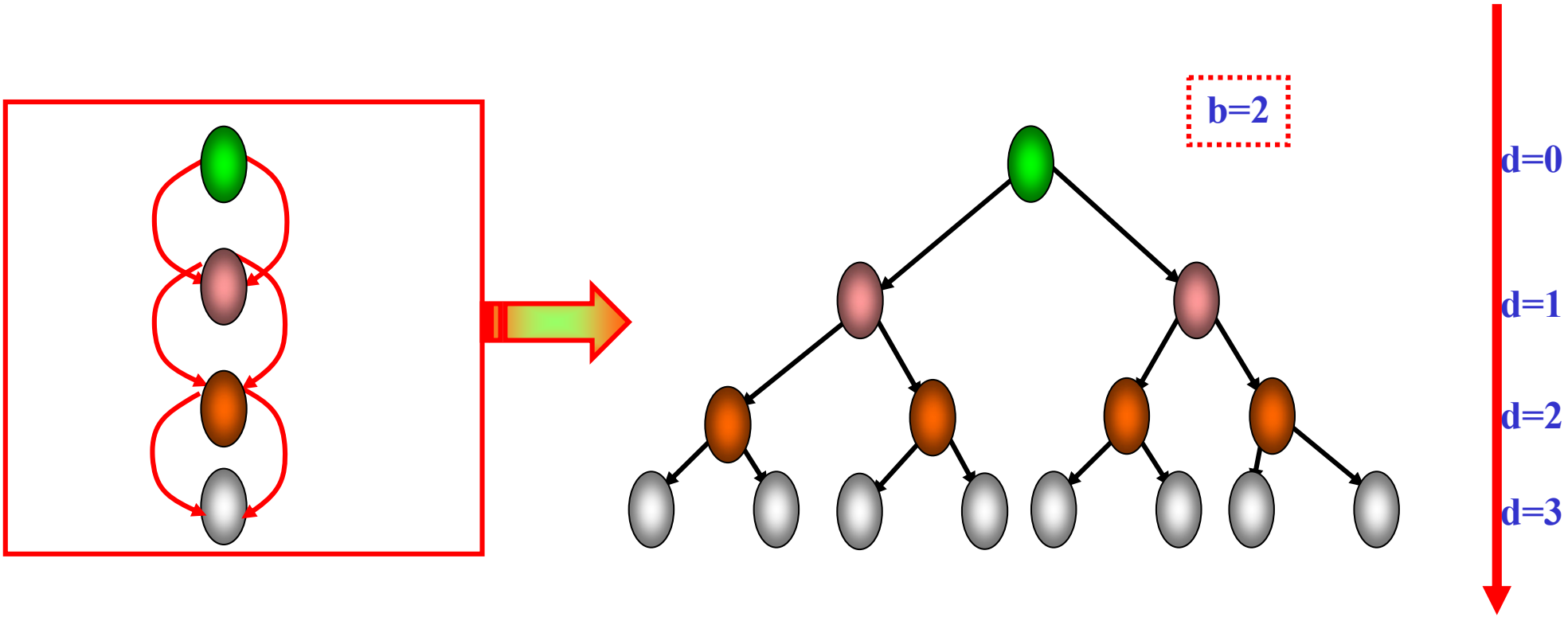
Optimal Search

A* and **Expanded List**

n.h.reyes@massey.ac.nz

Reference: MIT Open Courseware

States vs. Paths

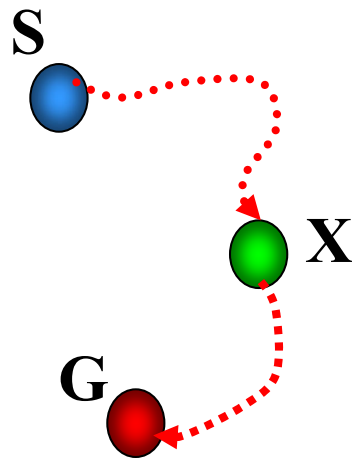


We have ignored the issue of revisiting states in our discussion of Uniform-Cost Search and A*. We indicated that we could not use the Visited List and still preserve optimality, but can we use something else that will keep the worst-case cost of a search proportional to the number of states in a graph rather than to the number of non-looping paths?

Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the **shortest path** from **S** to **X** and the **shortest path** from **X** to **G**. This is the "dynamic programming optimality principle".

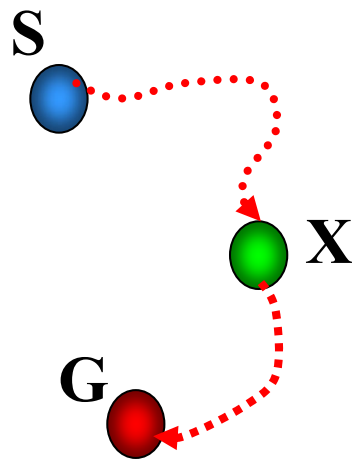


Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the **single best path** from **S** to any state **X**;
If we find a new path to a state already in **Q**, **discard the longer one**.



Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the single best path from **S** to any state **X**; If we find a new path to a state already in **Q**, discard the longer one.

Note that the first time **UC** pulls a search node off of **Q** whose state is **X**, this path is the shortest path from **S** to **X**. This follows from the fact that **UC** expands nodes in order of actual path length.

Uniform Cost Search

Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the single best path from **S** to any state **X**; If we find a new path to a state already in **Q**, discard the longer one.

Note that the first time **UC** pulls a search node off of **Q** whose state is **X**, this path is the shortest path from **S** to **X**. This follows from the fact that **UC** expands nodes in order of actual path length.

So, once we expand one path to state **X**, we don't need to consider (extend) any other paths to **X**. We can keep a list of these states, call it **Expanded List**. If the state of the search node we pull off of **Q** is in the Expanded List, we discard the node. When we use the Expanded List this way, we call it "**strict**"

Dynamic Programming Optimality Principle

and the EXPANDED LIST

Given that path length is additive, the shortest path from **S** to **G** via a state **X** is made up of the shortest path from **S** to **X** and the shortest path from **X** to **G**. This is the "dynamic programming optimality principle".

This means that we only need to keep the single best path from **S** to any state **X**; If we find a new path to a state already in **Q**, discard the longer one.

Note that the first time **UC** pulls a search node off of **Q** whose state is **X**, this path is the shortest path from **S** to **X**. This follows from the fact that **UC** expands nodes in order of actual path length.

So, once we expand one path to state **X**, we don't need to consider (extend) any other paths to **X**. We can keep a list of these states, call it **Expanded List**. If the state of the search node we pull off of **Q** is in the Expanded List, we discard the node. When we use the Expanded List this way, we call it "**strict**"

Note that **UC** without this is still correct, but inefficient for searching graphs.

SEARCH

Simple Optimal Search Algorithm: Uniform Cost

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry.
2. If Q is empty, fail. Else, pick search node N with **lowest path cost** from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. -
6. Find all the children of state(N) and create all the one-step extensions of N to each descendant.
7. Add all the extended paths to **front** of Q. (our convention in class)
8. Go to Step 2.

back

SEARCH

Simple Optimal Search Algorithm: Uniform Cost + Strict Expanded List

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Take note that we need to add some precautionary measures when adding extended paths to Q.

1. Initialise Q with search node (S) as only entry; **set Expanded = ( )**
2. If Q is empty, fail. Else, pick some search node N with **lowest path length** from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.

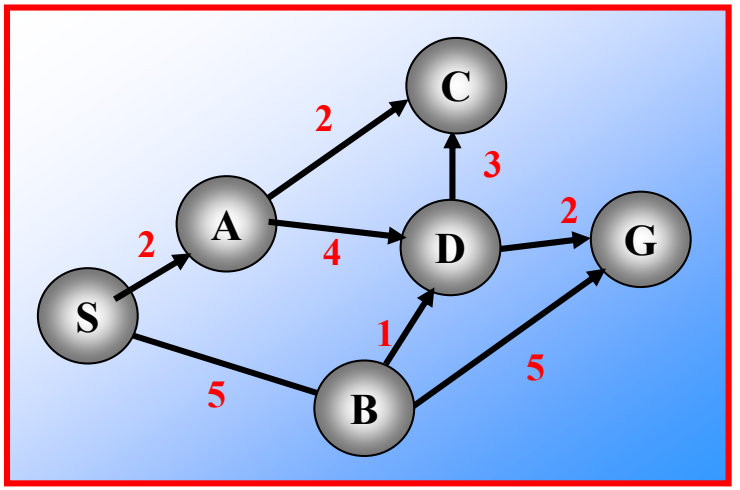
In effect, discard that search node N

5. **If state (N) in Expanded, go to Step 2; otherwise, add state (N) to Expanded List.**
6. Find all the children of state(N) **(Not in Expanded)** and create all the one-step extensions of N to each descendant.
7. **Add all the extended paths to front of Q. If descendant state already in Q, keep only shorter path to state in Q.**
8. Go to Step 2.

SEARCH

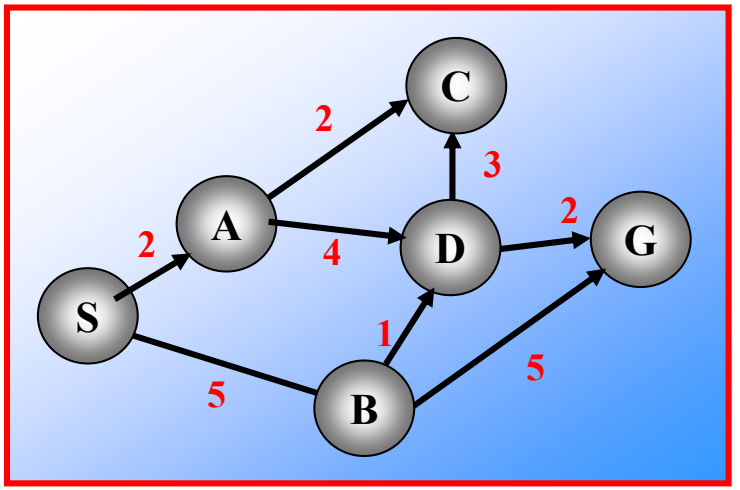
Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	(2 AS)(5 BS)	S



Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	(4 CAS)(6 DAS)(5 BS)	S, A

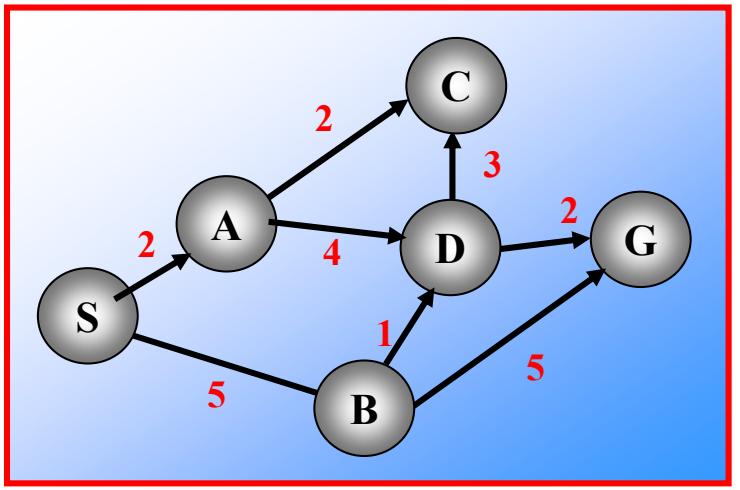


back

SEARCH

Uniform Cost (with Strict Expanded List)

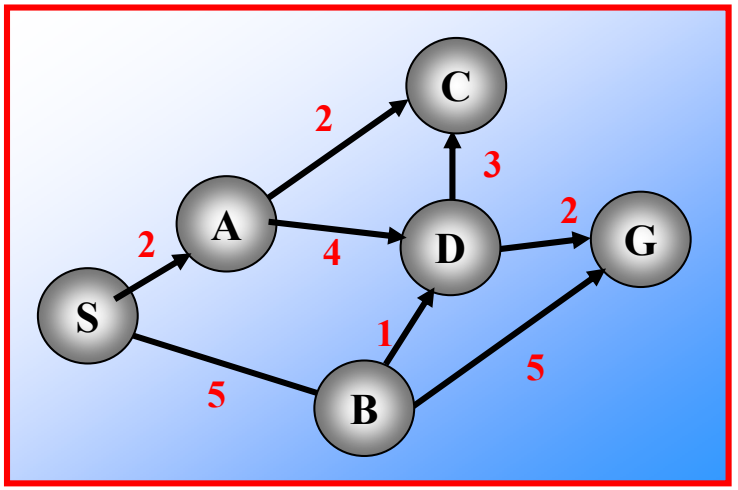
Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)(5 BS)	S, A, C



back

Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)~~(5 BS)~~	S, A, C
4	(5 BS)	~~(6 DBS)~~(10 GBS)~~(6 DAS)~~	S, A, C, B

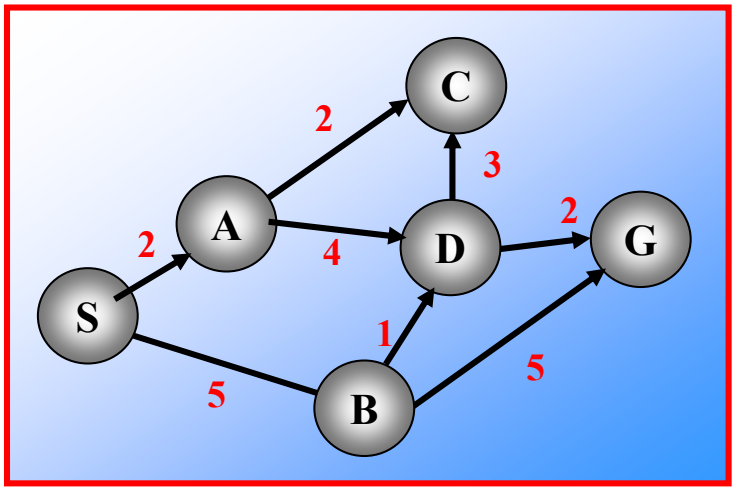


Removed because D is already in Q (our convention: take element at the front of the Q if a path leading to the same state is in Q).

SEARCH

Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)~~(5 BS)~~	S, A, C
4	(5 BS)	~~(6 DBS)~~(10 GBS)~~(6 DAS)~~	S, A, C, B
5	(6 DAS)	(8 GDAS)~~(10 GBS)~~	S, A, C, B, D

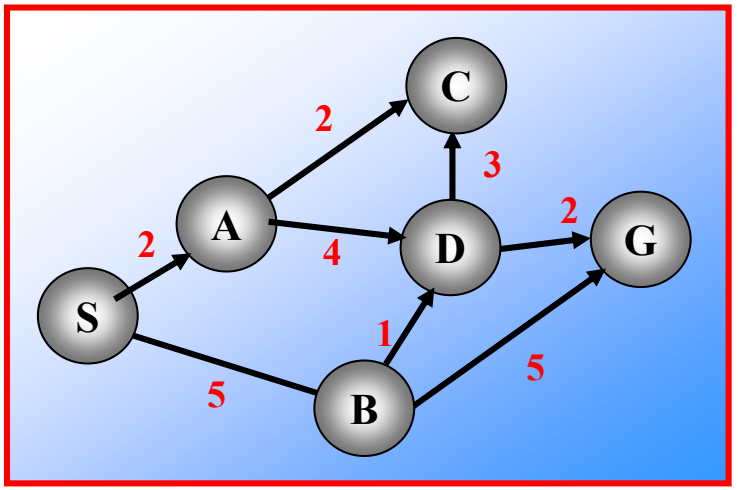


Removed because D is already in Q (our convention: take element at the front of the Q if a path leading to the same state is in Q).

Removed because there's a new shorter path to G

Uniform Cost (with Strict Expanded List)

Step		Q	Expanded
0		~~(0 S)~~	
1	(0 S)	~~(2 AS)~~(5 BS)	S
2	(2 AS)	~~(4 CAS)~~(6 DAS)(5 BS)	S, A
3	(4 CAS)	(6 DAS)~~(5 BS)~~	S, A, C
4	(5 BS)	~~(6 DBS)~~(10 GBS)~~(6 DAS)~~	S, A, C, B
5	(6 DAS)	~~(8 GDAS)~~~~(10 GBS)~~	S, A, C, B, D
6	(8 GDAS)		



Removed because D is already in Q (our convention: take element at the front of the Q if a path leading to the same state is in Q).

Removed because there's a new shorter path to G

Sequence of State Expansions: S – A – C – B – D - G

Sequence of state expansions = S–A–C–B–D-G

Path = S-A-D-G

Path length = 8

Max Q Length = 3

Number of state expansions = 6

UC with the Expanded List vs. UC without the Expanded List

	UC with the Expanded List	UC without the Expanded List
Sequence of state expansions	S–A–C–B–D-G	S– A – C – B – D – D – G
Path	S-A-D-G	S-A-D-G
Path Length	8	8
Max Q Length	3	5
Number of state expansions	6	7

back

A* (without Expanded List)

Let **g(N)** be the path cost of **N**, where **N** is a search tree node, i.e. a partial path.

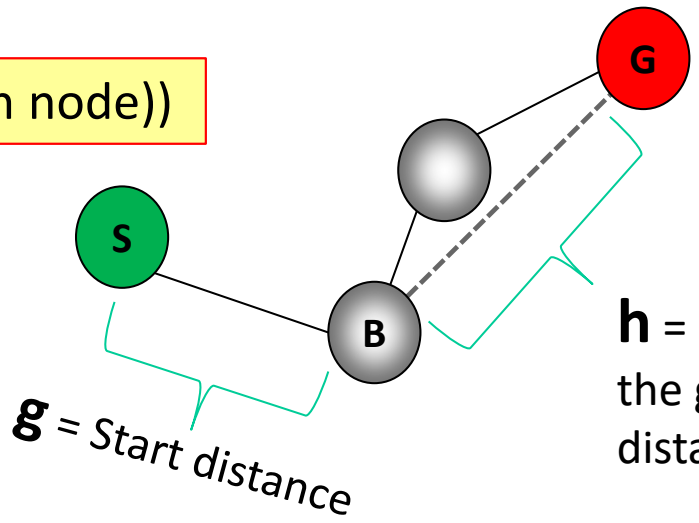
Let **h(N)** be h(state(N)), the heuristic estimate of the remaining path length to the goal from state(N).

Let **f(N) = g(N) + h(state(N))** be the total estimated path cost of a node, i.e. the estimate of a path to a goal that starts with the path given by N.

A* picks the node with **lowest f value** to expand.

f(search node)= g(search node) + h(state(search node))

e.g. f(BS) = g(S→B) + h(state(BS))



back

A* (without Expanded List)

Let g(N) be the path cost of N, where N is a search tree node, i.e. a partial path.

Let h(N) be h(state(N)), the heuristic estimate of the remaining path length to the goal from state(N).

Let f(N) = g(N) + h(state(N)) be the total estimated path cost of a node, i.e. the estimate of a path to a goal that starts with the path given by N.

A* picks the node with lowest f value to expand.

A* (**without Expanded List**) and with **admissible heuristic** is **guaranteed** to find optimal paths – those with smallest path cost.

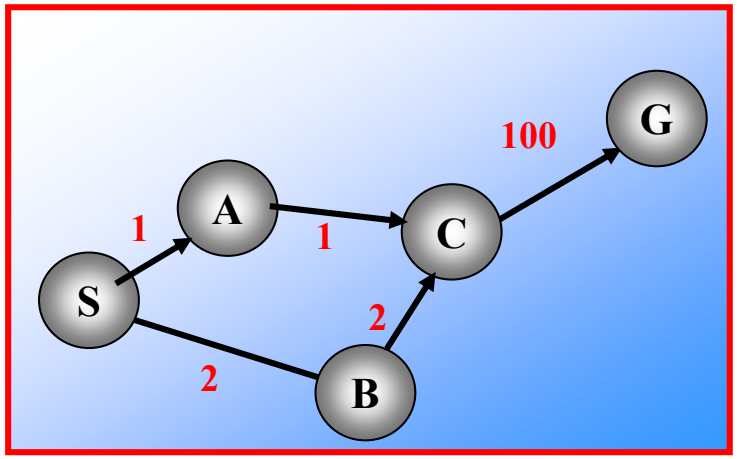
A* and the Strict Expanded List

The **Strict Expanded List** (also known as a **Closed List**) is commonly used in implementations of A* but, to guarantee finding optimal paths, this implementation requires a ***stronger condition*** for a heuristic than simply being an underestimate.

A* and the Strict Expanded List

The Strict Expanded List (also known as a Closed List) is commonly used in implementations of A* but, to guarantee finding optimal paths, this implementation requires a ***stronger condition*** for a heuristic than simply being an underestimate.

Here's a counterexample: The heuristic values listed below are all underestimates but A* using an Expanded List will not find the optimal path.



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

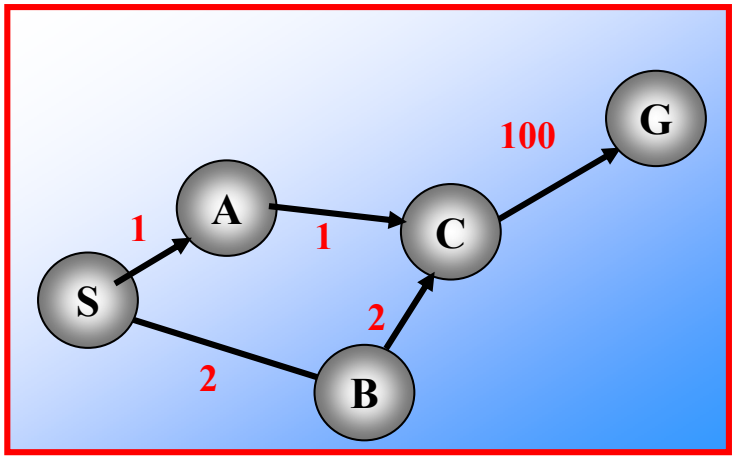
Step	Dequeued	Q	Expanded List
0		(90 S)	

f= g + h

f(S) = g(S) + h(state(S))

f(S) = 0 + h(S)

= 0 + 90 = 90



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions to **front** of Q.

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

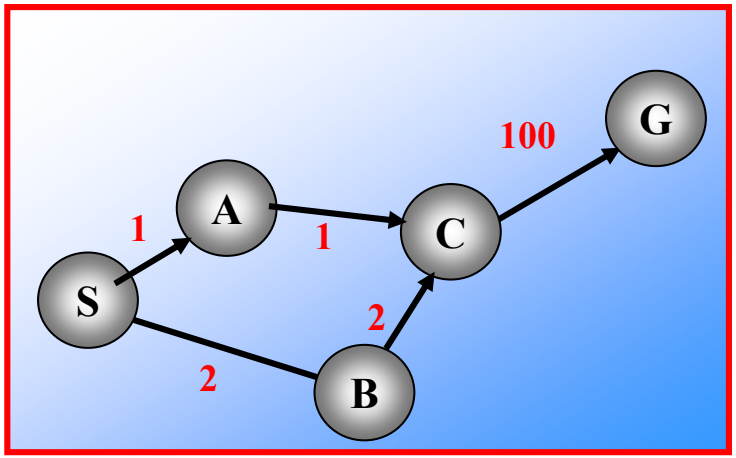
Step	Dequeued	Q	Expanded List
0		(90 S)	
1	(90 S)	(101 AS)(3 BS)	S

f= g + h

$$f(AS) = g(S \rightarrow A) + h(state(AS))$$

$$f(AS) = 1 + h(A)$$

$$= 1 + 100 = 101$$



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions to **front** of Q.

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

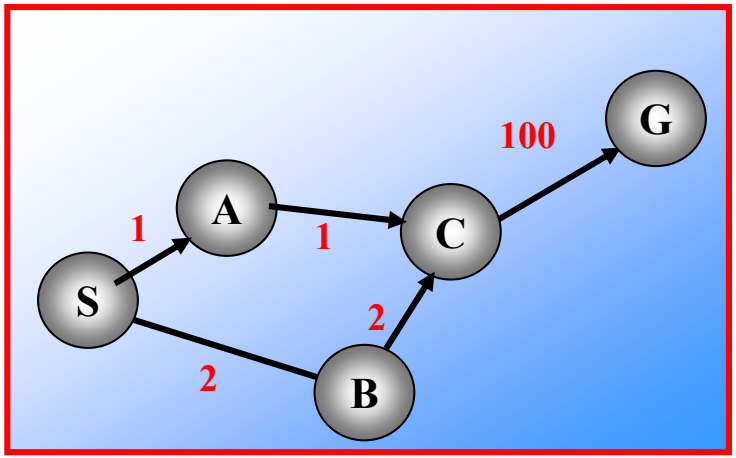
Step	Dequeued	Q	Expanded List
0		(90 S)	
1	(90 S)	(101 AS)(3 BS)	S

f= g + h

$$f(BS) = g(S \rightarrow B) + h(state(BS))$$

$$f(BS) = 2 + h(B)$$

$$= 2 + 1 = 3$$



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

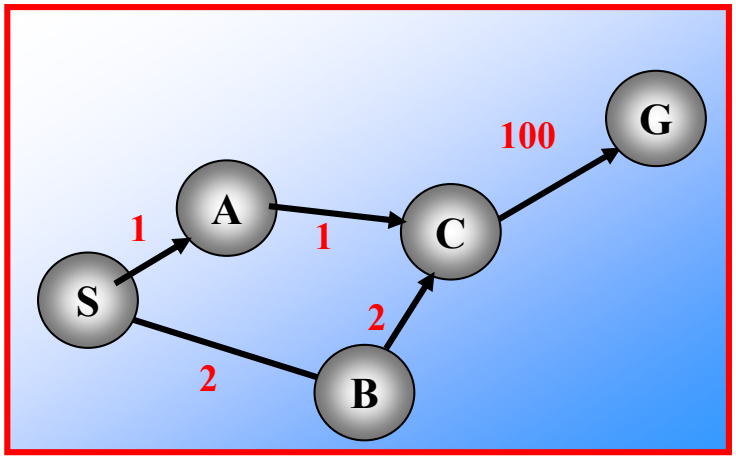
- Pick **best** (**by path length + heuristic**) element of Q, Add path extensions to **front** of Q.

A* and the Strict Expanded List

Here's a counterexample: The heuristic values listed below are all **underestimates** but A* using an Expanded List will not find the optimal path.

The misleading estimate at **B** **throws the algorithm off**; C is expanded before the optimal path to it is found.

Step	Dequeued	Q	Expanded List
0		(90 S)	
1	(90 S)	(101 AS)(3 BS)	S
2	(3 BS)	(94 CBS) (101 AS)	B, S
3	(94 CBS)	(104 GCBS)(101 AS)	C, B, S
4	(101 AS)	(104 GCBS)	A, C, B, S
5	(104 GCBS)		G, A, C, B, S



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

Underlined paths are chosen for extension

back

Consistency

To enable implementing A* using the strict Expanded List, **H** needs to satisfy the following **consistency** (also known as **monotonicity**) conditions

1. $h(S_i) = 0$, if n_i is a goal
2. $h(S_i) - h(S_j) <= c(S_i, S_j)$, if n_j a child of n_i

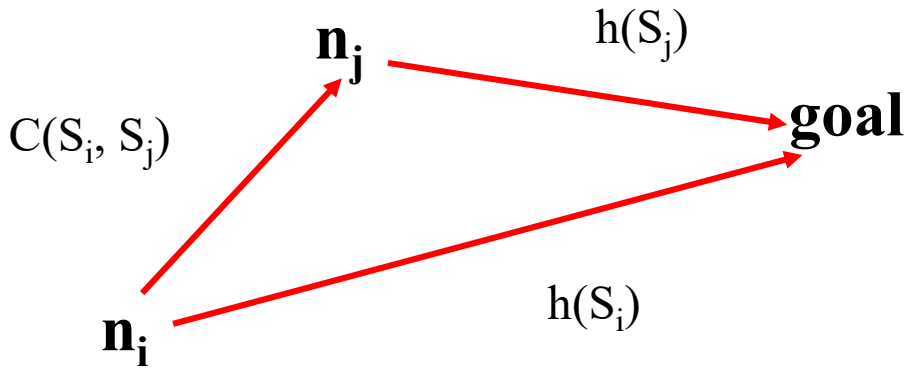
Consistency

To enable implementing A* using the strict Expanded List, H needs to satisfy the following **consistency** (also known as **monotonicity**) conditions

1. $h(S_i) = 0$, if n_i is a goal

2. $h(S_i) - h(S_j) <= c(S_i, S_j)$, if n_j a child of n_i

That is, the heuristic cost in moving from one entry to the next cannot decrease by more than the arc cost between the states. This is a kind of ***triangle inequality*** This condition is a highly desirable property of a heuristic function and often simply assumed (more on this later).



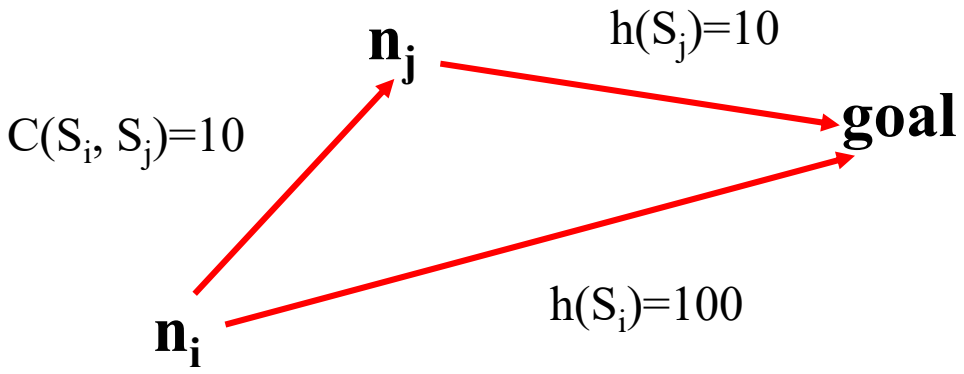
Consistency Violation!

A simple example of violation of consistency:

1. $h(S_i) = 0$, if n_i is a goal
2. $h(S_i) - h(S_j) <= c(S_i, S_j)$, if n_j a child of n_i

In the example, 100-10 > 10

If you believe that the **goal** is **100** units from n_i, then moving 10 units to n_j should not bring you to a distance of **10** units from the goal.



?

100-10 ≤ 10

This evaluates to false. Violates consistency requirement!

A* (without Expanded List)

Let **g(N)** be the path cost of n, where n is a search tree node, i.e. a partial path.

Let **h(N)** be **h(state(N))**, the heuristic estimate of the remaining path length to the goal from state(N).

Let **f(N) = g(N) + h(state(N))** be the total estimated path cost of a node, i.e. the estimate of a path to a goal that starts with the path given by N.

A* picks the node with **lowest f value** to expand.

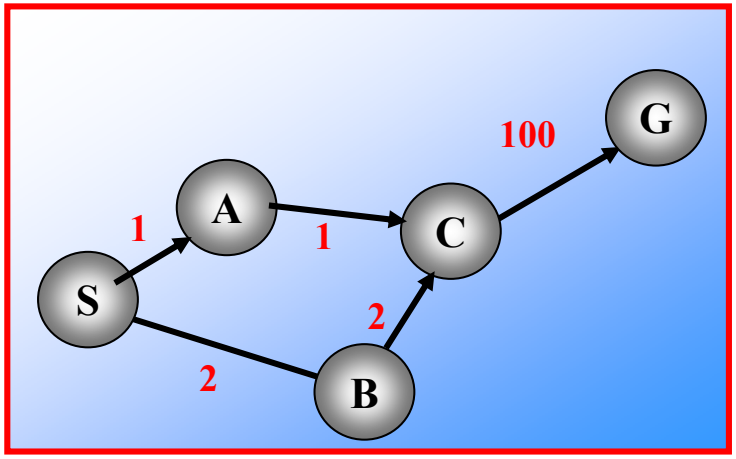
A* (without Expanded List) and **with admissible heuristic** is **guaranteed** to find optimal paths – those with smallest path cost.

This is true even if heuristic is **NOT** consistent.

A* (without the Strict Expanded List)

Step		Q
0		(90 S)
1	(90 S)	(3 BS)(101 AS)
2	(3 BS)	(94 CBS) (101 AS)
3	(94 CBS)	(101 AS)(104 GCBS)
4	(101 AS)	(92 CAS)(104 GCBS)
5	(92 CAS)	(102 GCAS)(104 GCBS)
6	(102 GCAS)	

Note that the heuristic is admissible but **NOT** consistent.



Heuristic Values:
A=100, B=1, C=90, S=90, G=0

inconsistent!

Underlined paths are chosen for extension.

This heuristic is not consistent but it is an **underestimate** and that is all that is needed for **A* without an Expanded List** to guarantee optimality.

A* (with Strict Expanded List)

Just like Uniform Cost Search.

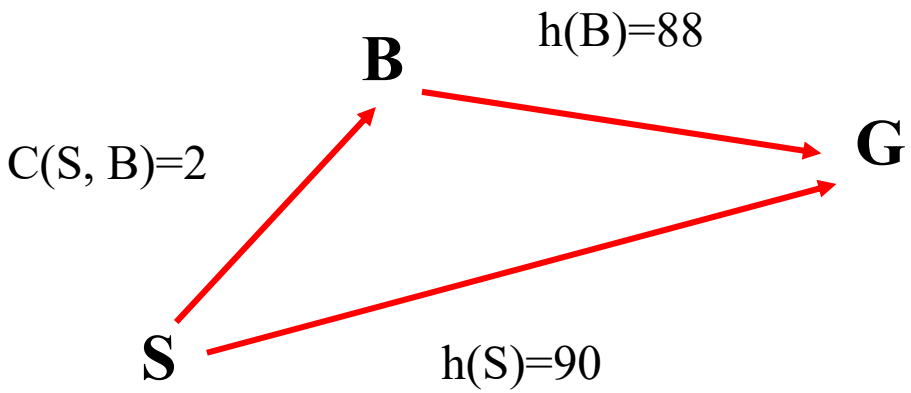
When a node **N** is expanded, if state **(N)** is in Expanded List, discard **N**, else add state **(N)** to Expanded List.

If some node in **Q** has the same state as some descendant of **N**, keep only node with smaller **f**, which will also corresponds to smaller **g**.

For **A* (with Strict Expanded List)** to be guaranteed to find the optimal path, the heuristic must be **consistent**.

A* (with Strict Expanded List)

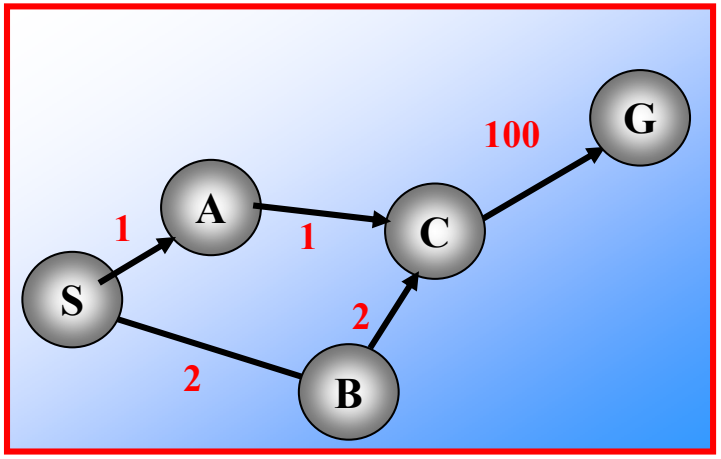
Note that the heuristic is **admissible and consistent**.



?

90-88 ≤ 2 True!

PSEUDOCODE



Heuristic Values:

A=100, B=**88**, C=100, S=90, G=0

For our convention in class, add the path extensions to the **front** of the Queue.

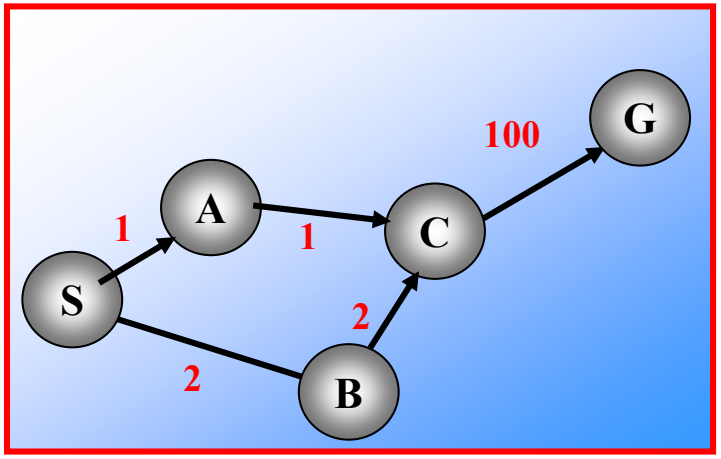
If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)(90 BS)	S
2			
3			
4			
5			

PSEUDOCODE



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

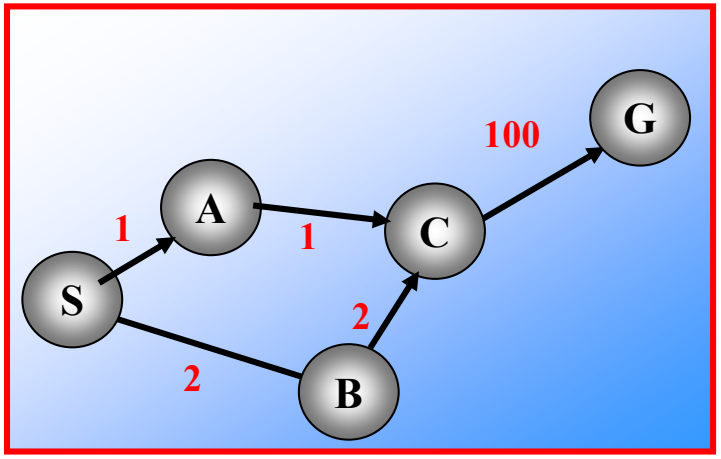
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)(101 AS)	S, B
3			
4			
5			



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

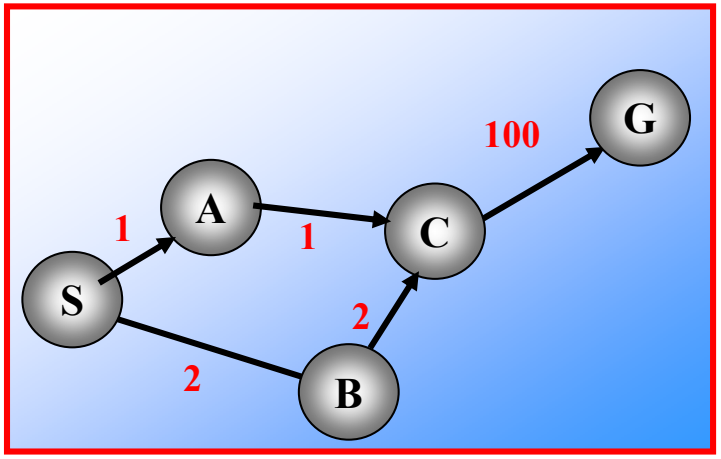
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)~~(101 AS)~~	S, B
3	(101 AS)	(102 CAS)~~(104 CBS)~~	S, B,A
4			
5			



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

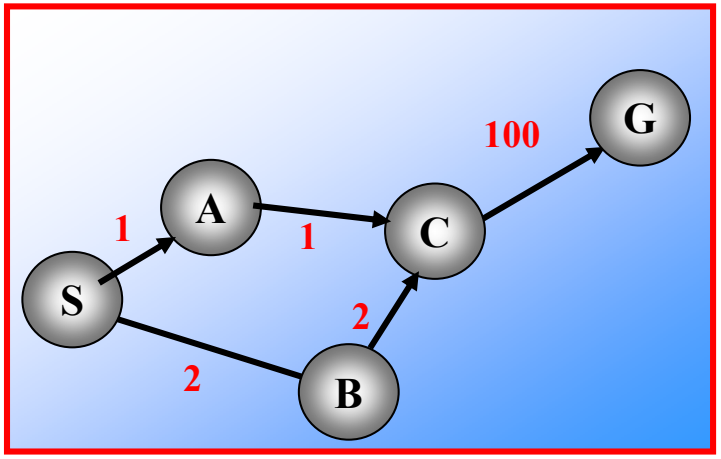
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)~~(101 AS)~~	S, B
3	(101 AS)	~~(102 CAS)~~(~~104 CBS~~)	S, B,A
4	(102 CAS)	(102 GCAS)	S, B, A, C
5			



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

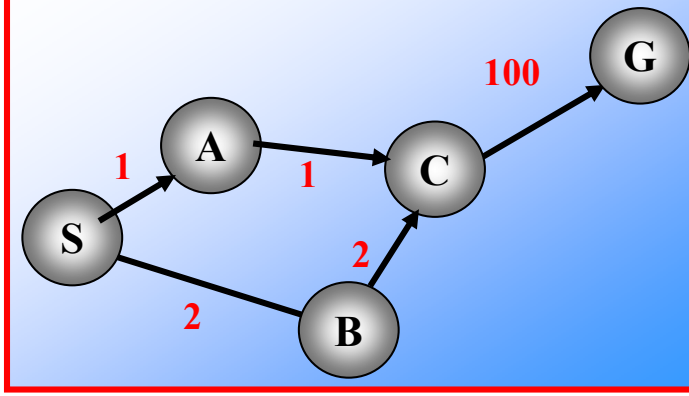
For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

A* (with Strict Expanded List)

Note that the heuristic is **admissible and consistent**.

Step	Dequeued	Enqueued	Expanded List
0		~~(90 S)~~	
1	(90 S)	(101 AS)~~(90 BS)~~	S
2	(90 BS)	(104 CBS)~~(101 AS)~~	S, B
3	(101 AS)	~~(102 CAS)~~~~(104 CBS)~~	S, B,A
4	(102 CAS)	~~(102 GCAS)~~	S, B, A, C
5	(102 GCAS)		S, B, A, C, G



Heuristic Values:
A=100, B=**88**, C=100, S=90, G=0

For our convention in class, add the path extensions to the **front** of the Queue.

If we modify the heuristic in the example we have been considering so that it is **consistent**, as we have done here **by increasing the value of h(B)**, then A* (with the Expanded List) will work.

Dealing with inconsistent heuristic

What can we do if we have an inconsistent heuristic but we still want optimal paths?

Dealing with inconsistent heuristic

What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it **detects** and **corrects** when inconsistency has led us astray.

Dealing with inconsistent heuristic

What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it detects and corrects when inconsistency has led us astray.

Assume we are adding $node_1$ to Q and $node_2$ is present in Expanded List with **$node_1$.state = $node_2$.state**.

Example:

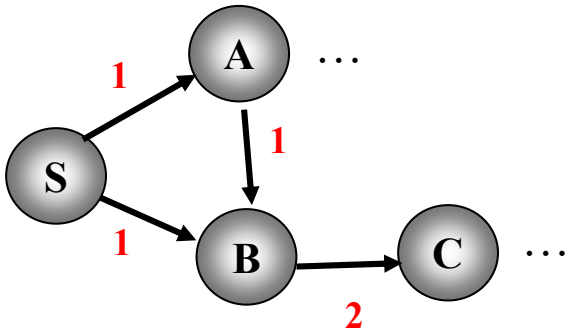
We have a node we are considering adding into Q:

(6 CBS) ???

Q
(10 FEDS) …

Search nodes are added in the expanded list

Strict Expanded List
(8 CDAS)



Dealing with inconsistent heuristic

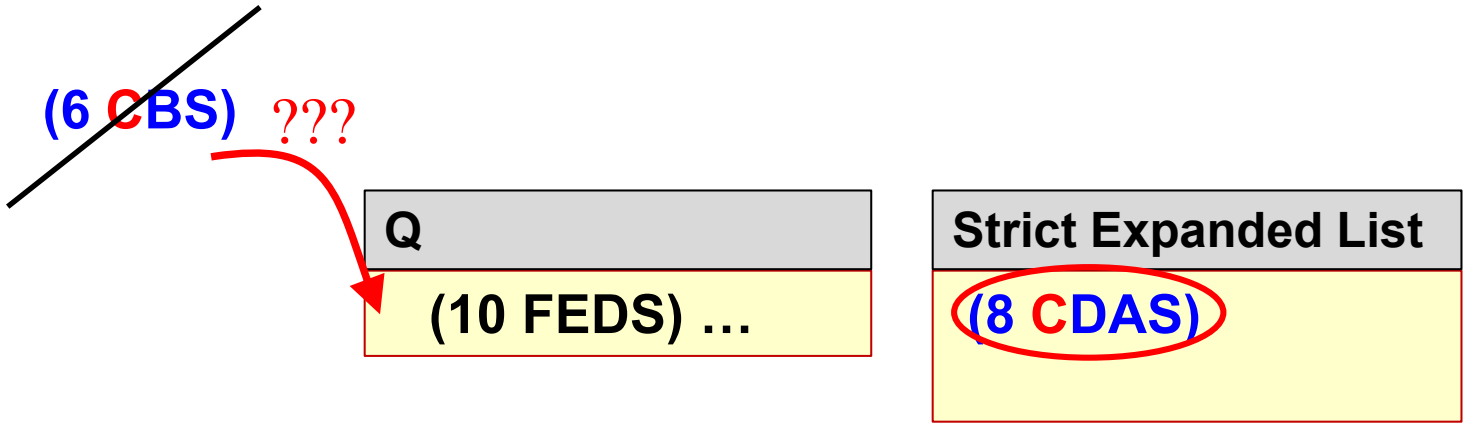
What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it detects and corrects when inconsistency has led us astray.

Assume we are adding $node_1$ to Q and $node_2$ is present in Expanded List with **$node_1$.state = $node_2$.state**.

Strict Expanded List:

- Do NOT add **$node_1$** to Q.



Dealing with inconsistent heuristic

What can we do if we have an inconsistent heuristic but we still want optimal paths?

Modify A* so that it detects and corrects when inconsistency has led us astray.

Assume we are adding $node_1$ to Q and $node_2$ is present in Expanded List with **$node_1$.state = $node_2$.state**.

Strict Expanded List:

- Do NOT add $node_1$ to Q.

Non-Strict Expanded List:

- If (**$node_1$.path_length** < **$node_2$.path_length**), then

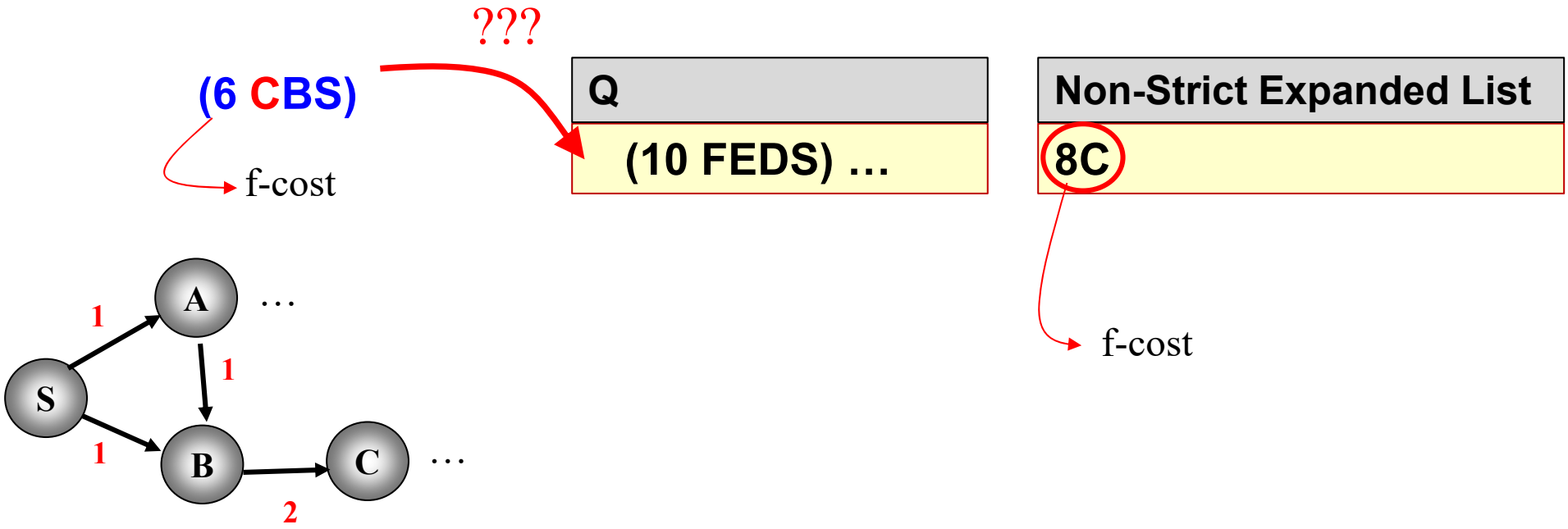
  1. Delete $node_2$ from Expanded List
  2. Add $node_1$ to Q.

Non-Strict Expanded List

Store the **f-cost** as part of each state recorded

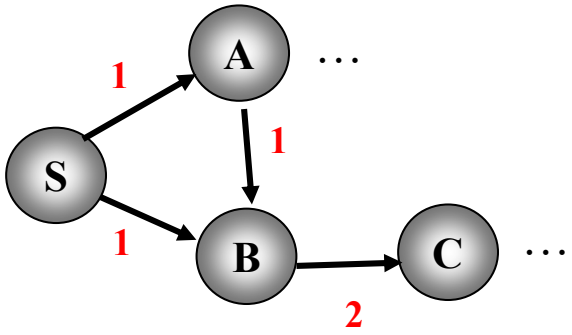
Dealing with inconsistent heuristic

Example: We have a node we are considering adding into Q:



Dealing with inconsistent heuristic

Example: We have a node we are considering adding into Q:



???

(6 CBS)

Q	Non-Strict Expanded List
(10 FEDS) …	8C

6 < 8, and so we should add **(6 CBS)** into the Q, then delete 8C from the Non-Strict Expanded list

Worst Case Complexity

The number of states in the search space may be exponential in some "depth" parameter, e.g. number of actions in a plan, number of moves in a game.

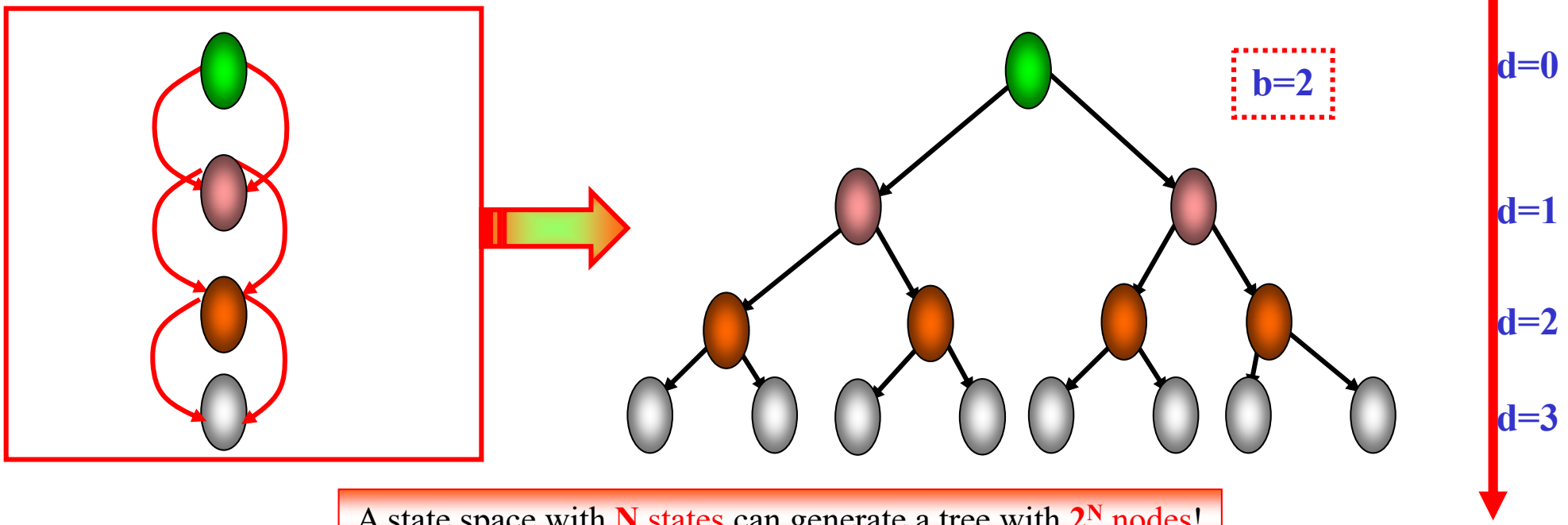
All the searches, with or without Visited or Expanded Lists, **may have to visit** (or expand) **each state** in the worst case.

So, all searches will have **worst case complexities** that are at least **proportional to the number of states** and therefore exponential in the "depth" parameter.

This is the bottom-line irreducible worst case cost of systematic searches.

Worst Case Complexity

- A state space with N states may give rise to a search tree that has a number of nodes that is exponential in N, as in this example.



A state space with N states can generate a tree with 2^N nodes!

- Searches **without** a Visited (Expanded) List may, in the worst case, visit (expand) every node in the search tree.

- Searches with **Strict Visited** (Expanded Lists) will **visit** (expand) **each state only** once

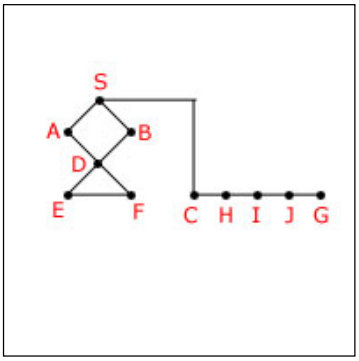
Optimality and Worst Case Complexity

Algorithm	Heuristic	Expanded List	Optimality Guaranteed?	Worst Case & Expansions
UC	None	Strict	Yes	N
A*	Admissible	None	Yes	>N
A*	Consistent	Strict	Yes	N
A*	Admissible	Strict	No	N
A*	Admissible	Non-Strict	Yes	>N

Tutorial 1 – Any Path Search

We are trying to find a path from **S** to **G** in the following graph:

```
Connections (all undirected):
 S: A B C            F: D E
 A: S D              G: J
 B: S D              H: C I
 C: S H              I: H J
 D: A B E F          J: I G
 E: D F
```



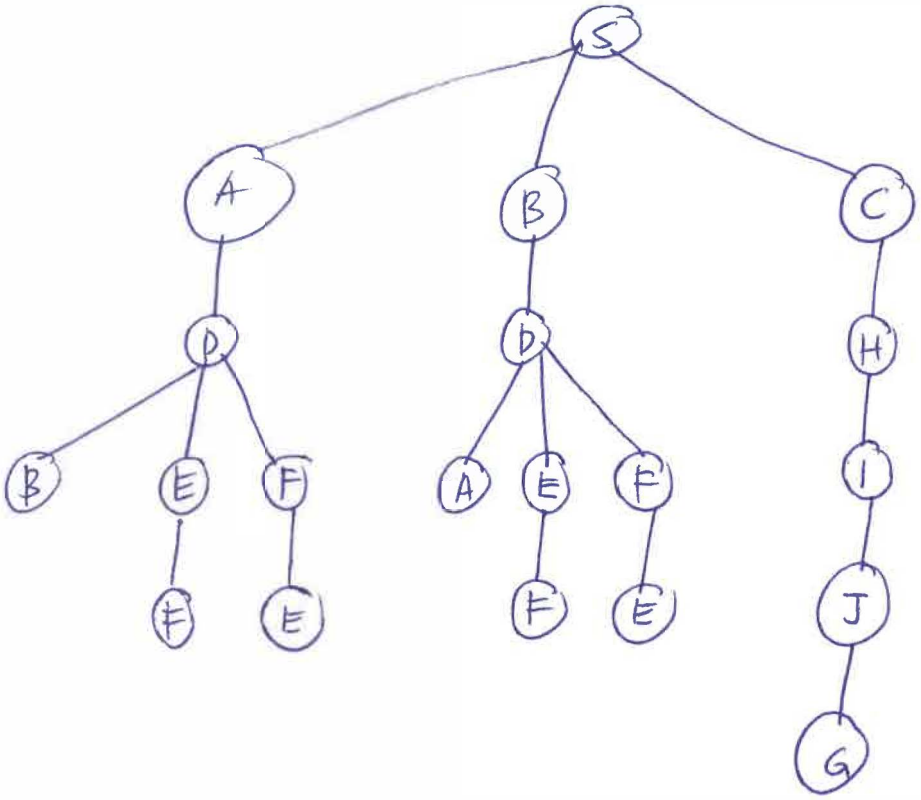
The heuristic distance estimates to the goal are:
`S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0`

Examine states in alphabetical order when no other order is specified. In particular, assume that the list of descendant states of a parent state is placed on Q so that they are examined in alphabetical order. So, this determines the order that both depth first and breadth first explore the space. Also, if you have ties in heuristic searching, use alphabetical order to break ties.

Write the sequence of **states expanded** by the specified search methods in our implementation. In each answer slot, enter a state name, including S and G. States may be repeated in the sequence. *You will save yourself a LOT of confusion if you first convert the graph above into the appropriate search tree.*

1. **Depth-First (using Visited List)**
2. **Depth-First (NOT using Visited List)**
3. **Breadth-First (using Visited List)**
4. **Breadth-First (NOT using Visited List)**
5. **Best-First (using Visited List)**

Search Tree



Heuristics

S = 10

A = 5

B = 8

C = 6

D = 7

E = 6

F = 5

H = 3

I = 2

J = 1

G = 0

1) DFS with Visited

Step	Deq	Enq	Visited list
0		(S)	S
1	(S)	~~(AS)~~ (BS) (CS)	S, A, B, C
2	(AS)	~~(DAS)~~ (BS) (CS)	", D
3	(DAS)	~~(EDAS)~~ (FDAS) (BS) (CS)	", E, F
4	(EDAS)	~~(FDAS)~~ (BS) (CS)	"
5	(FDAS)	~~(BS)~~ (CS)	"
6	(BS)	~~(CS)~~	"
7	(CS)	~~(HCS)~~	", H
8	(HCS)	~~(IHCS)~~	", I
9	(IHCS)	~~(JIHCS)~~	", J
10	(JIHCS)	~~(GJIHCS)~~	", G
11	(GJIHCS)		

Seq. of State Exp: S-A-D-E-F-B-C-H-I-J-G

path: S-C-H-I-J-G

max Q = 4

state expansions = 11

2) DFS without Visited List

Step	Dequened	Enqueued
0		(S)
1	(S)	(AS)(BS)(CS)
2	(AS)	(DAS)(BS)(CS)
3	(DAS)	(BDAS)(EDAS)(FDAS)(BS)(CS)
4	(BDAS)	(EDAS)(FDAS)(BS)(CS)
5	(EDAS)	(FEDAS)(FDAS)(BS)(CS)
6	(FEDAS)	(FDAS)(BS)(CS)
7	(FDAS)	(EFDAS)(BS)(CS)
8	(EFDAS)	(BS)(CS)
9	(BS)	(DBS)(CS)
10	(DBS)	(ADBS)(EDBS)(FDBS)(CS)
11	(ADBS)	(EDBS)(FDBS)(CS)
12	(EDBS)	(FEDBS)(FDBS)(CS)
13	(FEDBS)	(FDBS)(CS)
14	(FDBS)	(EFDBS)(CS)
15	(EFDBS)	(CS)
16	(CS)	(HCS)
17	(HCS)	(IHCS)
18	(IHCS)	(JIHCS)
19	(JIHCS)	(GJIHCS)
20	(GJIHCS)	

Fin

Sequence of State expansion : S-A-D-B-E-F-F-E-B-D-A-E-F-F-E-C-H-I-J-G

3) BFS w/ Visited List

Step	Deq	Enq	Visited List
0		~~(S)~~	S
1	(CS)	~~(AS)~~(BS)(CS)	S, A, B, C
2	(AS)	~~(BS)~~(CS)(DAS)	", D
3	(BS)	~~(CS)~~(DAS)	"
4	(CS)	~~(DAS)~~(HCS)	", H
5	(DAS)	~~(HCS)~~(EDAS)(FDAS)	", E, F
6	(HCS)	~~(EDAS)~~(FDAS)(IHCS)	", I
7	(EDAS)	~~(FDAS)~~(IHCS)	"
8	(FDAS)	~~(IHCS)~~	"
9	(IHCS)	~~(JIHCS)~~	", J
10	(JIHCS)	~~(GJIHCS)~~	", G
11	(GJIHCS)		

Sequence of State Exp: S - A - B - C - D - H - E - F - I - J - G

Num of State Expansions = 11

Max Q length = 3

path: S - C - H - I - J - G

path length: 5

4) BFS without Visited list

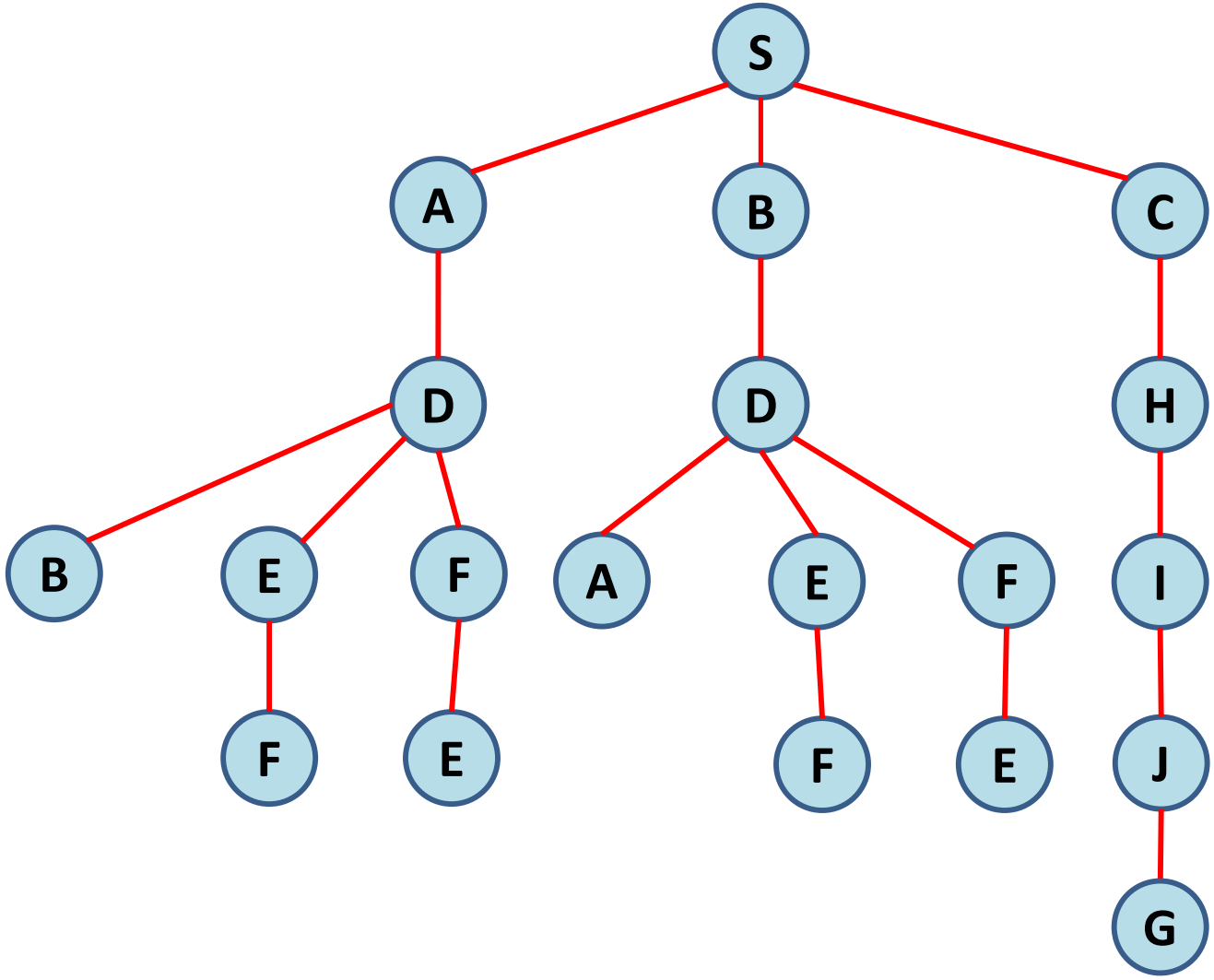
Step	Deq	Enq
0		(S)
1	(S)	~~(AS)~~ (BS) (CS)
2	(AS)	~~(BS)~~ (CS) (DAS)
3	(BS)	~~(CS)~~ (DAS) (DBS)
4	(CS)	~~(DAS)~~ (DBS) (HCS)
5	(DAS)	~~(DBS)~~ (HCS) (BDAS) (EDAS) (FDAS)
6	(DBS)	~~(HCS)~~ (BDAS) (EDAS) (FDAS) (ADBS) (EDBS) (FDBS)
7	(HCS)	~~(BDAS)~~ (EDAS) (FDAS) (ADBS) (EDBS) (FDBS) (IHCS)
8	(BDAS)	~~(EDAS)~~ (FDAS) (ADBS) (EDBS) (FDBS) (IHCS)
9	(EDAS)	~~(FDAS)~~ (ADBS) (EDBS) (FDBS) (IHCS) (FEDAS)
10	(FDAS)	~~(ADBS)~~ (EDBS) (FDBS) (IHCS) (FEDAS) (EFDAS)
11	(ADBS)	~~(EDBS)~~ (FDBS) (IHCS) (FEDAS) (EFDAS)
12	(EDBS)	~~(FDBS)~~ (IHCS) (FEDAS) (EFDAS) (FEDBS)
13	(FDBS)	~~(IHCS)~~ (FEDAS) (EFDAS) (FEDBS) (EFDBS)
14	(IHCS)	~~(FEDAS)~~ (EFDAS) (FEDBS) (EFDBS) (JIHCS)
15	(FEDAS)	~~(EFDAS)~~ (FEDBS) (EFDBS) (JIHCS)
16	(EFDAS)	~~(FEDBS)~~ (EFDBS) (JIHCS)
17	(FEDBS)	~~(EFDBS)~~ (JIHCS)
18	(EFDBS)	~~(JIHCS)~~
19	(JIHCS)	~~(GJIHCS)~~
20	(GJIHCS)	

5) Best-First Search with Visited List

Step	Deq	Enq	Visited List
0		~~(10S)~~	S
1	(10S)	~~(5AS)~~ (8BS) (6CS)	S, A, B, C
2	(5AS)	(7DAS) (8BS) ~~(6CS)~~	", D
3	(6CS)	~~(3HCS)~~ (7DAS) (8BS)	", H
4	(3HCS)	~~(2IHCS)~~ (7DAS) (8BS)	", I
5	(2IHCS)	~~(1JIHCS)~~ (7DAS) (8BS)	", J
6	(1JIHCS)	~~(0GJIHCS)~~ (7DAS) (8BS)	", G
7	(0GJIHCS)		

TUTORIAL#1: SEARCH

Search Tree



The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

Let’s apply the following algorithm to solve the problem:

Breadth-First Search without the Visited List

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry.
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

2) BFS: Pick **first** element of Q

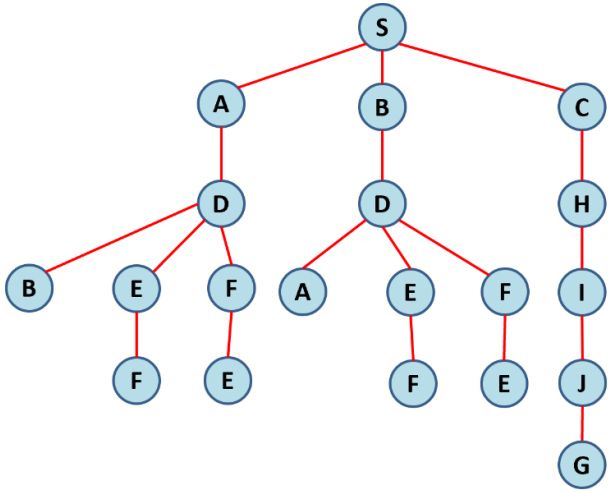
6) BFS: Add path extensions to **end** of Q.

SEARCH

Breadth-First Search without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)

Search Tree



Pick **first** element of Q; Add path extensions to **end** of Q.

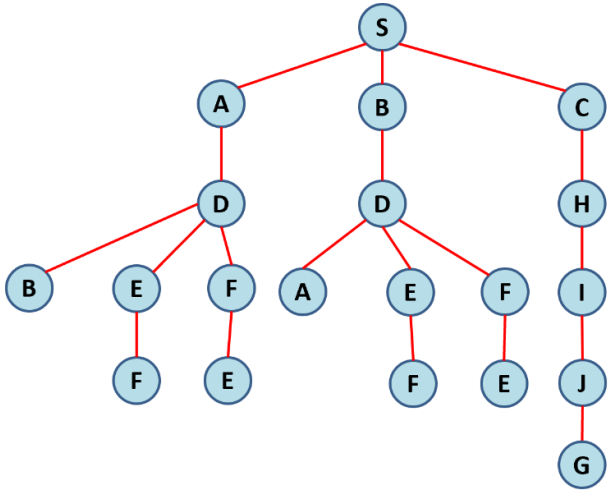
In **BFS** – nodes are pulled off the Q and inserted into the Q using a **queue**.

SEARCH

Breadth-First Search without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)
2	(AS)	(BS) (CS) (DAS)

Search Tree



Pick **first** element of Q; Add path extensions to **end** of Q.

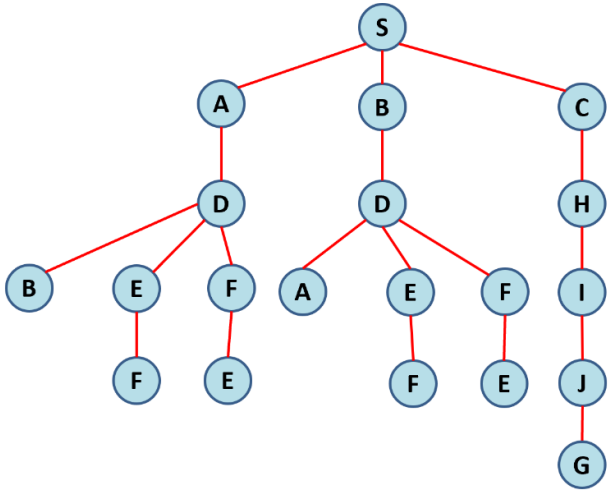
SEARCH

Breadth-First Search without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)
2	(AS)	(BS) (CS) (DAS)
3	(BS)	(CS)(DAS) (DBS)
4	(CS)	(DAS) (DBS) (HCS)
5	(DAS)	(DBS) (HCS) (BDAS)(EDAS)(FDAS)
6		

continued…

Search Tree



(DBS) is added into the Q as the algorithm is not checking for any visited states.

Pick **first** element of Q; Add path extensions to **end** of Q.

4) BFS without Visited list

Step	Deq	Enq
0		(S)
1	(S)	~~(AS)~~ (BS) (CS)
2	(AS)	~~(BS)~~ (CS) (DAS)
3	(BS)	~~(CS)~~ (DAS) (DBS)
4	(CS)	~~(DAS)~~ (DBS) (HCS)
5	(DAS)	~~(DBS)~~ (HCS) (BDAS) (EDAS) (FDAS)
6	(DBS)	~~(HCS)~~ (BDAS) (EDAS) (FDAS) (ADBS) (EDBS) (FDBS)
7	(HCS)	~~(BDAS)~~ (EDAS) (FDAS) (ADBS) (EDBS) (FDBS) (IHCS)
8	(BDAS)	~~(EDAS)~~ (FDAS) (ADBS) (EDBS) (FDBS) (IHCS)
9	(EDAS)	~~(FDAS)~~ (ADBS) (EDBS) (FDBS) (IHCS) (FEDAS)
10	(FDAS)	~~(ADBS)~~ (EDBS) (FDBS) (IHCS) (FEDAS) (EFDAS)
11	(ADBS)	~~(EDBS)~~ (FDBS) (IHCS) (FEDAS) (EFDAS)
12	(EDBS)	~~(FDBS)~~ (IHCS) (FEDAS) (EFDAS) (FEDBS)
13	(FDBS)	~~(IHCS)~~ (FEDAS) (EFDAS) (FEDBS) (EFDBS)
14	(IHCS)	~~(FEDAS)~~ (EFDAS) (FEDBS) (EFDBS) (JIHCS)
15	(FEDAS)	~~(EFDAS)~~ (FEDBS) (EFDBS) (JIHCS)
16	(EFDAS)	~~(FEDBS)~~ (EFDBS) (JIHCS)
17	(FEDBS)	~~(EFDBS)~~ (JIHCS)
18	(EFDBS)	~~(JIHCS)~~
19	(JIHCS)	~~(GJIHCS)~~
20	(GJIHCS)	

Sequence of state expansions = ?

Path = ?

Path length = ?

Max Q Length = ?

Number of state expansions = ?

SEARCH

Breadth-First Search without the Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Path length = ?

Max Q Length = ?

Number of state expansions = ?

SEARCH

Breadth-First Search without the Visited List

Characterisation:

Sequence of state expansions = S-A-B-C-D-D-H-B-E-F-A-E-F-I-F-E-F-E-J-G

Path = S-C-H-I-J-G

Path length = 5

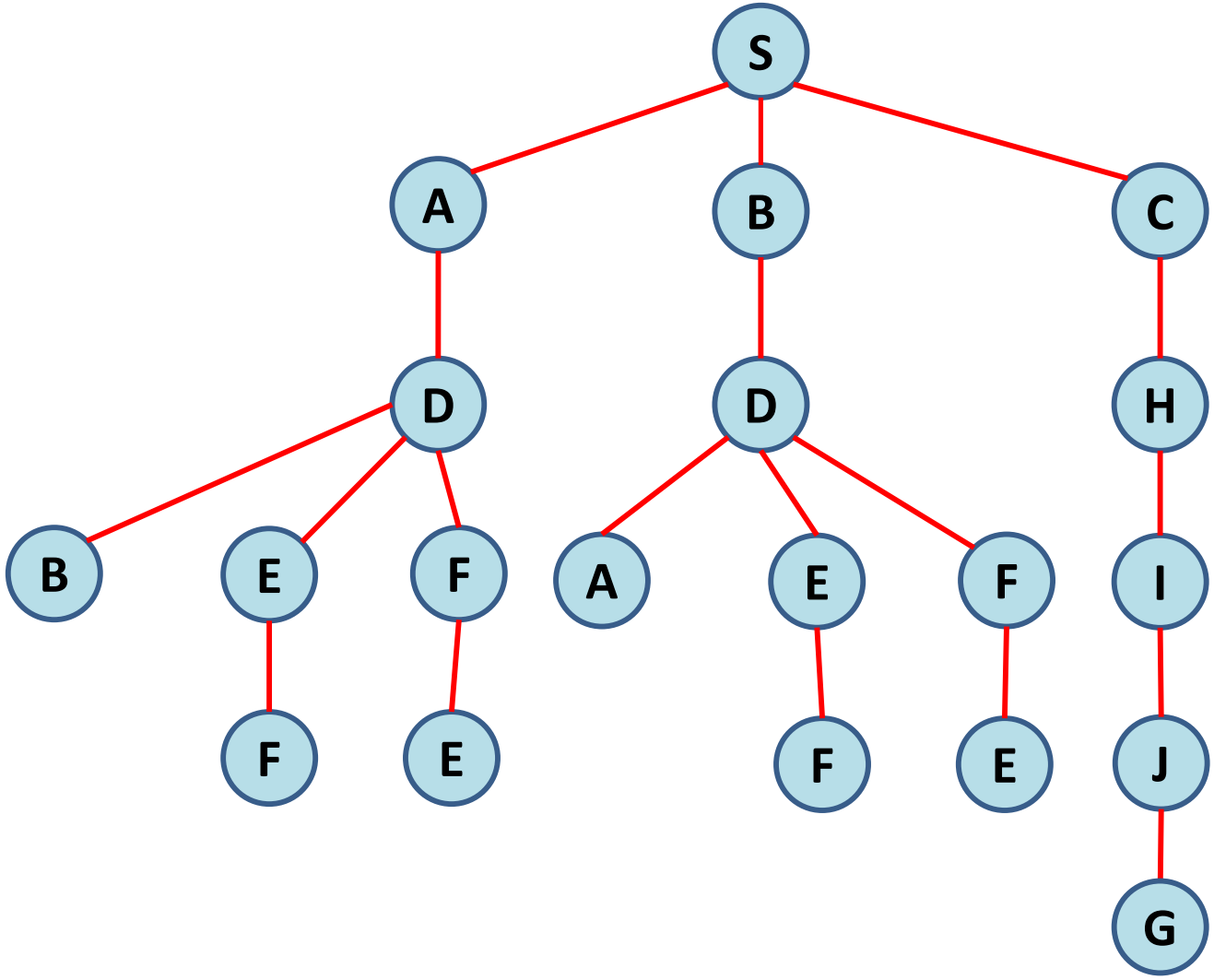
Max Q Length = 7

Number of state expansions = 20

Breadth-First Search without the Visited List expanded more states (20) than the one using the visited list. Moreover, it used more memory space.
For this simple example, we can see the consequence of not using the visited list. In the programming assignment, we will see much bigger differences in their performance.

TUTORIAL#1: SEARCH

Search Tree



The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

Let’s apply the following algorithm to solve the problem:

Depth-First Search without the Visited List

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry.
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

2) DFS: Pick **first** element of Q; Add path extensions to **front** of Q.

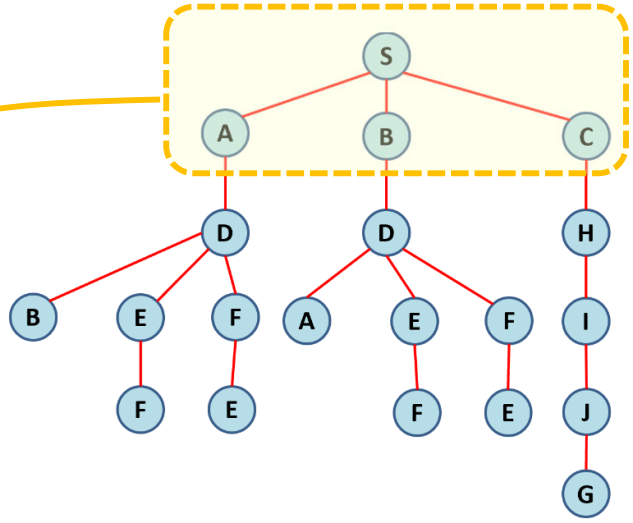
6) DFS: Nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

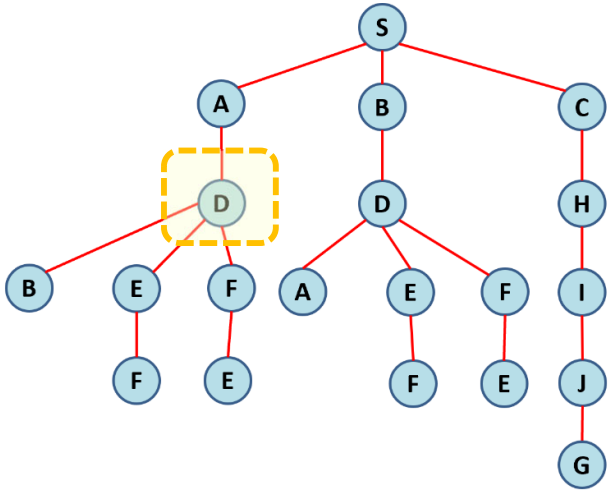
In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)
2	(AS)	(DAS) (BS) (CS)

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

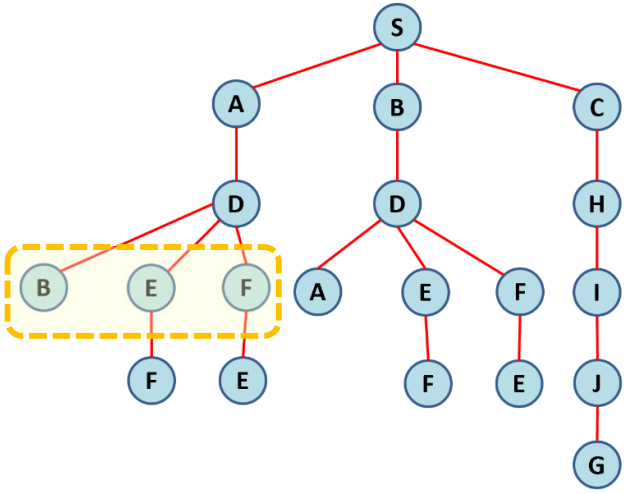
SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)
2	(AS)	(DAS) (BS) (CS)
3	(DAS)	(BDAS)(EDAS) (FDAS) (BS)(CS)
4	(BDAS)	(EDAS) (FDAS) (BS) (CS)
5	(EDAS)	(FEDAS) (FDAS) (BS) (CS)

continued…

Search Tree



(BDAS) is added as the algorithm has no way of telling whether it has already a node in the Q whose state is also B

Pick **first** element of Q; Add path extensions to **front** of Q.

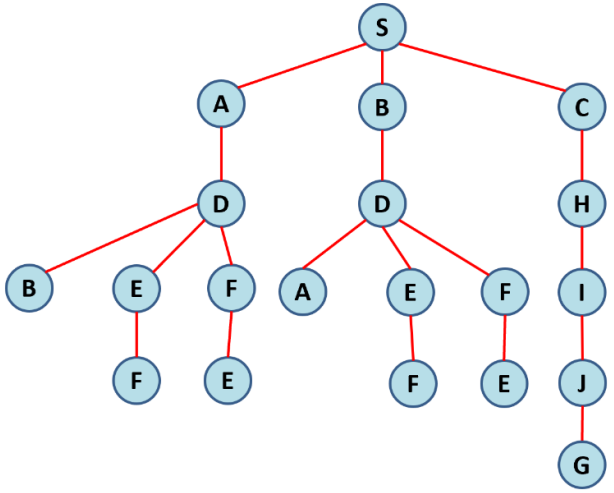
In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
6	(FEDAS)	(FDAS)(BS) (CS) dead end
7	(FDAS)	(EFDAS) (BS) (CS)
8	(EFDAS)	(BS)(CS)
9	(BS)	(DBS) (CS)
10	(DBS)	(ADBS)(EDBS) (FDBS)(CS)
11	(ADBS)	(EDBS) (FDBS)(CS)
…		

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

2): DFS without Visited List

Step	Dequened	Enqueued
0		(S)
1	(S)	(AS)(BS)(CS)
2	(AS)	(DAS)(BS)(CS)
3	(DAS)	(BDAS)(EDAS)(FDAS)(BS)(CS)
4	(BDAS)	(EDAS)(FDAS)(BS)(CS)
5	(EDAS)	(FEDAS)(FDAS)(BS)(CS)
6	(FEDAS)	(EDAS)(BS)(CS)
7	(FDAS)	(EFDAS)(BS)(CS)
8	(EFDAS)	(BS)(CS)
9	(BS)	(DBS)(CS)
10	(DBS)	(ADBS)(EDBS)(FDBS)(CS)
11	(ADBS)	(EDBS)(FDBS)(CS)
12	(EDBS)	(FEDBS)(FDBS)(CS)
13	(FEDBS)	(FDBS)(CS)
14	(FDBS)	(EFDBS)(CS)
15	(EFDBS)	(CS)
16	(CS)	(HCS)
17	(HCS)	(IHCS)
18	(IHCS)	(JIHCS)
19	(JIHCS)	(GJIHCS)
20	(GJIHCS)	

Sequence of State expansion: S-A-D-B-E-F-F-E-B-D-A-E-F-F-E-C-H-I-J-G

Depth-First without the Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Max Q Length = ?

Number of state expansions = ?

SEARCH

Depth-First without the Visited List

Characterisation:

Sequence of state expansions = S-A-D-B-E-F-F-E-B-D-A-E-F-F-E-C-H-I-J-G

Path = S-C-H-I-J-G

Max Q Length = 5

Number of state expansions = 20

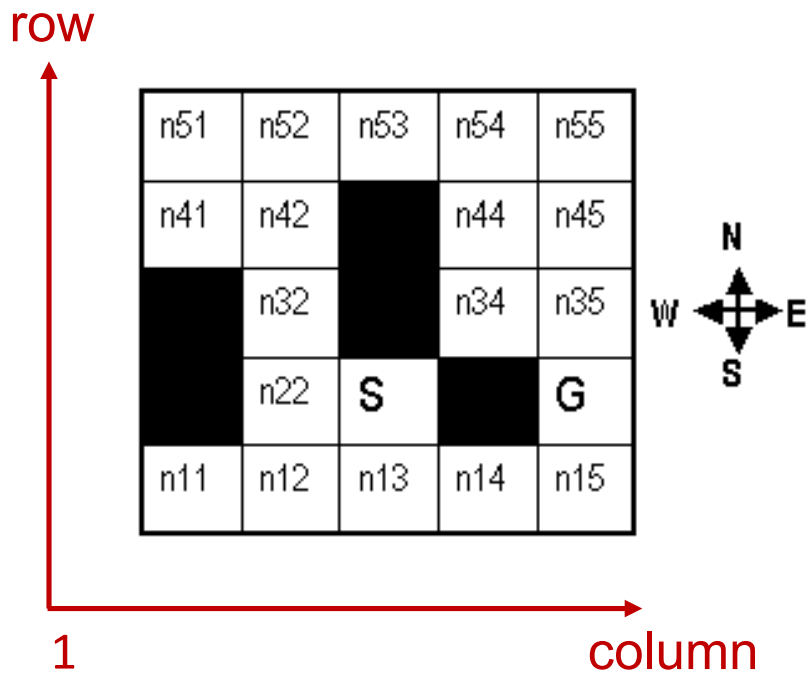
Depth-First Search without the Visited List expanded more states (20 vs. 11) than the one using the visited list. Moreover, it used more memory space (5 vs. 4). In this simple example, we can see the consequence of not using the visited list. In the programming assignment, we will see much bigger differences in their performance.

TUTORIAL#2: MAZE

Problem definition

We are trying to find a path from **S** to **G** in the following maze:

- **S** – start state
- **G** – goal state
- **Black** squares – blocked cells
- All squares, except for the start and goal states, are labelled, with row and column numbers.
  - For example, n54 is the cell in the fifth row, fourth column
- The graph **states** are the squares of the maze.



n.h.reyes@massey.ac.nz

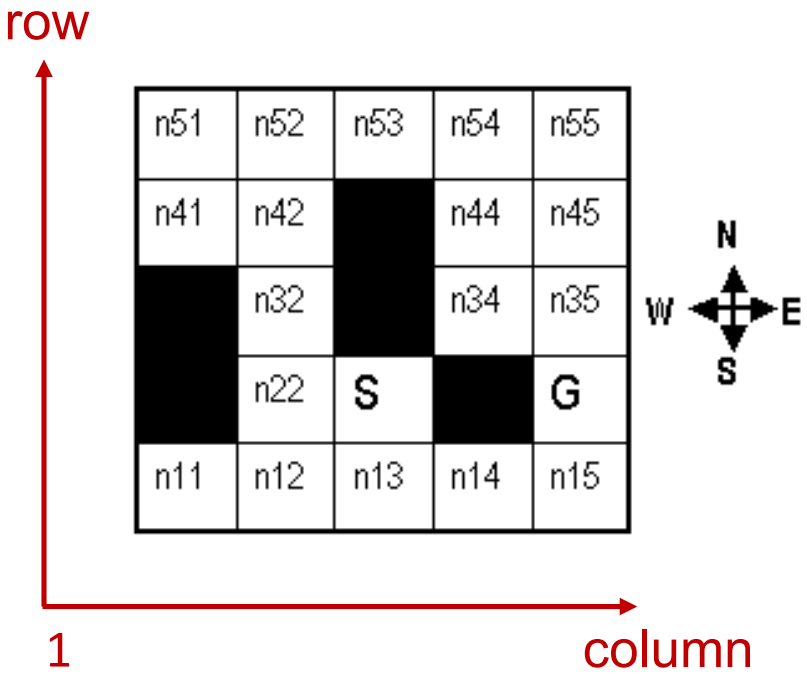
Reference: MIT OpenCourseWare

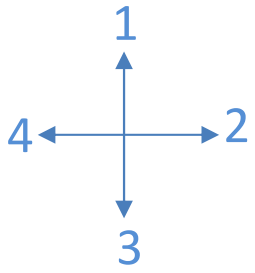
TUTORIAL#2: MAZE

Problem definition

We are trying to find a path from **S** to **G** in the following maze:

- There are **links** along the cardinal compass directions to neighbouring squares (that are not blocked). That is, diagonal moves are not permitted, nor are moves into or across a black square.
- The **successors** of a state are to be generated in a FIXED order to be drawn from left to right, namely N, E, S, W (Up, Right, Down, Left).

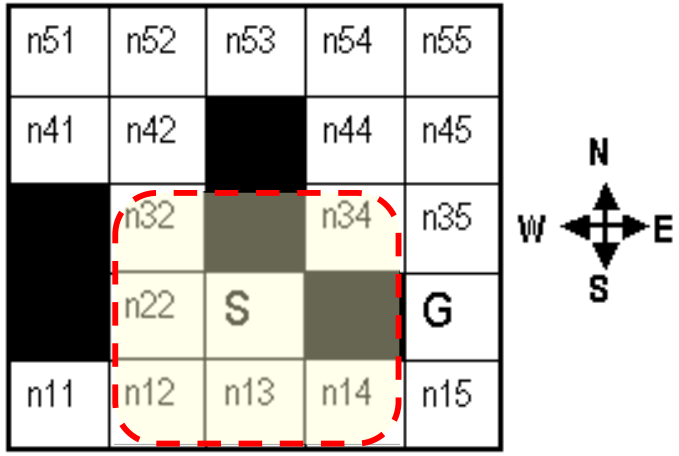




4-connected gridworld

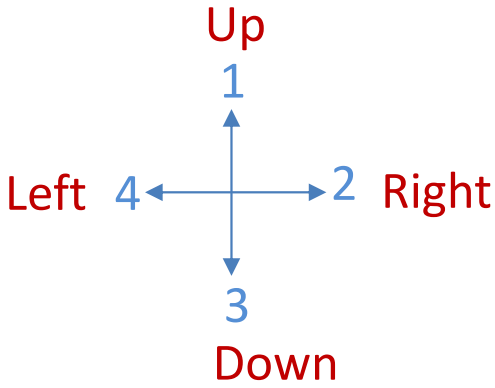
TUTORIAL#2: MAZE

Search Tree



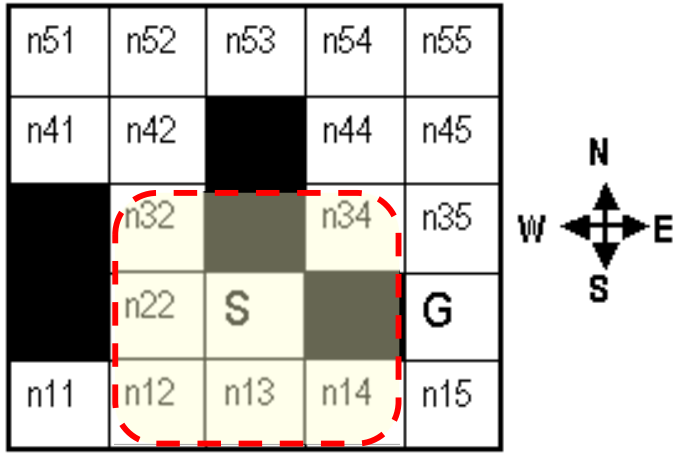
4-connected gridworld

- Start exploring the maze from S, considering the given successor generator ordering {up, right, down, left}.

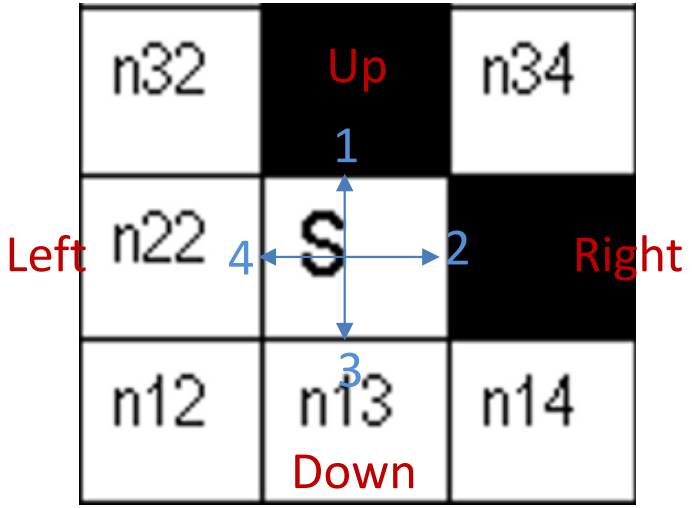


TUTORIAL#2: MAZE

Search Tree



4-connected gridworld



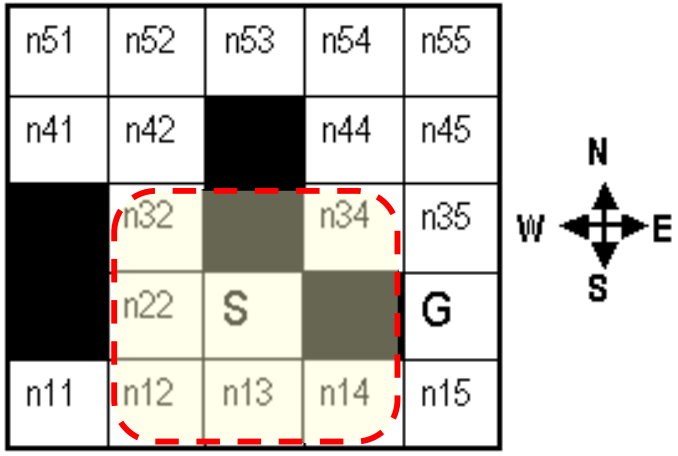
- From S, we cannot move up or right, as the cells in those neighbouring positions are all blocked. We can only move down or left.

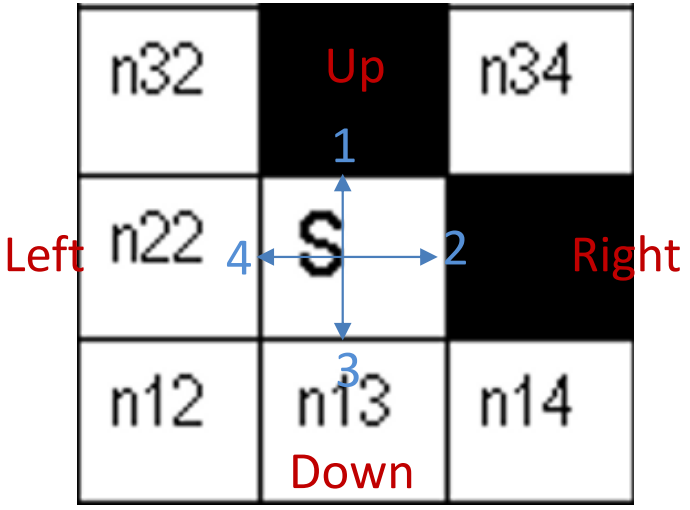
TUTORIAL#2: MAZE

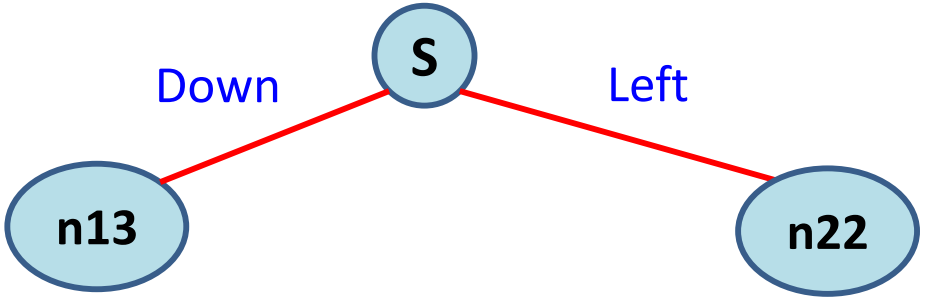
Search Tree

We are trying to find a path from **S** to **G** in the following maze:

4-connected gridworld







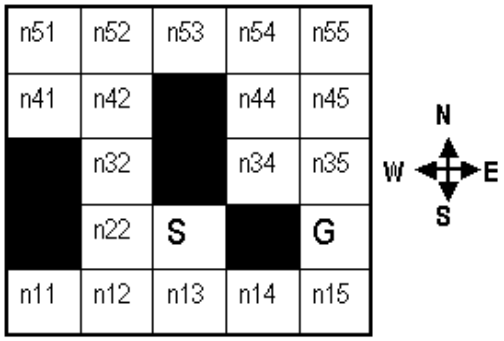
When generating a tree, remember to avoid local loops.

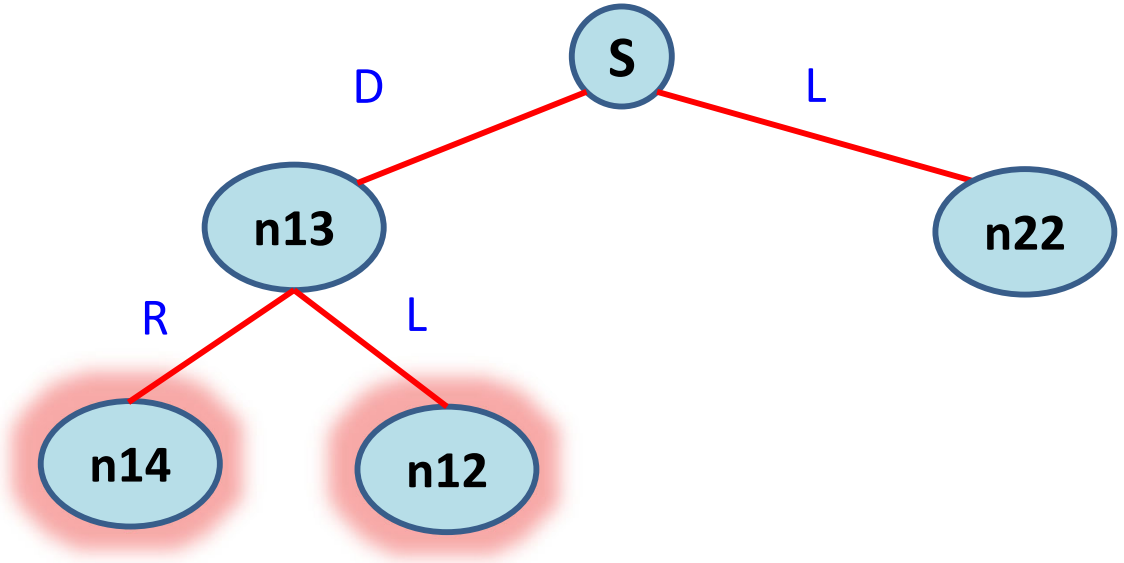
TUTORIAL#2: MAZE

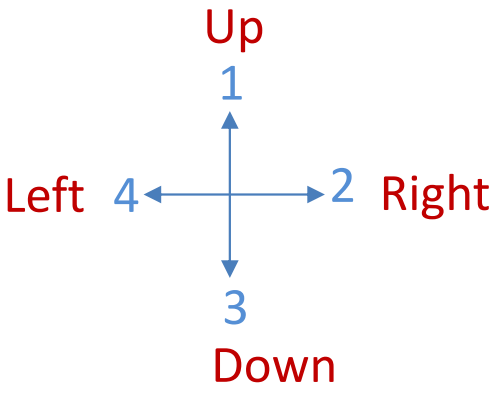
Search Tree

We are trying to find a path from **S** to **G** in the following maze:

4-connected gridworld







Extending the node n13, we can see that we can reach {S, n14 and n12}. However, we don't want to go back to S. Therefore, we are extending it to n14 and n12.

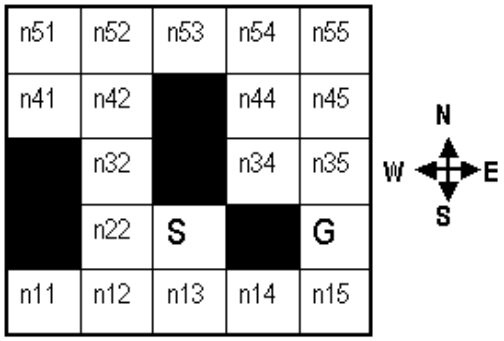
When generating a tree, remember to avoid local loops.

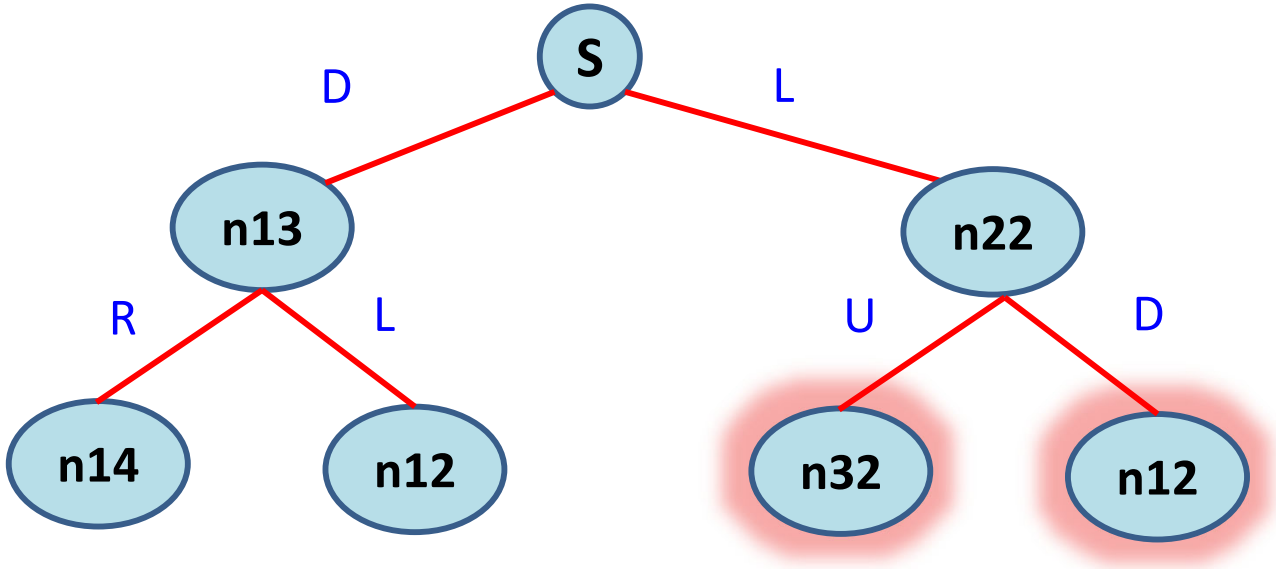
TUTORIAL#2: MAZE

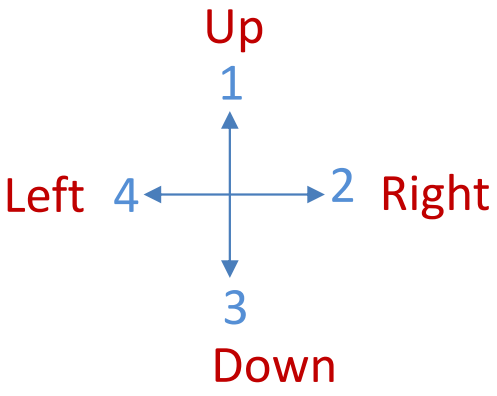
Search Tree

We are trying to find a path from **S** to **G** in the following maze:

4-connected gridworld





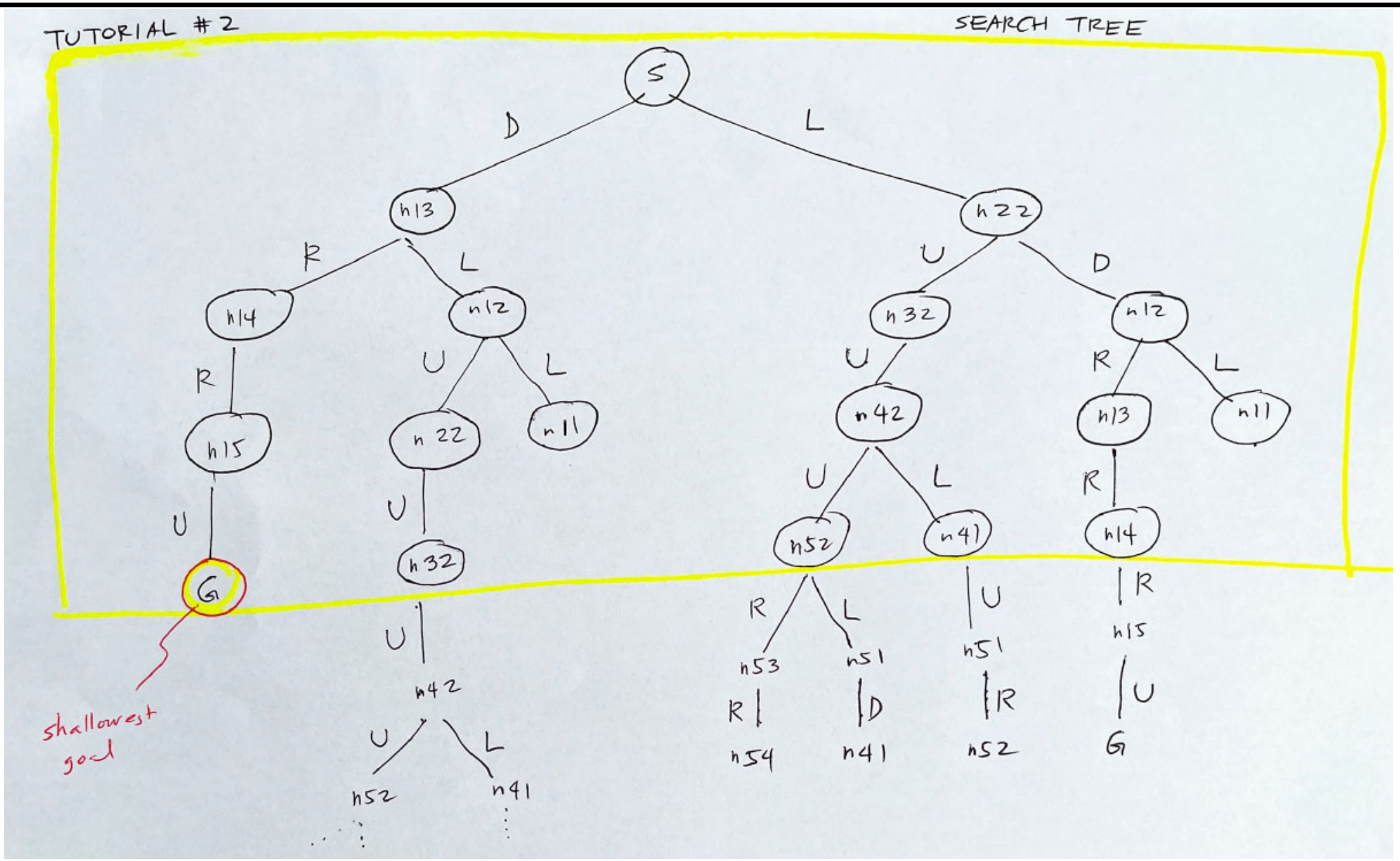


Extending the node n22, we can see that we can reach {n32,S and n12}. However, we don't want to go back to S. Therefore, we are extending it to n32 and n12.

When generating a tree, remember to avoid local loops.

TUTORIAL#2: MAZE

Search Tree

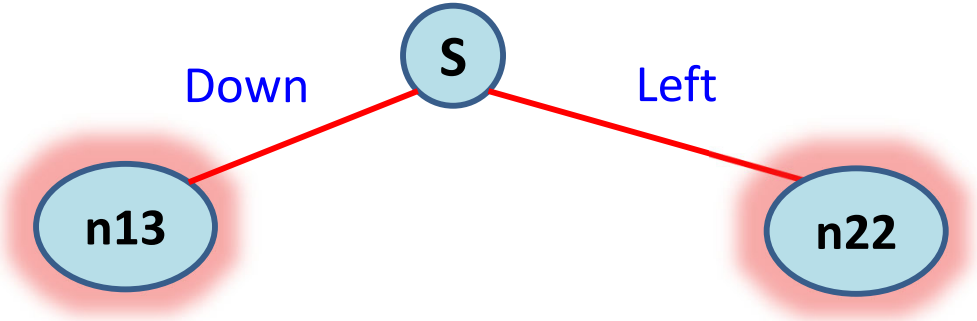


SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(n13,S) (n22,S)	S,n13,n22

Search Tree

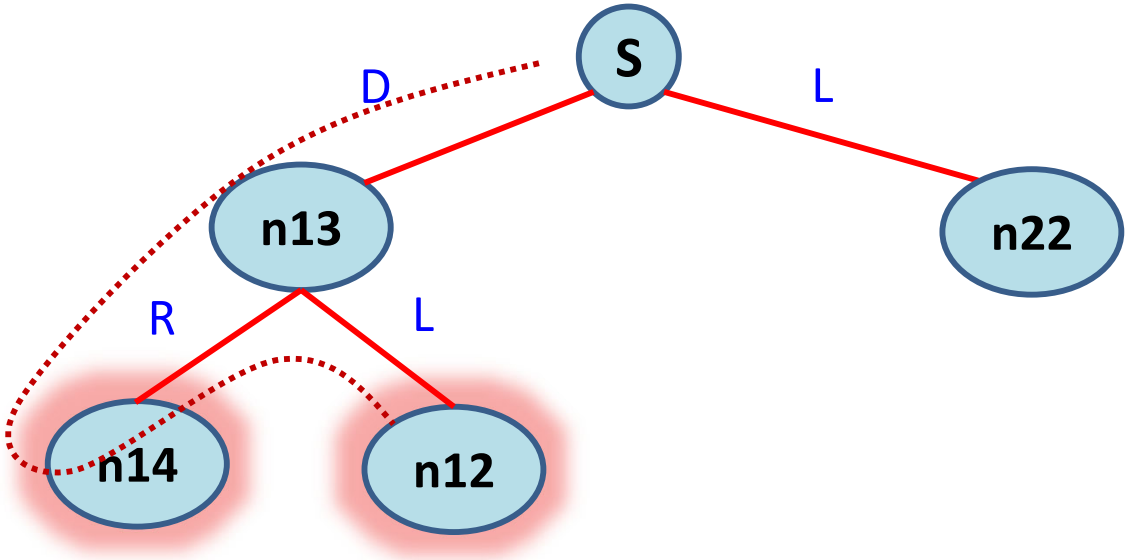


Pick **first** element of Q; Add path extensions to **end** of Q.

SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(n13,S) (n22,S)	S,n13,n22
2	(n13,S)	(n22,S) (n14,n13,S) (n12,n13,S)	“, n14, n12



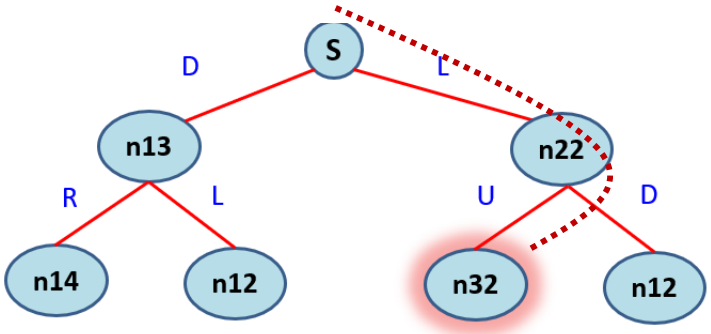
Pick **first** element of Q; Add path extensions to **end** of Q.

SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(n13,S) (n22,S)	S,n13,n22
2	(n13,S)	(n22,S) (n14,n13,S) (n12,n13,S)	“, n14, n12
3	(n22,S)	(n14,n13,S) (n12,n13,S) (n32,n22,S)	“, n32

We are skipping the path extension to **n12**, as we have visited that node previously already.



Pick **first** element of Q; Add path extensions to **end** of Q.

Breadth-First Search (with a Visited List)

Step	Dequeued	Enqueued	Visited List
0		~~(s)~~	s
1	(s)	~~(n13, s)~~ (n22, s)	s, n13, n22
2	(n13, s)	~~(n22, s)~~ (n14, n13, s) (n12, n13, s)	" n14, n12
3	(n22, s)	~~(n14, n13, s)~~ (n12, n13, s) (n32, n22, s)	" n32
4	(n14, n13, s)	~~(n12, n13, s)~~ (n32, n22, s) (n15, n14, n13, s)	" n15
5	(n12, n13, s)	~~(n32, n22, s)~~ (n15, n14, n13, s) (n11, n12, n13, s)	" n11
6	(n32, n22, s)	~~(n15, n14, n13, s)~~ (n11, n12, n13, s) (n42, n32, n22, s)	" n42
7	(n15, n14, n13, s)	~~(n11, n12, n13, s)~~ (n42, n32, n22, s) (G, n15, n14, n13, s)	" G
8	(n11, n12, n13, s)	~~(n42, n32, n22, s)~~ (G, n15, n14, n13, s)	"
9	(n42, n32, n22, s)	~~(G, n15, n14, n13, s)~~ (n52, n42, n32, n22, s) (n41, n42, n32, n22, s)	" n52, n41
10	(G, n15, n14, n13, s)		

Breadth-First Search (with a Visited List)

Step	Dequeued	Enqueued	
0		(S)	
1	(S)	(n13, S) (n22, S)	
2	(n13, S)	(n22, S) (n14, n13, S) (n12, n13, S)	
3	(n22, S)	(n14, n13, S) (n12, n13, S) (n32, n22, S)	" n32
4	(n14, n13, S)	(n12, n13, S) (n32, n22, S) (n15, n14, n13, S)	" n15
5	(n12, n13, S)	(n32, n22, S) (n15, n14, n13, S) (n11, n12, n13, S)	" n11
6	(n32, n22, S)	(n15, n14, n13, S) (n11, n12, n13, S) (n42, n32, n22, S)	" n42
7	(n15, n14, n13, S)	(n11, n12, n13, S) (n42, n32, n22, S) (G, n15, n14, n13, S)	" G
8	(n11, n12, n13, S)	(n42, n32, n22, S) (G, n15, n14, n13, S)	"
9	(n42, n32, n22, S)	(G, n15, n14, n13, S) (n52, n42, n32, n22, S) (n41, n42, n32, n22, S)	" n52, n41
10	(G, n15, n14, n13, S)		

Sequence of state expansions = S-n13-n22-n14-n12-n32-n15-n11-n42-G

Path = S-n13-n14-n15-G

Path length = 4

Max Q Length = 3

Number of state expansions = 10

Breadth- First Search (with a Visited List)

Step	Dequeued	Enqueued	Visited List
0		(S)	
1	(S)	(n13, S) (n22, S)	S
2	(n13, S)	(n22, S) (n14, n13, S) (n12, n13, S)	S, n13, n22
3	(n22, S)	(n14, n13, S) (n12, n13, S) (n32, n22, S)	" n14, n12
4	(n14, n13, S)	(n12, n13, S) (n32	" n32
5	(n12, n13, S)	(n32, n22, S) (	
6	(n32, n22, S)	(n15, n14, n13, S) (	
7	(n15, n14, n13, S)	(n11, n12, n13, S) (	
8	(n11, n12, n13, S)	(n42, n32, n22	
9	(n42, n32, n22, S)	(G, n1	
10	(G, n15, n14, n13, S)		

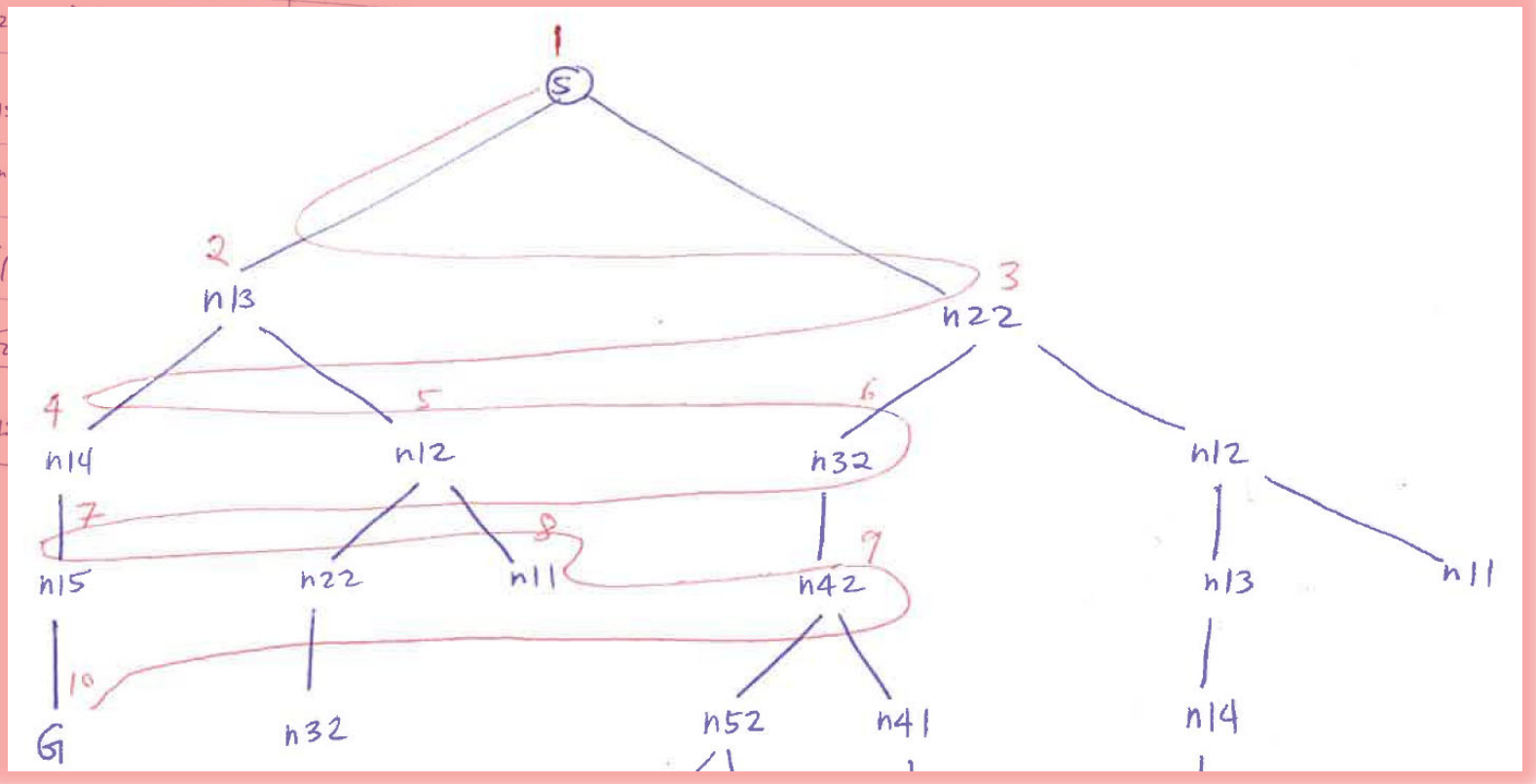
Sequence of state expansions = S-n13-n22-n14-n12-n32-n15-n11-n42-G

Path = S-n13-n14-n15-G

Path length = 4

Max Q Length = 3

Number of state expansions = 10



Tracing the sequence of state expansions shows us that the algorithm traverses the search tree following the breadth-first approach indeed.

SEARCH

Breadth-First Search with Visited List

Characterisation:

Sequence of state expansions = S-n13-n22-n14-n12-n32-n15-n11-n42-G

Path = S-n13-n14-n15-G

Path length = 4

Max Q Length = 3

Number of state expansions = 10

The Farmer, Goose, Fox and Grain Problem

- Learn how to perform abstractions in order to make the problem solvable using Search
- States are generated on-the-fly; that is, neither the graph, nor the tree is given beforehand

n.h.reyes@massey.ac.nz

The farmer-fox-goose-grain problem.

A farmer has to cross a river with his fox, goose and grain. However, his little boat can only accommodate himself and one of his possessions. Therefore, in order to bring all of them across the river he needs to cross it several times. During planning, he is cautious of two unwanted situations: an unguarded fox will eat the goose, and an unguarded goose will eat the grain.

How to approach this problem using AI?

How can he cross the river without losing his possessions?

How to represent the problem as a Search problem?

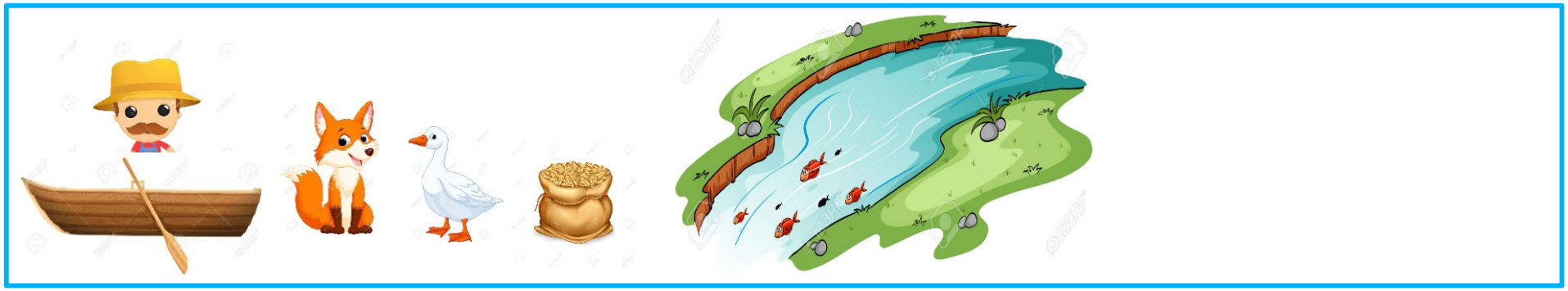
What are the constraints to consider in this problem?

Image: http://cairnarvon.rotahall.org/2010/02/28/search-trees-are-easy/

How many possible arrangements (states) are there? How many valid states?

Preconditions

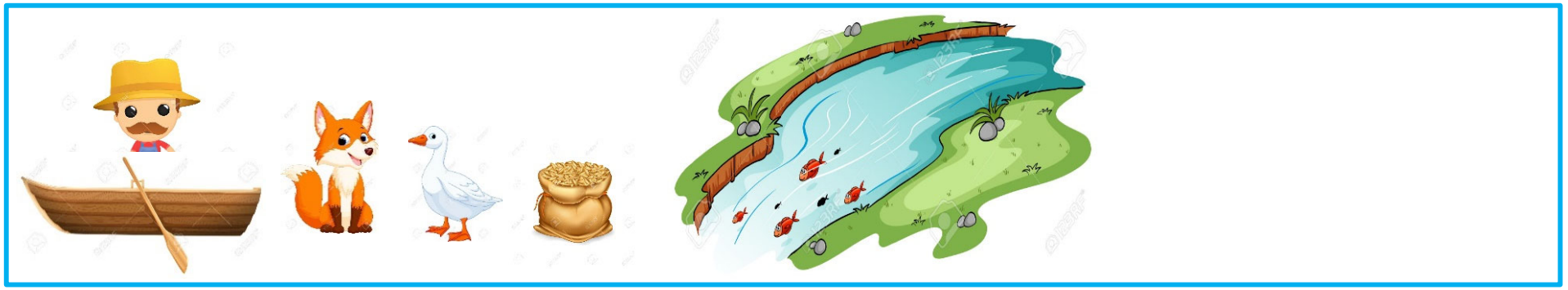
- The **fox** and the **goose** should not be in the same side of the river, not unless the farmer is with them.
- The **goose** and the **grain** should not be in the same side of the river, not unless the farmer is with them.



Left river bank river right river bank

Rules

1. Farmer can travel alone to the right river bank.
2. Farmer can travel alone to the left river bank.
3. Farmer can bring one of his possessions to the right river bank, provided that all preconditions are met.
4. Farmer can bring one of his possessions to the left river bank, provided that all preconditions are met.
5. The farmer should cross the river, every time he makes a move.



Left river bank river right river bank

Representation

How to represent the problem as a Search problem?

The objective is to move all of the farmer's possessions to the right side of the river bank.

What are the **States? Actions? Goal test?**

Initial state of the problem:

Left river bank river right river bank

States

The **state** is a tuple of **4 booleans**, representing the position of the farmer, goose, fox and grain, relative to the river.

Fa, Fo, Go, Gr

Initial state of the problem:

Left river bank river right river bank

Notation: When all 4 boolean variables are set to false, that means the farmer and all of his possessions are in the left side of the river bank.

Fa=false, Fo=false, Go=false, Gr=false → 0, 0, 0, 0

States

The **state** is a tuple of **4 booleans**, representing the position of the farmer, goose, fox and grain, relative to the river.

Fa, Fo, Go, Gr

Initial state of the problem:

Left river bank river right river bank

Notation: When all 4 boolean variables are set to false, that means the farmer and all of his possessions are in the left side of the river bank.

Fa=false, Fo=false, Go=false, Gr=false → 0, 0, 0, 0

In this hand-simulation, to make the notations more intuitive, we can write the same state as follows:

< Fa, Fo, Go, Gr | > The bar indicates the position of the river, relative to all characters.

Actions/Operators

There are **4 operators:** the farmer crosses alone, or with one of his three possessions. In each case all the preconditions must be met.

e.g. the farmer travels to the right river bank, the precondition is goose not equal fox and goose not equal grain.

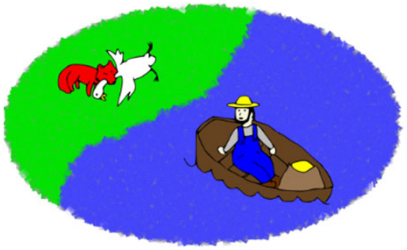
Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.

Invalid States

Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.



The following are invalid states that we should avoid:

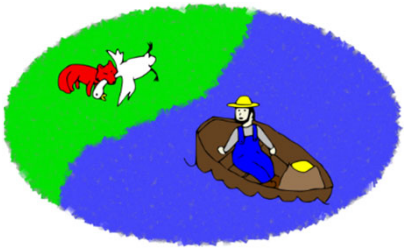
When the farmer travels alone to one of the river banks:

Fo, Go, Gr | **Fa**	The goose eats the grain, or the fox eats the goose first.
Fa | Fo, Go, Gr	The goose eats the grain, or the fox eats the goose first.

Invalid States

Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.



The following are invalid states that we should avoid:

When the farmer travels alone to one of the river banks:

Fo, Go, Gr | **Fa**	The goose eats the grain, or the fox eats the goose first.
Fa | Fo, Go, Gr	The goose eats the grain, or the fox eats the goose first.

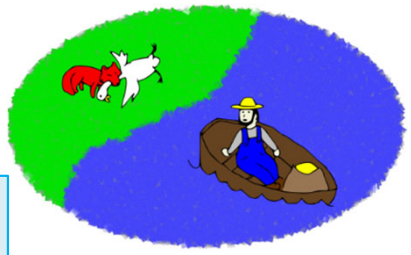
When the farmer travels with one of his possessions to one of the river banks:

Fo, Go | **Fa**, Gr	the fox eats the goose.
Fa, Gr | Fo, Go	the fox eats the goose.
Go, Gr | **Fa**, Fo	the goose eats the grain.
Fa, Fo | Go, Gr	the goose eats the grain.

There are (2^4)=**16 states** in total, but **6 are invalid**. Therefore, there are only **10 valid** states.

Goal Test

How to check if the algorithm has succeeded already?



Goal test:

All Booleans are true. That means all possessions, including the farmer are already on the right river bank.

i.e. farmer (**Fa**) == fox (**Fo**) == goose (**Go**) == grain (**Gr**) == true

Using our notation for the hand-simulation:

| Fa, Fo, Go, Gr

*To keep track of the characters in the puzzle more easily, enumerate them in alphabetical order

Exercise

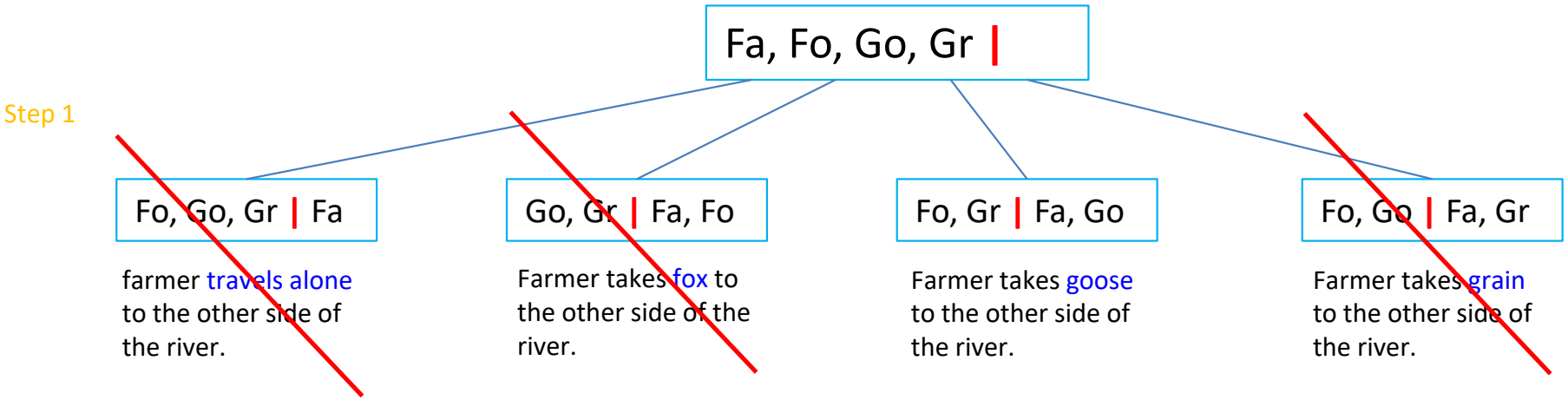
Solve the problem using Depth-First Search

In your solution, consider the following sequence of moves:

1. Farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.

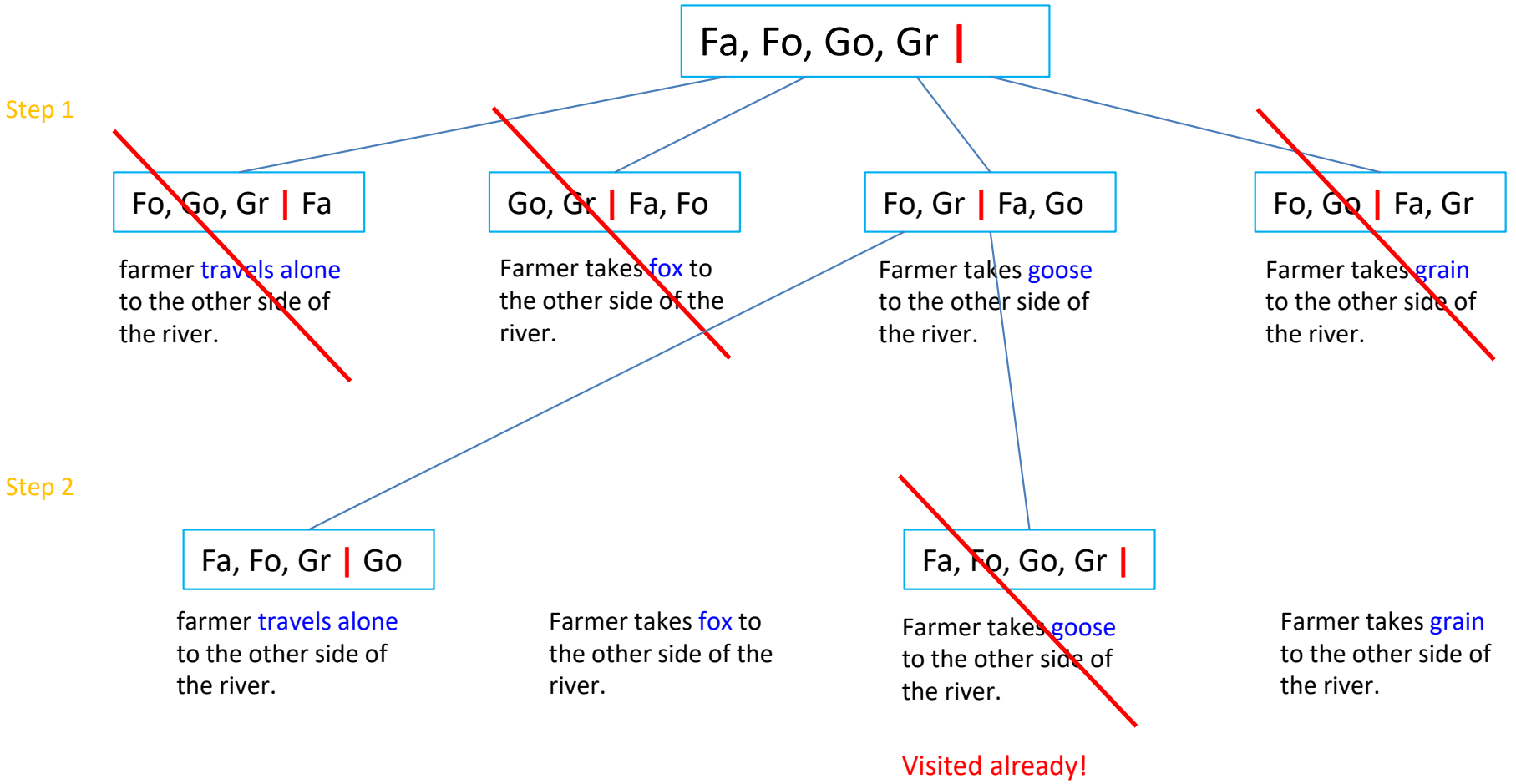
Initial state of the problem:

Left river bank river right river bank



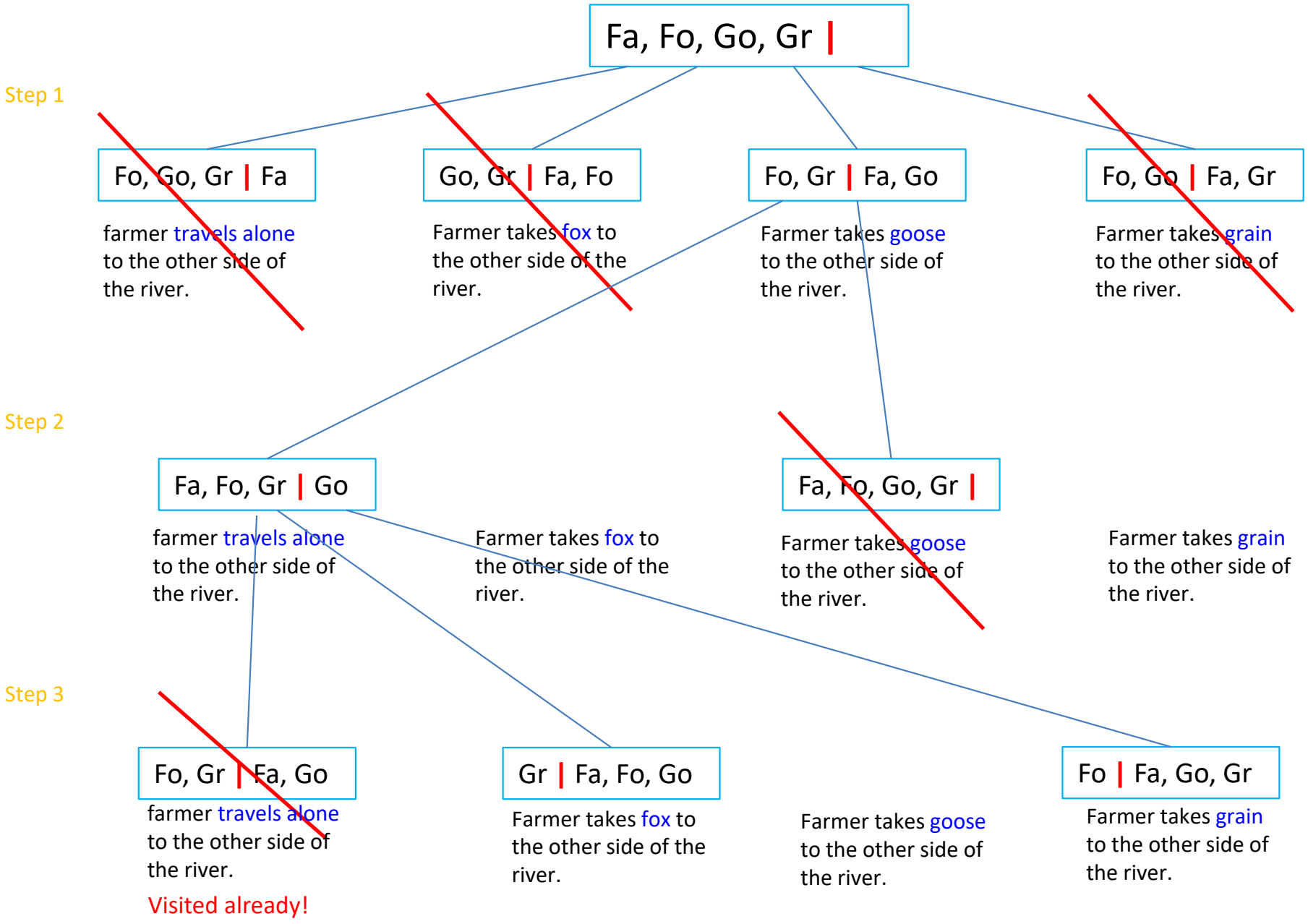
SEQUENCE OF MOVES

1. farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.



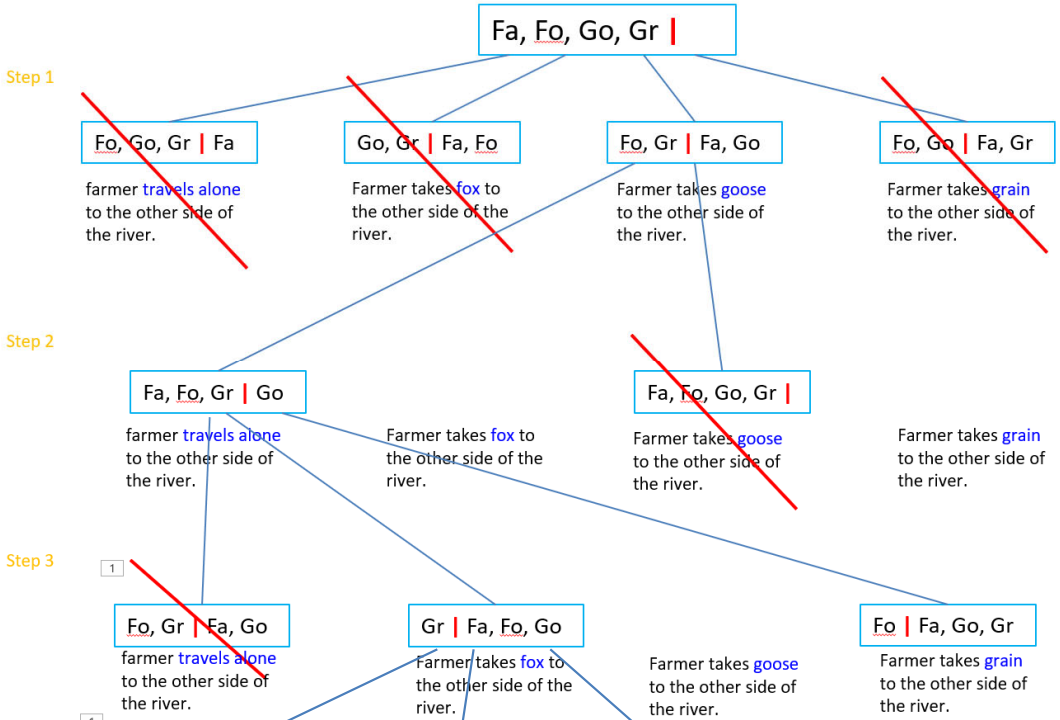
SEQUENCE OF MOVES

1. farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.



SEQUENCE OF MOVES

1. farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.



Step 4

Fa, Gr | Fo, Go

farmer travels alone to the other side of the river.

Fox eats goose!

Fa Fo Gr | Go

Farmer takes fox to the other side of the river.

Visited already!

Fa Go Gr | Fo

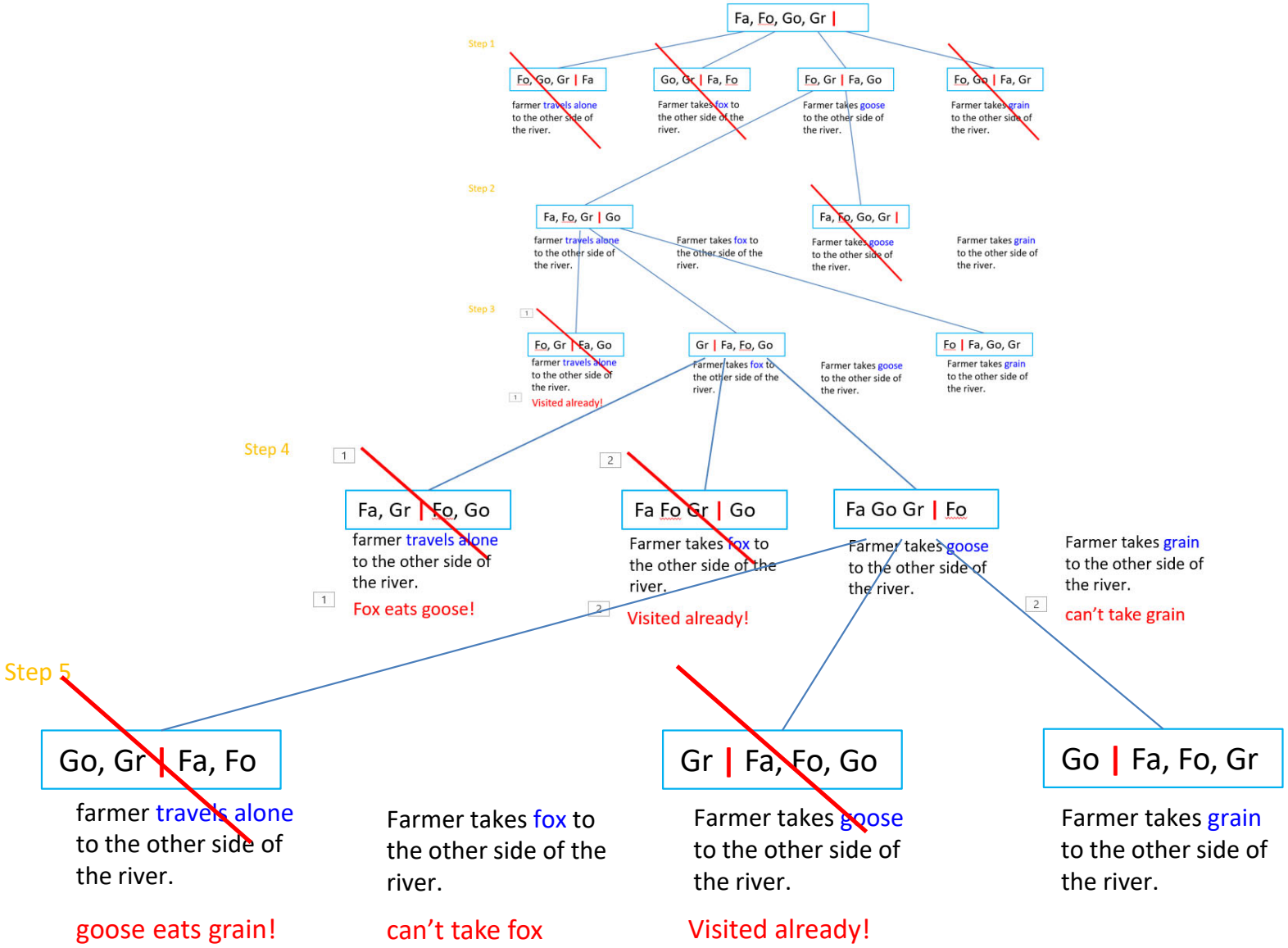
Farmer takes goose to the other side of the river.

Farmer takes grain to the other side of the river.

can't take grain

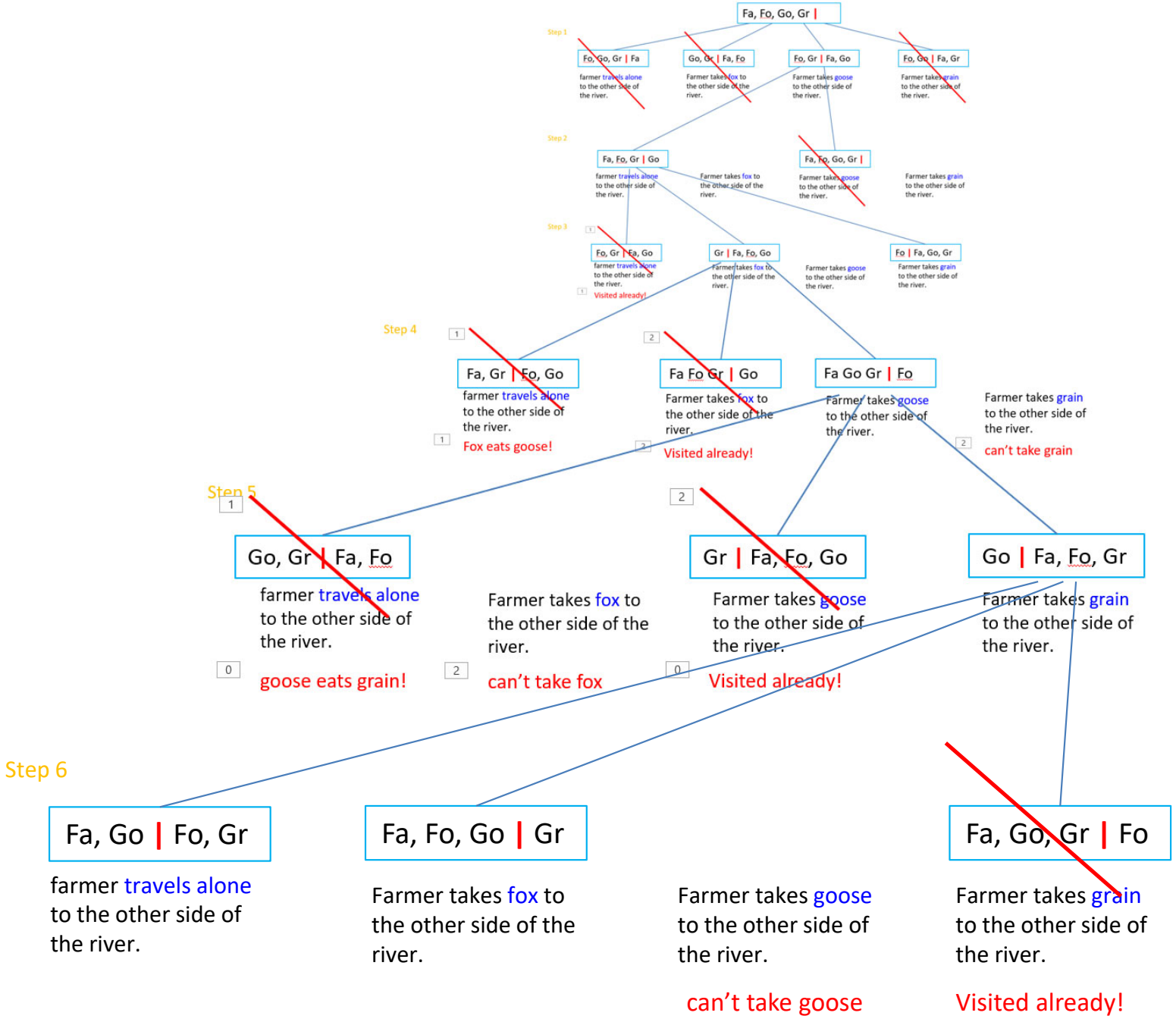
SEQUENCE OF MOVES

1. farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.



SEQUENCE OF MOVES

1. farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.

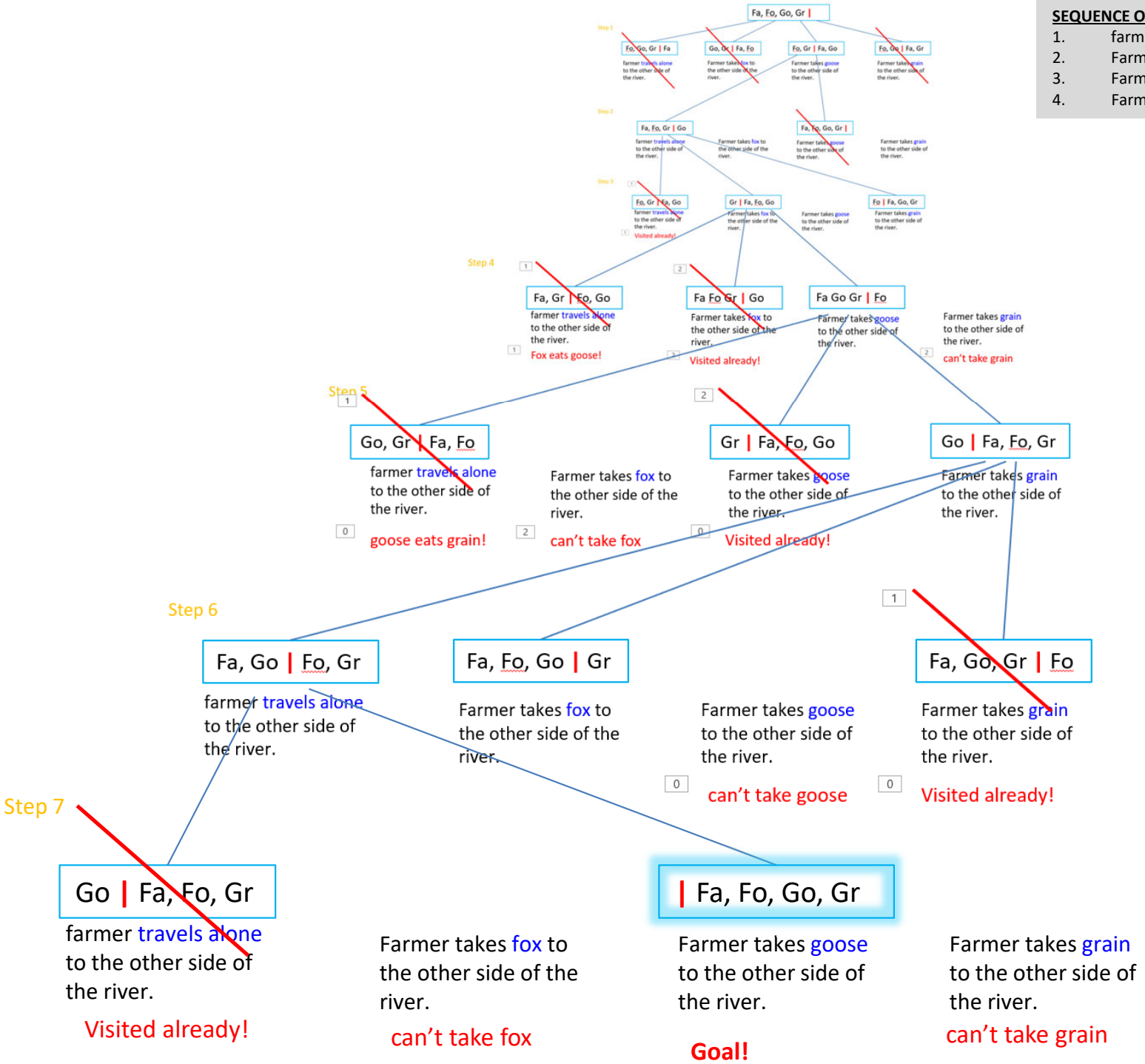


SEQUENCE OF MOVES

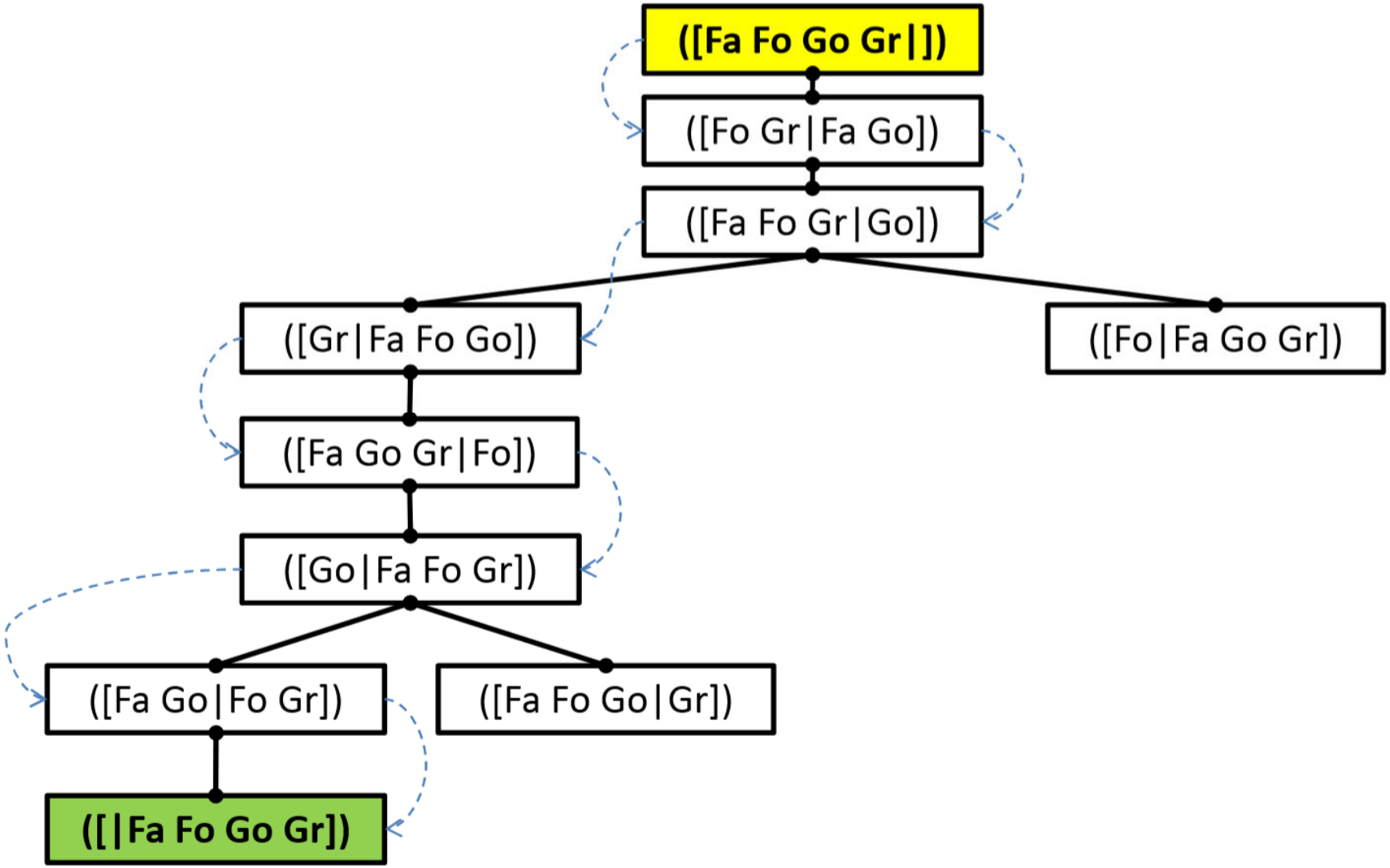
1. farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.

SEQUENCE OF MOVES

1. farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.



DFS Search Results



Hand-simulation

- Next let’s see the solution to the problem in tabular form.

Solution

Step 0: (Start state) Initially, the farmer and all of his possessions are on the left river bank.

STEP	DEQUEUED	ENQUEUED	VISITED LIST
0		(<Fa Fo Go Gr | >)	<Fa Fo Go Gr |>

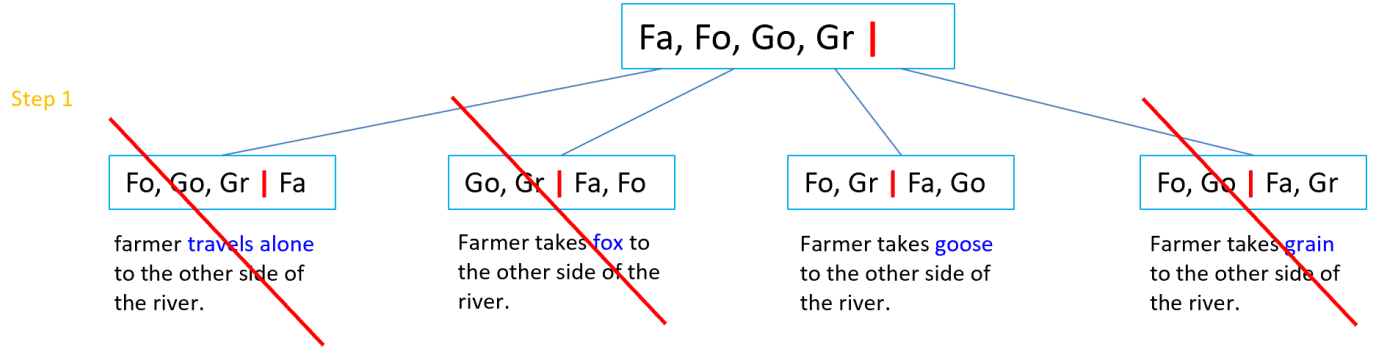
< Fa, Fo, Go, Gr | > The bar indicates the position of the river, relative to all characters.

Notation: When all 4 boolean variables are set to false, that means the farmer and all of his possessions are on the left side of the river bank.

Solution

Step 1: After expanding the path (<Fa Fo Go Gr | >) and applying the preconditions, we can see that only one child node is valid.

STEP	DEQUEUED	ENQUEUED	VISITED LIST
0		(<Fa Fo Go Gr | >)	<Fa Fo Go Gr |>
1	(<Fa Fo Go Gr|>)	(<Fo Gr | Fa Go> <Fa Fo Go Gr | >)	", <Fo Gr | Fa Go>

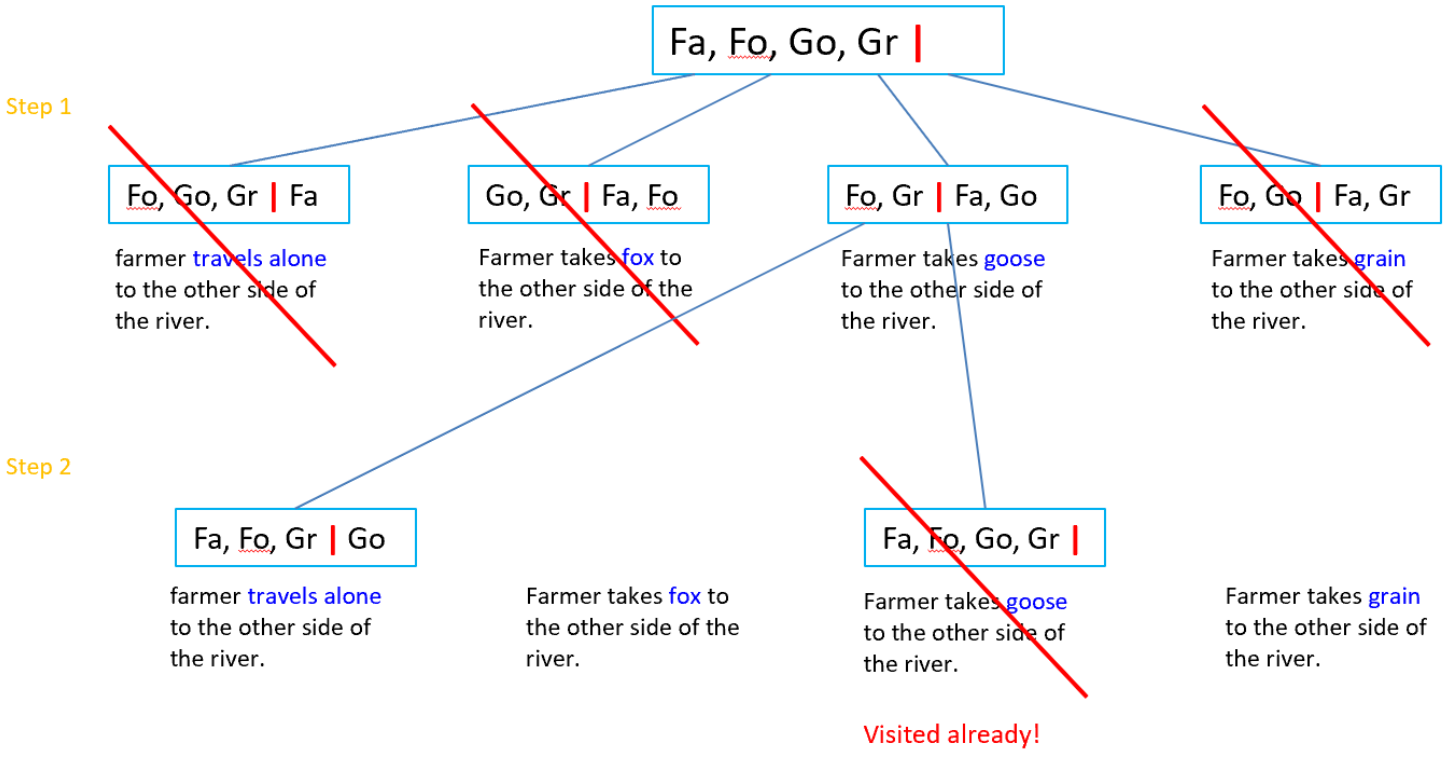


Note: The path extension requires that we concatenate the states together, writing the new state to the left of the previous state.

Solution

Step 2: After expanding the path (<Fo Gr | Fa Go> <Fa Fo Go Gr | >)and applying the preconditions, we can see that only one child node is valid.

STEP	DEQUEUED	ENQUEUED	VISITED LIST
0		(<Fa Fo Go Gr | >)	<Fa Fo Go Gr |>
1	(<Fa Fo Go Gr|>)	(<Fo Gr | Fa Go> <Fa Fo Go Gr | >)	", <Fo Gr | Fa Go>
2	(<Fo Gr | Fa Go> <Fa Fo Go Gr|>)	(<Fa Fo Gr | Go> <Fo Gr | Fa Go> <Fa Fo Go Gr |>)	", <Fa Fo Gr | Go>



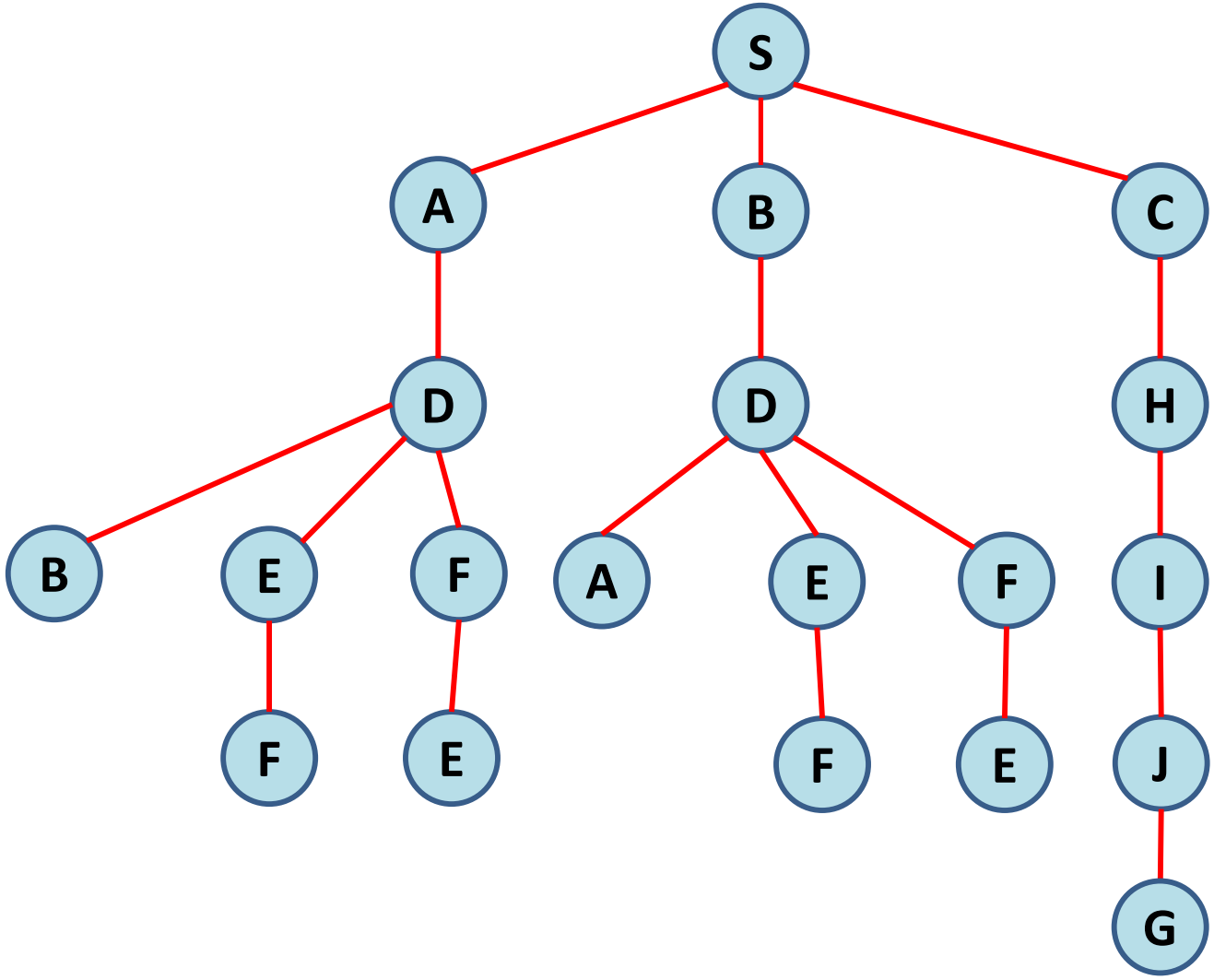
Solution

The process repeats until on the 8th Step, we are able to pull off a node whose state is the Goal state.

STEP	DEQUEUED	ENQUEUED	VISITED LIST
0		<Fa Fo Go Gr>	<Fa Fo Go Gr>
1	<Fa Fo Go Gr>	~~(<Fo Gr | Fa Go><Fa Fo Go Gr>)~~	", <Fo Gr | Fa Go>
2	(<Fo Gr | Fa Go><Fa Fo Go Gr>)	~~(<Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)~~	", <Fa Fo Gr | Go>
3	(<Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	~~(<Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)~~, (<Fo | Fa Go Gr><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	", <Gr | Fa Fo Go>, <Fo | Fa Go Gr>
4	(<Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	~~(<Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)~~, (<Fo | Fa Go Gr><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	", <Fa Go Gr | Fo>
5	(<Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	~~(< Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)~~, (<Fo | Fa Go Gr><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	", < Go | Fa Fo Gr >
6	(< Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	~~(< Fa Go | Fo Gr > < Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)~~, (< Fa Fo Go | Gr > < Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>), (<Fo | Fa Go Gr><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	", < Fa Go | Fo Gr >, < Fa Fo Go | Gr >
7	(< Fa Go | Fo Gr > < Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	~~(< | Fa Fo Go Gr > < Fa Go | Fo Gr > < Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)~~, (< Fa Fo Go | Gr > < Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>), (<Fo | Fa Go Gr><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	", < | Fa Fo Go Gr>
8	(< | Fa Fo Go Gr > < Fa Go | Fo Gr > < Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	(< Fa Fo Go | Gr > < Go | Fa Fo Gr > <Fa Go Gr | Fo> <Gr | Fa Fo Go><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>), (<Fo | Fa Go Gr><Fa Fo Gr | Go><Fo Gr | Fa Go><Fa Fo Go Gr>)	

TUTORIAL#1: SEARCH

Search Tree



The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

Let’s apply the following algorithm to solve the problem:

Best-First Search with the Visited List

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (h-val S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q, including their h-values; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

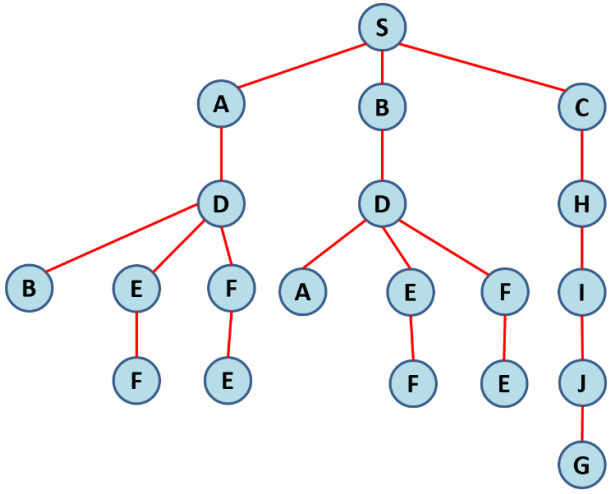
2) Best-First Search: Pick element from Q according to lowest heuristic .

6) Best-First Search : Add path extensions to **front** of Q. (our convention)

Best-First Search with Visited List

Step	Dequeued	Q	Visited
0		(10S)	S
1	(10S)	(5AS) (8BS) (6CS)	S,A,B,C

Search Tree



Heuristic values:

S = 10, A = 5, B = 8, C = 6,
D = 7, E = 6, F = 5, H = 3,
I = 2, J = 1, G = 0

Pick element from Q, based on lowest heuristic

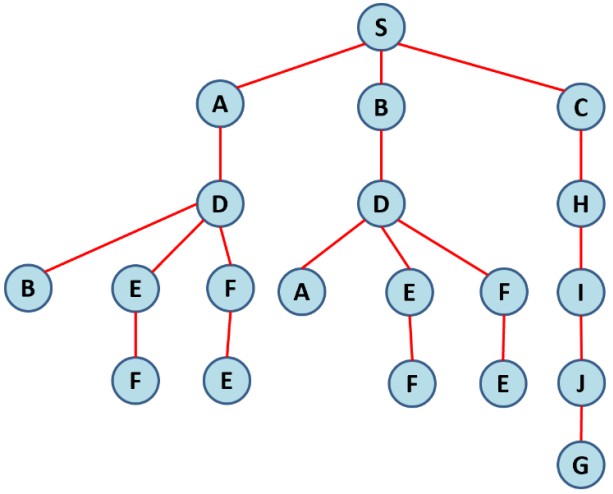
Add path extensions to **front** of Q (convention). Q is best implemented using a **priority queue**.

SEARCH

Best-First Search with Visited List

Step	Dequeued	Q	Visited
0		(10S)	S
1	(10S)	(5AS) (8BS) (6CS)	S,A,B,C
2	(5AS)	(7DAS) (8BS) (6CS)	“, D

Search Tree



“ – means including all previous states visited

Heuristic values:

S = 10, A = 5, B = 8, C = 6,
D = 7, E = 6, F = 5, H = 3,
I = 2, J = 1, G = 0

Pick element from Q, based on lowest heuristic

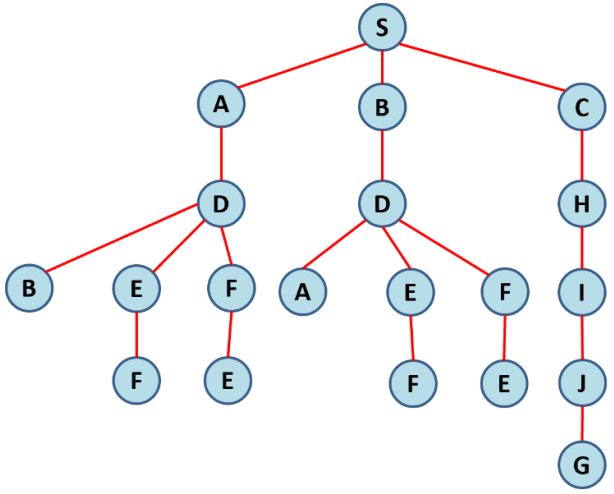
Add path extensions to **front** of Q (convention). Q is best implemented using a **priority queue**.

SEARCH

Best-First Search with Visited List

Step	Dequeued	Q	Visited
0		(10S)	S
1	(10S)	(5AS) (8BS) (6CS)	S,A,B,C
2	(5AS)	(7DAS) (8BS) (6CS)	“, D
3	(6CS)	(3HCS)(7DAS)(8BS)	“, H
4	(3HCS)	(2 IHCS) (7DAS)(8BS)	“, I
5	(2 IHCS)	(1 JIHCS) (7DAS)(8BS)	“, J
6	(1 JIHCS)	(0 GJIHCS) (7DAS)(8BS)	“, G
7	(0 GJIHCS)	(7DAS)(8BS)	

Search Tree



Heuristic values:

S = 10, A = 5, B = 8, C = 6,
D = 7, E = 6, F = 5, H = 3,
I = 2, J = 1, G = 0

Pick element from Q, based on lowest heuristic

Add path extensions to **front** of Q (convention). Q is best implemented using a **priority queue**.

Best-First Search with Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Path length = ?

Max Q Length = ?

Number of state expansions = ?

SEARCH

Best-First Search with Visited List

Characterisation:

Sequence of state expansions = S-A-C-H-I-J-G

Path = S-C-H-I-J-G

Path length = 5

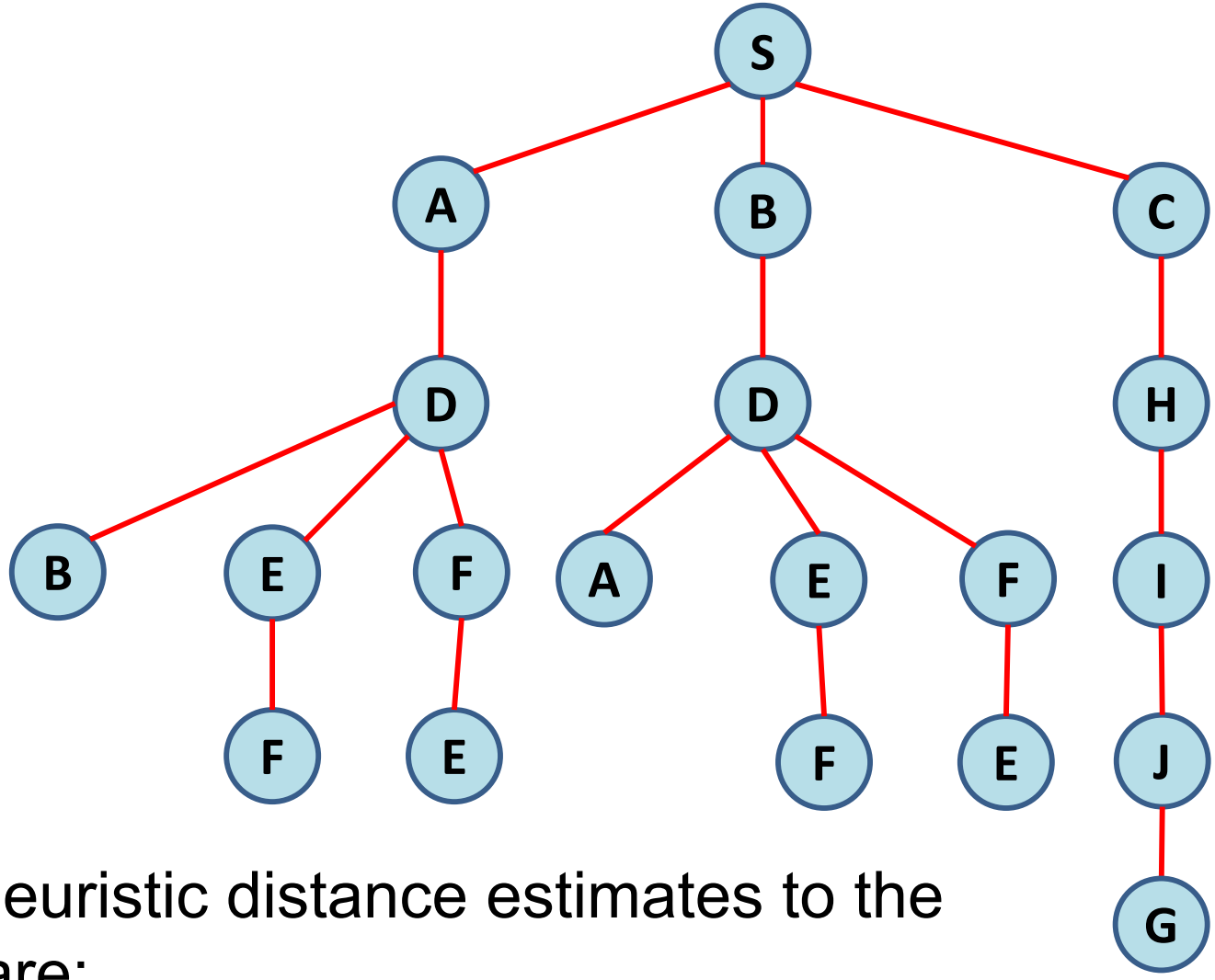
Max Q Length = 3

Number of state expansions = 7

Best-First Search with the Visited List expanded the least number of states (7) among the 3 algorithms being compared. Moreover, it used memory space efficiently. This is due to the accuracy of the heuristic used. In the programming assignment, we will see some variations in the performance when the heuristic function is varied.

TUTORIAL#1: SEARCH

Search Tree



The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5,
H = 3, I = 2, J = 1, G = 0

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Let’s apply the following algorithm to solve the problem:

Breadth-First Search with the Visited List

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

2) BFS: Pick **first** element of Q

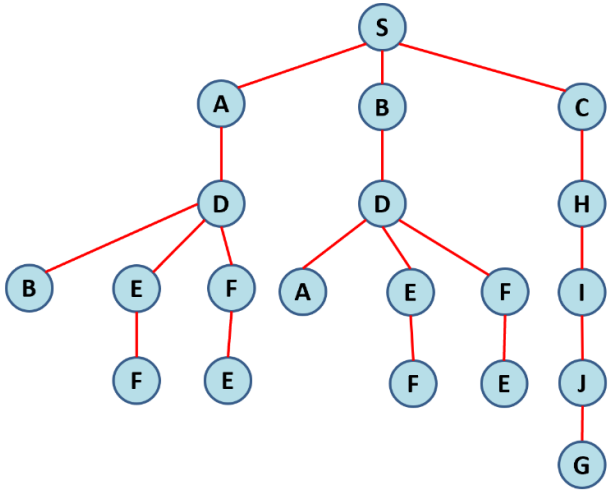
6) BFS: Add path extensions to **end** of Q.

SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS) (BS) (CS)	S,A,B,C

Search Tree



Pick **first** element of Q; Add path extensions to **end** of Q.

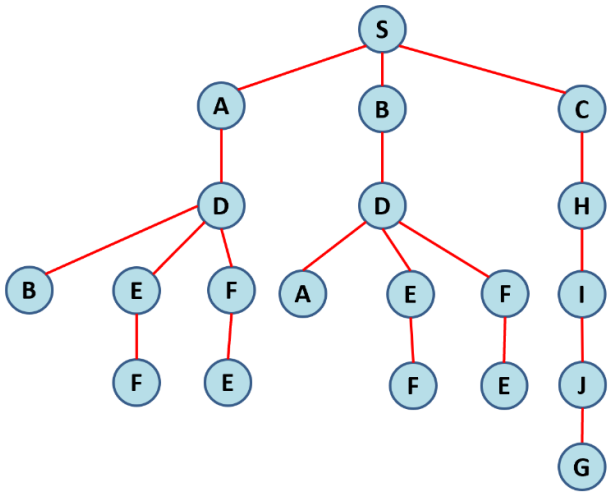
In **BFS** – nodes are pulled off the Q and inserted into the Q using a **queue**.

SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS) (BS) (CS)	S,A,B,C
2	(AS)	(BS) (CS) (DAS)	“, D

Search Tree



“ – means including all previous states visited

Pick **first** element of Q; Add path extensions to **end** of Q.

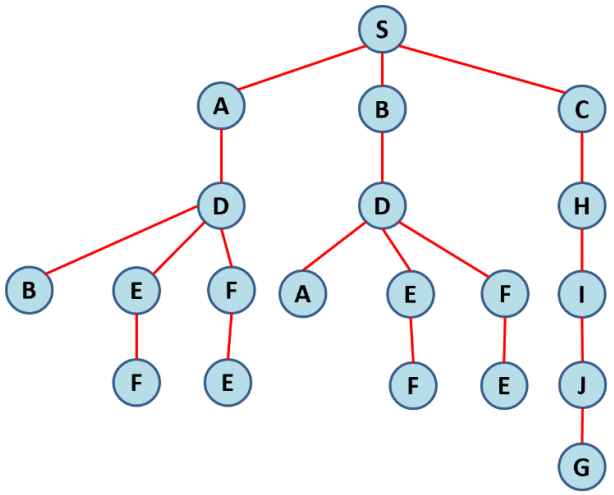
SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS) (BS) (CS)	S,A,B,C
2	(AS)	(BS) (CS) (DAS)	“, D
3	(BS)	(CS)(DAS)	“
4	(CS)	(DAS) (HCS)	“, H
5	(DAS)	(HCS) (EDAS)(FDAS)	“, E, F

continued…

Search Tree



“ – means including all previous states visited

Not extended to (DBS) anymore, as D is already in the visited list.

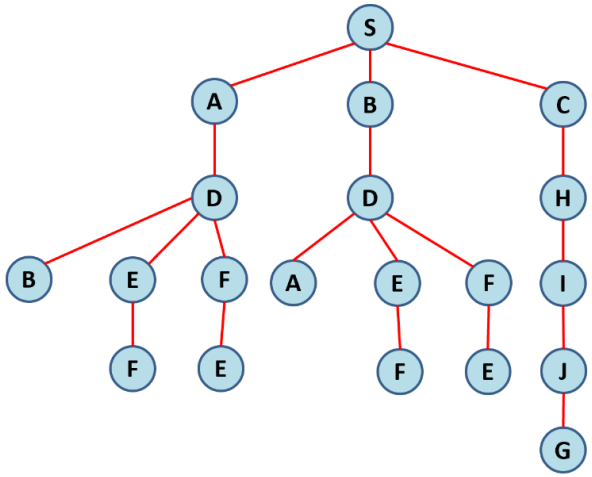
Pick **first** element of Q; Add path extensions to **end** of Q.

SEARCH

Breadth-First Search with Visited List

Step	Dequeued	Q	Visited
5	(DAS)	(HCS) (EDAS) (FDAS)	S,A,B,C,D,E,F,H
6	(HCS)	(EDAS) (FDAS) (IHCS)	“, I
7	(EDAS)	(FDAS) (IHCS)	“
8	(FDAS)	(IHCS)	“
9	(IHCS)	(JIHCS)	“, J
10	(JIHCS)	(GJIHCS)	“, G
11	(GJIHCS)		

Search Tree



Path = S-C-H-I-J-G

Pick **first** element of Q; Add path extensions to **end** of Q.

Breadth-First Search with Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Path length = ?

Max Q Length = ?

Number of state expansions = ?

Breadth-First Search with Visited List

Characterisation:

Sequence of state expansions = S-A-B-C-D-H-E-F-I-J-G

Path = S-C-H-I-J-G

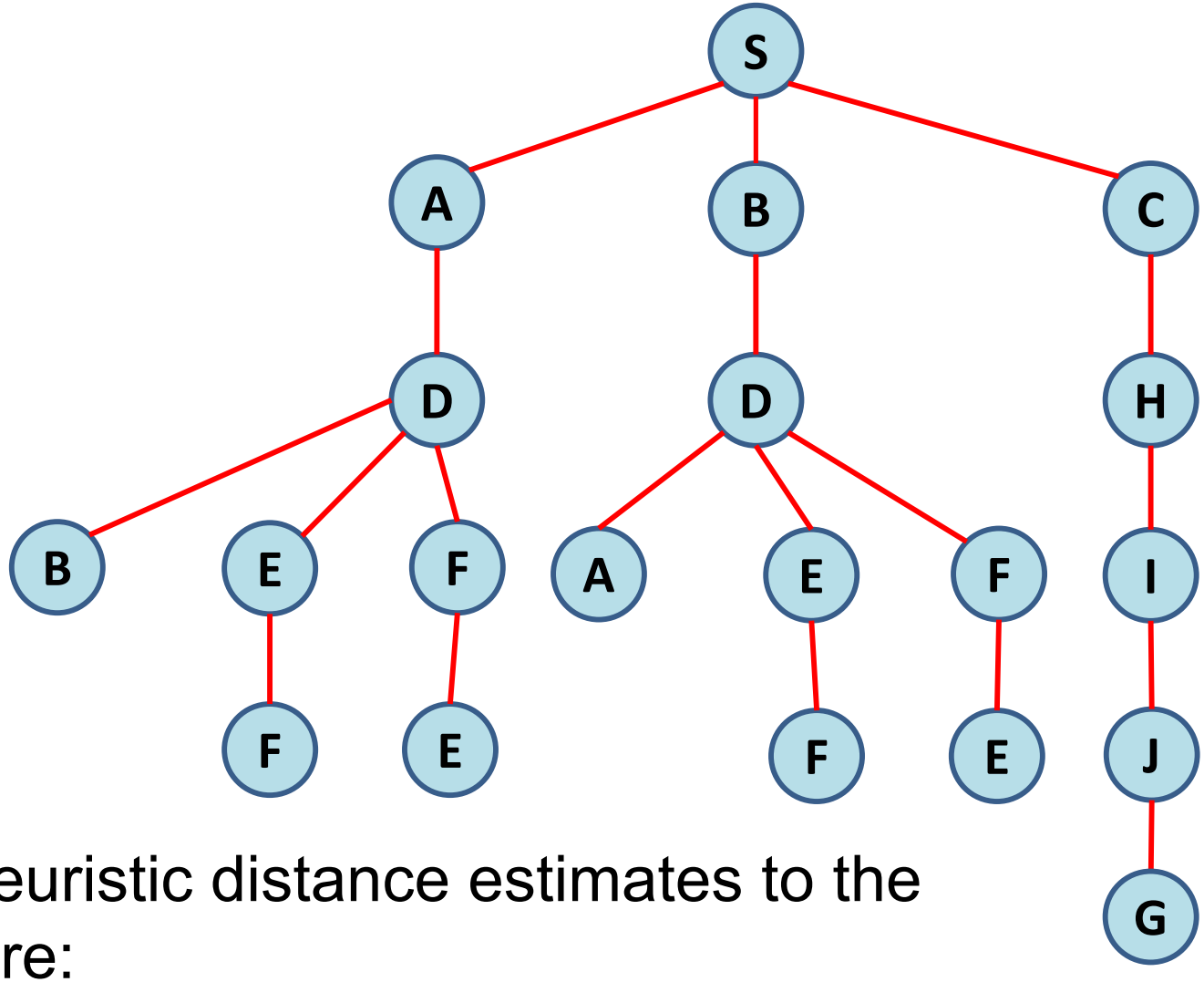
Path length = 5

Max Q Length = 3

Number of state expansions = 11

TUTORIAL#1: SEARCH

Search Tree



The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Let’s apply the following algorithm to solve the problem:

Depth-First Search with the Visited List

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

Guarantees that each state is reached only once.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

2) DFS: Pick **first** element of Q; Add path extensions to **front** of Q.

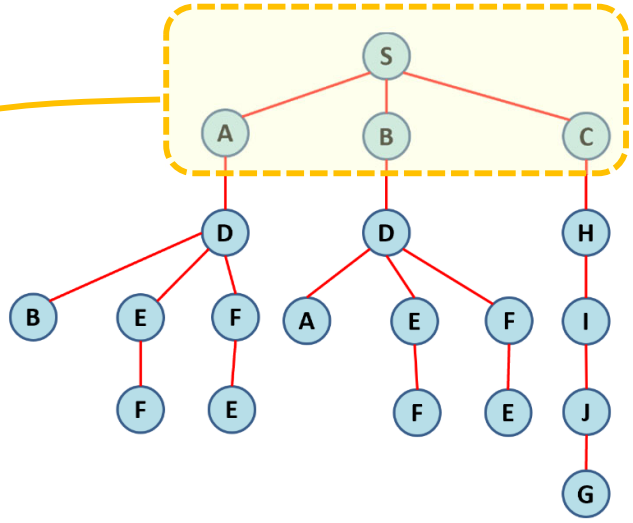
6) DFS: Nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS) (BS) (CS)	S,A,B,C

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

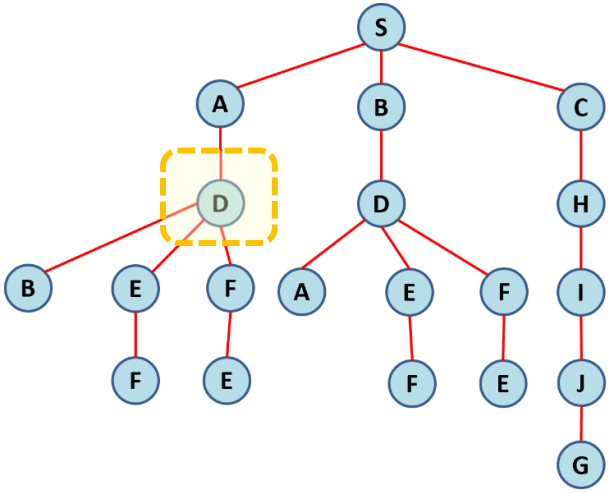
In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS) (BS) (CS)	S,A,B,C
2	(AS)	(DAS) (BS) (CS)	“, D

Search Tree



“ – means including all previous states visited

Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

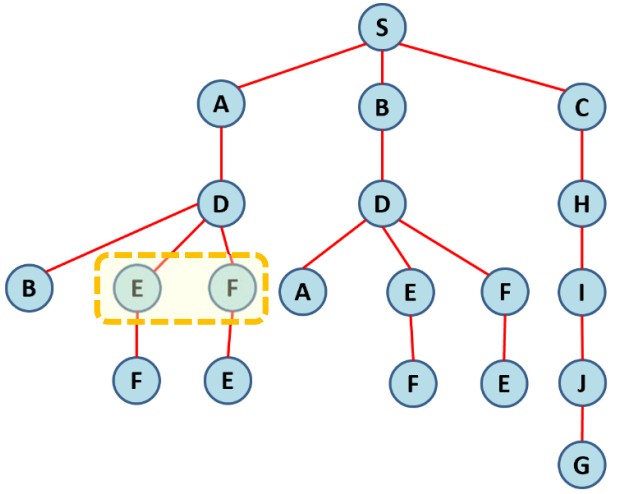
SEARCH

Depth-First with Visited List

Step	Dequeued	Q	Visited
0		(S)	S
1	(S)	(AS) (BS) (CS)	S,A,B,C
2	(AS)	(DAS) (BS) (CS)	“ D
3	(DAS)	(EDAS) (FDAS) (BS)(CS)	“, E, F
4	(EDAS)	(FDAS) (BS) (CS)	“
5	(FDAS)	(BS) (CS)	“

continued…

Search Tree



“ – means including all previous states visited

Not extended to (BDAS) anymore as state B has been visited already

Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

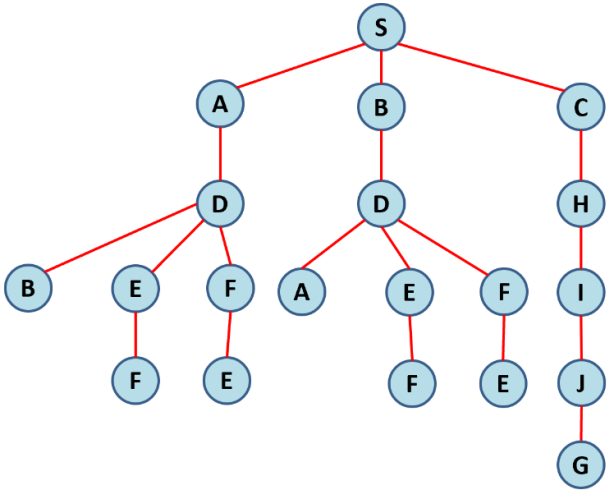
SEARCH

Depth-First with Visited List

Step	Dequeued	Q	Visited
5	(FDAS)	(BS) (CS)	S,A,B,C,D,E,F
6	(BS)	(CS)	“
7	(CS)	(HCS)	“, H
8	(HCS)	(IHCS)	“, I
9	(IHCS)	(JIHCS)	“, J
10	(JIHCS)	(GJIHCS)	“, G
11	(GJIHCS)		“

Path = S-C-H-I-J-G

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

Depth-First with Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Max Q Length = ?

Number of state expansions = ?

Depth-First with Visited List

Characterisation:

Sequence of state expansions = S-A-D-E-F-B-C-H-I-J-G

Path = S-C-H-I-J-G

Path length = 5

Max Q Length = 4

Number of state expansions = 11

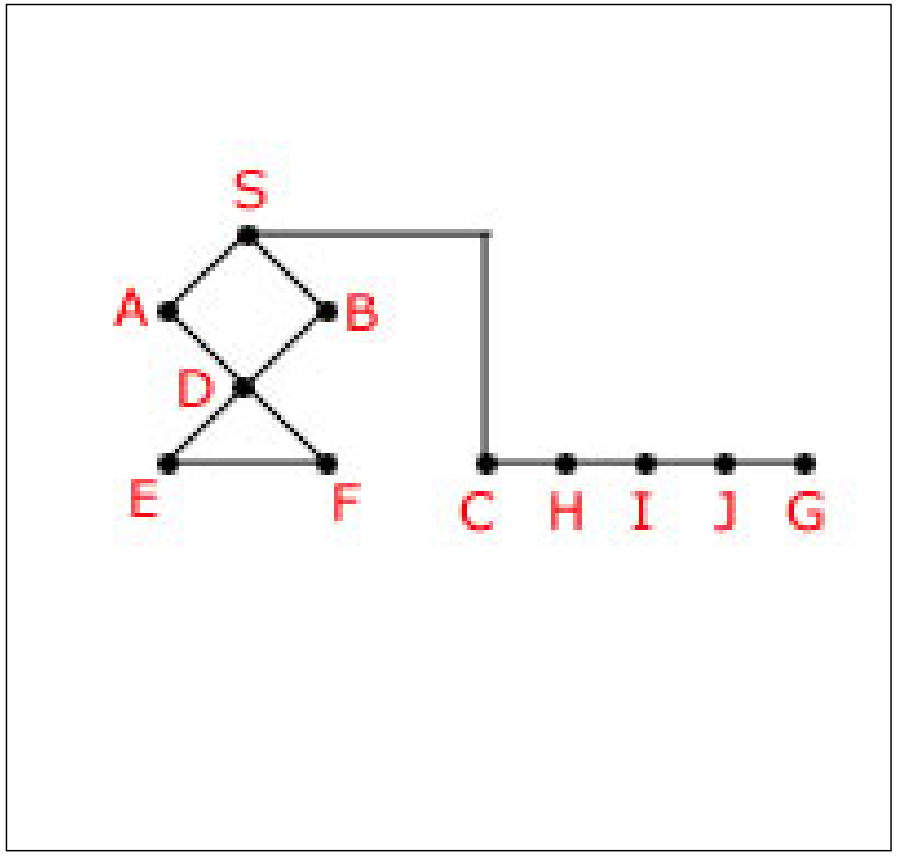
TUTORIAL#1: SEARCH

Problem definition

We are trying to find a path from **S** to **G** in the following graph:

S – start state
G – goal state

Connections (all undirected):
S: A B C
F: D E
A: S D
G: J
B: S D
H: C I
C: S H
I: H J
D: A B E F
J: I G
E: D F



The heuristic distance estimates to the goal are:

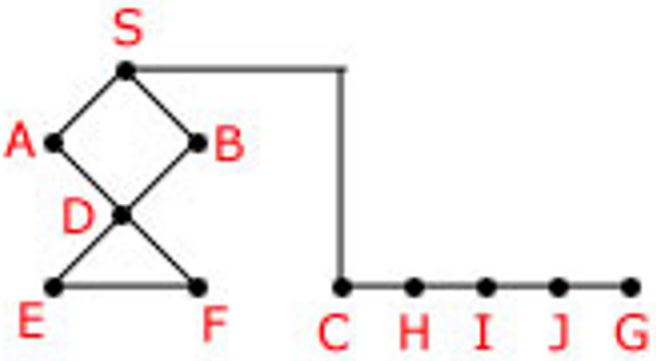
S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

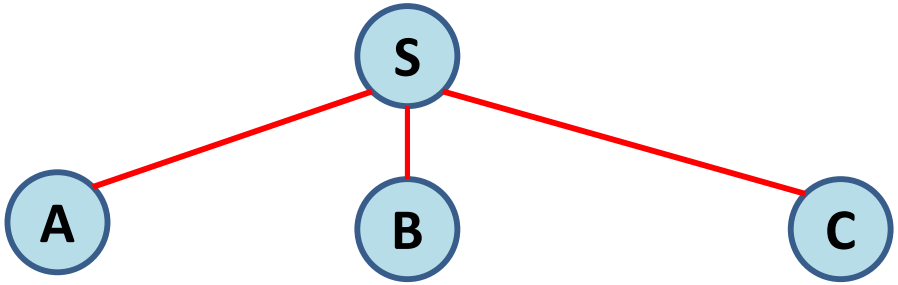
n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

TUTORIAL#1: SEARCH

Search Tree





Connections (all undirected):

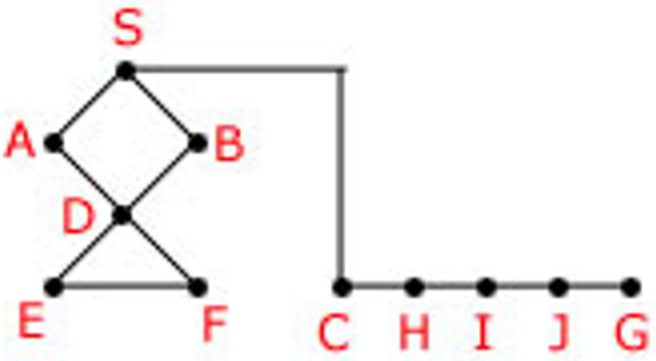
S: A B C
F: D E
A: S D
G: J
B: S D
H: C I
C: S H
I: H J
D: A B E F
J: I G
E: D F

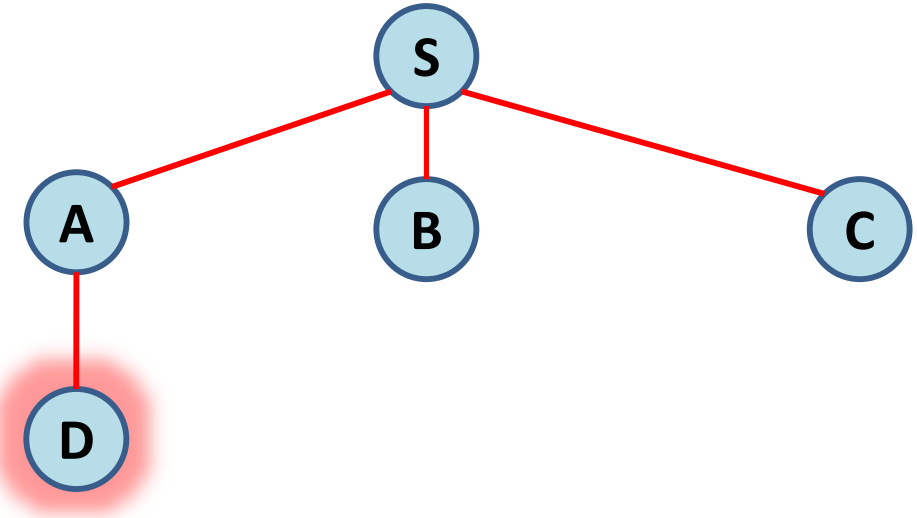
As our convention, for uniformity of solutions, especially in the exam, we are arranging the placement of the nodes alphabetically.

Remember to avoid generating local loops.

TUTORIAL#1: SEARCH

Search Tree





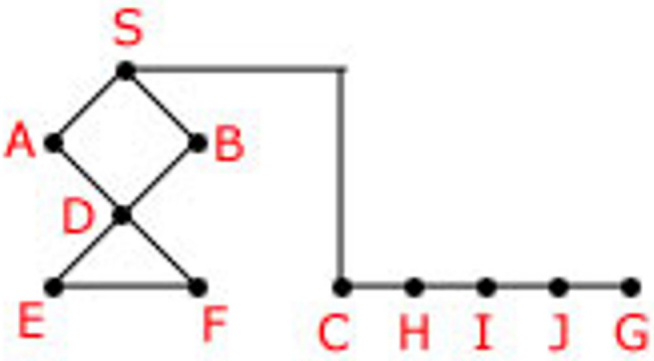
Connections (all undirected):

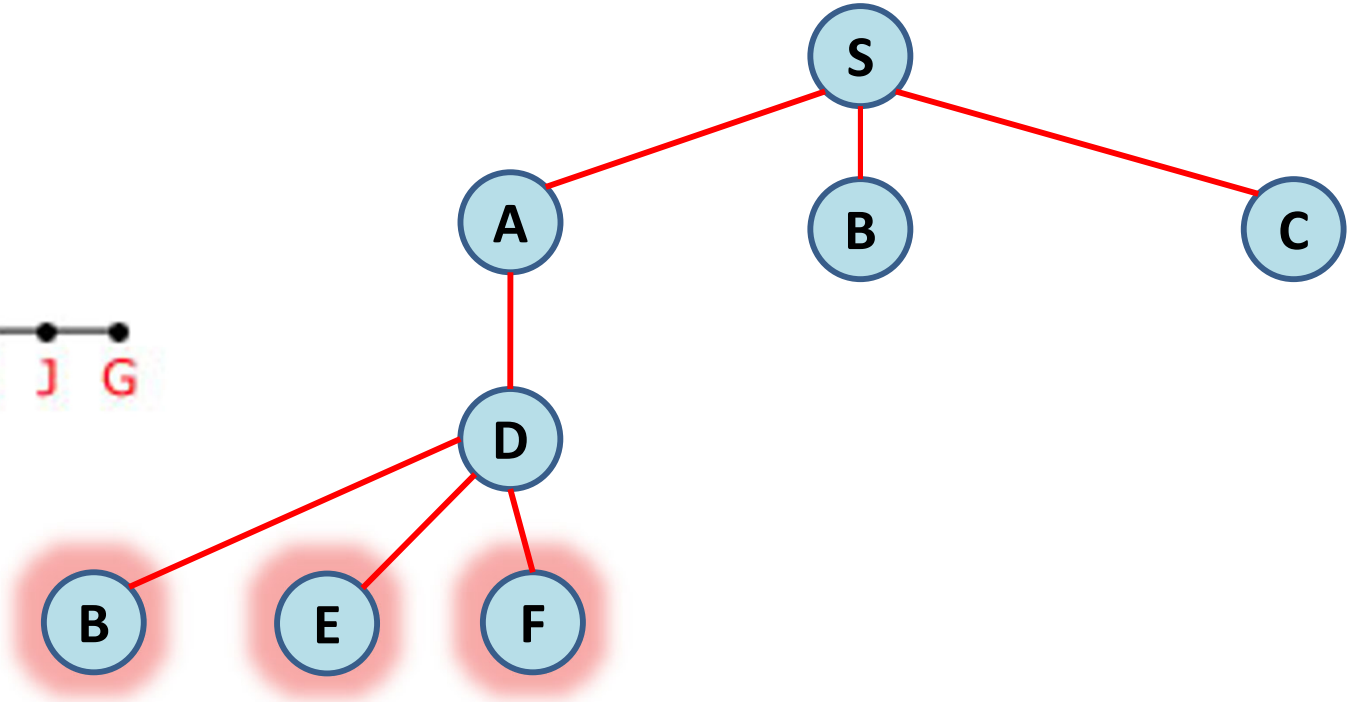
S: A B C
F: D E
A: S D
G: J
B: S D
H: C I
C: S H
I: H J
D: A B E F
J: I G
E: D F

Don’t go back to **S** to avoid a local loop.

TUTORIAL#1: SEARCH

Search Tree



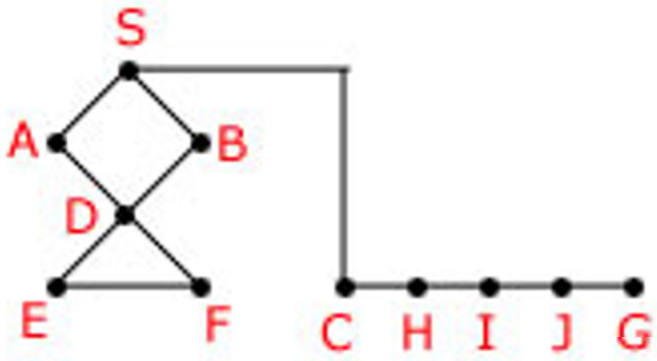


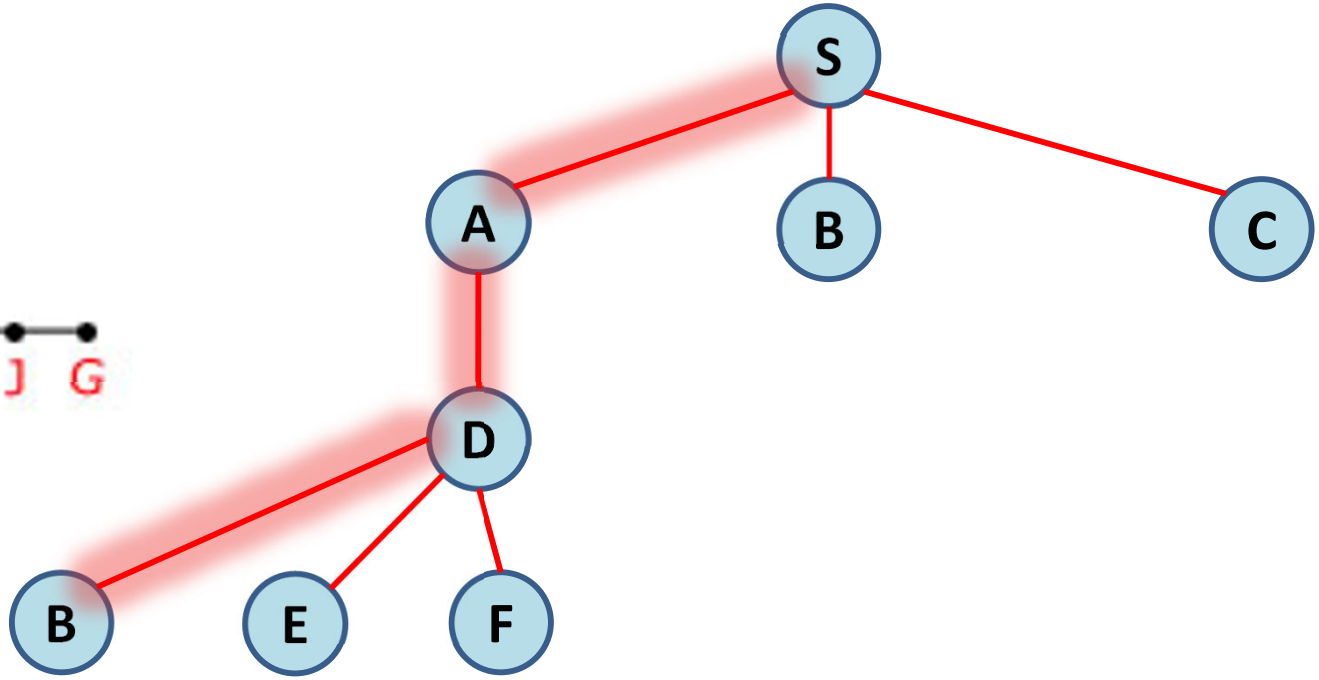
Connections (all undirected):

S: A B C
F: D E
A: S D
G: J
B: S D
H: C I
C: S H
I: H J
D: A B E F
J: I G
E: D F

TUTORIAL#1: SEARCH

Search Tree





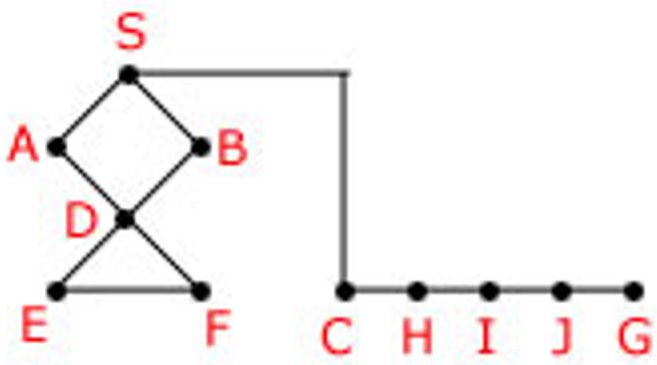
Connections (all undirected):

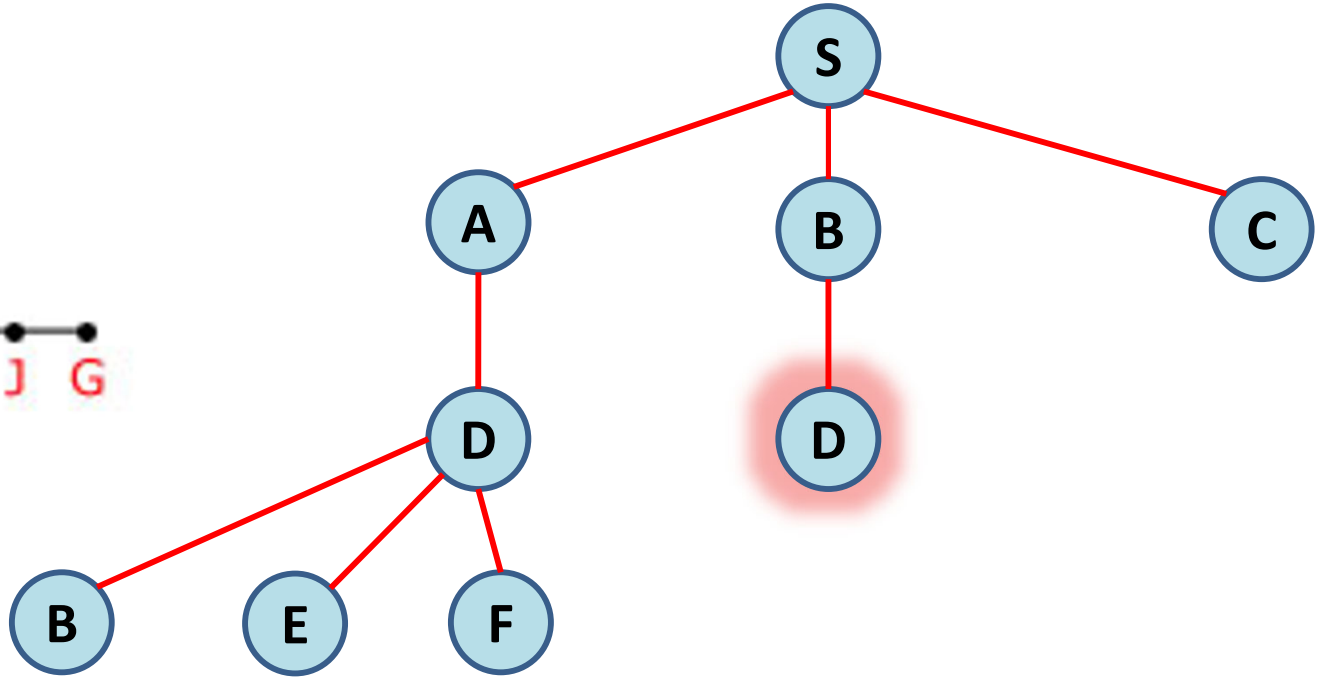
S: A B C
F: D E
A: S D
G: J
B: S D
H: C I
C: S H
I: H J
D: A B E F
J: I G
E: D F

We have nothing to extend along this path anymore.

TUTORIAL#1: SEARCH

Search Tree



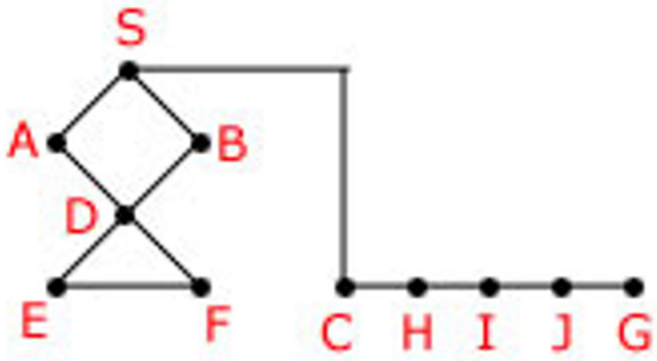


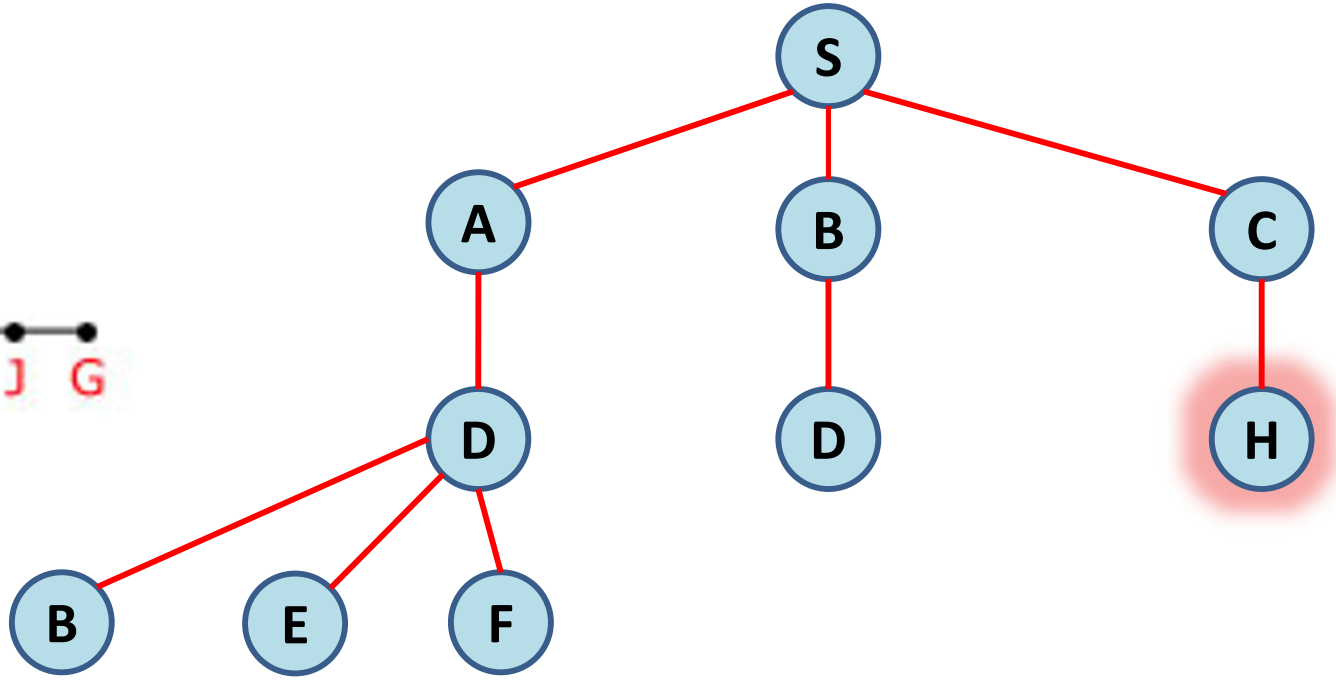
Connections (all undirected):

S: A B C
F: D E
A: S D
G: J
B: S D
H: C I
C: S H
I: H J
D: A B E F
J: I G
E: D F

TUTORIAL#1: SEARCH

Search Tree



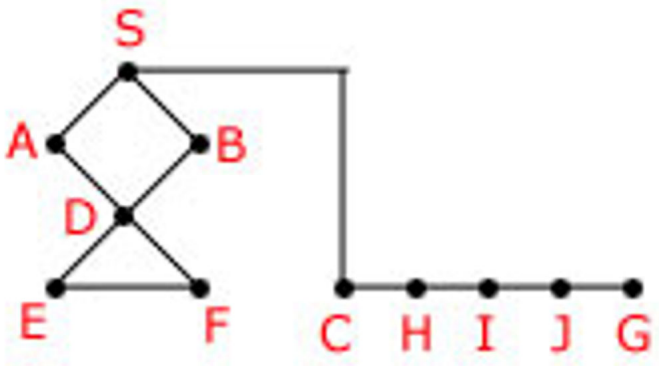


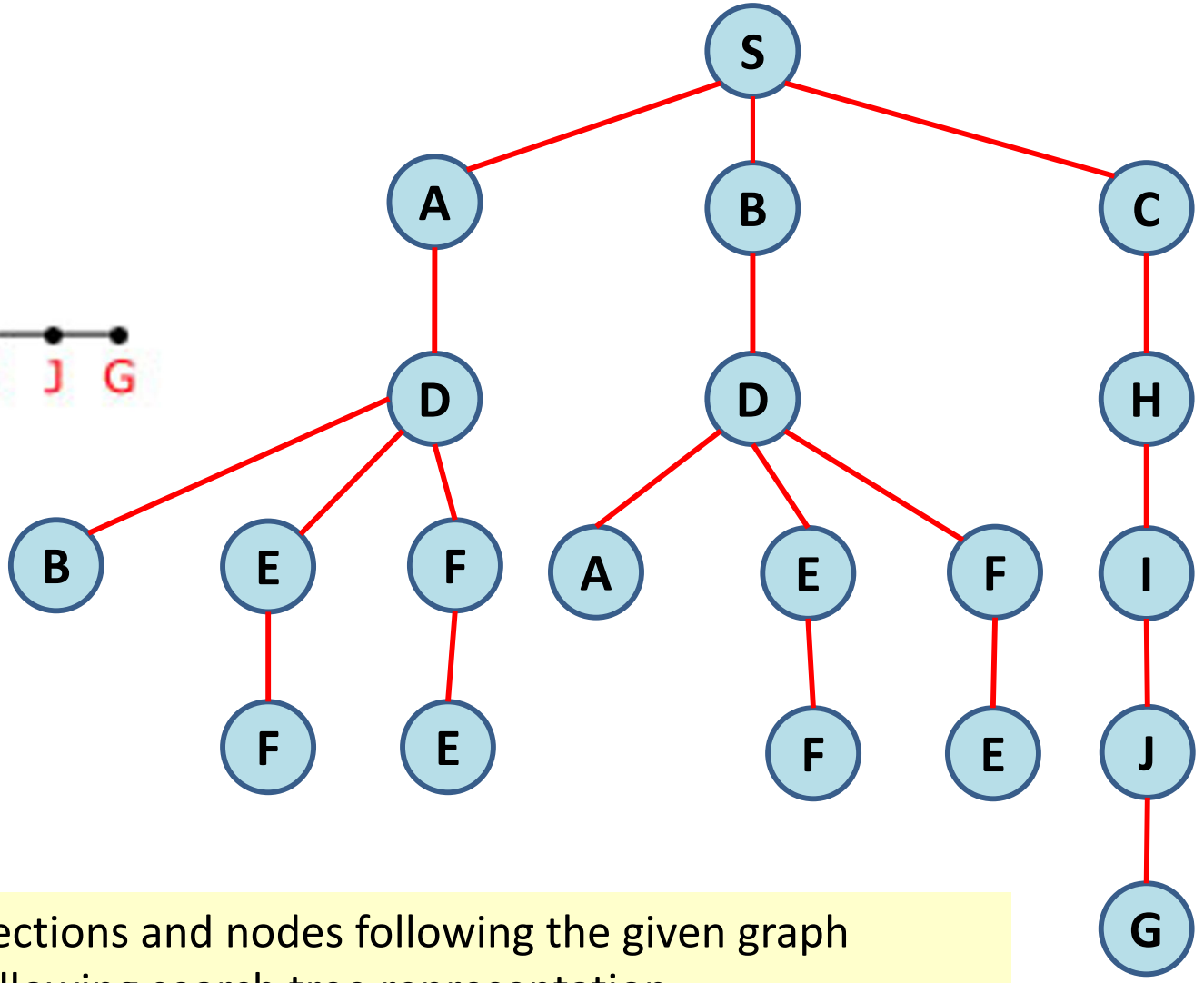
Connections (all undirected):

S: A B C
F: D E
A: S D
G: J
B: S D
H: C I
C: S H
I: H J
D: A B E F
J: I G
E: D F

TUTORIAL#1: SEARCH

Search Tree

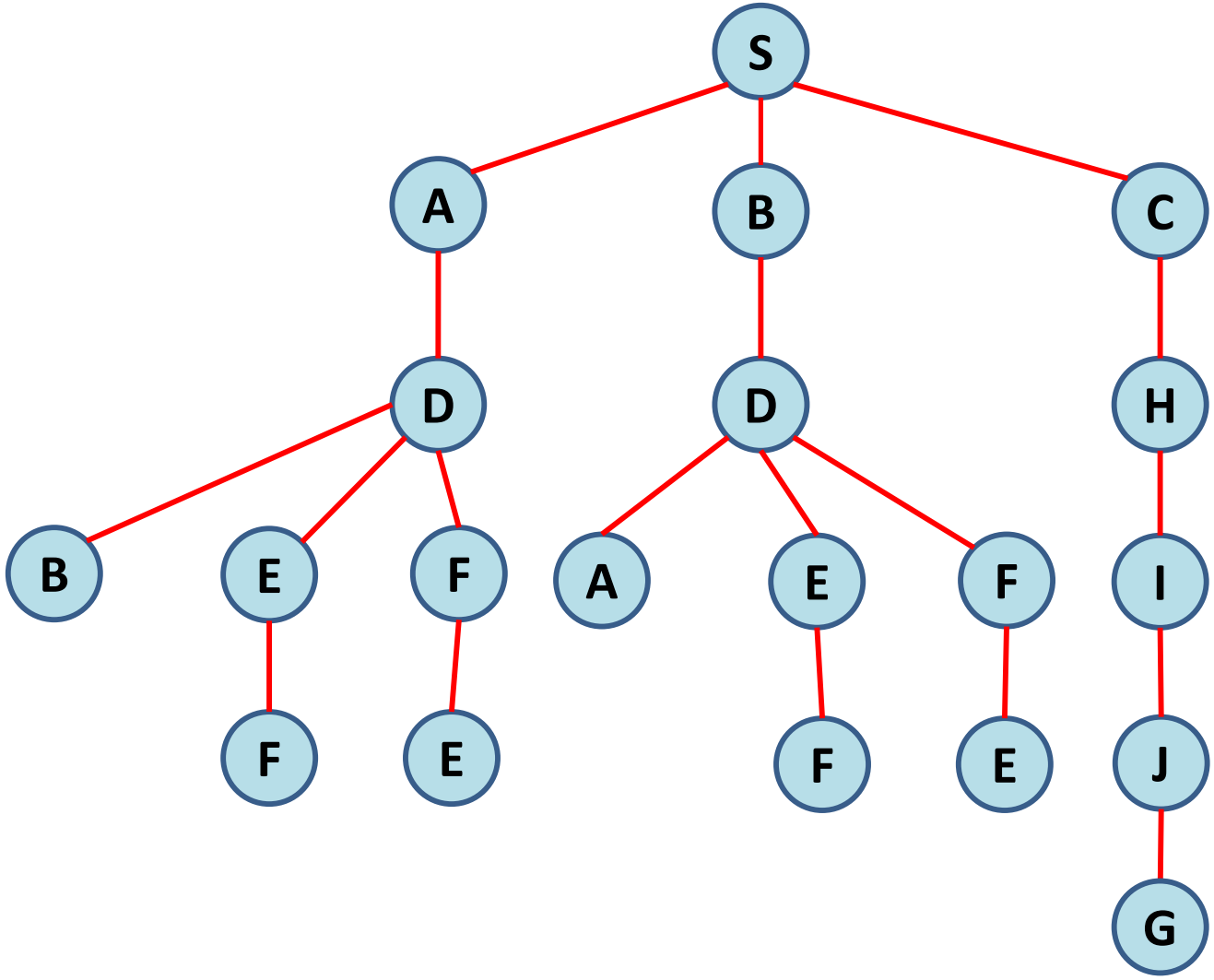




After putting all the connections and nodes following the given graph definition, we have the following search tree representation.

TUTORIAL#1: SEARCH

Search Tree



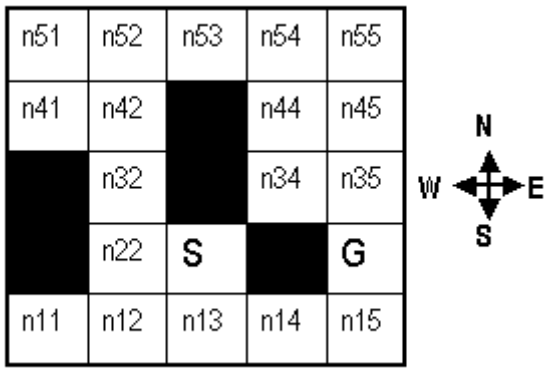
The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

Artificial Intelligence 159302
Tutorial

Maze Problem

Consider the maze shown in the figure below. **S** is the initial state and **G** is the goal state. Dark squares are blocked cells. All squares, except for the start and goal states, are labelled, with row and column numbers. For example, n54 is the cell in the fifth row, fourth column.



For all the questions in this problem:

- the graph states are the squares of the maze.
- there are links along the cardinal compass directions to neighboring squares (that are not blocked). That is, diagonal moves are not permitted, nor are moves into or across a black square.
- the successors of a state are to be generated in a FIXED order to be drawn from left to right, namely N, E, S, W.

We say that a state is **expanded** when a path to that state is pulled off of Q.

For Breadth-First search (with a Visited List), indicate the order in which states are expanded. Enter a sequence of state names starting with S and ending with G, e.g. `S n13 n22 ... G`.

n.h.reyes@massey.ac.nz

The Farmer, Goose, Fox and Grain Problem

- Learn how to perform abstractions in order to make the problem solvable using Search
- States are generated on-the-fly; that is, neither the graph, nor the tree is given beforehand

n.h.reyes@massey.ac.nz

How to approach this problem using AI?

The farmer-fox-goose-grain problem.

A farmer has to cross a river with his fox, goose and grain. However, his little boat can only accommodate himself and one of his possessions. Therefore, in order to bring all of them across the river, he needs to cross it several times. During planning, he is cautious of two unwanted situations: an unguarded fox will eat the goose, and an unguarded goose will eat the grain.

Image: http://cairnarvon.rotahall.org/2010/02/28/search-trees-are-easy/

How can he cross the river without losing his possessions?

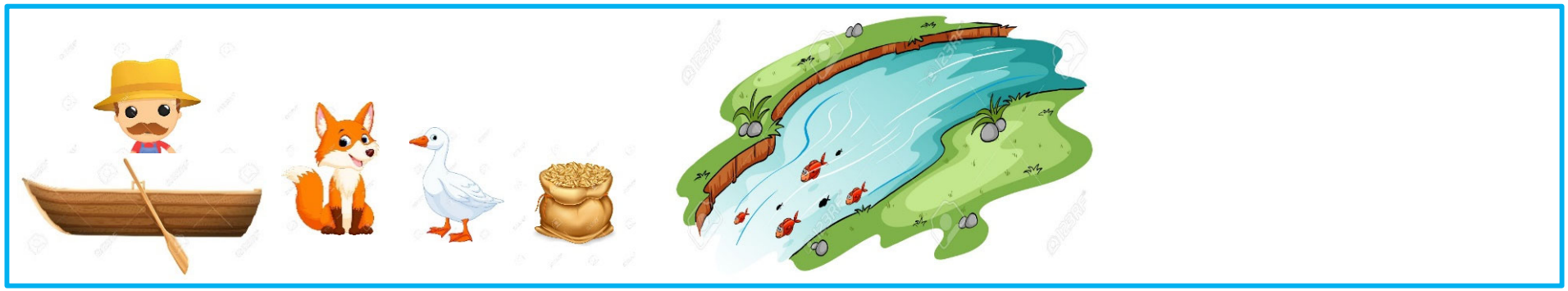
How to represent the problem as a Search problem?

What are the constraints to consider in this problem?

How many possible arrangements (states) are there? How many valid states?

Preconditions

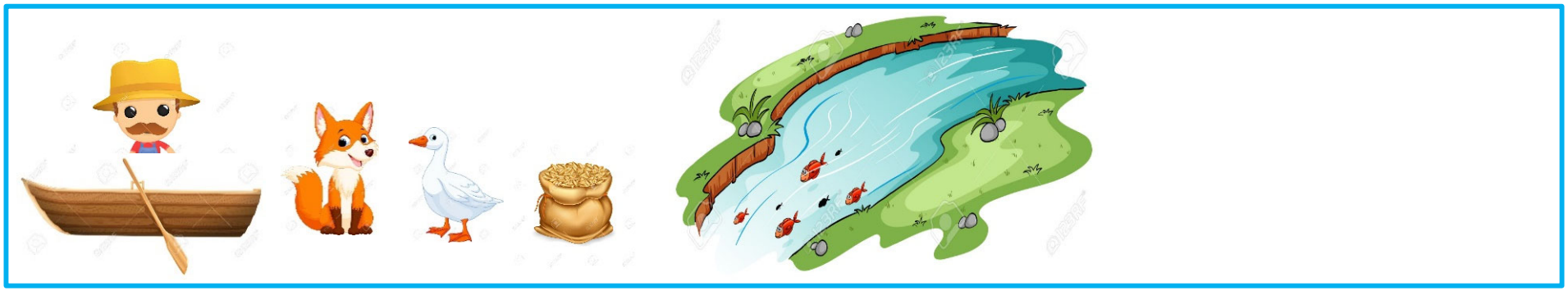
- The **fox** and the **goose** should not be in the same side of the river, not unless the farmer is with them.
- The **goose** and the **grain** should not be in the same side of the river, not unless the farmer is with them.



Left river bank river right river bank

Rules

1. Farmer can travel alone to the right river bank.
2. Farmer can travel alone to the left river bank.
3. Farmer can bring one of his possessions to the right river bank, provided that all preconditions are met.
4. Farmer can bring one of his possessions to the left river bank, provided that all preconditions are met.
5. The farmer should cross the river, every time he makes a move.



Left river bank river right river bank

Representation

How to represent the problem as a Search problem?

The objective is to move all of the farmer's possessions to the right side of the river bank.

What are the **States? Actions? Goal test?**

Initial state of the problem:

Left river bank

river

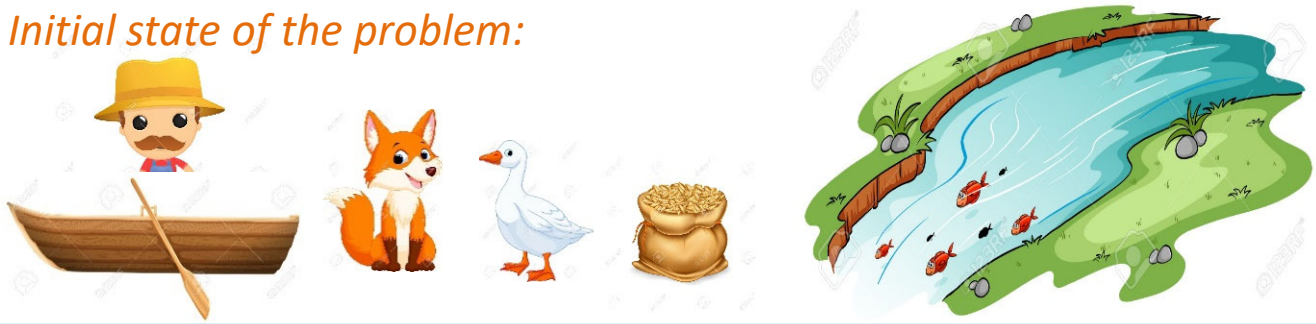
right river bank

States

The **state** is a tuple of **4 booleans**, representing the position of the farmer, goose, fox and grain, relative to the river.

Fa, Fo, Go, Gr

Initial state of the problem:



Left river bank river right river bank

Notation: When all 4 boolean variables are set to false, that means the farmer and all of his possessions are in the left side of the river bank.

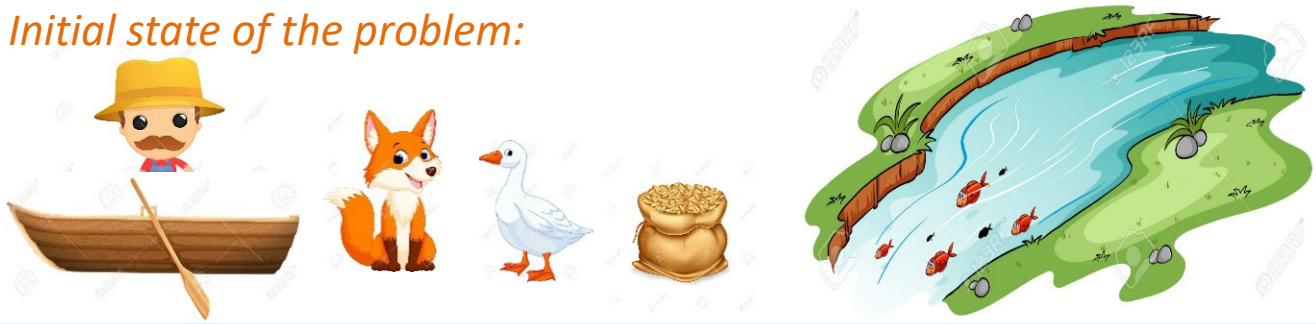
Fa=false, Fo=false, Go=false, Gr=false → 0, 0, 0, 0

States

The **state** is a tuple of **4 booleans**, representing the position of the farmer, goose, fox and grain, relative to the river.

Fa, Fo, Go, Gr

Initial state of the problem:



Left river bank river right river bank

Notation: When all 4 boolean variables are set to false, that means the farmer and all of his possessions are in the left side of the river bank.

Fa=false, Fo=false, Go=false, Gr=false → 0, 0, 0, 0

In this hand-simulation, to make the notations more intuitive, we can write the same state as follows:

< Fa, Fo, Go, Gr | >

The bar indicates the position of the river, relative to all characters.

Actions/Operators

There are **4 operators:** the farmer crosses alone, or with one of his three possessions. In each case all the preconditions must be met.

e.g. the farmer travels to the right river bank, the precondition is goose not equal fox and goose not equal grain.

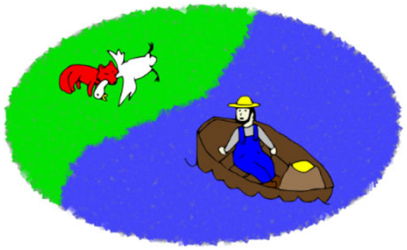
Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.

Invalid States

Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.



The following are invalid states that we should avoid:

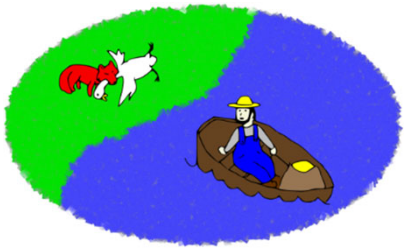
When the farmer travels alone to one of the river banks:

Fo, Go, Gr | **Fa**	The goose eats the grain, or the fox eats the goose first.
Fa | Fo, Go, Gr	The goose eats the grain, or the fox eats the goose first.

Invalid States

Actions/Operators/Moves:

1. Farmer travels alone to the left river bank.
2. Farmer travels alone to the right river bank.
3. Farmer travels with one of his possessions to the left river bank.
4. Farmer travels with one of his possessions to the right river bank.



The following are invalid states that we should avoid:

When the farmer travels alone to one of the river banks:

Fo, Go, Gr | **Fa**	The goose eats the grain, or the fox eats the goose first.
Fa | Fo, Go, Gr	The goose eats the grain, or the fox eats the goose first.

When the farmer travels with one of his possessions to one of the river banks:

Fo, Go | **Fa**, Gr	the fox eats the goose.
Fa, Gr | Fo, Go	the fox eats the goose.
Go, Gr | **Fa**, Fo	the goose eats the grain.
Fa, Fo | Go, Gr	the goose eats the grain.

There are (2^4)=**16 states** in total, but **6 are invalid**. Therefore, there are only **10 valid** states.

Goal Test

How to check if the algorithm has succeeded already?

Goal test:

All Booleans are true. That means all possessions, including the farmer are already on the right river bank.

i.e. farmer (**Fa**) == fox (**Fo**) == goose (**Go**) == grain (**Gr**) == true

Using our notation for the hand-simulation:

| Fa, Fo, Go, Gr

*To keep track of the characters in the puzzle more easily, enumerate them in alphabetical order

Exercise

Solve the problem using Depth-First Search

In your solution, consider the following sequence of moves:

1. Farmer travels alone to the other side of the river.
2. Farmer takes fox to the other side of the river.
3. Farmer takes goose to the other side of the river.
4. Farmer takes grain to the other side of the river.

Initial state of the problem:

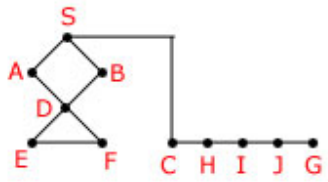
Left river bank

river

right river bank

Tutorial #4: More Search (the Sequel)

We are trying to find a path from S to G in the following graph:




```
Connections (all undirected):
 S: A B C            F: D E
 A: S D              G: J
 B: S D              H: C I
 C: S H              I: H J
 D: A B E F          J: I G
 E: D F
```

The path length of all the links is 1, except for S-C which is of length 10.

The heuristic distance estimates to the goal are:

```
S = 10, A = 5, B = 11, C = 1, D = 7, E = 6,
F = 5, H = 3, I = 2, J = 1, G = 0
```

Assume that the implementation of Uniform Cost Search and A* places newly extended paths at the **front** of Q, with the new paths ordered by alphabetical order of the head state. The order on Q of paths will be used to resolve any ties in choosing the "best" path to pull off of Q, the path that appears **closer to the front** of Q is chosen.

Write the sequence of **states expanded** by the specified search methods in our implementation. In some searches, states may be repeated in the sequence. When using an expanded list, if the state pulled off Q is found in the expanded list, you should not include it in your answer (since its descendants are not generated). You will save yourself a LOT of confusion if you first convert the graph above into the appropriate search tree.

1. **Uniform Cost (NOT using Expanded List)**. This is tricky, please pay careful attention to the rule for settling ties given above. In each slot, enter a state name, including S and G:

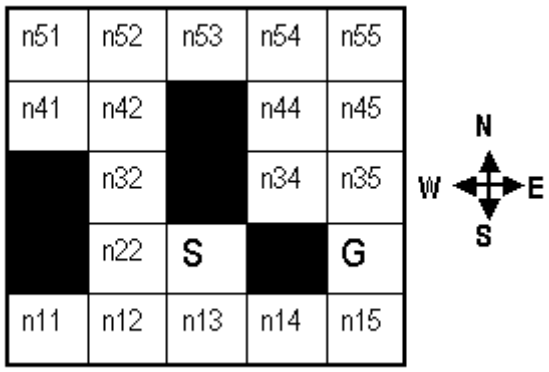
2. **Uniform Cost (using strict Expanded List)**. In each slot, enter a state name, including S and G:

3. **A* (using strict Expanded List)**. For this answer, enter into each box both the state and the value of the f-value for the corresponding node. For example - S 10:

Source: MIT OpenCourseWare

Maze

Consider the maze shown in the figure below. **S** is the initial state and **G** is the goal state. Dark squares are blocked cells. All squares, except for the start and goal states, are labeled, with row and column numbers. For example, n54 is the cell in the fifth row, fourth column.



- the graph states are the squares of the maze.
- there are links along the cardinal compass directions to neighbouring squares (that are not blocked). That is, diagonal moves are not permitted, nor are moves into or across a black square.
- the successors of a state are to be generated in a FIXED order to be drawn from left to right, namely N, E, S, W.
- We say that a state is **expanded** when a path to that state is pulled off of Q.

Find a path from **S** to **G** by applying the **A* algorithm with the Strict Expanded List, and the Manhattan distance heuristic**, indicate the order in which states are expanded. Enter a sequence of state names starting with S and ending with G, e.g. `S n13 n22 ... G`.

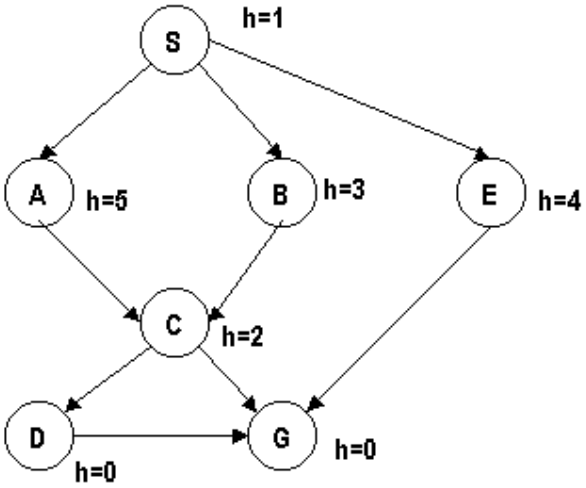
Tutorial #6: Optimal Searches

I. True or False

1. The use of a Visited list enhances the performance of Uniform Cost search without affecting its correctness.
2. The Dynamic Programming optimality principle says that all we ever need is to find the single shortest path to any state; any other paths can never be part of the best path from the start to the goal.
3. The use of an Expanded list enhances the performance of Uniform Cost search without affecting its correctness.
4. Uniform Cost (without an Expanded list) looks at every possible path to the goal so that it can be sure it has found the best one.
5. Admissible heuristics for path searching always use straight-line distance to the goal.
6. A* will generally be more efficient than Uniform Cost search since, in the worst case, an admissible heuristic gives no additional help but it never hurts.
7. To guarantee that A* finds optimal paths (when using a strict Expanded list) we must use a consistent heuristic function (as defined in the notes).

II. A*

Consider the directed graph shown below with start state S and goal state G. The figure shows the heuristic values at each state, i.e., the **estimates** of the length of a path from that state to the goal.



Unfortunately we do not know the link costs (lengths), and we really would like to know them. However and fortunately:

- We know that the given heuristic is admissible.
- There is a record of the expansion produced by A*, as listed below, where each list represents the A* evaluation of each path followed by the states in the path.
- We assume here that Q is kept sorted by the value of the search nodes.
- There is no Expanded list in use, but after adding a new node to Q, we go through and discard any node where a shorter path to the corresponding state exists in Q.

Step	Path expanded	Paths Added to Q	Sorted Resulting Q
1.	(1 S)	(6 A S) (5 B S) (8 E S)	(5 B S) (6 A S) (8 E S)
2.	(5 B S)	(7 C B S)	(6 A S) (7 C B S) (8 E S)
3.	(6 A S)	(6 C A S)	(6 C A S) (8 E S)
4.	(6 C A S)	(5 D C A S) (10 G C A S)	(5 D C A S) (8 E S) (10 G C A S)
5.	(5 D C A S)	(8 G D C A S)	(8 E S) (8 G D C A S)
6.	(8 E S)	(12 G E S)	(8 G D C A S)
7.	(8 G D C A S)	--	--

1. Fill in the link costs on the graph for all the connecting links.
   Hint 0: Note that each path added to Q will let you fill in the cost of exactly one link in the graph.

   S-A
   S-B
   S-E
   A-C
   B-C
   C-D
   C-G
   D-G
   E-G

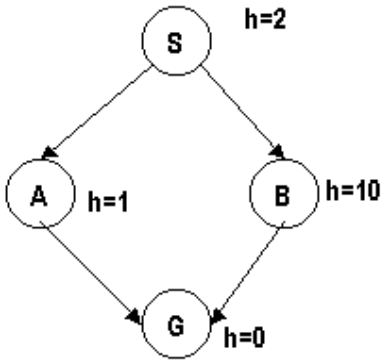
2. Why was the path (7 C B S) removed from the resulting sorted Q at step 3?
   a) No answer
   b) It forms a loop
   c) There are no descendants of C
   d) A shorter path to C was found.
   e) C is on the Visited List
   f) C is on the Expanded List

3. Why couldn't A* finish its search at steps 4 or 5?
   a) No answer
   b) No path has reached the goal
   c) The paths to the goal are not unambiguously the best ones in Q
   d) There exists an unambiguously better path in Q

4. A friend of yours claims that: In general, if we are given an arbitrary search problem, such that:
    a) We know that the given heuristic estimate is admissible (the actual distance is greater that the value of the heuristic estimate).
    b) There is a record of the complete A* search episode, like that in the table above.

    Then, the costs of **all** the links in this arbitrary search problem can be uniquely determined.

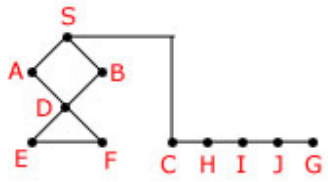
    You think your friend is wrong. To convince him, assign the link costs in the following simple graph (the search is from S to G) to provide a counterexample:



    Enter your answer as a sequence of four numbers for the link costs of S-A, A-G, S-B, and B-G (in that order): Many answers are possible so we will only show you one answer as "the correct answer", but we will accept any combination that proves the point.

Tutorial #4: More Search (the Sequel)

We are trying to find a path from S to G in the following graph:




```
Connections (all undirected):
 S: A B C          F: D E
 A: S D            G: J
 B: S D            H: C I
 C: S H            I: H J
 D: A B E F        J: I G
 E: D F
```

The path length of all the links is 1, except for S-C which is of length 10.

The heuristic distance estimates to the goal are:

```
S = 10, A = 5, B = 11, C = 1, D = 7, E = 6,
F = 5, H = 3, I = 2, J = 1, G = 0
```

Assume that the implementation of Uniform Cost Search and A* places newly extended paths at the **front** of Q, with the new paths ordered by alphabetical order of the head state. The order on Q of paths will be used to resolve any ties in choosing the "best" path to pull off of Q, the path that appears **closer to the front** of Q is chosen.

Write the sequence of **states expanded** by the specified search methods in our implementation. In some searches, states may be repeated in the sequence. When using an expanded list, if the state pulled off Q is found in the expanded list, you should not include it in your answer (since its descendants are not generated). You will save yourself a LOT of confusion if you first convert the graph above into the appropriate search tree.

1. **Uniform Cost (NOT using Expanded List)**. This is tricky, please pay careful attention to the rule for settling ties given above. In each slot, enter a state name, including S and G:

S	A	B	D	D	B	E	F	A	E	F	E	F	E	F	C	H	I	J	G

2. **Uniform Cost (using strict Expanded List)**. In each slot, enter a state name, including S and G:

S	A	B	D	E	F	C	H	I	J	G

3. **A* (using strict Expanded List)**. For this answer, enter into each box both the state and the value of the f-value for the corresponding node. For example - S 10:

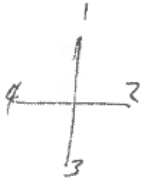
S10	A6	D9	F8	E9	C11	B12	H14	I14	J14	G14

n.h.reyes@massey.ac.nz

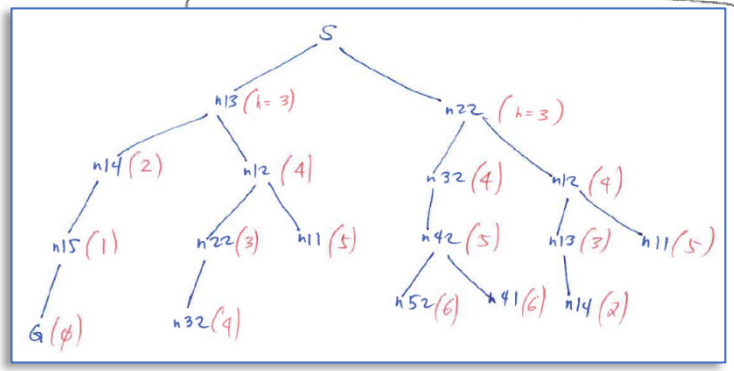
MAZE PROBLEM

A* with Strict Expanded list (4-connected Gridworld)

h = Manhattan distance



Step	Deq	Enq	Expanded
0		~~(25)~~	
1	(2 5)	~~(4 n12, 5)~~ (4 n22, 5)	5
2	(4 n13, 5)	~~(4 n14, n15, 5)~~ ~~(6 n12, n13, 5)~~ (4 n22, 5)	5, n13
3	(4 n14, n13, 5)	~~(4 n15, n14, n13, 5)~~ ~~(6 n12, n13, 5)~~ (4, n22, 5)	5, n13, n14,
4	(4 n15, n14, n13, 5)	(4G, n15, n14, n13, 5) ~~(6 n12, n13, 5)~~ (4, n22, 5)	", n15
5	(4G, n15, n14, n13, 5)	path to Goal is found!	
6			



n51	n52	n53	n54	n55
n41	n42		n44	n45
	n32		n34	n35
	n22	S		G
n11	n12	n13	n14	n15

N
W E
S

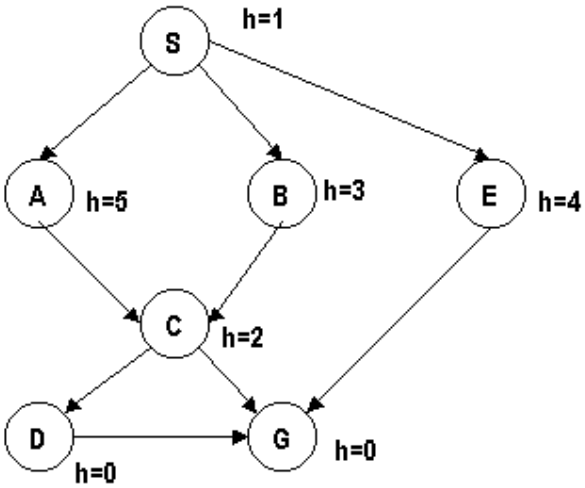
Successor generator = {N, E, S, W}

I. True or False

1. The use of a Visited list enhances the performance of Uniform Cost search without affecting its correctness. - F
2. The Dynamic Programming optimality principle says that all we ever need is to find the single shortest path to any state; any other paths can never be part of the best path from the start to the goal.- T
3. The use of an Expanded list enhances the performance of Uniform Cost search without affecting its correctness. - T
4. Uniform Cost (without an Expanded list) looks at every possible path to the goal so that it can be sure it has found the best one.- F
5. Admissible heuristics for path searching always use straight-line distance to the goal.- F
6. A* will generally be more efficient than Uniform Cost search since, in the worst case, an admissible heuristic gives no additional help but it never hurts.- T
7. To guarantee that A* finds optimal paths (when using a strict Expanded list) we must use a consistent heuristic function (as defined in the notes).- T

II. A*

Consider the directed graph shown below with start state S and goal state G. The figure shows the heuristic values at each state, i.e., the **estimates** of the length of a path from that state to the goal.



Unfortunately we do not know the link costs (lengths), and we really would like to know them. However and fortunately:

- We know that the given heuristic is admissible.
- There is a record of the expansion produced by A*, as listed below, where each list represents the A* evaluation of each path followed by the states in the path.
- We assume here that Q is kept sorted by the value of the search nodes.

- There is no Expanded list in use, but after adding a new node to Q, we go through and discard any node where a shorter path to the corresponding state exists in Q.

Step	Path expanded	Paths Added to Q	Sorted Resulting Q
1.	(1 S)	(6 A S) (5 B S) (8 E S)	(5 B S) (6 A S) (8 E S)
2.	(5 B S)	(7 C B S)	(6 A S) (7 C B S) (8 E S)
3.	(6 A S)	(6 C A S)	(6 C A S) (8 E S)
4.	(6 C A S)	(5 D C A S) (10 G C A S)	(5 D C A S) (8 E S) (10 G C A S)
5.	(5 D C A S)	(8 G D C A S)	(8 E S) (8 G D C A S)
6.	(8 E S)	(12 G E S)	(8 G D C A S)
7.	(8 G D C A S)	--	--

1. Fill in the link costs on the graph for all the connecting links.
   Hint 0: Note that each path added to Q will let you fill in the cost of exactly one link in the graph.

S-A [ ] 1
S-B [ ] 2
S-E [ ] 4
A-C [ ] 3
B-C [ ] 3
C-D [ ] 1
C-G [ ] 6
D-G [ ] 3
E-G [ ] 8

2. Why was the path (7 C B S) removed from the resulting sorted Q at step 3?
   a) No answer
   b) It forms a loop
   c) There are no descendants of C
   d) A shorter path to C was found.
   e) C is on the Visited List
   f) C is on the Expanded List

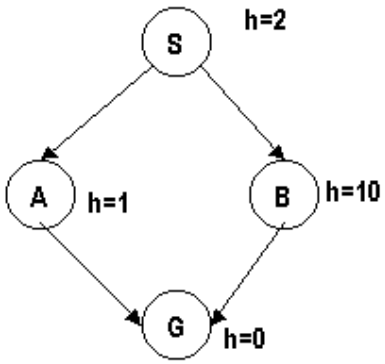
3. Why couldn't A* finish its search at steps 4 or 5?
   a) No answer
   b) No path has reached the goal
   c) The paths to the goal are not unambiguously the best ones in Q

d) There exists an unambiguously better path in Q

4. A friend of yours claims that: In general, if we are given an arbitrary search problem, such that:
   a) We know that the given heuristic estimate is admissible (the actual distance is greater that the value of the heuristic estimate).
   b) There is a record of the complete A* search episode, like that in the table above.

   Then, the costs of **all** the links in this arbitrary search problem can be uniquely determined.

   You think your friend is wrong. To convince him, assign the link costs in the following simple graph (the search is from S to G) to provide a counterexample:



   Enter your answer as a sequence of four numbers for the link costs of S-A, A-G, S-B, and B-G (in that order): _____ Many answers are possible so we will only show you one answer as "the correct answer", but we will accept any combination that proves the point.

   Note: Many possible answers:

2 10 11 10 – correct

1 5 1 10 - correct

3 20 2 10 – wrong!

Fuzzy Logic

Theory and Applications

n.h.reyes@massey.ac.nz

Fuzzy Logic

- Introduction
  - What is Fuzzy Logic?
  - Applications of Fuzzy Logic
  - Classical Control System vs. Fuzzy Control
- Developing a Fuzzy Control System
- Examples
- Theory of Fuzzy Sets
- Fuzzy Inference Systems
- Assignment #2

Topics

- Introduction
- Basic Algorithm
- Control Systems
- Sample Computations
- Inverted Pendulum
- Fuzzy Inference Systems
  - Mamdani Type
  - Sugeno Type
- Fuzzy Sets & Operators
- Defuzzification
- Membership Functions

Basics

Control Systems

Computations

Inverted Pendulum

Mamdani

Sugeno

Fuzzy Sets

Defuzzification

Mem. Fcns

back

menu

Motivation

Previously

Systematic exploration of alternatives

find a path or a plan of action

e.g. sequence of moves to solve a puzzle

States finite and complete

e.g. arrangement of tiles in a puzzle

Moves deterministic and discrete

e.g. up, right, down, left

Next set of problems

- States with continuous-valued and ill-defined inputs
- Dynamically-changing world

e.g. target is moving to the left and its speed is increasing fast

Continuous-valued actions
uncertainty in inputs

e.g. exact steering angle, speed, exact Force

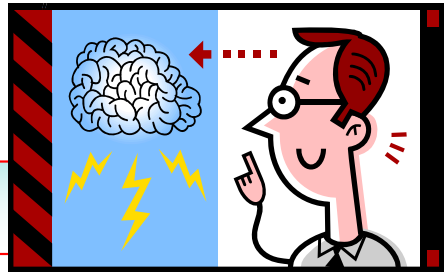
What is Fuzzy Logic?

‘Fuzzy’ – a misnomer

n.h.reyes@massey.ac.nz

Fuzzy Logic

What is Fuzzy Logic?



A **computational paradigm that mimics how humans think**.

Fuzzy Logic looks at the world in **imprecise terms**, in much the same way that our brain takes in information (e.g. temperature is hot, speed is slow), then responds with **precise actions**.

The human brain can reason with uncertainties, vagueness, and judgments. Computers can only manipulate precise valuations. Fuzzy logic is an attempt to combine the two techniques.

Fuzzy Logic

What is Fuzzy Logic?

FL is in fact, **a precise problem-solving methodology**.

It is able to simultaneously handle numerical data and linguistic knowledge.

A technique that facilitates the control of a complicated system without knowledge of its mathematical description.

"Fuzzy" – a misnomer, has resulted in the mistaken suspicion that FL is somehow less exacting than traditional logic

e.g. *Why would you purchase a fuzzy-controlled auto-focusing camera?... Wouldn't it produce fuzzy images?*

What is Fuzzy Logic?

History

n.h.reyes@massey.ac.nz

Fuzzy Logic

History of Fuzzy Logic

an American, mathematically oriented, computer scientist, electrical engineer of Iranian descent, born in Russia.

Professor Lotfi A. Zadeh

http://www.cs.berkeley.edu/~zadeh/

In **1965**, **Lotfi A. Zadeh** of the University of California at Berkeley published "**Fuzzy Sets**," which laid out the mathematics of fuzzy set theory and, by extension, fuzzy logic. Zadeh had observed that conventional computer logic couldn't manipulate data that represented subjective or vague ideas, so he created fuzzy logic to allow computers to determine the distinctions among data with shades of gray, similar to the process of human reasoning.

This paper drew 90,000 Google Scholar citations (as of 2017).
It is the highest cited paper in the literature of Computer Science (Web of Science);

Source: August 30, 2004
(Computerworld)

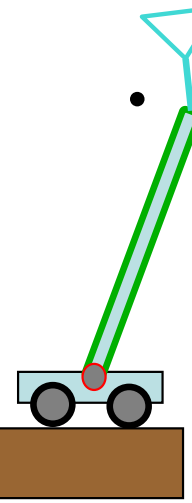
http://www.cs.berkeley.edu/~zadeh/suprco.html

http://www.computerworld.com/news/2004/story/0,11280,95282,00.html

Pioneering works

20 years later after its inception

- Interest in fuzzy systems was sparked by **Seiji Yasunobu** and **Soji Miyamoto** of **Hitachi**, who in **1985** provided simulations that demonstrated the superiority of fuzzy control systems for the **Sendai railway**. Their ideas were adopted, and fuzzy systems were used to control accelerating and braking when the line opened in **1987**.



- Also in **1987**, during an international meeting of fuzzy researchers in Tokyo, **Takeshi Yamakawa** demonstrated the use of fuzzy control, through a set of simple dedicated fuzzy logic chips, in an "**inverted pendulum**" experiment. This is a classic control problem, in which a vehicle tries to keep a pole mounted on its top by a hinge upright by moving back and forth.

- Observers were impressed with this demonstration, as well as later experiments by **Yamakawa** in which he mounted a wine glass containing water or even a live mouse to the top of the pendulum. The system maintained stability in both cases. Yamakawa eventually went on to organize his own fuzzy-systems research lab to help exploit his patents in the field.

What is Fuzzy Logic?

Applications

n.h.reyes@massey.ac.nz

Fuzzy Logic

Introduction of FL in the Engineering world (1990's),

(*News excerpt from the 1990s*) **Fuzzy Logic** is one of the most talked-about technologies to hit the embedded control field in recent years. It has already transformed many product markets in Japan and Korea, and has begun to attract a widespread following In the United States. Industry watchers predict that fuzzy technology is on its way to becoming a multibillion-dollar business.

Fuzzy Logic enables low cost microcontrollers to perform functions traditionally performed by more powerful expensive machines enabling lower cost products to execute advanced features.

Intel Corporation's Embedded Microcomputer Division Fuzzy Logic Operation

Motorola 68HC12 MCU

http://www.intel.com/design/mcs96/designex/2351.htm

back

menu

Sample Applications

In the city of Sendai in Japan, a 16-station subway system is controlled by a fuzzy computer (Seiji Yasunobu and Soji Miyamoto of Hitachi) – the ride is so smooth, riders do not need to hold straps

Nissan – fuzzy automatic transmission, fuzzy anti-skid braking system

CSK, Hitachi – Hand-writing Recognition

Sony - Hand-printed character recognition

Ricoh, Hitachi – Voice recognition

Tokyo's stock market has had at least one **stock-trading portfolio based on Fuzzy Logic** that outperformed the Nikkei exchange average

Sample Applications

NASA has studied fuzzy control for **automated space docking**: simulations show that a fuzzy control system can greatly reduce fuel consumption

Canon developed an **auto-focusing camera** that uses a charge-coupled device (CCD) to measure the clarity of the image in six regions of its field of view and use the information provided to determine if the image is in focus. It also tracks the rate of change of lens movement during focusing, and controls its speed to prevent overshoot.

The camera's fuzzy control system uses **12** inputs: 6 to obtain the current clarity data provided by the CCD and 6 to measure the rate of change of lens movement. The output is the position of the lens. The **fuzzy control system** uses **13 rules** and requires **1.1 kilobytes** of memory.

http://en.wikipedia.org/wiki/Fuzzy_system

Sample Applications

Segway Robotics Mobility Platform

- Algorithm: combination of Fuzzy Logic and Expert System
- Sensors: multiple gyroscopes and accelerometers
- Estimates the 3D position of the mobile robot
- The control system dynamically prevents the robot from falling over.

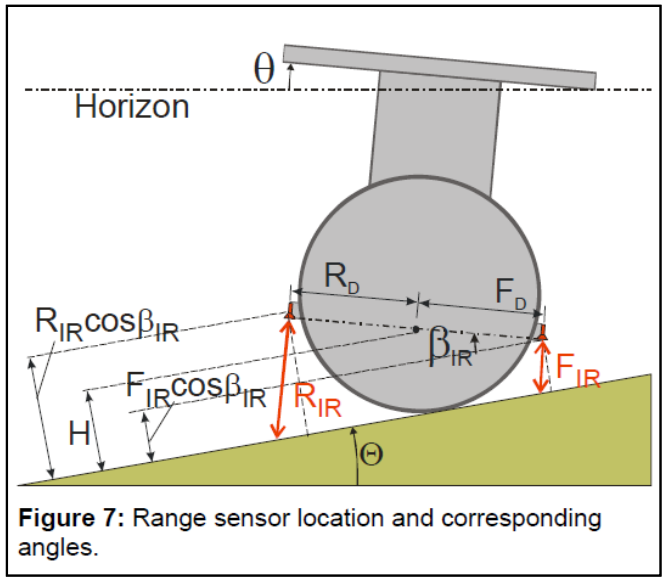
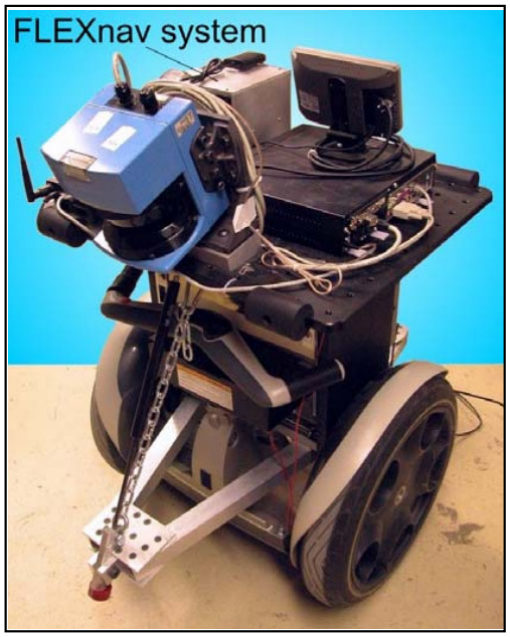


Figure 7: Range sensor location and corresponding angles.



2004, FLEXnav: A Fuzzy Logic Expert Dead-reckoning System for the Segway RMP
Lauro Ojeda, Mukunda Raju and Johann Borenstein
The University of Michigan, Advanced Technologies Lab

Sample Applications

What about hoverboards? how to build its control system?

- self-balancing, two-wheeled scooters
- you just lean to move forward and put weight on whichever foot to go left or right.

Sample Applications

In high-end washing machines, dishwashers and refrigerators, fuzzy logic is now commonplace.

- These machines offer the advantages of performance, simplicity, productivity, and less cost.
- Sensors continually monitor varying conditions inside the machine and accordingly adjust operations for the best wash results.
- Typically, fuzzy logic controls the washing process, water intake, water temperature, wash time, rinse performance, and spin speed. This optimises the life span of the washing machine.
- (Neuro-Fuzzy) Some machines even learn from past experience, memorising programs and adjusting them to minimise running costs.

GE WPRB9110WH Top Load Washer

Haier ESL-T21 Top Load Washer

LG WD14121 Front Load Washer

Miele WT945 Front Load All-in-One Washer / Dryer

AEG LL1610 Front Load Washer

Zanussi ZWF1430W Front Load Washer

Others: Samsung, Toshiba, National, Matsushita, etc.

- Fuzzy Logic: The Revolutionary Computer Technology That Is Changing Our World, by Daniel Mcneill, Paul Freiberger
- http://www.samsung.com/in/support/skp/faq/138486

Sample Applications

Fuzzy logic washing machines vs. PID washing machines

- Standard **PID controller** would require [**1,000-2,000**] rules to build the controller.
- **Fuzzy Logic** achieve the same results with **200** rules. (Productivity gain!)

Fuzzy logic dishwashers

- Using Fuzzy Logic the dishwasher can determine the dishwasher load. The duration and the water usage are then adapted accordingly, to ensure that no excess water is used. Rest assured that whether you wash a full load or a half load the dishwasher will adapt accordingly.

- (Accessed 2018) http://www.electrolux.co.nz/support/usage-advice/dishwashers/

Heat Pumps

consumer. RECOMMENDS
Mitsubishi Heavy Industries SRK20ZSXA-W

Cooling ENERGY RATING
Heating ENERGY RATING

Recommended by Consumer and with an industry leading 7 star "Super Efficiency" energy rating the 2.0kW Avanti PLUS® heat pump is one of the most efficient split system heat pumps on the market.

Incorporating a range of advanced features including an energy saving motion sensor and eco-operation, improved automatic mode for precise temperature control, LED display with adjustable brightness and optional wi-fi control, the 2.0kW Avanti PLUS® split system heat pump is perfect for smaller spaces such as a single bedroom, guest bedroom, kid's room or a home office.

ENERGY SAVING

FUZZY AUTO MODE

Using fuzzy logic algorithms, the unit determines the operating mode and temperature settings automatically and adjusts the inverter frequency.

KEY FEATURES

- 7 star "Super Efficiency" energy rating
- Award winning, modern design
- Energy saving motion sensor and eco operation as standard
- Allergen clear system to eliminate airborne allergens and deliver clean air throughout your home
- Improved auto mode allowing for precise temperature control

(Accessed, 2021) https://mhiheatpumps.co.nz/products/model/split-system-heat-pump-avanti-plus-srk20zsxa-w/

Meeting Lotfi in Germany

My Fuzzy Logic-based Research

- Navigation in unknown terrain (Hybrid Fuzzy-D*Lite)
- Robot soccer navigation
  - Real-time path-planning (Hybrid Fuzzy A*)
- Machine Vision
  - Real-time colour-object recognition
  - Fuzzy Colour Contrast Fusion

9th Fuzzy Days (2006), Dortmund, Germany

Meeting Prof. Yamakawa in Japan

ICONIP 2007, Kitakyushu, Japan

back

menu

Control Systems in General

n.h.reyes@massey.ac.nz

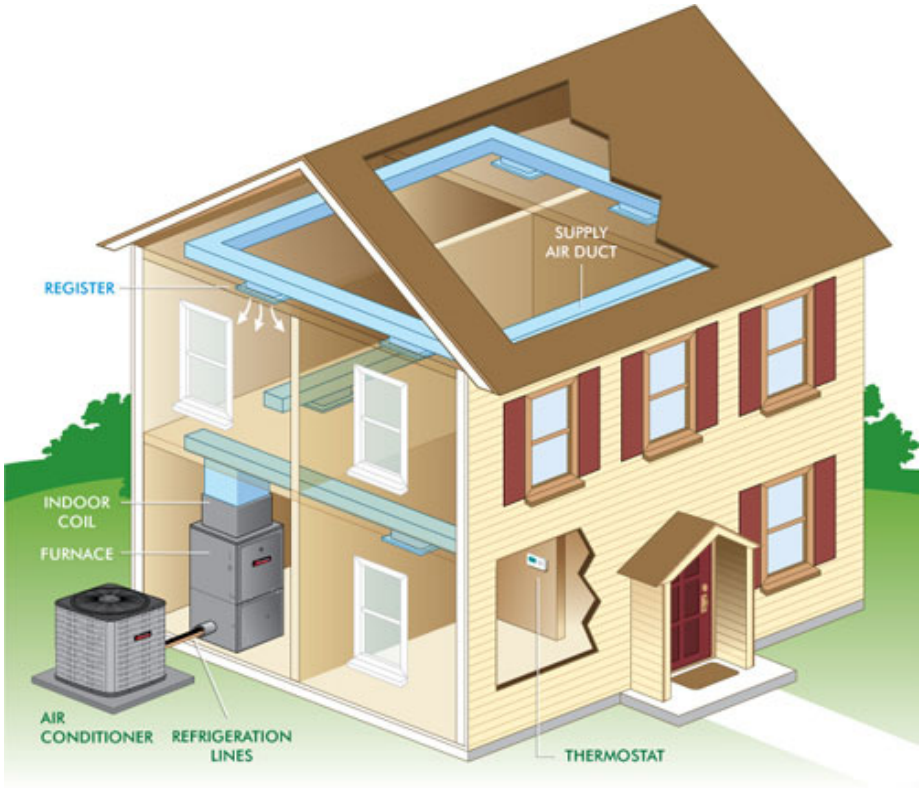
Control Systems in General

Objective

The aim of any control system is **to produce a set of desired outputs for a given set of inputs**.

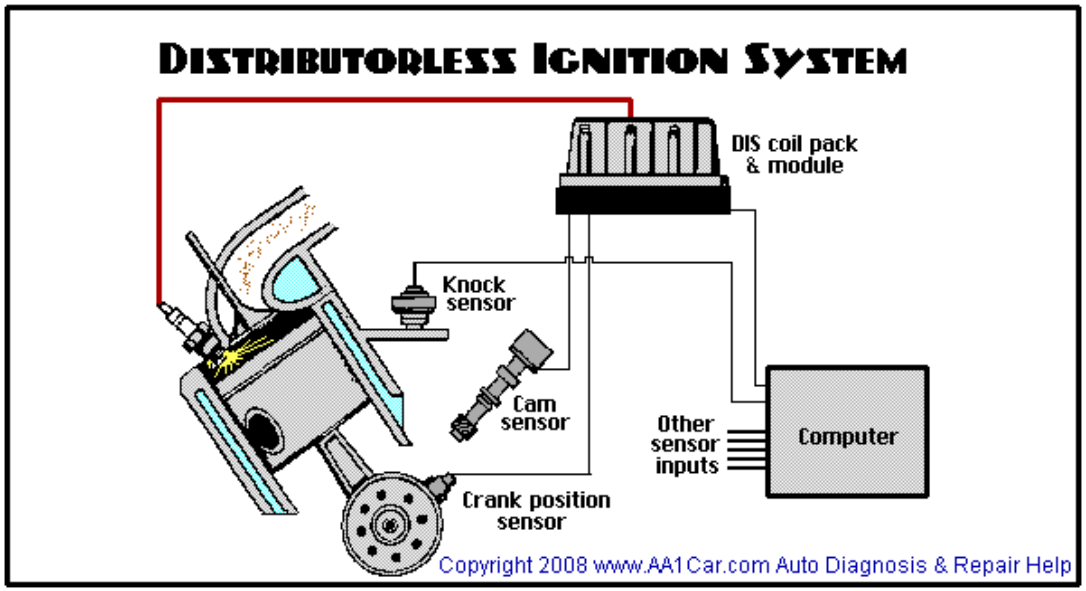
Example

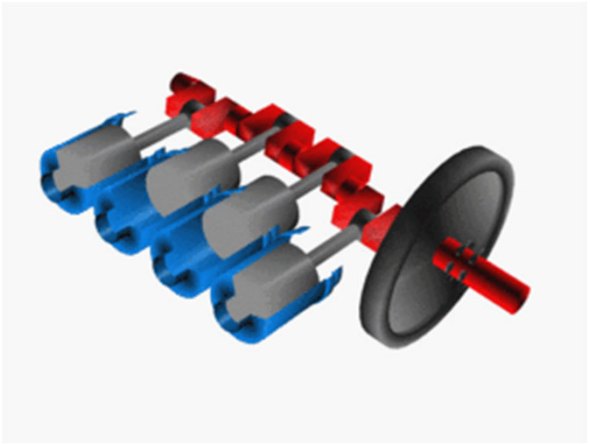
A household thermostat takes a temperature input and sends a control signal to a furnace.



Control Systems in General

Example





Crankshaft (red), pistons (gray) in their cylinders (blue), and flywheel (black)

Image: http://en.wikipedia.org/wiki/Crankshaft

A car engine controller responds to variables such as engine position, manifold pressure and cylinder temperature to regulate fuel flow and spark timing.

back

menu

Control Systems

Conventional Control vs. Fuzzy Control

n.h.reyes@massey.ac.nz

Conventional Control vs. Fuzzy

1. Look-up table

In the simplest case, a controller takes its cues from a look-up table, which tells what output to produce for every input or combination of inputs.

Sample

The table might tell the controller,
"**IF temperature is 85, THEN increase furnace fan speed to 300 RPM.**"

Drawbacks

The problem with the tabular approach is that the **table can get very long**, especially in situations where there are many inputs or outputs. And that, in turn, **may require more memory than the controller can handle**, or more than is cost-effective.

Tabular control mechanisms may also give a **bumpy, uneven response**, as the controller jumps from one table-based value to the next.

Conventional Control vs. Fuzzy

2. Mathematical formula

The usual alternative to look-up tables is to have the controller execute a mathematical formula – **a set of control equations that express the output as a function of the input**.

Ideally, these equations represent an accurate model of the system behaviour.

For example:

$$\left[m \frac{\partial^2}{\partial t^2} (x + l \sin\theta) \right] l \cos\theta - \left[m \frac{\partial^2}{\partial t^2} (l \cos\theta) \right] l \sin\theta = mgl \sin\theta$$

Downside of mathematical modeling

The formulas can be very complex, and working them out in real-time may be more than an affordable controller (or machine) can manage.

It may be difficult or impossible to derive a workable mathematical model in the first place, making both tabular and formula-based methods impractical.

Conventional Control vs. Fuzzy

Why use Fuzzy Logic?

FL overcomes the disadvantages of both table-based and formula-based control.

Fuzzy has **no unwieldy memory requirements** of look-up tables, and **no heavy number-crunching demands** of formula-based solutions.

Troubleshooting a car (an analogy)

- A fuzzy system is analogous to a human expert who learned through experience.
- Though an automotive engineer might understand the general relationship between say, ignition timing, air flow, fuel mix and engine RPM, the exact math that underlies those interactions may be completely obscure.

Conventional Control vs. Fuzzy

Why use Fuzzy Logic?

FL can make development and implementation much simpler.

It needs no intricate mathematical models, only a practical understanding of the overall system behaviour.

FL mechanisms can result to **higher accuracy** and **smoother control** as well.

Fuzzy Set Theory

n.h.reyes@massey.ac.nz

Fuzzy Logic Explained

Fuzzy Set Theory

Fuzzy logic differs from classical logic in that statements are no longer black or white, true or false, on or off.
In traditional logic an object takes on a value of either zero or one.
In fuzzy logic, a statement can assume any real value between 0 and 1, representing the degree to which an element belongs to a given set.

In other words, FL recognizes not only clear-cut, black-and-white alternatives, but also the infinite gradations in between.

Fuzzy reasoning eliminates the vagueness by assigning specific numbers to those gradations. These numeric values are then used to derive exact solutions to problems.

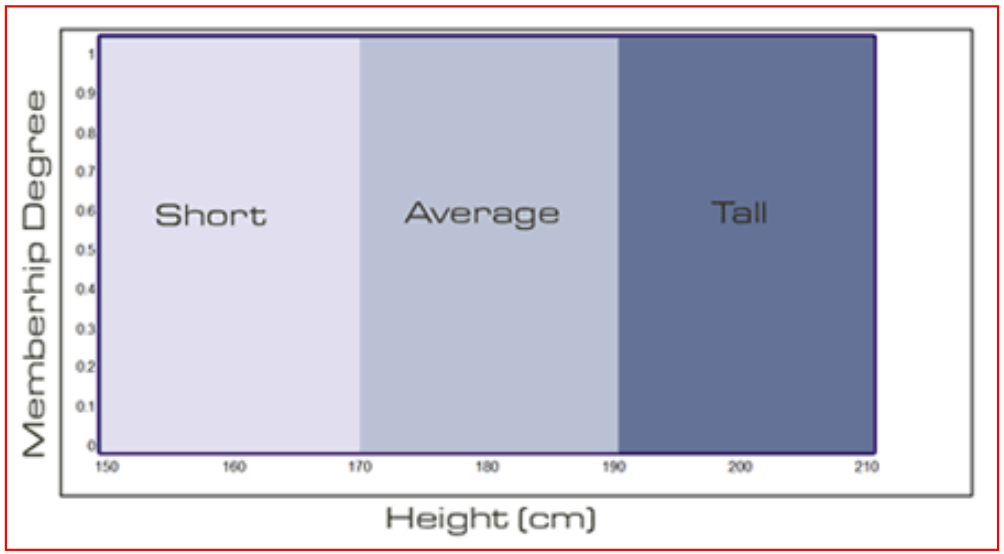
Fuzzy Logic Explained

Fuzzy Set Theory

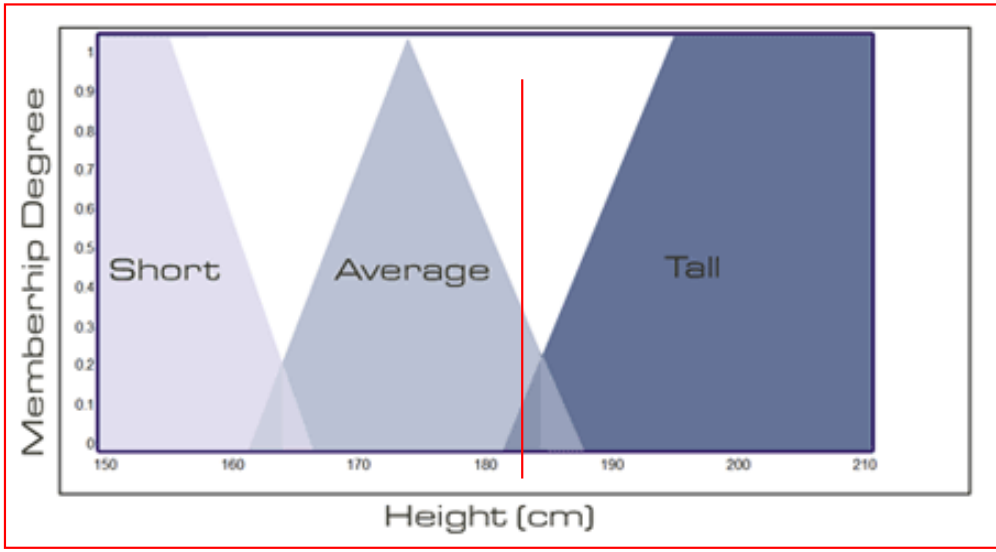
Is a man whose height is 5’ 11-1/2” average or tall?

A fuzzy system might say that he is partly medium and partly tall.

Boolean representation



Fuzzy representation



http://blog.peltarion.com/2006/10/25/fuzzy-math-part-1-the-theory/

In fuzzy terms, the height of the man would be classified within a range of [0, 1] as average to a degree of 0.6, and tall to a degree of 0.4.

Fuzzy Rules

Fuzzy rules may come in various forms

Examples:

(*a*) if pressure is high then volume is small — represents dependency

(*b*) if pressure is high and temperature is low then volume is very small — represents dependency, multiple antecedents

(*c*) if pressure is high then lower temperature slightly — command

(*d*) if pressure is high then volume is small unless temperature is high — represents dependency

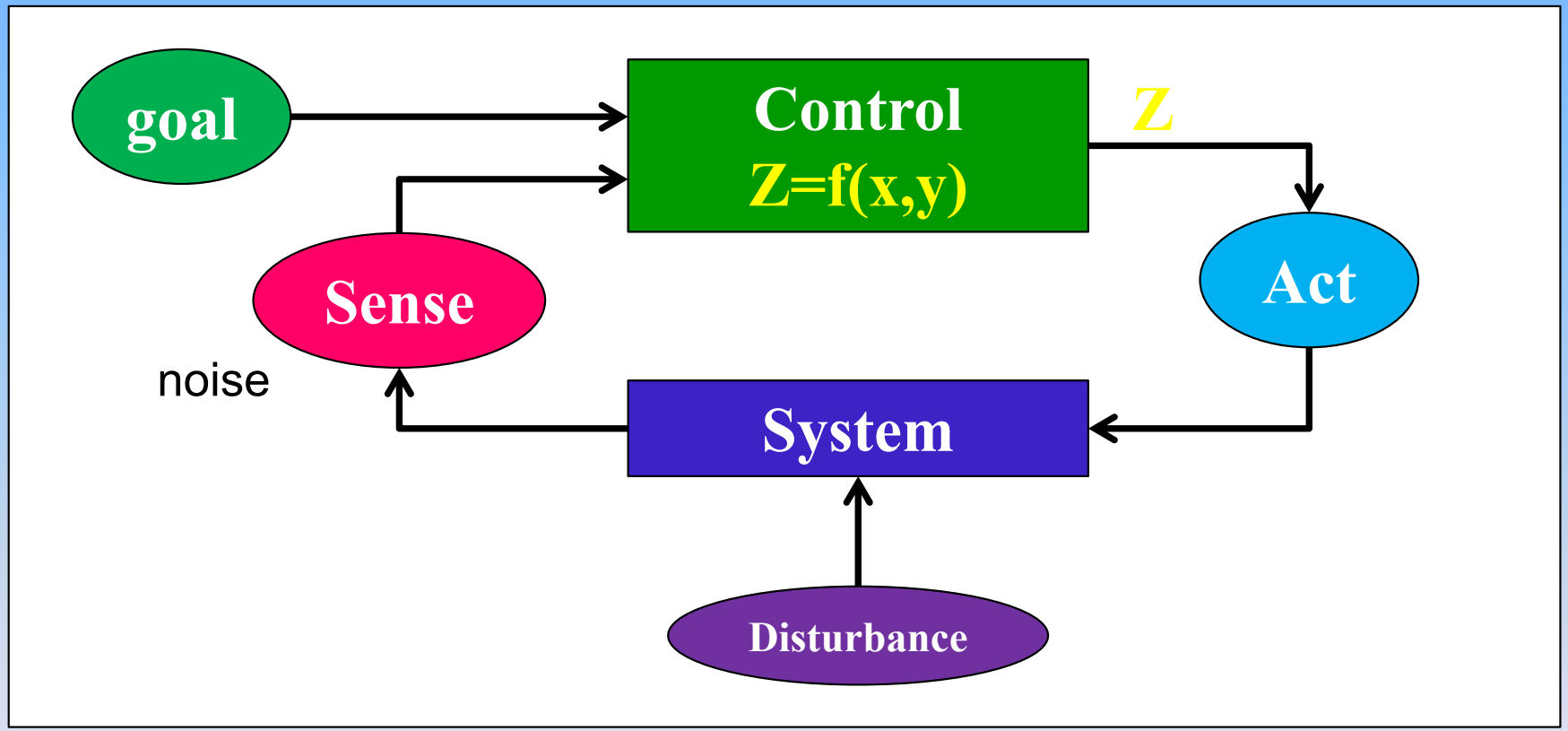
(*e*) if pressure is high then usually volume is small. — represents dependency, dispositional

In these examples, pressure, volume and temperature are linguistic variables and small, low and high are their linguistic values.

- All of the rules except (c) represent dependencies, with (c) representing a command.
- All of the rules except (d) and (e) are categorical.
- Rules (d) and (e) are qualified, with (d) qualified through an exception and (e) through usuality.
- (e) exemplifies what is referred to as a dispositional rule.
- All of the rules except for (b) involve a single variable in the antecedent

Feedback Control/Closed Loop Control

- Closed-loop control allows for **uncertainty** in the model as well as **noise** and **disturbances** in the system under control



*__Controllers__ are used in the industry to **regulate** temperature, pressure, flow rate, chemical composition, speed and practically every other variable for which a measurement exists.

Fuzzy Inference Process

- What are the steps involved in creating a Fuzzy Control System?

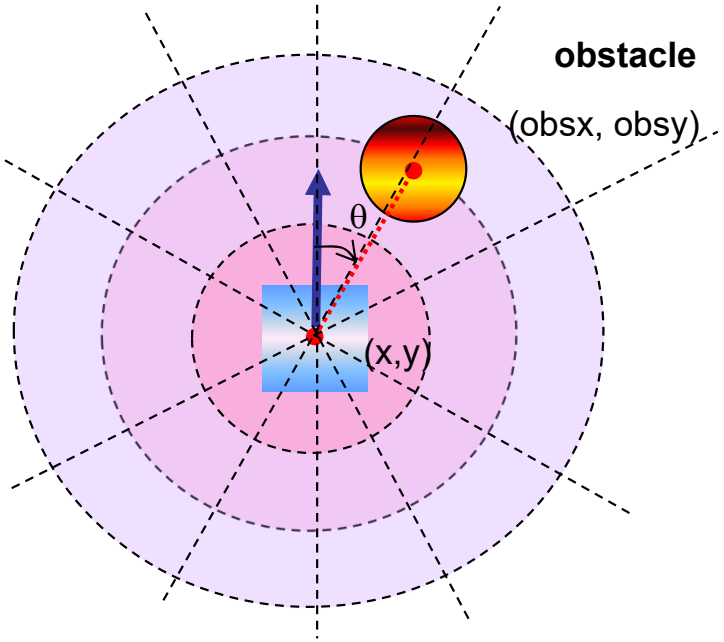
Fuzzy Inference Process

Fuzzy Inference Process

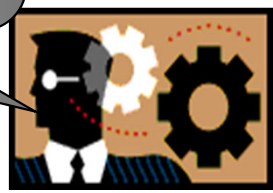
Fuzzification: Translate input into truth values
Rule Evaluation: Compute output truth values
Defuzzification: Transfer truth values into output

Obstacle Avoidance Problem

Robot Navigation



Can you describe how the robot should turn based on the position and angle of the obstacle?

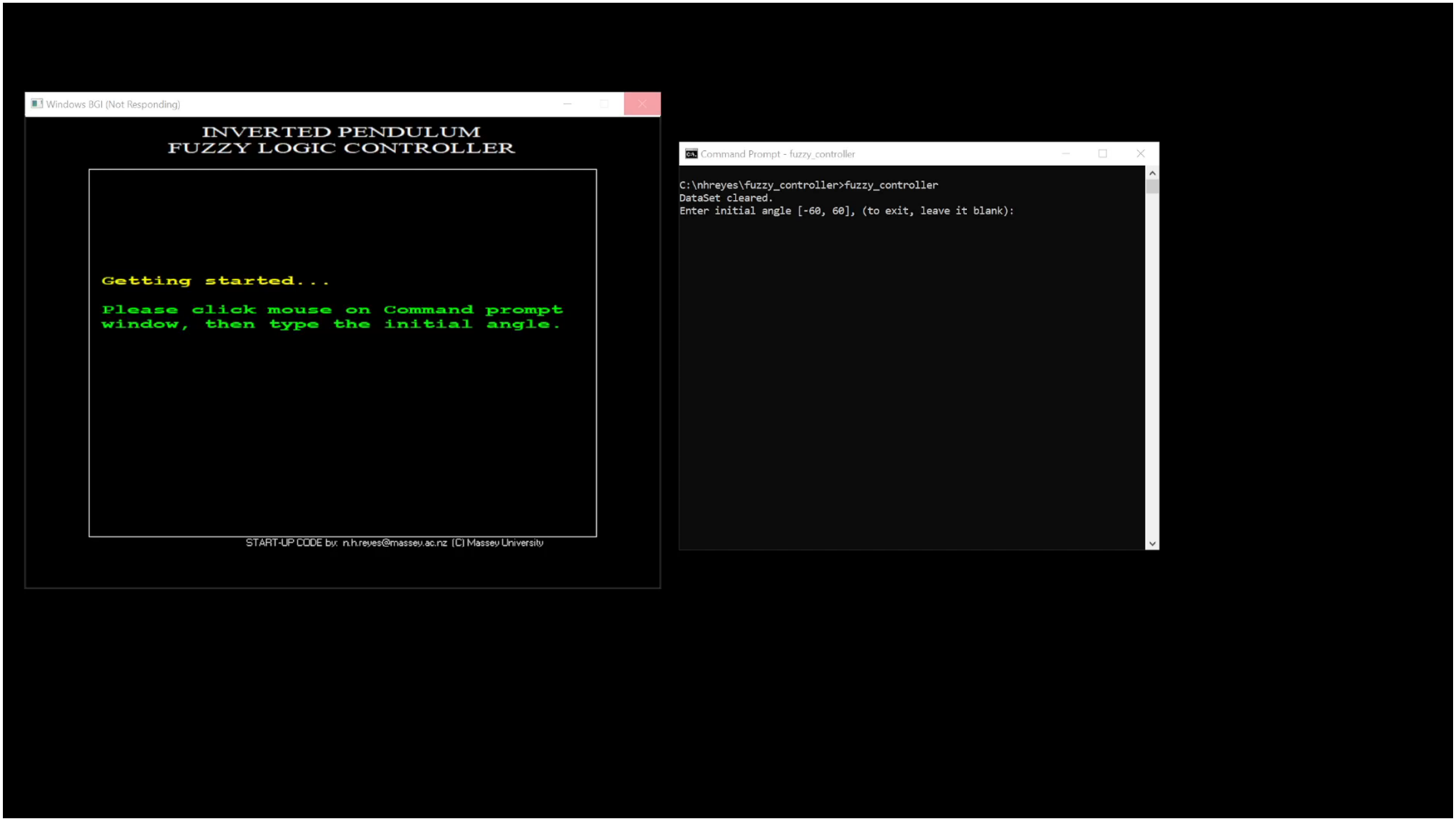


Obstacle Avoidance & Target Pursuit

Demonstration

back

Demonstration of a well-calibrated Fuzzy System for controlling the Inverted Pendulum



Fuzzy System Design #1

Reference: (Book) Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead

Fuzzy System Design #2 (best solution!)

Reference: (Journal article), Takeshi Yamakawa, "A Fuzzy Inference Engine in Nonlinear Analog Mode and its Application to a Fuzzy Logic Control"

Fuzzy System Design #1

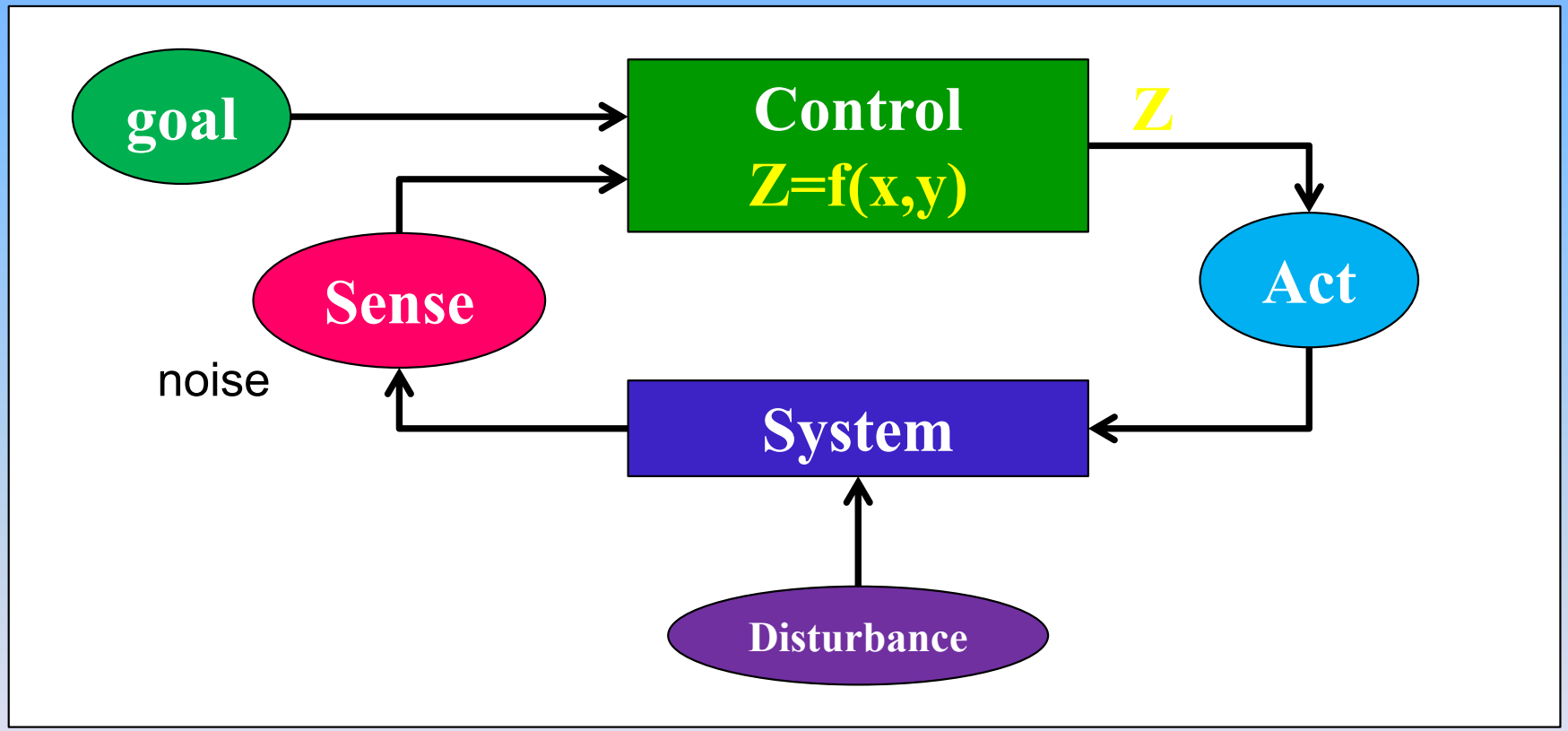
Inverted Pendulum Problem

Takagi-Sugeno Fuzzy Inference System

Reference: Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead

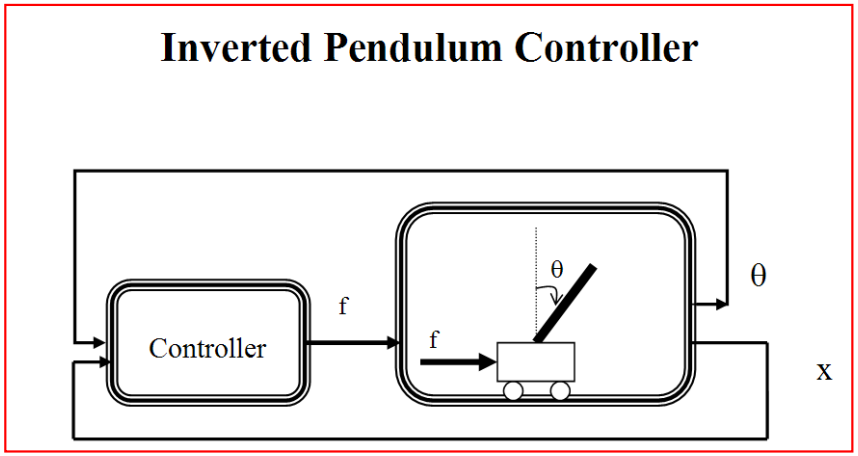
Feedback Control/Closed Loop Control

- Closed-loop control allows for **uncertainty** in the model as well as **noise** and **disturbances** in the system under control



*__Controllers__ are used in the industry to **regulate** temperature, pressure, flow rate, chemical composition, speed and practically every other variable for which a measurement exists.

Inverted Pendulum Problem



A Classic test case in embedded control

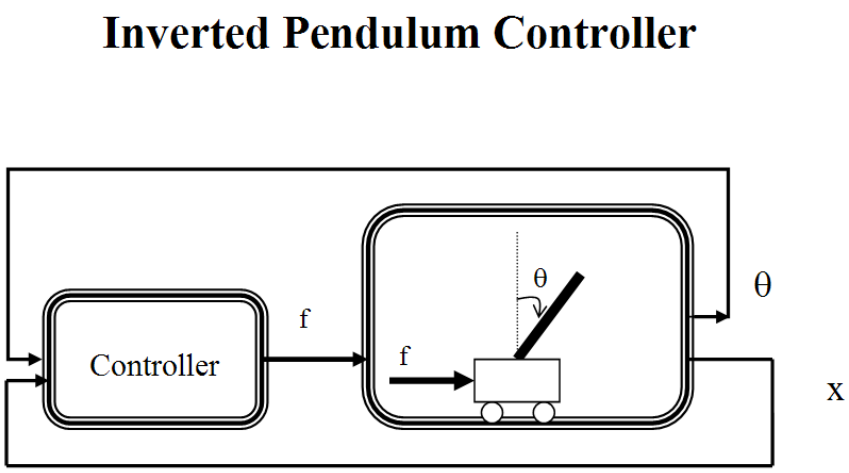
A pole with a weight on top is mounted on a motor-driven cart. The pole can swing freely, and the cart must move back and forth to keep it vertical.

A controller monitors the angle and motion of the pole and directs the cart to execute the necessary balancing movements.

A Glimpse at History: International Conference in Tokyo (1987) **Takeshi Yamakawa** demonstrated the use of fuzzy control, through a set of simple dedicated fuzzy logic chips, in an "**inverted pendulum**" experiment. (Later experiments: mounted a wine glass containing water or even a live mouse to the top of the pendulum).

back
menu

Inverted Pendulum Problem



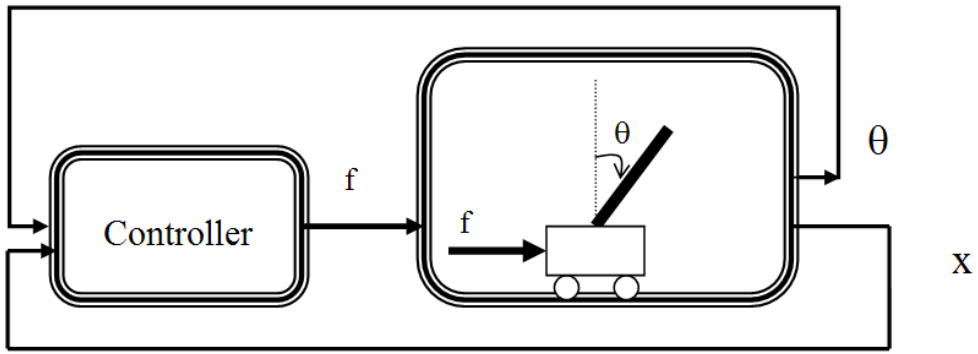
Conventional mathematical solution

The solution uses a second-order differential equation that describes cart motion as a function of pole position and velocity:

$$\left[m \frac{\partial^2}{\partial t^2} (x + l \sin\theta) \right] l \cos\theta - \left[m \frac{\partial^2}{\partial t^2} (l \cos\theta) \right] l \sin\theta = mgl \sin\theta$$

Inverted Pendulum Problem

Inverted Pendulum Controller



Sensed values:

X – position of object with respect to the horizontal axis

θ - angle of pole relative to the vertical axis

Derived values:

X' - Velocity along the x-axis

θ' - Angular velocity

Input variables: sensed and derived values

Controller output: **F** – force to be applied to the cart

back

menu

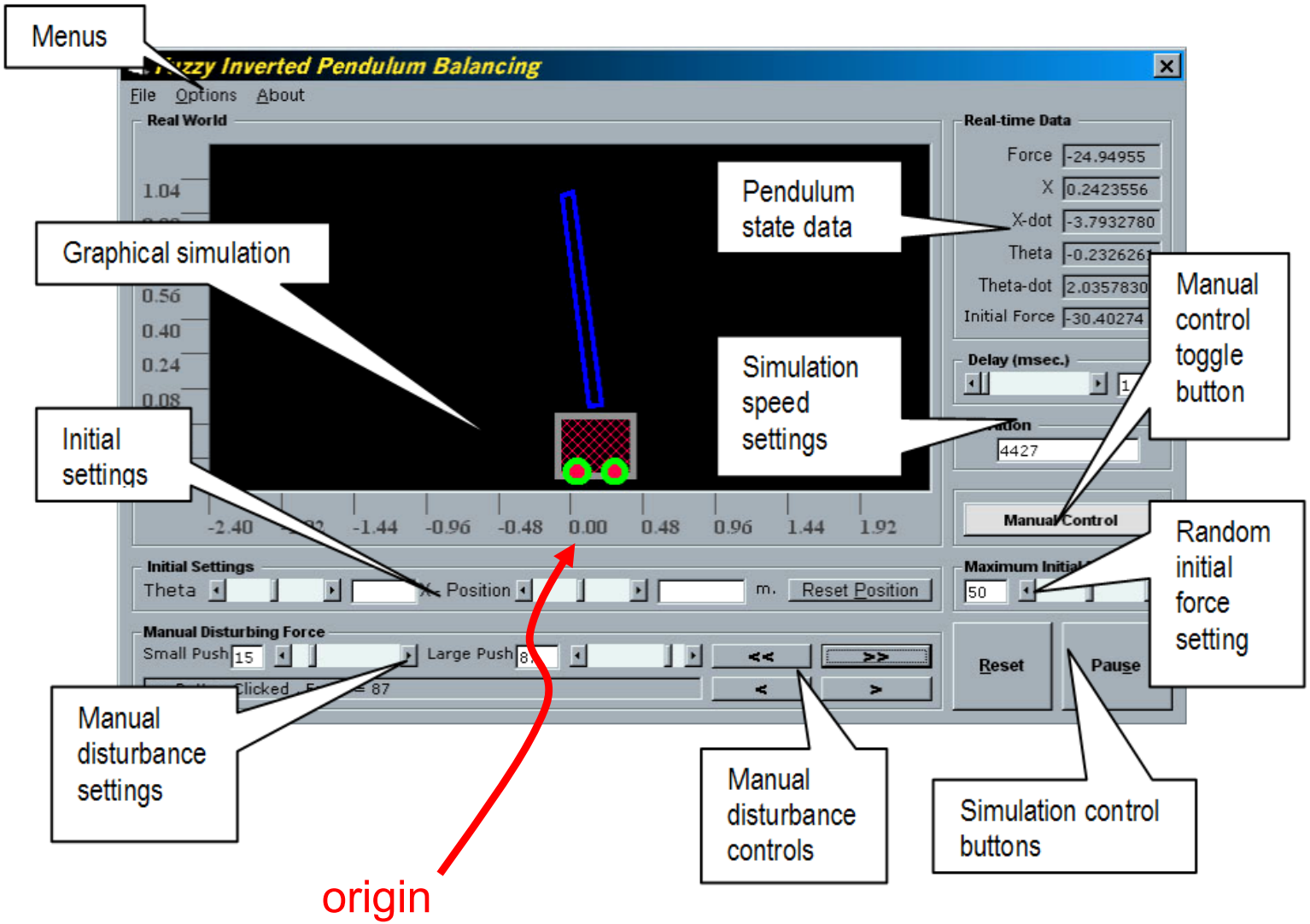
Derived Input Values

We can derive new input values for our Fuzzy Control System using Physics equations.

A sample calculation of some of the derived values: angular velocity (theta')

theta	time	theta'
2	1	
10	2	8
30	3	20
40	4	10
47	5	7
32	6	-15
28	7	-4
19	8	-9

Inverted Pendulum Problem



back

menu

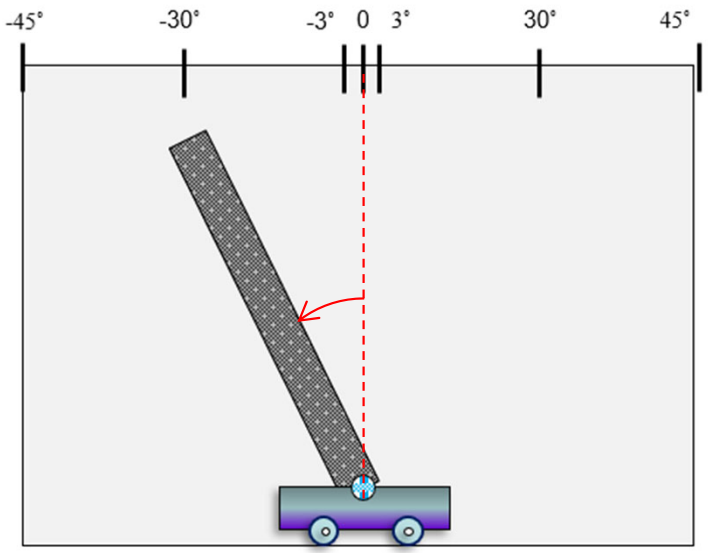
Parameters for a Fuzzy System

Once you have determined the appropriate inputs and outputs for your application, there are three steps to designing the parameters for a fuzzy system:

1. specify the **fuzzy sets** to be associated with each variable.
2. decide on what the **fuzzy rules** are going to be.
3. specify the **shape of the membership functions**.

Fuzzy Sets

Input **angle** = the pole angle measured relative to the vertical axis.



Note: the figure is not drawn to scale

We can begin designing a fuzzy system by subdividing the two input variables (pole angle and angular velocity) into membership sets.

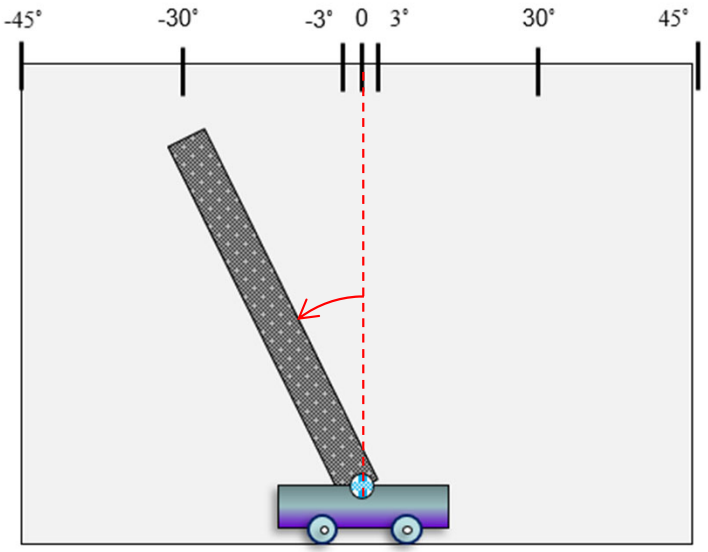
The **angle** could be described as:

1. Inclined to the Left (Negative).
2. Vertical (Zero).
3. Inclined to the Right (Positive).

Linguistic terms for describing an angle measurement

Fuzzy Sets

Input **angle** = the pole angle measured relative to the vertical axis.



Note: the figure is not drawn to scale

Fuzzy logic provides the mathematical facility for transforming vague linguistic terms into their equivalent mathematical function representation.

Linguistic terms		Fuzzy sets
1. Inclined to the Left (Negative).	→	$F_{negative}(angle)$
2. Vertical (Zero).	→	$F_{zero}(angle)$
3. Inclined to the Right (Positive).	→	$F_{positive}(angle)$

Implemented as **membership functions**

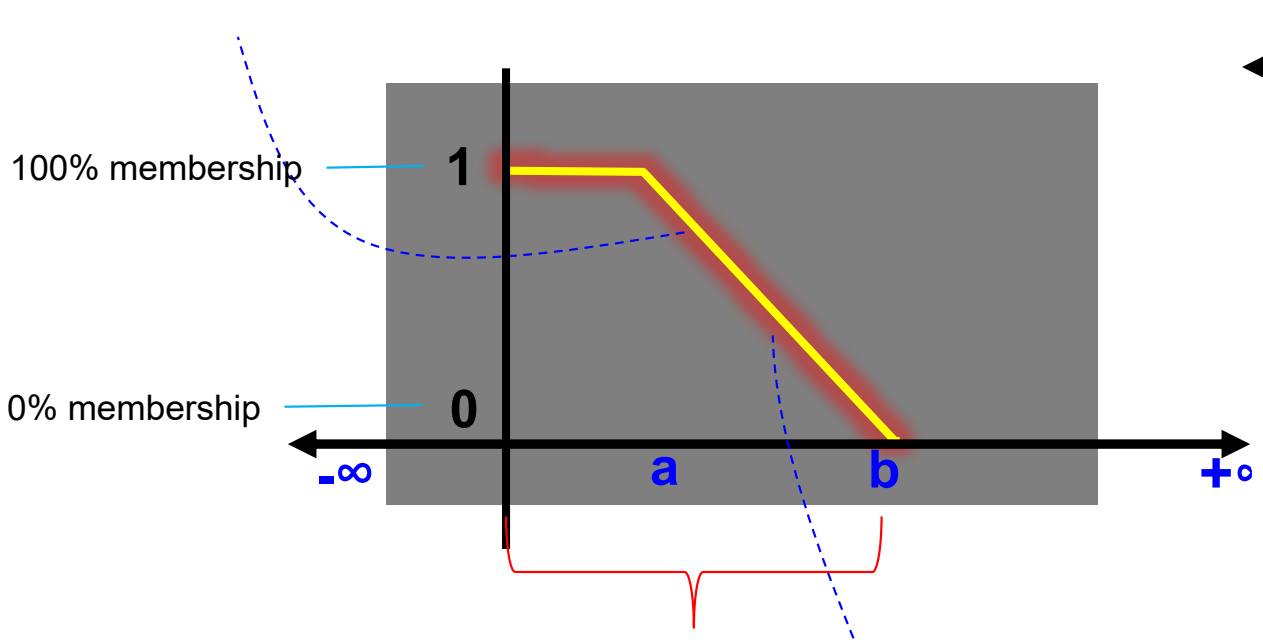
back

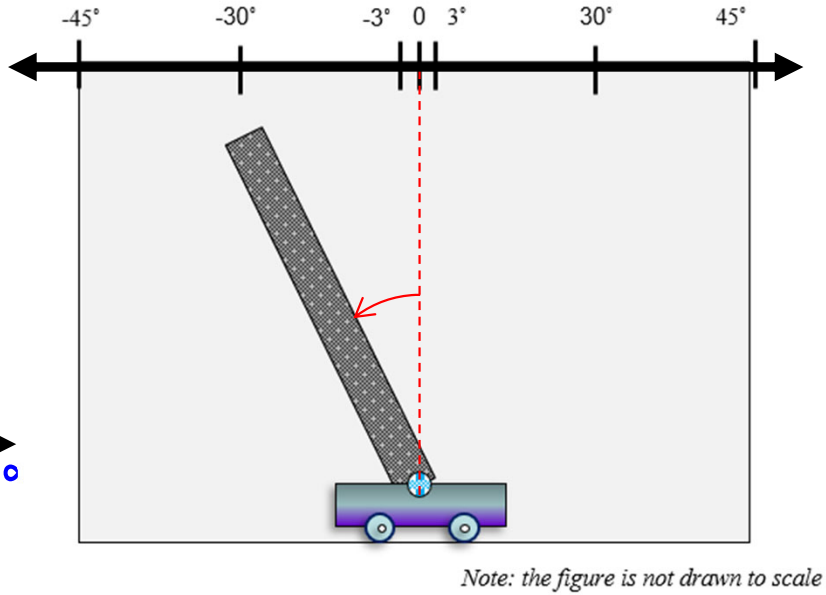
menu

Membership Functions

Membership Functions for the Pole Angle: Inclined to the Left (Negative)

Inclined to the Left (also referred to as Negative angle)





Note: the figure is not drawn to scale

Range of values associated with the term "Inclined to the Left"

Linguistic term **Fuzzy set**

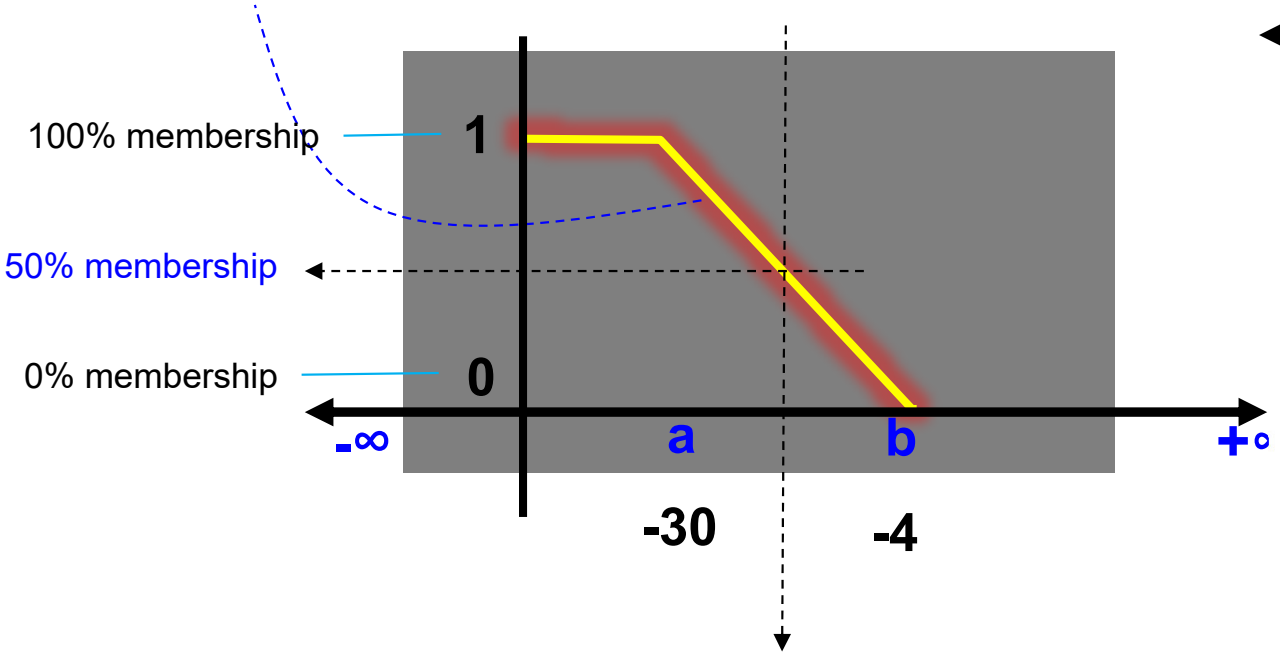
1. Inclined to the Left (Negative). → $F_{negative}(angle) = [0, 1]$

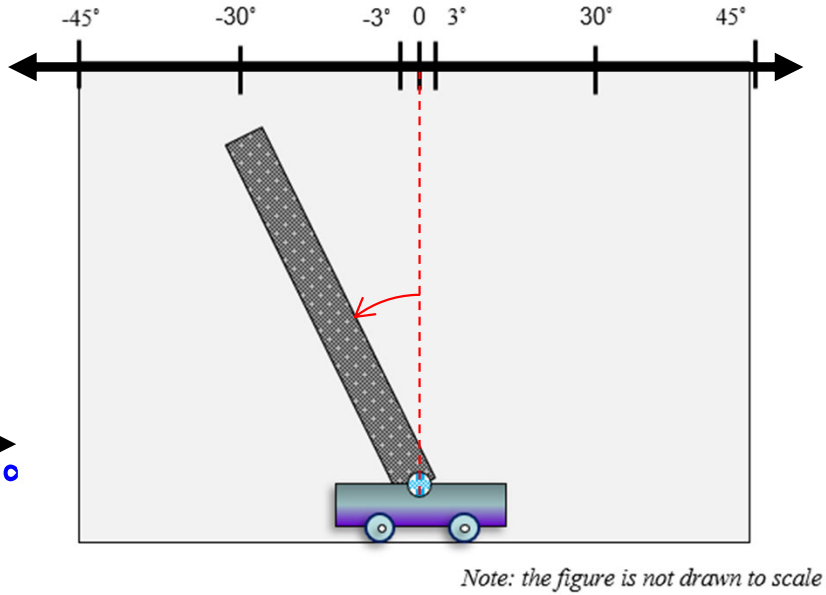
input

Membership Functions

Membership Functions for the Pole Angle: Inclined to the Left (Negative)

Inclined to the Left (also referred to as Negative angle)





Given an **input angle** = **-13**

Fuzzy set

$F_{negative}(angle) = ?$

input

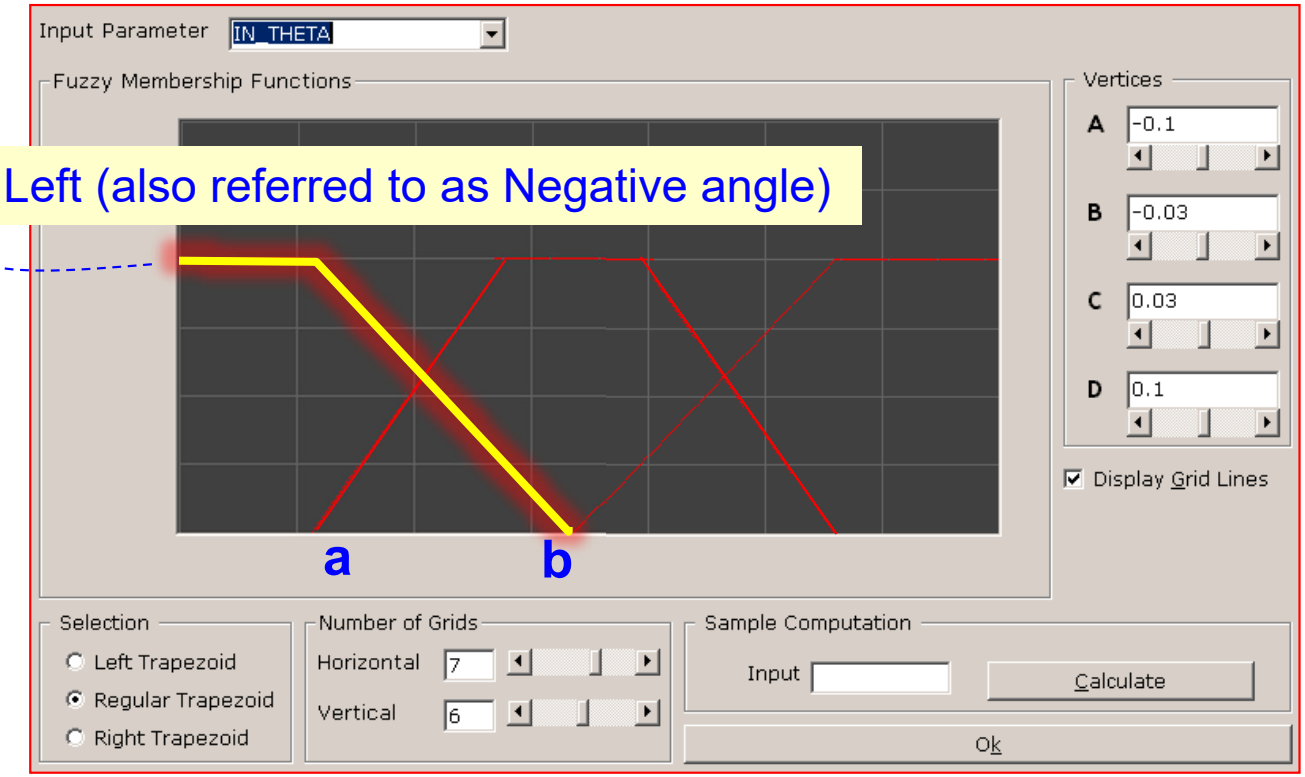
Degree of membership of the input angle = **-13** to the fuzzy set $F_{negative}$

$F_{negative}(-13) \sim 0.5$

Membership Functions

Membership Functions for the Pole Angle: Inclined to the Left (Negative)

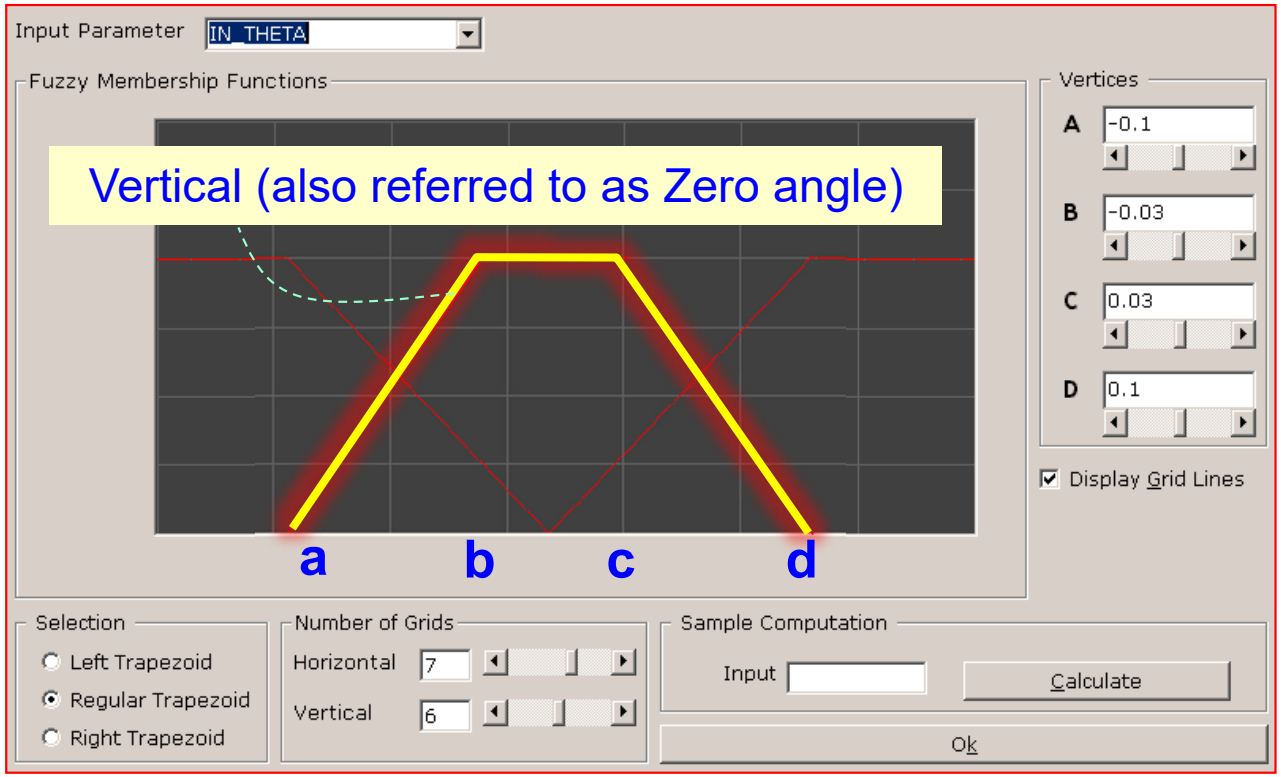
Inclined to the Left (also referred to as Negative angle)



Trapezoid Vertices		
Left Trapezoid	**Regular**	**Right**
A = -0.1 rad. = -5.73 deg.	A = -0.1 rad = -5.73 deg.	A = 0
B = 0	B = -0.03 = -1.72 deg.	B = 0.1 rad = 5.73 deg.
C = 0	C = 0.03 = 1.72 deg.	C = 0
D = 0	D = 0.1 rad = 5.73 deg.	D = 0

Membership Functions

Membership Functions for the Pole Angle: Inclined to the Left (Negative)

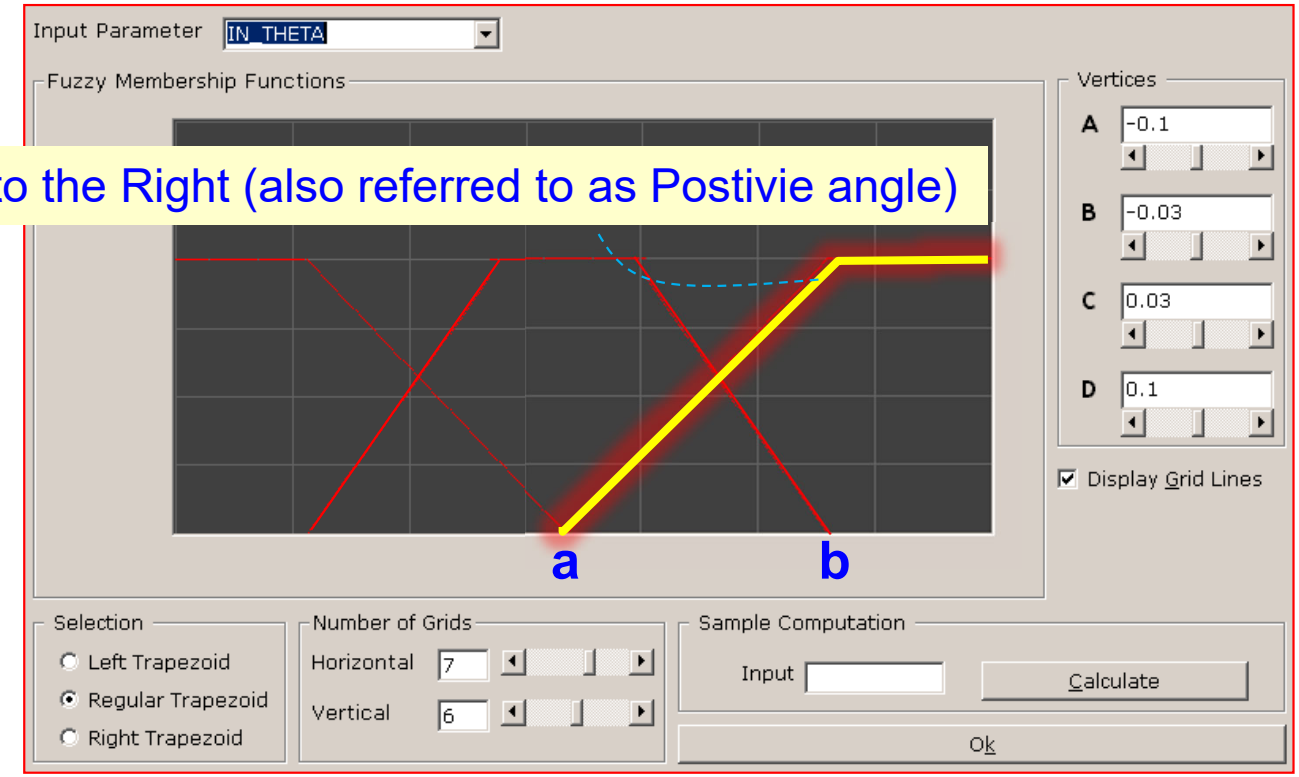


Trapezoid Vertices		
Left Trapezoid	**Regular**	**Right**
A = -0.1 rad. = -5.73 deg.	A = -0.1 rad = -5.73 deg.	A = 0
B = 0	B = -0.03 = -1.72 deg.	B = 0.1 rad = 5.73 deg.
C = 0	C = 0.03 = 1.72 deg.	C = 0
D = 0	D = 0.1 rad = 5.73 deg.	D = 0

Membership Functions

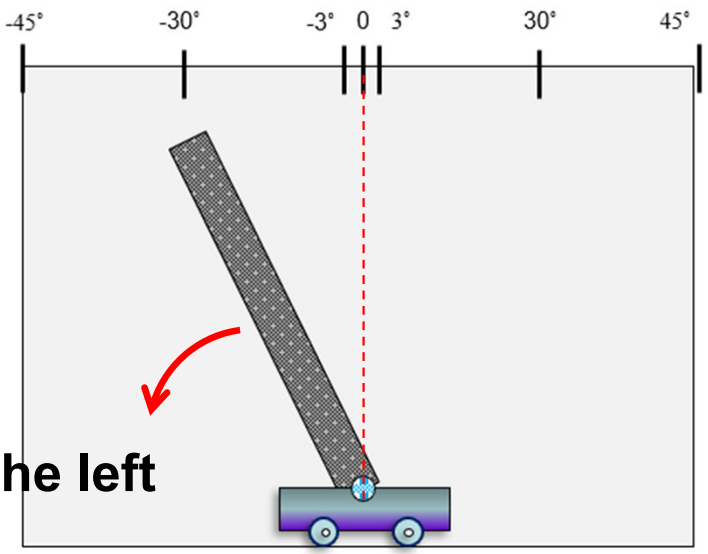
Membership Functions for the Pole Angle: Inclined to the Left (Negative)

Inclined to the Right (also referred to as Postivie angle)



Trapezoid Vertices		
Left Trapezoid	**Regular**	**Right**
A = -0.1 rad. = -5.73 deg.	A = -0.1 rad = -5.73 deg.	A = 0
B = 0	B = -0.03 = -1.72 deg.	B = 0.1 rad = 5.73 deg.
C = 0	C = 0.03 = 1.72 deg.	C = 0
D = 0	D = 0.1 rad = 5.73 deg.	D = 0

Fuzzy Sets



Falling to the left

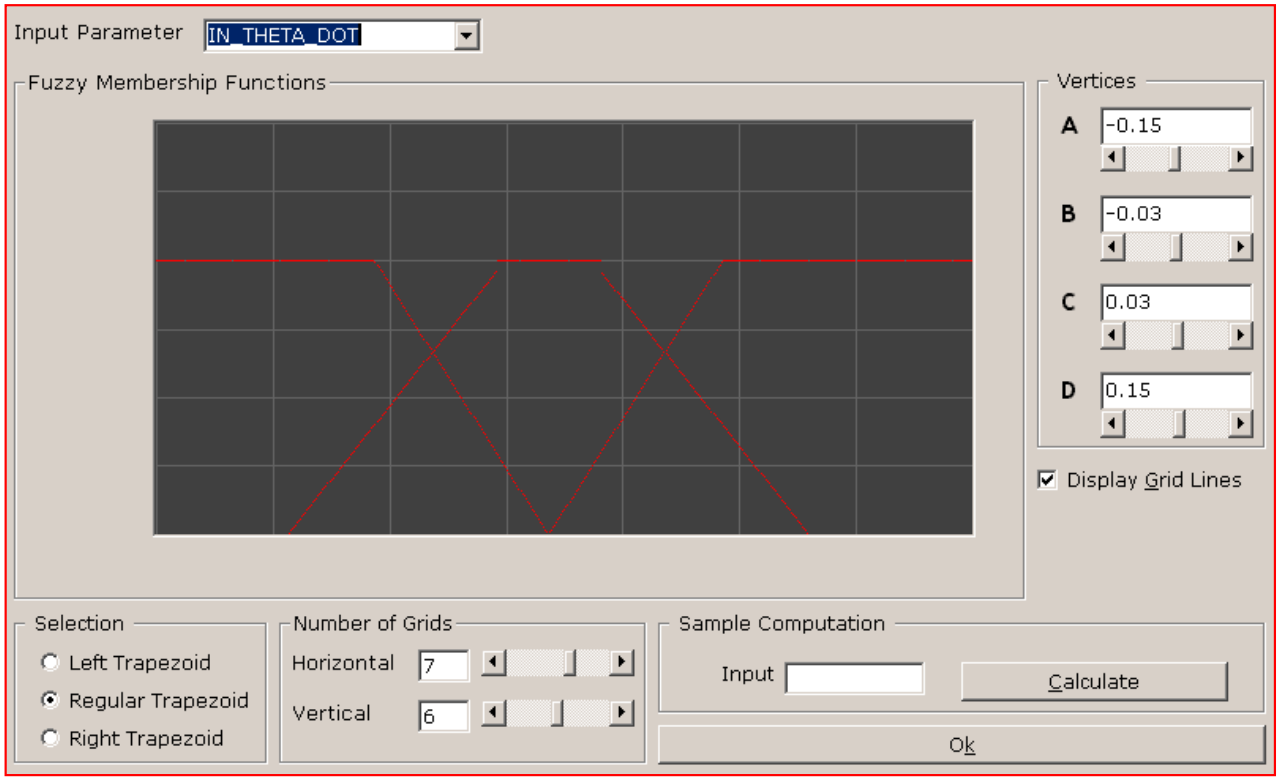
Note: the figure is not drawn to scale

The **angular velocity** could be described as:

1. Falling to the Left (N).
2. Still (Zero).
3. Falling to the Right (P).

Membership Functions

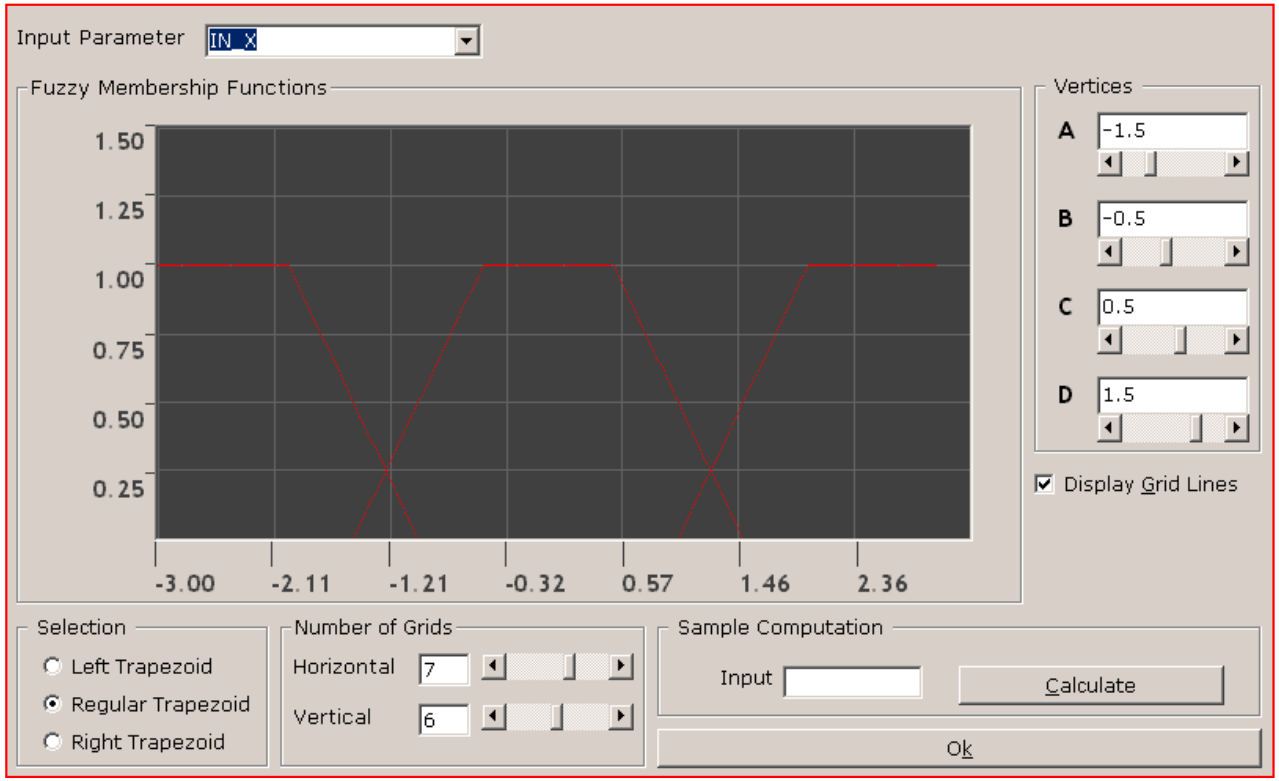
Membership Functions for the Pole Angular Velocity



Trapezoid Vertices		
Left Trapezoid	**Regular**	**Right**
A = -0.1=-5.73 deg/s.	A = -0.15 rad/s = -8.59 deg/s	A = 0
B = 0	B = -0.03 rad/s =-1.72 deg/s	B = 0.1 rad/s = 5.73 deg/s
C = 0	C = 0.03=1.72 deg/s	C = 0
D = 0	D = 0.15= 8.59 deg/s	D = 0

Membership Functions

Membership Functions for the Cart Position x

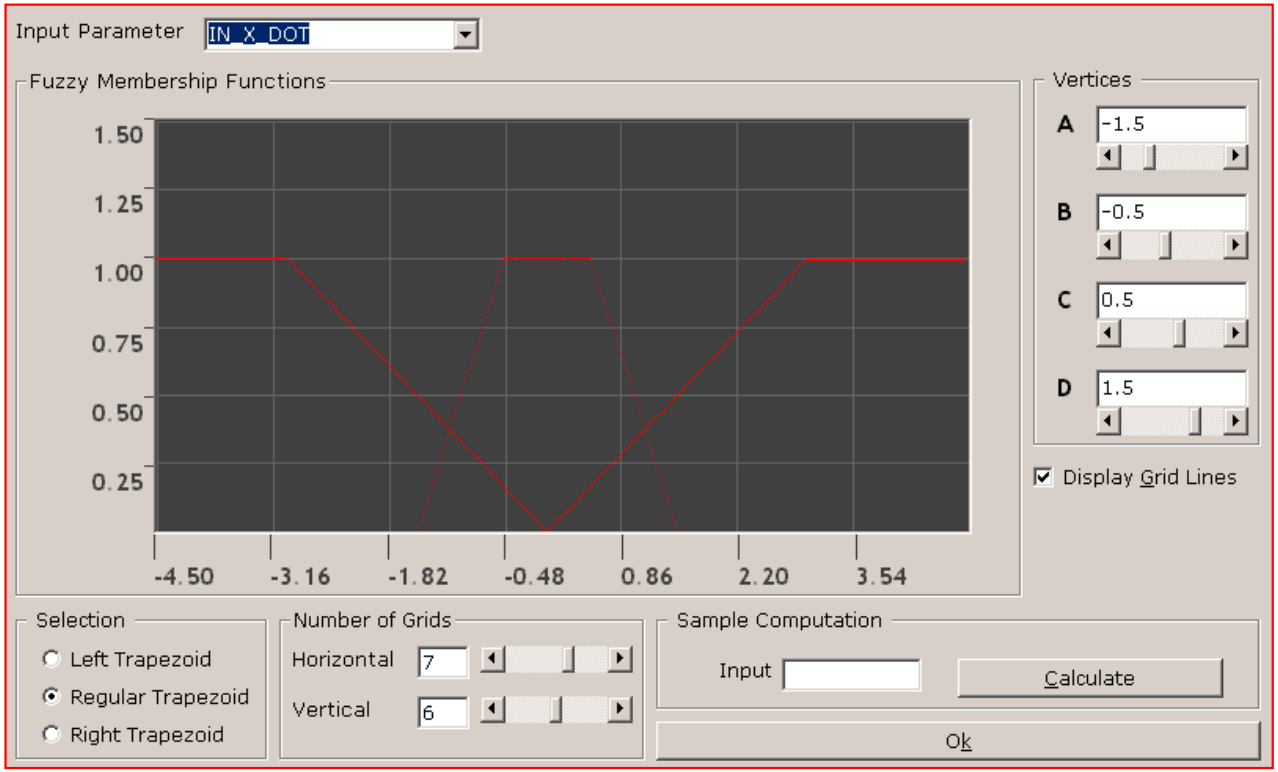


Take note of the position of the origin.

Trapezoid Vertices		
Left Trapezoid	**Regular**	**Right**
A = -2 m	A = -1.5 m	A = 1 m
B = -1 m	B = -0.5 m	B = 2 m
C = 0	C = 0.5 m	C = 0
D = 0	D = 1.5 m	D = 0

Membership Functions

Membership Functions for the Cart Velocity



Trapezoid Vertices		
Left Trapezoid	**Regular**	**Right**
A = -3 m/s	A = -1.5 m/s	A = 0
B = 0	B = -0.5 m/s	B = 3 m/s
C = 0	C = 0.5 m/s	C = 0
D = 0	D = 1.5 m/s	D = 0

FUZZY LOGIC SYSTEM

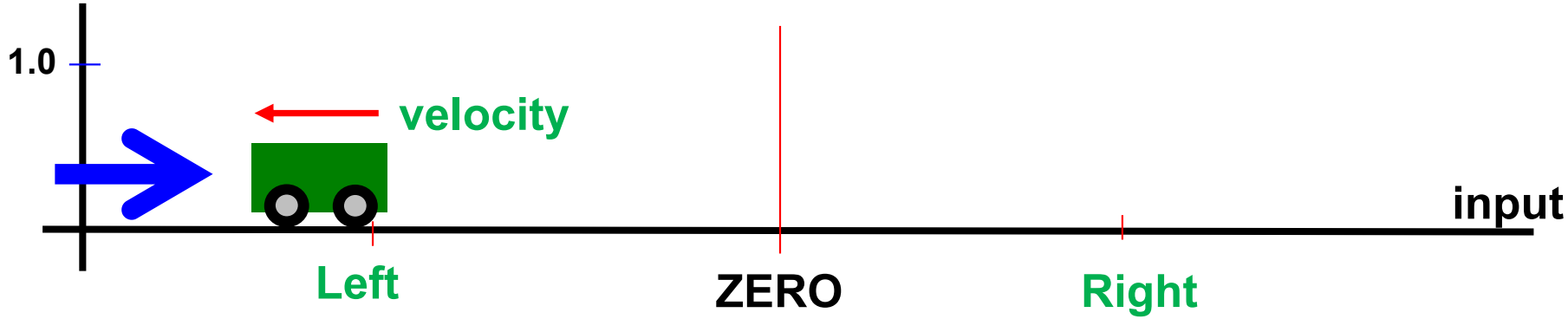
FAMM1: x vs. x'

Position vs. Velocity

n.h.reyes@massey.ac.nz

Fuzzy Rules (FAMM 1)

Fuzzy rule base and for the **position and velocity** vectors of the inverted pendulum-balancing problem



1. IF cart is on the left AND cart is going left THEN largely push cart to the right

Input X: {N, ZE, P} = {left, centre, right}

Input X': {N, ZE, P} = {going to the left, still, going to the right}

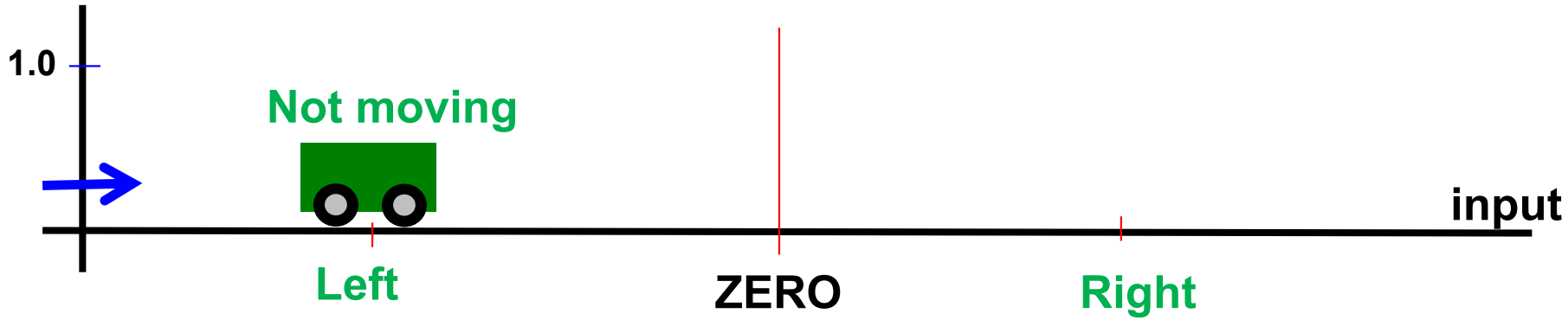
F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Fuzzy Rules (FAMM 1)

Fuzzy rule base and for the **position and velocity** vectors of the inverted pendulum-balancing problem



2. IF cart is on the left AND cart is not moving THEN slightly push cart to the right

Input X: {N, ZE, P} = {left, centre, right}

Input X': {N, ZE, P} = {going to the left, still, going to the right}

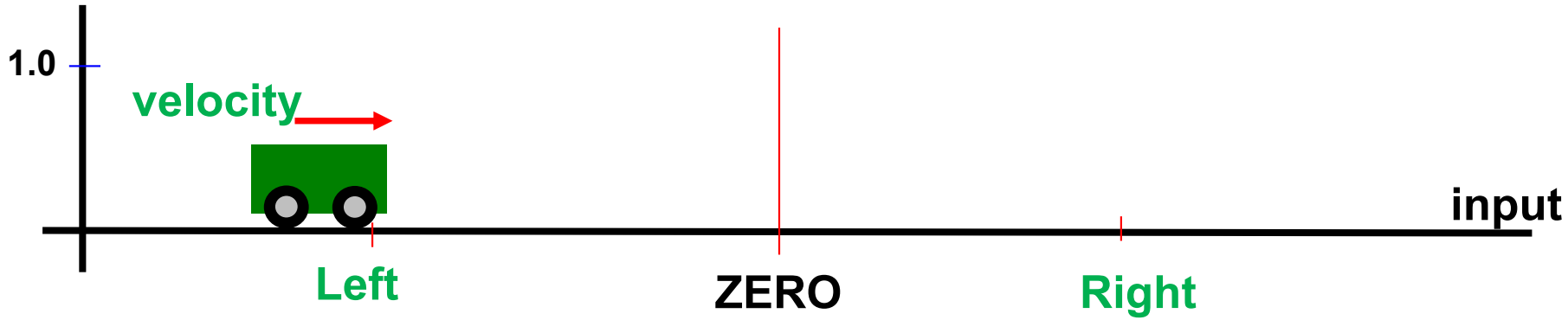
F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Fuzzy Rules (FAMM 1)

Fuzzy rule base and for the **position and velocity** vectors of the inverted pendulum-balancing problem



3. IF cart is on the left AND cart is going right THEN don't push cart

Input X: {N, ZE, P} = {left, centre, right}

Input X': {N, ZE, P} = {going to the left, still, going to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Fuzzy Rules (FAMM 1)

Fuzzy rule base and for the **position and velocity** vectors of the inverted pendulum-balancing problem

1. IF cart is on the left AND cart is going left THEN largely push cart to the right
2. IF cart is on the left AND cart is not moving THEN slightly push cart to the right
3. IF cart is on the left AND cart is going right THEN don't push cart
4. IF cart is centered AND cart is going left THEN slightly push cart to the right
5. IF cart is centered AND cart is not moving THEN don't push cart
6. IF cart is centered AND cart is going right THEN slightly push cart to the left
7. IF cart is on the right AND cart is going left THEN don't push cart
8. IF cart is on the right AND cart is not moving THEN push cart to the left
9. IF cart is on the right AND cart is going right THEN largely push cart to the left

Input X: {N, ZE, P} = {left, centre, right}

Input X': {N, ZE, P} = {going to the left, still, going to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Position vs. Velocity (FAMM 1)

If the cart is too near the end of the path, then regardless of the state of the pole angle push the cart towards the other end.

		X		
		N	ZE	P
	N	PL	PS	ZE
X'	ZE	PS	ZE	NS
	P	ZE	NS	NL

3 x 3 FAMM

Input X: {N, ZE, P} = {left, centre, right}

Input X': {N, ZE, P} = {going to the left, still, going to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

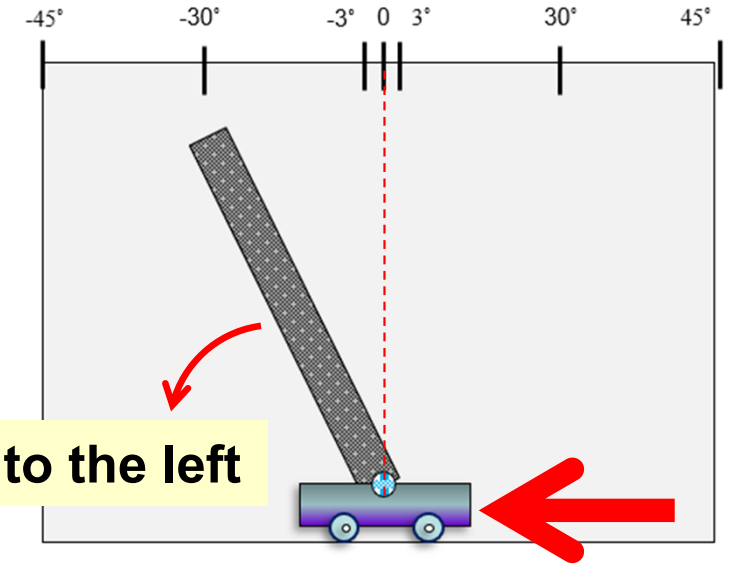
FUZZY LOGIC SYSTEM

FAMM2: θ vs. θ'

n.h.reyes@massey.ac.nz

Fuzzy Rules (FAMM 2)

Fuzzy rule base for the **angle and angular velocity** vectors of the inverted pendulum-balancing problem.



1. IF pole is leaning to the left AND pole is dropping to the left THEN largely push cart to the left

Input θ: {N, ZE, P}={leaning to the left, centre, leaning to the right}

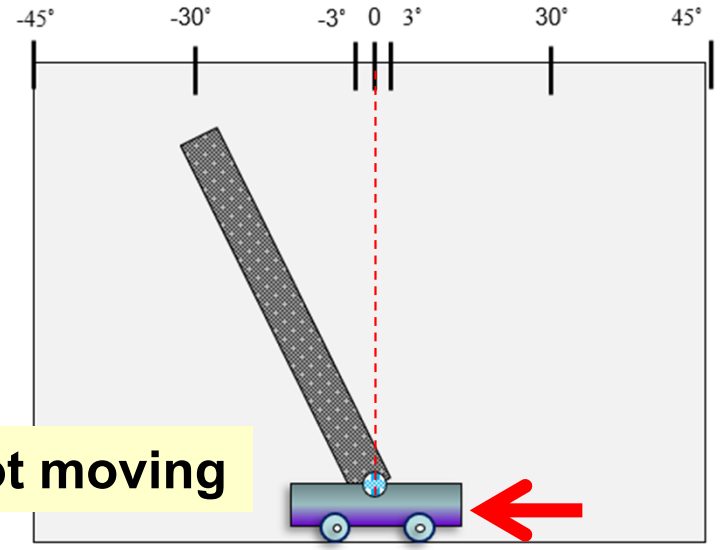
Input θ': {N, ZE, P}={moving to the left, still, moving to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back
menu

Fuzzy Rules (FAMM 2)

Fuzzy rule base for the **angle and angular velocity** vectors of the inverted pendulum-balancing problem.



Pole is not moving

Note: the figure is not drawn to scale

2. IF pole is leaning to the left AND pole is not moving THEN slightly push cart to the left

Input θ: {N, ZE, P}={leaning to the left, centre, leaning to the right}

Input θ': {N, ZE, P}={moving to the left, still, moving to the right}

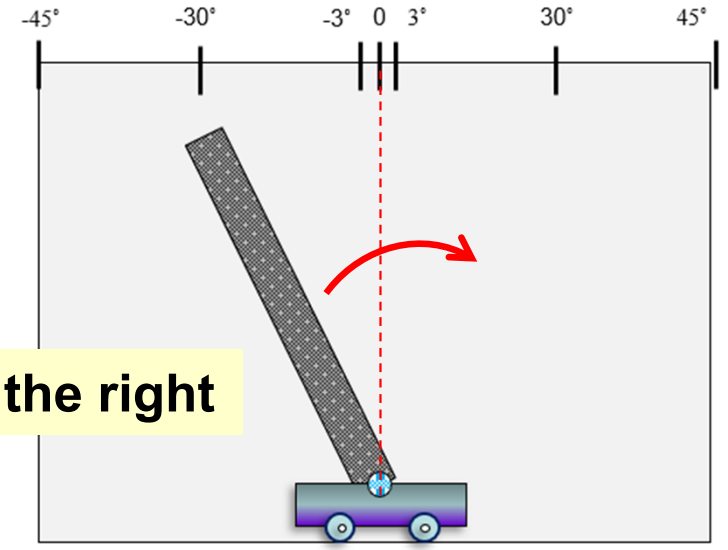
F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Fuzzy Rules (FAMM 2)

Fuzzy rule base for the **angle and angular velocity** vectors of the inverted pendulum-balancing problem.



Note: the figure is not drawn to scale

Pole is moving to the right

3. IF pole is leaning to the left AND pole is moving to the right THEN don't push the cart

Input θ: {N, ZE, P}={leaning to the left, centre, leaning to the right}

Input θ': {N, ZE, P}={moving to the left, still, moving to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Fuzzy Rules (FAMM 2)

Fuzzy rule base and the corresponding FAMM for the **angle and angular velocity** vectors of the inverted pendulum-balancing problem

1. IF pole is leaning to the left AND pole is dropping to the left THEN largely push cart to the left

2. IF pole is leaning to the left AND pole is not moving THEN slightly push cart to the left

3. IF pole is leaning to the left AND pole is moving to the right THEN don't push the cart

…. and so on, and so forth

……..

Input θ: {N, ZE, P}={leaning to the left, centre, leaning to the right}

Input θ': {N, ZE, P}={moving to the left, still, moving to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Angle vs. Angular Velocity (FAMM 2)

If the pole angle is too big or changing too quickly, then regardless of the location of the cart on the cart path, push the cart towards the direction it is leaning to.

		θ	
		N	ZE	P
	N	NL	NM	ZE
θ'	ZE	NS	ZE	PS
	P	ZE	PM	PL

3 x 3 FAMM

Input θ: {N, ZE, P}={leaning to the left, centre, leaning to the right}

Input θ': {N, ZE, P}={moving to the left, still, moving to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back

menu

Fuzzy Rule

If (θ is NEGATIVE **and** θ' is **NEGATIVE**)

Then NEGATIVELY_LARGE_FORCE.

$F_{SMALL}(\theta')$ = degree of membership of the given **ANGULAR VELOCITY** in the Fuzzy Set NEGATIVE

Could be a constant or another Membership Function

$F_{NEGATIVE}(\theta)$ = degree of membership of the given **ANGLE** in the Fuzzy Set NEGATIVE

Input θ: {N, ZE, P}={leaning to the left, centre, leaning to the right}

Input θ': {N, ZE, P}={dropping to the left, still, dropping to the right}

F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

(FAMM 1) and (FAMM 2)

The final output can be calculated by combining all the rule outputs from the two FAMMS, into one big centre of mass formula.

		X		
		N	ZE	P
	N	PL	PS	ZE
X'	ZE	PS	ZE	NS
	P	ZE	NS	NL

FAMM 1

		θ		
		N	ZE	P
	N	NL	NM	ZE
θ'	ZE	NS	ZE	PS
	P	ZE	PM	PL

FAMM 2

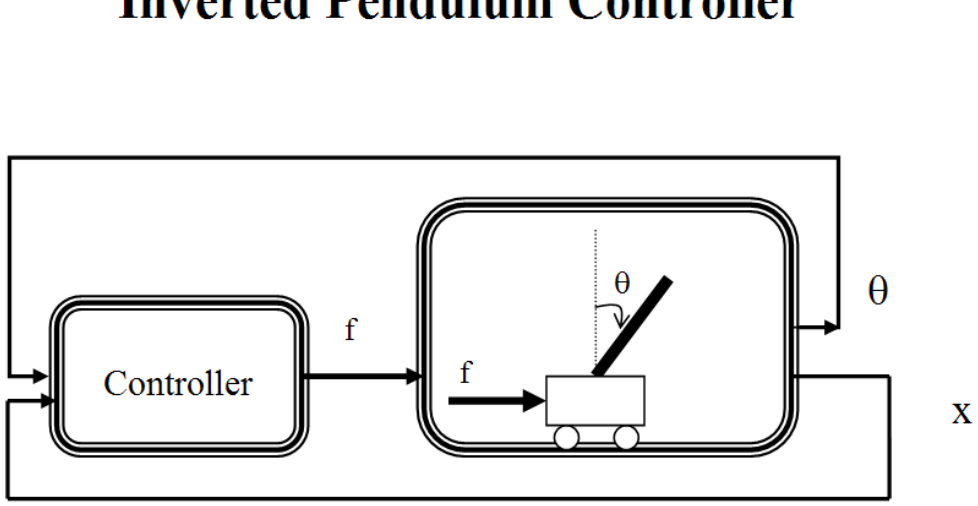
Final output

Force (magnitude and direction) for controlling the Inverted Pendulum

Fuzzy Controller for the Inverted Pendulum

Part 2

Recall: Inverted Pendulum Problem



Sensed values:

X – position of object with respect to the horizontal axis

θ - angle of pole relative to the vertical axis

Derived values:

X' - Velocity along the x-axis

θ' - Angular velocity

Input variables: sensed and derived values

Controller output: **F** – force to be applied to the cart

back

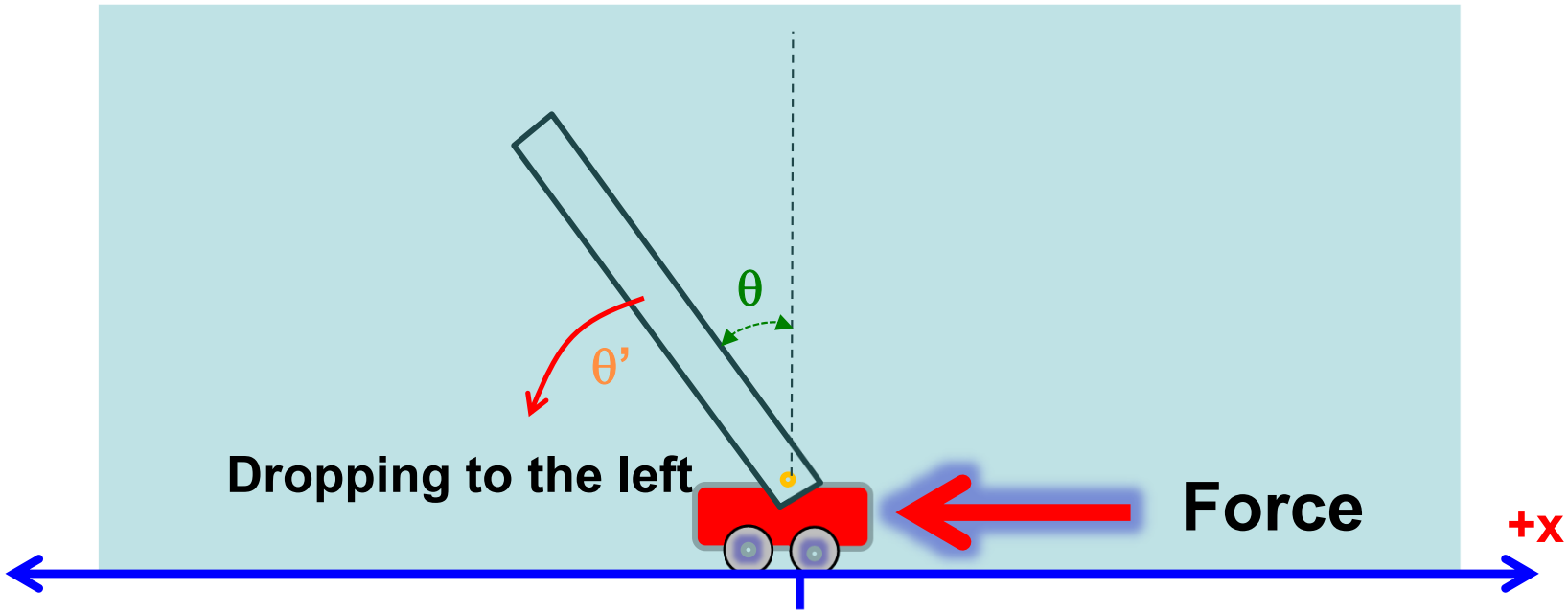
menu

Fuzzy Rule

e.g.

If (θ is **NEGATIVE** and θ' is **NEGATIVE**)

Then Apply a NEGATIVELY LARGE FORCE.



If (pole is leaning to the left AND pole is dropping to the left)
THEN largely push the cart to the left.

Fuzzy Rule

e.g.

If (θ is **NEGATIVE** **and** θ' is **NEGATIVE**)

Then Apply a NEGATIVELY LARGE FORCE.

The linguistic term **NEGATIVE** associated with the input angle is represented as a **fuzzy set**, named $\mathbf{F}_{\mathbf{NEGATIVE}\ \theta}$.

Fuzzy Rule

e.g.

If (θ is NEGATIVE **and** θ' is **NEGATIVE**)

Then NEGATIVELY LARGE FORCE.

The linguistic term **NEGATIVE** is represented as a **fuzzy set**, named $\mathbf{F}_{\mathbf{NEGATIVE}\ \theta}$.

On the other hand, the fuzzy set, $\mathbf{F}_{\mathbf{NEGATIVE}\ \theta}$, is implemented as a membership function, $\mathbf{F}_{\mathbf{NEGATIVE}}(\theta)$.

$\mathbf{F}_{\mathbf{NEGATIVE}}(\theta)$ = [0,1] computes for the degree of membership of any given input **angle** θ, in the fuzzy Set **NEGATIVE** θ

Fuzzy Rule

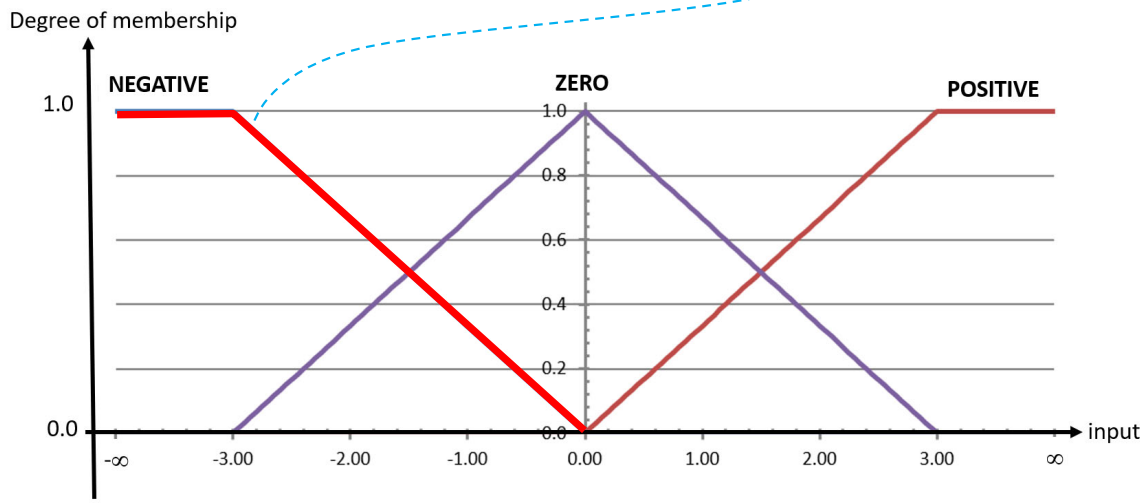
e.g.

If (θ is NEGATIVE **and** θ' is **NEGATIVE**)

Then NEGATIVELY_LARGE_FORCE.

$F_{NEGATIVE}(\theta)$ = [0,1] computes for the degree of membership of any given input **angle** θ, in the fuzzy Set **NEGATIVE** θ

The shape of this membership function, $F_{NEGATIVE}(\theta)$, may take the form of a trapezoid opening to the left.



back

menu

Fuzzy Rule

e.g.

If (θ is NEGATIVE **and** θ' is **NEGATIVE**)

Then NEGATIVELY LARGE FORCE.

The linguistic term **NEGATIVE** associated with the input, angular velocity θ' is represented as a **fuzzy set**, named $F_{\text{NEGATIVE } \theta'}$.

On the other hand, the fuzzy set, $F_{\text{NEGATIVE } \theta'}$, is implemented as a membership function, $F_{\text{NEGATIVE}}(\theta')$.

$F_{\text{NEGATIVE}}(\theta')$ = [0,1] computes for the degree of membership of any given input **anglular velocity θ'**, in the fuzzy Set **NEGATIVE θ'**

Fuzzy Rule

If (θ is NEGATIVE **and** θ' is **NEGATIVE**)

Then NEGATIVELY_LARGE_FORCE.

Could be a constant or another Membership Function

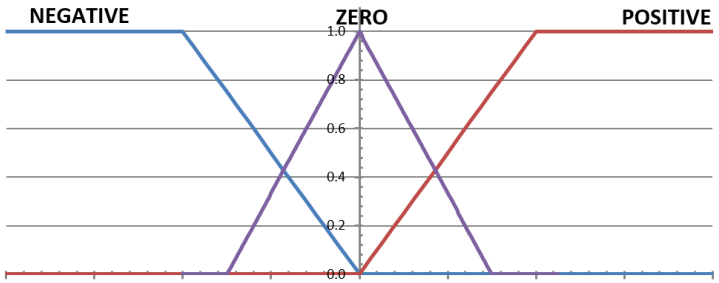
e.g. Negatively Large Force may be set to -100 N

Fuzzy Rule

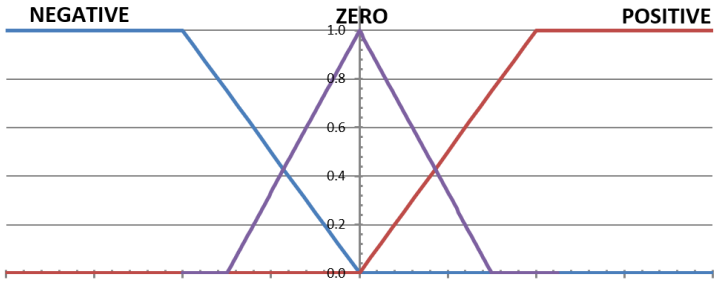
The linguistic terms should take into account the range of all possible values for each of the inputs.

Note: The range of input values and the shape of the membership functions are determined empirically

Input Angle θ: {N, ZE, P}={leaning to the left, centre, leaning to the right}



Input Angular velocity θ': {N, ZE, P}={dropping to the left, still, dropping to the right}



F: {PL, PS, ZE, NS, NL}={Large push to the right, small push to the right, don't push, small push to the left, Large push to the left**}**

back
menu

FUZZY LOGIC SYSTEM

Best solution
(Yamakawa's approach)

Reference journal article: Takeshi Yamakawa, "A Fuzzy Inference Engine in Nonlinear Analog Mode and its Application to a Fuzzy Logic Control"

n.h.reyes@massey.ac.nz

Inputs

We can reduce the number of rules further by combining related inputs together in a linear equation.

X = (A * theta) + (B * theta_dot)

Y = (C*x) + (D * x_dot)

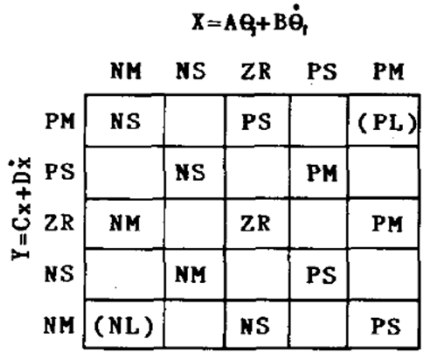
Note: A, B, C & D are determined empirically.

This approach requires only a single FAMM to solve the control problem.

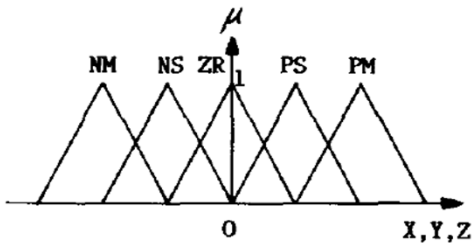
Fuzzy System Design

Yamakawa published his calibrated fuzzy rules and membership functions. Out of the 25 rules possible, he is only using 13 of them.

Fuzzy rules



(a)



(b)

Fig. 29. (a) A rule map including fuzzy rules for stabilizing a glass with wine. (b) Membership functions which characterize the fuzzy linguistic terms used in the rule map.

- 2 inputs, 5 fuzzy sets per input, 5x5=25 possible rules
- 3 inputs, 5 fuzzy sets per input, 5x5x5=125 possible rules
- 4 inputs, 5 fuzzy sets per input, 5x5x5x5=625 possible rules

X = (A * theta) + (B * theta_dot)

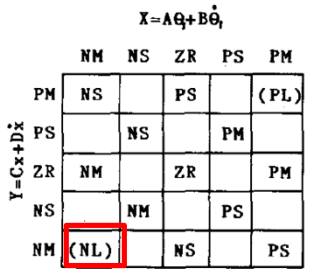
Y = (C*x) + (D * x_dot)

menu

Fuzzy System Design

Here are some graphical interpretations of some of the rules.

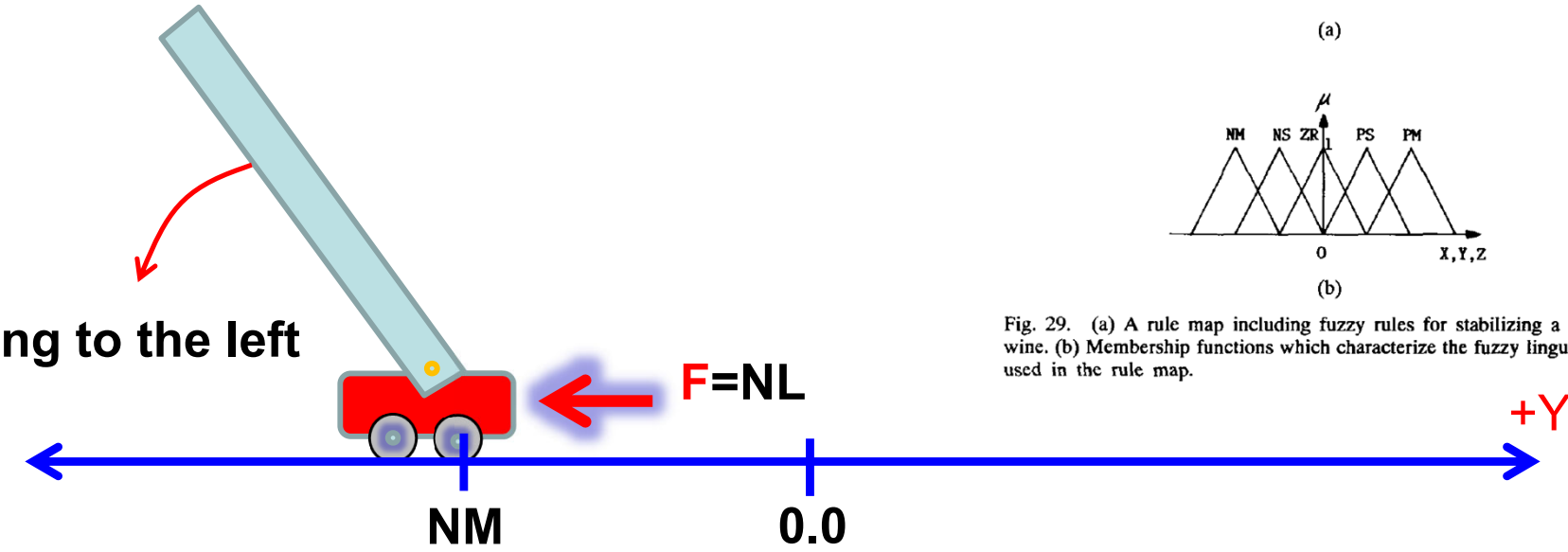
Rule: If (X is NM) and (Y is NM) THEN F = NL.



(a)

(b)

Fig. 29. (a) A rule map including fuzzy rules for stabilizing a glass with wine. (b) Membership functions which characterize the fuzzy linguistic terms used in the rule map.



X = (A * theta) + (B * theta_dot)

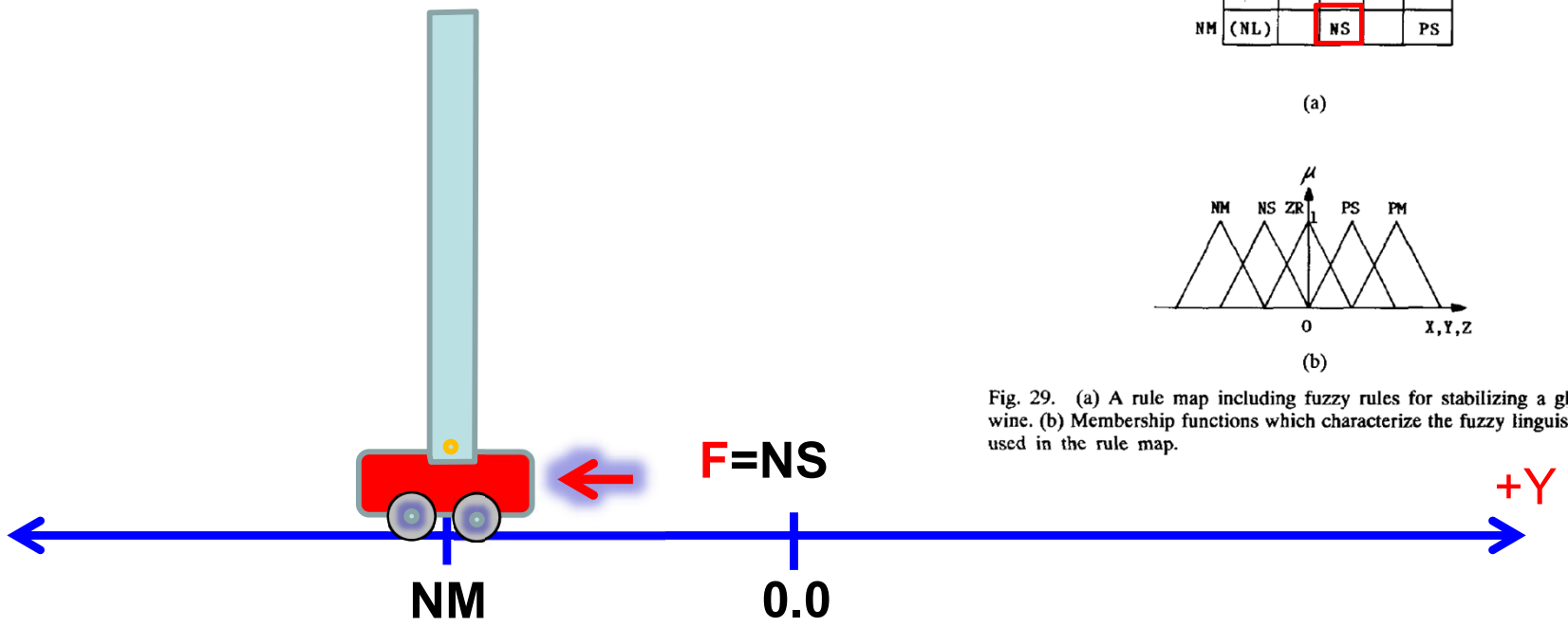
Y = (C*x) + (D * x_dot)

menu

Fuzzy System Design

Here are some graphical interpretations of some of the rules.

Rule: If (X is ZE) and (Y is NM) THEN F = NS.



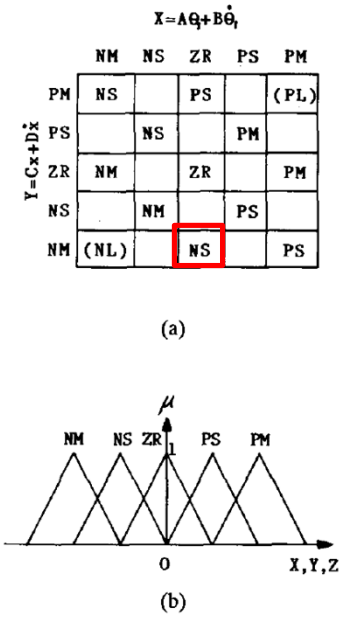


Fig. 29. (a) A rule map including fuzzy rules for stabilizing a glass with wine. (b) Membership functions which characterize the fuzzy linguistic terms used in the rule map.

X = (A * theta) + (B * theta_dot)

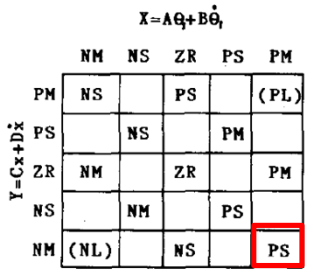
Y = (C*x) + (D * x_dot)

menu

Fuzzy System Design

Here are some graphical interpretations of some of the rules.

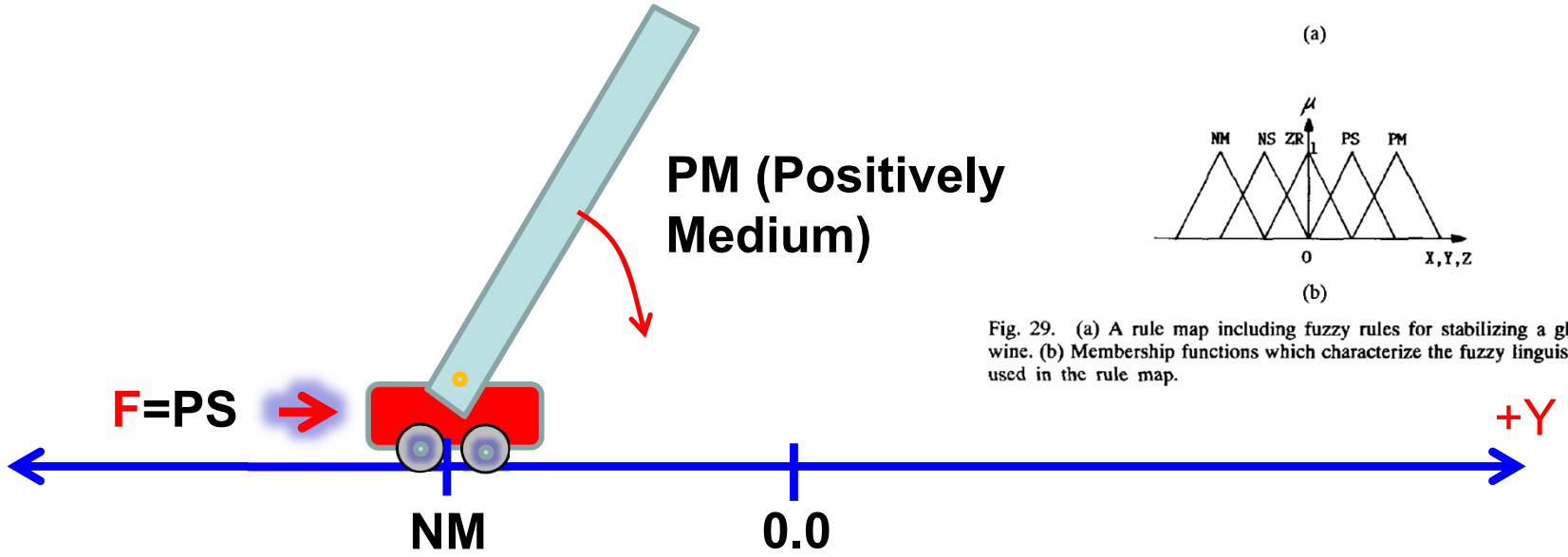
Rule: If (X is PM) and (Y is NM) THEN F = PS.



(a)

(b)

Fig. 29. (a) A rule map including fuzzy rules for stabilizing a glass with wine. (b) Membership functions which characterize the fuzzy linguistic terms used in the rule map.



X = (A * theta) + (B * theta_dot)

Y = (C*x) + (D * x_dot)

menu

Hand-Simulation

Fuzzy Control

Different stages of Fuzzy control

1. Fuzzification

Input variables are assigned degrees of membership in various classes

e.g. A temperature input might be graded according to its degree of coldness, coolness, warmth or heat.

The purpose of **fuzzification** is to **map the inputs** from a set of sensors (or features of those sensors) **to values from 0 to 1** using a set of input membership functions.

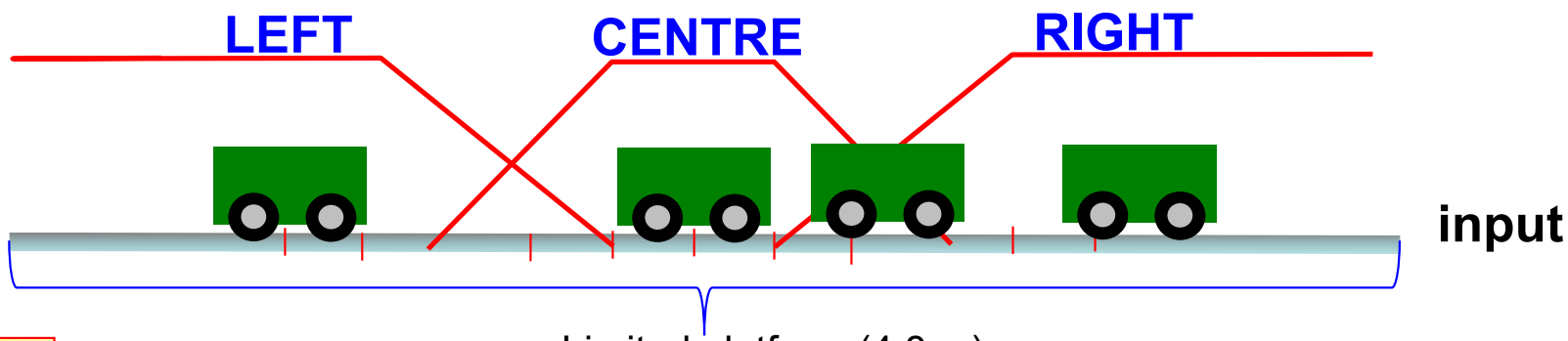
Membership Function: $F_{Set}(x) = [0, 1]$ = *assigns a degree of membership of x, in the fuzzy set defined.*

e.g. $F_{NEAR}(x=0.2) = [0, 1]$

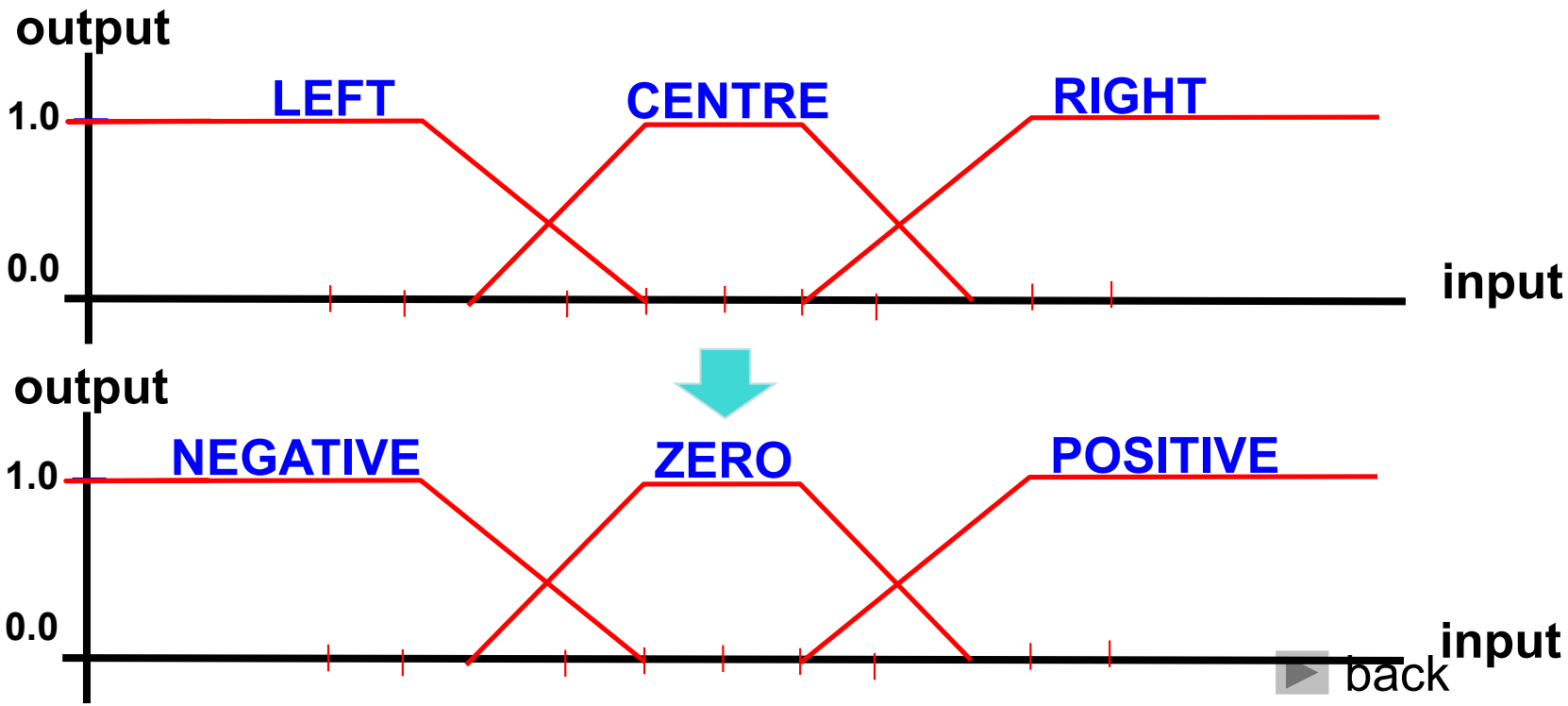
We will see a complete example of the steps involved later.

Fuzzification

Let's consider one of the inputs, x (position of cart):



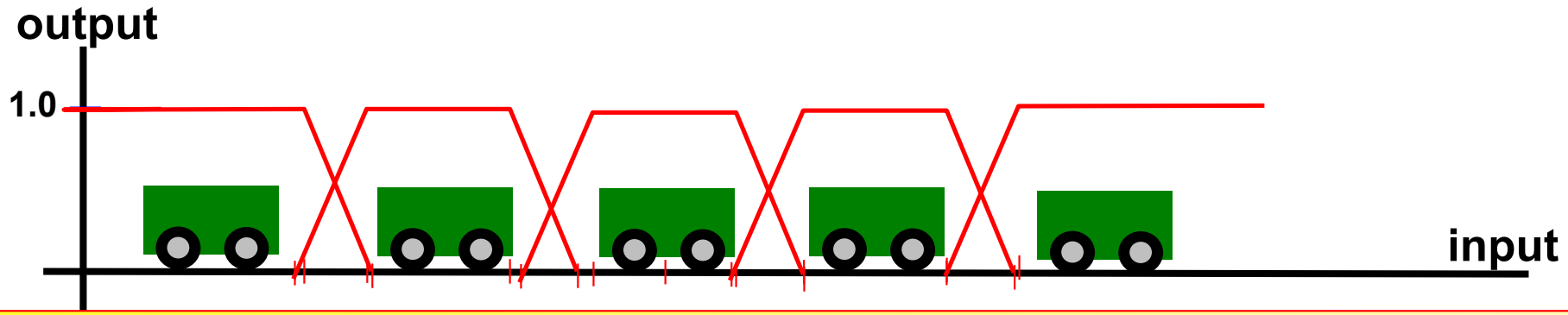
Input x



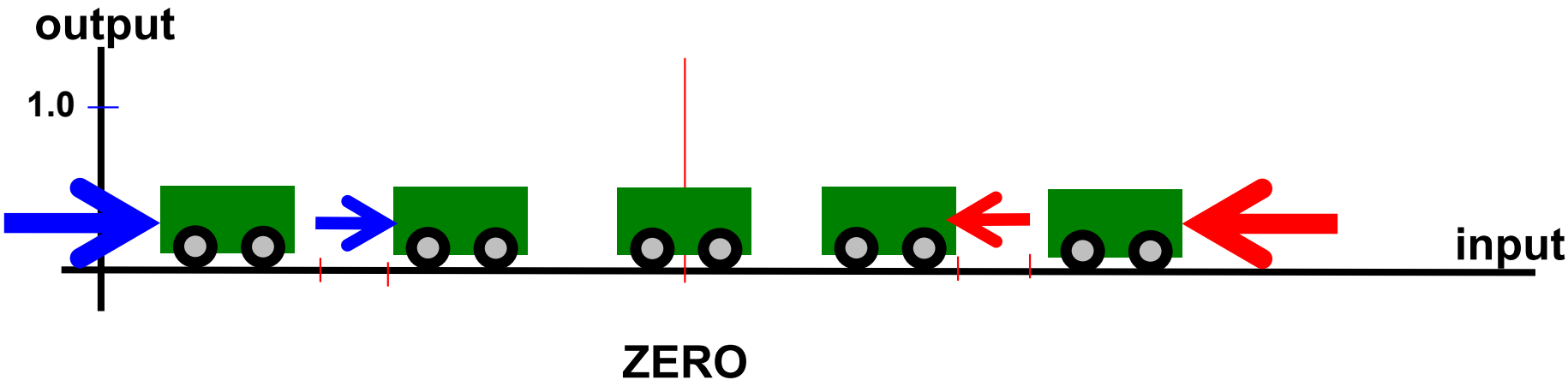
back

menu

Fuzzification



By defining more fuzzy sets, more accurate control rules can be defined.



As desired, we can have more rule outputs:

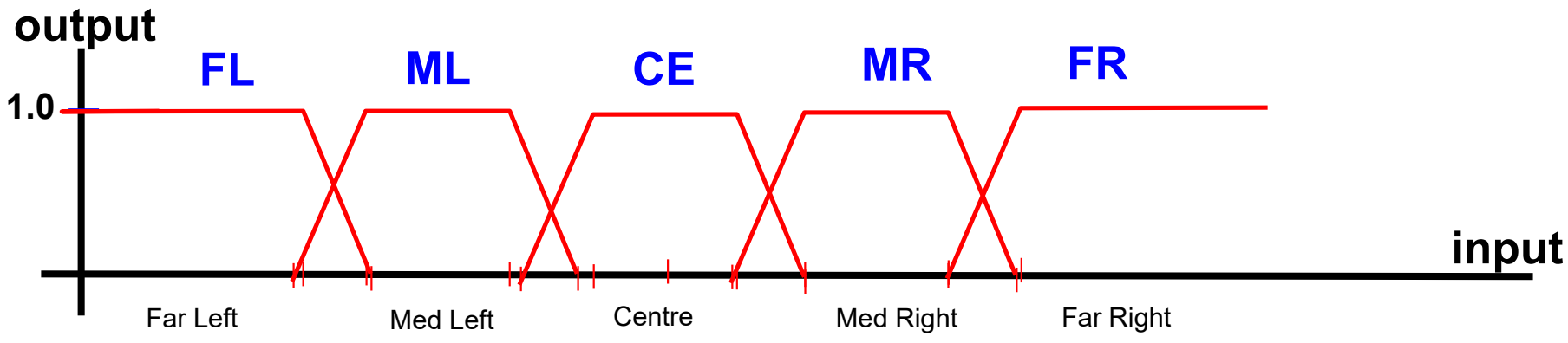
PL PM PS NS NM NL

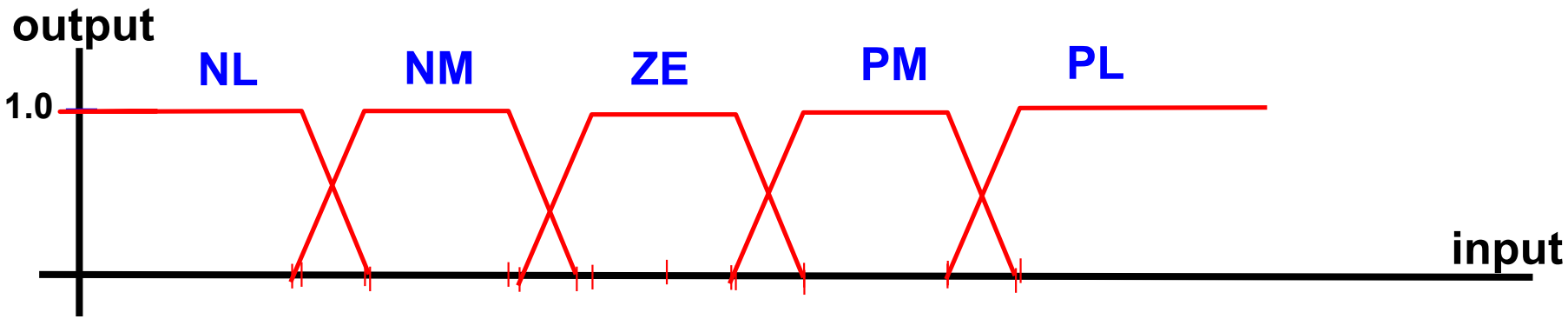
back

menu

Fuzzification

We can describe the position of the cart using the following 5 fuzzy sets.





These terms are more commonly used in the literature.

- (next) Derivation of formulas

Assignment #2

Yamakawa's Fuzzy Controller

INSTRUCTIONS

DOCUMENTATION GUIDE

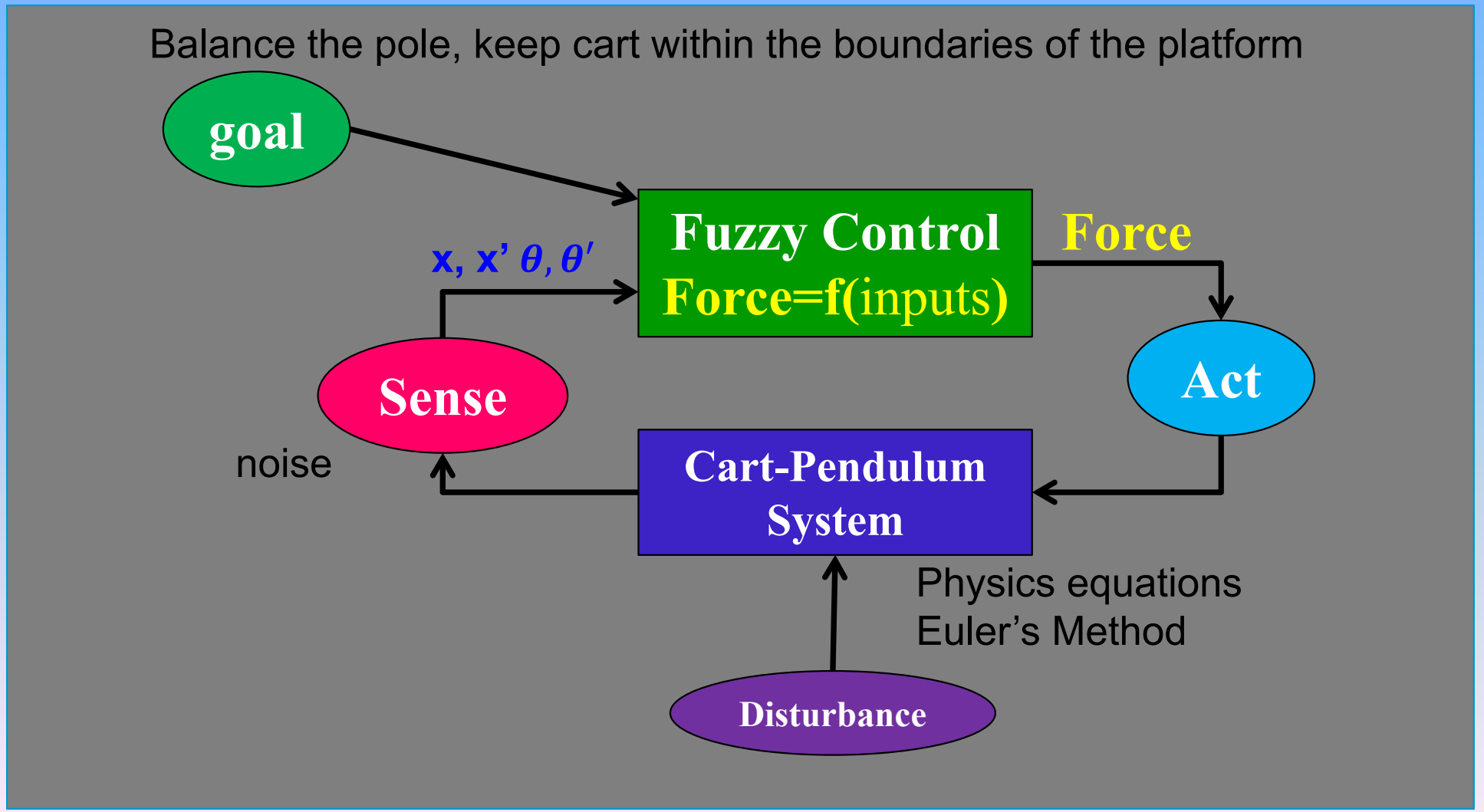
MEMBERSHIP FUNCTION VIEWER

CHECKLIST

n.h.reyes@massey.ac.nz

Feedback Control/Closed Loop Control

- Closed-loop control allows for **uncertainty** in the model as well as **noise** and **disturbances** in the system under control



Solve the tutorial first, before studying this lecture.

Fuzzy Logic Tutorial

2 Inputs, 9 rules

Fuzzy Controller

Start-up codes

n.h.reyes@massey.ac.nz

SEARCH

Start-up Codes: makefile

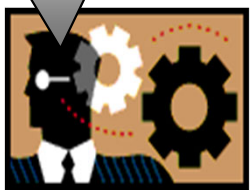
The start-up system is comprised of multiple files.

Executable filename

```
main.exe       :   main.o transform.o sprites.o fuzzylogic.o graphics.o
    g++ -O2 -std=c++11  -Wl,-s -o main.exe main.o transform.o sprites.o fuzzylogic.o graphics.o -l gdi32 -static-libgcc -static-libstdc++

main.o      :   main.cpp graphics.h transform.h sprites.h fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings main.cpp

transform.o     :    transform.cpp transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings transform.cpp

sprites.o       :    sprites.cpp sprites.h graphics.h transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings sprites.cpp

fuzzylogic.o    :    fuzzylogic.cpp fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings fuzzylogic.cpp

graphics.o      :    graphics.cpp graphics.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space -Wno-write-strings  graphics.cpp

clean:
    del *.o
    del *.exe
```

Indented statements must be preceded by a **tab**.

The program requires that you "**build**" it using a **makefile**.



back

SEARCH

Start-up Codes: makefile

```
main.exe        :   main.o transform.o sprites.o fuzzylogic.o graphics.o
    g++ -O2 -std=c++11  -Wl,-s -o main.exe main.o transform.o sprites.o fuzzylogic.o graphics.o -l gdi32 -static-libgcc -static-libstdc++

main.o      :   main.cpp graphics.h transform.h sprites.h fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings main.cpp

transform.o     :    transform.cpp transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings transform.cpp

sprites.o       :    sprites.cpp sprites.h graphics.h transform.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings sprites.cpp

fuzzylogic.o    :    fuzzylogic.cpp fuzzylogic.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space  -Wno-write-strings fuzzylogic.cpp

graphics.o      :    graphics.cpp graphics.h
    g++ -O2  -std=c++11  -c -fpermissive -fconserve-space -Wno-write-strings  graphics.cpp

clean:
    del *.o
    del *.exe
```

The program uses a simple graphics library (**graphics.cpp and graphics.h**) and links to the built-in graphics device interface (**gdi32**) of the Windows operating system. Because of this, the program only compiles in the Windows environment.

Fuzzy Logic

Variations of FIS

n.h.reyes@massey.ac.nz

More details…

- Fuzzy Inference Systems (FIS)
- Fuzzy Sets
- Fuzzy Combination Operators
- Membership functions
- Implication and Aggregation Operators
- Defuzzification Methods

Fuzzy Inference

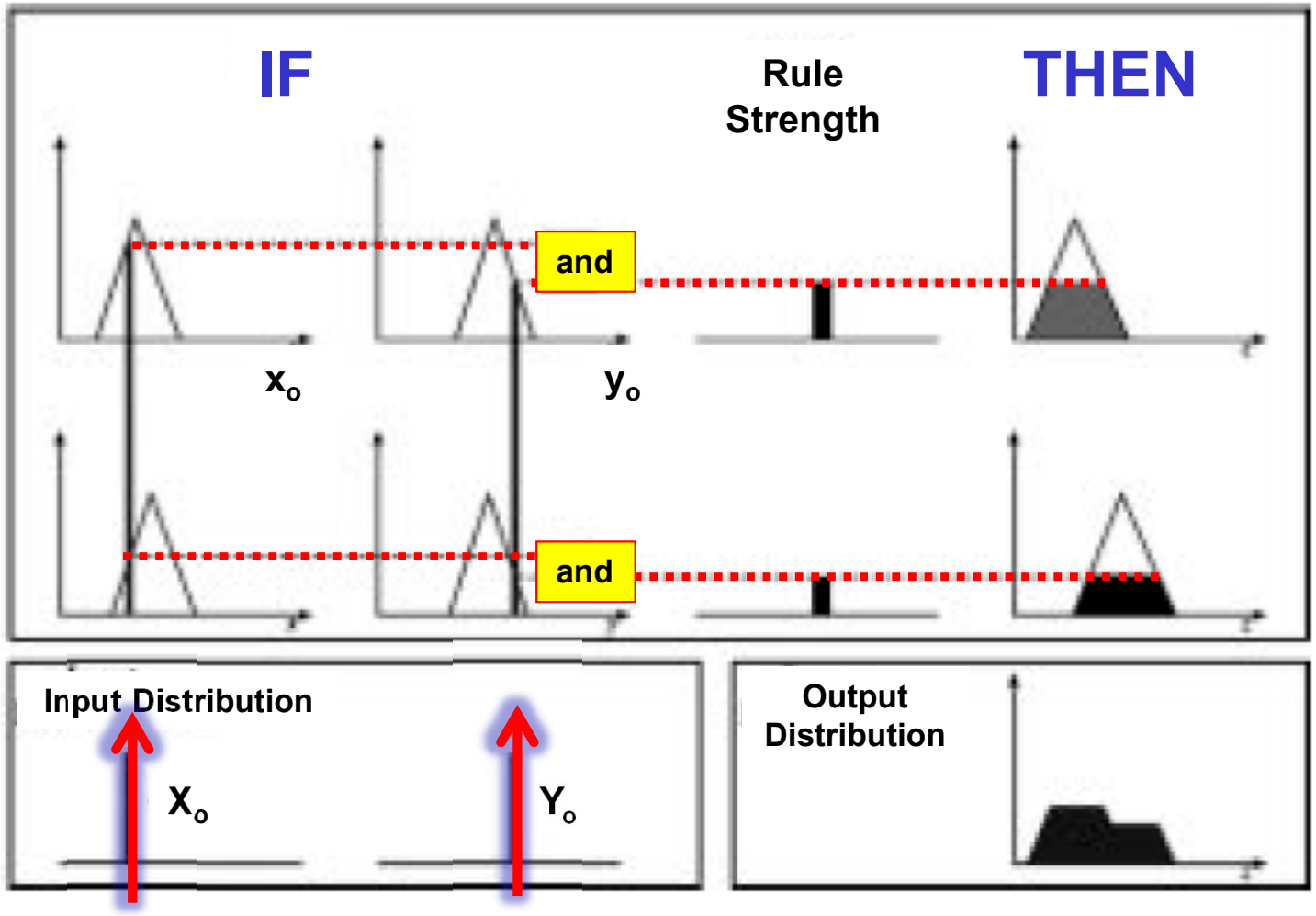
Fuzzy inference is the process of formulating the mapping from a given input to an output using fuzzy logic. The mapping then provides a basis from which decisions can be made, or patterns discerned. The process of fuzzy inference involves all of the pieces that were described in the previous sections: Membership Functions, Logical Operations, and If-Then Rules.

http://www.mathworks.com/help/fuzzy/fuzzy-inference-process.html

Mamdani Fuzzy Inference System

Mamdani Inference System

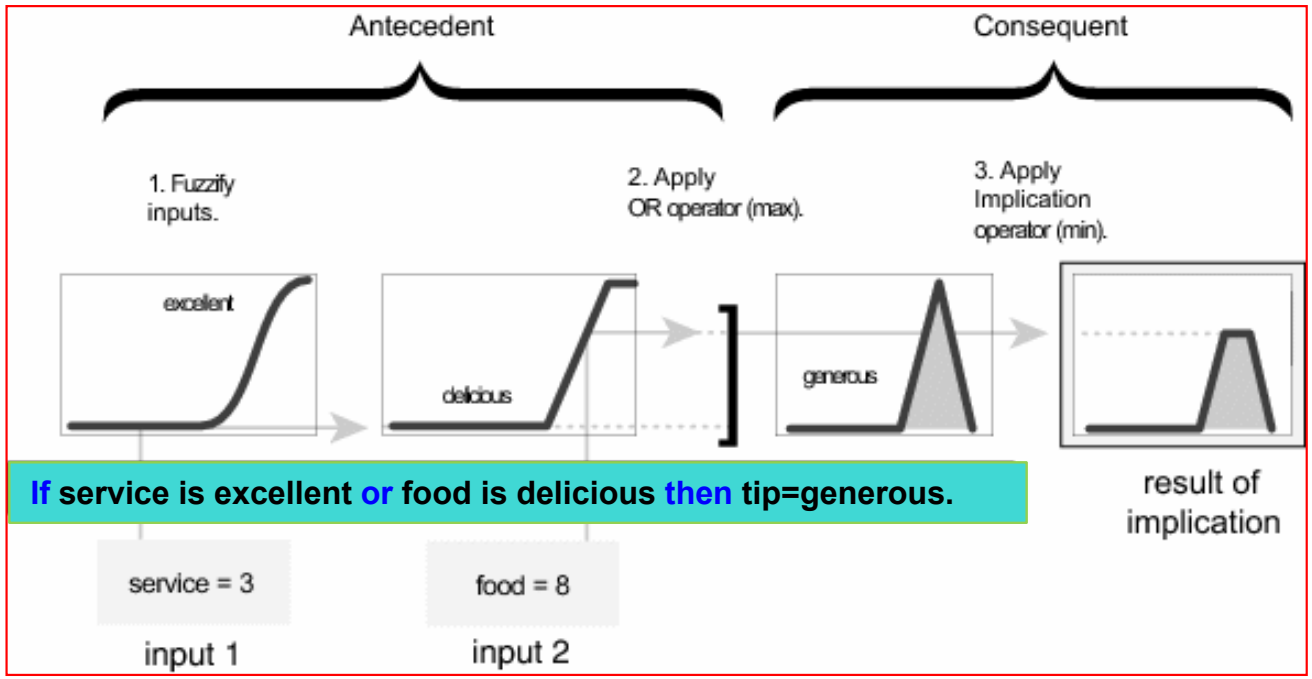
Two input, two rule Mamdani FIS with crisp inputs



Fuzzy rules are a collection of linguistic statements that describe how the FIS should make a decision regarding classifying an input or controlling an output.

Mamdani FIS

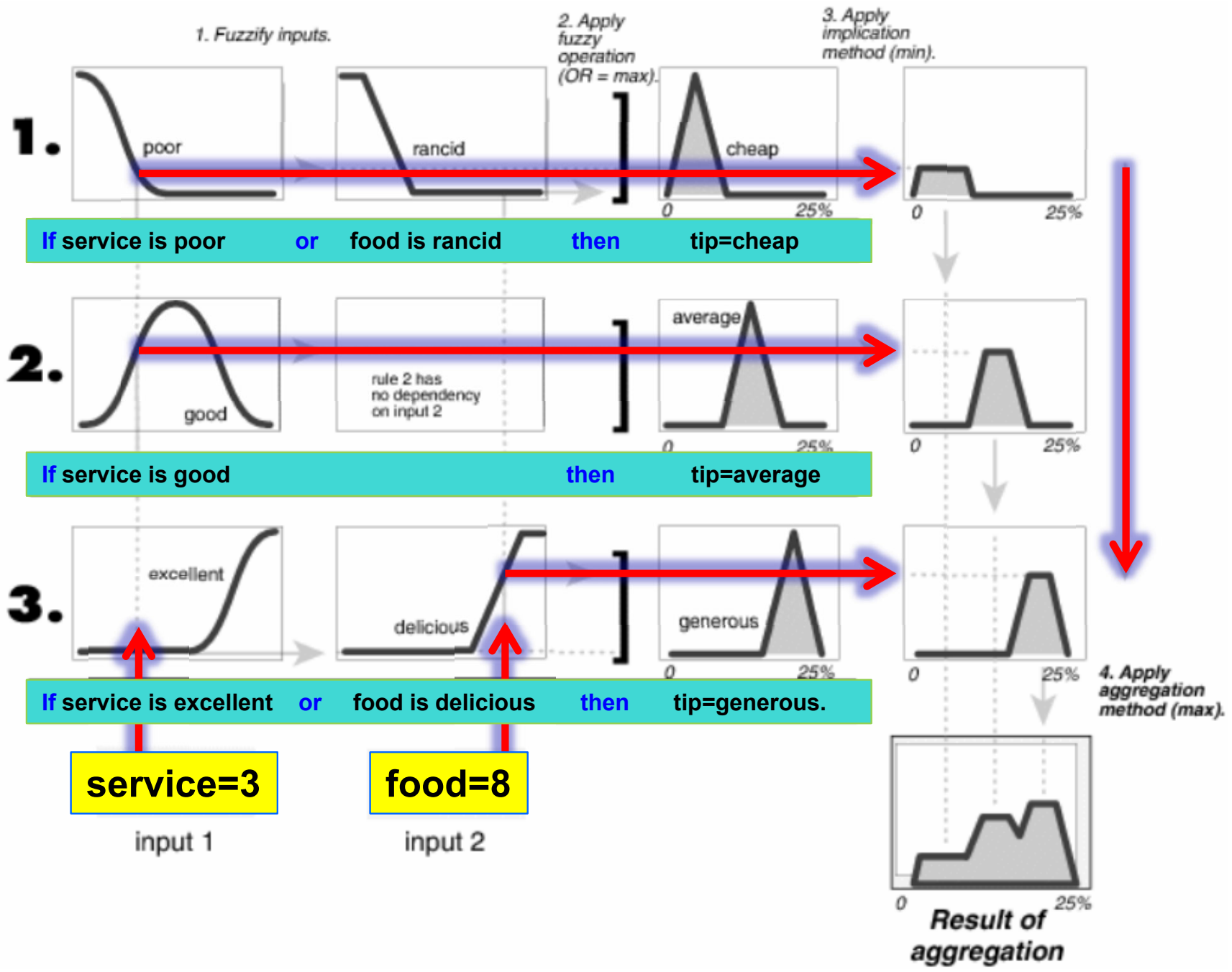
Mamdani-type inference, expects the output membership functions to be fuzzy sets. After the aggregation process, there is a fuzzy set for each output variable that needs defuzzification.



http://www.mathworks.com/access/helpdesk/help/toolbox/fuzzy/index.html?/access/helpdesk/help/toolbox/fuzzy/bp78l6_-1.html&http://www.mathworks.com/cgi-bin/texis/webinator/search/?db=MSS&prox=page&rorder=750&rprox=750&rdfreq=500&rwfreq=500&rlead=250&sufs=0&order=r&is_summary_on=1&ResultCount=10&query=fuzzy+logic&submitButtonName=Search

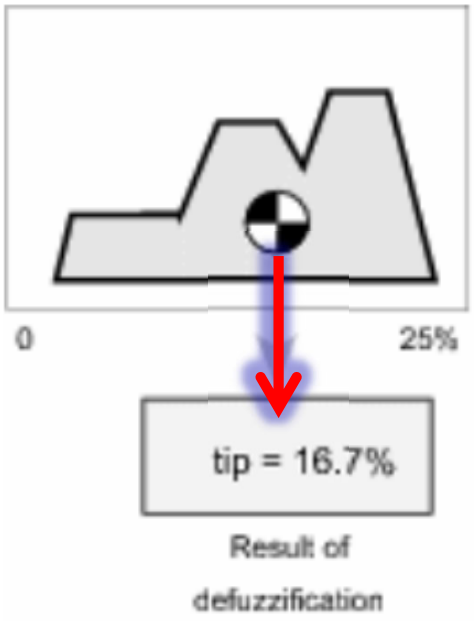
back
menu

Mamdani FIS



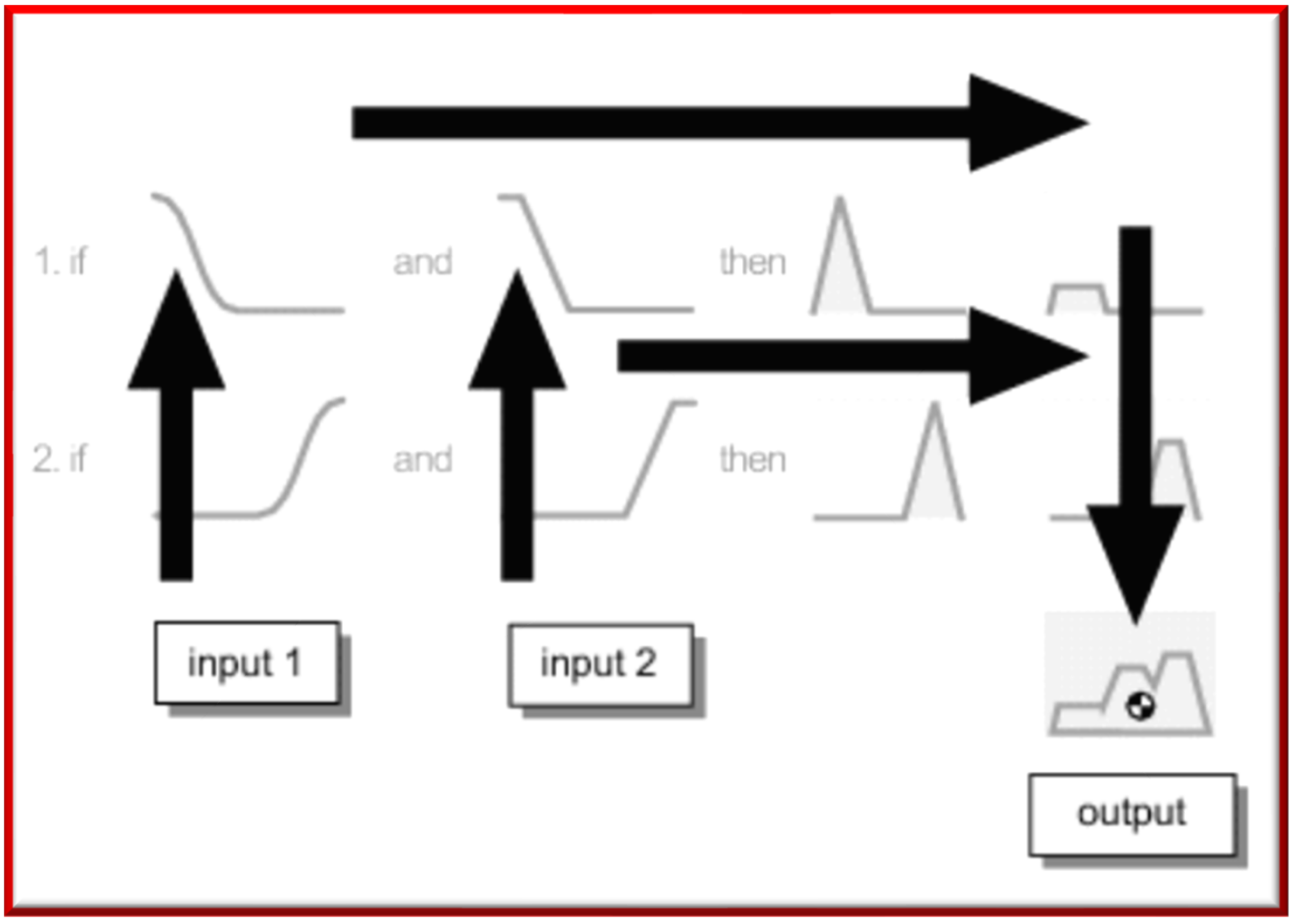
menu

Mamdani FIS



5. Defuzzify the aggregate output (centroid).

Flow of Fuzzy Inference



In this figure, the flow proceeds up from the inputs in the lower left, then across each row, or rule, and then down the rule outputs to finish in the lower right. This compact flow shows everything at once, from linguistic variable fuzzification all the way through defuzzification of the aggregate output.

Mamdani FIS

OUTPUT MEMBERSHIP FUNCTION

- It is possible, and in many cases much more efficient, to use a **single spike** as the **output membership function** rather than a distributed fuzzy set.

- This type of output is sometimes known as a **singleton output membership function**, and it can be thought of as a **pre-defuzzified fuzzy set**.

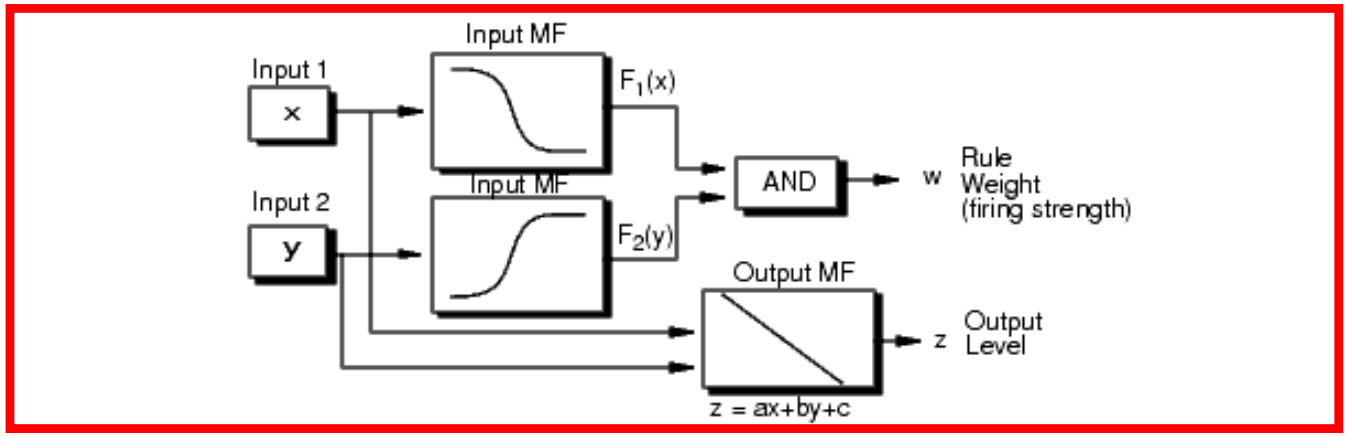
- It enhances the efficiency of the defuzzification process because it **greatly simplifies the computation** required by the more general Mamdani method, which finds the **centroid of a 2-D function**.

- Rather than integrating across the two-dimensional function to find the centroid, you use the **weighted average of a few data points**.

Sugeno Fuzzy Inference System

Sugeno FIS

Sugeno FIS is similar to the Mamdani method in many respects. The first two parts of the fuzzy inference process, fuzzifying the inputs and applying the fuzzy operator, are exactly the same. The main difference between Mamdani and Sugeno is that the Sugeno output membership functions are either linear or constant.



A typical rule in a Sugeno fuzzy model has the form:

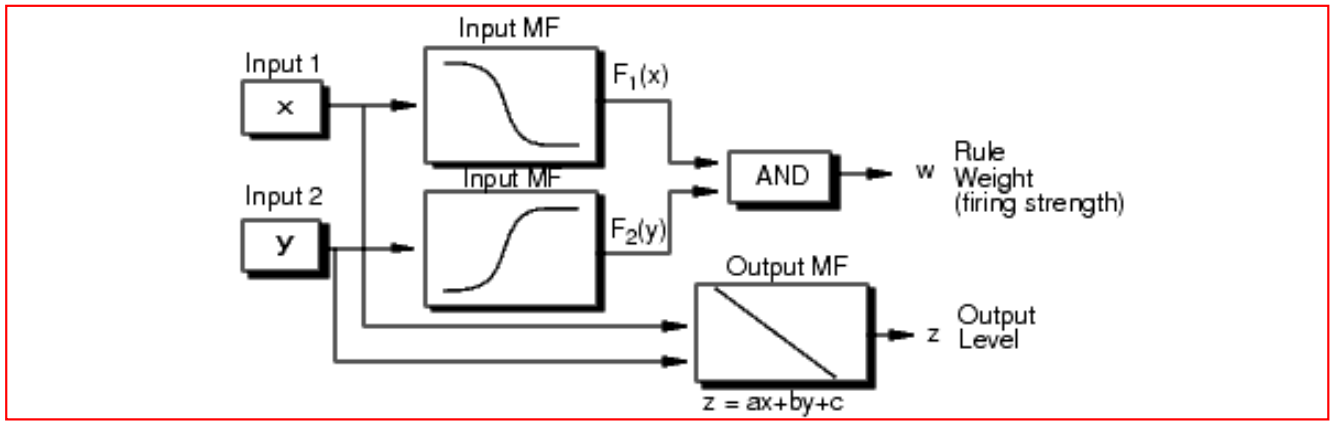
If (Input 1 = x and Input 2 = y) **then** Output is **z = ax + by + c**

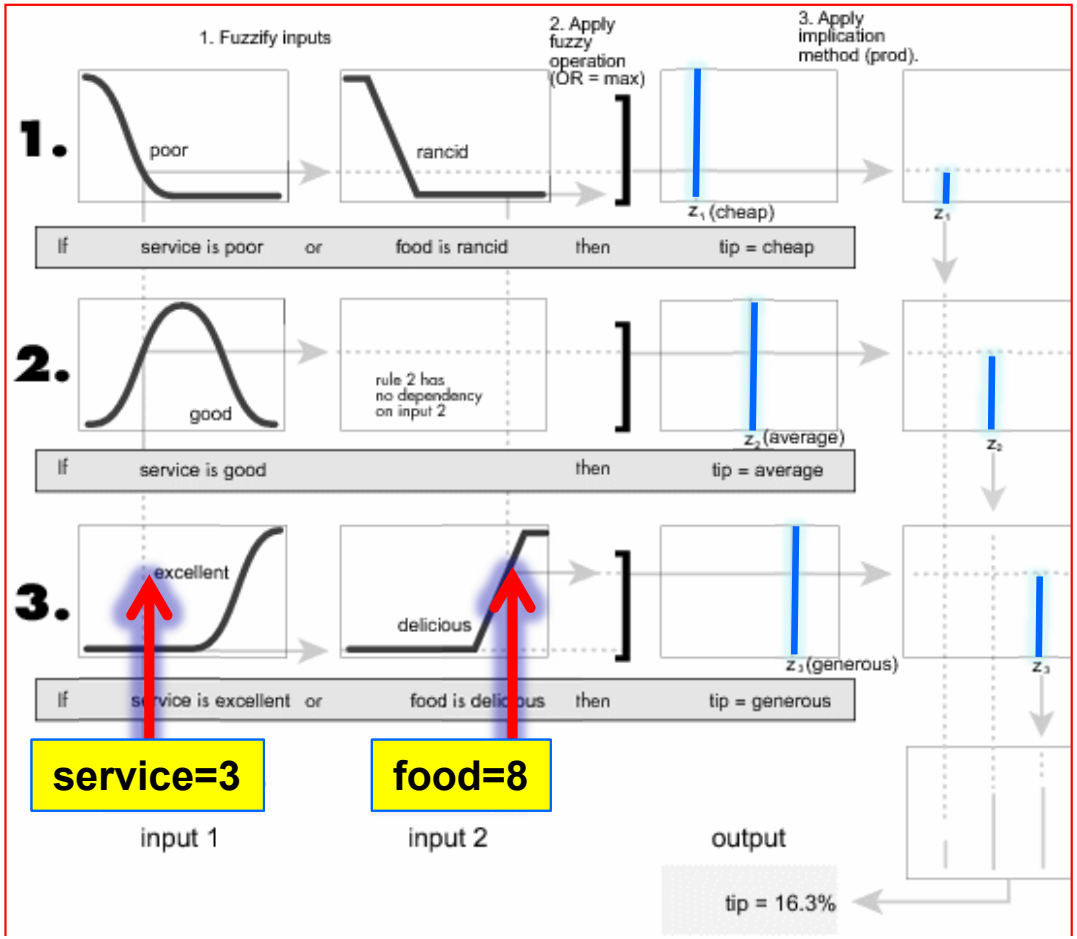
For a **zero-order Sugeno model**, the output **z** is a **constant** (where a=b=0).

back

menu

Sugeno FIS





$$\text{Final Output} = \frac{\sum_{i=1}^{N} w_i z_i}{\sum_{i=1}^{N} w_i}$$

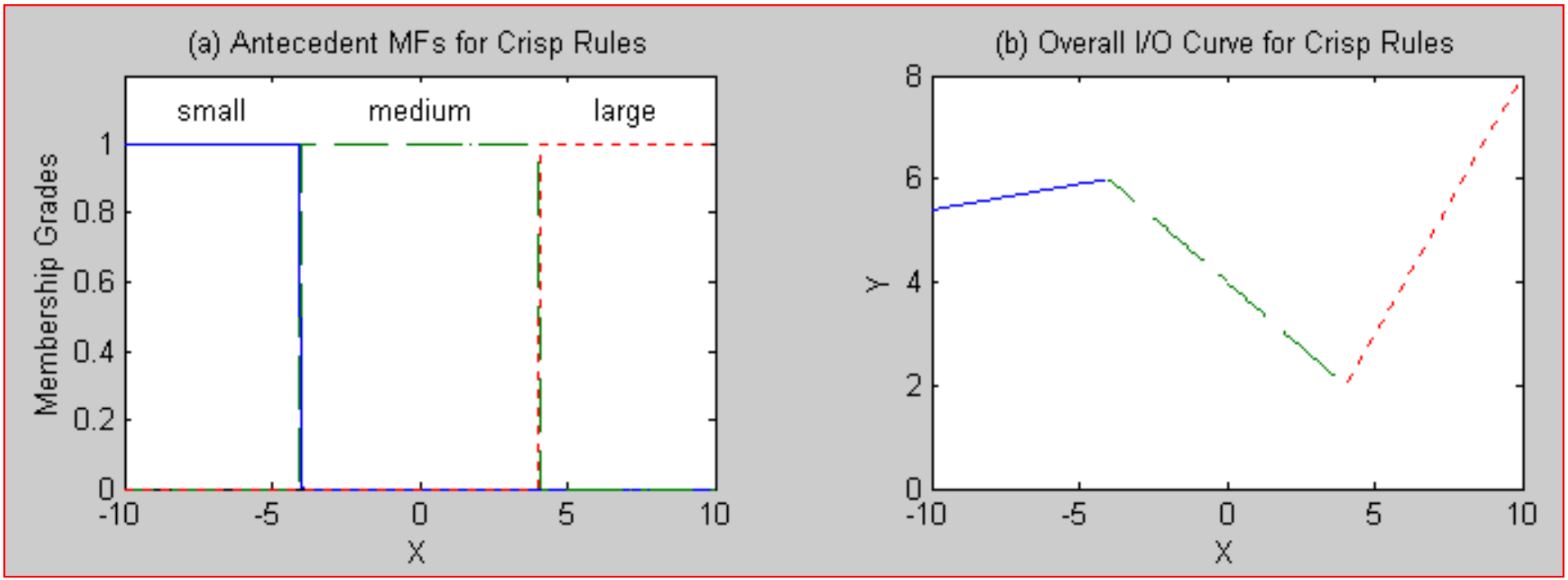
back

menu

Sugeno FIS Example

X = input $\in [-10, 10]$

R1: If X is small then $Y = 0.1X + 6.4$
R2: If X is medium then $Y = -0.5X + 4$
R3: If X is large then $Y = X - 2$

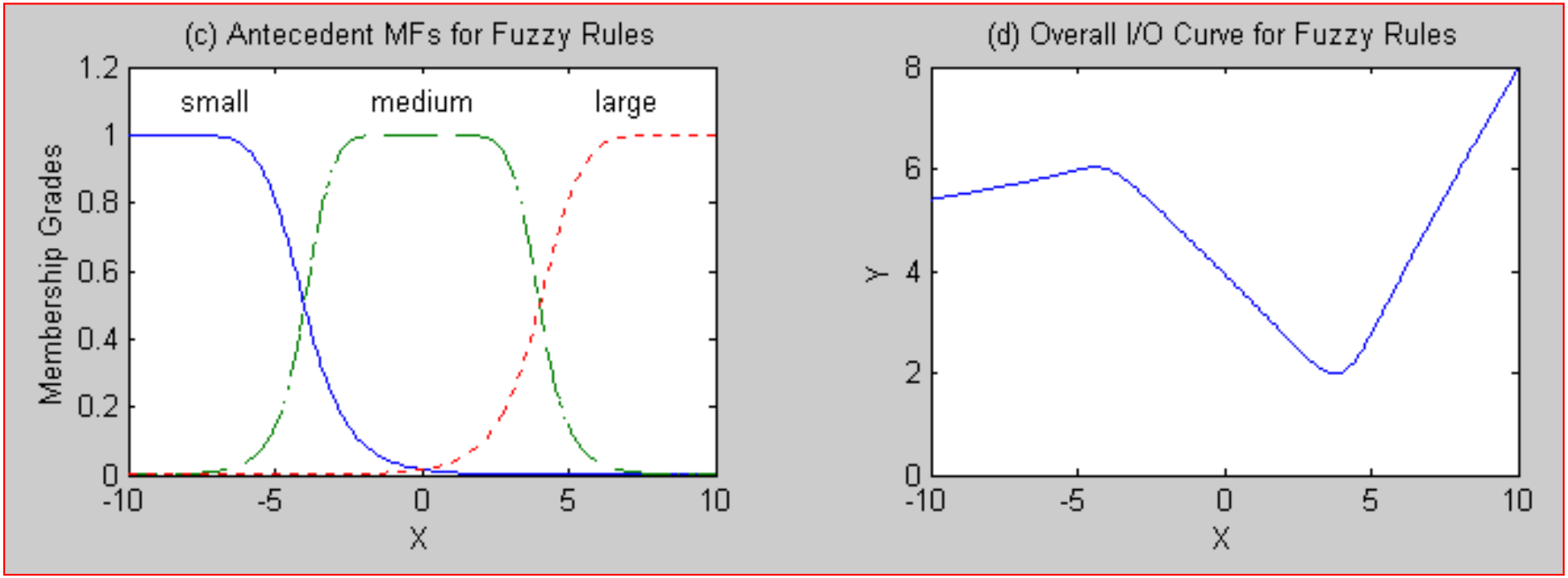


Using **jagged** membership functions, the overall input-output curve produced contains sharp edges.

Sugeno FIS Example

X = input $\in [-10, 10]$

R1: If X is small then $Y = 0.1X + 6.4$
R2: If X is medium then $Y = -0.5X + 4$
R3: If X is large then $Y = X - 2$

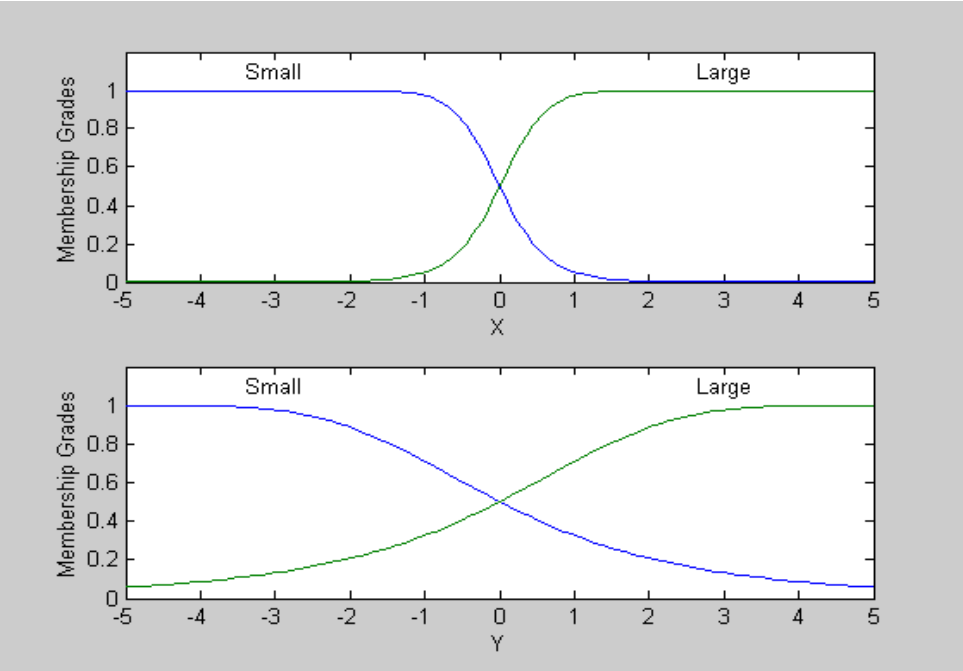


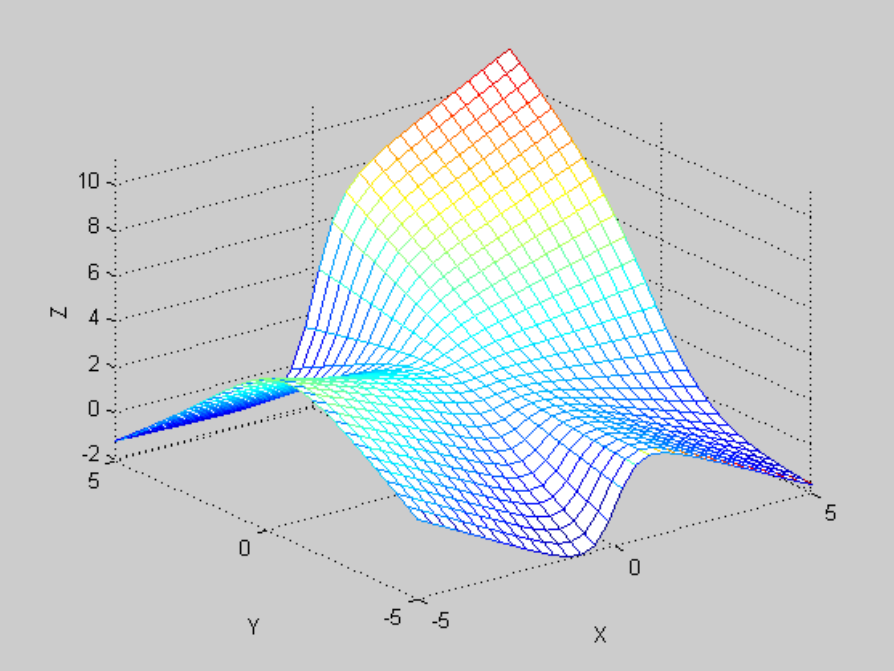
Using **smooth** membership functions, the overall input-output curve produced becomes smoother.

Sugeno FIS Example

R1: if X is small and Y is small then $z = -x + y + 1$
R2: if X is small and Y is large then $z = -y + 3$
R3: if X is large and Y is small then $z = -x + 3$
R4: if X is large and Y is large then $z = x + y + 2$

$X, Y \in [-5, 5]$





Highly non-linear problems could be solved.

http://aimm02.cse.ttu.edu.tw/

back

menu

FIS: Sugeno vs. Mamdani

Advantages of the Sugeno Method

It is computationally efficient.
It can be used to model any inference system in which the output membership functions are either linear or constant.
It works well with linear techniques (e.g., PID control).
It works well with optimization and adaptive techniques.
It has guaranteed continuity of the output surface.
It is well suited to mathematical analysis.

Advantages of the Mamdani Method

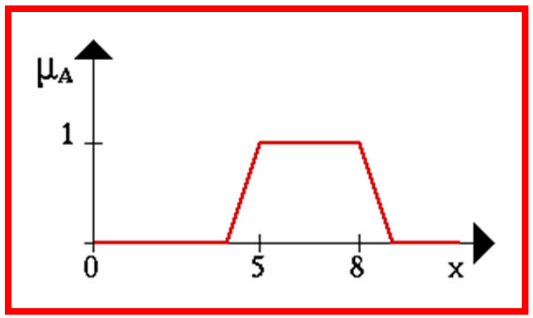
It is intuitive.
It has widespread acceptance.
It is well suited to human input.

Properties of Fuzzy Sets

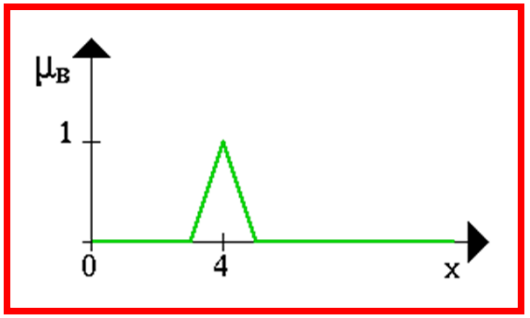
Fuzzy Sets

We will use the following fuzzy sets in explaining the different fuzzy operators that follows next.

Examples:



Fuzzy Set **A**



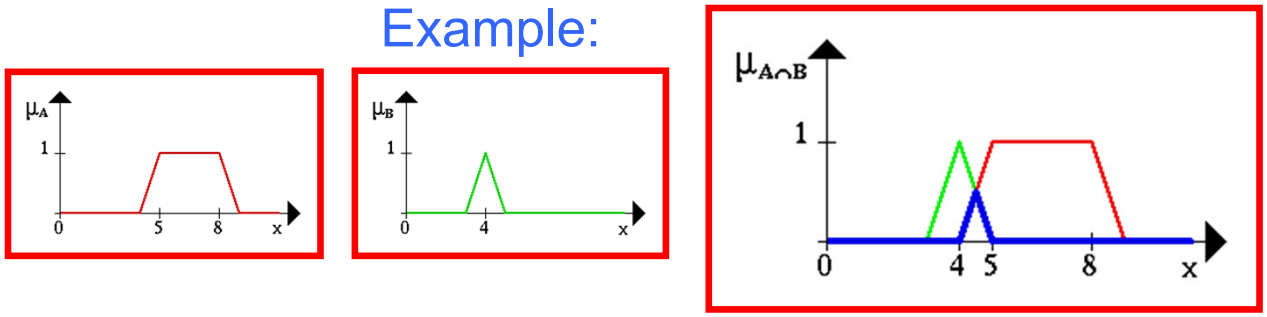
Fuzzy Set **B**

Fuzzy combinations (T-norms)

In making a fuzzy rule, we use the concept of "**and**", "**or**", and sometimes "**not**". The sections below describe the most common definitions of these "fuzzy combination" operators. **Fuzzy intersections** are also referred to as "**T-norms**".

Fuzzy "and"

Example:



Intersection of A and B

The fuzzy "and" is written as:

$$u_{A \cap B} = T(u_A(x), u_B(x))$$

where μ_A is read as "the membership in class A" and μ_B is read as "the membership in class B".

Fuzzy "and"

There are many ways to compute "**and**". The two most common are:

Zadeh And - $\min(\mu_A(x), \mu_B(x))$

This technique, named after the inventor of fuzzy set Theory; it simply computes the "and" by taking the **minimum** of the two (or more) membership values. This is the most common definition of the fuzzy "and".

Product - $\mu_A(x) * \mu_B(x)$

This technique computes the fuzzy "and" by multiplying the two membership values.

Fuzzy "and"

Both techniques have the following two properties:

$$T(0,0) = T(a,0) = T(0,a) = 0$$
$$T(a,1) = T(1,a) = a$$

One of the nice things about both definitions is that they also can be used to compute the Boolean "and". The **fuzzy "and"** is an extension of the **Boolean "and"** to numbers that are not just 0 or 1, but between 0 and 1.

A AND B (B / A)	0	0.25	0.5	0.75	1.0
0	0	0	0	0	0
0.25	0	0.25	0.25	0.25	0.25
0.5	0	0.25	0.5	0.5	0.5
0.75	0	0.25	0.5	0.75	0.75
1	0	0.25	0.5	0.75	1

back

menu

Fuzzy "or"

Fuzzy "or"

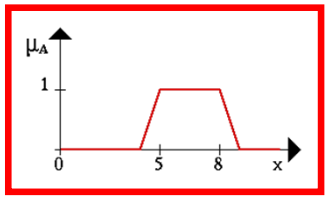
The fuzzy "or" is written as:

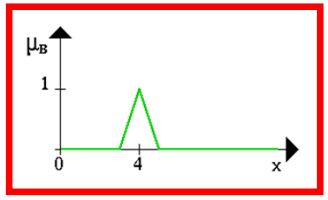
$$u_{A\cup B} = T(u_A(x), u_B(x))$$

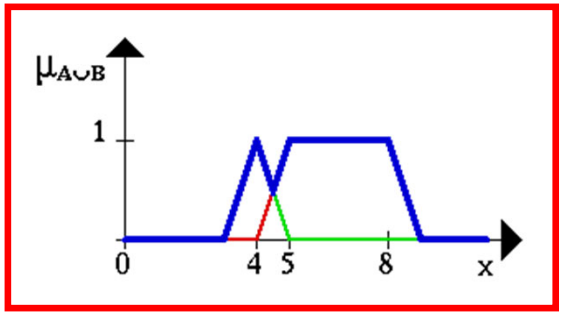
where μ_A is read as "the membership in class A" and μ_B is read as "the membership in class B".

Fuzzy unions are also referred to as "**T-conorms**" or "**S-norms**".

Example:







Union of **A** and **B**

back

menu

Fuzzy "or"

The fuzzy "or" is an extension of the Boolean "or" to numbers that are not just 0 or 1, but between 0 and 1.

A OR B (A \ B)	0	0.25	0.5	0.75	1.0
0	0	0.25	0.5	0.75	1.0
0.25	0.25	0.25	0.5	0.75	1.0
0.5	0.5	0.5	0.5	0.75	1.0
0.75	0.75	0.75	0.75	0.75	1.0
1	1.0	1.0	1.0	1.0	1.0

Fuzzy "or"

There are many ways to compute "**or**". The two most common are:

$$\sigma(x, y) = \max(x, y)$$

Zadeh OR

$$\max(\mu_A(x), \mu_B(x))$$

it simply computes the "or" by taking the **maximum** of the two (or more) membership values. This is the most common definition of the fuzzy "or".

$$\sigma(x, y) = x + y - xy$$

Product

$$(\mu_A(x) + \mu_B(x)) - (\mu_A(x) * \mu_B(x))$$

This technique uses the difference between the sum of the two (or more) membership values and the product of their membership values.

Similar to the fuzzy "and", both definitions of the fuzzy "or" also can be used to compute the Boolean "or".

Fuzzy "or"

Other ways to compute Fuzzy "**or**":

$$\sigma(x, y) = \min(1, x + y)$$

Lukasiewicz Disjunction

$$\min(1, \mu A\ (x) + \mu B\ (x))$$

$$\sigma(x, y) = \frac{x + y - 2xy}{1 - xy}$$

Hamacher Disjunction

$$\frac{\mu_A(x) + \mu_B(x) - 2\mu_A(x)\mu_B(x)}{1 - \mu_A(x)\mu_B(x)}$$

Fuzzy "or"

Other ways to compute Fuzzy "**or**":

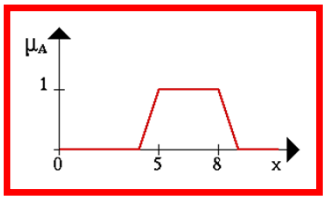
$$\sigma(x,y) = \frac{x+y}{1+xy}$$

Einstein Disjunction

$$\frac{\mu_A(x) + \mu_B(x)}{1 + \mu_A(x)\mu_B(x)}$$

Fuzzy “not”

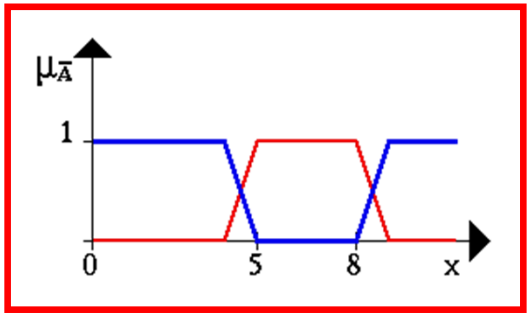
NOT (A) = 1 - A



Fuzzy set **A**

A	NOT A
0	1
0.25	0.75
0.5	0.5
0.75	0.25
1	0

Example:



Negation of **A**

Frequently used properties

Define three fuzzy sets $\underset{\sim}{A}$, $\underset{\sim}{B}$, and $\underset{\sim}{C}$ on the universe X.

- *commutativity*

$$\underset{\sim}{A} \cup \underset{\sim}{B} = \underset{\sim}{B} \cup \underset{\sim}{A}$$

$$\underset{\sim}{A} \cap \underset{\sim}{B} = \underset{\sim}{B} \cap \underset{\sim}{A}$$

- *associativity*:

$$\underset{\sim}{A} \cup \left(\underset{\sim}{B} \cup \underset{\sim}{C}\right) = \left(\underset{\sim}{A} \cup \underset{\sim}{B}\right) \cup \underset{\sim}{C}$$

$$\underset{\sim}{A} \cap \left(\underset{\sim}{B} \cap \underset{\sim}{C}\right) = \left(\underset{\sim}{A} \cap \underset{\sim}{B}\right) \cap \underset{\sim}{C}$$

- *distributivity*:

$$\underset{\sim}{A} \cup \left(\underset{\sim}{B} \cap \underset{\sim}{C}\right) = \left(\underset{\sim}{A} \cup \underset{\sim}{B}\right) \cap \left(\underset{\sim}{A} \cup \underset{\sim}{C}\right)$$

$$\underset{\sim}{A} \cap \left(\underset{\sim}{B} \cup \underset{\sim}{C}\right) = \left(\underset{\sim}{A} \cap \underset{\sim}{B}\right) \cup \left(\underset{\sim}{A} \cap \underset{\sim}{C}\right)$$

- *idempotency*:

$$\underset{\sim}{A} \cup \underset{\sim}{A} = \underset{\sim}{A} \quad \text{and} \quad \underset{\sim}{A} \cap \underset{\sim}{A} = \underset{\sim}{A}$$

Frequently used properties

Define three fuzzy sets $\underset{\sim}{A}$, $\underset{\sim}{B}$, and $\underset{\sim}{C}$ on the universe X.

- *identity*

$$\underset{\sim}{A} \cup \emptyset = \underset{\sim}{A} \quad \text{and} \quad \underset{\sim}{A} \cap X = \underset{\sim}{A}$$

$$\underset{\sim}{A} \cap \emptyset = \emptyset \quad \text{and} \quad \underset{\sim}{A} \cup X = X$$

- *transitivity*:

$$\text{If } \underset{\sim}{A} \subseteq \underset{\sim}{B} \text{ and } \underset{\sim}{B} \subseteq \underset{\sim}{C}, \text{ then } \underset{\sim}{A} \subseteq \underset{\sim}{C}$$

$A \subset B$	$\Rightarrow$	A is fully contained in B (if $x \in A$, then $x \in B$)
$A \subseteq B$	$\Rightarrow$	A is contained in or is equivalent to B
$(A \leftrightarrow B)$	$\Rightarrow$	$A \subseteq B$ and $B \subseteq A$ (A is equivalent to B)

- *involution*:

$$\overline{\overline{\underset{\sim}{A}}} = \underset{\sim}{A}$$

Fuzzy Set operations

Fuzzy logic is a superset of conventional (Boolean) logic

All other operations on classical sets also hold for fuzzy sets, except for the **excluded middle laws**.

$$A \cup \overline{A} \neq X$$

$$A \cap \overline{A} \neq 0$$

for any proposition, either that proposition is true or its negation is true

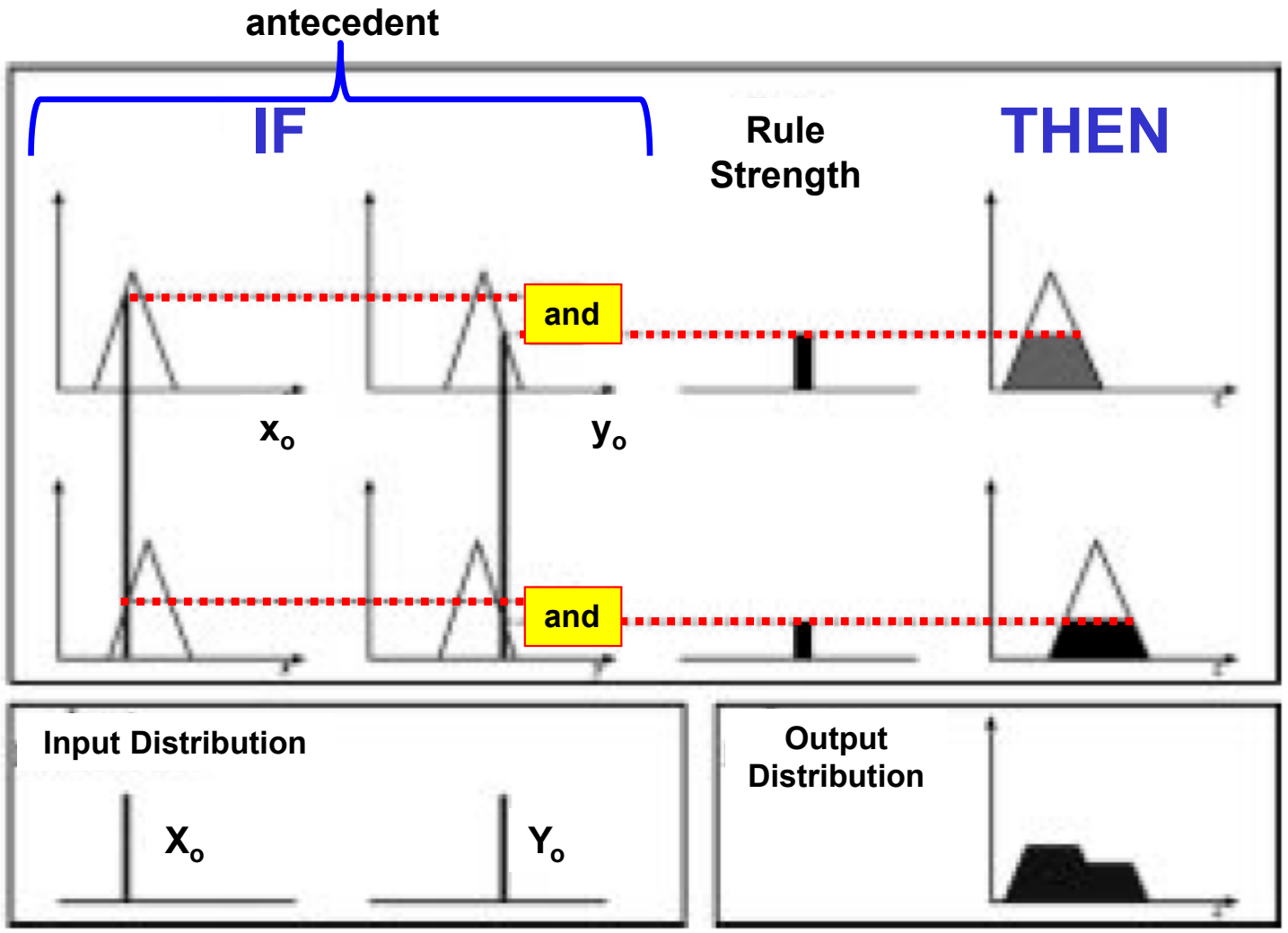
The **whole set X** is the set of all elements in the universe

Variations in computing for the rule consequence

Consequence

The consequence of a fuzzy rule is computed using two steps:

1 Computing the **rule strength** by combining the **fuzzified** inputs using the **fuzzy combination** process



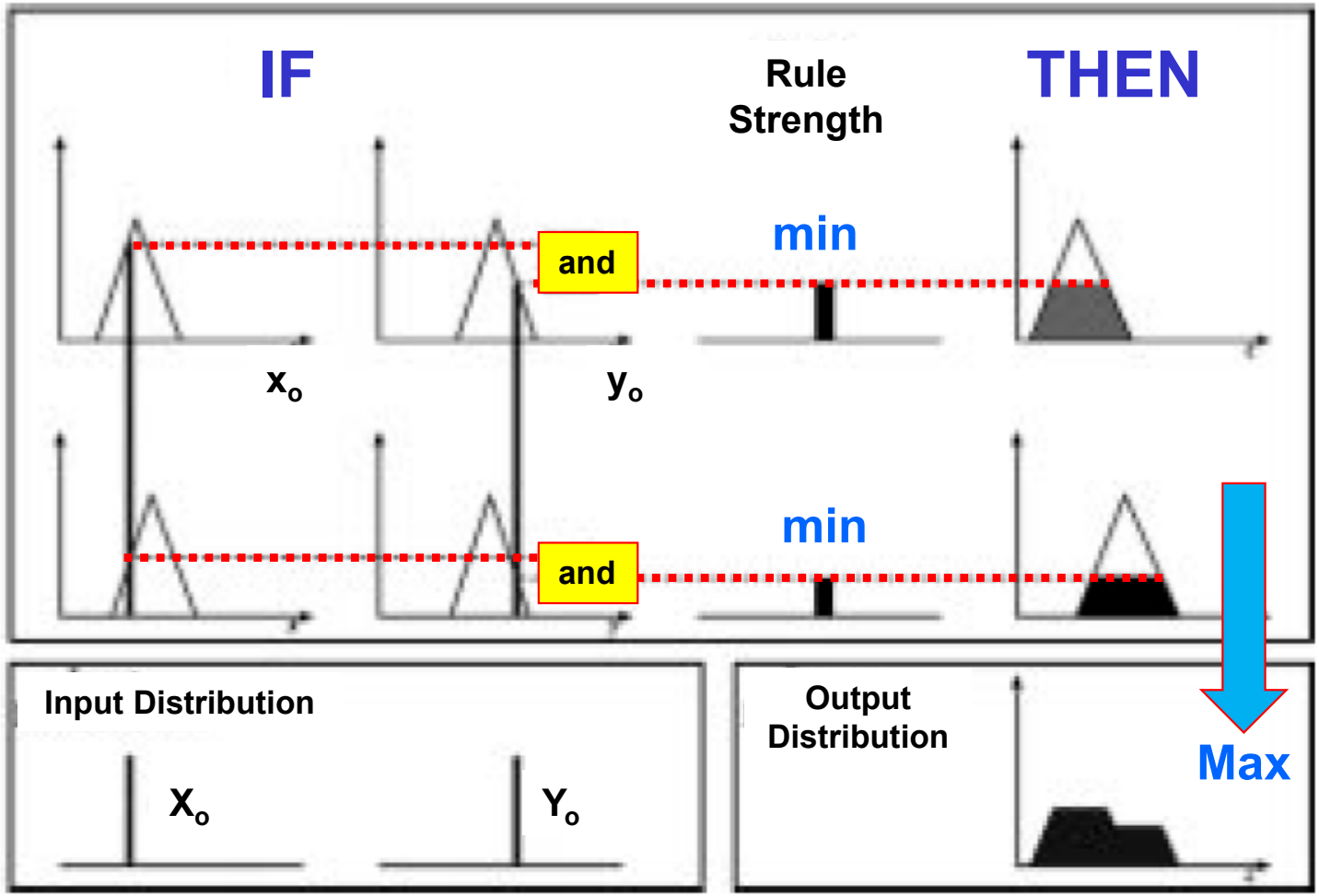
In this example, the fuzzy “and” is used to combine the membership functions to compute the rule strength.

back
menu

Consequence

2a Max-Min Composition

Clipping the output membership function at the rule strength.



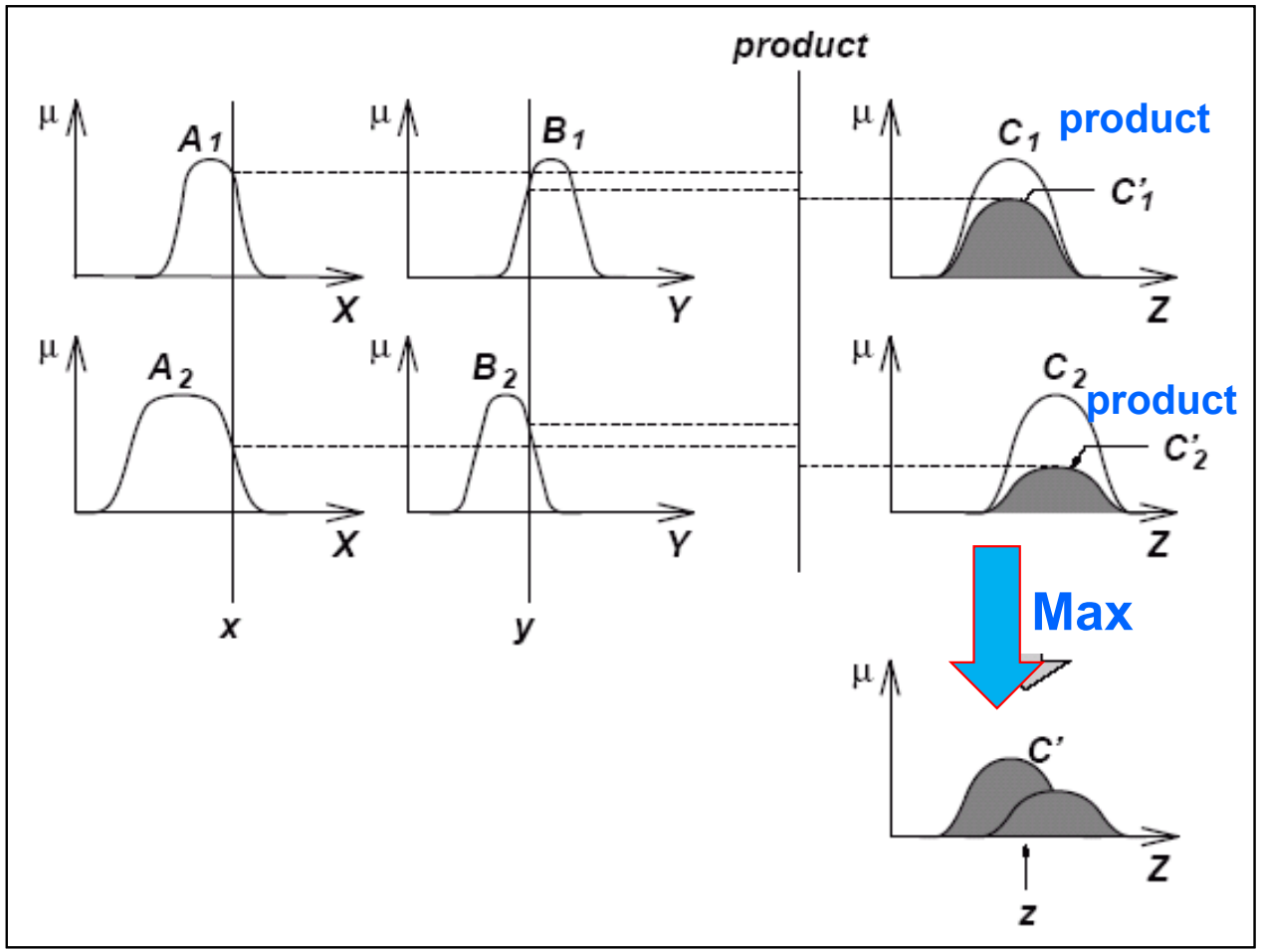
In this example, the fuzzy “and” is used to combine the membership functions to compute the rule strength.

back
menu

Consequence

2b **(Alternatively) Max-Product Composition**

Multiplying the output membership function by the rule strength.

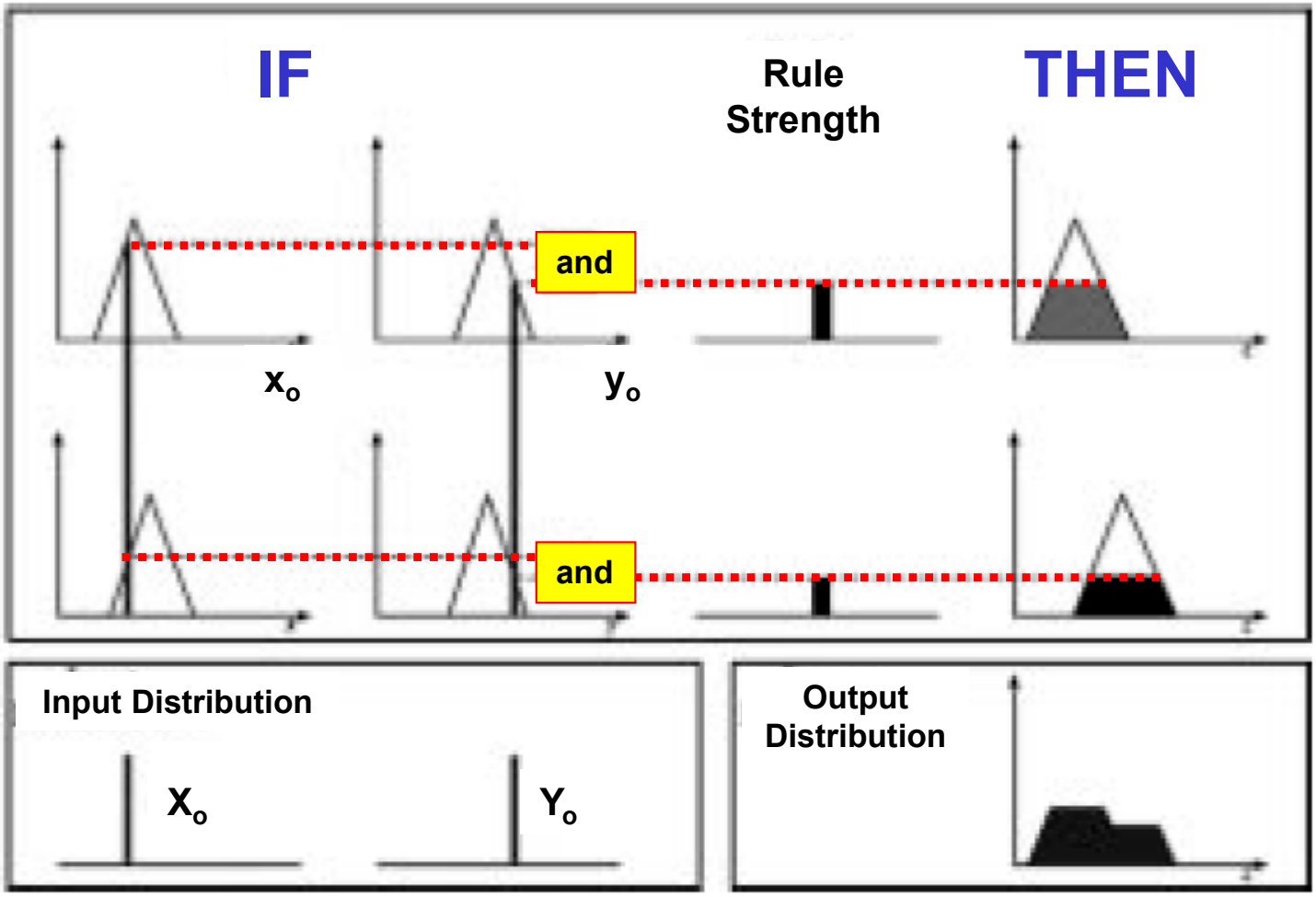


In this example, the fuzzy “and” is used to combine the membership functions to compute the rule strength.

back
menu

Consequence

The outputs of all of the fuzzy rules must now be combined to obtain one **fuzzy output distribution**. This is usually, but not always, done by using the fuzzy "or". The figure below shows an example of this.



Aggregation Method

The output membership functions on the right hand side of the figure are combined using the fuzzy "**or**" to obtain the output distribution shown on the lower right corner of the figure.

Defuzzification techniques

3 Defuzzification of Output Distribution

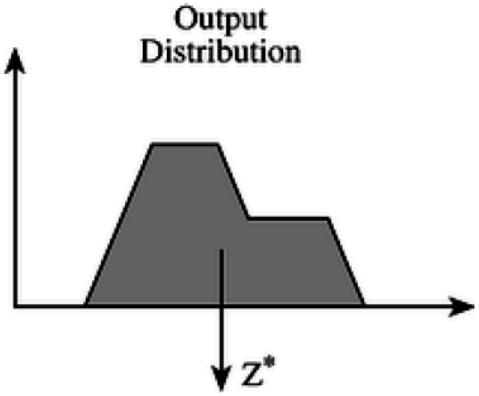
In many instances, it is desired to come up with a single crisp output from a FIS. For example, if one was trying to classify a letter drawn by hand on a drawing tablet, ultimately the FIS would have to come up with a crisp number to tell the computer which letter was drawn. This crisp number is obtained in a process known as defuzzification.
There are two common techniques for **defuzzifying**:

a) Center of mass

Center of mass - This technique takes the output distribution found in the previous slide and finds its center of mass to come up with one crisp number. This is computed as follows:

$$z = \frac{\sum_{j=1}^{q} Z_{outj} u_c(Z_j)}{\sum_{j=1}^{q} u_c(Z_j)}$$

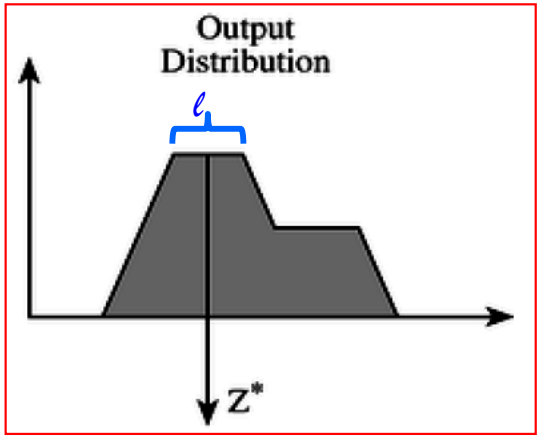
where z is the center of mass and μ_c is the membership in class c at value Z_j. An example outcome of this computation is shown in the figure at the right.



Defuzzification of Output Distribution

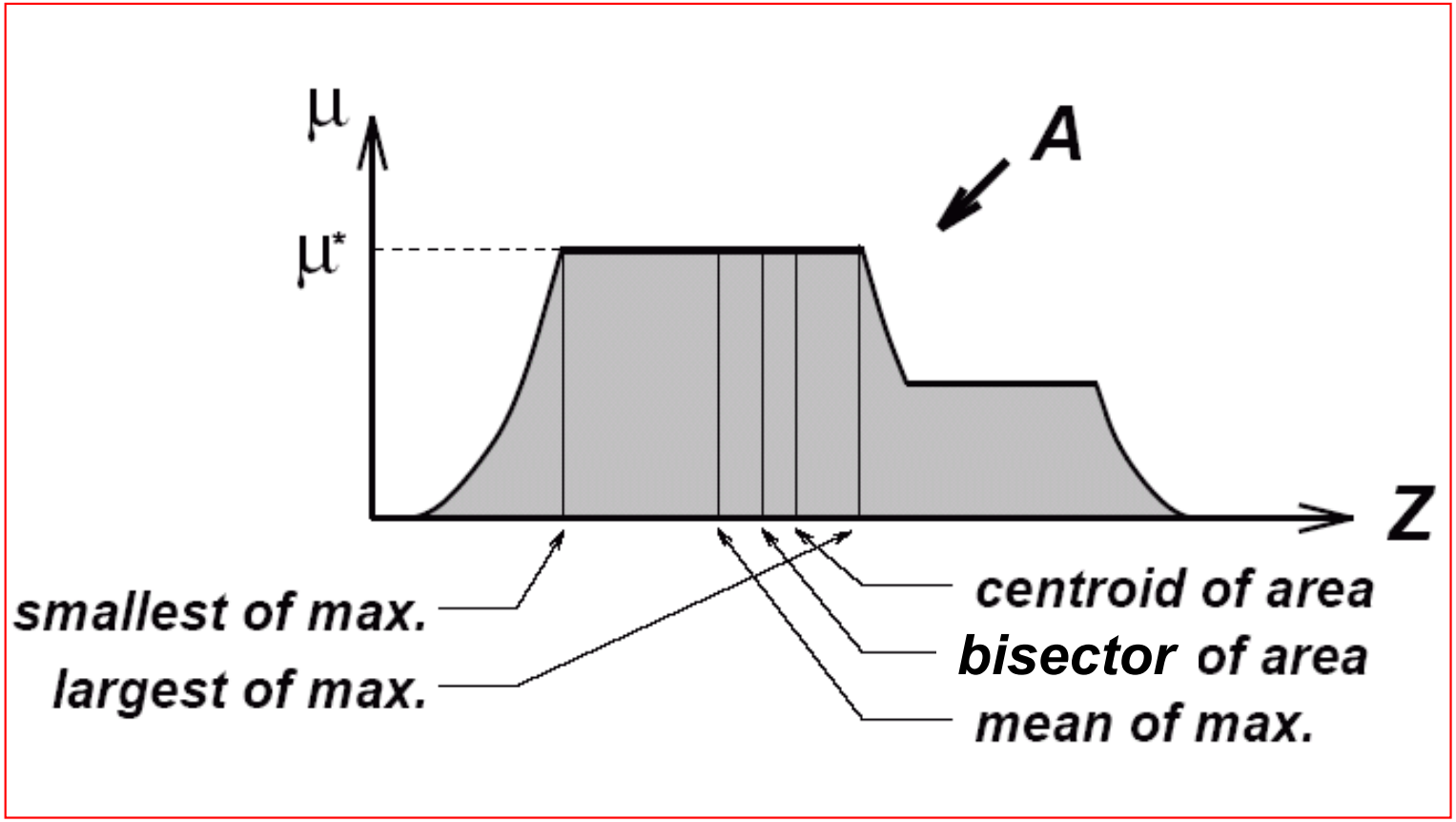
b) Mean of maximum - This technique takes the output distribution found in the previous section and finds its mean of maxima to come up with one crisp number. This is computed as follows:

$$z = \frac{\sum_{j=1}^{l} z_j}{l}$$



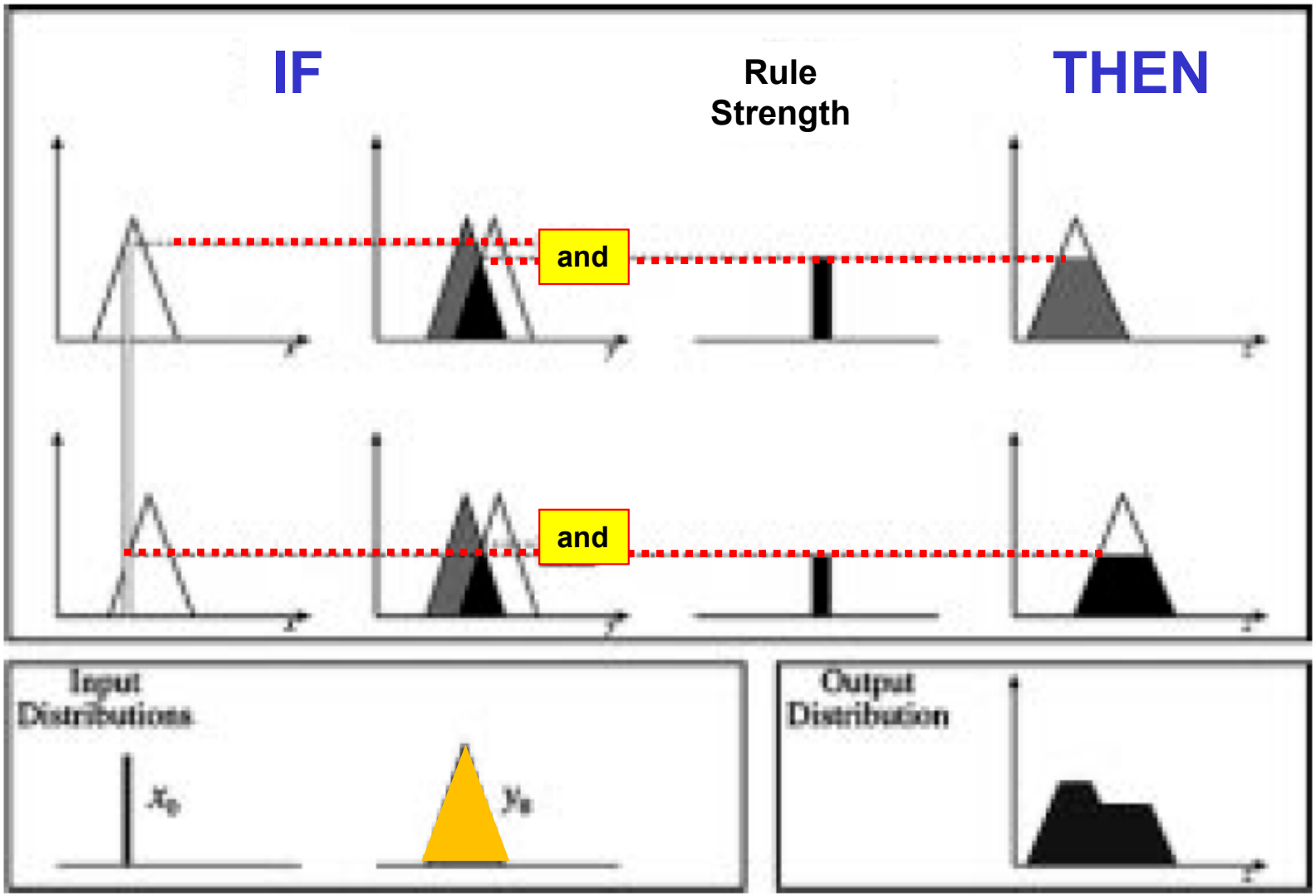
where **z** is the mean of maximum, z_j is the point at which the membership function is maximum, and l is the number of times the output distribution reaches the maximum level.
An example outcome of this computation is shown on the figure at the right.

Other Defuzzification Methods



Mamdani FIS with a Fuzzy Input

A two Input, two rule Mamdani FIS with a fuzzy input



shows a modification of the Mamdani FIS where the input **y0** is **fuzzy**, not crisp.
This can be used to model inaccuracies in the measurement.
For example, we may be measuring the output of a pressure sensor. Even with the exact same pressure applied, the sensor is measured to have slightly different voltages.
The fuzzy input membership function models this uncertainty.

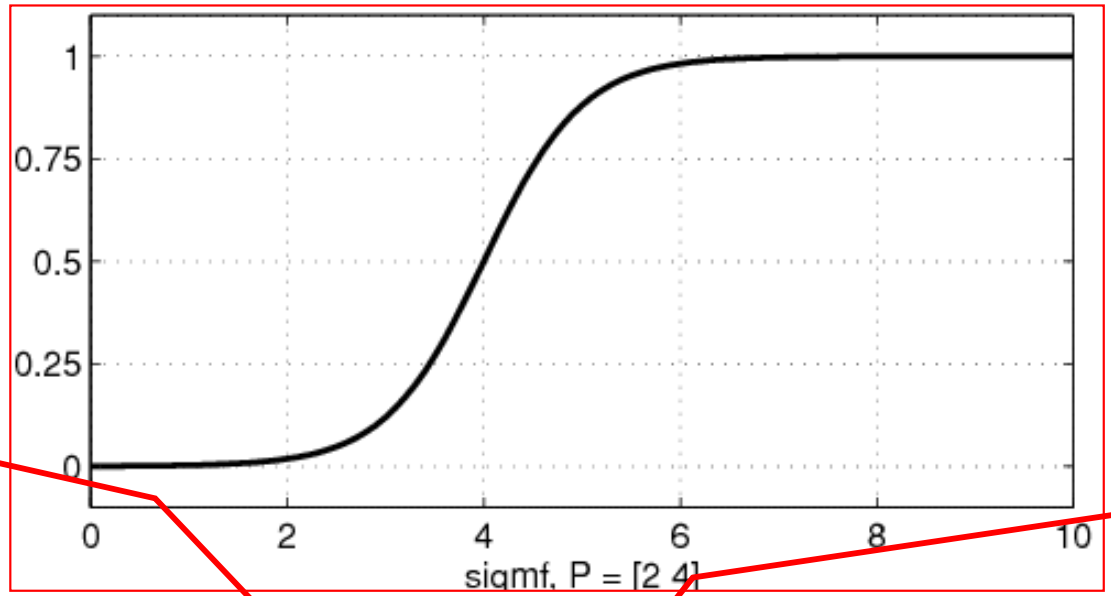
The input fuzzy function is combined with the rule input membership function by using the fuzzy "**and**"

back
menu

Membership Functions

The Sigmoidal function

sigmf(x,[**a** **c**]), as given in the following equation by f(x,a,c) is a mapping on a vector x, and depends on two parameters **a** and **c**.



Slope

crossover point

$$f(x,a,c)=\frac{1}{1+e^{-a(x-c)}}$$

back

menu

Membership Functions

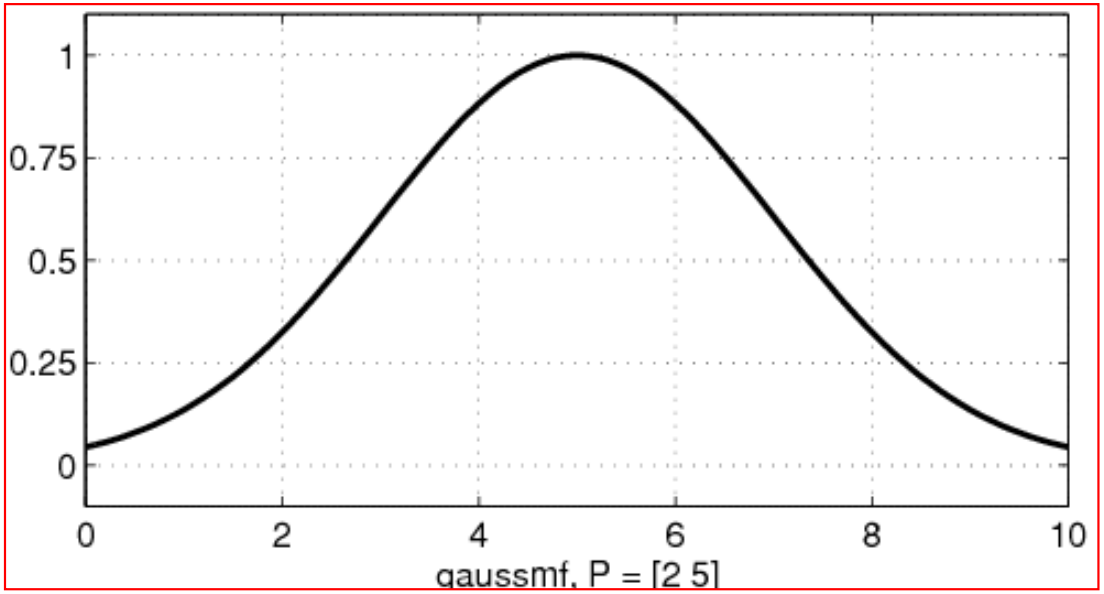
The Gaussian function

centre

The symmetric Gaussian function depends on two parameters σ and c as given by

$$f(x;\sigma,c) = e^{\frac{-(x-c)^2}{2\sigma^2}}$$

width



back

menu

Membership Functions

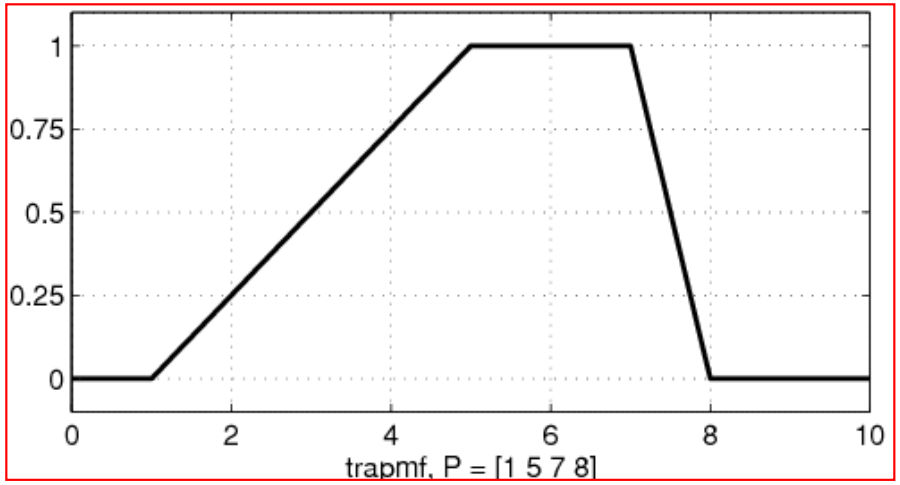
The Trapezoidal function

The trapezoidal curve is a function of a vector, x, and depends on four scalar parameters a, b, c, and d, as given by

$$f(x;a,b,c,d)=\begin{cases}0, & x\le a\\ \dfrac{x-a}{b-a}, & a\le x\le b\\ 1, & b\le x\le c\\ \dfrac{d-x}{d-c}, & c\le x\le d\\ 0, & d\le x\end{cases}$$

or

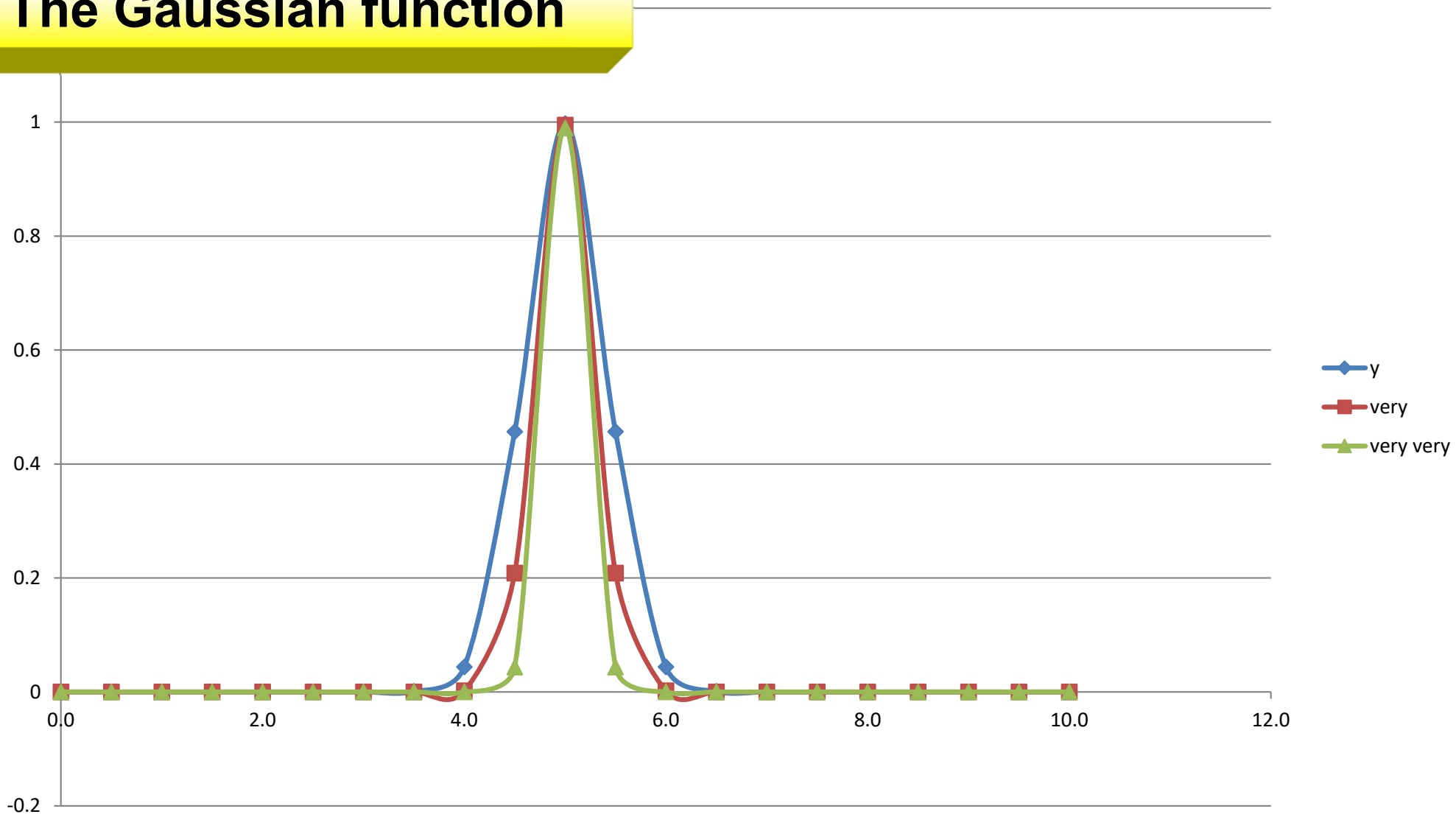
$$f(x;a,b,c,d)=\max\left(\min\left(\frac{x-a}{b-a},1,\frac{d-x}{d-c}\right),o\right)$$



The parameters a and d locate the "feet" of the trapezoid and the parameters b and c locate the "shoulders."

Fuzzy Hedges

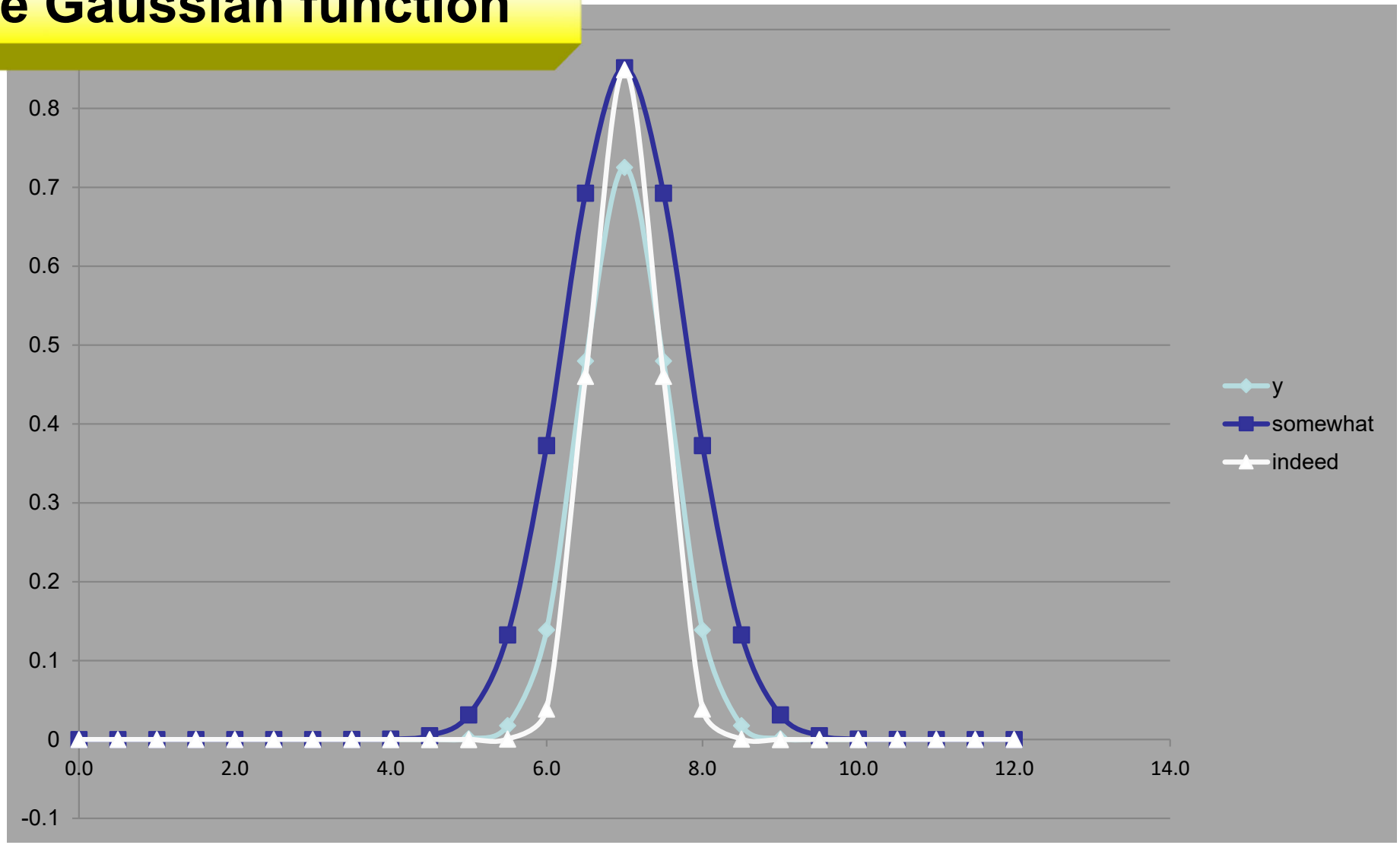
The Gaussian function



sigma	0.4
c	5

Fuzzy Hedges

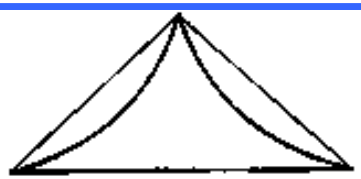
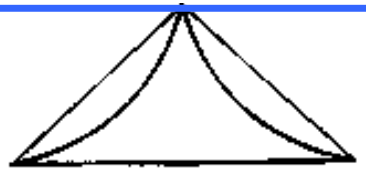
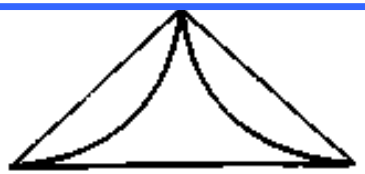
The Gaussian function



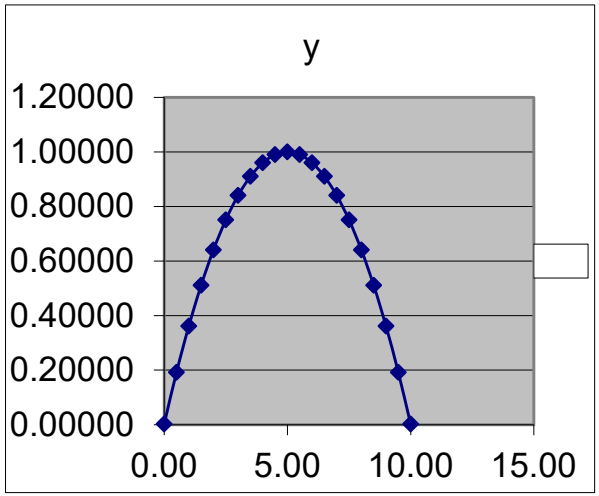
sigma	0.55
c	7

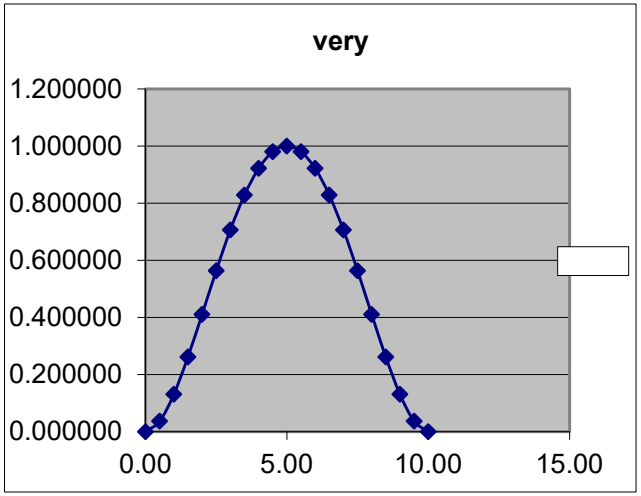
Fuzzy Hedges

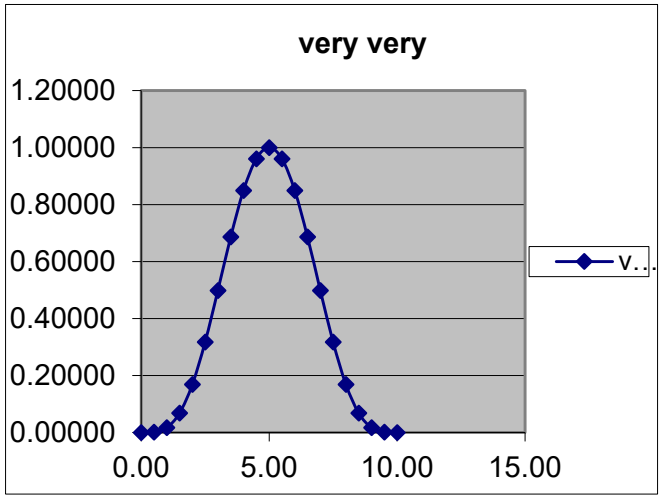
http://blog.peltarion.com/2006/10/25/fuzzy-math-part-1-the-theory/

Hedge	Operator	Effect
A little	$\mu_A(x)^{1.3}$	
Slightly	$\mu_A(x)^{1.7}$	
Very	$\mu_A(x)^{2}$	
Extremely	$\mu_A(x)^{3}$	
Very very	$\mu_A(x)^{4}$	
Somewhat	$\mu_A(x)^{\frac{1}{2}}$	
Indeed	$2\mu_A(x)^2 \quad \text{if } 0 \le \mu_A(x) \le 0.5$ $1-2(1-\mu_A(x))^2 \quad \text{if } 0.5 < \mu_A(x) \le 1$	

More Examples







Sigma	3.54
c	5

References

- Genetic fuzzy systems by Oscar Cordón, Francisco Herrera, Frank Hoffmann
- Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead
- Fuzzy Logic with Engineering Applications by Timothy Ross
- Fuzzy Sets and Pattern Recognition by Benjamin Knapp

The End.

back

menu

Fuzzy Logic Engine

C/C++ version

Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead

Fuzzy Inference System

float inputs[MAX_NO_OF_INPUTS]

fuzzy_system_rec fz

```
crisp_output = fuzzy_system(inputs, fz);
```

A function that can execute a complete fuzzy inference system.

We would like a function that feeds on an array of inputs and a complete specification of a fuzzy system, and in turn returns the final crisp output.

Enumerated data types – for easy indexing

fuzzylogic.h

```
// Indices
////////////////////////////////////////////////////////////////////////////////
typedef   enum {in_angle, in_distance};  //input indices

//indices of fuzzy sets for angle input
typedef   enum {in_negatively_small, in_negatively_medium, in_negatively_large, in_zero,
in_positively_small, in_positively_medium, in

//indices of fuzzy sets for distance input
typedef   enum {in_near, in_medium, in_f

//indices for output steering angle
typedef   enum {out_ze_turn, out_negativ
out_negatively_sharp_turn, out_negativel
out_positively_very_mild_turn, out_positi
out_positively_very_sharp_turn
};

//indices for output speed
typedef   enum {out_very_slow, out_slow, out_medium, out_fast, out_very_fast,
                out_wicked_fast};
```

Define **enum** variables for the linguistic terms.

We would like to refer to the different fuzzy sets and input parameters using descriptive names (e.g. {in_angle, in_position}, {in_negatively_small, in_negatively_large, etc.), instead of numbers whenever we use them.

Add linguistic terms tailored to the problem domain

Trapezoidal Membership Function

fuzzylogic.h

```
//Trapezoidal membership function types
typedef   enum { regular_trapezoid, left_trapezoid, right_trapezoid }  trapz_type;


// Trapezoidal Fuzzy Set ////////////////////

struct  trapezoid {

   trapz_type    tp;  //type of trapezo

   float  a, b, c, d, l_slope, r_slope;

};
```

Define a structure for holding the properties of the membership functions to use.

Here we have a structure implementing the trapezoidal membership function.

Fuzzy Rule

fuzzylogic.h

```
// Fuzzy Rule ////////////////////////////////////////////////////////////

typedef   struct {

   short     inp_index[MAX_NO_OF_INPUTS],  //input index
             inp_fuzzy_set[MAX_NO_OF_INPUTS], //input fuzzy set index
             out_fuzzy_set;  //output index

} rule;
```

Define a structure for holding the components of a fuzzy rule.

Note that we are only storing the indices of the components of a rule.

inp_index[0]: **in_angle**

inp_index[1]: **in_distance**

out_fuzzy_set

If (**angle** is **small**) AND (**distance** is **VERY_FAR**) **Then** output is **VERY_FAST**.

inp_fuzzy_set[0]: **in_small**

inp_fuzzy_set[1]: **in_very_far**

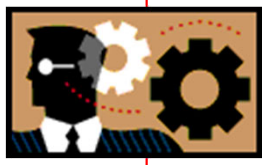
Fuzzy System

fuzzylogic.h

```
// The complete Fuzzy System /////

typedef  struct fuzzy_system_rec {

    bool allocated;

    trapezoid   inp_mem_fns [MAX_NO_OF_INPUTS] [MAX_NO_OF_INP_REGIONS];

    rule*  rules; //note that we need to allocate memory for the rules

    int       no_of_inputs,        //number of inputs
              no_of_inp_regions,   //number of fuzzy sets associated with each input
              no_of_rules,         //number of rules
              no_of_outputs;       //number of fuzzy outputs

    float  output_values[MAX_NO_OF_OUTPUT_VALUES]; //the values of the outputs

};
```

Define a structure for holding the complete description of a **fuzzy inference system**.

Here we have a structure implementing a Zero-Order Sugeno Fuzzy inference system.



Defining the Fuzzy Rule

If (**angle** is small) **AND** (**distance** is VERY_FAR) **Then** output is **VERY_FAST**.

```
fuzzy_system_rec *fl;
//…allocate memory
//…initialise fuzzy parameters

    fl->rules[0].inp_index[0] = in_angle;
    fl->rules[0].inp_index[1] = in_distanc

    fl->rules[0].inp_fuzzy_set[0] = in_sm
    fl->rules[0].inp_fuzzy_set[1] = in_ver
    fl->rules[0].out_fuzzy_set = out_very
```

How to customise a fuzzy rule?

Here is an example of how we can define a rule with two inputs.

```
// Fuzzy Rule ////////////////////////////////////
typedef   struct {
  short     inp_index[MAX_NO_OF_INPUTS],
            inp_fuzzy_set[MAX_NO_OF_INPUTS],
            out_fuzzy_set;  //output index
} rule;
```

SMALL	Med Speed	Fast Sp	ry Fast
MEDIUM	Slow Speed	Med Speed	Fa
LARGE	Very Slow	Slow Speed	Sl

Defining the Fuzzy Rule

Main.cpp

inp_index[0]: **in_angle**

inp_index[1]: **in_distance**

out_fuzzy_set

If (**angle** is small) **AND** (**distance** is VERY_FAR) **Then** output is **VERY_FAST**.

```
fuzzy_system_rec *fl;
//…allocate memory
//…initialise fuzzy parameters

    fl->rules[0].inp_index[0] = in_angle;
    fl->rules[0].inp_index[1] = in_distance;

    fl->rules[0].inp_fuzzy_set[0] = in_small;
    fl->rules[0].inp_fuzzy_set[1] = in_very_far;
    fl->rules[0].out_fuzzy_set = out_very_fast;
```

We are only storing the indices in the rule.

```
// Fuzzy Rule ////////////////////////////////////////////
typedef   struct {
  short     inp_index[MAX_NO_OF_INPUTS],
            inp_fuzzy_set[MAX_NO_OF_INPUTS],
            out_fuzzy_set;  //output index
} rule;
```

FAMM

	NEAR	*FAR*	*VERY FAR*
SMALL	Med Speed	Fast Speed	Very Fast
MEDIUM	Slow Speed	Med Speed	Fast Speed
LARGE	Very Slow	Slow Speed	Slow Speed

Membership Function

If (**angle** is small) AND (**distance** is VERY_FAR) Then output is **VERY_FAST**.

fl->inp_mem_fns[variable_index][fuzzy_set]

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance

fl->rules[0].inp_fuzzy_set[0] = in_sma
fl->rules[0].inp_fuzzy_set[1] = in_very
fl->rules[0].out_fuzzy_set = out_very
```

How to specify which **membership function** we want to use?

Using the index of the rule input, and index of the fuzzy set, we can easily identify the membership function.

SMALL	Med Speed	Fast Speed	Fast
MEDIUM	Slow Speed	Med Speed	Fa
LARGE	Very Slow	Slow Speed	Sl

Membership Function

inp_index[0]: **in_angle**

inp_index[1]: **in_distance**

out_fuzzy_set

If (**angle** is small) AND (**distance** is VERY_FAR) Then output is **VERY_FAST**.

fl->inp_mem_fns[variable_index][fuzzy_set]

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

	NEAR	*FAR*	*VERY FAR*
SMALL	Med Speed	Fast Speed	Very Fast
MEDIUM	Slow Speed	Med Speed	Fast Speed
LARGE	Very Slow	Slow Speed	Slow Speed

Membership Function data structure

Declaration:

trapezoid **inp_mem_fns** [MAX_NO_OF_INPUTS] [MAX_NO_OF_INP_REGIONS];

inp_mem_fns is a structure containing all the parameters of a membership function.

fl->inp_mem_fns[variable_index][fuzzy_set]

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

Membership Function Initialisation

inp_mem_fns is a structure containing all the parameters of a membership function.

The **init_trapz()** function is used to initialize **inp_mem_fns**.

- fl->**inp_mem_fns**[in_angle][in_small] **=**
  - **init_trapz**(14.0f, 20.0f, 0.0f, 0.0f, **left_trapezoid**);

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

Calculation of degree of Membership

inp_mem_fns is a structure containing all the parameters of a membership function.

The **trapz** function calculates the **actual degree of membership** of a given input value in a fuzzy set.

which input

which membership function

trapz(inputs[variable_index],fl.inp_mem_fns[variable_index][fuzzy_set])

```
fl->rules[i].inp_index[0] = in_angle;
fl->rules[i].inp_index[1] = in_distance;

fl->rules[0].inp_fuzzy_set[0] = in_small;
fl->rules[0].inp_fuzzy_set[1] = in_very_far;
fl->rules[0].out_fuzzy_set = out_very_fast;
```

Calculation of degree of Membership

fuzzylogic.cpp

which input

which membership function

trapz(inputs[variable_index],fl.inp_mem_fns[variable_index][fuzzy_set])

```
float trapz (float x, trapezoid trz) {
  switch (trz.tp) {

    case left_trapezoid: //opening to the left
                if (x <= trz.a)
                  return 1.0;
                if (x >= trz.b)
                  return 0.0;
                /* a < x < b */
                return trz.r_slope * (x - trz.b);


    case right_trapezoid: //opening to the right
                if (x <= trz.a)
                  return 0.0;
                if (x >= trz.b)
                  return 1.0;
                /* a < x < b */
                return trz.l_slope * (x - trz.a);
```

```
    case regular_trapezoid:
                if ((x <= trz.a) || (x >= trz.d))
                  return 0.0;
                if ((x >= trz.b) && (x <= trz.c))
                  return 1.0;
                if ((x >= trz.a) && (x <= trz.b))
                  return trz.l_slope * (x - trz.a);
                if ((x >= trz.c) && (x <= trz.d))
                  return  trz.r_slope * (x - trz.d);

                } /* End switch  */

  return 0.0;  /* should not get to this point */
} /* End function */
```

Fuzzy System – from fuzzification to defuzzification

File: **fuzzylogic.cpp**

```
float  fuzzy_system ( float  inputs[ ],  fuzzy_system_rec   fz) {
   int  i, j;
   short  variable_index, fuzzy_set;
   float  sum1 = 0.0f,  sum2 = 0.0f,  weight;
   float  m_values[MAX_NO_OF_INPUTS];

   for (i = 0; i < fz.no_of_rules; i++) {
      for (j = 0; j < fz.no_of_inputs; j++) {
                  variable_index = fz.rules[i].inp_index[j];
                  fuzzy_set = fz.rules[i].inp_fuzzy_set[j];
                  m_values[j] = trapz(inputs[variable_index], fz.inp_mem_fns[variable_index][fuzzy_set]);
       } /* end j  */

      weight = min_of (m_values, fz.no_of_inputs);
      sum1 += weight * fz.output_values[ fz.rules[i].out_fuzzy_set ];
      sum2 += weight;

   } /* end i  */

   if (fabs(sum2) < TOO_SMALL) {  // TOO_SMALL = 1e-6
      cout << "\r\nFLPRCS Error: sum2 in fuzzy_system is 0." << endl;
      //exit(1);
      return 0.0;
   }

   return (sum1/sum2);
}
```

which input

which fuzzy set

How to use the Fuzzy Logic Engine?

Fuzzy System Development

1. Declare a global variable for the fuzzy system.

File: **Main.cpp**

```
e.g.  fuzzy_system_rec  g_fuzzy_system;
```

Define the maximum number of Fuzzy Sets to be used.

File: **fuzzylogic.h**

```
#define  MAX_NO_OF_INP_REGIONS  7
```

e.g. 7 Maximum Number of Fuzzy Sets allowed

Fuzzy System Development

2. Initialise the fuzzy system.

File: **fuzzylogic.cpp**

```
void   initFuzzySystem (fuzzy_system_rec*  fl) {

   fl->no_of_inputs = 2;  //inputs are handled 2 at a time only
   fl->no_of_inp_regions = 7; //number of fuzzy sets per input
   fl->no_of_rules = 49*2;          //number of rules
//----
   fl->output_values [out_small_push_left] = -15f;   //-15 Newtons
   fl->output_values [out_large_push_left] = -100f; //-100 Newtons

   //… and so on…
   …
   fl->rules = new rule [fl->no_of_rules];  //allocate memory for the rules

   initFuzzyRules (fl); //initialise the rules; this is user-defined
   initMembershipFunctions(fl); //initialise the membership functions; this is user-
                                //defined
}
```

Fuzzy System Development

3. Initialise fuzzy rules.

File: **fuzzylogic.cpp**

```
void initFuzzyRules (fuzzy_system_rec*  fl) {
  int i;
  for (i = 0;i < fl->no_of_rules;i++) {
     //(*fl).rules[i].inp_index[0] = in_angle; //alternatively
     fl->rules[i].inp_index[0] = in_angle;
     fl->rules[i].inp_index[1] = in_angle_dot;
  }

  fl->rules[0].inp_fuzzy_set[0] = in_negatively_small;
  fl->rules[0].inp_fuzzy_set[1] = in_falling_to_left_fast;
  fl->rules[0].out_fuzzy_set = out_large_push_left;

// define the other remaining rules next
// and so on...

}
```

Rule #0

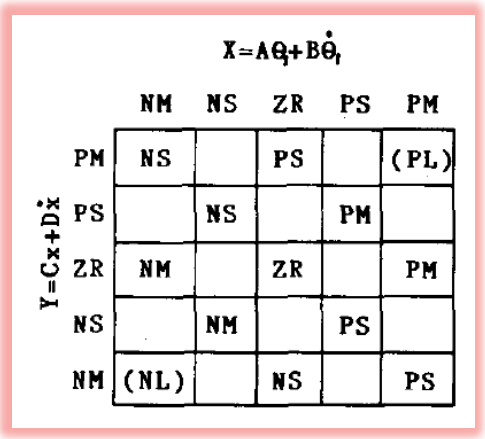
This is an example only. The inputs and rule parameters will depend on your system design.

Fuzzy System Development

3. Initialise fuzzy rules.

File: **fuzzylogic.cpp**

```
void initFuzzyRules (fuzzy_system_rec*  fl) {
  int i;
  for (i = 0;i < fl->no_of_rules;i++) {
     //(*fl).rules[i].inp_index[0] = in_X; //alternatively
     fl->rules[i].inp_index[0] = in_X;
     fl->rules[i].inp_index[1] = in_Y;
  }

  fl->rules[0].inp_fuzzy_set[0] = in_NM;
  fl->rules[0].inp_fuzzy_set[1] = in_PM;
  fl->rules[0].out_fuzzy_set = out_NS;

// define the other remaining rules next
// and so on...

}
```



Rule #0

Yamakawa's strategy

Fuzzy System Development

4. Initialise membership functions.

File: **fuzzylogic.cpp**

```
void   initMembershipFunctions ( fuzzy_system_rec*   fl ) {

  //angle
  fl->inp_mem_fns[in_angle][in_negatively_large] = init_trapz(14.0f,20.0f,0.0f,0.0f,
                                                                    left_trapezoid);
  fl->inp_mem_fns[in_angle][in_negatively_medium] = init_trapz(14.0f,20.0f,34.0f,40.0f,
                                                                    regular_trapezoid);
 //...
  fl->inp_mem_fns[in_angle][in_positively_large] = init_trapz (34.0f, 40.0f, 0.0f, 0.0f,
                                                                    right_trapezoid);

  //angular velocity
  //...
  //...
  //...
  //and so on...

}
```

which fuzzy set

Inverted Pendulum

4. Modify runInvertedPendulum ().

File: Main.cpp

```
void   runInvertedPendulum () {
  //...
   initFuzzySystem(&g_fuzzy_system);
   while((GetAsyncKeyState(VK_ESCAPE)) == 0 ) { //while ESC key is not pressed
      //...
       inputValues [ in_angle ] = prevState.angle;
       inputValues[ in_angle_dot ] = prevState.angle_dot;
       prevState.Force= fuzzy_system(inputValues, g_fuzzy_system);
       //...  Calculate new state of the world
       newState.angle_double_dot = calc_angular_acceleration(prevState);
       //and so on...
       cart.setX(newState.x);
       rod.setX(newState.x);
       rod.setAngle(newState.angle);
       cart.draw();   rod.draw();
}
    free_fuzzy_rules(&g_fuzzy_system);
   //and so on...
}
```

Specify the inputs and fuzzy system structure, then pass them to the fuzzy_system() function.

Skeleton of Simulation

void runInvertedPendulum()

```
initPendulumWorld();
initFuzzySystem(&g_fuzzy_system);

while((GetAsyncKeyState(VK_ESCAPE)) == 0 ) {

        setactivepage(page);
        cleardevice();

        senseEnvironment (inputs);

        F = fuzzy_system(inputs, g_fuzzy_system);

        updateWorld(newWorldState)
        drawCartAndRod(newWorldState)

        setvisualpage(page);
        toggle page  //switch to another page
}
```

Input and Output

INPUTS

1. Angle of the rod with respect to the vertical axis (in radians)
2. Angular velocity of the rod (in radians per second)
3. Cart position (in meters)
4. Cart velocity (in meters per second)

OUTPUT

OUTPUT

1. Force to apply (in Newtons)

World State

```
struct WorldStateType{
    void init(){
        x=0.0;
        x_dot=0.0;
        x_double_dot = 0.0;
        angle = 0.0;
        angle_dot = 0.0;
        angle_double_dot = 0.0;
        F = 0.0;
    }

    float x;
    float x_dot;
    float x_double_dot;
    float angle;
    float angle_dot;
    float angle_double_dot;
```

```
float const mb=0.1; // mass of broom
float const g=9.8; // pull of gravity
float const m=1.1; // mass of cart & broom
float const l=0.5; // length of broom from
//pivot point to centre of mass

float F;

};
```

```
WorldStateType prevState, newState;
```

Dynamics of the System

```
float calc_angular_acceleration( const WorldStateType& s){
    float a_double_dot=0.0;

    a_double_dot = (s.m * s.g * sin(s.angle) - (cos(s.angle)
                        * (s.F + ((s.mb) * s.l * s.angle_dot * s.angle_dot * sin(s.angle) ) ) )  )
                       / (  ((4/3)*s.m * s.l) - (s.mb * s.l * cos(s.angle) * cos(s.angle)));

    return a_double_dot;
}
```

```
float calc_horizontal_acceleration( const WorldStateType& s){
     float x_double_dot=0.0;

     x_double_dot = ( s.F + s.mb * s.l * (s.angle_dot * s.angle_dot)* sin(s.angle) -
                         s.angle_double_dot * cos(s.angle)   )/ s.m;

    return x_double_dot;
}
```

void runInvertedPendulum()

```
//*********************************************************
//Set the initial angle of the pole with respect to the vertical
prevState.angle = 8 * (3.14/180);  //initial angle  = 8 degrees

initFuzzySystem(&g_fuzzy_system);

while((GetAsyncKeyState(VK_ESCAPE)) == 0 ) {

 setactivepage(page);
 cleardevice();
 drawInvertedPendulumWorld();

 //retrieve inputs
 inputs[in_theta] = prevState.angle;
 inputs[in_theta_dot] = prevState.angle_dot;
 inputs[in_x] = prevState.x;
 inputs[in_x_dot] = prevState.x_dot;

 //1) Enable this only after your fuzzy system has been completed already.
 //Remember, you need to define the rules, membership function parameters and rule outputs.
 //prevState.F = fuzzy_system(inputs, g_fuzzy_system); //call the fuzzy controller

 externalForce=0.0;
 externalForce = getKey(); //manual operation

 if(externalForce != 0.0)
    prevState.F = externalForce;
//-------------------------------------------------------------------------
//continued…
```

Force to apply.

void runInvertedPendulum()

```
//-------------------------------------------------------------------------
 // ************************************************************************
 // BEGIN - DYNAMICS OF THE SYSTEM
 //Calculate the new state of the world
 newState.angle_double_dot = calc_angular_acceleration(prevState);
 newState.angle_dot = prevState.angle_dot + (h * newState.angle_double_dot);
 newState.angle = prevState.angle + (h * newState.angle_dot);
 newState.F = prevState.F;
 newState.x_double_dot = calc_horizontal_acceleration(prevState);
 newState.x_dot = prevState.x_dot + (h * newState.x_double_dot);
 newState.x = prevState.x + (h * newState.x_dot);
 prevState.x = newState.x;
 prevState.angle = newState.angle;
 prevState.x_dot = newState.x_dot;
 prevState.angle_dot = newState.angle_dot;
 prevState.angle_double_dot = newState.angle_double_dot;
 prevState.x_double_dot = newState.x_double_dot;
 //-------------------------
 cart.setX(newState.x);
 rod.setX(newState.x);
 rod.setAngle(newState.angle);
 cart.draw();
 rod.draw();
 // END - DYNAMICS OF THE SYSTEM
 // ************************************************************************
 //-------------------------------------------------------------------------
 displayInfo(newState);
 setvisualpage(page);
 page = !page;  //switch to another page
}
/2) Enable this only after your fuzzy system has been completed already.
//free_fuzzy_rules(&g_fuzzy_system);
```

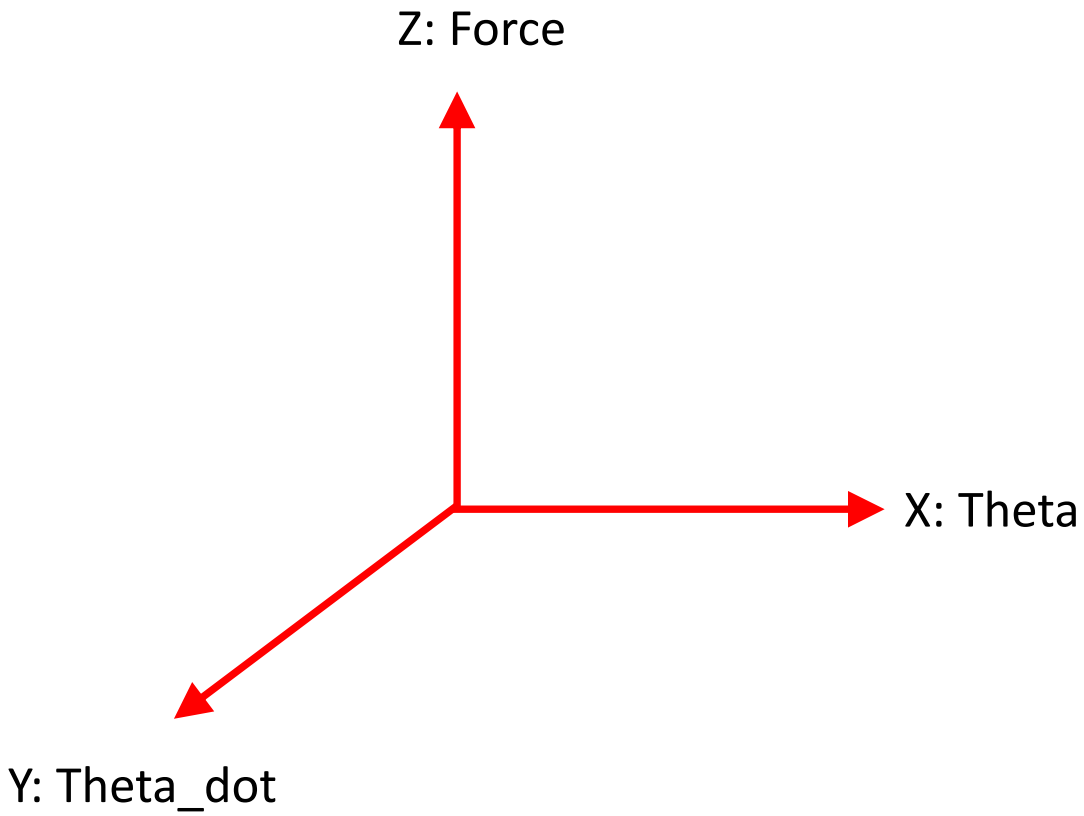
Physics equations and Euler's method (please do not modify these statements)

Control surface generation

int main()

```
int main() {

   int graphDriver = 0,graphMode = 0;

   initgraph(&graphDriver, &graphMode, "", 1280, 1024); // Start Window
   clearDataSet();
   try{
         runInvertedPendulum();

         //3) Enable this only after your fuzzy system has been completed already.
         //generateControlSurface_Angle_vs_Angle_Dot();

         //4) Enable this only after your fuzzy system has been completed already.
         //saveDataToFile("data_angle_vs_angle_dot.txt");

   }
   catch(...){
    cout << "Exception caught!\n";
   }
return 0;
}
```

Control surface generation



X: Theta [-40 deg., 40 deg] = [-0.7 rad., 0.7 rad.]

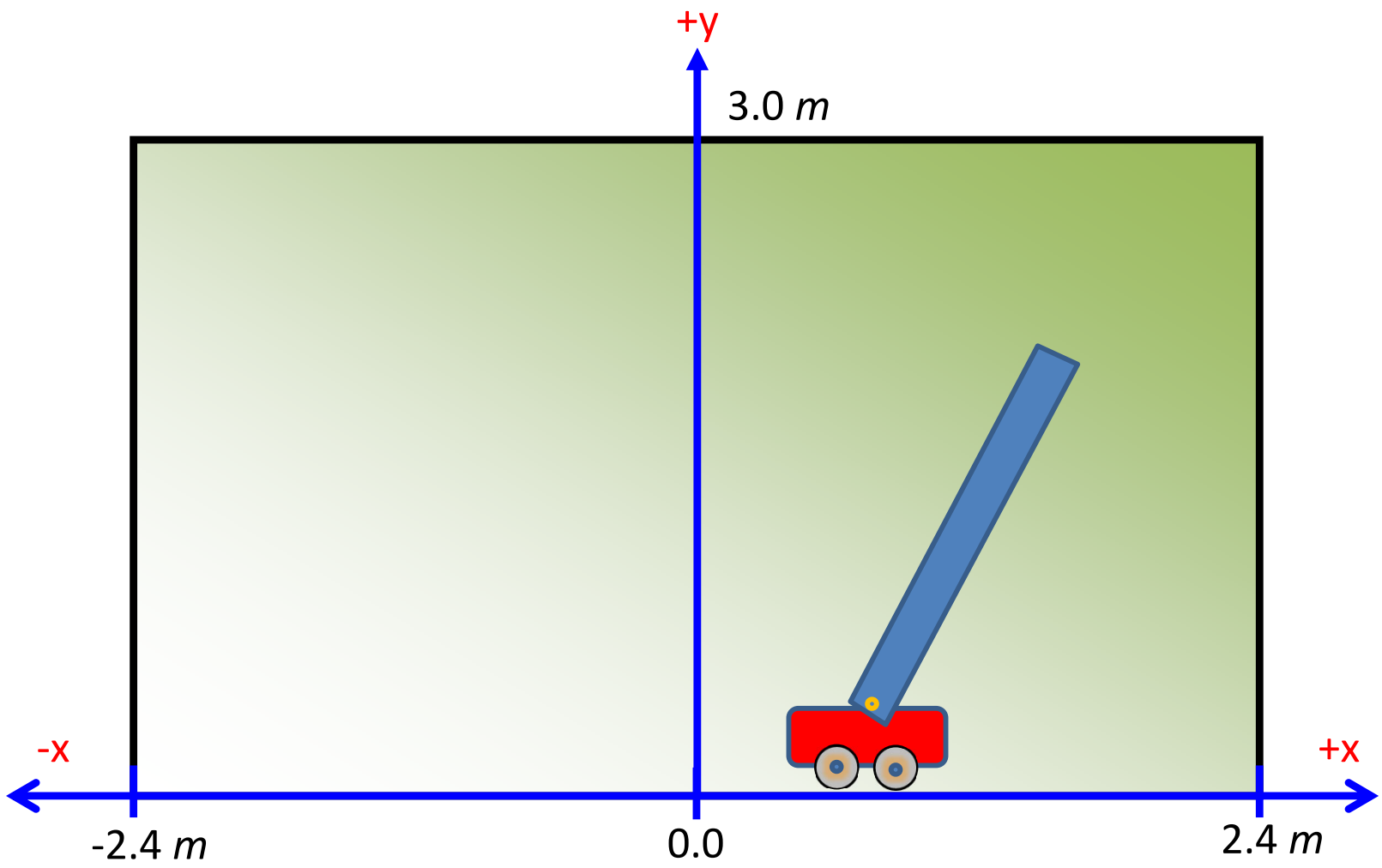
Y: Theta_dot = [-3 rad./sec, 3 rad./sec]

World Coordinate to Device Coordinate Transformation

for defining the system of coordinates, scaling and zooming-in or out.

Transformation Equations

WORLD SYSTEM OF COORDINATES



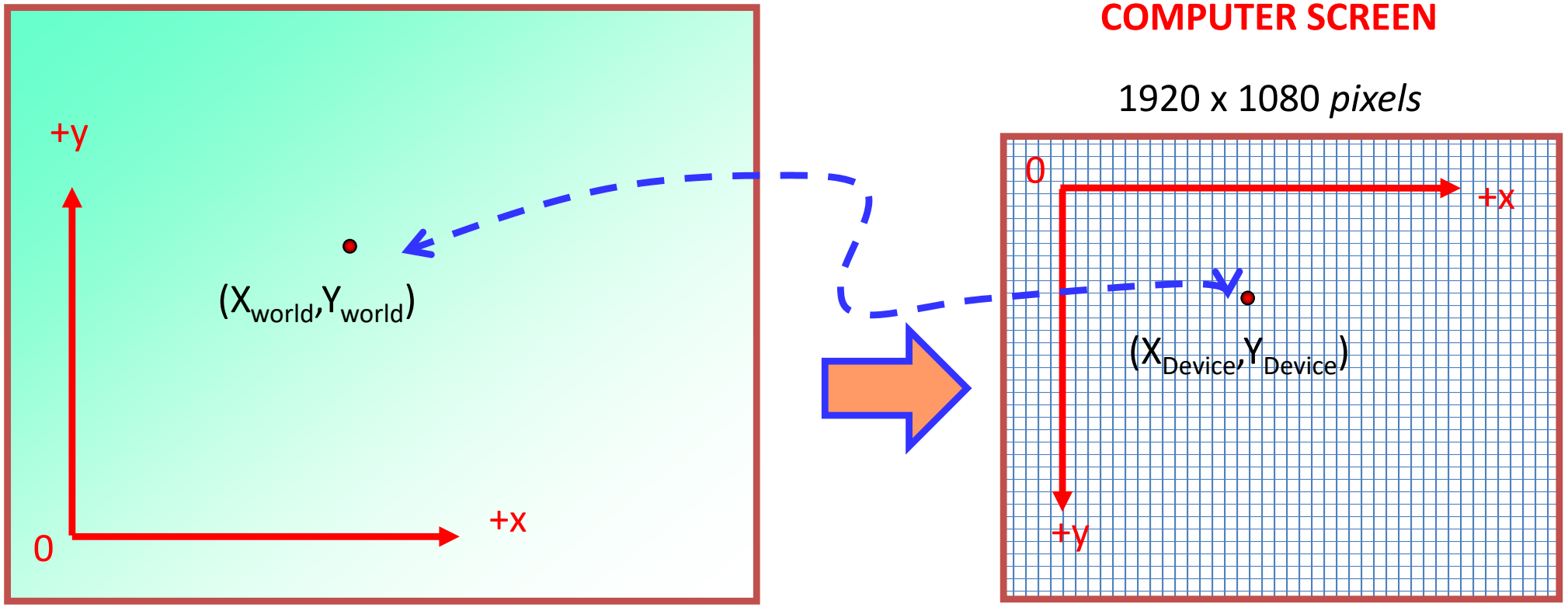
World System of Coordinates

(X_{world}, Y_{world})

Transformation Equations

WORLD-to-DEVICE COORDINATES

100,000,000 *m* x 500,000 *m*



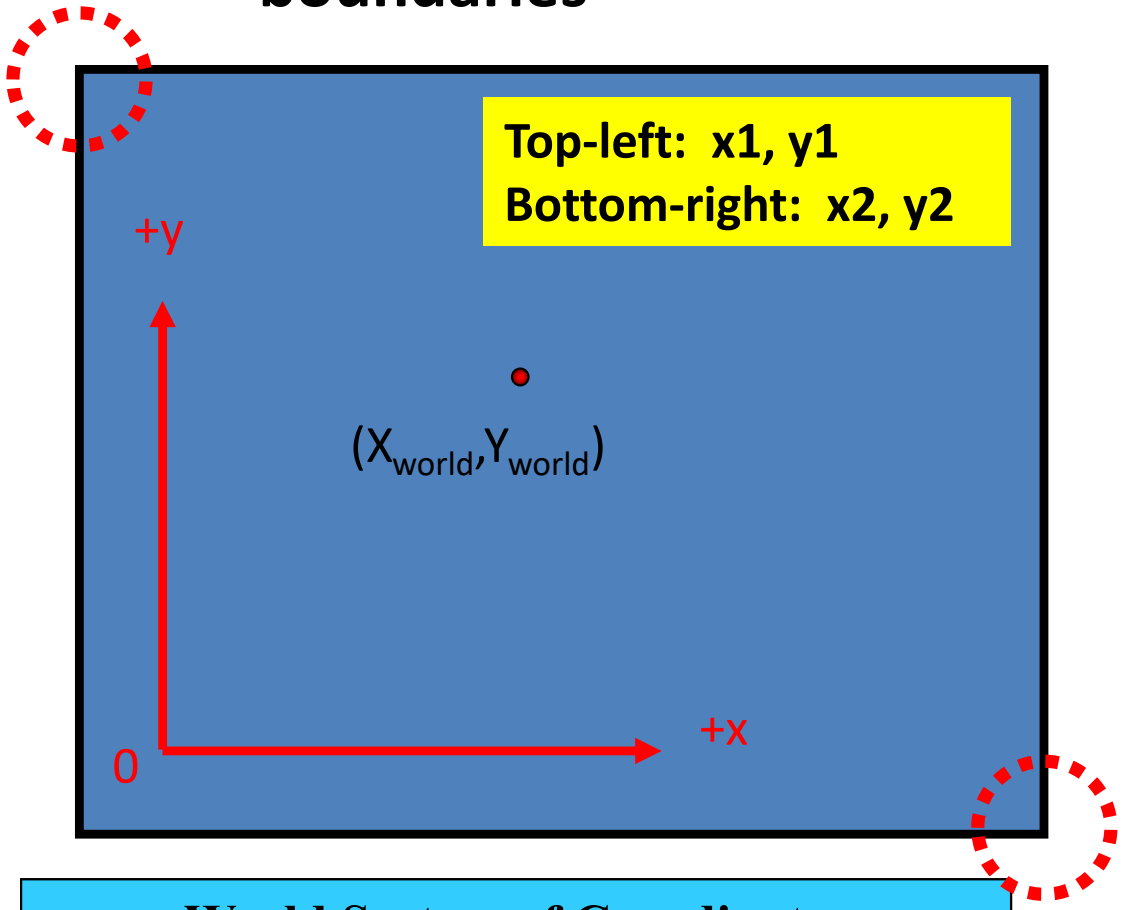
World System of Coordinates

Device System of Coordinates

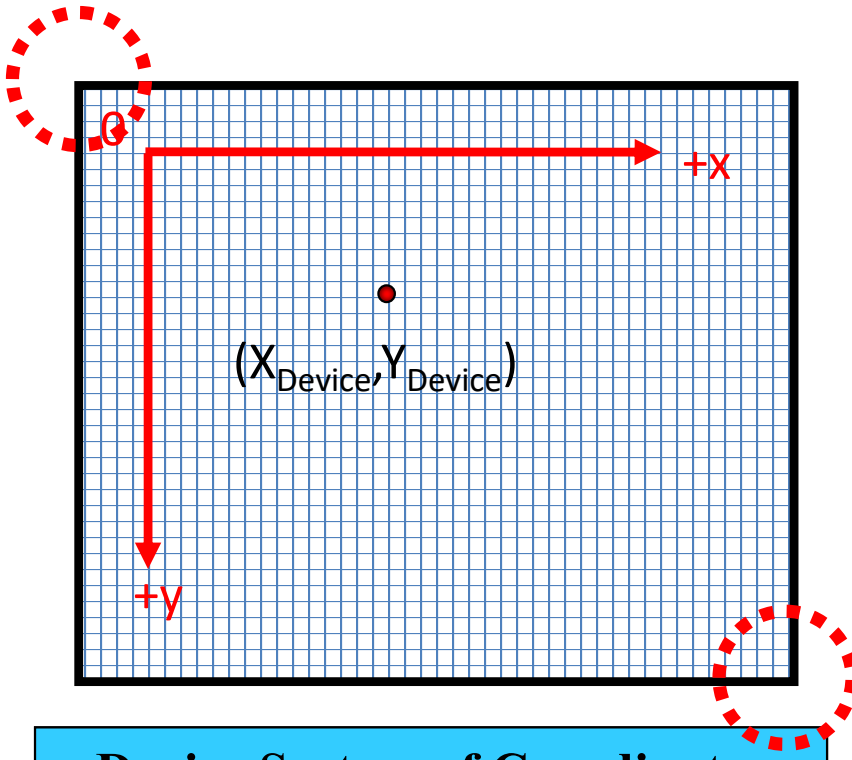
World Boundaries

SETTING THE BOUNDARIES

Use the upper-left and bottom-right coordinates to set the boundaries



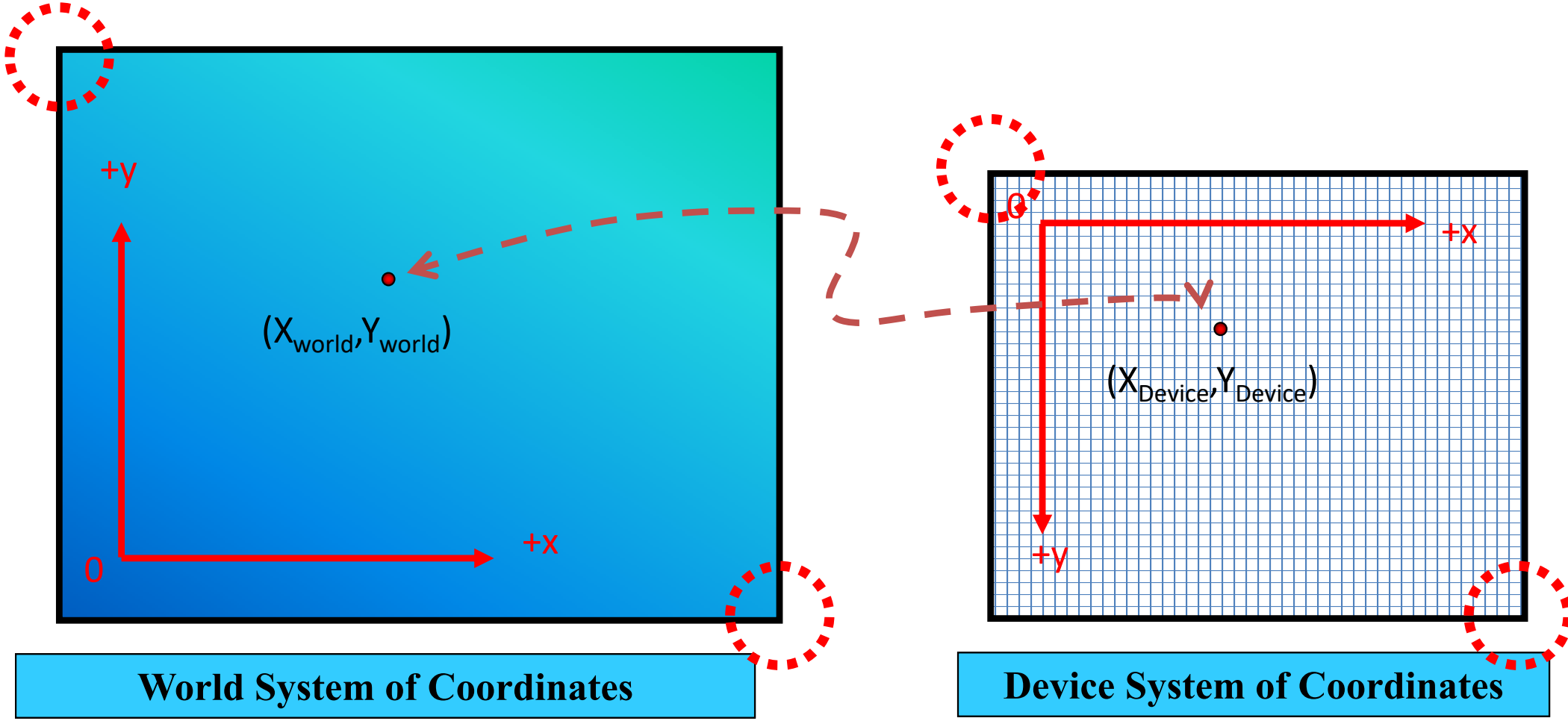
World System of Coordinates



Device System of Coordinates

Transformation

Given a world coordinate, what's the equivalent device coordinate?



World-to-Device Coordinates

- **TRANSFORMATION EQUATIONS**

$$Xslope = \frac{DeviceBound_{X_2} - DeviceBound_{X_1}}{WorldBound_{X_2} - WorldBound_{X_1}}$$

$$\text{x_intercept} = DeviceBound_{X_1} - Xslope * WorldBound_{X_1}$$

$$Device_X = Xslope * \text{World}_X + \text{x_intercept}$$

World x-coordinate

For calculating Device_y, replace all **x**'s with **y**

Demonstration

Let’s see a sample solution…

How do we generate the 3D control surface?

Some Extras...

Randomisation

Element of Surprise

srand()

Seed for random-number generation

Seed the random-number generator with current time so that the numbers will be different every time we run.

```
srand( (unsigned)time( NULL ) );

/* Display 10 numbers. */
for( i = 0;   i < 10;i++ )
   printf( "  %6d\n", rand() );
```

Element of Surprise

- **rand()**

  generates a pseudorandom number - returns int

```
int  randomVal(int min, int max)
{
        return (min + (rand() % ((max-min)+1)  ));
}
```

rand() returns a pseudo-random integral number in the range (0 to RAND_MAX-1)

Element of Surprise

- rand()

```
float randomVal(float min, float max)
{
        float r;

        r = (float)rand()/RAND_MAX;
        r = min + (r*(max-min));

        return r;
}
```

rand() returns a pseudo-random integral number in the range (0 to RAND_MAX)-1

Time elapsed, wait…

- clock()

```
void wait ( int seconds )
{
  clock_t  endwait;

  endwait = clock () + seconds * CLOCKS_PER_SEC ;
  while (clock() < endwait) {}
}
```

```
clock_t  startTime, elapsedTime;

startTime = clock();
…
...
elapsedTime = (clock() - startTime) / CLOCKS_PER_SEC;
```

Keyboard Handling

Function keys, arrow keys, Control and Shift keys, combination of keys

Keyboard Handling

GetAsyncKeyState

The GetAsyncKeyState function determines whether a key is up or down at the time the function is called, and whether the key was pressed after a previous call to GetAsyncKeyState.

```
SHORT  GetAsyncKeyState( int vKey );    // vKey - virtual-key code
```

Virtual-key code

e.g.

vk_shift

vk_control

To find other pre-defined constants:
Using ***google***, type the following keywords: **msdn vk_shift**

http://msdn.microsoft.com/en-us/library/ms645540(VS.85).aspx

Keyboard Handling

GetAsyncKeyState

```
void moveSprite()
{

  if(GetAsyncKeyState(VK_UP) < 0) {
   SpriteY = SpriteY + 2;  //up
   outtext("UP");
  }

  if(GetAsyncKeyState(VK_DOWN) < 0) {
    SpriteY = SpriteY - 2;  //down
    outtext("DOWN");
    ....
```

To find other pre-defined constants:

Using google, type the following keywords: **msdn virtual key codes**

http://msdn.microsoft.com/en-us/library/ms645540(VS.85).aspx

Keyboard Handling

GetAsyncKeyState

```
//general key press event
if(GetAsyncKeyState(VK_SPACE) < 0) {
...
}


//key down event
if((GetAsyncKeyState(VK_SPACE) & 0X0001) ==0X0001){
...
}

//key up event
if((GetAsyncKeyState(VK_SPACE) & 0X8000) ==0X8000){
...
}
```

Keyboard Handling

Monitoring the Control and Shift keys:

```
bool ControlFlag, ShiftFlag;

if(GetAsyncKeyState(VK_CONTROL)<0)
 {
     ControlFlag =! ControlFlag;
 }

if(GetAsyncKeyState(VK_SHIFT)<0)
 {
     ShiftFlag =! ShiftFlag;
 }
```

For the Tank to Jump to the Right: Control + Shift + Right Arrow key

For the Tank to Jump to the Left: Control + Shift + Left Arrow key

Keyboard Handling

Possible approach in monitoring key combinations :

```
if(GetAsyncKeyState(VK_RIGHT)<0)
  {
       XDir=RIGHT;
       if(ShiftFlag)
        {
          outtext("SHIFT + RIGHT");
          ShiftFlag=!ShiftFlag;
        }

       if(ControlFlag)
        {
          outtext("CTRL + RIGHT");
          if (TankX < getmaxx()-W) TankX += 2;
          Angle=Angle-5;
          RaiseWheelFlag=TRUE;
          ControlFlag=!ControlFlag;
        }

       ...
```

End of Presentation

Let me answer some of your questions…

*

Tutorial: FUZZY LOGIC

Using the given FAMM, output values and membership functions, calculate the crisp output of the fuzzy system when **x** = -0.7 and **y** = -3 .

Fuzzy Associative Memory Matrix (FAMM)

Outputs:
NL=-5
NS=-2.5
ZE=0
PS=2.5
PL=5.0

y \ x	N	ZE	P
S	NL	NS	NS
M	NS	ZE	PS
L	PS	PS	PL

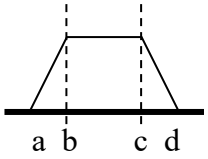
Fuzzy Associative Memory Matrix (FAMM)

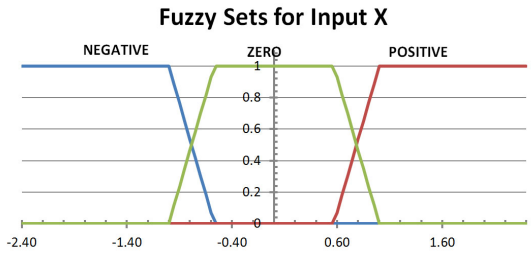
Assume that the fuzzy combination operator used is the Zadeh AND.

y \ x	N	ZE	P
S	**NL**	**NS**	**NS**
M	**NS**	**ZE**	**PS**
L	**PS**	**PS**	**PL**

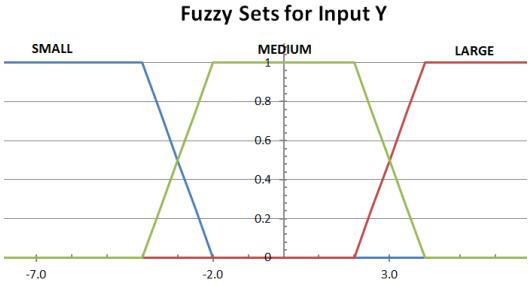
y \ x	N	ZE	P
S	**W1**	**W4**	**W7**
M	**W2**	**W5**	**W8**
L	**W3**	**W6**	**W9**

All fuzzy sets are of type trapezoidal.





	a	b	c	d
NEG	0	0	-1	-0.57
ZE	-1	-0.57	0.57	1
POS	0.57	1	0	0



	a	b	c	d
S	0	0	-4	-2
M	-4	-2	2	4
L	2	4	0	0

n.h.reyes@massey.ac.nz

Fuzzy Logic Tutorial

2 Inputs, 9 rules

n.h.reyes@massey.ac.nz

Fuzzy Logic Tutorial

Using the given FAMM, output values and membership functions, calculate the crisp output of the fuzzy system when **x** = -0.7 and **y** = -3 .

Assume that the fuzzy combination operator used is the **Zadeh AND**.

Outputs:
NL=-5
NS=-2.5
ZE=0
PS=2.5
PL=5.0

y \ x	N	ZE	P
S	**NL**	**NS**	**NS**
M	**NS**	**ZE**	**PS**
L	**PS**	**PS**	**PL**

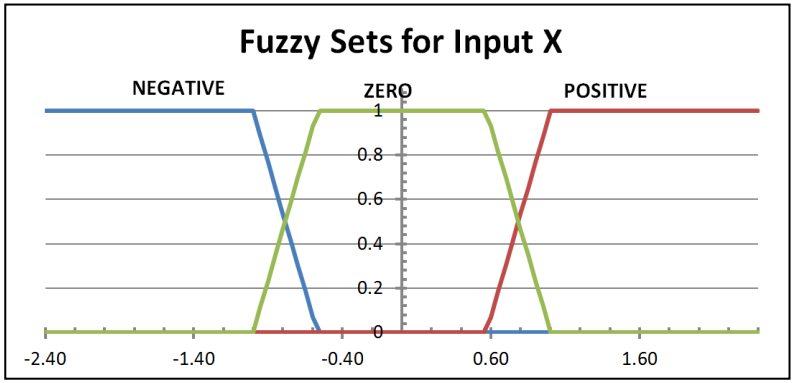
y \ x	N	ZE	P
S	**W1**	**W4**	**W7**
M	**W2**	**W5**	**W8**
L	**W3**	**W6**	**W9**

This is used only to refer to each of the rules individually

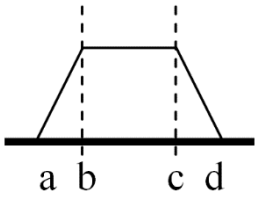
Fuzzy Logic Tutorial

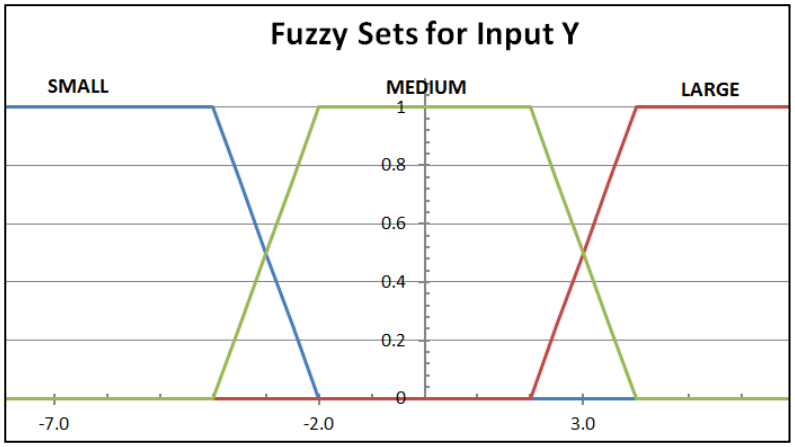
y \ x	N	ZE	P
S	NL	NS	NS
M	NS	ZE	PS
L	PS	PS	PL

All fuzzy sets are of type trapezoidal.



	a	b	c	d
NEG	0	0	-1	-0.57
ZE	-1	-0.57	0.57	1
POS	0.57	1	0	0

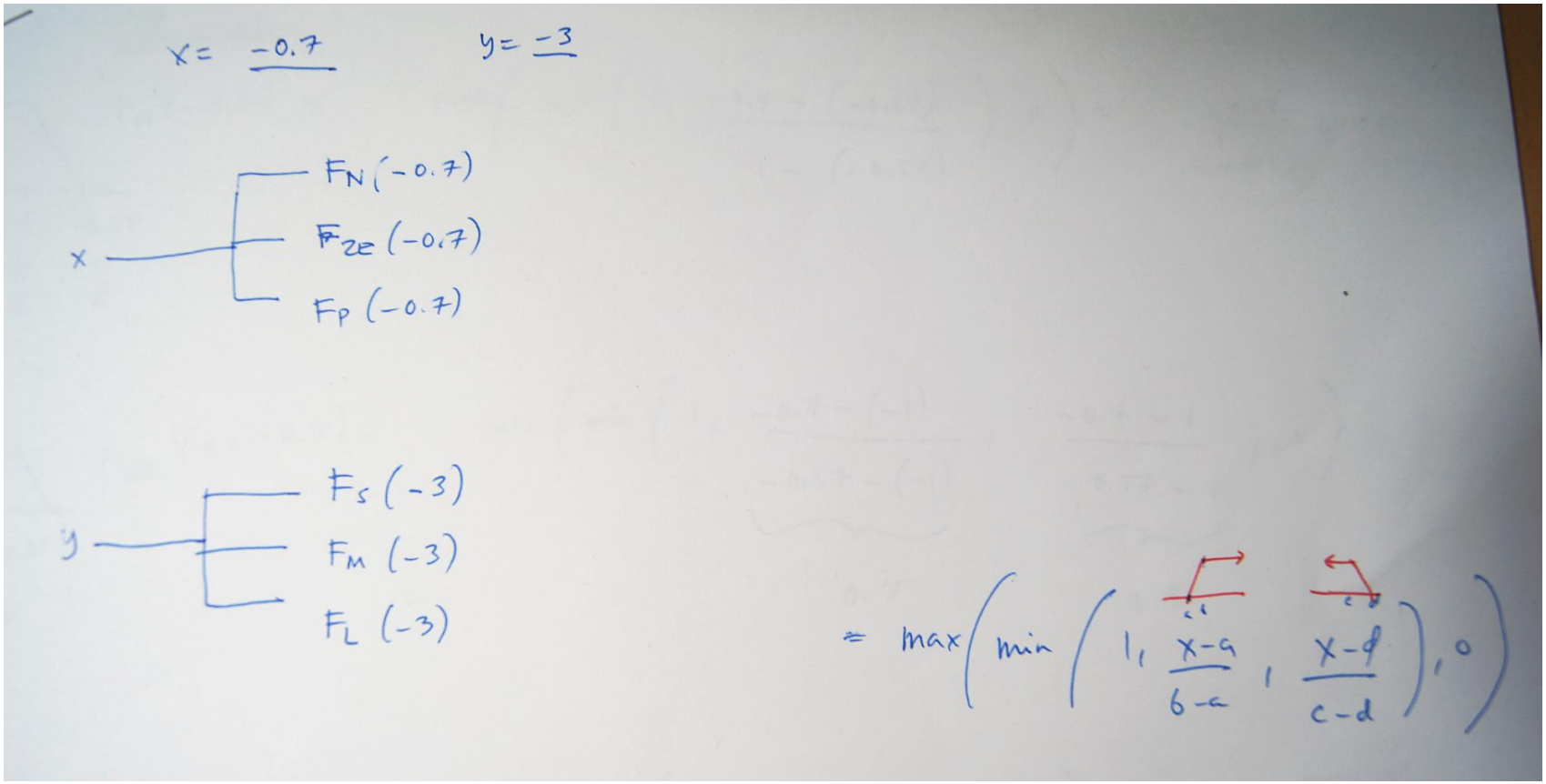




	a	b	c	d
S	0	0	-4	-2
M	-4	-2	2	4
L	2	4	0	0

Fuzzy Logic Tutorial

Fuzzify the inputs



Fuzzy Logic Tutorial

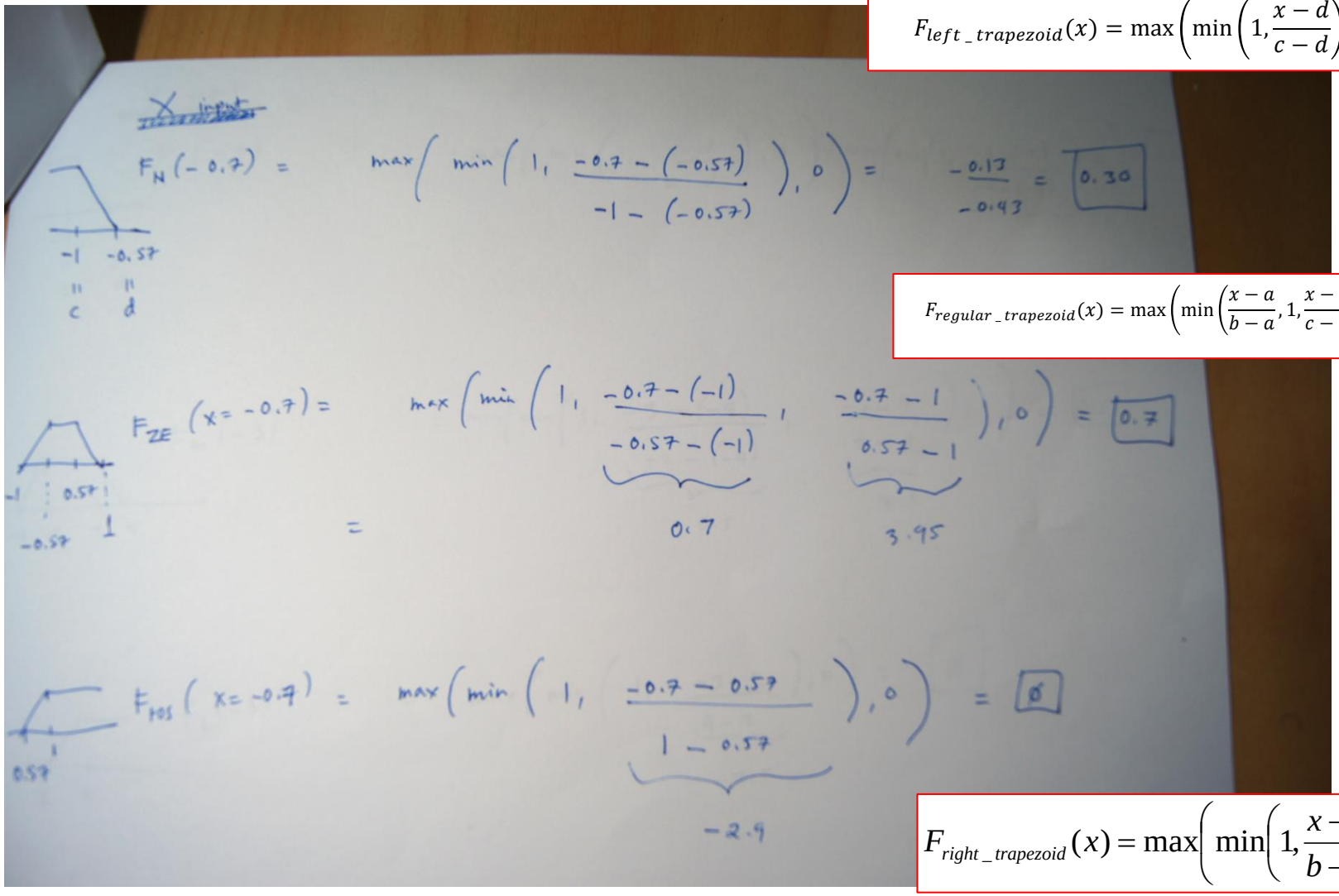
	a	b	c	d
NEG	0	0	-1	-0.57
ZE	-1	-0.57	0.57	1
POS	0.57	1	0	0

Fuzzify the inputs: **INPUT VARIABLE X**

$$F_{left_trapezoid}(x) = \max\left(\min\left(1, \frac{x-d}{c-d}\right), 0\right)$$

$$F_{regular_trapezoid}(x) = \max\left(\min\left(\frac{x-a}{b-a}, 1, \frac{x-d}{c-d}\right), 0\right)$$

$$F_{right_trapezoid}(x) = \max\left(\min\left(1, \frac{x-a}{b-a}\right), 0\right)$$



Fuzzy Logic Tutorial

	a	b	c	d
S	0	0	-4	-2
M	-4	-2	2	4
L	2	4	0	0

Fuzzify the inputs: **INPUT VARIABLE Y**

$$F_{left_trapezoid}(x) = \max\left(\min\left(1, \frac{x-d}{c-d}\right), 0\right)$$

$$F_{regular_trapezoid}(x) = \max\left(\min\left(\frac{x-a}{b-a}, 1, \frac{x-d}{c-d}\right), 0\right)$$

$$F_{right_trapezoid}(x) = \max\left(\min\left(1, \frac{x-a}{b-a}\right), 0\right)$$

Fuzzy Logic Tutorial

Compute degree of applicability of rules: **WEIGHTS**

DEFUZZIFICATION: compute crisp output using **CENTRE OF MASS**

F_Neg(X) = 0.3023256

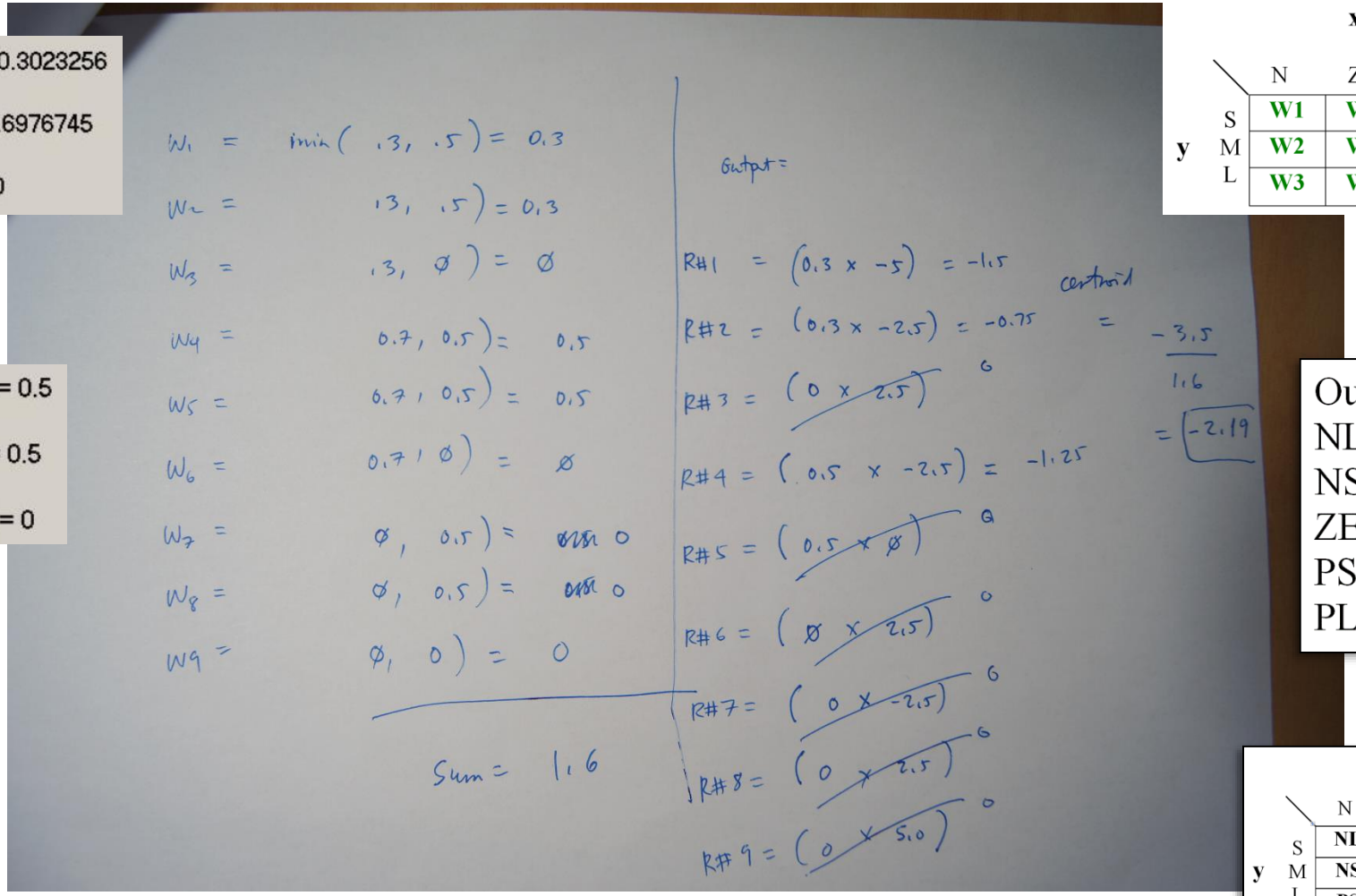
F_ZE(X) = 0.6976745

F_Pos(X) = 0

F_Small(Y) = 0.5

F_Med(Y) = 0.5

F_Large(Y) = 0



y \ x	N	ZE	P
S	W1	W4	W7
M	W2	W5	W8
L	W3	W6	W9

Outputs:
NL=-5
NS=-2.5
ZE=0
PS=2.5
PL=5.0

y \ x	N	ZE	P
S	NL	NS	NS
M	NS	ZE	PS
L	PS	PS	PL

Fuzzy Logic Tutorial

A) Degree of Memberships of Input X

F_Neg(X) = 0.3023256

F_ZE(X) = 0.6976745

F_Pos(X) = 0

B) Degree of Memberships of Input Y

F_Small(Y) = 0.5

F_Med(Y) = 0.5

F_Large(Y) = 0

C) Weights

Weight(1) = Min(F_Neg(X),F_Small(Y)) = Min(0.3023256, 0.5)= 0.3023256
Weight(2) = Min(F_Neg(X),F_Med(Y)) = Min(0.3023256, 0.5)= 0.3023256
Weight(3) = Min(F_Neg(X),F_Large(Y)) = Min(0.3023256, 0)= 0
Weight(4) = Min(F_ZE(X),F_Small(Y)) = Min(0.6976745, 0.5)= 0.5
Weight(5) = Min(F_ZE(X),F_Med(Y)) = Min(0.6976745, 0.5)= 0.5
Weight(6) = Min(F_ZE(X),F_Large(Y)) = Min(0.6976745, 0)= 0
Weight(7) = Min(F_Pos(X),F_Small(Y)) = Min(0, 0.5)= 0
Weight(8) = Min(F_Pos(X),F_Med(Y)) = Min(0, 0.5)= 0
Weight(9) = Min(F_Pos(X),F_Large(Y)) = Min(0, 0)= 0

D) Crisp Output

Numerator/Denominator = -3.517442/1.604651

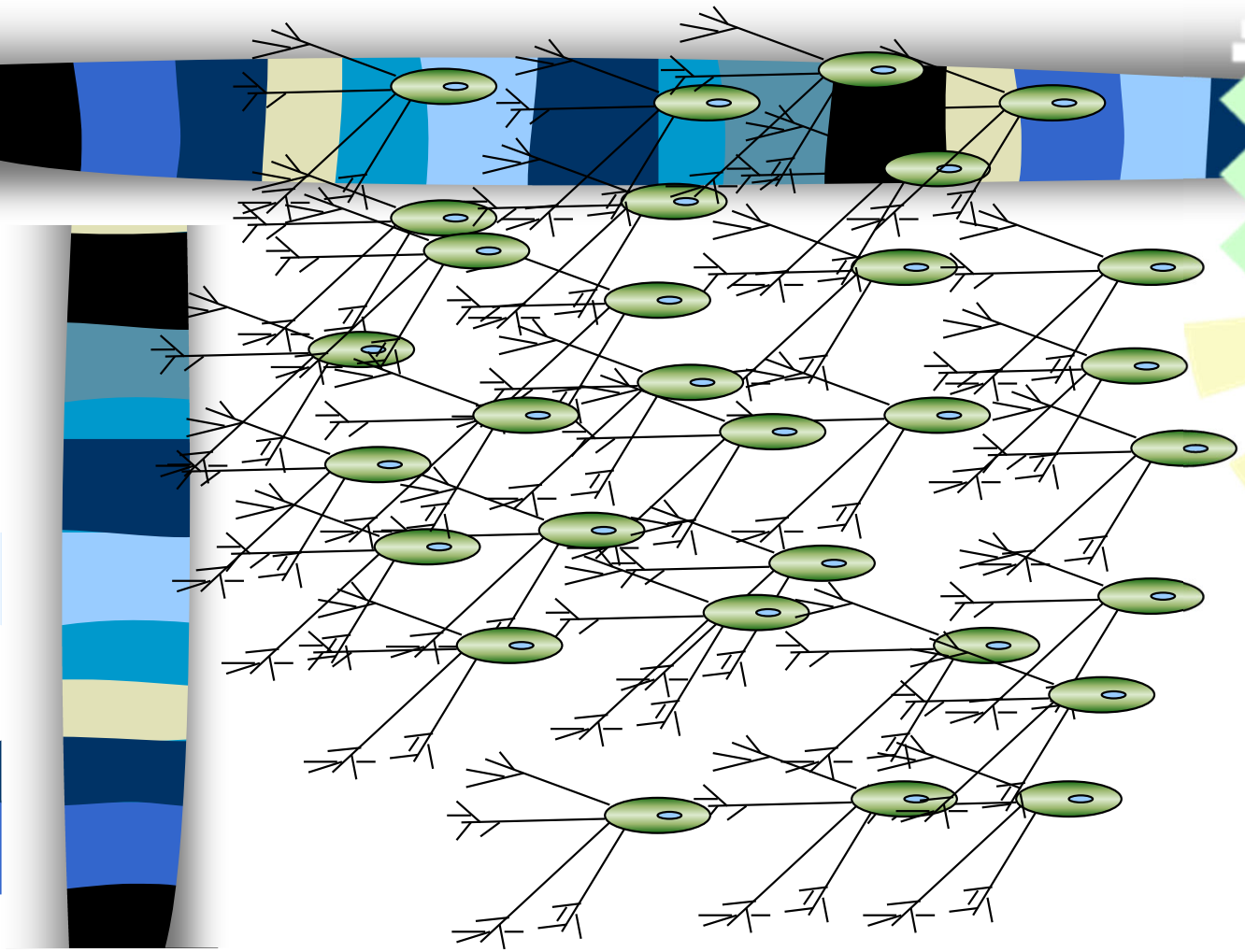
CrispOutput = -2.192029

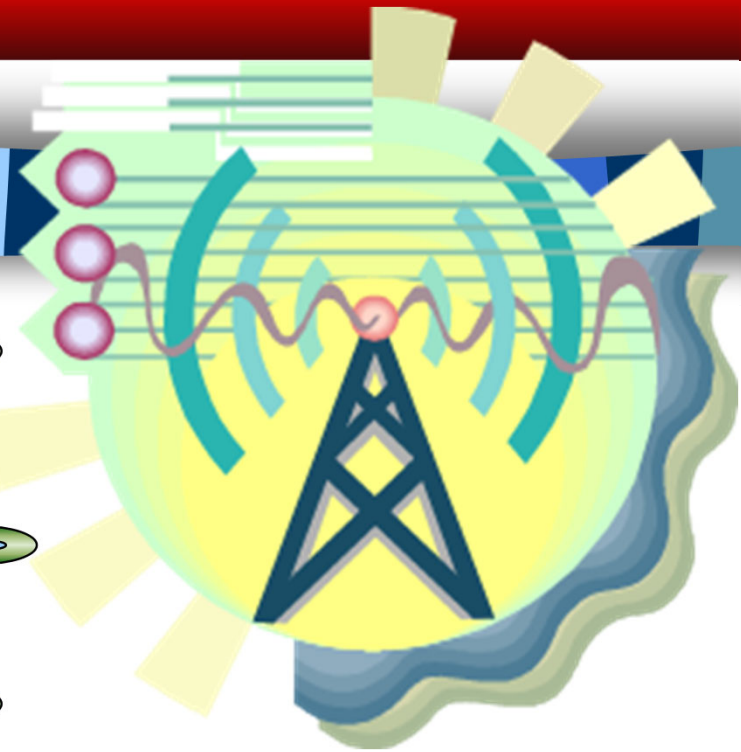
INPUT VALUES

X -0.7 Y -3

Summary of steps

Perceptrons





n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Multi-layer Perceptron

Overview

In order to understand the modern deep learning systems of today, we need to take a step back and learn Perceptrons first.

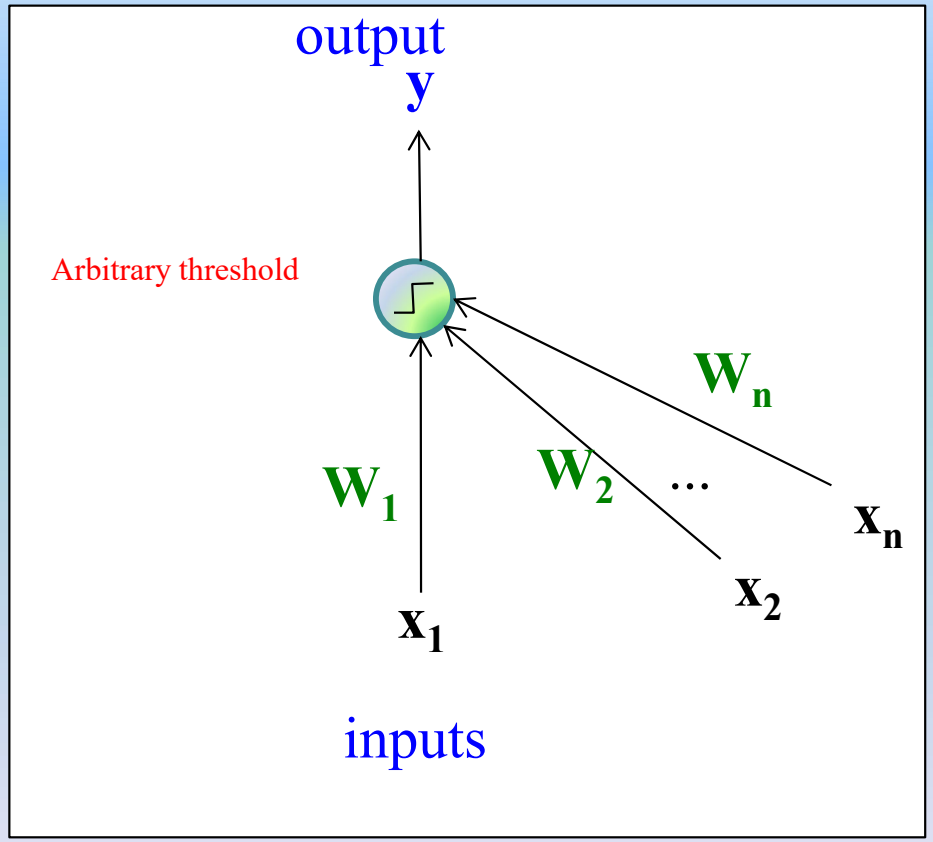
What is the role of a perceptron unit?

What is the advantage of organising them into layers?

What can we achieve if we construct a deep network of perceptron units?

How to train a network of perceptrons?

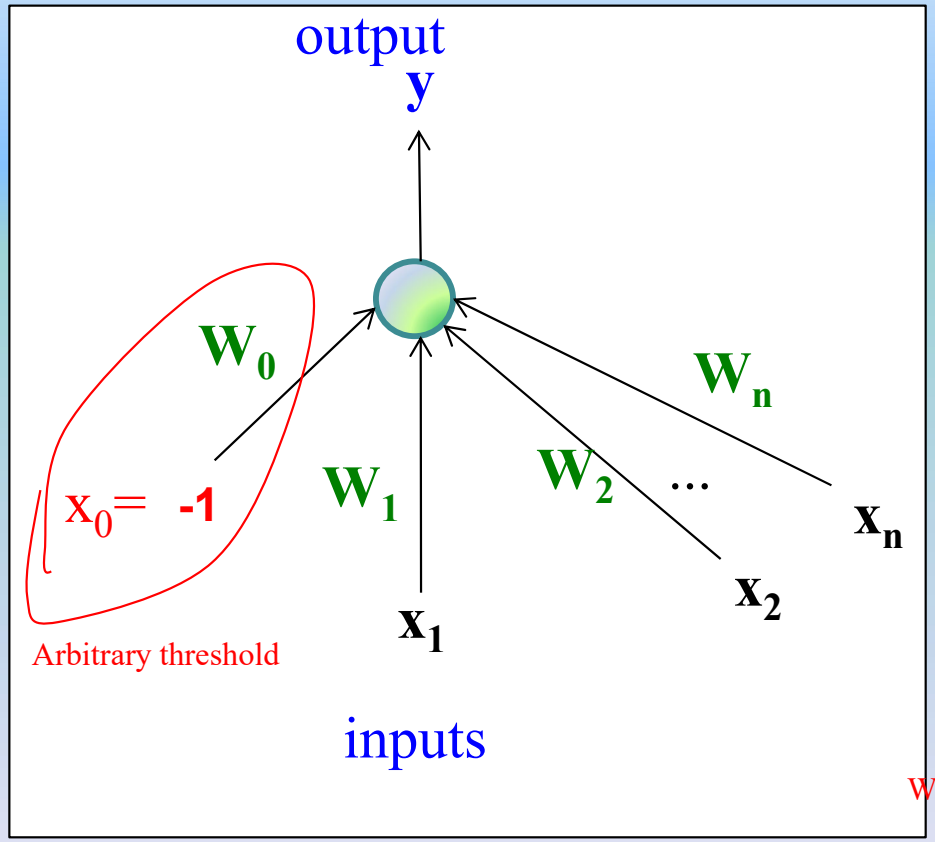
Single Perceptron Unit



A perceptron unit basically compares a weighted combination of its inputs against a **threshold** value and then outputs a **1** if the weighted inputs exceed the threshold.

$$h(x) = \theta(\mathbf{w} \cdot \mathbf{x} + b) \equiv \theta(\overline{\mathbf{w}} \cdot \overline{\mathbf{x}})$$

Single Perceptron Unit



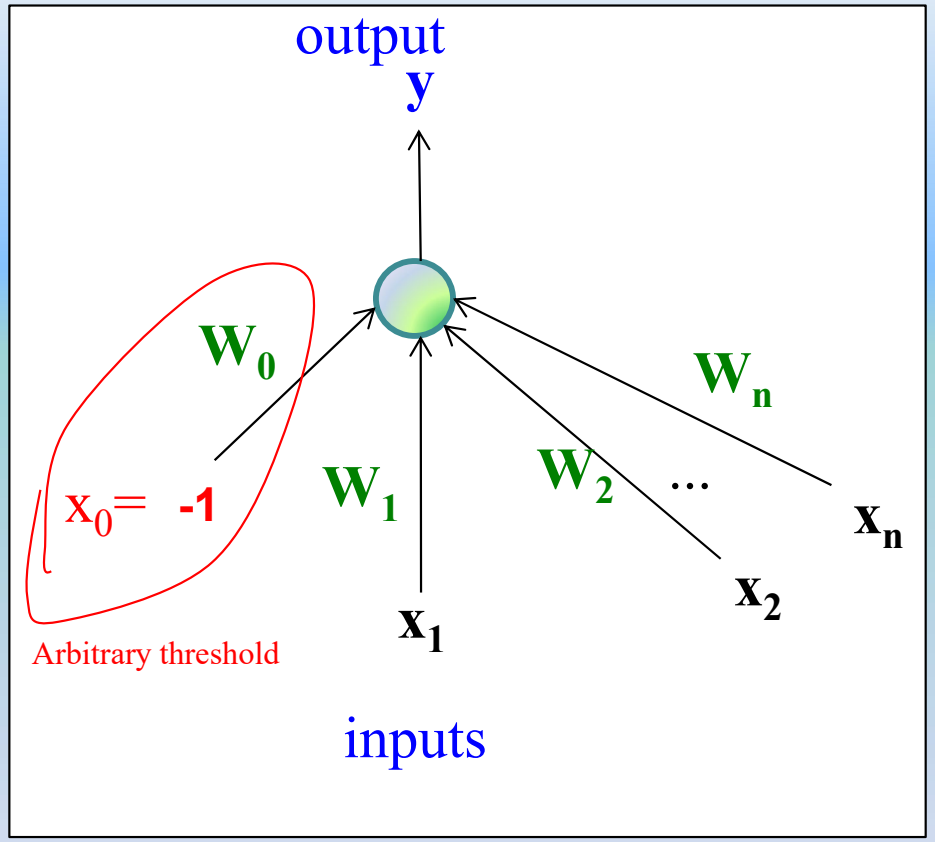
Basic rule of operation:

Compute the weighted sum of all the inputs then compare the result to zero.

$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Weighted input

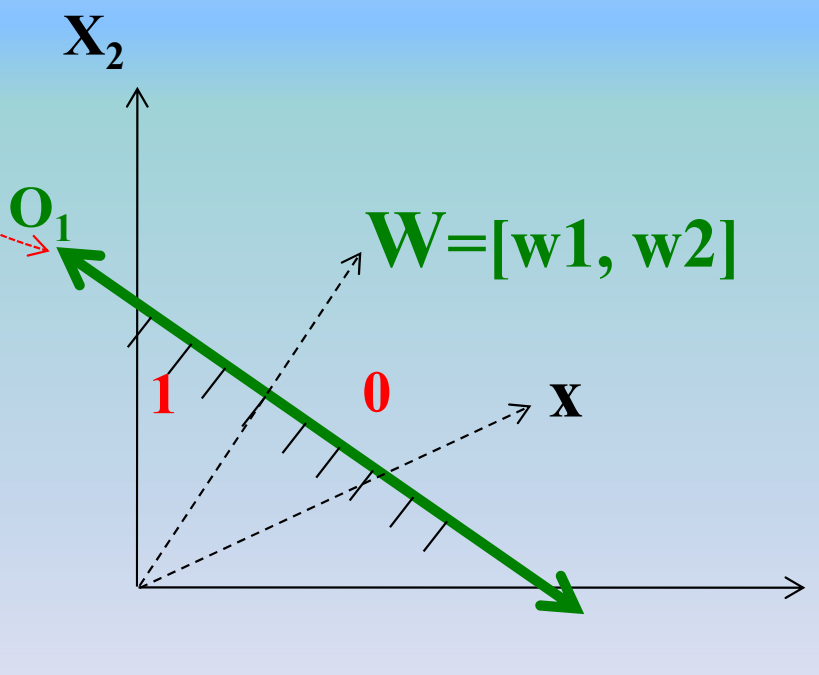
Linear Classifier Single Perceptron Unit



$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

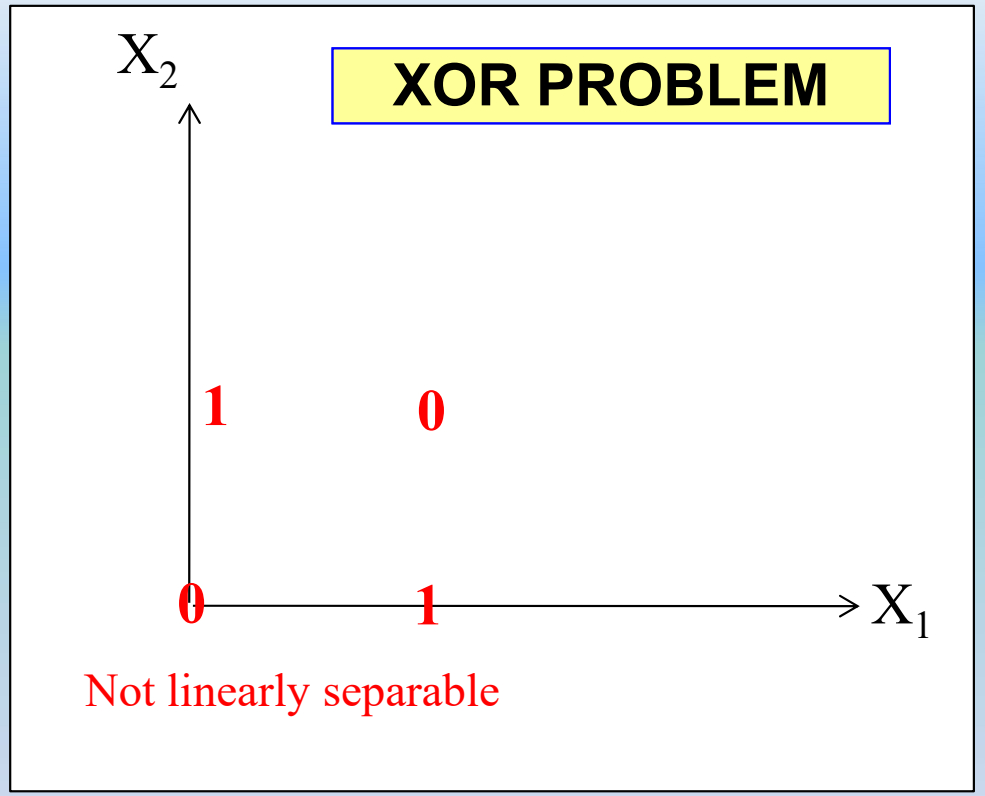
Weighted input

The decision boundary for a single perceptron is a hyperplane in the feature space.



Linear Classifier Single Perceptron Unit

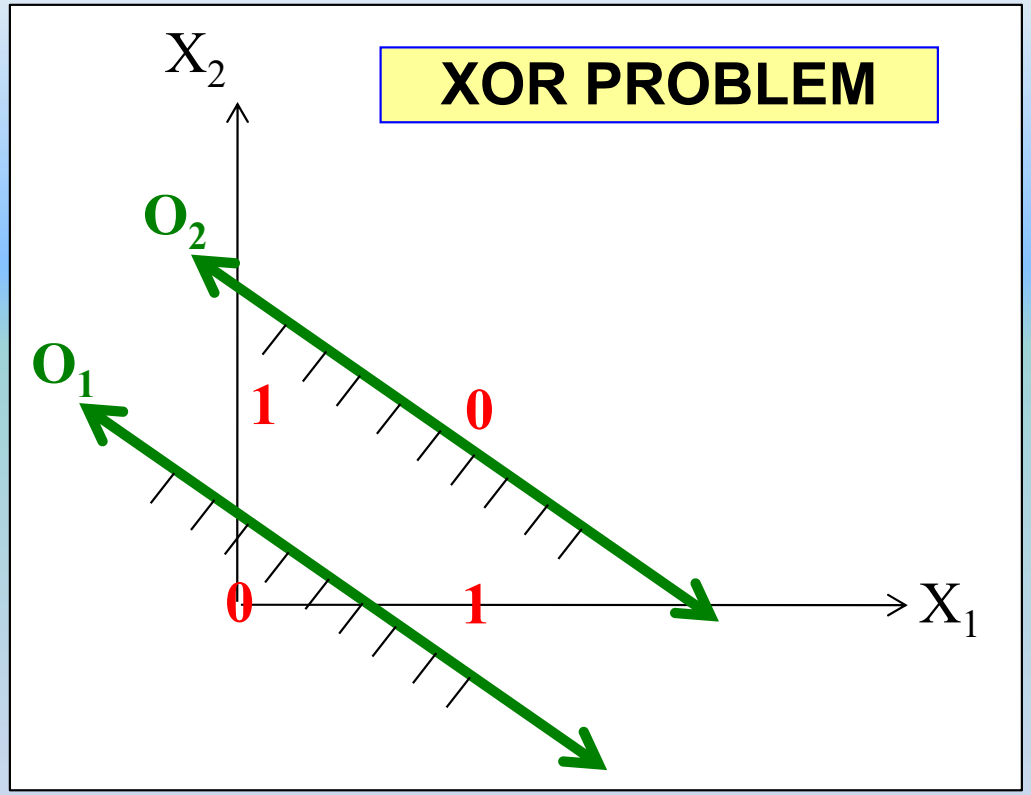
Beyond Linear Separability



Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

A problem like that illustrated by the values of the **XOR** boolean function **cannot** be solved by a **single perceptron unit**.

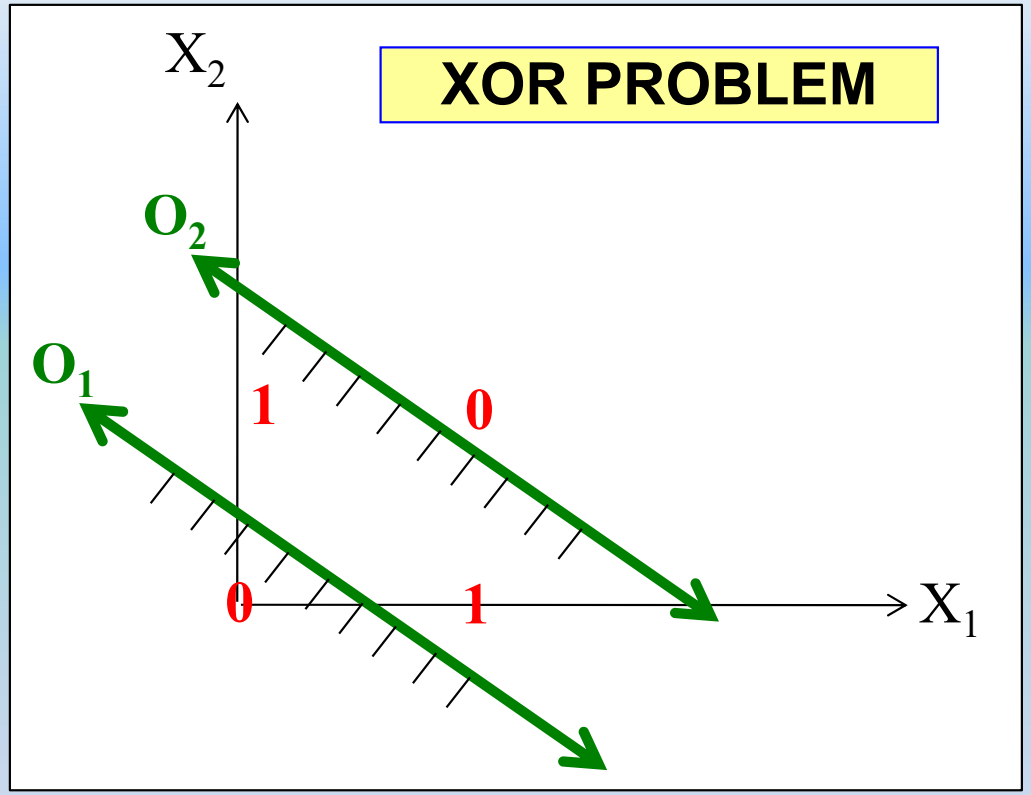
Beyond Linear Separability



Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Beyond Linear Separability

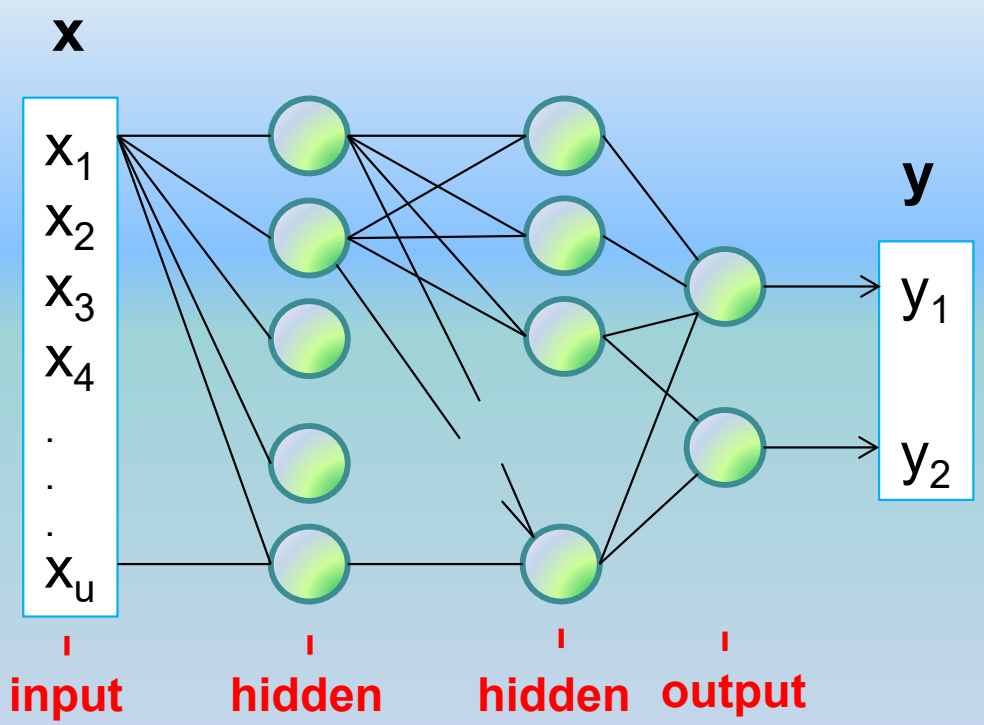


Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

Multi-Layer Perceptron



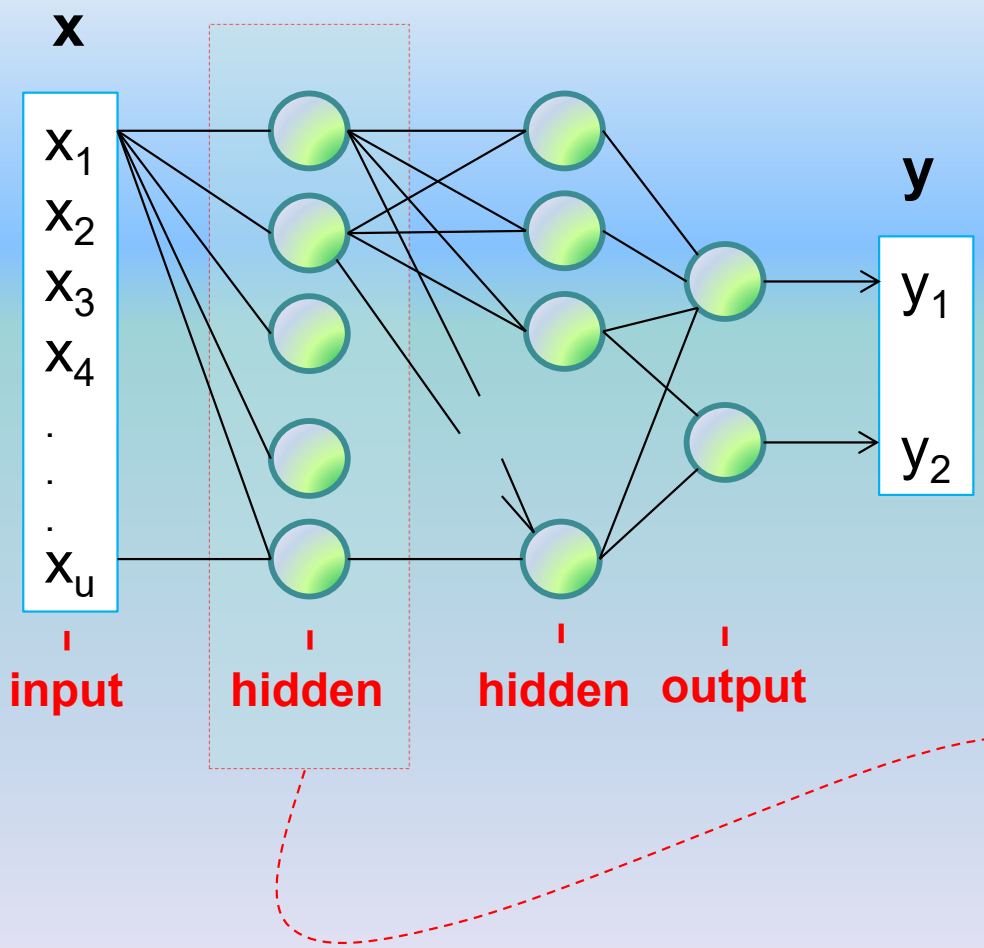
Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

The introduction of "hidden" (not connected directly to the output) units into these networks make them much more powerful: they are no longer limited to linearly separable problems.

Multi-Layer Perceptron



Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

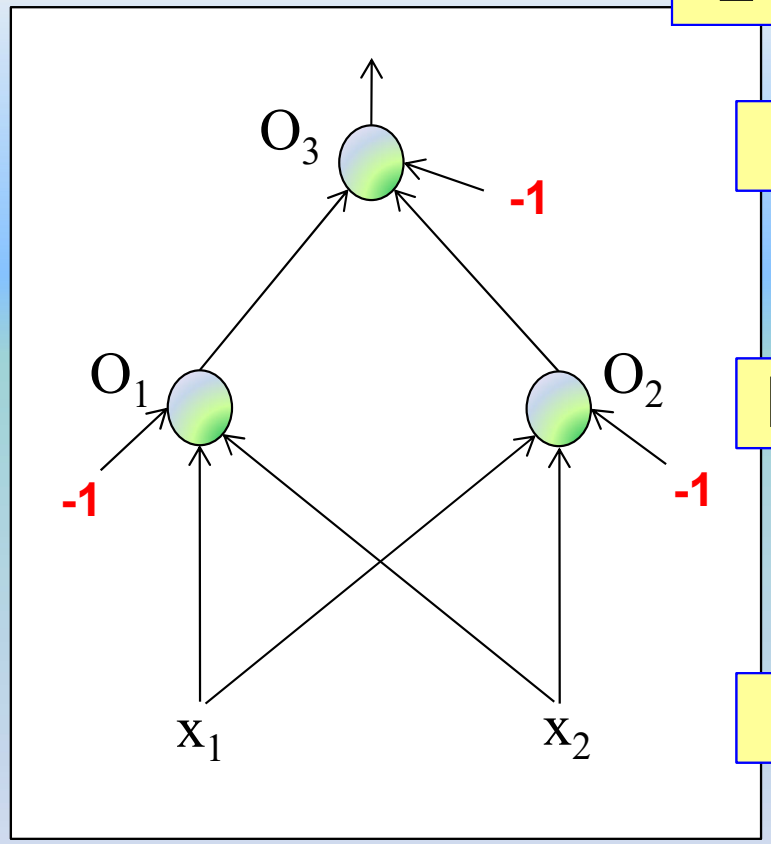
The introduction of “hidden” (not connected directly to the output) units into these networks make them much more powerful: they are no longer limited to linearly separable problems.

The lower layers transform the input problem into more tractable (linearly separable) problems for subsequent layers.

MLP for Solving the XOR Problem

Example

2-Layer Perceptron Network



Output units

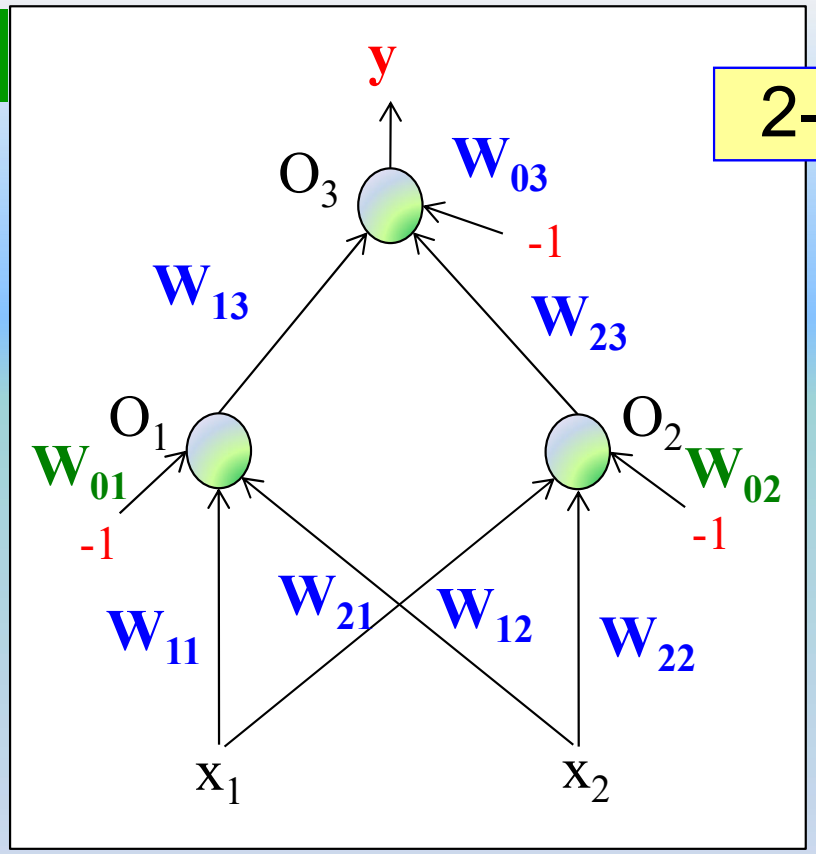
Hidden units

Input units

Interconnected network of simple units in which each connection has a weight.

Multilayer Layer Perceptron

Example



2-Layer Perceptron Network

Hidden units:

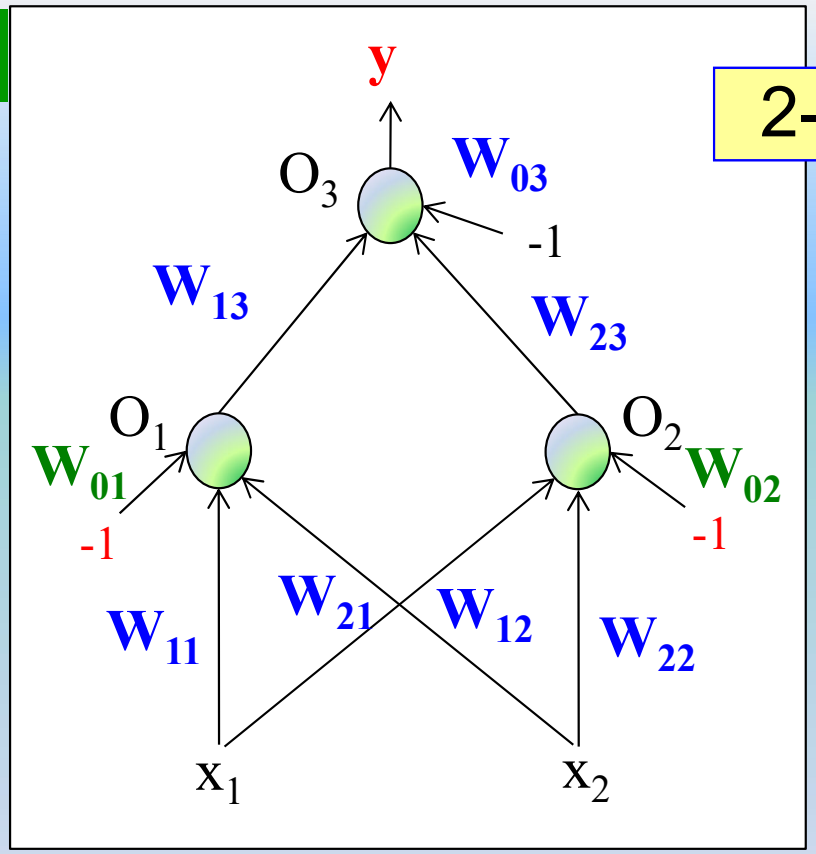
Weight for the link connecting input $\mathbf{x_1}$ to hidden node 2.

$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Multilayer Layer Perceptron

Example



2-Layer Perceptron Network

Hidden units:

$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

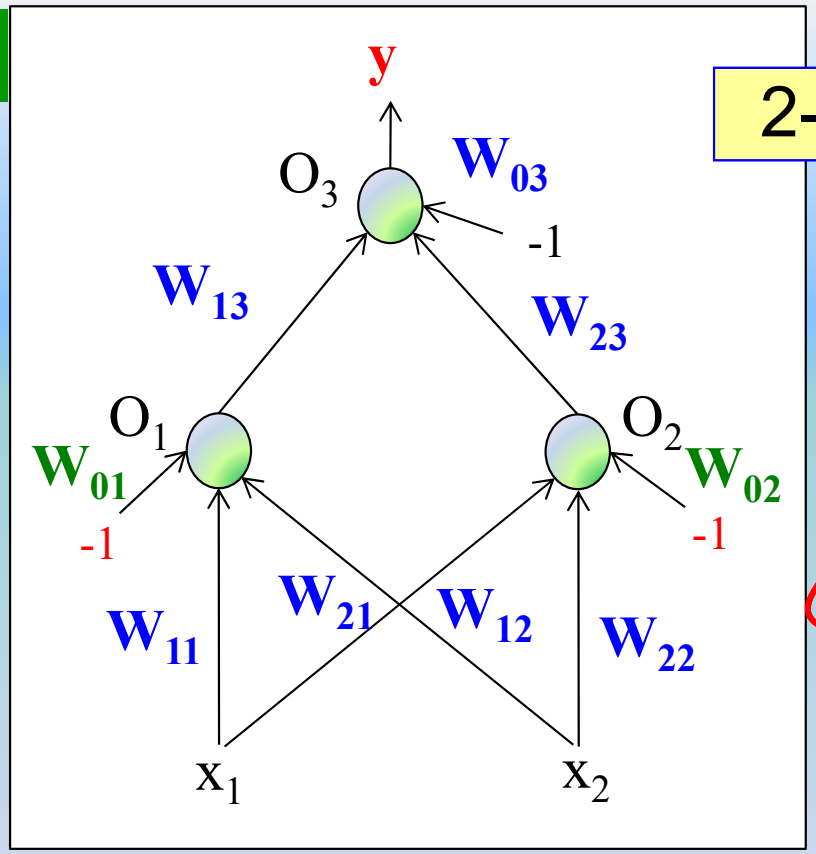
$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Each perceptron unit is a linear classifier, whose arbitrary threshold is defined by a weight W_{0i} and fixed input -1.

W_{0i} affects the **slope** and **offset** of the linear classifier from the origin (more on this later).

Multilayer Layer Perceptron

Example



2-Layer Perceptron Network

Hidden units:

$$(x_1 * w_{11}) + (x_2 * w_{21}) - w_{01} = O_1$$

$$(x_1 * w_{12}) + (x_2 * w_{22}) - w_{02} = O_2$$

Normal vector to the decision boundary

$$\mathbf{w}_1 = [w_{11}, w_{21}]$$

$$\mathbf{w}_2 = [w_{12}, w_{22}]$$

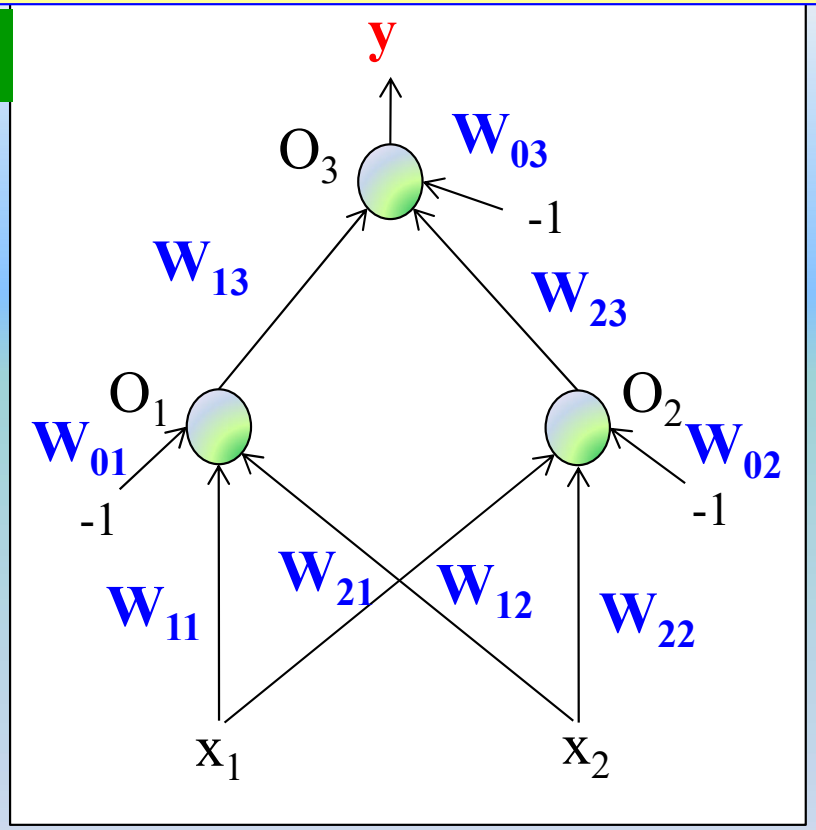
$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Multilayer Layer Perceptron

2-Layer Perceptron Network

Example



$$(x_1 * w_{11}) + (x_2 * w_{21}) - w_{01} = O_1$$

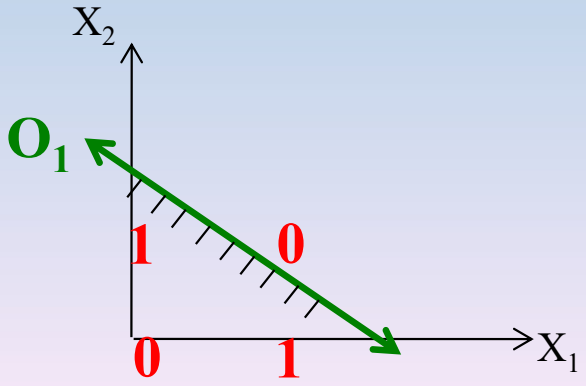
1 1 3/2

Slope of linear classifier $O_1 = -1$

Normal vector to the decision boundary

$$\mathbf{w}_1 = [w_{11}, w_{21}]$$

Offset (distance from origin) = 3/2

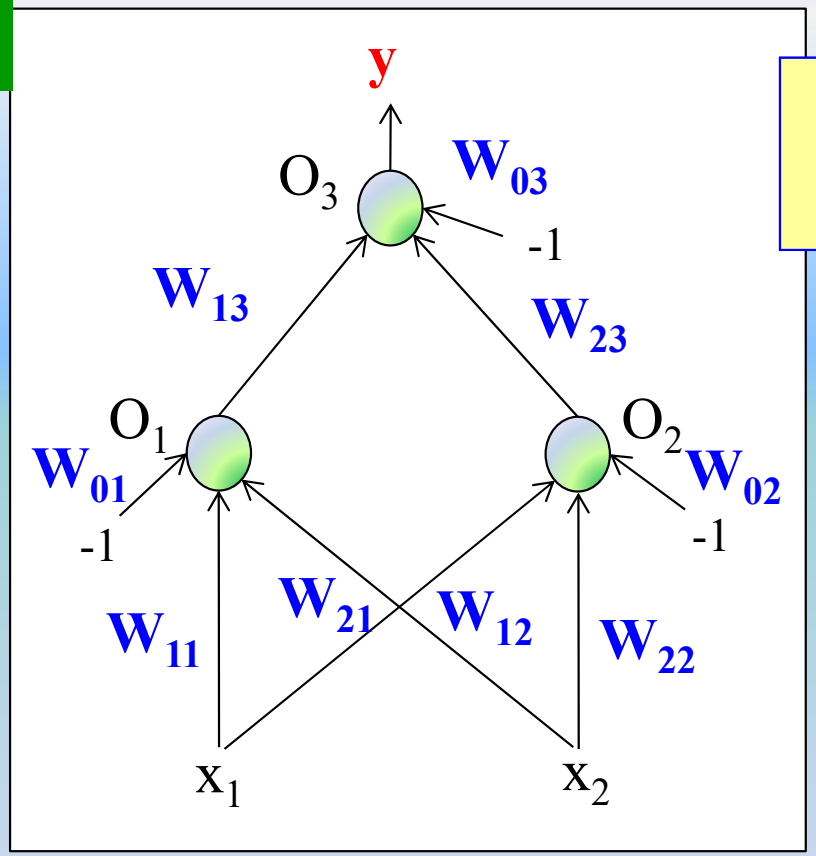


$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Multilayer Layer Perceptron

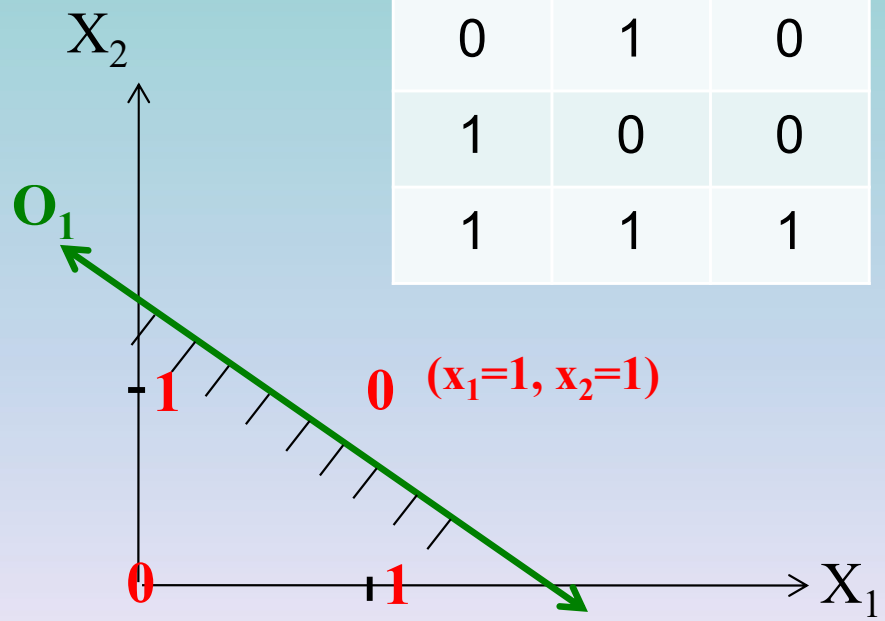
Example



HIDDEN UNIT (O_1)

DECISION BOUNDARY

X_1	X_2	O_1
0	0	0
0	1	0
1	0	0
1	1	1



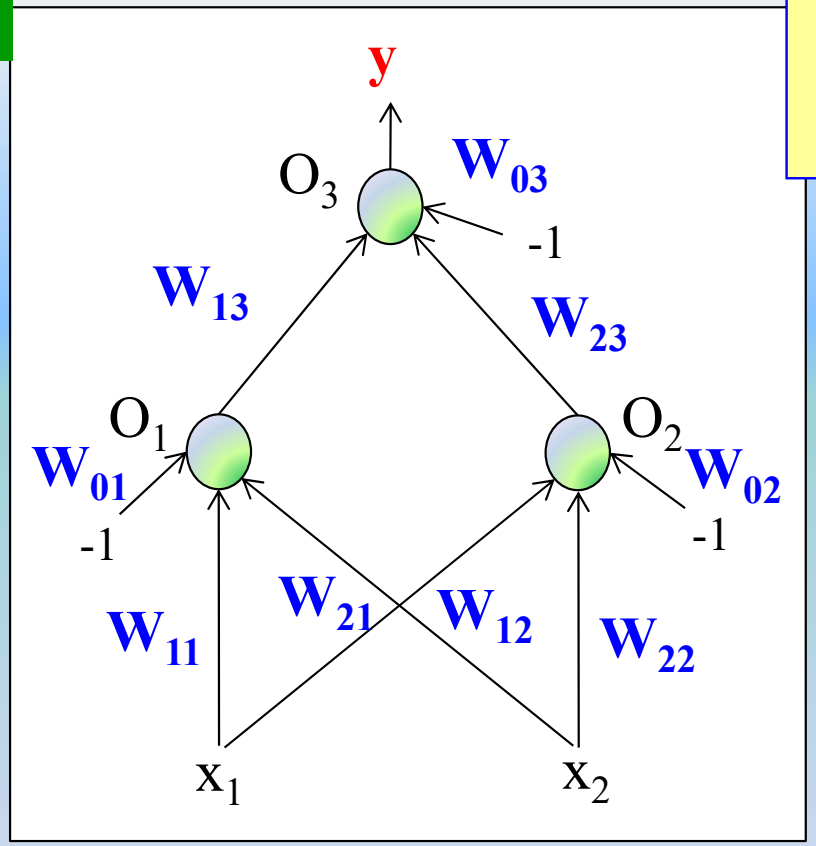
$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

Each hidden unit defines its own "decision boundary" and the output from each of these units is fed to the output unit.

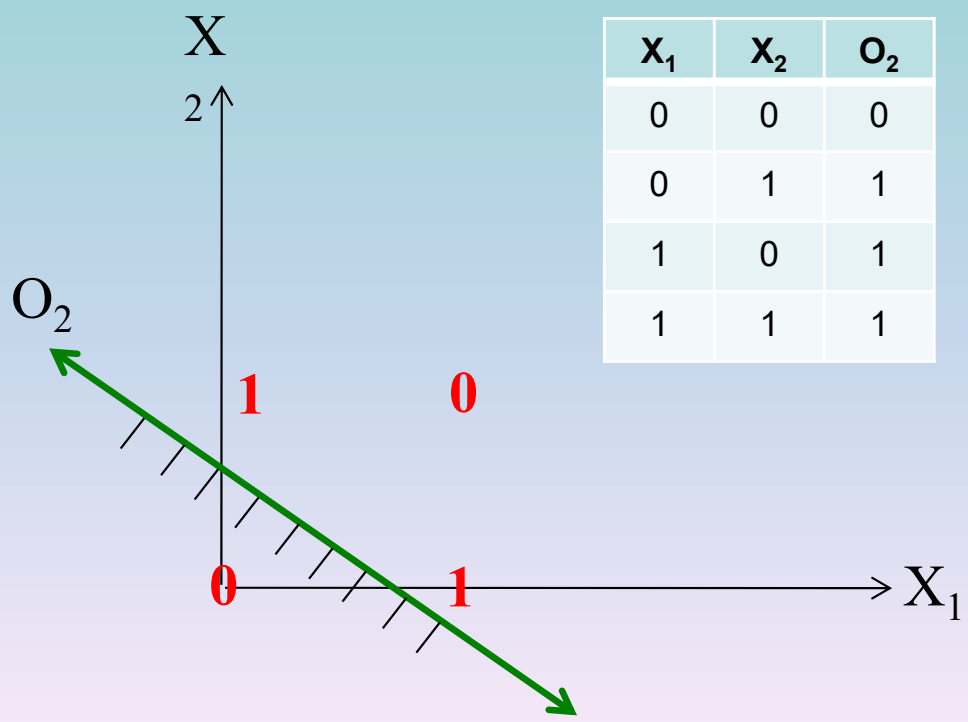
Multilayer Layer Perceptron

Example



HIDDEN UNIT (O_2)
DECISION BOUNDARY

$$(x_1 * w_{12}) + (x_2 * w_{22}) - w_{02} = O_2$$



X_1	X_2	O_2
0	0	0
0	1	1
1	0	1
1	1	1

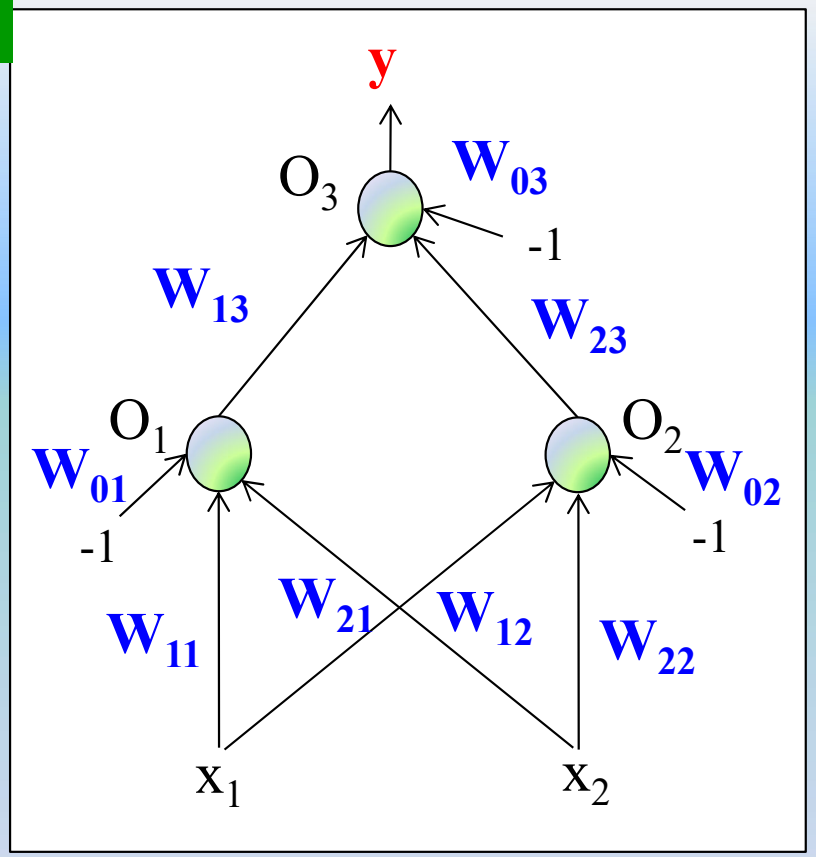
HIDDEN UNIT (O_2) DECISION BOUNDARY

$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

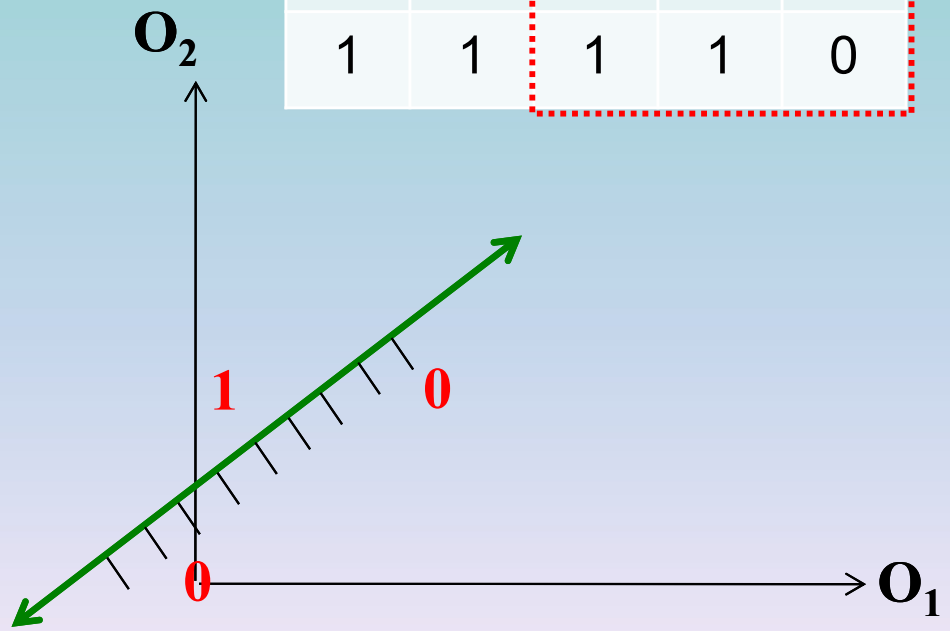
Multilayer Layer Perceptron

Example



XOR PROBLEM

X_1	X_2	O_1	O_2	Y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0



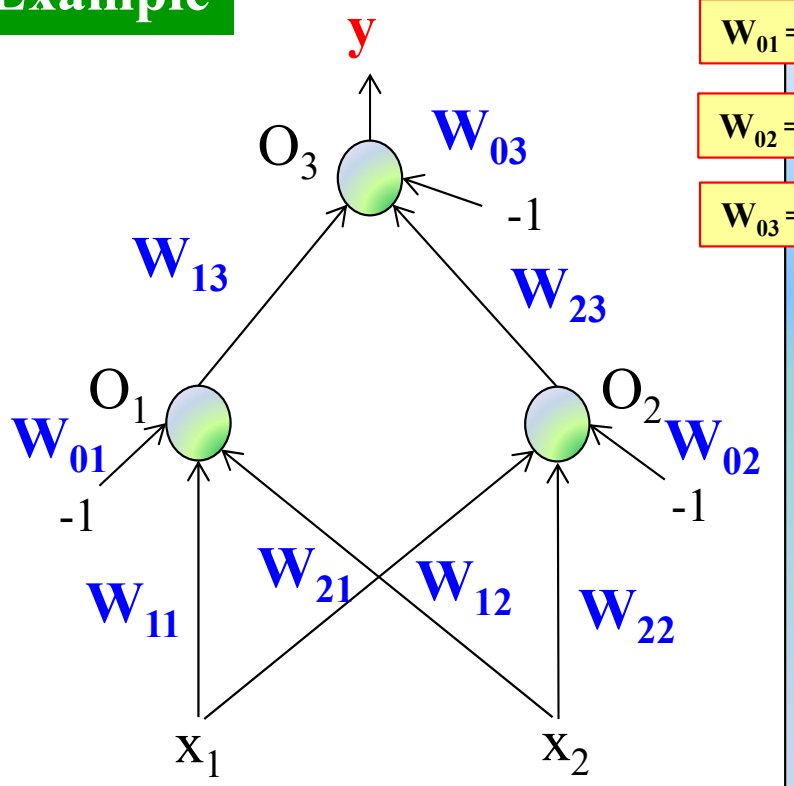
$W_{01} = 3/2$ $W_{11} = 1$ $W_{12} = 1$

$W_{02} = 1/2$ $W_{21} = 1$ $W_{22} = 1$

$W_{03} = 1/2$ $W_{13} = -1$ $W_{23} = 1$

Multilayer Layer Perceptron

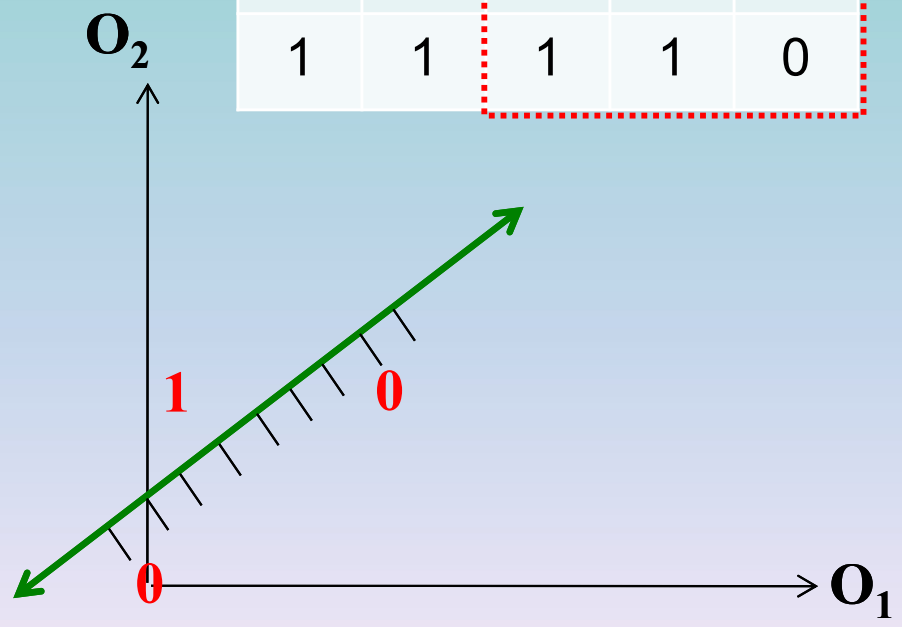
Example



$W_{01} = 3/2$	$W_{11} = 1$	$W_{12} = 1$
$W_{02} = 1/2$	$W_{21} = 1$	$W_{22} = 1$
$W_{03} = 1/2$	$W_{13} = -1$	$W_{23} = 1$

XOR PROBLEM

X_1	X_2	O_1	O_2	Y
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0



The problem has been mapped into a linearly separable problem in the space of the outputs of the hidden units. We can now find a linear separator that solves the XOR problem.

Multilayer Layer Perceptron

Any set of training points can be separated by a three-layer perceptron network (with sufficiently many units).

“Almost any” set of points are separable by a two-layer perceptron network (with lots of units).

But no efficient learning rule is known.

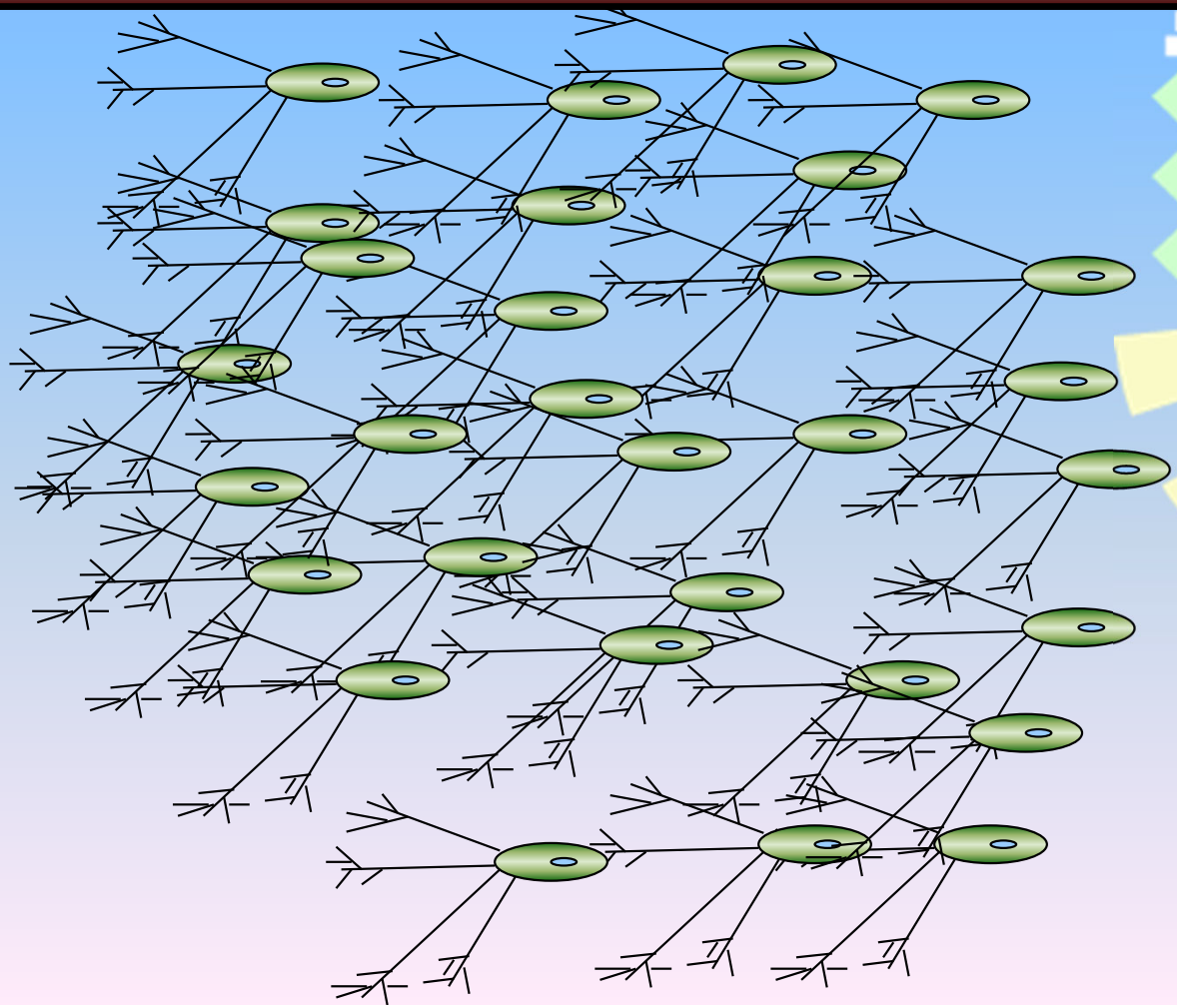
Could we use gradient ascent/descent?

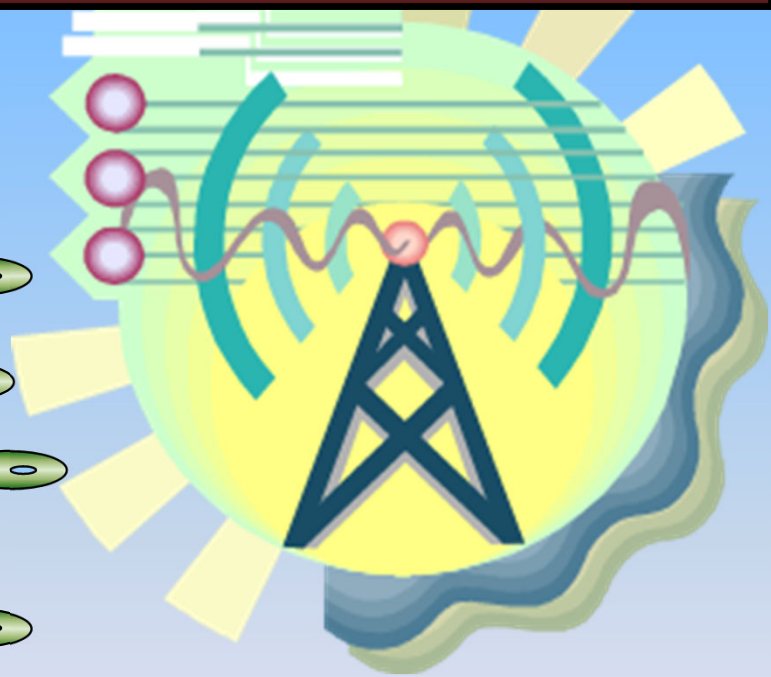
Threshold function is not smooth.
We need a smooth function in lieu of the threshold function.

We need a smooth output (as a function of changes in the network weights) in order for us to use gradient descent.

Neural Networks

Part 1





n.h.reyes@massey.ac.nz

Neural Networks

- Introduction to Multi-layer Feed-forward Network
- Evaluating a Feed-forward Network
- Training a Feed-forward Network
- Theory (Derivation of Formulas)
- Training Tips
- Encoding Inputs, Softmax units
- Performance Measure
- Applications, Tutorial Questions

Artificial Neural Networks

- Computational models made up of simple, highly interconnected neuron-like units which operate in some parallel and distributed mode to perform some common global task

- connectionist systems, PDP networks, neural computing devices, empirical learning systems, etc.

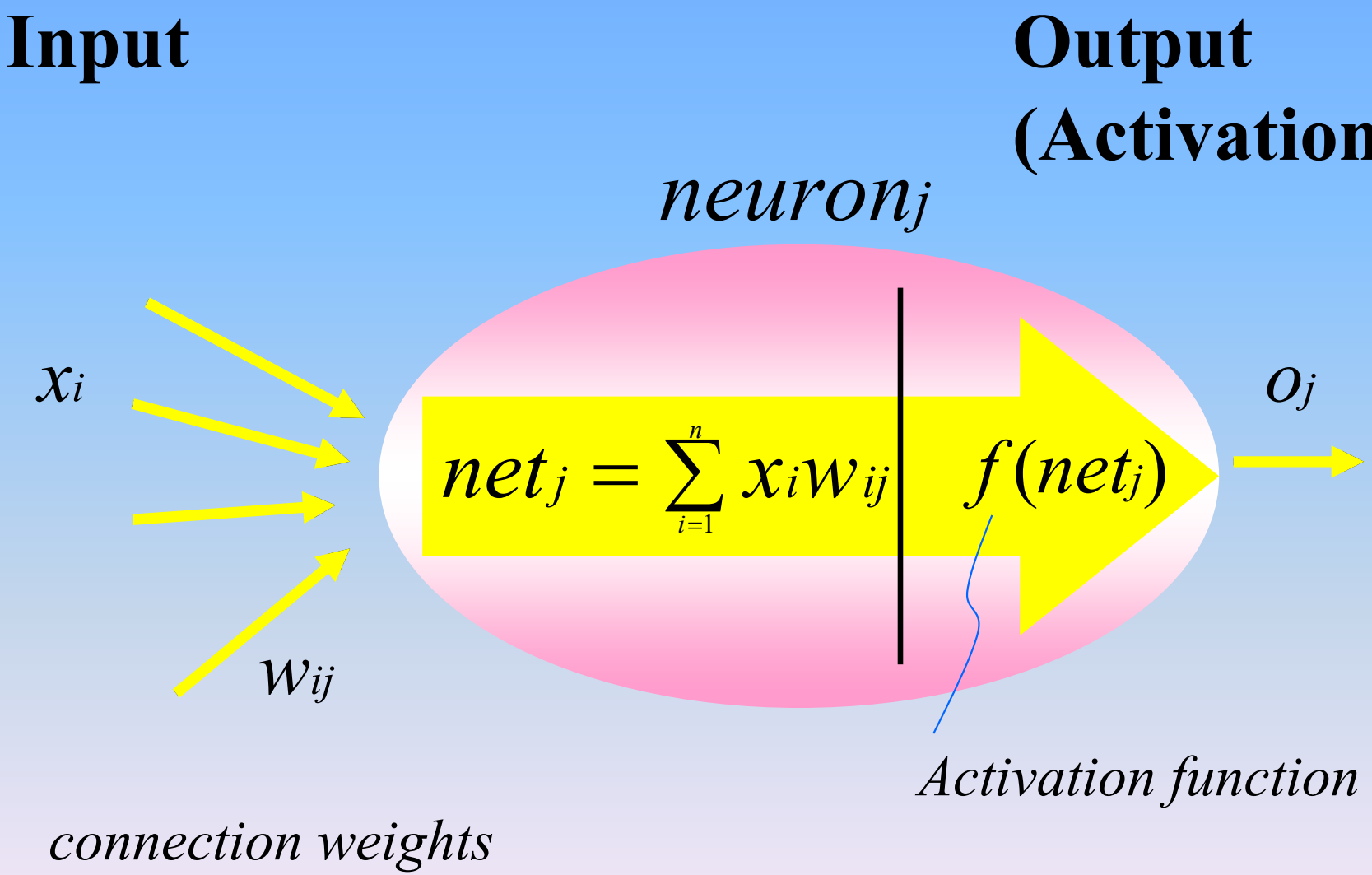
Artificial Neural Networks

- Neural nets are quantitative, numerical and don't require a knowledge engineer to extract expert information
- Neural networks are inductive programs; they take in a great amount of information all at once and then draw a conclusion.

Artificial Neural Networks

- Learning ability
- inherent parallelism
- distributed mode of operation
- simplicity of units' behavior
- absence of centralized control

the artificial neuron



Activation/ Squashing/Transfer Function

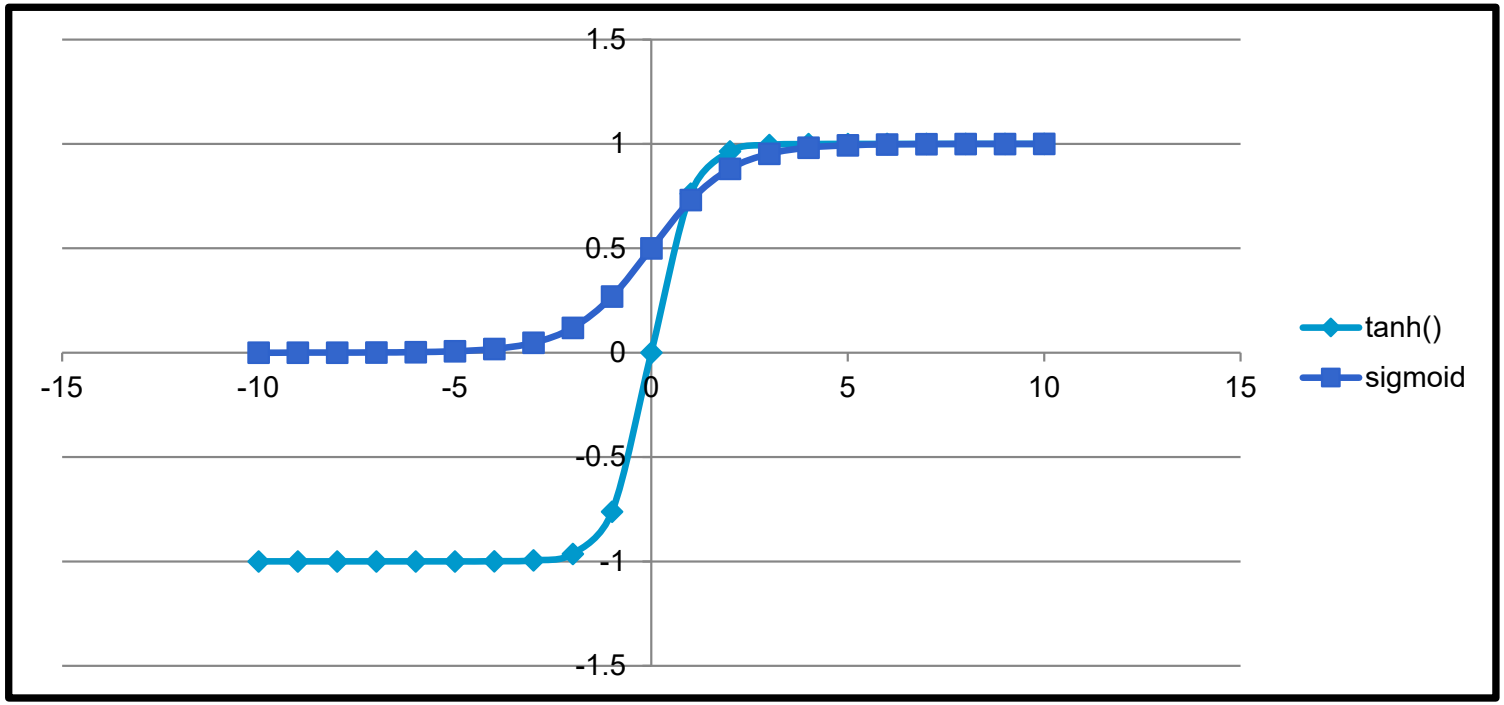
where

$$f(net_j) = \frac{1}{1+\exp^{(-netj)}}$$

Activation Functions

- Logistic function or Sigmoid function
- Hyperbolic tangent,
- ReLU, etc.

Activation Functions



• The **sigmoid function** varies between zero and one. This is inconvenient in some calculations as it cannot handle negative values.

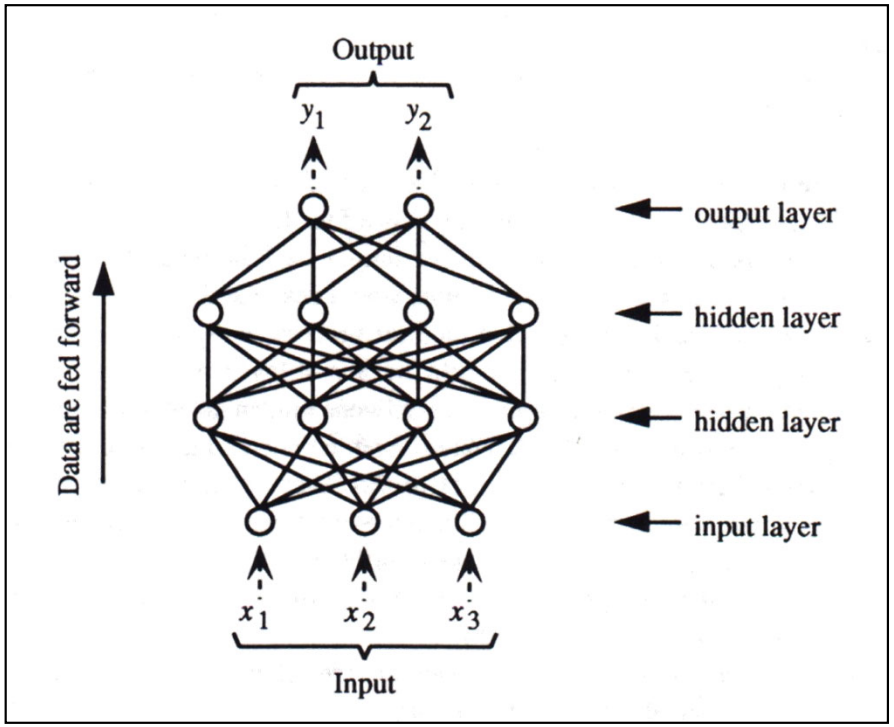
• It is sometimes more convenient to use the ***tanh()* function**, which has exactly the same shape but which swings between +/- 1.

Activation Functions and their derivatives

Activation Function	$g(a)$	$g'(a)$
Linear	$g(a) = a$	$g'(a) = 1$
Logistic	$g(a) = \frac{1}{1+e^{-a}}$	$g'(a) = g(a)(1-g(a))$
Hyperbolic-tangent	$g(a) = tanh(a)$	$g'(a) = sech^2(a) = 1-tanh^2(a)$
Squash	$g(a) = \frac{a}{1+\lvert a\rvert}$	$g'(a) = \frac{1}{(1+\lvert a\rvert)^2} = (1-\lvert g(a)\rvert)^2$

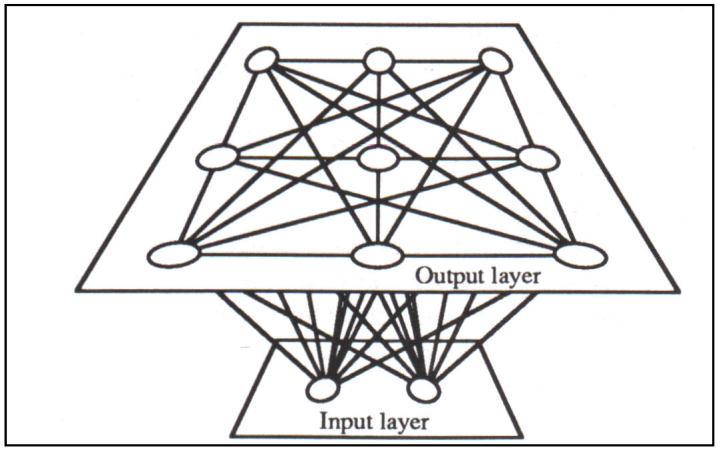
Bipolar sigmoidal $g(a) = \frac{2}{1+\exp(-a)} - 1$ $g'(a) = \frac{2\exp(-a)}{(1+\exp(-a))^2} = \frac{1}{2}(1-g(a))$

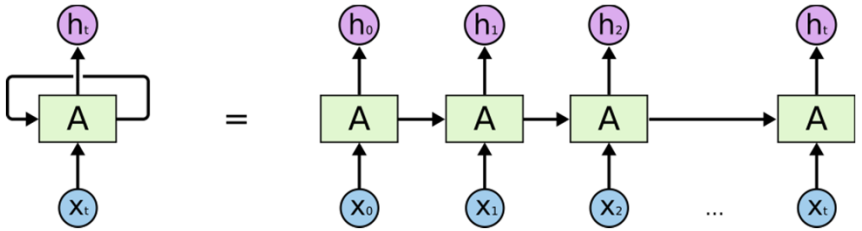
Various NN Architectures



Feed-forward Net

SOM





RNN (e.g. Long Short Term Memory networks)

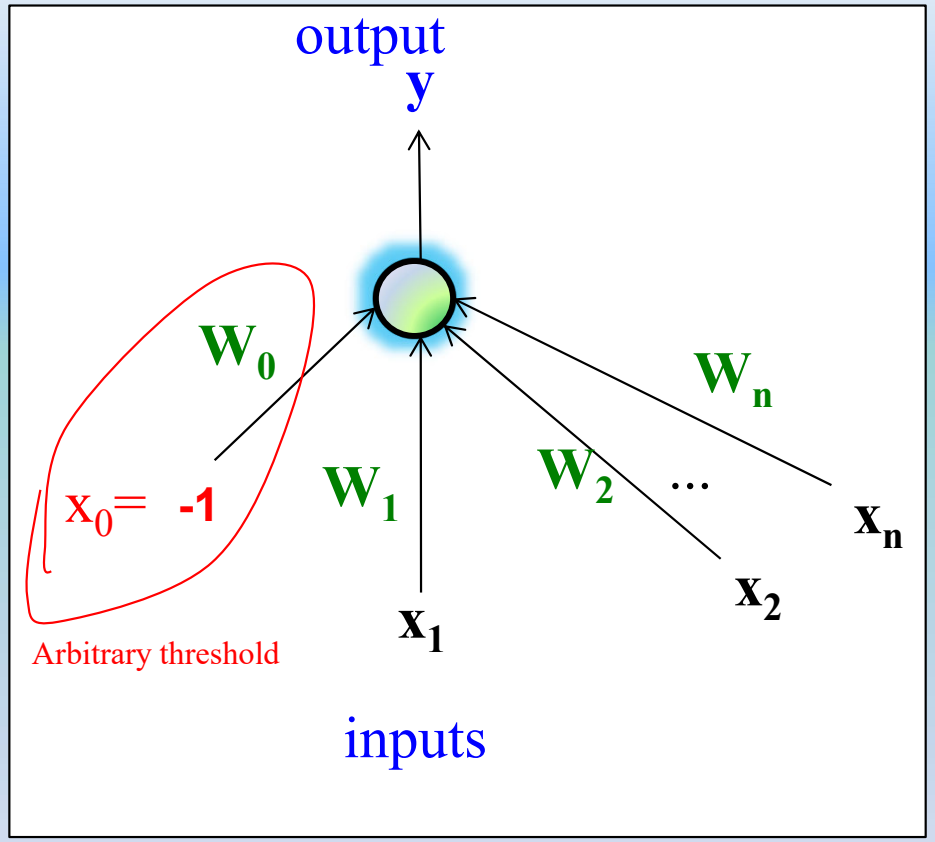
Neural Network Models

- **Perceptron**
- Hopfield Networks
- Bi-Directional Associative Memory
- Self-Organizing Maps
- Neocognitron
- Adaptive Resonance Theory
- Boltzmann Machine
- Radial Basis Function Networks
- Cascade-Correlation Networks
- Reduced-Coulomb Energy Networks
- **Multi-layer Feed-forward Network**

What have we learned previously?

- Single Perceptron Unit
- Multi-layer Perceptrons
- Introduction to Neural Networks

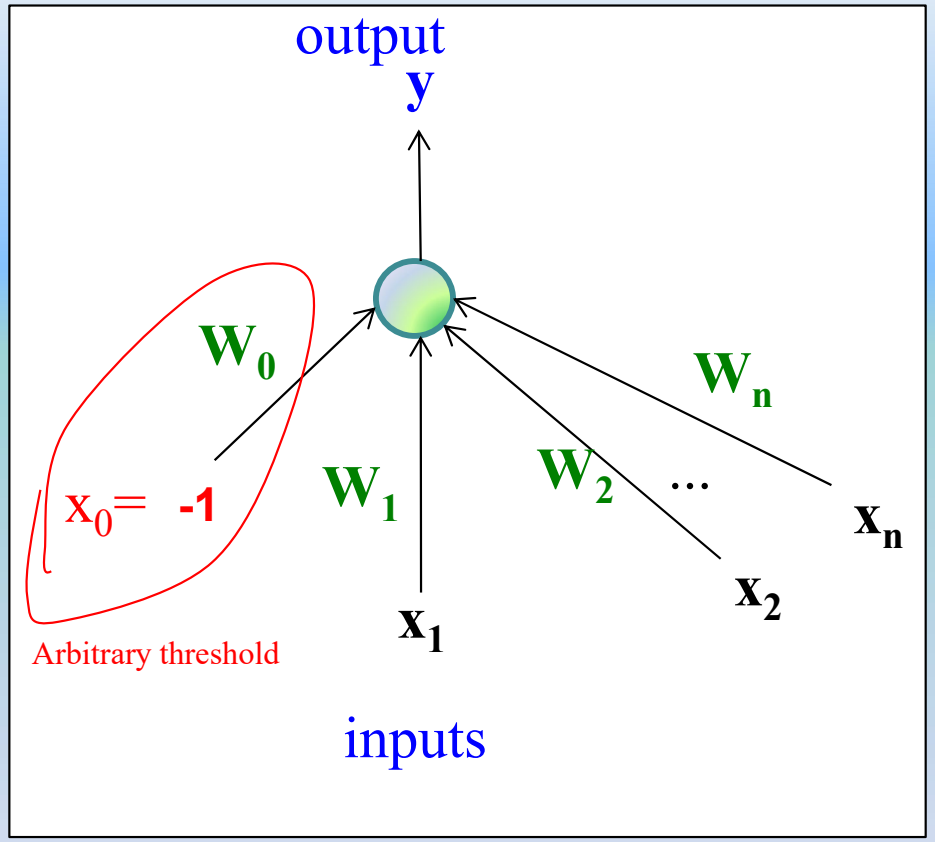
Linear Classifier Single Perceptron Unit



$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Weighted input

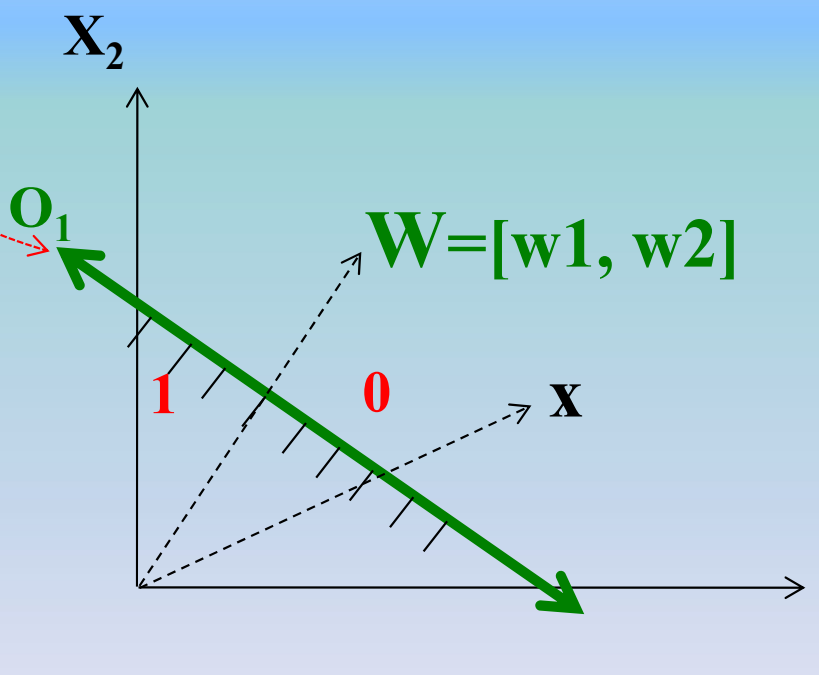
Linear Classifier Single Perceptron Unit



$$\theta(z) = \begin{cases} 1 & z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

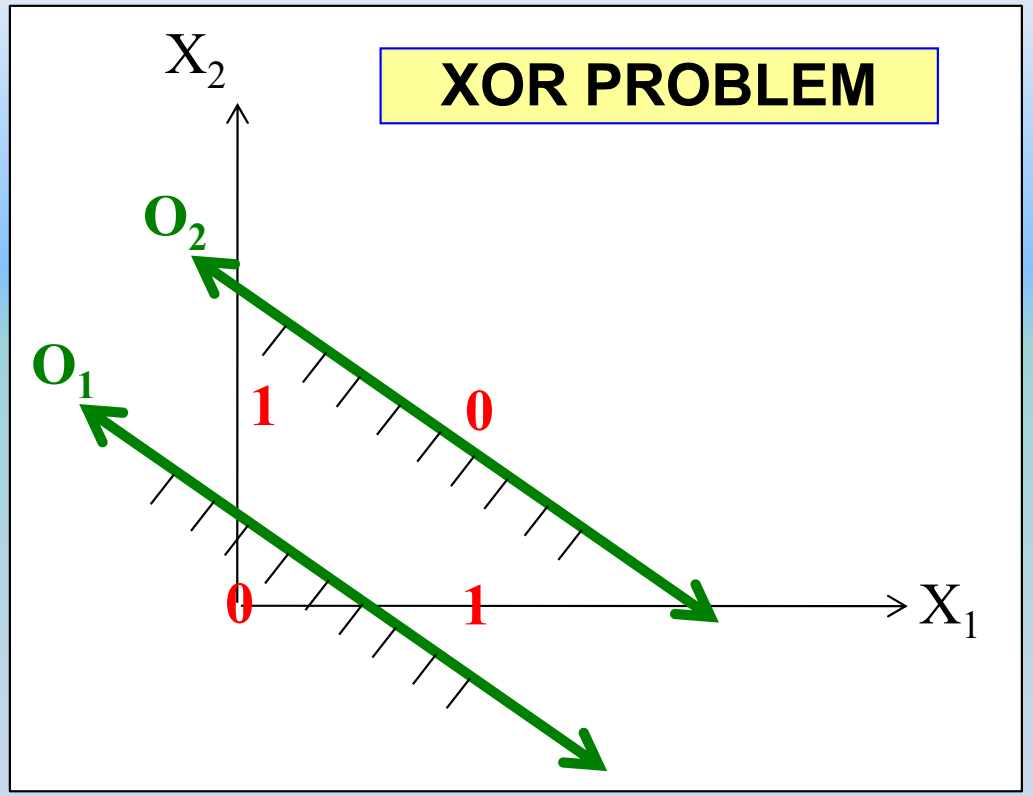
Weighted input

The decision boundary for a single perceptron is a **hyperplane** in the feature space.



Linear Classifier Single Perceptron Unit

Beyond Linear Separability

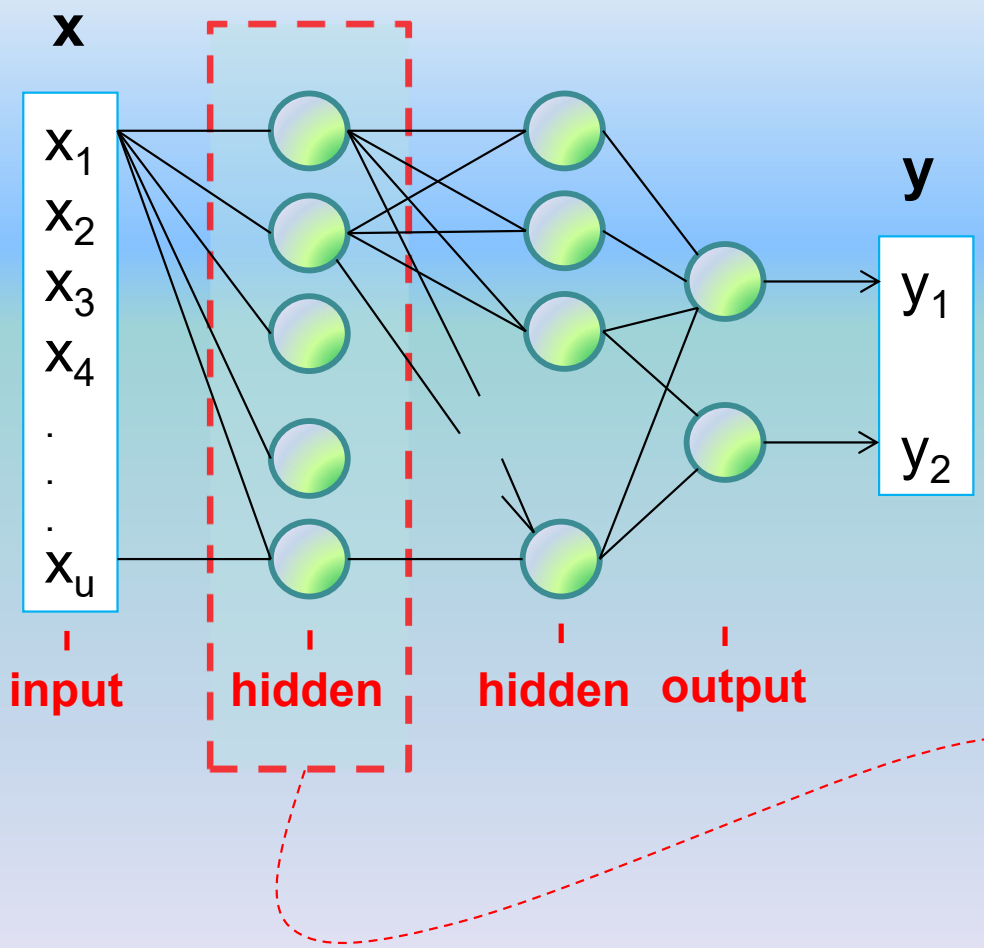


Since a single perceptron unit can only define a single linear boundary, it is limited to solving linearly separable problems.

What about if we consider more than one linear separator and combine their outputs; can we get a more powerful classifier?

Yes, since a single perceptron unit is so limited, a network of these units will be less limited.

Multi-Layer Perceptron



The introduction of “hidden” (not connected directly to the output) units into these networks make them much more powerful: they are no longer limited to linearly separable problems.

The lower layers transform the input problem into more tractable (linearly separable) problems for subsequent layers.

Multilayer Layer Perceptron

Any set of training points can be separated by a three-layer perceptron network (with sufficiently many units).

"Almost any" set of points are separable by a two-layer perceptron network (with lots of units).

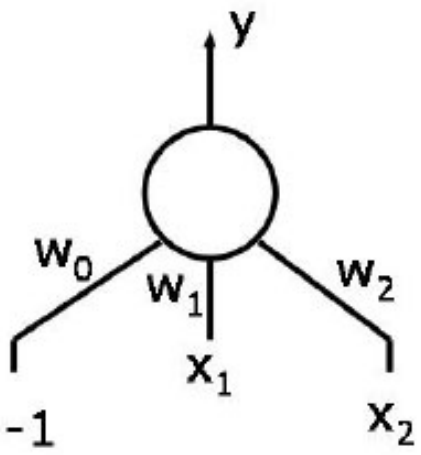
For example, (2^n)/n hidden units are needed to encode all boolean functions of n inputs.

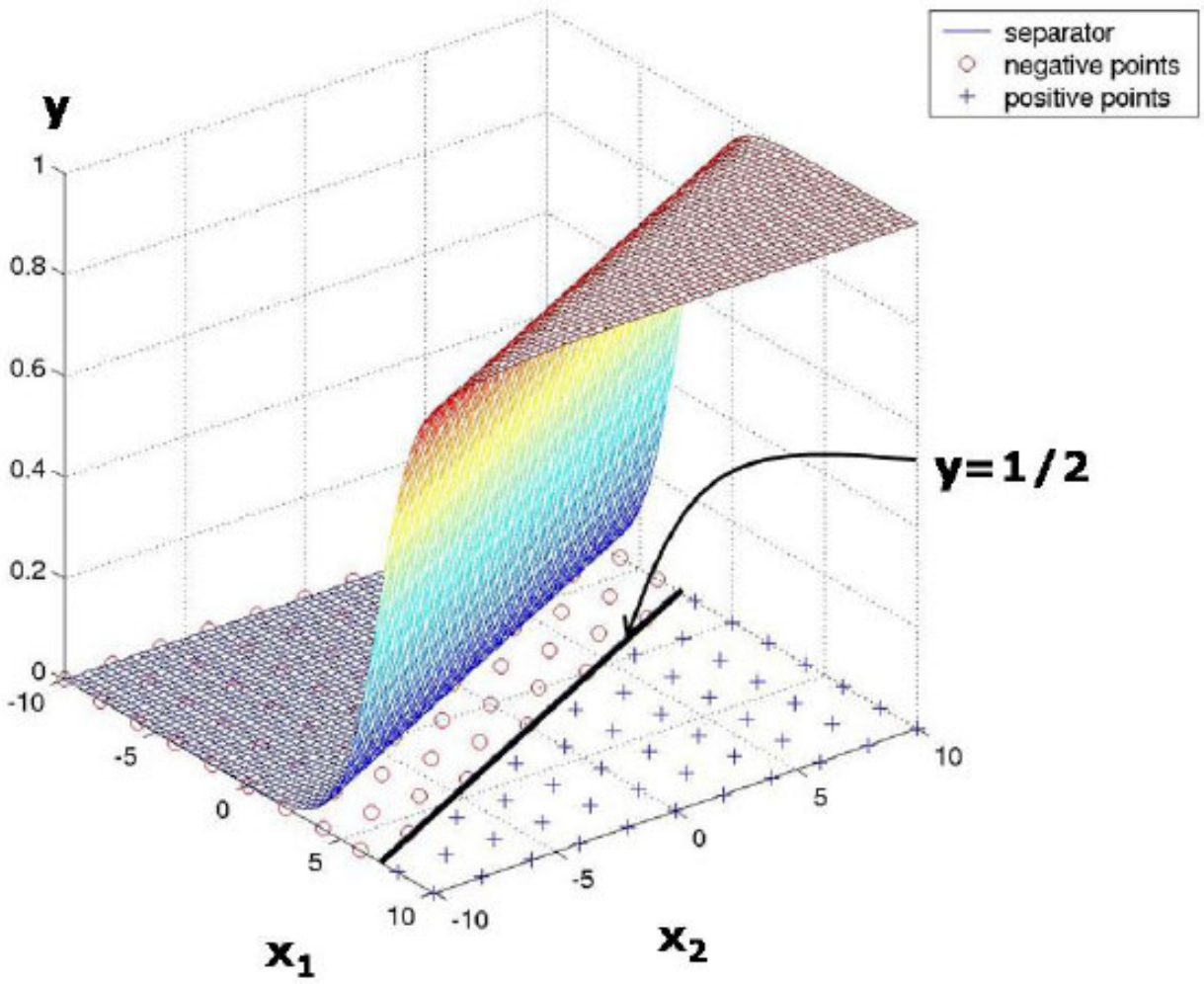
But no efficient learning rule is known.

Threshold function is not smooth.
We need a smooth function in lieu of the threshold function.

We need a smooth output (as a function of changes in the network weights) in order for us to use gradient descent.

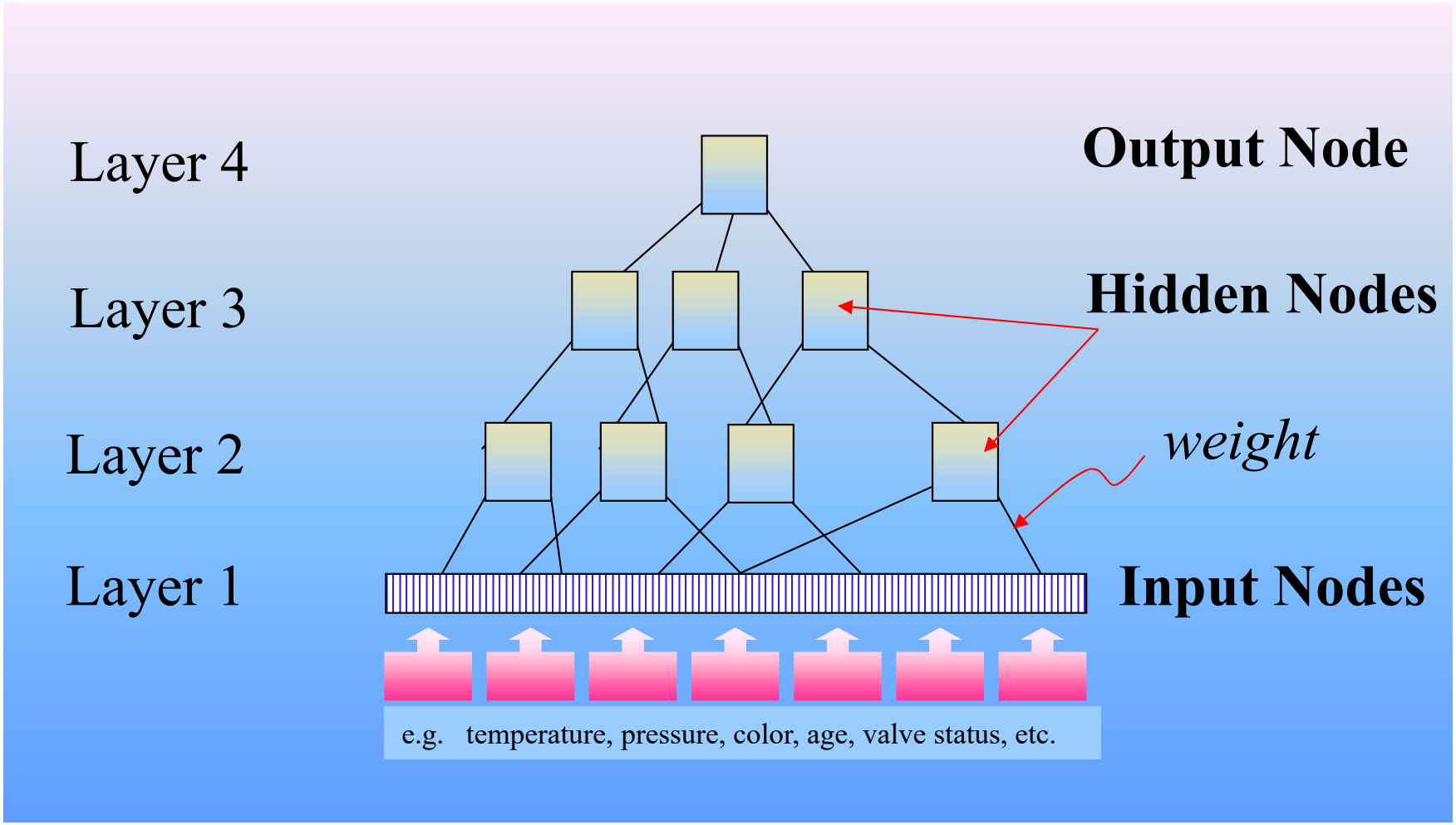
Sigmoid Unit





Neural Network Architecture

Multi-layer Feed-forward Network

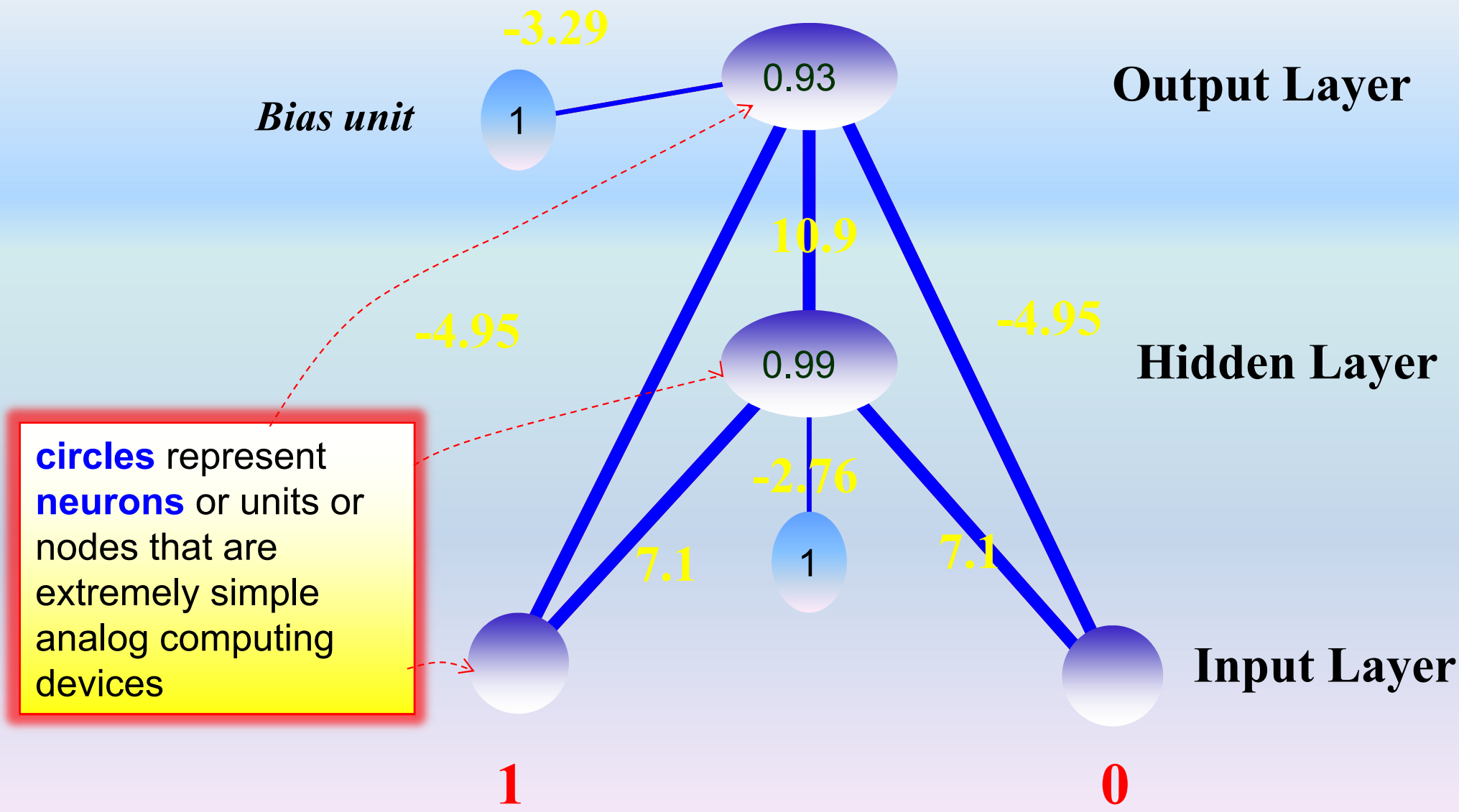


Example

Evaluating a Feed-forward Network

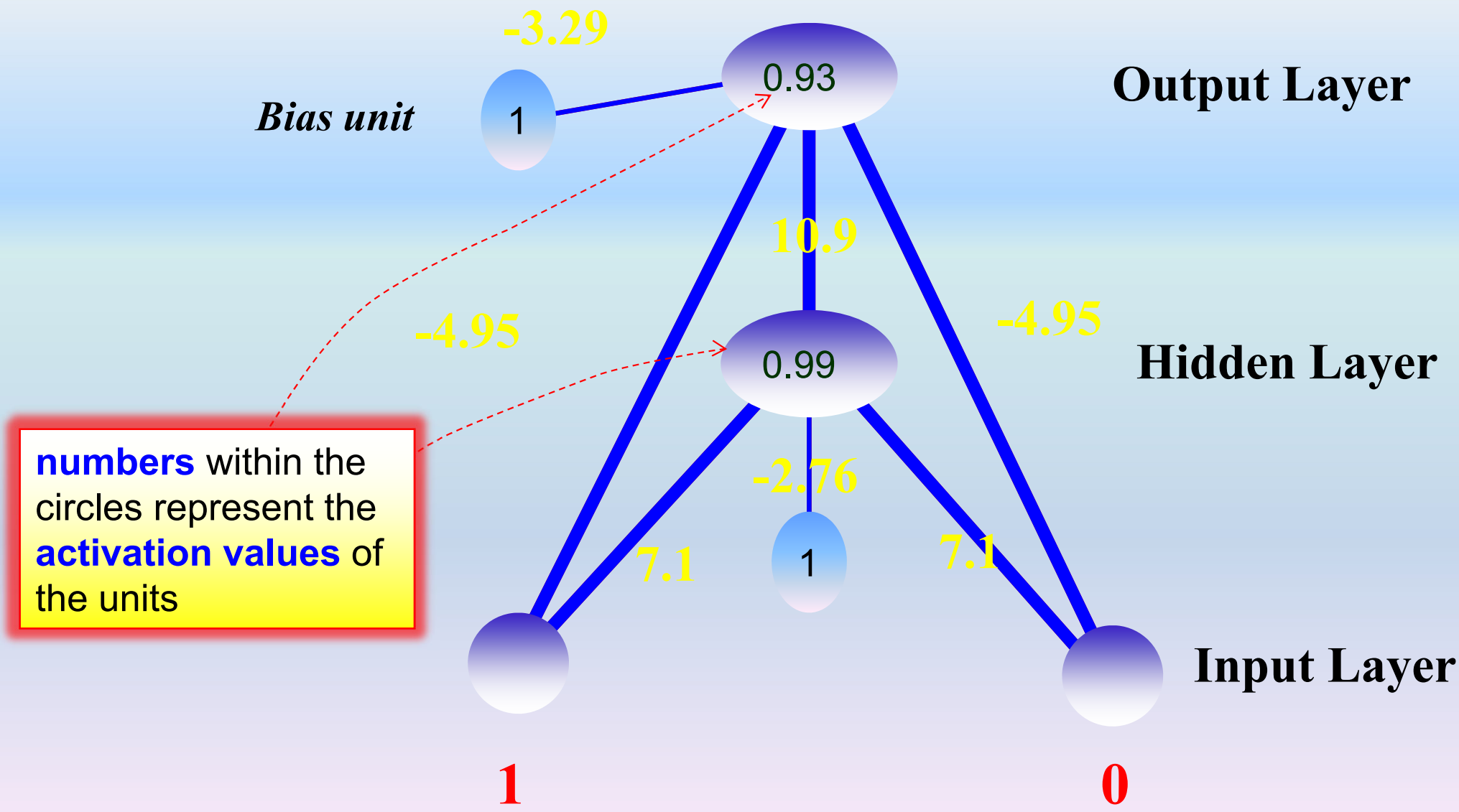
Multi-layer Feed-forward Network

Three-layer Network (2-1-1) for solving the XOR Problem

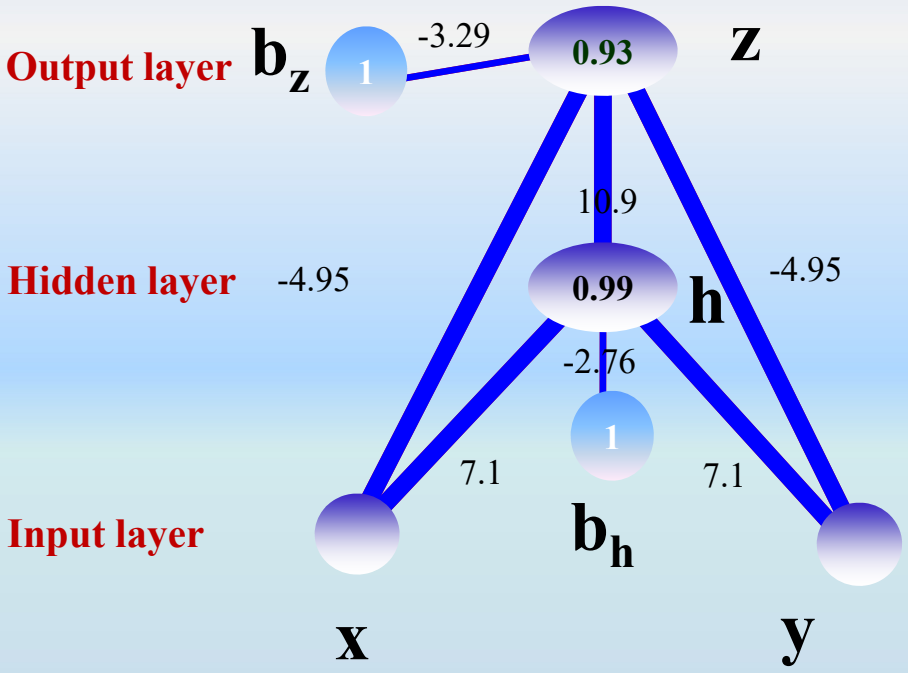


Multi-layer Feed-forward Network

Three-layer Network (2-1-1) for solving the XOR Problem

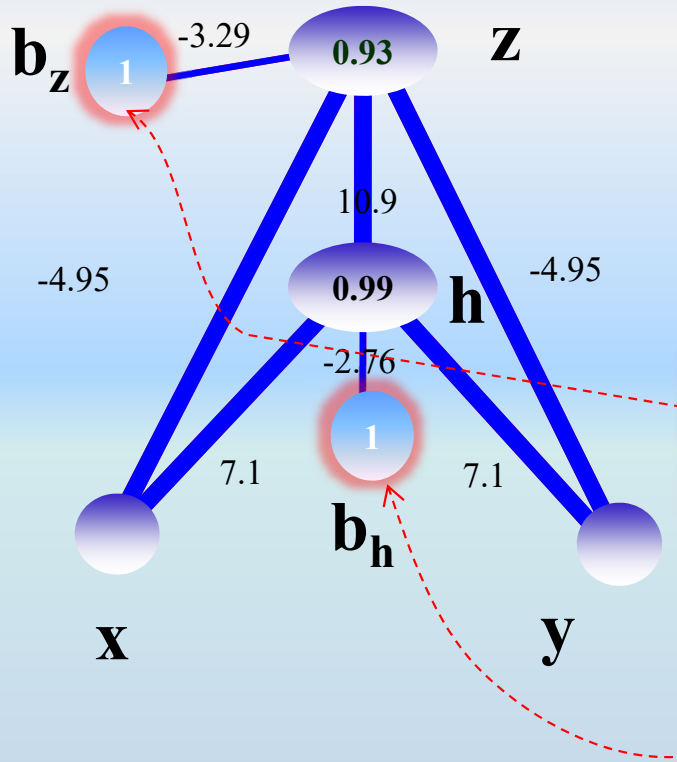


Multi-layer Feed-forward Network



There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

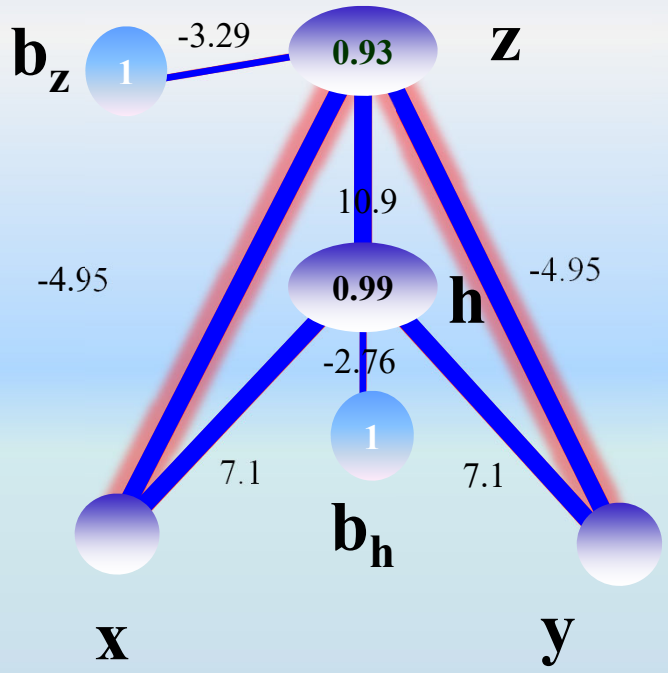
Multi-layer Feed-forward Network



There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

There are two other units present called **bias units** whose values are always **1.0**

Multi-layer Feed-forward Network

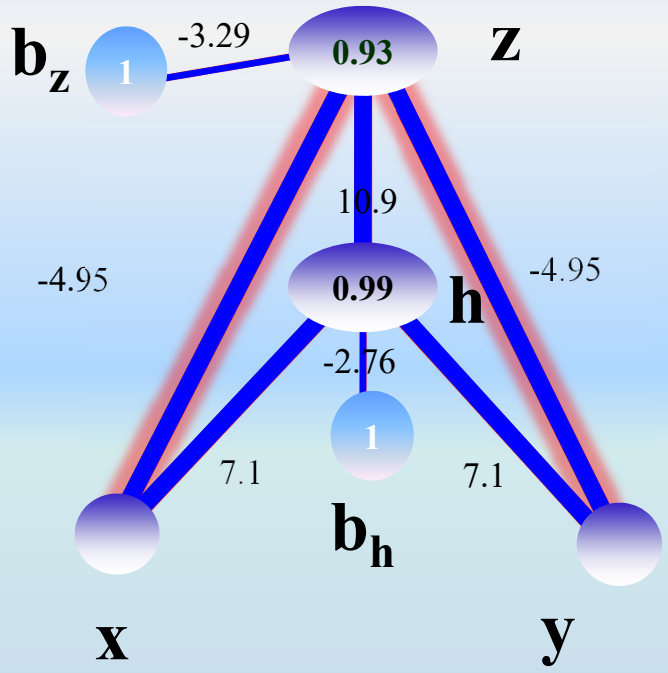


There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

There are two other units present called bias units whose values are always 1.0

Much of the time feed-forward networks only have connections within adjacent layers; however, this one has two extra connections that go directly from the input units to the output unit.

Multi-layer Feed-forward Network



There are three layers, the input layer that contains the values for x and y, a hidden layer that contains one node h, and an output unit that gives the value of the output value, z

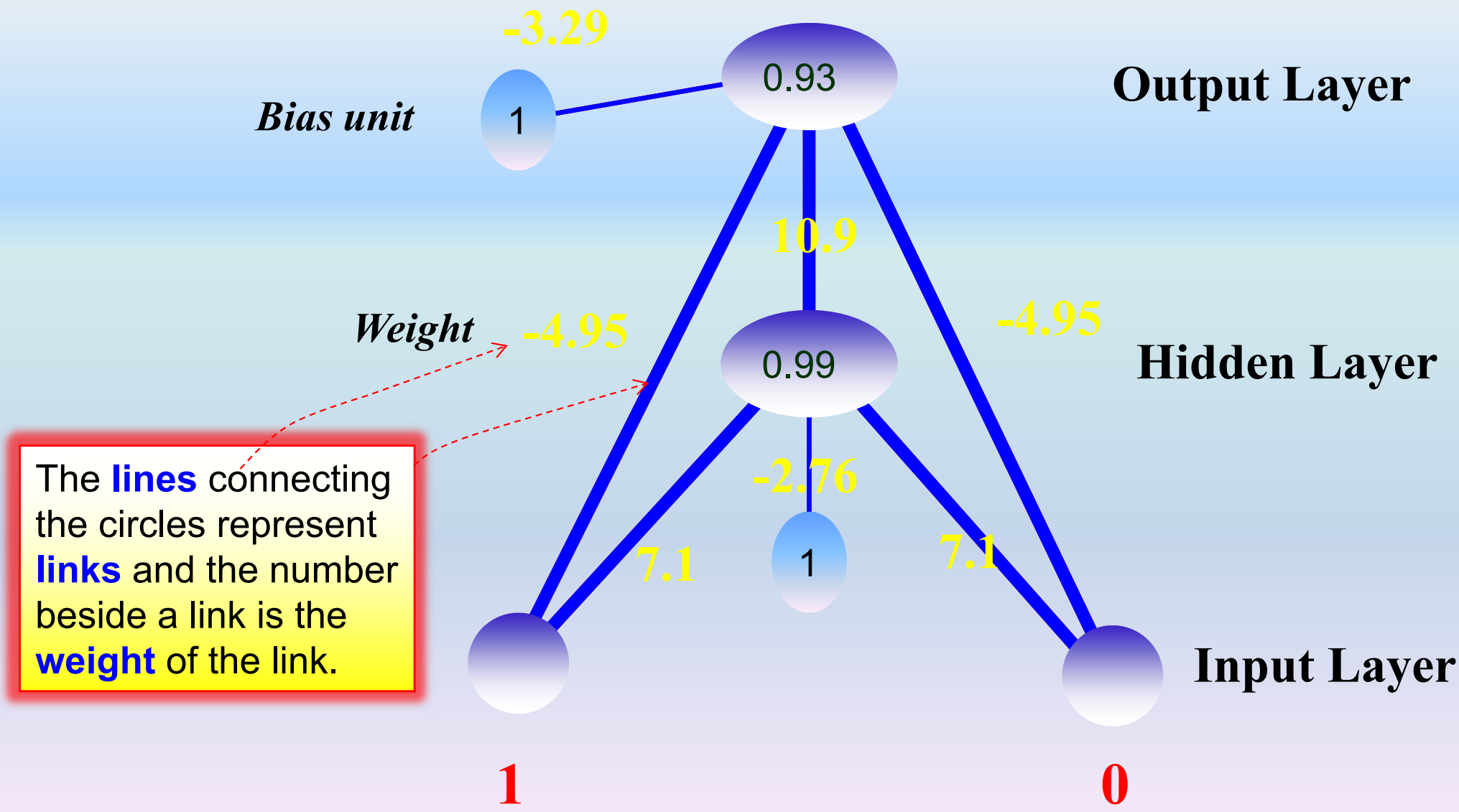
There are two other units present called bias units whose values are always 1.0

Much of the time feed-forward networks only have connections within adjacent layers; however, this one has two extra connections that go directly from the input units to the output unit.

In some problems, like XOR these extra input-output connections make training the network much faster.

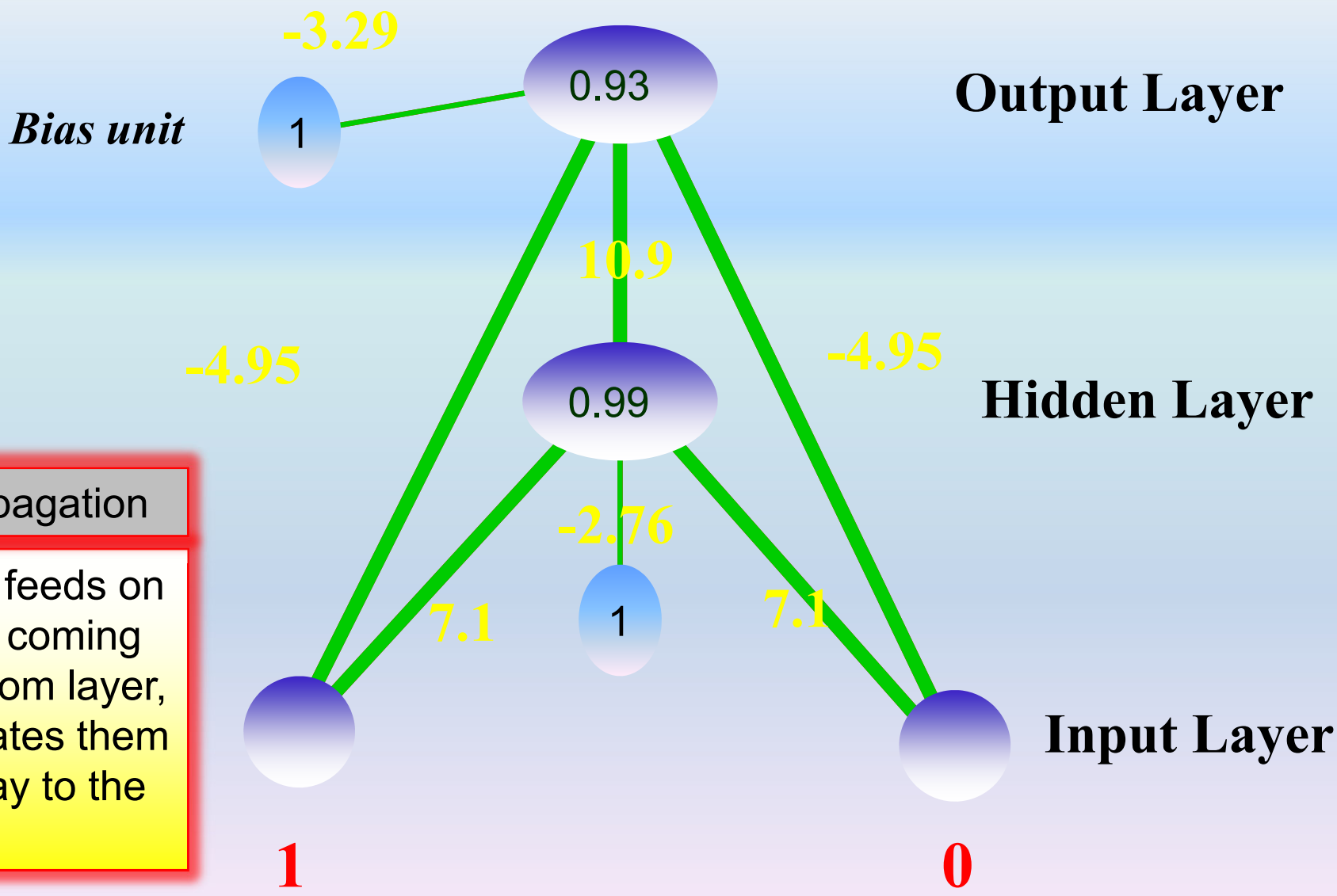
Multi-layer Feed-forward Network

Three-layered Network (2-1-1) for solving the XOR Problem



Multi-layer Feed-forward Network

Three-layered Network (2-1-1) for solving the XOR Problem

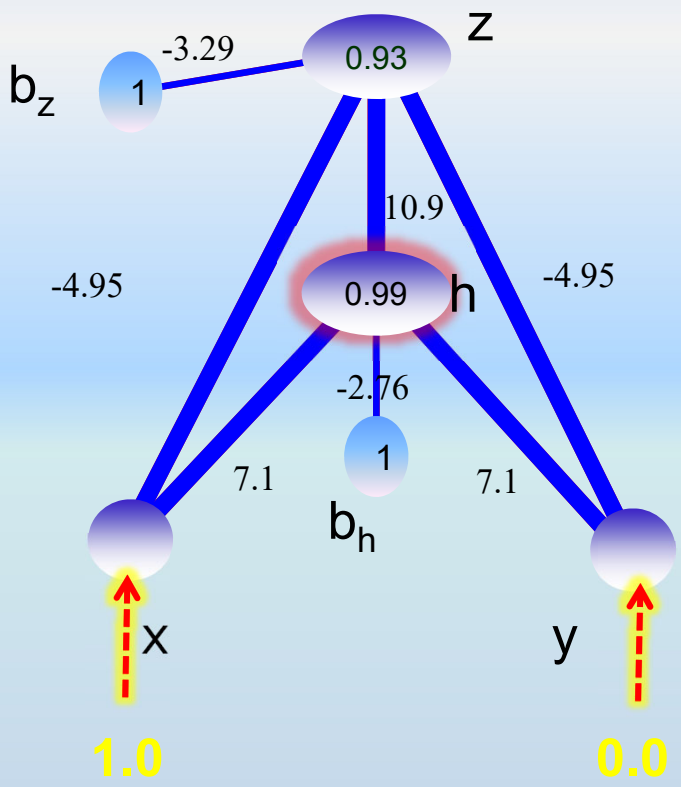


Forward propagation

The network feeds on input signals coming from the bottom layer, then propagates them up, all the way to the top.

Evaluating a network

Evaluating a Network



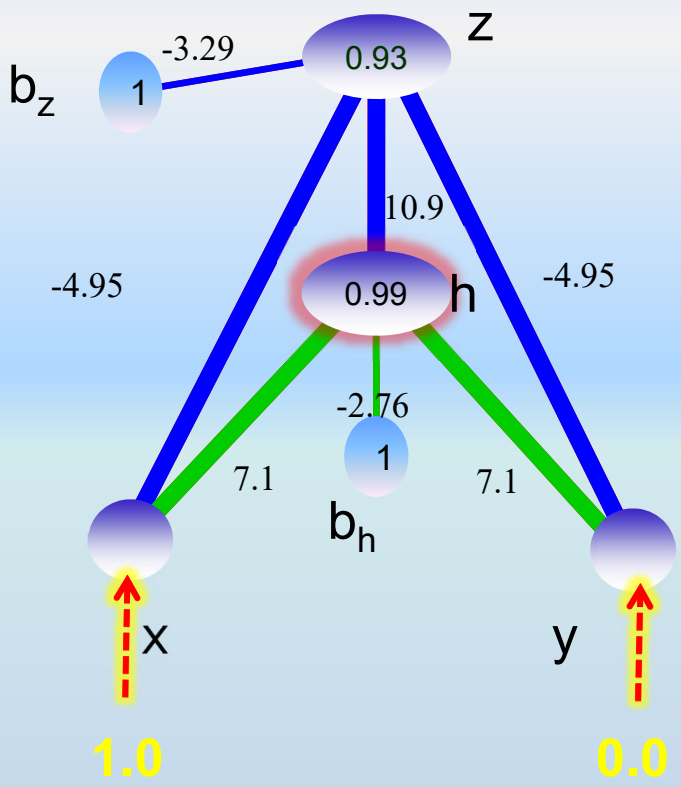
Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

To compute the value of the output unit, z, we place values for x and y on the input layer units,
say **x** = 1.0 and **y** = 0.0, then propagate the signals up to the next succeeding layer.

Evaluating a Network (XOR Problem)



Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

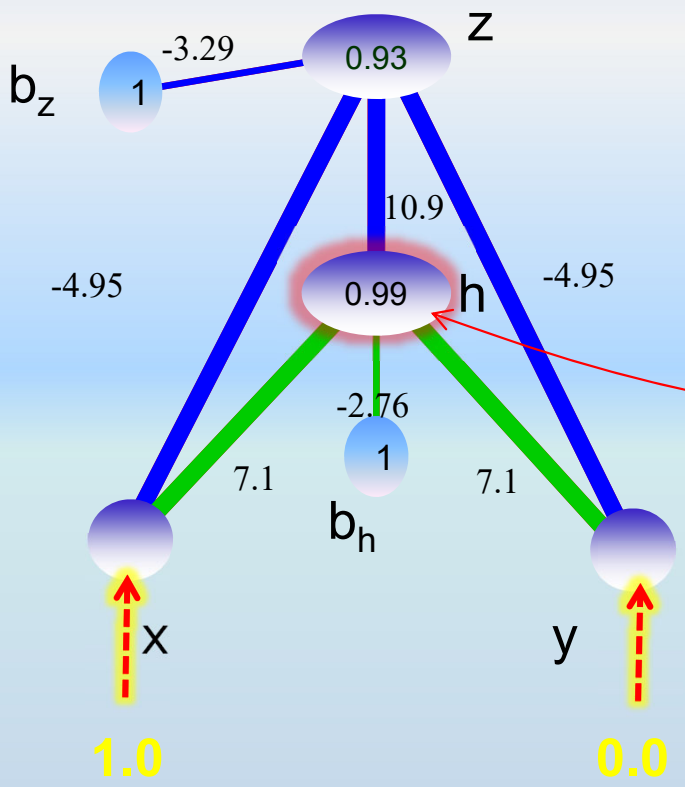
For the hidden node ***h***, find all lower level units connected to it.

For each of the connections, multiply the weight attached to the link by the value of the unit, then sum them all up. This sum is the net input of node ***h***.

Hidden Node h:

1.00	*	7.1	=7.1
1.00	*	-2.76	=-2.76
0.00	*	7.1	=0
		sum	4.34

Evaluating a Network (XOR Problem)



Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

The output or activation value of node h is computed using a non-linear function:

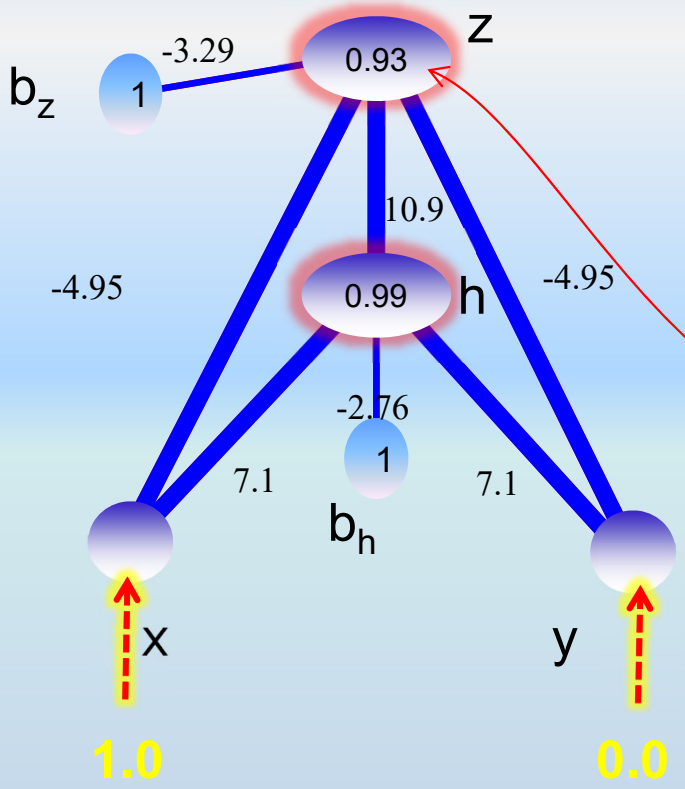
$$activationValue = \frac{1}{1+e^{-sum}}$$

Hidden Node h:

1.00	*	7.1	7.1
1.00	*	-2.76	-2.76
0.00	*	7.1	0
		sum	4.34

$$Node_h_{activation} = \frac{1}{1+e^{-4.34}} = 0.987131$$

Evaluating a Network (XOR Problem)



Example

Given a trained NN, and the following input values, calculate the output of the NN.

x = 1.0 and **y** = 0.0,

Continuing with the process, after propagating the signals toward output node **z**, we have:

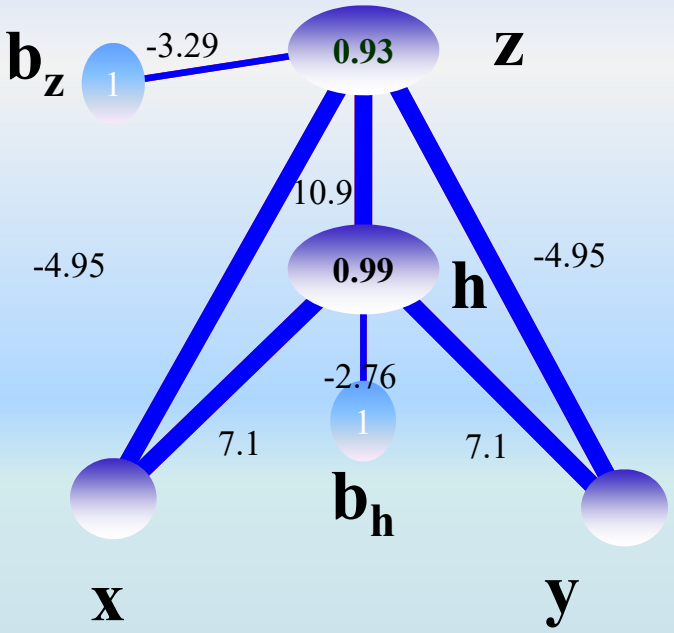
Hidden Node h:

1.00	*	7.1	7.1
1.00	*	-2.76	-2.76
0.00	*	7.1	0
		sum	4.34
		activation	0.987131

Output Node z

1.00	*	-4.95	-4.95
0.00	*	-4.95	0
0.99	*	10.9	10.791
1.00	*	-3.29	-3.29
		sum	2.551
		activation	0.93

Evaluating a Network (XOR Problem)



1.00	*	7.1	7.1
1.00	*	-2.76	-2.76
0.00	*	7.1	0
		sum	4.34
		v	0.987131

Output Node z

1.00	*	-4.95	-4.95
0.00	*	-4.95	0
0.99	*	10.9	10.791
1.00	*	-3.29	-3.29
		sum	2.551
		v	0.93

In some neural networks we might just leave the activation value of the unit to be **4.34**. In this case we would say that we are using the **linear activation function**, however backprop is at its best when this value is passed to certain types of **nonlinear functions**.

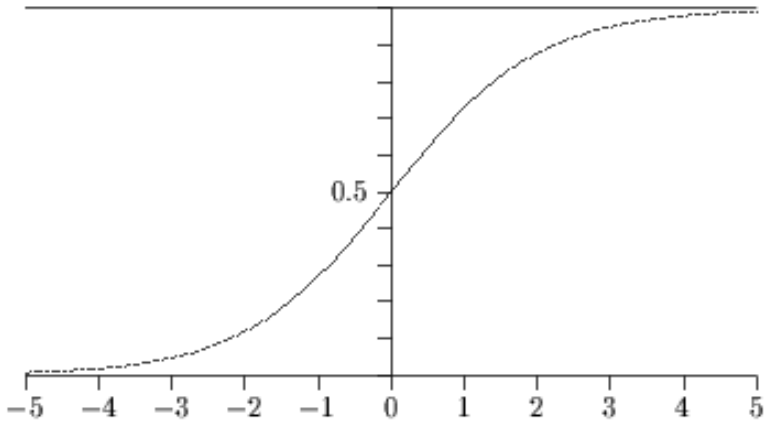
The most commonly used nonlinear function is:

Standard Sigmoid: $v = \frac{1}{1 + e^{-s}}$

where **s** is the sum of the inputs to the neuron and **v** is the value of the neuron. Thus, with **s** = 4.34, **v** = 0.99

Of course, 0.93 is not quite 1 but for this example it is close enough. When using this particular activation function for a problem where the output is supposed to be a 0 or 1, getting the output to within 0.1 of the target value is a very common standard.

Evaluating a Network



With this particular activation function it is actually somewhat hard to get very close to **1** or **0** because the function only approaches **1** and **0** as the input to the function approaches

∞ and $-\infty$

Standard Sigmoid:

$$v = \frac{1}{1 + e^{-s}}$$

Evaluating a Network

Computing the activation value of a Neuron

The formulas for computing the activation value for a neuron, **j** can be written more concisely as follows:

Let the **weight** between neuron **i** and neuron **j** be $\mathbf{w_{ij}}$. Let the net input to neuron **j** be $\mathbf{net_j}$

Let the activation value for neuron **j** be $\mathbf{O_j}$.

Let the **activation function** be the general function, **f**, then

$$net_j = \sum_{i=1,n} w_{ij} o_i$$

where **n** is the number of units feeding into unit **j** and

$$o_j = f(net_j)$$

Neural Networks

- [x] Introduction to Multi-layer Feed-forward Network
- [x] Evaluating a Feed-forward Network
- [ ] Training a Feed-forward Network
- [ ] Theory (Derivation of Formulas)
- [ ] Training Tips
- [ ] Encoding Inputs, Softmax units
- [ ] Performance Measure
- [ ] Applications, Tutorial Questions

Training a Network

Back propagation Algorithm

Backpropagation Training

On-line / Continuous Update : Method 1

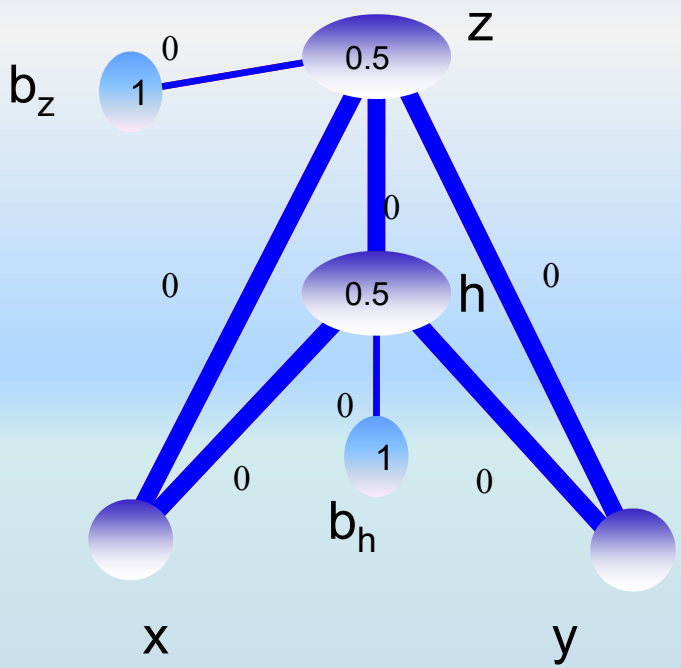
Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
5. Use another formula to find out errors for the hidden layer unit.
6. Adjust the weights leading into the hidden layer unit via another formula.
7. Repeat steps 1 thru 6 for the second, third patterns,…

Example: Training a network

Considering the same neural network example used for solving the XOR problem, let's examine how a neural network learns.

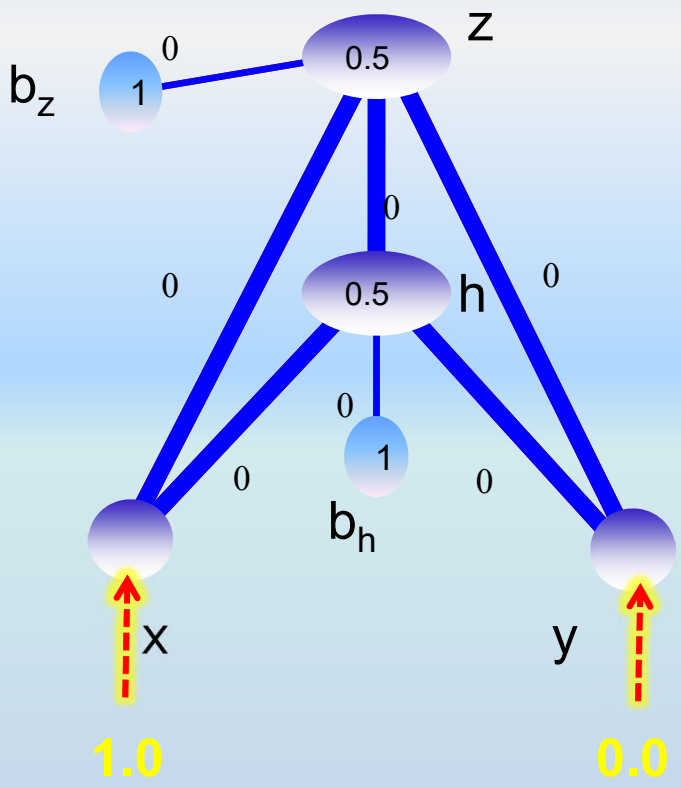
Training a Network



Firstly, give the network some initial weights.

For brevity of discussions, this example is using zero initial weights, but in practice, neural networks are initialised with randomly generated near-zero weights.

Training a Network



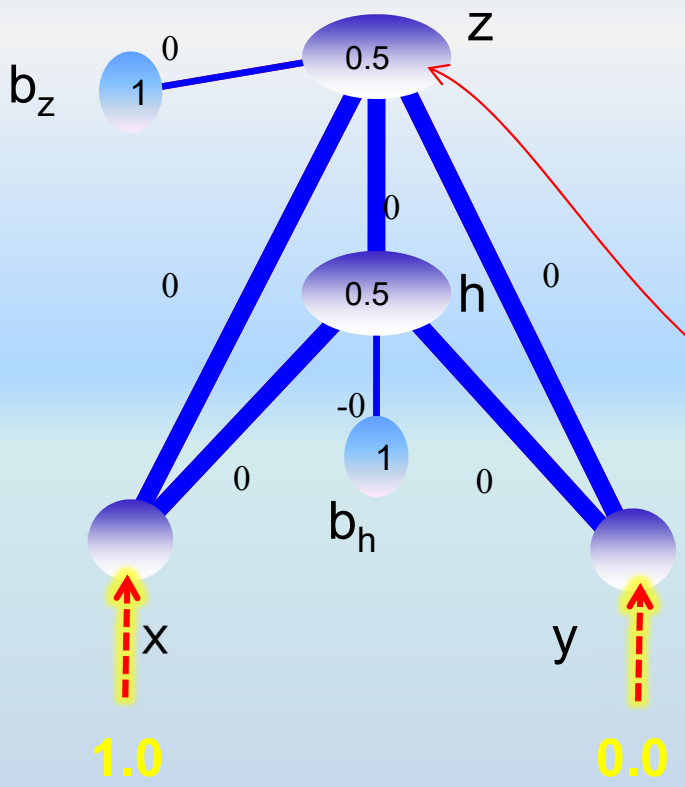
Randomly pick one training pattern from the training set, then feed it to the network.

e.g.
x = 1.0 and **y** = 0.0 , **desired output** = 1.0

TRAINING SET

x	y	desired
0	0	0
0	1	1
1	0	1
1	1	0

Training a Network



Randomly pick one training pattern from the training set, then feed it to the network.

e.g.
x = 1.0 and **y** = 0.0 , **desired output** = 1.0

Perform **forward propagation** until the activation value of the output neuron is found.

$$activationValue = \frac{1}{1+e^{-sum}} = \frac{1}{1+e^{0}} = \frac{1}{2}$$

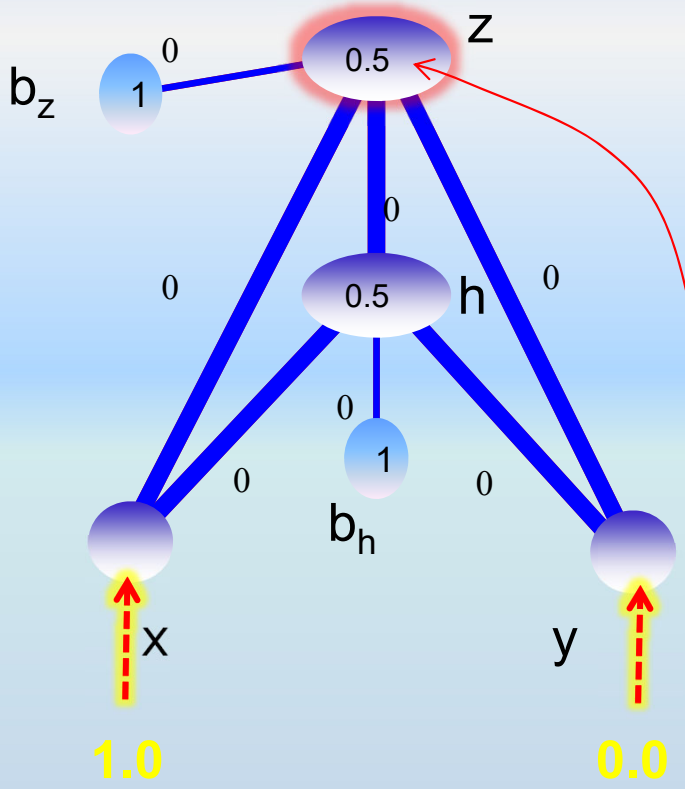
Hidden Node h:

1.00	*	0	0
1.00	*	0	0
0.00	*	0	0
		sum	0
		activation	0.5

Output Node z

1.00	*	0	0
0.00	*	0	0
0.99	*	0	0
1.00	*	0	0
		sum	0
		activation	0.5

Training a Network



Training pattern:

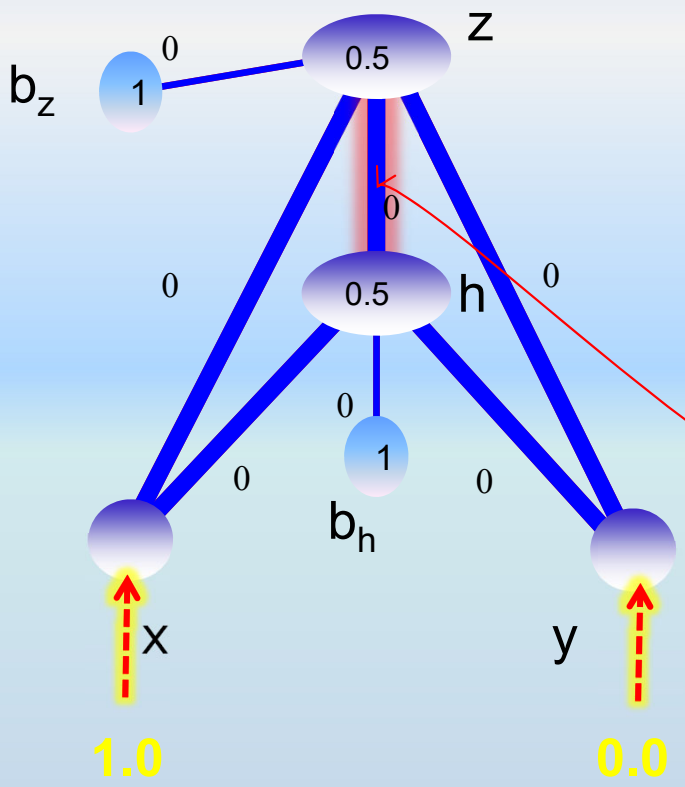
x = 1.0 and **y** = 0.0 , **desired output** = 1.0

Calculate the error on the output unit.

$$\partial_z = (t_z - o_z) * o_z * (1 - o_z)$$

$$\partial_z = (1 - 0.5) * 0.5 * (1 - 0.5) = 0.125$$

Training a Network



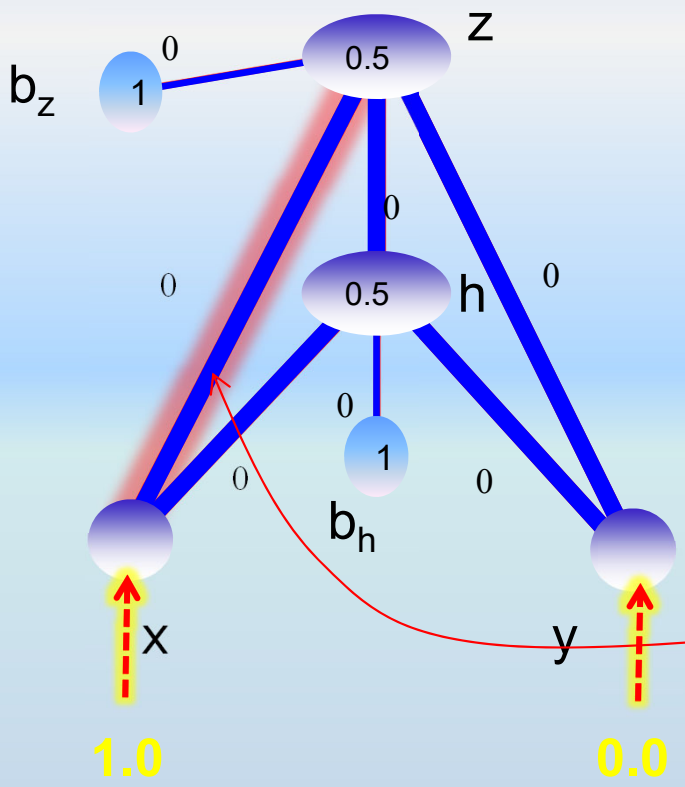
$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

Learning rate

Training a Network



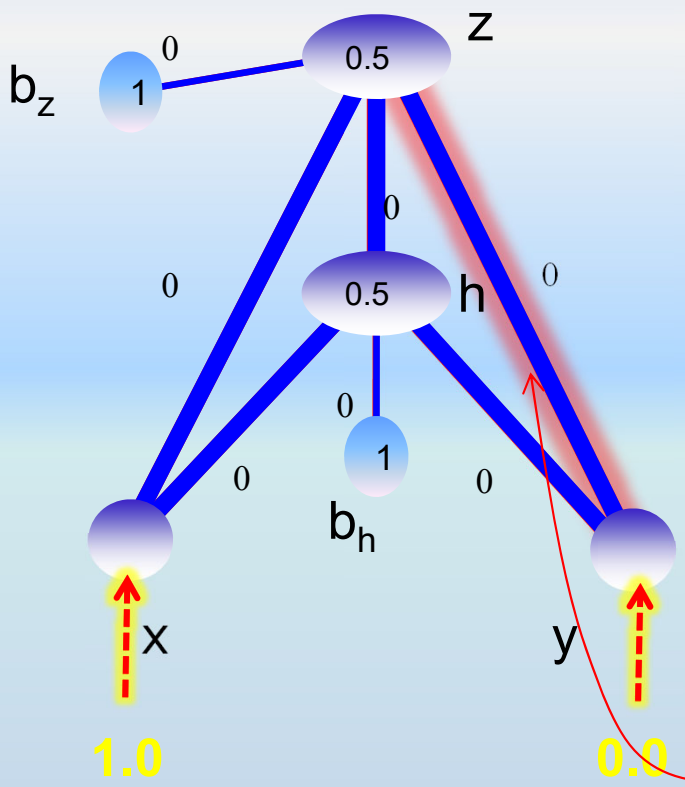
$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x)$$

Training a Network



$$\partial_z = 0.125$$

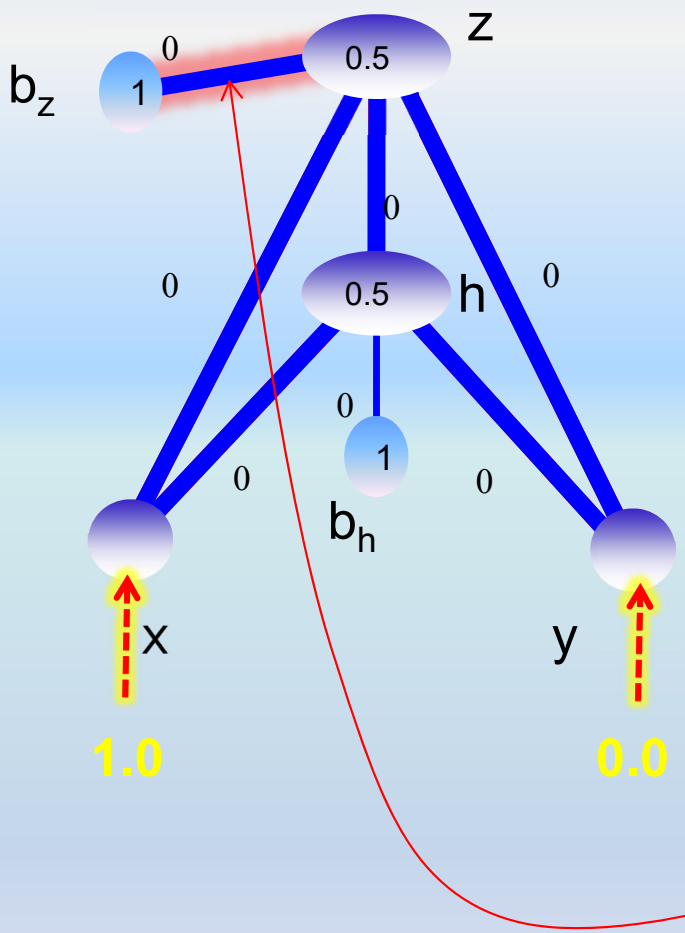
Back-propagate the error to adjust all the weights leading into the output unit.

$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x)$$

$$w_{yz} = w_{yz} + (\eta * \partial_z * o_y)$$

Training a Network



$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

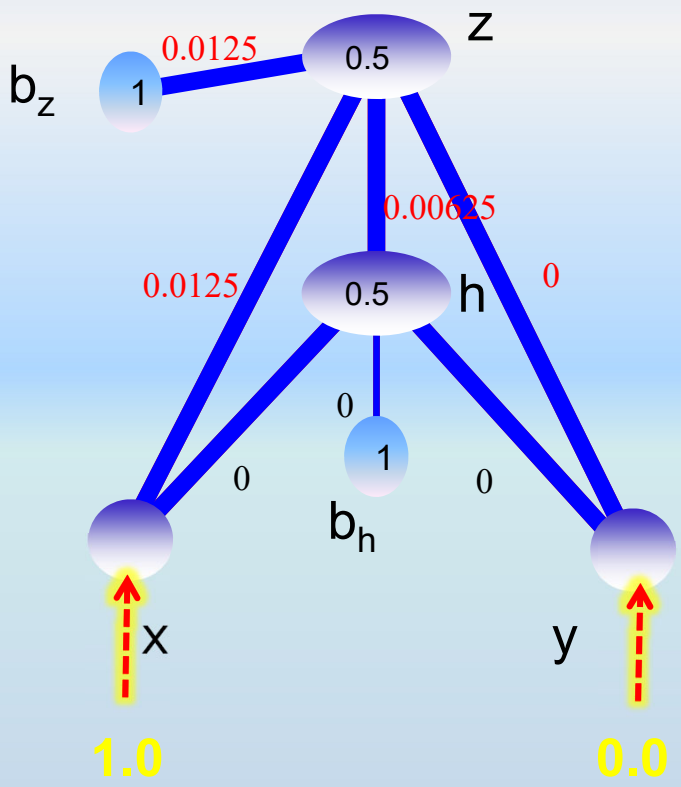
$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h)$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x)$$

$$w_{yz} = w_{yz} + (\eta * \partial_z * o_y)$$

$$w_{b_z z} = w_{b_z z} + (\eta * \partial_z * o_{b_z})$$

Training a Network



$$\partial_z = 0.125$$

Back-propagate the error to adjust all the weights leading into the output unit.

If we set $\eta = 0.1$, we will get the following values:

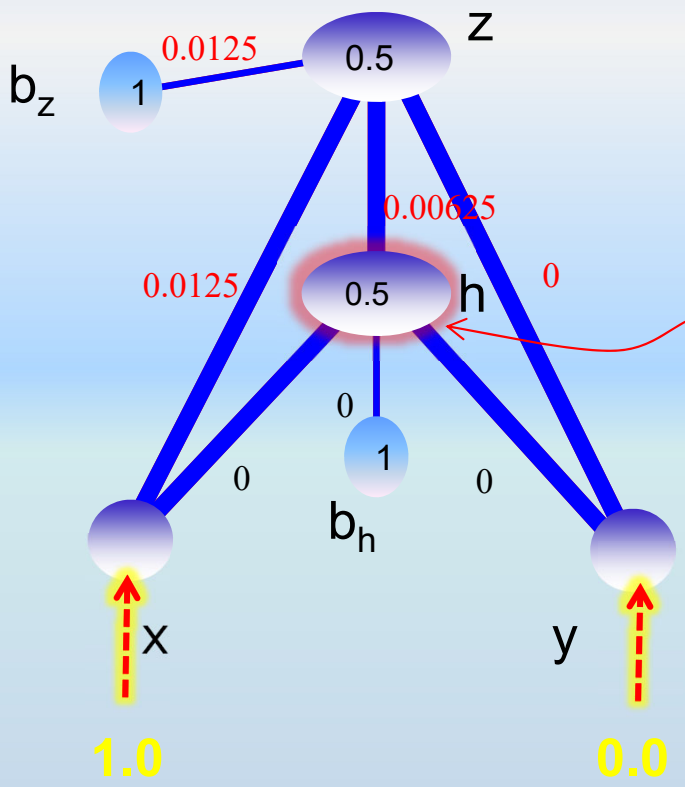
$$w_{hz} = w_{hz} + (\eta * \partial_z * o_h) = 0.00625$$

$$w_{xz} = w_{xz} + (\eta * \partial_z * o_x) = 0.0125$$

$$w_{yz} = w_{yz} + (\eta * \partial_z * o_y) = 0.0$$

$$w_{b_z z} = w_{b_z z} + (\eta * \partial_z * o_{b_z}) = 0.0125$$

Training a Network



Calculate error on the hidden unit ***h***.

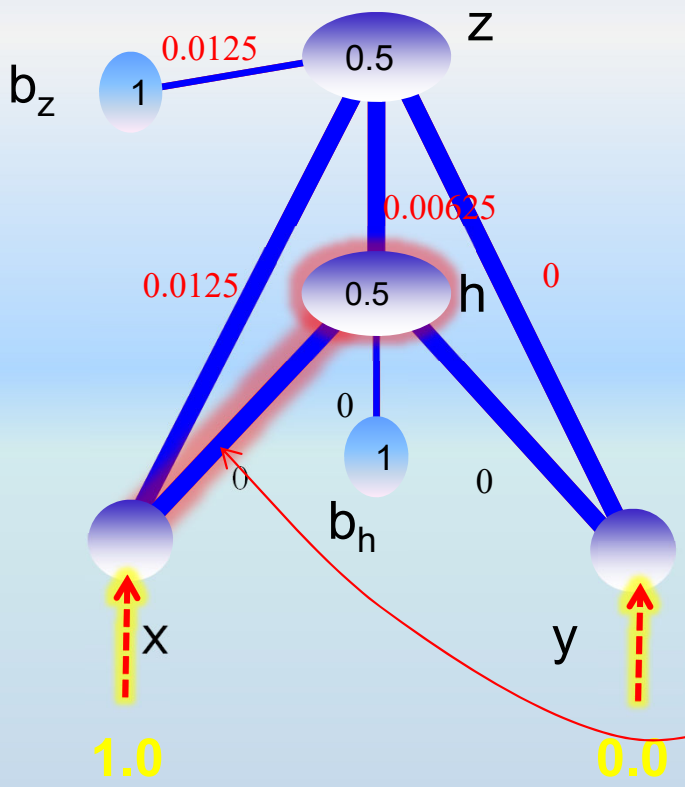
$$\partial_h = o_h * (1 - o_h) \sum_{k=1}^{m} (\partial_k * W_{hk})$$

$$\partial_h = o_h * (1 - o_h) * (\partial_z * W_{hz})$$

The **k** subscript is for all the units in the output layer that are connected directly to node **h**; however, in this example there is only one unit.

$$\partial_h = 0.5 * (1 - 0.5) * 0.125 * 0.00625 = 0.000195$$

Training a Network



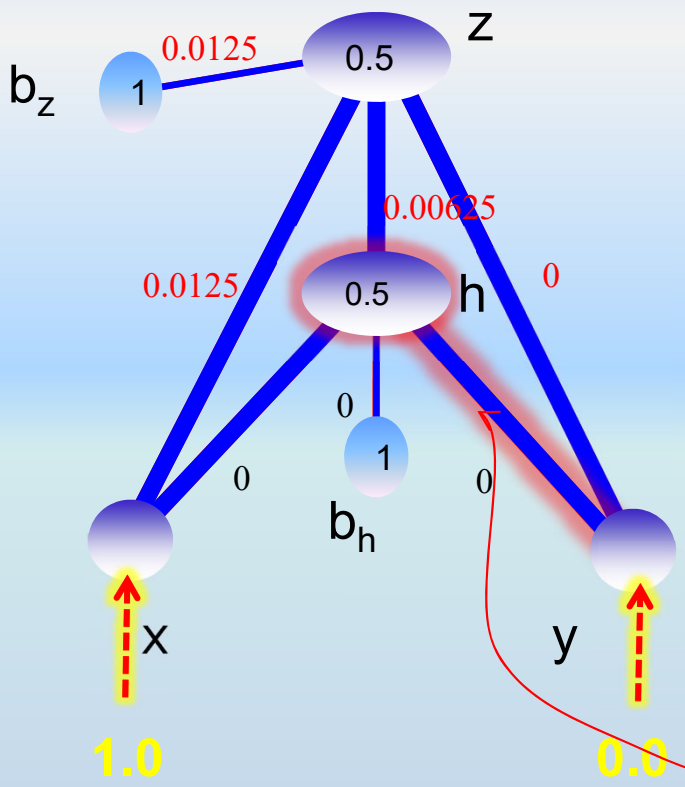
Calculate error on the hidden unit h.

$$\partial_h = o_h * (1 - o_h) * (\partial_z * W_{hz})$$

Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x)$$

Training a Network



Calculate error on the hidden unit h.

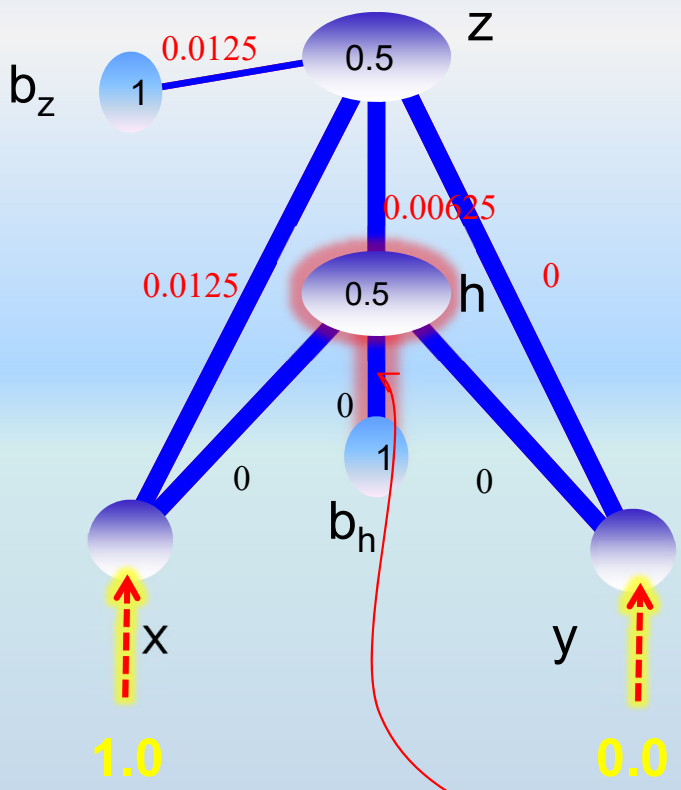
$$\partial_h = (\partial_z * W_{hz}) * o_h * (1 - o_h)$$

Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x)$$

$$w_{yh} = w_{yh} + (\eta * \partial_h * o_y)$$

Training a Network



Calculate error on the hidden unit h.

$$\partial_h = (\partial_z * W_{hz}) * o_h * (1 - o_h)$$

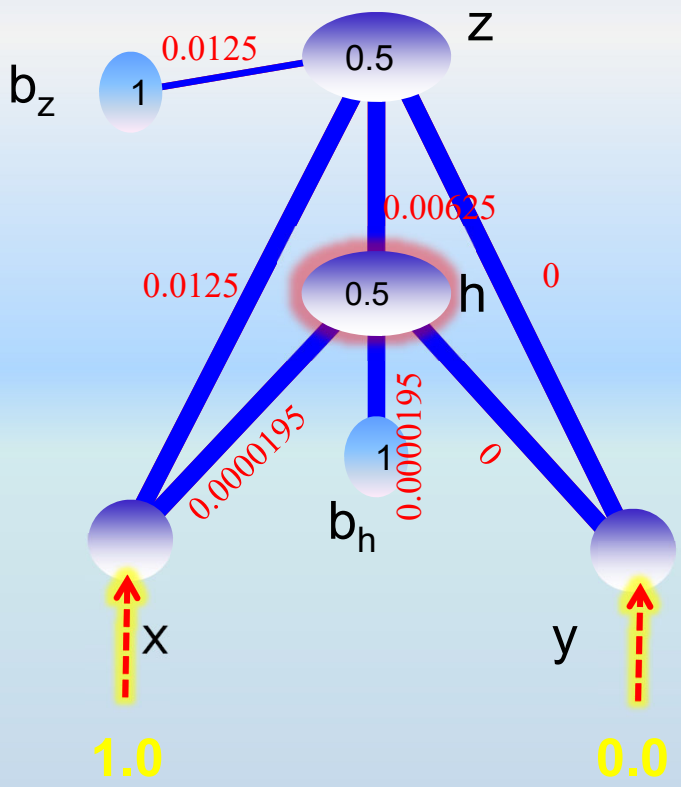
Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x)$$

$$w_{yh} = w_{yh} + (\eta * \partial_h * o_y)$$

$$w_{b_h h} = w_{b_h h} + (\eta * \partial_h * o_{b_h})$$

Training a Network



Calculate error on the hidden unit h.

$$\partial_h = (\partial_z * W_{hz}) * o_h * (1 - o_h) = 0.000195$$

Back-propagate the error to adjust all the weights leading into the hidden unit.

$$w_{xh} = w_{xh} + (\eta * \partial_h * o_x) = 0.0000195$$

$$w_{yh} = w_{yh} + (\eta * \partial_h * o_y) = 0$$

$$w_{b_h h} = w_{b_h h} + (\eta * \partial_h * o_{b_h}) = 0.0000195$$

Backpropagation (Gradient Ascent)

Summary of Weight Update Formulas

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial P}{\partial w}$$

$$w = w + \eta \frac{\partial P}{\partial w}$$

To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (t_k - O_k) Ok (1 - O_k)$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

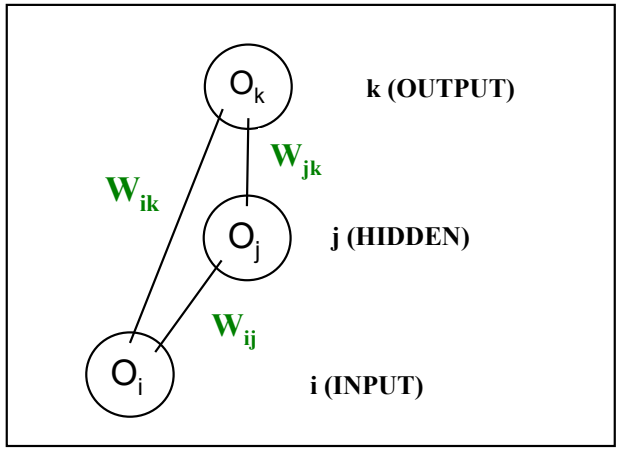
where:

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$

$\eta - learning_rate$

$\delta_k - error_signal$

$O_j - output_of_unit_j$



To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$

$\eta - learning_rate$

$\delta_j - error_signal$ at unit j

$O_i - output_of_unit_i$

Training a Network

BACKPROPAGATION TRAINING

After using one training pattern, the activation value for the output layer will now be **0.5070308**. If you will inspect the output of the network on the other three patterns, we have:

x	y	desired	actual
0	0	0	0.5039062
0	1	1	0.5039062
1	0	1	0.5070308
1	1	0	0.5070308

Sad to say but to get the outputs to within **0.1** requires **20,862 *iterations***, a very long time especially for such a short problem. Fortunately, there are a large number of things that can be done to speed-up the training time. The very simplest thing to do is to increase the learning rate, η. The following table shows how many iterations are used for different values of η.

η	iterations
0.1	20,862
0.5	2,455
1.0	1,060
2.0	480
3.0	(fails)

Derivation of Formulas

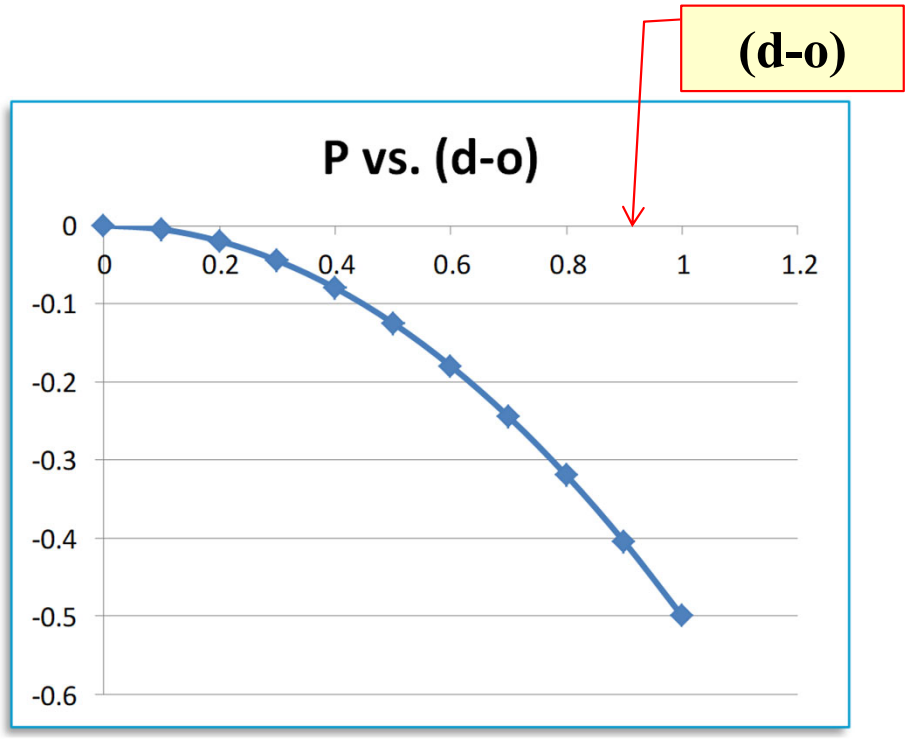
Let’s see how calculus was applied to extract the back propagation formulas…

Gradient Ascent

Performance Function

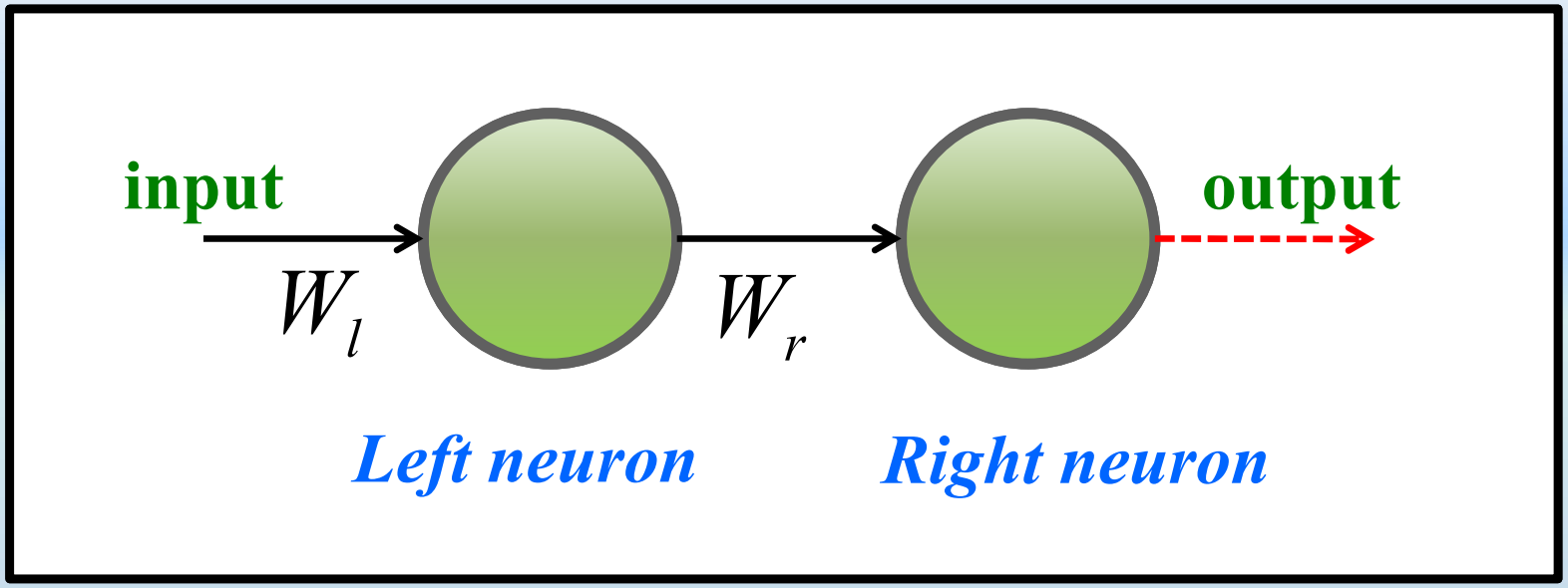
$$P = (-1/2) * ((desired_output - actual_output)^2)$$

desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	-0.005
1	0.8	0.2	-0.02
1	0.7	0.3	-0.045
1	0.6	0.4	-0.08
1	0.5	0.5	-0.125
1	0.4	0.6	-0.18
1	0.3	0.7	-0.245
1	0.2	0.8	-0.32
1	0.1	0.9	-0.405
1	0	1	-0.5



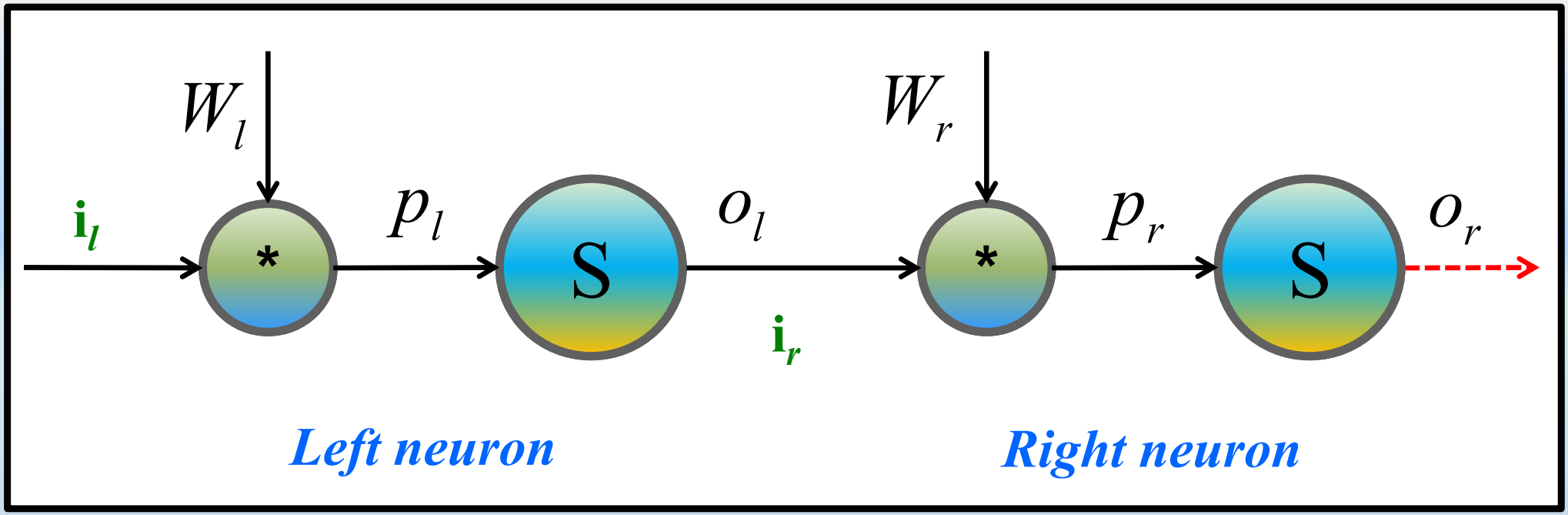
When training the network, we would like to maximise P

Let's consider the simplest possible neural network that will help explain how the back propagation formulas were derived.



2 neurons, 1 input, 1 output

In terms of the processes involved, let's expand the graphical representation of our network of neurons.



See the details of the formula derivations in a separate pdf file (downloadable via Stream).

2 neurons, 1 input, 1 output

Gradient Ascent

Summary of Weight Update Formulas

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial \mathrm{P}}{\partial w}$$

$$w = w + \eta \frac{\partial \mathrm{P}}{\partial w}$$

To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (t_k - O_k) Ok (1 - O_k)$$

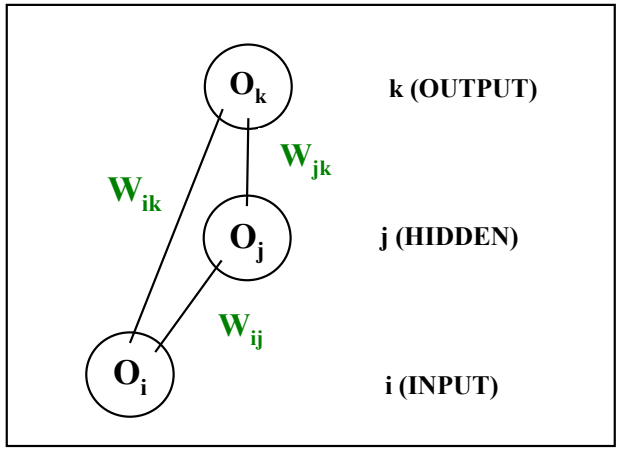
$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$

$\eta - learning_rate$

$\delta_k - error_signal$

$O_j - output_of_unit_j$



To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$

$\eta - learning_rate$

$\delta_j - error_signal$ at unit j

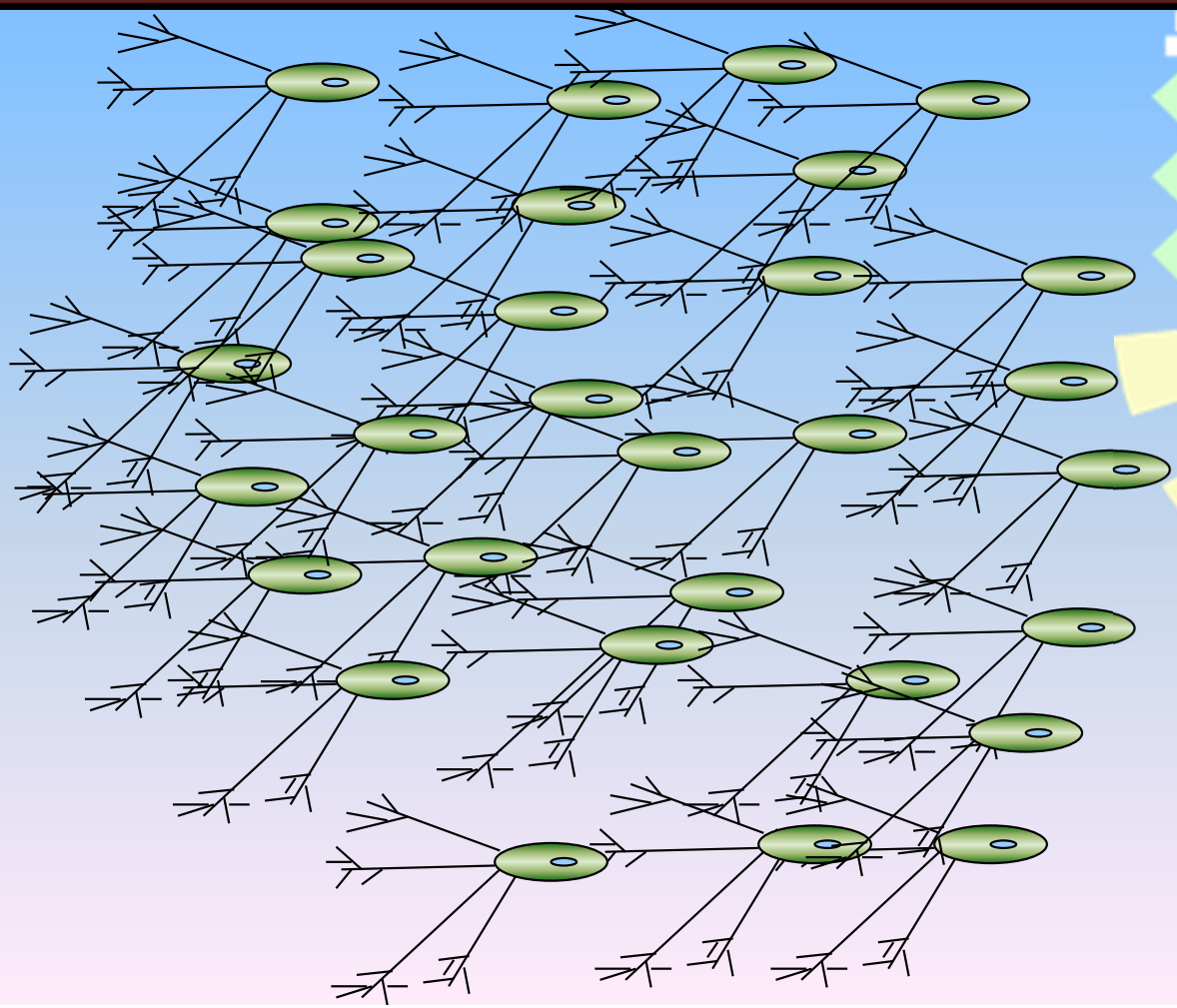
$O_i - output_of_unit_i$

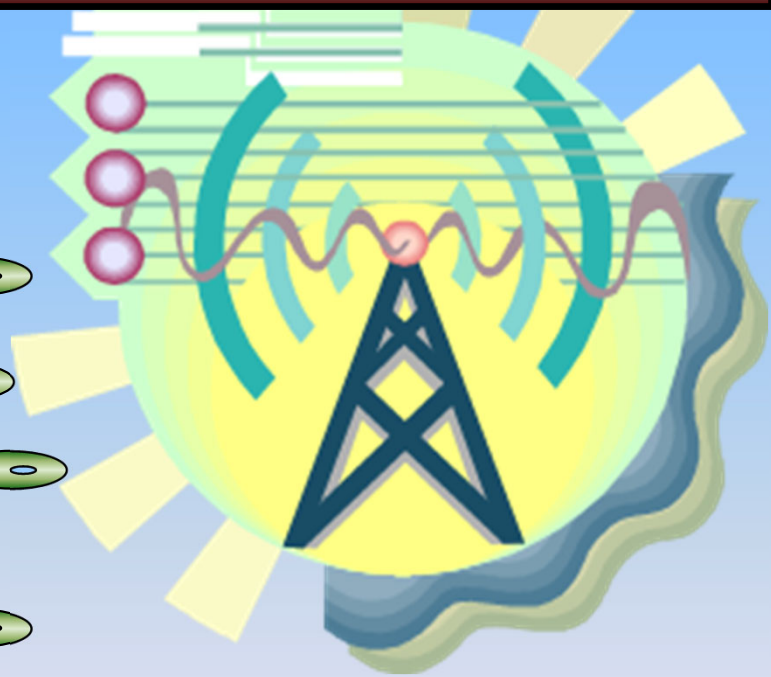
References

- MIT OpenCourseWare
- www.cim.mcgill.ca/~jer/courses/ai/readings/backprop.pdf
- Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead

Neural Networks

Part 2





n.h.reyes@massey.ac.nz

Neural Networks

- [x] Introduction to Multi-layer Feed-forward Network
- [x] Evaluating a Feed-forward Network
- [x] Training a Feed-forward Network
- [x] Theory (Derivation of Formulas)
- [ ] Review of Formulas, Variations in Weight Updates
- [ ] Training Tips
- [ ] Encoding Inputs, Softmax units
- [ ] Performance Measure, Over-fitting problem

Applications, Tutorial Questions

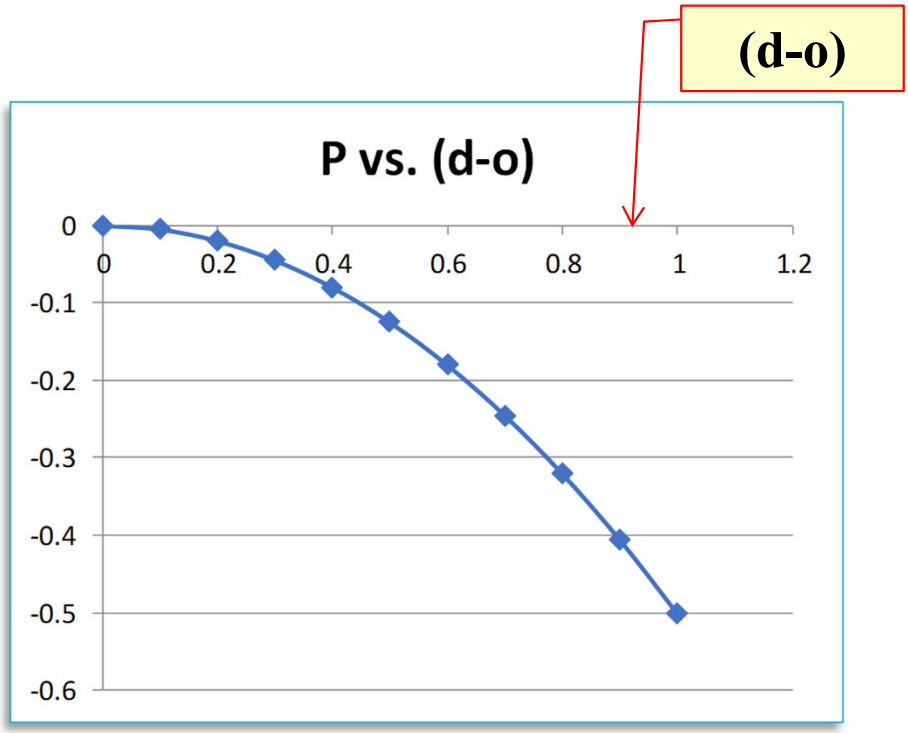
Review of Formulas, Variations in Weight Updates

Gradient Ascent

Performance Function

$$P = (-1/2) * ((desired_output - actual_output)^2)$$

desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	-0.005
1	0.8	0.2	-0.02
1	0.7	0.3	-0.045
1	0.6	0.4	-0.08
1	0.5	0.5	-0.125
1	0.4	0.6	-0.18
1	0.3	0.7	-0.245
1	0.2	0.8	-0.32
1	0.1	0.9	-0.405
1	0	1	-0.5



When training the network, we would like to maximise P

Gradient Ascent

Performance Function

P = (-1/2) * ((desired_output – actual_output)^2)

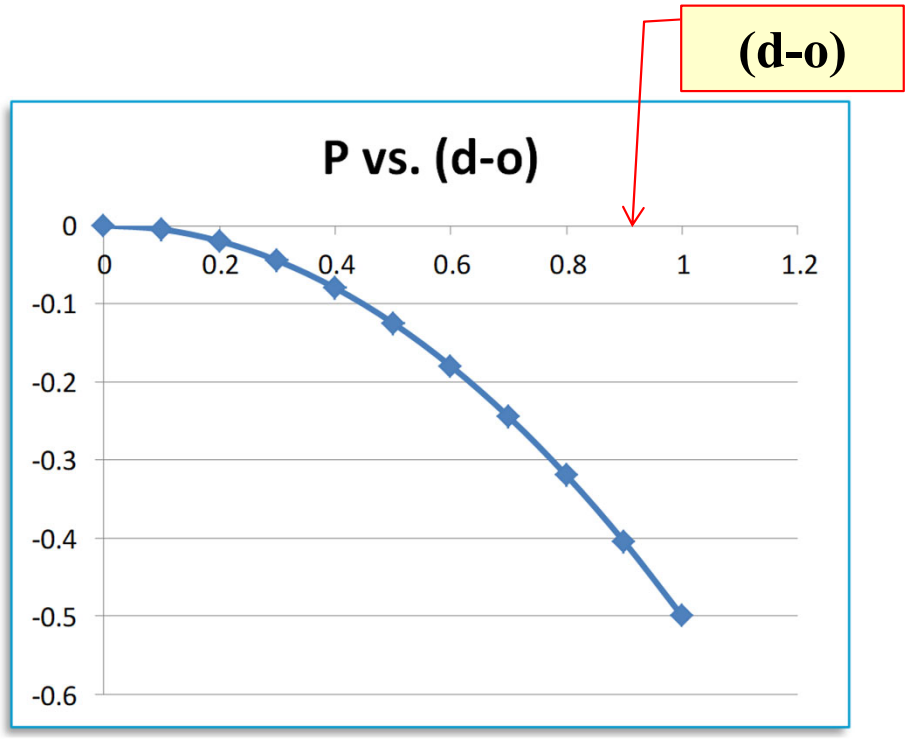
desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	-0.005
1	0.8	0.2	-0.02
1	0.7	0.3	-0.045
1	0.6	0.4	-0.08
1	0.5	0.5	-0.125
1	0.4	0.6	-0.18
1	0.3	0.7	-0.245
1	0.2	0.8	-0.32
1	0.1	0.9	-0.405
1	0	1	-0.5

desired output-actual output

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial \mathrm{P}}{\partial w}$$

$$w = w + \eta \frac{\partial \mathrm{P}}{\partial w}$$



When training the network, we would like to maximise P

Error delta at the output node

Gradient Ascent

Error delta at Output node k:

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs

Gradient Ascent

Error delta at Output node k:

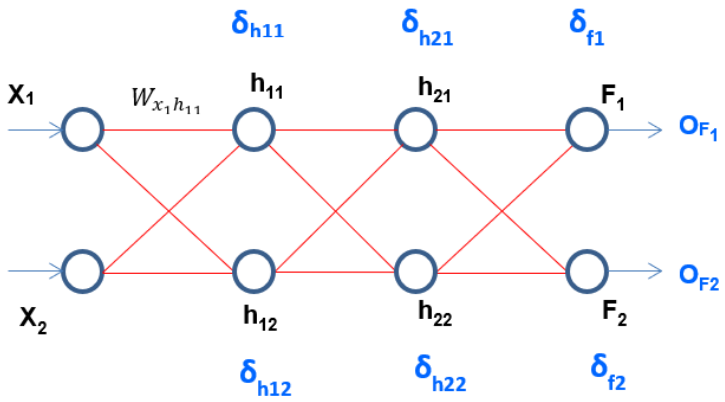
$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

Example:

We can use the error delta formula to compute for error deltas at output nodes F_1 and F_2.

Compute for: δ_{F_1} and δ_{F_2}



Updating weights leading into the output node(s).

Gradient Ascent

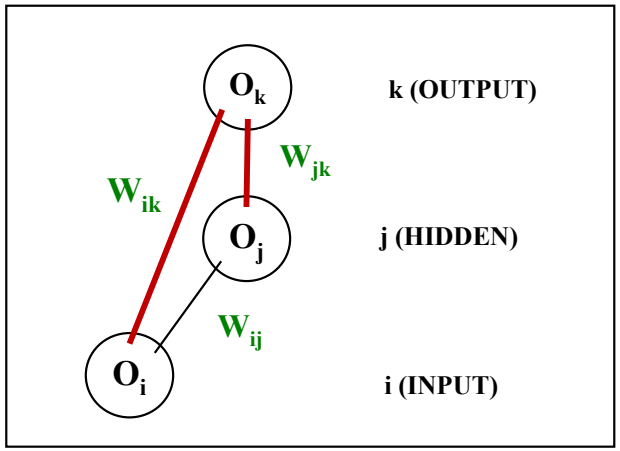
$$\delta_k = \underbrace{(t_k - O_k)}_{\frac{dP}{do}} \underbrace{Ok(1 - O_k)}_{\frac{dO_k}{dp_r}}$$

To adjust weights leading into an output unit:

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$\eta - learning\ rate$
$k - output\ unit$
$J - node\ that\ is\ directly\ connected\ at\ output\ unit\ k$
$\delta_k - error\ signal\ at\ output\ unit\ k$
$O_j - output\ of\ unit\ j$

Gradient Ascent



Example #1:

To adjust weights leading into the output unit k, (i.e. W_{jk} and W_{ik}), we can use the formulas below:

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$

$\eta - learning_rate$
$\delta_k - error_signal$
$O_j - output_of_unit_j$
$O_i - output_of_unit_\boldsymbol{i}$

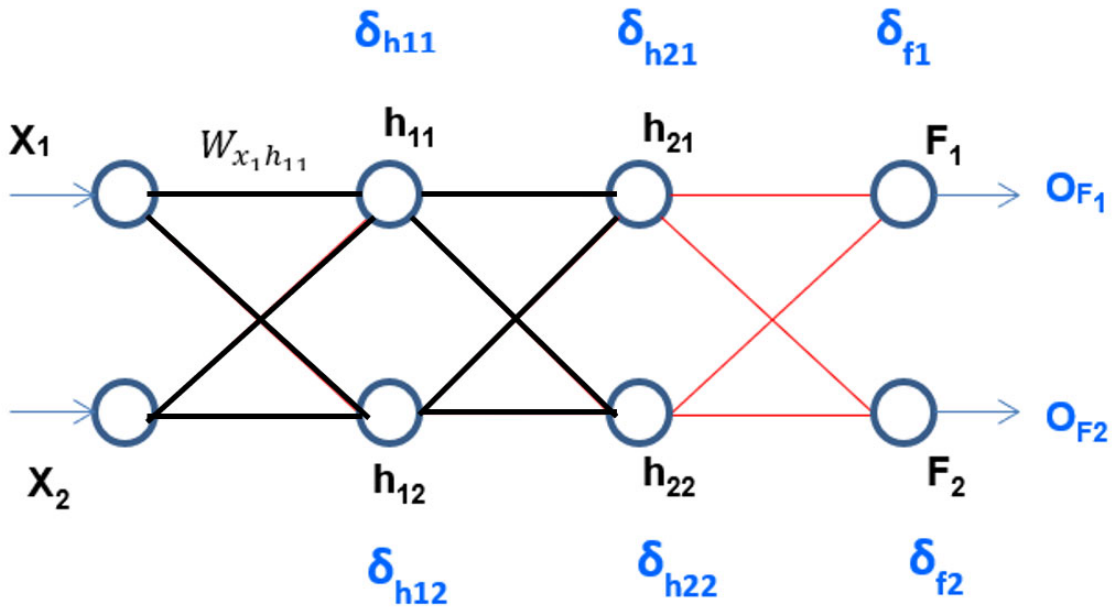
Gradient Ascent

Example #2:

In the figure below, all weights attached to the red-coloured links can be adjusted using the general weight-update formula represented by W_{jk}.

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

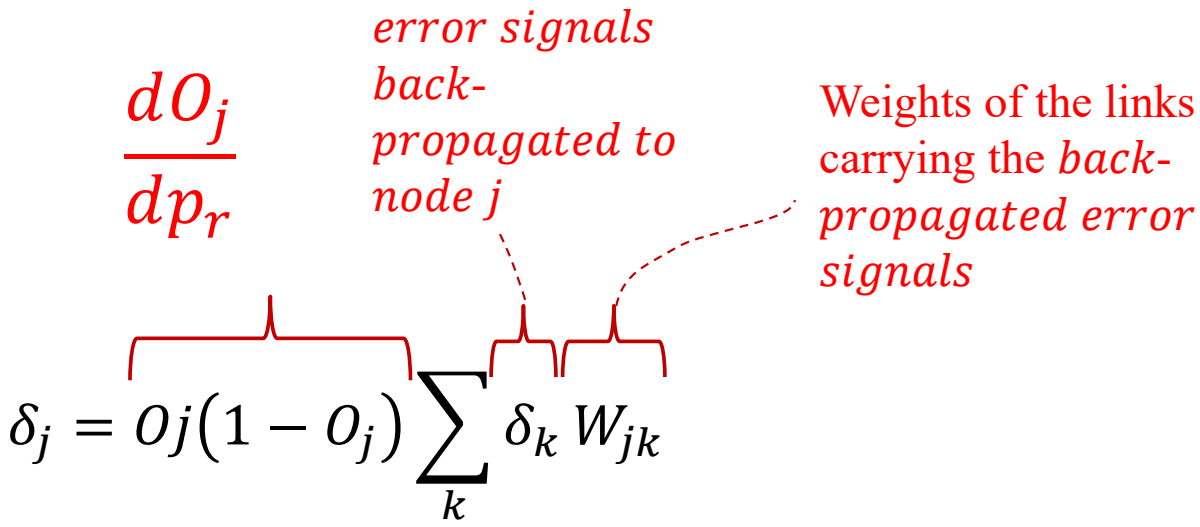
$$W_{jk} = W_{jk} + \eta \delta_k O_j$$



Error delta at the hidden node(s)

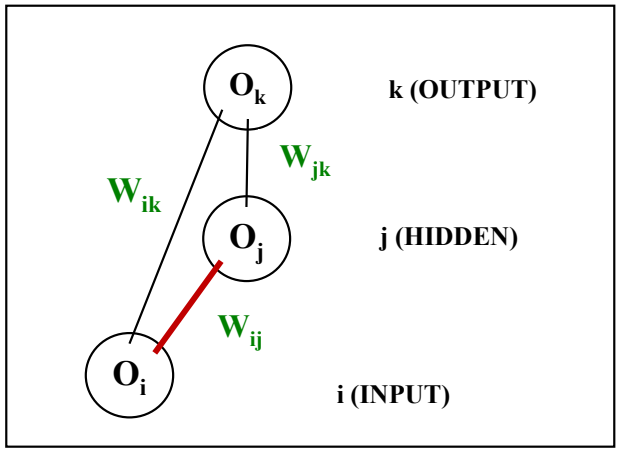
Gradient Ascent

Error delta at a hidden node:



$\eta -$ *learning rate*
k – output unit
J – hidden node that is directly connected at node k (which is residing at a higher layer)
$\delta_k -$ *error delta at node k*
$O_j -$ *output of unit j*

Gradient Ascent



Example#1:

We can compute the error delta at hidden node j in the figure on the right.

$$\frac{dO_j}{dp_r}$$

error signals back-propagated to node j

Weights of the links carrying the *back-propagated error signals*

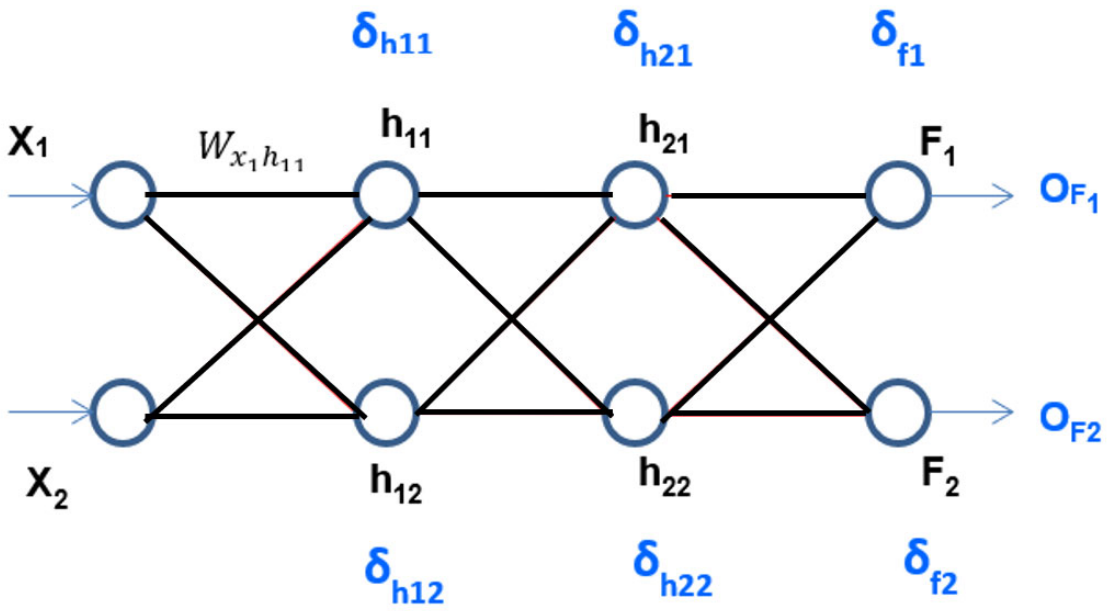
$$\delta_j = Oj(1 - O_j)\sum_k \delta_k W_{jk}$$

$\eta -$ *learning rate*
k – output unit
J – hidden node that is directly connected at node k (which is residing at a higher layer)
$\delta_k -$ *error delta at node k*
$O_j -$ *output of unit j*

Gradient Ascent

Example#2:

In the figure below, we can compute the error deltas at hidden nodes h_{21}, h_{22}, h_{11} and h_{12}, using the general error delta formula below.



$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

Updating weights leading into hidden node(s)

Gradient Ascent

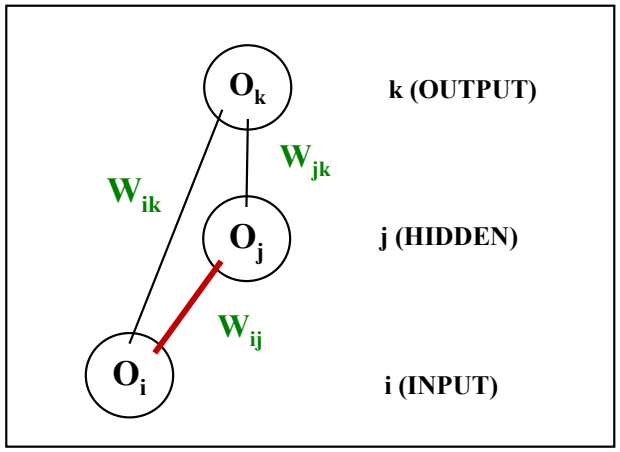
To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j)\sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta\delta_j O_i$$

$\eta - learning_{rate}$
J – *hidden node*
$\delta_j - error\ signal\ at\ node\ j$
i – *node that is directly connected to hidden node j; node j is at a higher layer than node i*
$O_i - output\ of\ node\ i$

Gradient Ascent



Example #1:

Refer to the figure. To adjust the weight W_{ij}, leading into the hidden unit j, we can apply the formula below.

$$\delta_j = Oj(1 - O_j)\sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta\delta_j O_i$$

$\eta - learning_{rate}$
J – *hidden node*
$\delta_j - error_signal\ at\ node\ j$
i – *node that is directly connected to hidden node j;*
$O_i - output_of_unit_i$

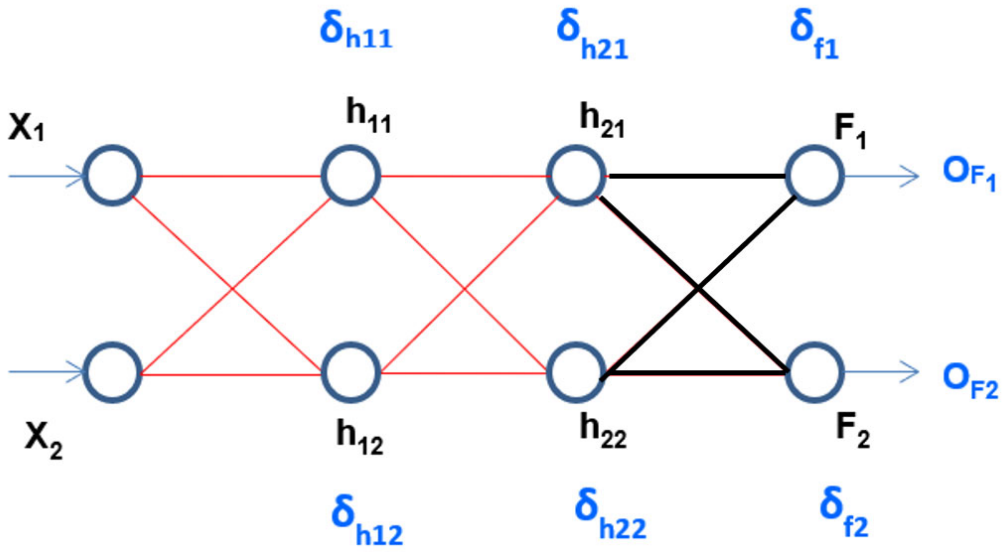
Gradient Ascent

Example #2:

Refer to the figure. To adjust all the weights leading into the hidden nodes (*i.e.* the weights attached to the red-coloured links), we can apply the general weight-update formula below.

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$



$\eta - learning_{rate}$
J – *hidden node*
$\delta_j - error_signal\ at\ node\ j$
i – *node that is directly connected to hidden node j;*
$O_i - output_of_unit_i$

Gradient Ascent

Summary of Weight Update Formulas

$$\frac{dP}{do} = (d - o)$$

$$\Delta w \ \alpha \ \frac{\partial \mathrm{P}}{\partial w}$$

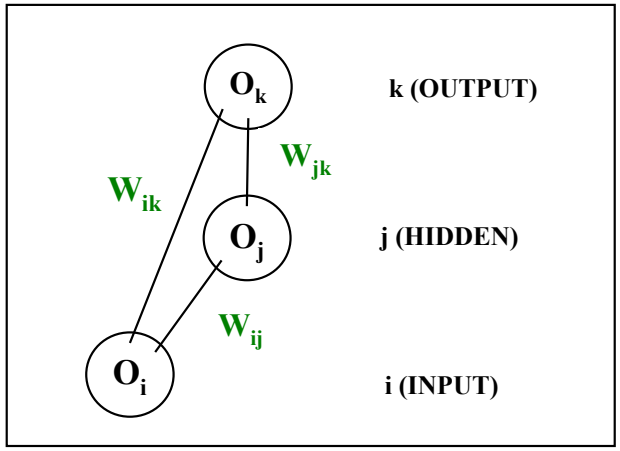
$$w = w + \eta \frac{\partial \mathrm{P}}{\partial w}$$

To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (t_k - O_k)Ok(1 - O_k)$$

$$W_{jk} = W_{jk} + \eta \delta_k O_j$$

$$W_{ik} = W_{ik} + \eta \delta_k O_i$$



$\eta - learning_rate$
$\delta_k - error_signal$
$O_j - output_of_unit_j$

To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} + \eta \delta_j O_i$$

$\eta - learning_rate$
$\delta_j - error_signal$ at unit j
$O_i - output_of_unit_i$

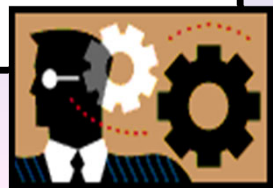
Variations of Backpropagation

Backpropagation Training

On-line / Continuous Update/Stochastic Gradient Descent : Method 1

Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
5. Use another formula to find out error for the hidden layer unit.
6. Adjust the weights leading into the hidden layer unit via another formula.
7. Repeat steps 1 thru 6 for the second, third patterns,…



Backpropagation Training

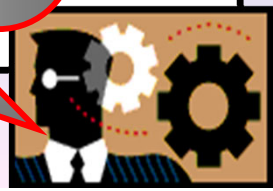
On-line / Continuous Update/Stochastic Gradient Descent : Method 1

Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
5. Use another formula to find out error for the hidden layer unit.
6. Adjust the weights leading int another formula.
7. Repeat steps 1 thru 6 for the

The weight changes are applied after each pattern is presented to the network.

This is the method that we used in solving the tutorial problems!



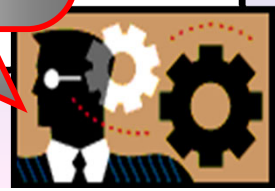
Backpropagation Training

On-line / Continuous Update/Stochastic Gradient Descent : Method 2

Iterative minimization of error over training set

1. Put one of the training patterns to be learned on the input units.
2. Find the values for the hidden unit and output unit.
3. Find out how large the error is on the output unit.
4. Use another formula to find out error for the hidden layer unit.
5. Use one of the back-propagation formulas to adjust the weights leading into the output unit.
6. Adjust the weights leading into the hid… another formula.
7. Repeat steps 1 thru 6 for the second,…

It is possible to calculate all the errors first, then adjust all the weights afterwards.



Backpropagation Training

Off-line (Mini-Batch) Update: Method 3

Iterative minimization of error over training set

- Group training patterns into **batches** = a fraction of the total number of training patterns to be learned.
- For every pattern within a 'batch', calculate the weight gradient (Δw) for each weight in the network. Accumulate the weight gradients(Δw) for each weight. After all the patterns within a batch have made their contributions, all the weights are updated.
- For example, if there were 50,000 training patterns to be learned, an update might happen for every 'batch' of 1000 patterns (1/50 of the total number of training patterns).
- 1 Iteration = equivalent to using all training patterns in a batch once (e.g. 1,000 patterns)
- 1 Epoch = equivalent to using all training patterns in the complete training set once (e.g. 50,000 patterns)

Backpropagation Training

Off-line (Mini-Batch) Update: Method 3

Iterative minimization of error over training set

- Group training patterns into **batches** = a fraction of the total number of training patterns to be learned.
- For every pattern within a 'batch', calculate the weight changes (Δw) for each weight in the network. Accumulate the weight changes (Δw) for each weight. After all the patterns within a batch have made their contributions, all the weights are updated.
- For example, if there were 50,000 training patterns to be learned, an update might happen for every 'batch' of 1000 patterns (1/50 of the total number of training patterns).
- 1 Epoch = equivalent to using all training patterns in the complete training set once
- Neural Network training usually takes many training epochs (e.g. until its performance is at an acceptable level)

Backpropagation Training

Off-line (Mini-Batch) Update: Method 3

Repeat until Epoch > MAX_EPOCHS

For i=1 to NUM_OF_MINI_BATCHES

For each pattern in MiniBatch i

- forward propagate inputs
- compute error deltas at the output units and hidden units
- backpropagate error deltas (needed for computing all weight gradients)
- compute weight gradients and accumulate them, but do not apply them to the weights yet

  e.g.

  $$\Delta W_{jk} = \Delta W_{jk}^{(1)} + \Delta W_{jk}^{(2)} + \Delta W_{jk}^{(3)} + \cdots + \Delta W_{jk}^{(size\ of\ MiniBatch)}$$

End For

Adjust all weights using the computed weight gradients

Compute network performance (P) on Validation Set (or both Training and Test sets)

If (P is at an acceptable level) then End_Training

End For

End Repeat

Backpropagation Training

Off-line (Full-Batch) Update: Method 4

Iterative minimization of error over training set

- For every pattern in the training set, calculate the weight changes (Δw) for each weight in the network. Accumulate the weight changes (Δw) for each weight. After all the patterns have made their contributions, all the weights are updated.

Backpropagation Training

Off-line (Full-Batch) Update: Method 4

Repeat until Epoch > **MAX_EPOCHS**

For each pattern in the **Training Set**

- forward propagate inputs
- compute error deltas at the output units and hidden units
- backpropagate error deltas (needed for computing all weight gradients)
- compute weight gradients and accumulate them, but do not apply them to the weights yet

  e.g.

$$\Delta W_{jk} = \Delta W_{jk}^{(1)} + \Delta W_{jk}^{(2)} + \Delta W_{jk}^{(3)} + \cdots + \Delta W_{jk}^{(number\ of\ patterns)}$$

End For

Adjust all weights using the computed weight gradients

Compute network performance (P) on Validation Set (or both Training and Test sets)

If (P is at an acceptable level) then End_Training

End Repeat

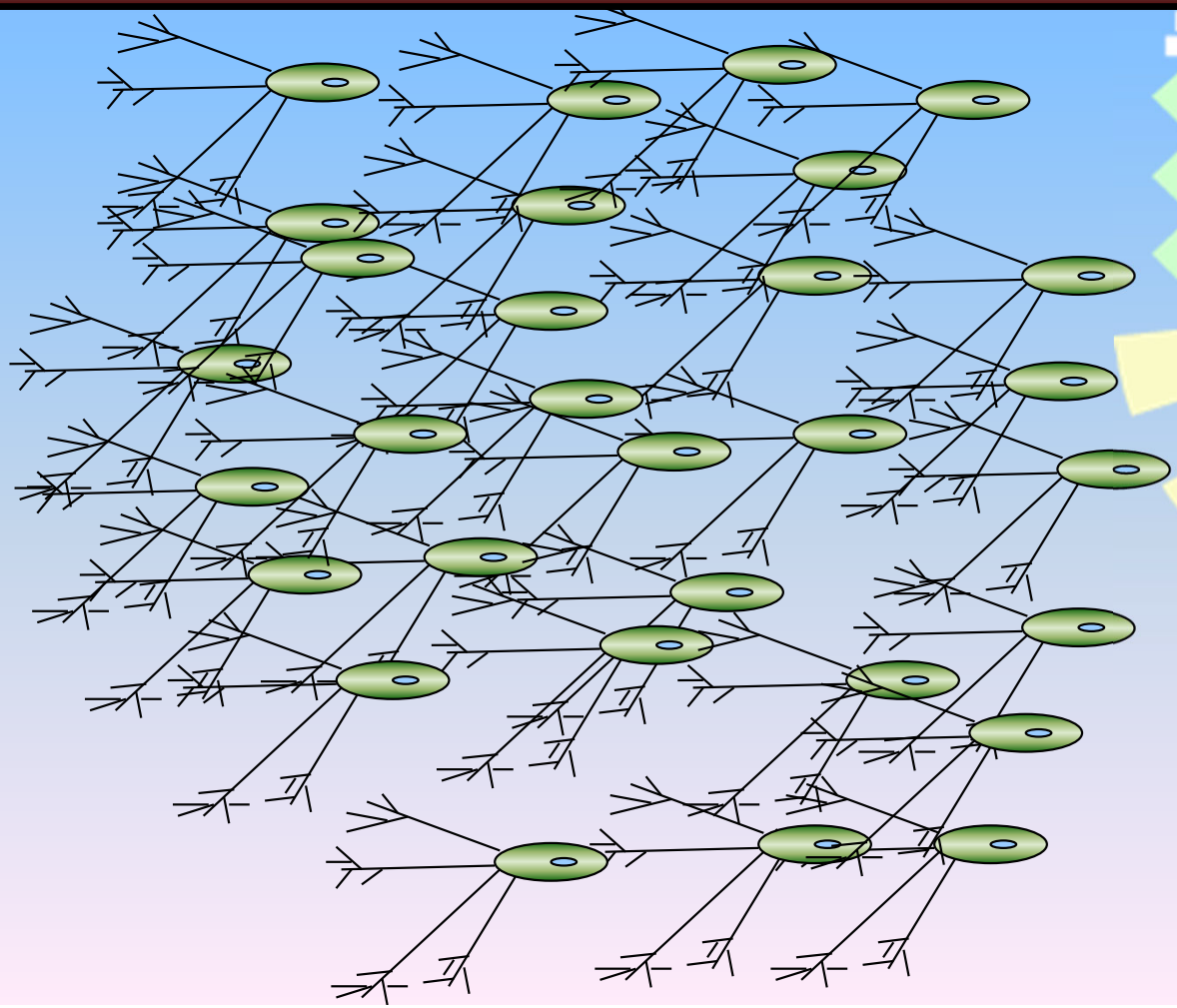
Backpropagation Training

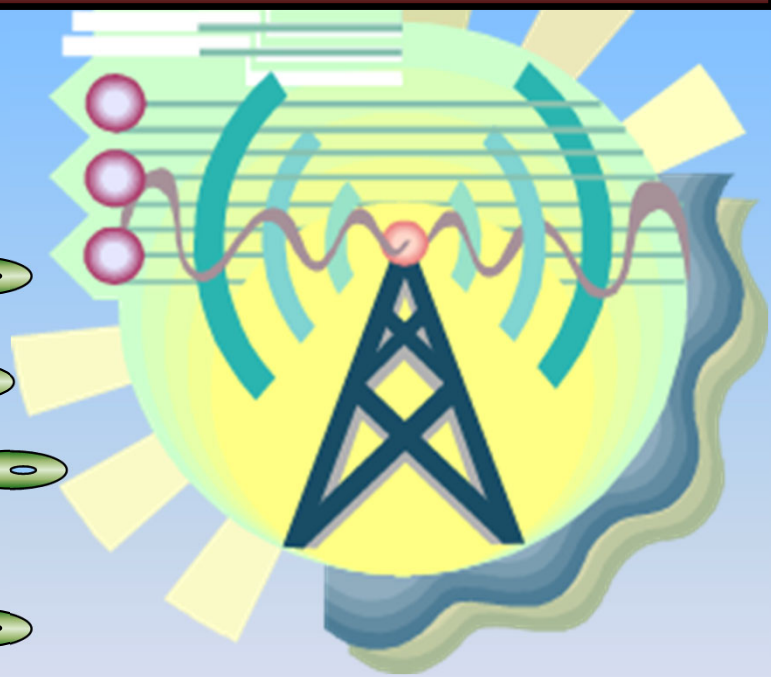
On-line vs. Off-line Update

- The **on-line update/SGD method** is not a "correct" implementation of gradient ascent/descent for the total training error. However, it is often argued that allowing the weights to change immediately after the presentation of an instance of the training set produces a beneficial effect on training – it tends to push the system out of some shallow local minimum.
- The **off-line batch update** method of actually adding up the gradients for all the instances and changing the weights based on that complete gradient is also used and has some advantages for smaller systems. (training set with <= 2,000 patterns).
- For larger nets and larger data sets, the **mini-batch method** has substantial performance advantages. (typical mini-batch size: 64, 128, 256, 512). Benefits from vectorisation using GPUs.

Neural Networks

Part 3





n.h.reyes@massey.ac.nz

Neural Networks

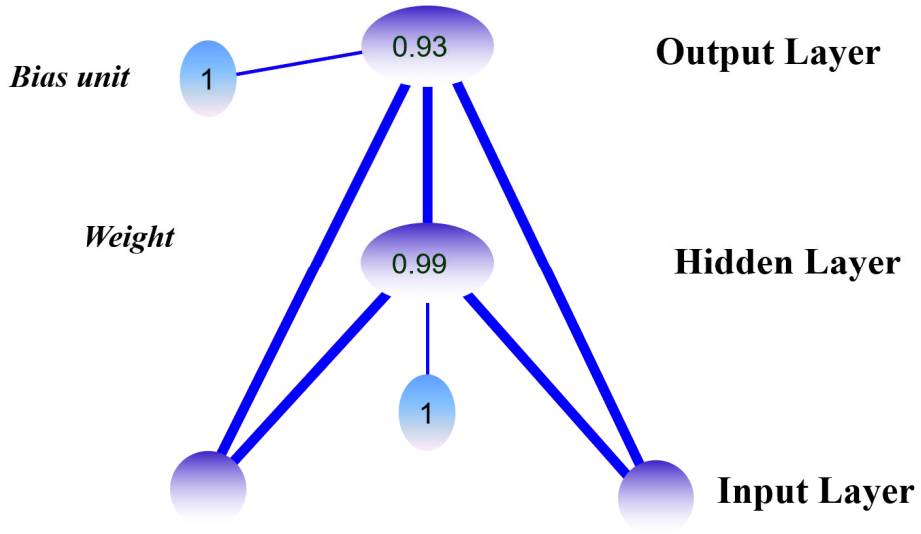
- [x] Introduction to Multi-layer Feed-forward Network
- [x] Evaluating a Feed-forward Network
- [x] Training a Feed-forward Network
- [x] Theory (Derivation of Formulas)
- [x] Review of Formulas, Variations in Weight Updates
- [ ] Overfitting Problem, Softmax units, Performance Metrics
- [ ] Variations in Activation Function
- [ ] Input/Output Representations, Bias Units, Hyperparameter, Data Shuffling
- [ ] Experiments using Tensorflow playground

Possible Training Problem: getting stuck at a local minimum

Training a Network

BACKPROPAGATION TRAINING

One unfortunate problem with backprop is that **when the learning rate is too large** the **training can fail** as it did in the case of the neural network we considered for solving the XOR problem, when $\eta = 3.0$. Here, after **10,000** epochs the results were:



x	y	$z_{desired}$	z_{actual}
1	0	1	0.994
0	0	0	0.009
0	1	1	0.994
1	1	0	1.000

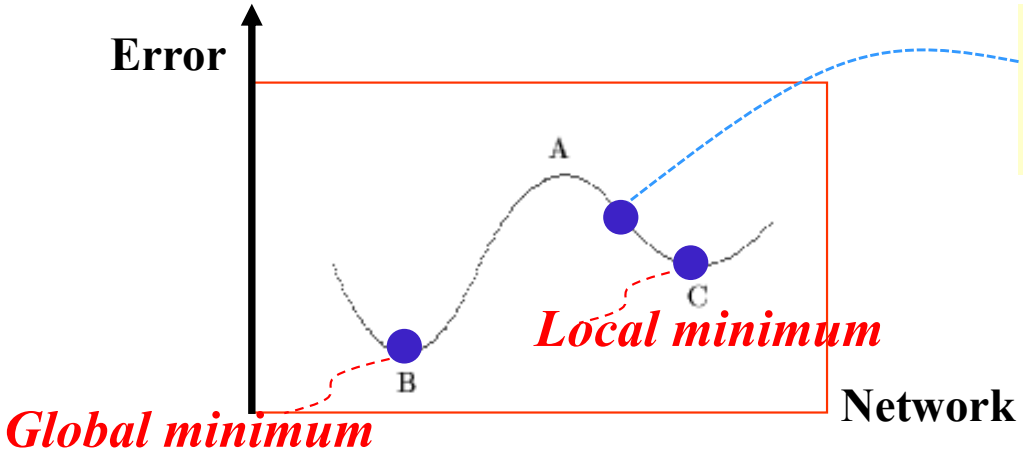
Incorrect result!

the output for the last pattern is **1** not **0**.

Training a Network

BACKPROPAGATION TRAINING

The geometric interpretation of this problem is that when the network tries to make the error go down the network may get stuck in a valley that is not the lowest possible valley.



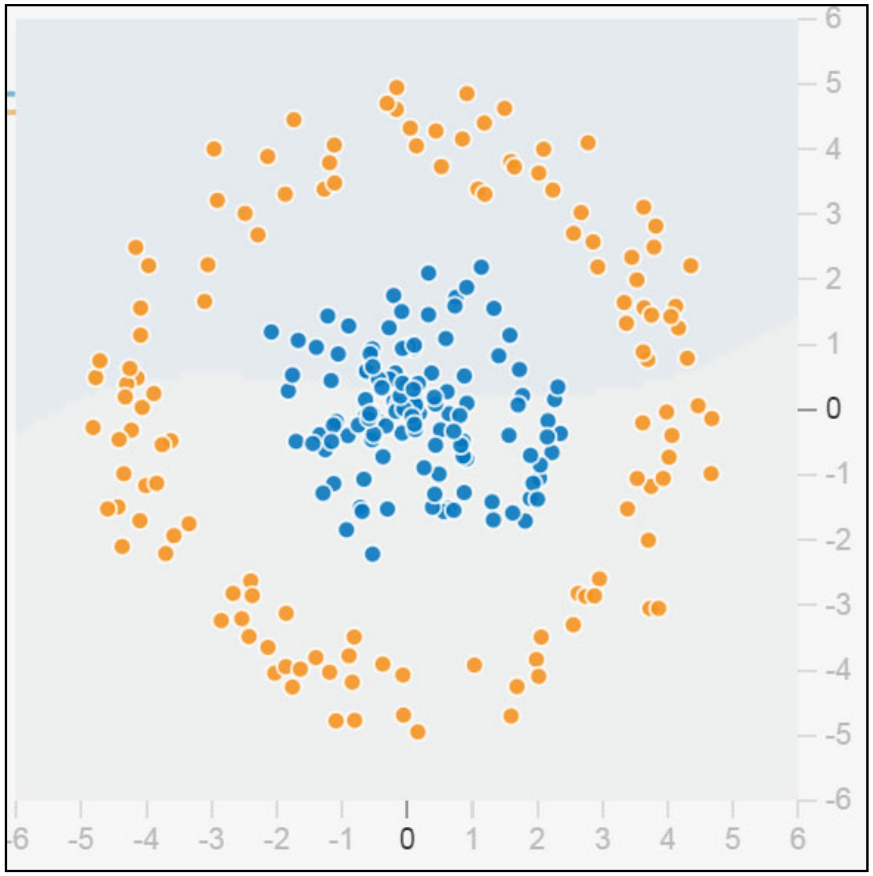
Each point in the curve corresponds to a complete neural network configuration

When backprop starts at point ***A*** and tries to minimize the error, you hope the process will stop when it hits the low point at ***B*** however you could get unlucky and hit the not so low point at *C*. The lowest point is a ***global minimum*** and the not so low point is a ***local minimum***.

Training a Network

Example:

Classification problem: distinguish the blue dots from the orange dots



Demonstration: **playground.tensorflow.org**

Training a Network

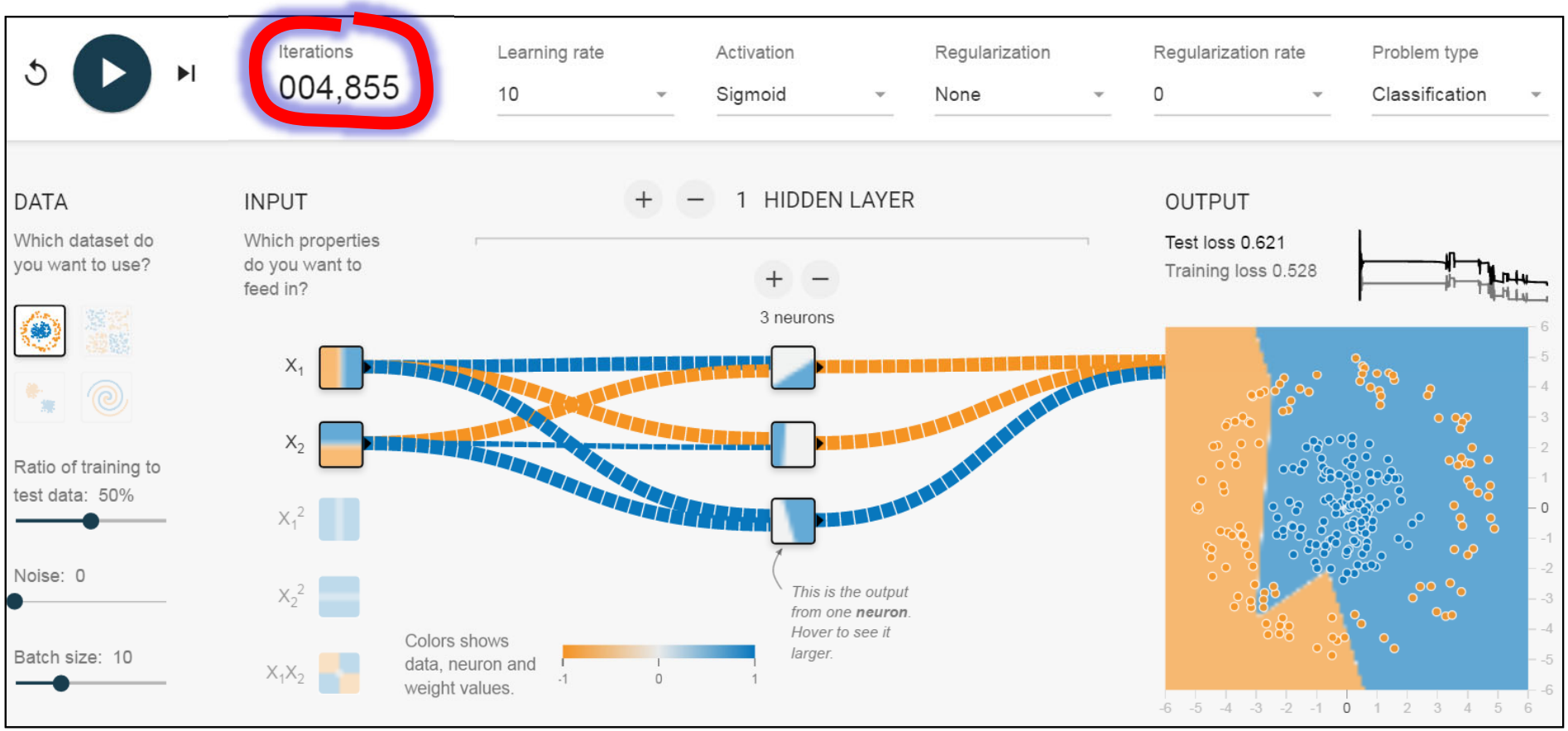
Example: Consequence of setting the learning rate too high.

Regardless of how long you train the network, its performance does not significantly improve anymore.

playground.tensorflow.org

Training a Network

Example: Consequence of setting the learning rate too high.

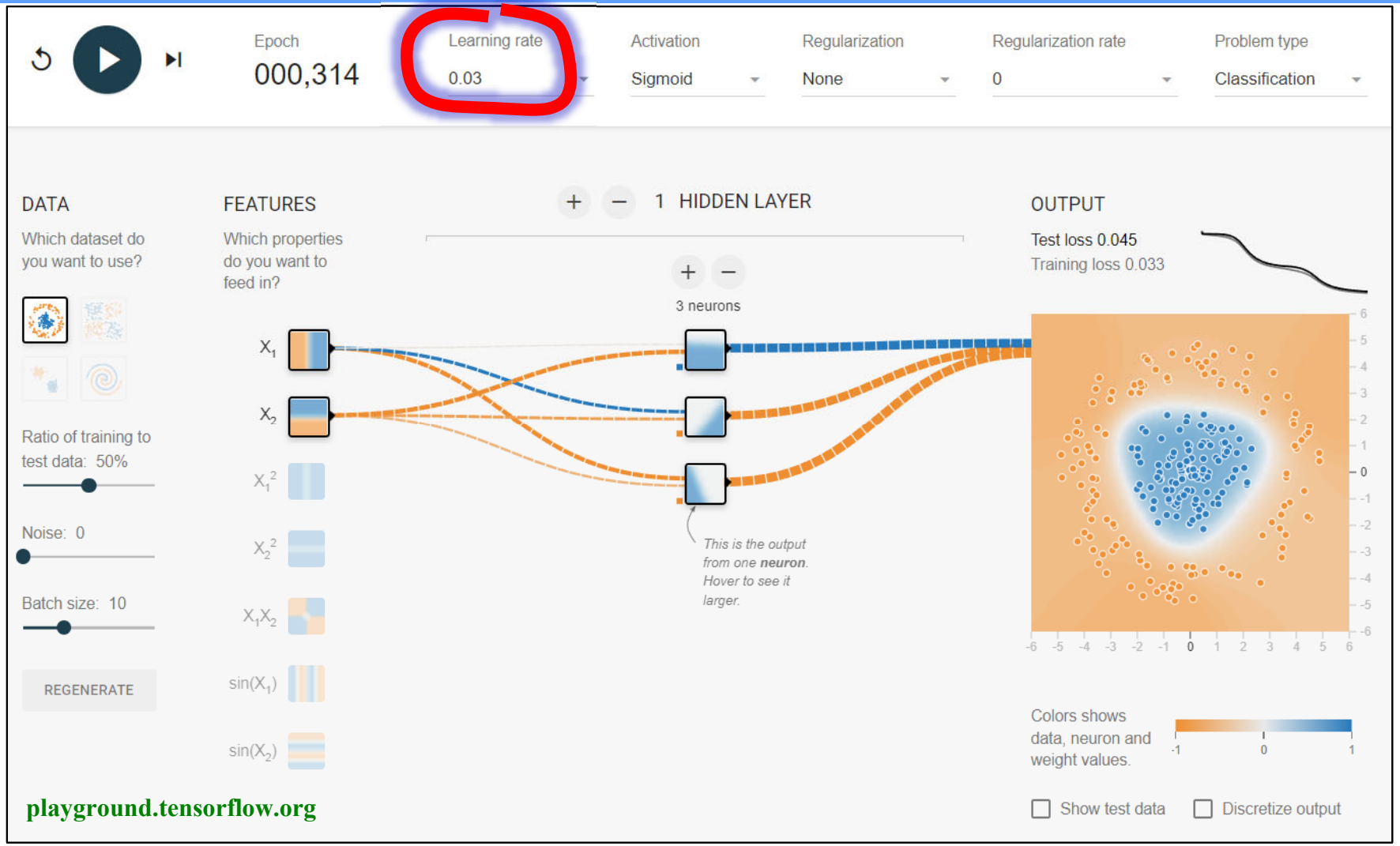


Regardless of how long you train the network, its performance does not significantly improve anymore.

playground.tensorflow.org

Training a Network

Example: Problem fixed!



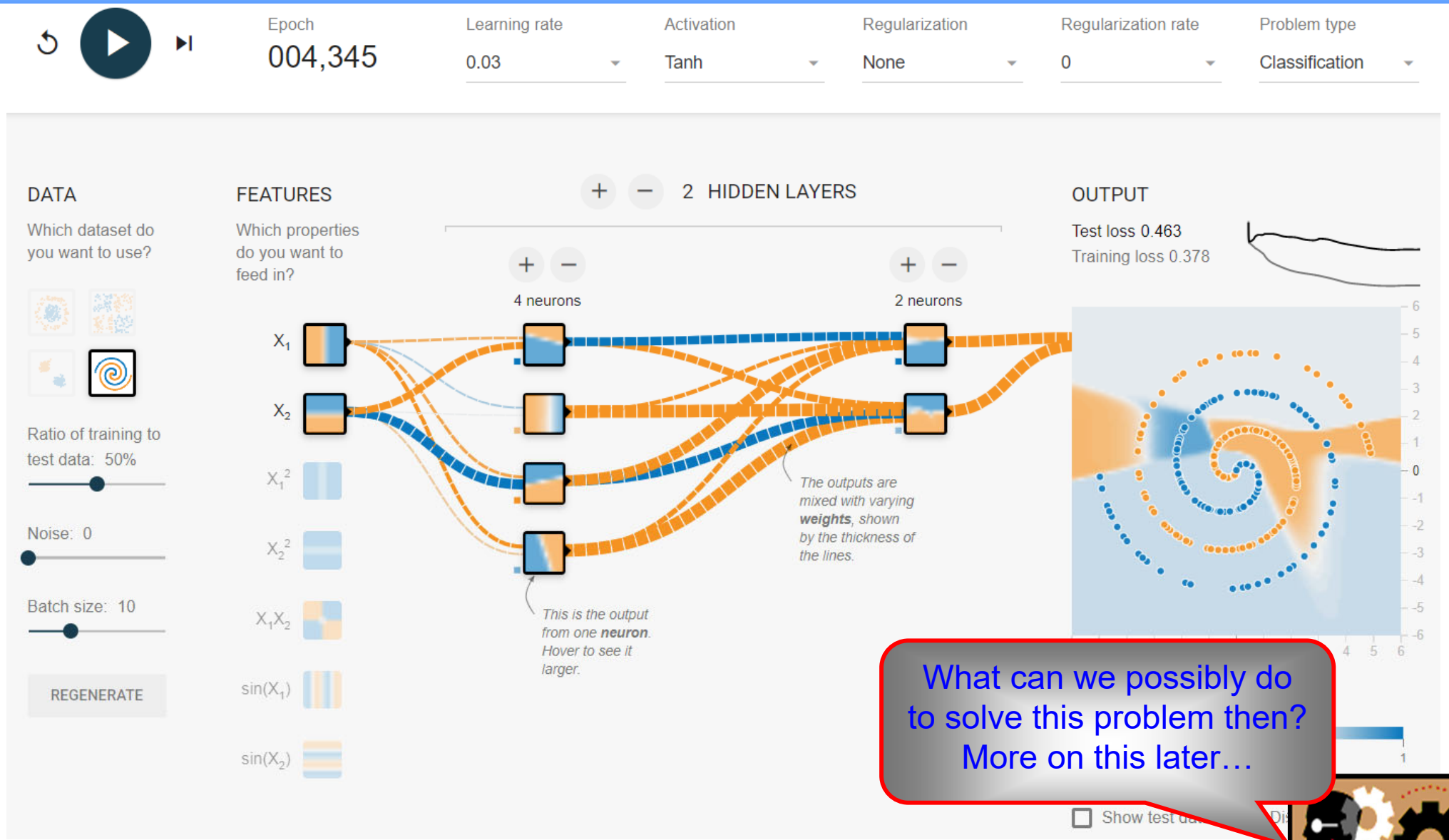
After setting the learning rate to **0.03**, it can be observed that the network is now able to isolate the blue dots from the orange dots.

Spiral Data Set

More challenging classification problem

Training a Network

Example: Unable to solve problem!



Complete Training

- **Given:** Data set, desired outputs and a Neural Net with m weights.
- **Find a setting for the weights that will give good predictive performance on new data. Estimate expected performance on new data.**

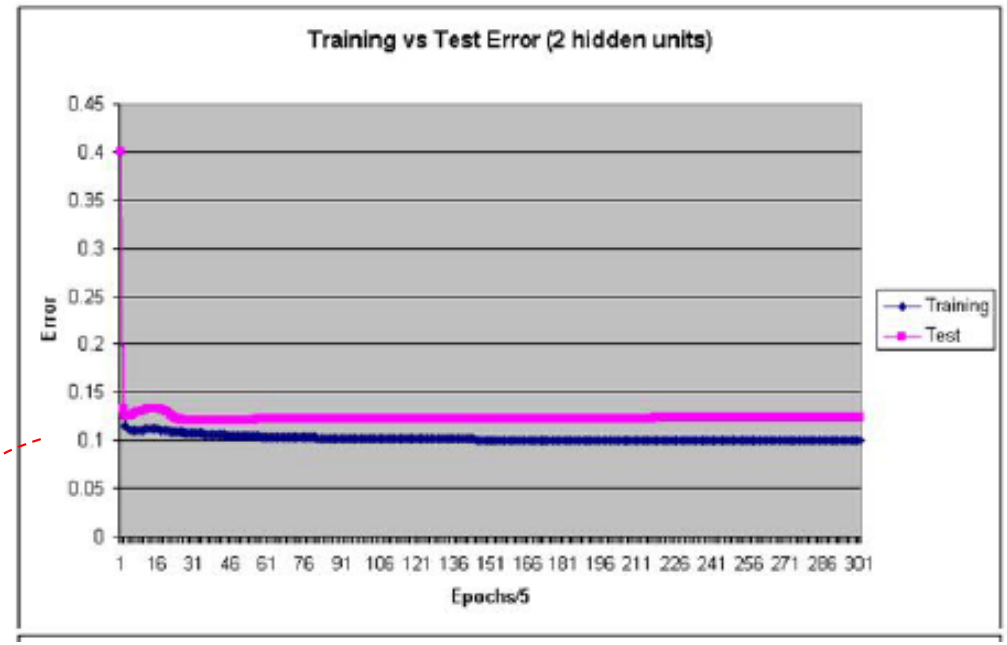
1. **Split data set (randomly) into three subsets:**
   - **Training set – used for picking weights**
   - **Validation set – used to stop training**
   - **Test set – used to evaluate performance**
2. **Pick random, small weights as initial values.**
3. **Perform iterative minimization of error over training set.**
4. **Stop when error on validation set reaches a minimum (to avoid over-fitting).**
5. **Repeat training (from Step 2) several times (to avoid local minima).**
6. **Use best weights to compute error on test set, which is the estimate of performance on new data. Do not repeat training to improve this.**

Notes on Training

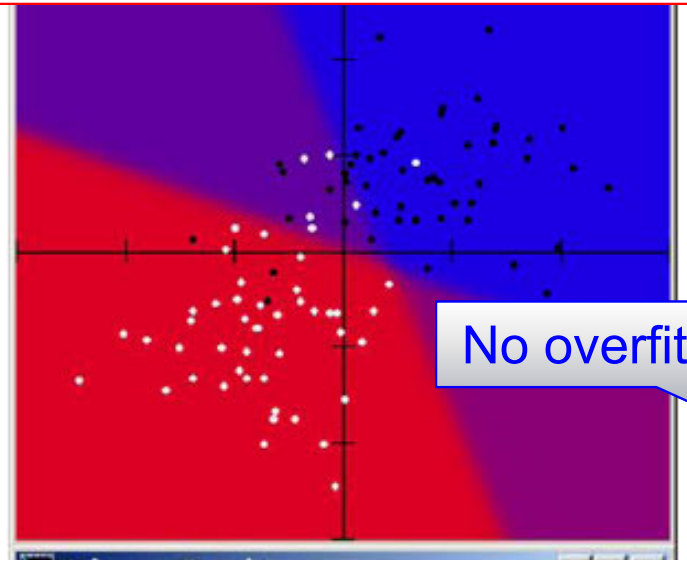
1. pick a set of **random small weights** as the initial values of the weights. This reduces the chance of saturating any of the units initially.
2. we do not want to simply keep going until we reduce the training error to its minimum value. This is likely to **overfit** the **training data**.
3. stop when we get best the performance on the **validation set**
4. Keeping the weights small is a strategy for reducing the size of the hypothesis space. It also reduces the variance of the hypothesis since it limits the impact that any particular data point can have on the output.

How can we tell if a network is learning well?

Training without a validation set



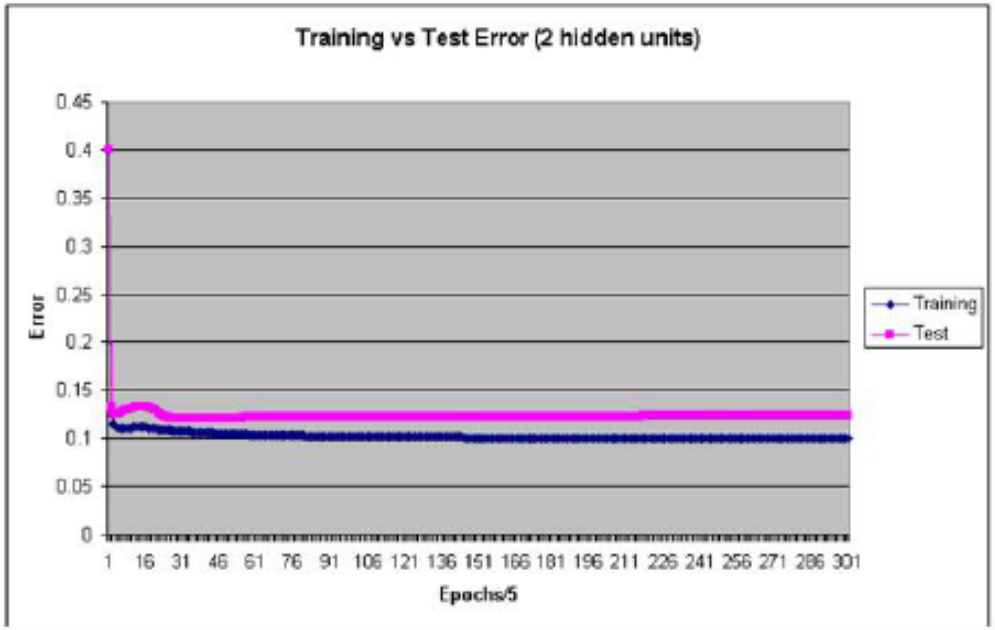
2 inputs, 2 hidden units, 1 output

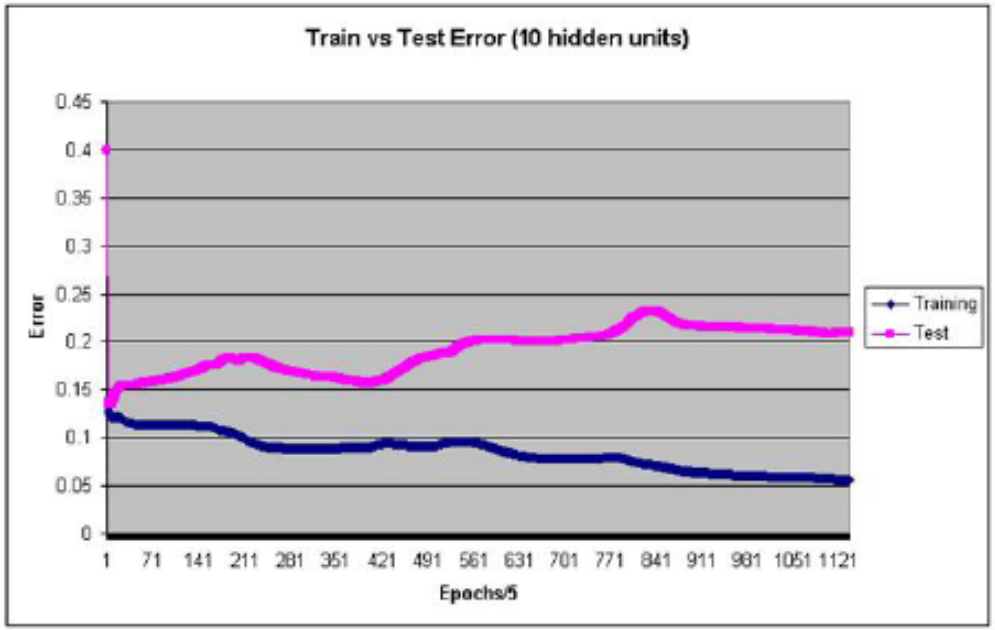


No overfitting!

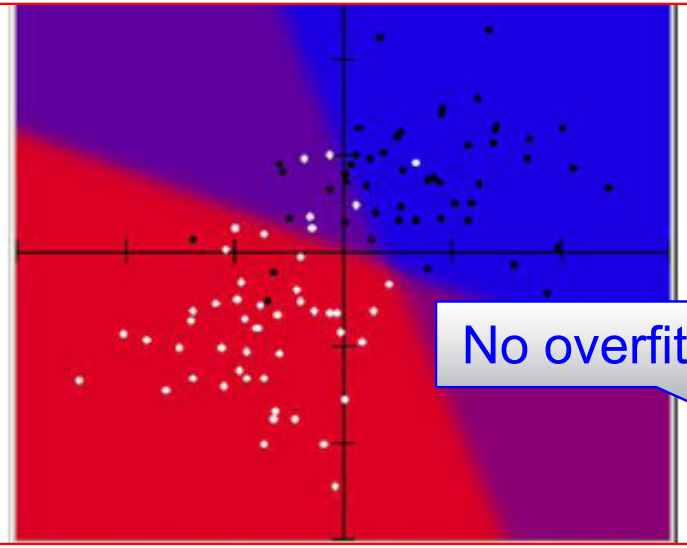
Examine the performance of the network on both the training set and test set.

Training without a validation set



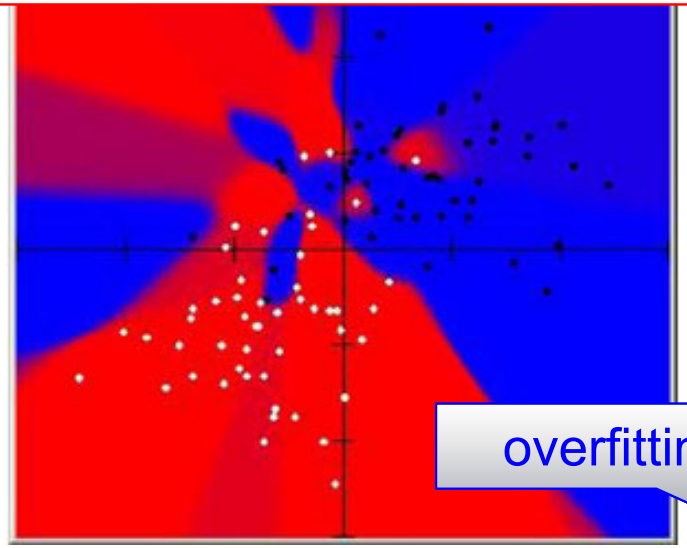


2 inputs, 2 hidden units, 1 output



No overfitting!

2 inputs, 10 hidden units, 1 output



overfitting!

Early Stopping

- Choose the stopping point (stop training before going too far astray) based on the network's performance on **validation data.**
- This technique is also used to prevent overfitting.

Ockham's Razor

- *entia non sunt multiplicanda praeter necessitatem*
- "entities must not be multiplied beyond necessity"
  - William of Ockham
  - (14th century Franciscan friar, logician, theologian)

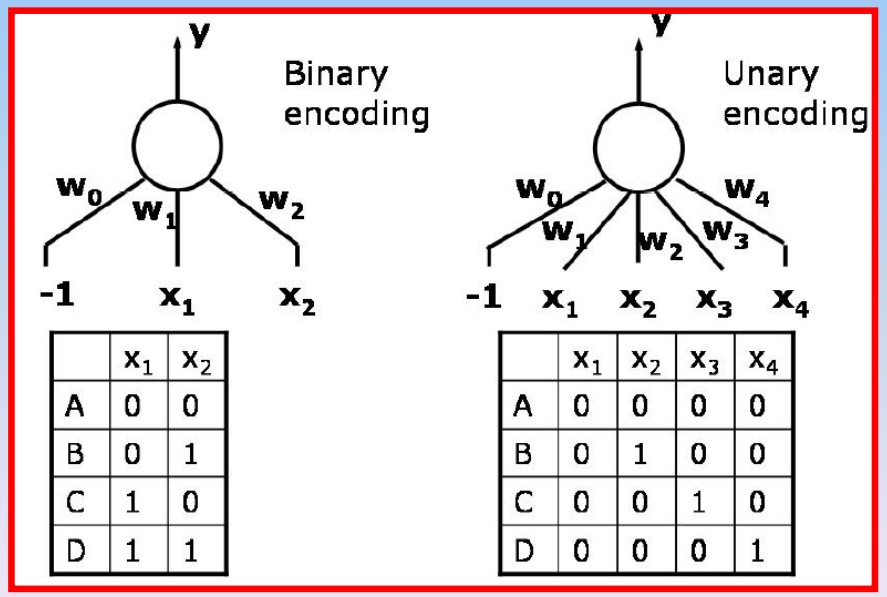
Input & Output Representations

Input Representation

- All the signals in a neural net are **[0, 1]**. Input values should also be **scaled** **to this range** (or approximately so) so as to speed up training.

Input Representation

- All the signals in a neural net are **[0, 1]**. Input values should also be **scaled to this range** (or approximately so) so as to speed up training.
- If the input values are **discrete**, **e.g. {A,B,C,D}** or **{a,g,t,c},** they need to be coded in unary form.



	x_1	x_2
A	0	0
B	0	1
C	1	0
D	1	1

	x_1	x_2	x_3	x_4
A	0	0	0	0
B	0	1	0	0
C	0	0	1	0
D	0	0	0	1

Output Representation

- A neural net with a single sigmoid output is aimed at binary classification.

  Class is **0** if **y** < **0.5** and **1** otherwise.

- **For multi-class problems**
  - Can use one output per class (unary encoding)
  - There may be confusing outputs (two outputs > 0.5 in unary encoding)
  - More sophisticated method is to use special ***softmax units***, which force outputs to sum to 1.

Softmax Units

- For multi-class problems, use softmax units
- Softmax activation function:

net input

$$f_q(k) = z_k^* = \frac{\exp(z_k)}{\sum_{k_1} \exp(z_{k_1})}$$

Actual output

e.g.

net input 1	net input 2	net input 3	net input 4	net input 5		
0.776315846	0.528655919	0.498515491	0.613505773	0.339863291		
Number of Classes: 5						
denominator	8.768					
Softmax Units (Outputs)						
Output 1	Output 2	Output 3	Output 4	Output 5		SUM
0.247883624	0.193504245	0.187758962	0.210639745	0.160213424		1

Softmax Units

- When the inputs to the Softmax unit gets too high, it is possible for the output to become NAN
- The reason is that the computer cannot handle such big numbers; therefore, returning Not A Number result.
- One possible approach to solving this problem is to shift the highest value in the inputs to zero, then compute for the Softmax.

- For example, to compute the softmax of [1, 3, 5] use [1-5, 3-5, 5-5] which is [-4, -2, 0].

Softmax and Cross Entropy Pairing

Softmax activation function

$$f_q(k) = z_k^* = \frac{\exp(z_k)}{\sum_{k_1} \exp(z_{k_1})}$$

Cross-Entropy Penalty Function

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z_{nk}^*}\right).$$

Target output for output node k, for the given pattern n

Actual output of output node k, given pattern n

*Cross-entropy can be used as an error measure when a network's outputs can be thought of as representing independent hypotheses (e.g. each node stands for a different concept), and the node activations can be understood as representing the probability (or confidence) that each hypothesis might be true.

http://www.cse.unsw.edu.au/~billw/cs9444/crossentropy.html

*excerpt from p. 166 of Plunkett and Elman: Exercises in Rethinking Innateness, MIT Press, 1997

Error delta at the output node(s)

$$\delta_k = \underbrace{(t_k - O_k)}_{\frac{dP}{do}} \underbrace{Ok(1 - O_k)}_{\frac{dO_k}{dp_r}}$$

Derivative of the Sigmoid function with respect to its input

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs

Softmax and Cross Entropy Pairing

Cross-Entropy Penalty Function

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z^*_{nk}}\right).$$

Actual output of output node k, for the given pattern n

Derivative

$$\frac{\partial \rho_c}{\partial z_k} = \left(z^*_k - t_k\right)$$

Reference: On the Pairing of the Softmax Activation and Cross-Entropy Penalty Functions and the Derivation of the Softmax Activation Function by R.A. Dunne & N.A. Campbell

$$\frac{\partial \rho_c}{\partial W_{jk}} = \left(z^*_k - t_k\right) * \frac{\partial z_k}{\partial W_{jk}}$$

Z_k is the net input

Weight-update

$$W'_{jk} = W_{jk} - \eta * \left(z^*_k - t_k\right) * \frac{\partial z_k}{\partial W_{jk}}$$

Actual output of output node k, given pattern n

Softmax and Cross Entropy Pairing

Cross-Entropy Penalty Function

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z^*_{nk}}\right).$$

http://neuralnetworksanddeeplearning.com/chap3.html#the_cross-entropy_cost_function

Weight-update

$$W'_{jk} = W_{jk} - \eta * \left(z^*_k - t_k\right) * O_j$$

Actual output of output node k, given pattern n

Target Value

- During training it is impossible for the outputs to reach **0** or **1** (with finite weights)
- Customary to use **0.1** and **0.9** as targets
- But, most **termination criteria**,

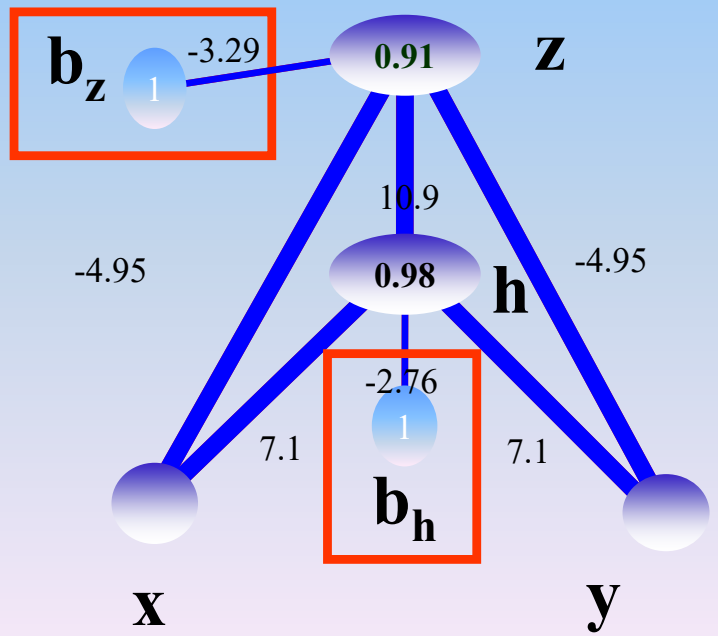
  e.g. **small change in training** or **validation error** will stop training before targets are reached.

Bias Units

Multi-Layer Feed-Forward Neural Network

Why do we need BIAS UNITS?

Apart from improving the speed of learning for some problems (XOR problem), bias units or threshold nodes are required for universal approximation. Without them, the feedforward network always assigns 0 output to 0 input. **Without thresholds, it would be impossible to approximate functions which assign nonzero output to zero input.** Threshold nodes are needed in much the same way that the constant polynomial '1' is required for approximation by polynomials.



Using Different Activation Functions

Using Different Activation Functions

Now that we understand the composition of the weight update formulas at the different layers of the network, it is now easy for us to change the activation function used in the network, as well as the required weight update formula.

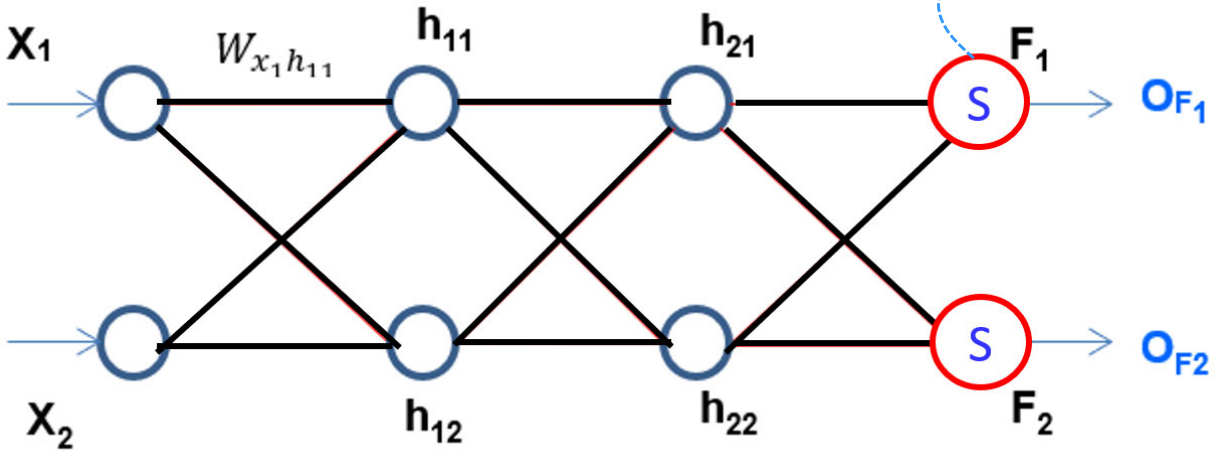
If we opt to change the **activation function** used by a node in the network, the **derivative of the activation function** in the weight update formula needs to be changed as well.

Using the Sigmoid Activation Function

Gradient Ascent

Output nodes using the sigmoid function as activation function.

Uses the Sigmoid Activation function



Error delta at the output node(s)

Derivative of the Sigmoid function with respect to its input

$$\delta_k = \overbrace{(t_k - O_k)}^{\frac{dP}{do}} \overbrace{Ok(1 - O_k)}^{\frac{dO_k}{dp_r}}$$

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

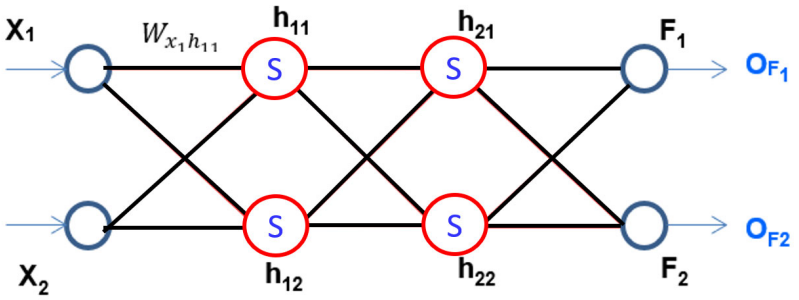
$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs

Gradient Ascent

Hidden nodes using the sigmoid function as activation function.

S - uses the Sigmoid Activation function



Error delta at a hidden node:

error signals back-propagated to node j

Weights of the links carrying the *back-propagated error signals*

$$\delta_j = O_j(1 - O_j) \sum_k \delta_k W_{jk}$$

$\frac{dO_j}{dp_r}$

Derivative of the Sigmoid function with respect to its input

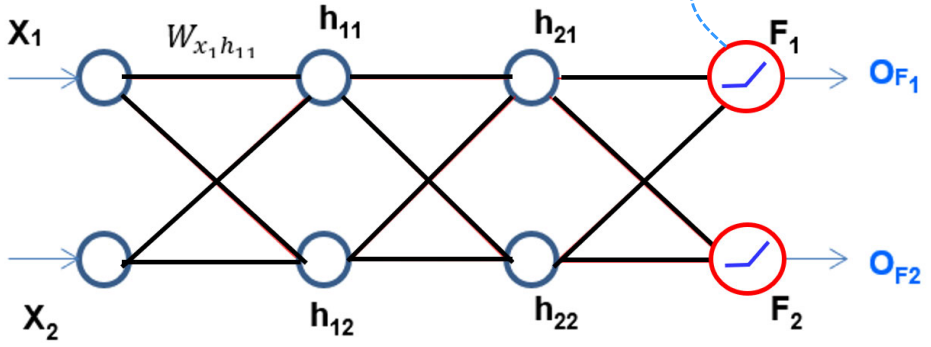
η – *learning rate*
k – *output unit*
J – *hidden node that is directly connected at node k (which is residing at a higher layer)*
δ_k – *error delta at node k*
O_j – *output of unit j*

Using the ReLU Activation Function

Gradient Ascent

Uses the **ReLU** Activation function

Output nodes using the ReLU function as activation function.



Error delta at the output node(s)

- If input to node is greater than zero

Derivative of the **ReLU** function with respect to its input

$$\delta_k = \underbrace{(t_k - O_k)}_{\frac{dP}{do}} \underbrace{(\mathbf{1})}_{\frac{dO_k}{dp_r}}$$

- Else

$$\delta_k = \mathbf{0}$$

$O_k - activation\ function\ of\ output\ node\ k$
$p_r - weighted\ input\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

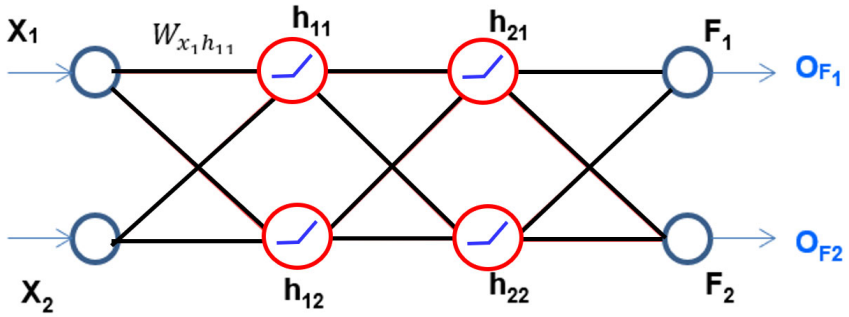
$\frac{dP}{do}$ - derivative of the performance function with respect to the actual output of the network

$\frac{dO_k}{dp_r}$ - derivative of the activation function at output node k, with respect to its weighted inputs

Gradient Ascent

Hidden nodes using the ReLU function as activation function.

- uses the **ReLU** Activation function



Error delta at a hidden node:

- If input to node is greater than zero

Derivative of the **ReLU** function with respect to its input

error signals back-propagated to node j

Weights of the links carrying the *back-propagated error signals*

$$\delta_j = \frac{dO_j}{dp_r}(1)\sum_k \delta_k W_{jk}$$

- Else

$$\delta_j = 0$$

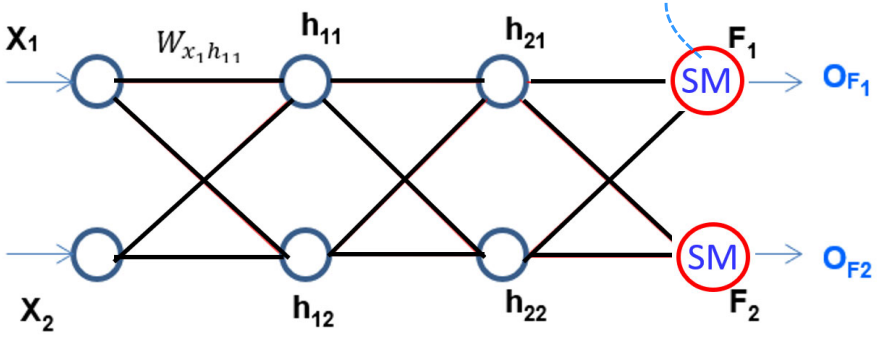
η – *learning rate*
k – *output unit*
J – *hidden node that is directly connected at node k (which is residing at a higher layer)*
δ_k – *error delta at node k*
O_j – *output of unit j*

Using the Softmax Activation Function at the Output Layer

Gradient Ascent

Uses the **Softmax** Activation function

Output nodes using the Softmax function as activation function.



Error delta at the output node(s)

- The error delta is computed based on the derivative of the **cross-entropy penalty function** and the **derivative of the softmax function**. Interestingly, using the pairing of the two, the **error delta** is significantly simplified.

$$\delta_k = (t_k - O_k)$$

$O_k - activation\ function\ of\ output\ node\ k$
$t_k - target\ (desired)\ output\ of\ output\ node\ k$

Derivative of Activation Functions

https://github.com/tensorflow/playground/blob/master/nn.ts

```
/** Built-in activation functions */
export class Activations {
  public static TANH: ActivationFunction = {
    output: x => (<any>Math).tanh(x),
    der: x => {
      let output = Activations.TANH.output(x);
      return 1 - output * output;
    }
  };
  public static RELU: ActivationFunction = {
    output: x => Math.max(0, x),
    der: x => x <= 0 ? 0 : 1
  };
  public static SIGMOID: ActivationFunction = {
    output: x => 1 / (1 + Math.exp(-x)),
    der: x => {
      let output = Activations.SIGMOID.output(x);
      return output * (1 - output);
    }
  };
  public static LINEAR: ActivationFunction = {
    output: x => x,
    der: x => 1
  };
}
```

Gradient Descent

Performance Function

P = (1/2) * ((desired_output – actual_output)^2)

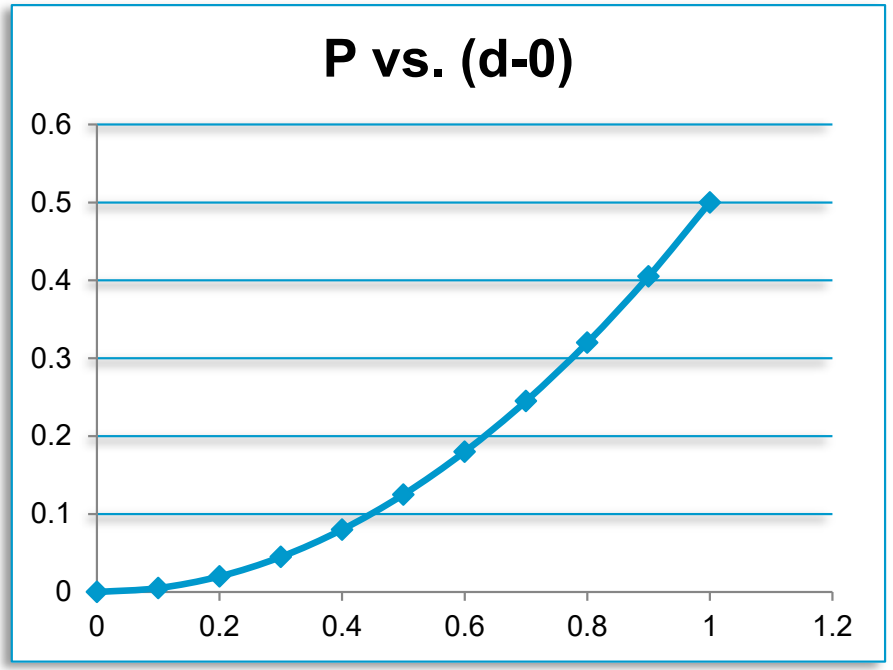
desired output	actual output	difference	Performance
1	1	0	0
1	0.9	0.1	0.005
1	0.8	0.2	0.02
1	0.7	0.3	0.045
1	0.6	0.4	0.08
1	0.5	0.5	0.125
1	0.4	0.6	0.18
1	0.3	0.7	0.245
1	0.2	0.8	0.32
1	0.1	0.9	0.405
1	0	1	0.5

actual output - desired output

$$\frac{dP}{do} = (o - d)$$

$$\Delta w \ \alpha \ \frac{\partial \mathrm{P}}{\partial w}$$

$$w = w - \eta \frac{\partial \mathrm{P}}{\partial w}$$



When training the network, we would like to minimise P

Gradient Descent

Summary of Weight Update Formulas

$$\frac{dP}{do} = (o - d)$$

$$\Delta w \ \alpha \ \frac{\partial P}{\partial w}$$

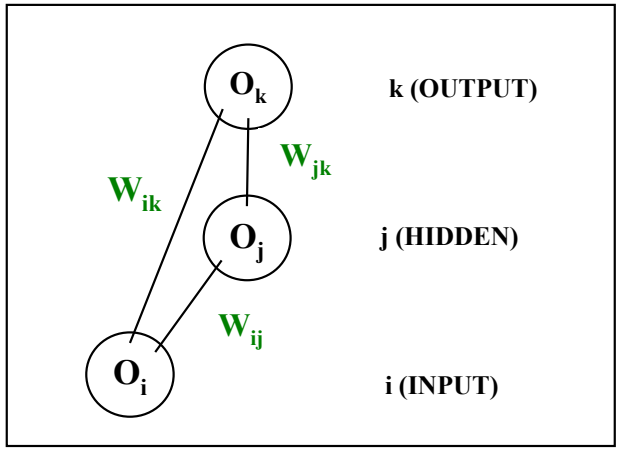
$$w = w - \eta \frac{\partial P}{\partial w}$$

To adjust weights leading into an output unit, W_{jk}:

$$\delta_k = (O_k - t_k) Ok(1 - O_k)$$
$$W_{jk} = W_{jk} - \eta \delta_k O_j$$

$$W_{ik} = W_{ik} - \eta \delta_k O_i$$

η – learning_rate
δ_k – error_signal
O_j – output_of_unit_j



To adjust weights leading into a hidden unit, W_{ij}:

$$\delta_j = Oj(1 - O_j) \sum_k \delta_k W_{jk}$$

$$W_{ij} = W_{ij} - \eta \delta_j O_i$$

η – learning_rate
δ_j – error_signal at unit j
O_i – output_of_unit_i

Hyperparameter

- A **hyperparameter** is any number used in the neural network which cannot be learned by the network. An example would be the **learning rate**, **number of neurons in the hidden layer** or **number of hidden layers, momentum rate**.

- This means that learning cannot adjust hyperparameters; They have to be adjusted manually by training the network on training set (typically 80% of data set) and testing it on the **validation set**. We do this process of tweaking hyperparameters, training and then testing until we get the desired classification accuracy.

Shuffling the Examples

- Shuffle the training set so that successive examples never (or rarely) belong to the same class.

Example:

Syntax: Class, 16 input features

G,4,9,5,6,6,8,5,4,3,7,5,11,6,8,5,11
X,5,11,8,8,4,8,8,1,8,10,5,7,3,8,4,8
X,3,9,5,7,4,8,7,3,8,5,6,8,2,8,6,7
C,2,3,3,1,1,5,9,4,6,11,9,11,1,9,2,8
C,7,10,5,5,2,6,8,6,8,11,7,11,2,8,5,9
E,3,7,4,5,4,7,7,5,8,8,8,9,3,9,6,9
E,6,9,4,4,2,7,7,4,7,10,6,10,1,9,7,9
E,3,4,3,6,2,3,8,6,10,7,6,15,0,8,7,8
...
and so on.

This data set needs some shuffling!

Measure of Performance

Sum of Squared Error

- This error is minimised during training.

$$E = \frac{1}{2}\sum_{i=1}^{n}\sum_{k=1}^{m}(T_{ik} - O_{ik})^2$$

where
T_{ik} is the target output of node k for input pattern **i**;
O_{ik} is the actual output of node k for input pattern **i**;
m is the total number of output units;
n is the number of patterns;
i is the input pattern

Mean squared error

$$MSE = \frac{\sum_{i=1}^{n}\sum_{j=1}^{m}(o_{ij} - t_{ij})^2}{n}$$

- **n** is the number of patterns
- **m** is the number of components in the output vector
- **o** is the output of a single neuron j
- **t** is the target for the single neuron j
- **i** denotes the input pattern

Root mean squared error

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}\sum_{j=1}^{m}(o_{ij} - t_{ij})^2}{n}}$$

- **n** is the number of patterns
- **m** is the number of components in the output vector
- **o** is the output of a single neuron j
- **t** is the target for the single neuron j
- **i** denotes the input pattern

- This measure corresponds to the **Euclidean norm or L_2 norm,** denoted by $\|\cdot\|_2$.
- RMSE is generally the preferred performance measure for regression tasks

Mean absolute error

$$MAE = \frac{\sum_{i=1}^{n}\sum_{j=1}^{m}|o_{ij} - t_{ij}|}{n}$$

- **n** is the number of patterns
- **m** is the number of components in the output vector
- **o** is the output of a single neuron j
- **t** is the target for the single neuron j
- **i** denotes the input pattern

- for regression tasks with many outliers, you may consider using the Mean Absolute Error
- This measure corresponds to the **Manhattan norm** or **L_1 norm**, denoted by $\|\cdot\|_1$.

Cross-Entropy Error

$$\rho_c = \sum_{n=1}^{N} \sum_{k=1}^{q} t_{nk} \log\left(\frac{t_{nk}}{z^*_{nk}}\right)$$

- **N** is the number of patterns
- **q** is the number of output neurons
- $\mathbf{Z^*_{nk}}$ is the actual output of neuron k, for pattern n
- $\mathbf{t_{nk}}$ is the target output for neuron k, using pattern n

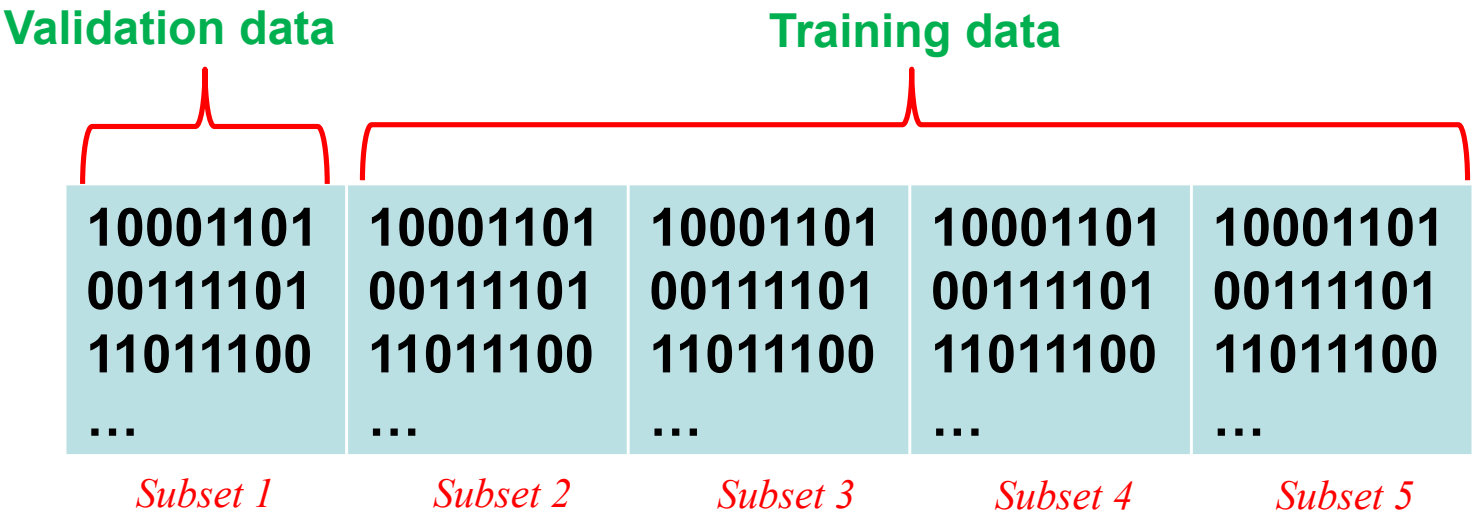
- Generally for multiple classification tasks

Reference: On the Pairing of the Softmax Activation and Cross-Entropy Penalty Functions and the Derivation of the Softmax Activation Function by R.A. Dunne & N.A. Campbell

Cross Validation

- Split data set (randomly) into **k equal size subsets**.

e.g. (k=5)



- Of the k subsets, a single subset is retained as the **validation data** for testing the model, and the remaining k − 1 subsets are used as **training data**.
- Repeat k times
- Average the results on validation data

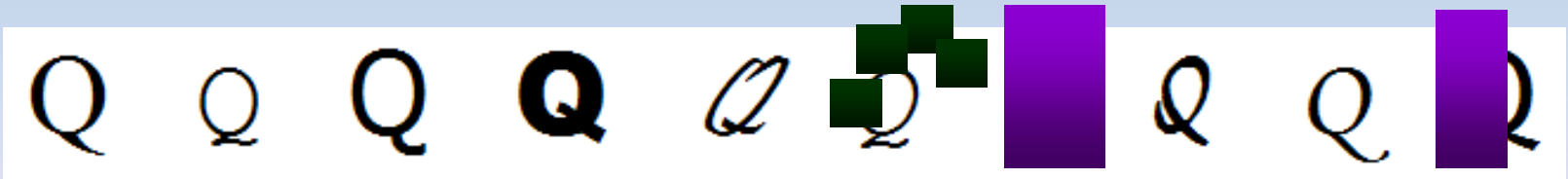
Use if data is scarce.

Training the Network

Other Tips for Training (from the Welstead book)

In a system to be used for a **real-world application**, such as character recognition, you would want the network to be able to handle **not only pixel noise**, but also **size variance** (slightly smaller letters), some **rotation**, **reflections**, as well as some **variations in font style**.

To produce such a robust network classifier, you would need to add representative samples to the training set. For example, samples of rotated versions of a character, reflected images, etc. could be added to the training set.



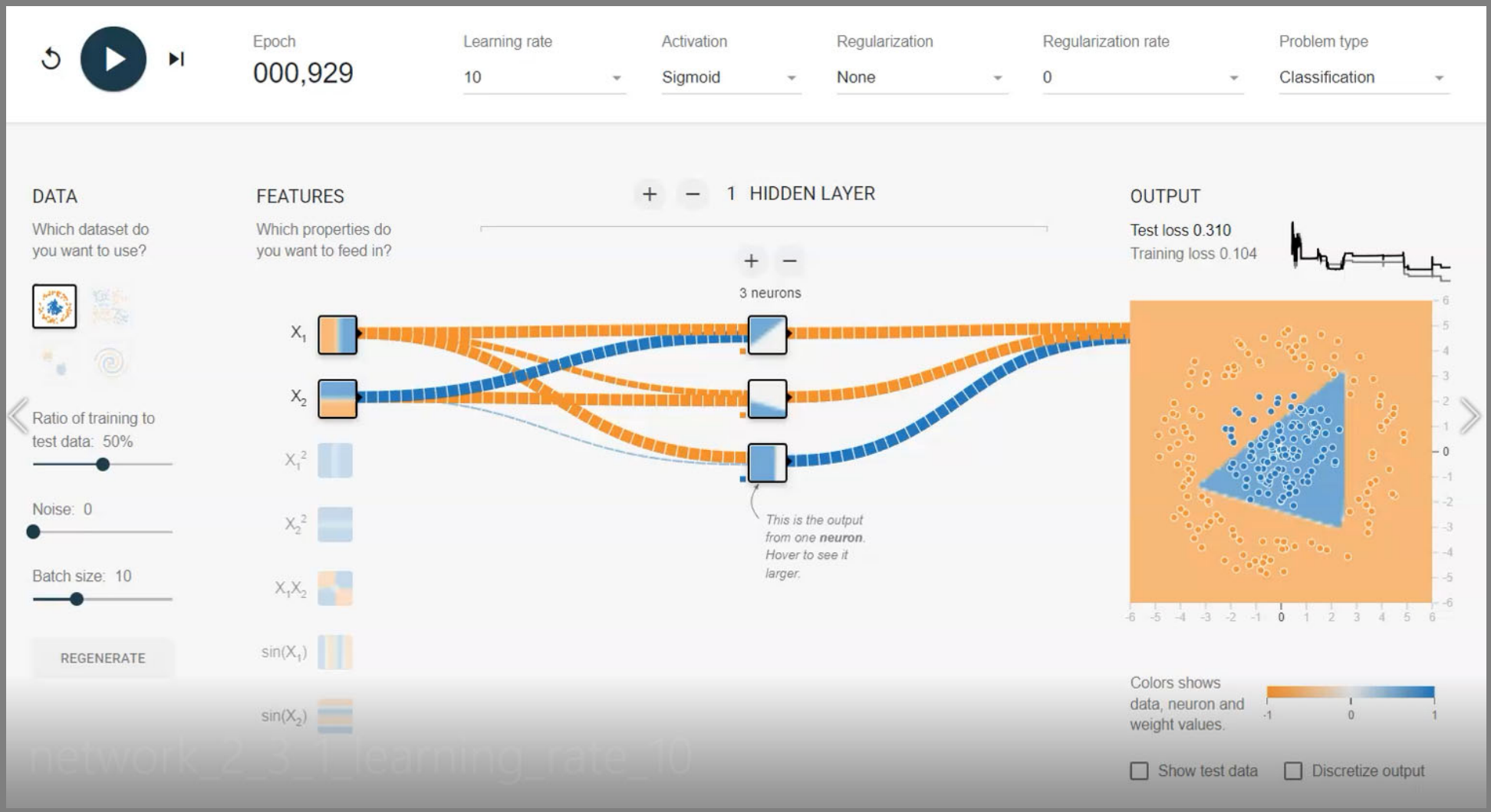
Experimenting with Neural Nets using Tensorflow playground

https://playground.tensorflow.org

Concentric Circles Data Set

Too high learning rate

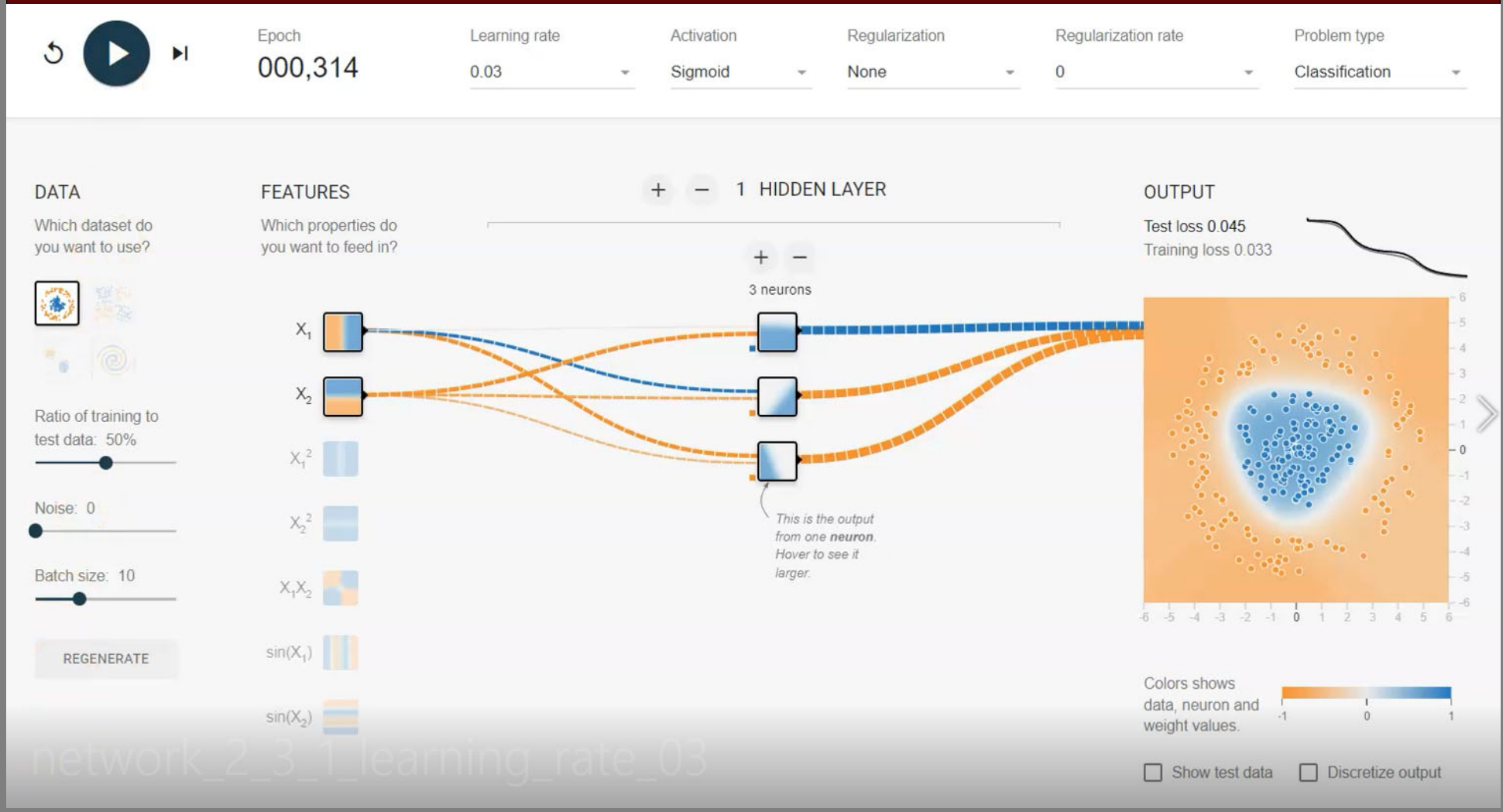
CIRCLES DATASET



network_2_3_1_learning_rate_10

Adjusted learning rate

CIRCLES DATASET

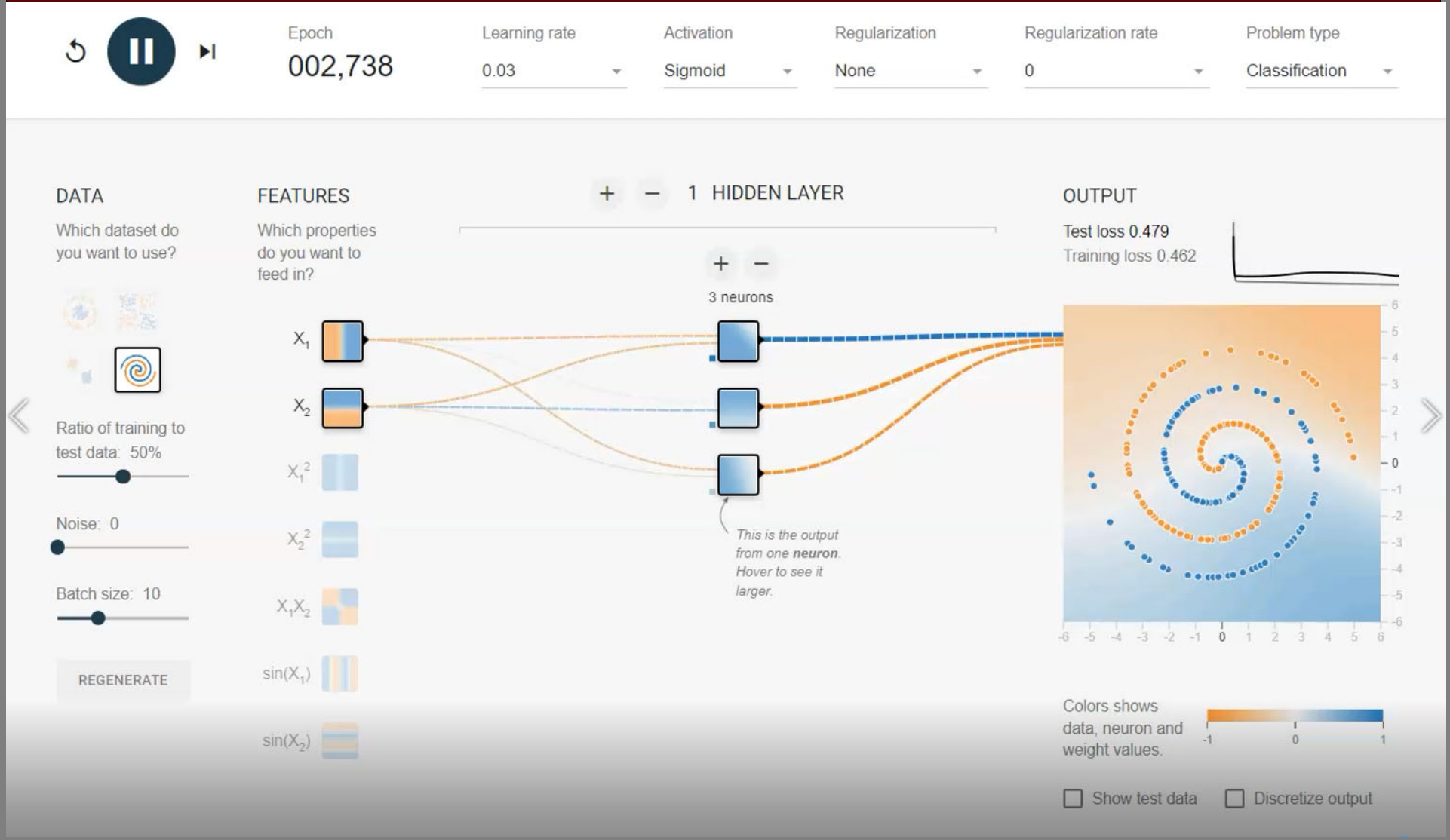


Spiral Data Set

A more challenging classification problem

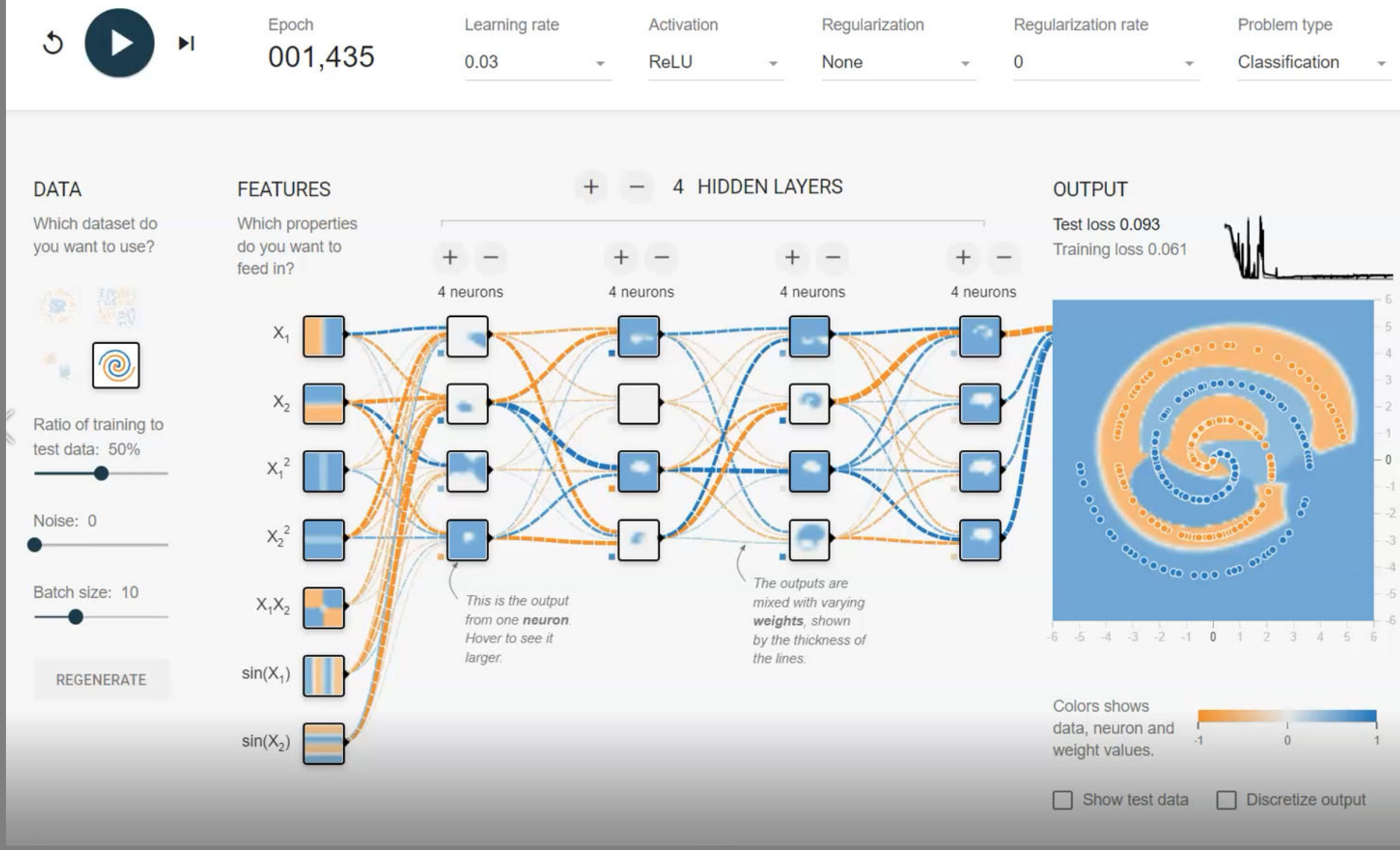
Failed attempt

SPIRAL DATASET



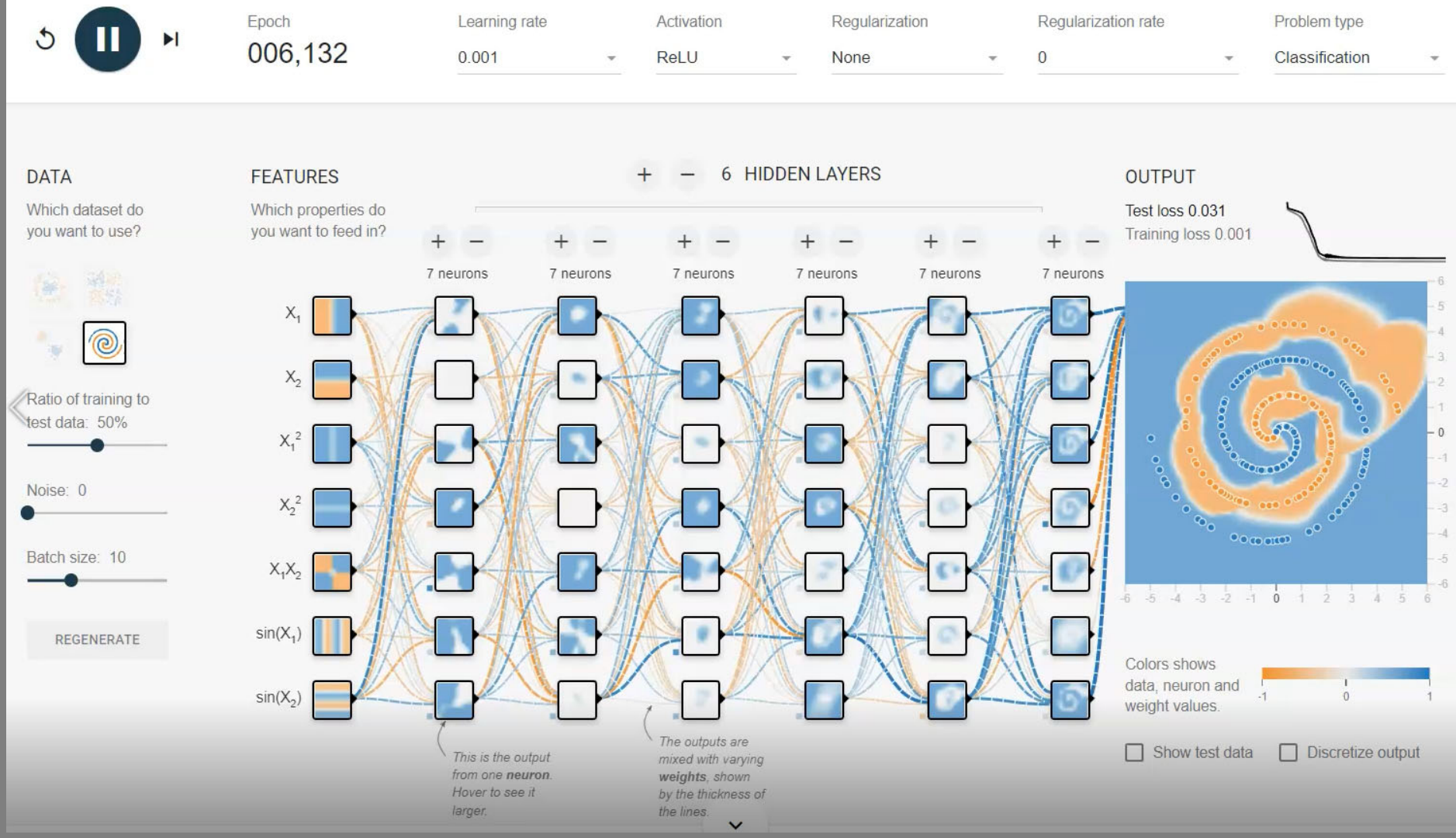
Feature Engineering

SPIRAL DATASET



Feature Engineering + Deep Network

SPIRAL DATASET



Deep Learning

SPIRAL DATASET

Epoch 001,078 | Learning rate 0.03 | Activation ReLU | Regularization None | Regularization rate 0 | Problem type Classification

DATA

Which dataset do you want to use?

Ratio of training to test data: 50%

Noise: 0

Batch size: 10

REGENERATE

FEATURES

Which properties do you want to feed in?

X_1, X_2, X_1^2, X_2^2, X_1X_2, $\sin(X_1)$, $\sin(X_2)$

6 HIDDEN LAYERS

7 neurons | 7 neurons | 7 neurons | 7 neurons | 7 neurons | 7 neurons

*This is the output from one **neuron**. Hover to see it larger.*

*The outputs are mixed with varying **weights**, shown by the thickness of the lines.*

OUTPUT

Test loss 0.029

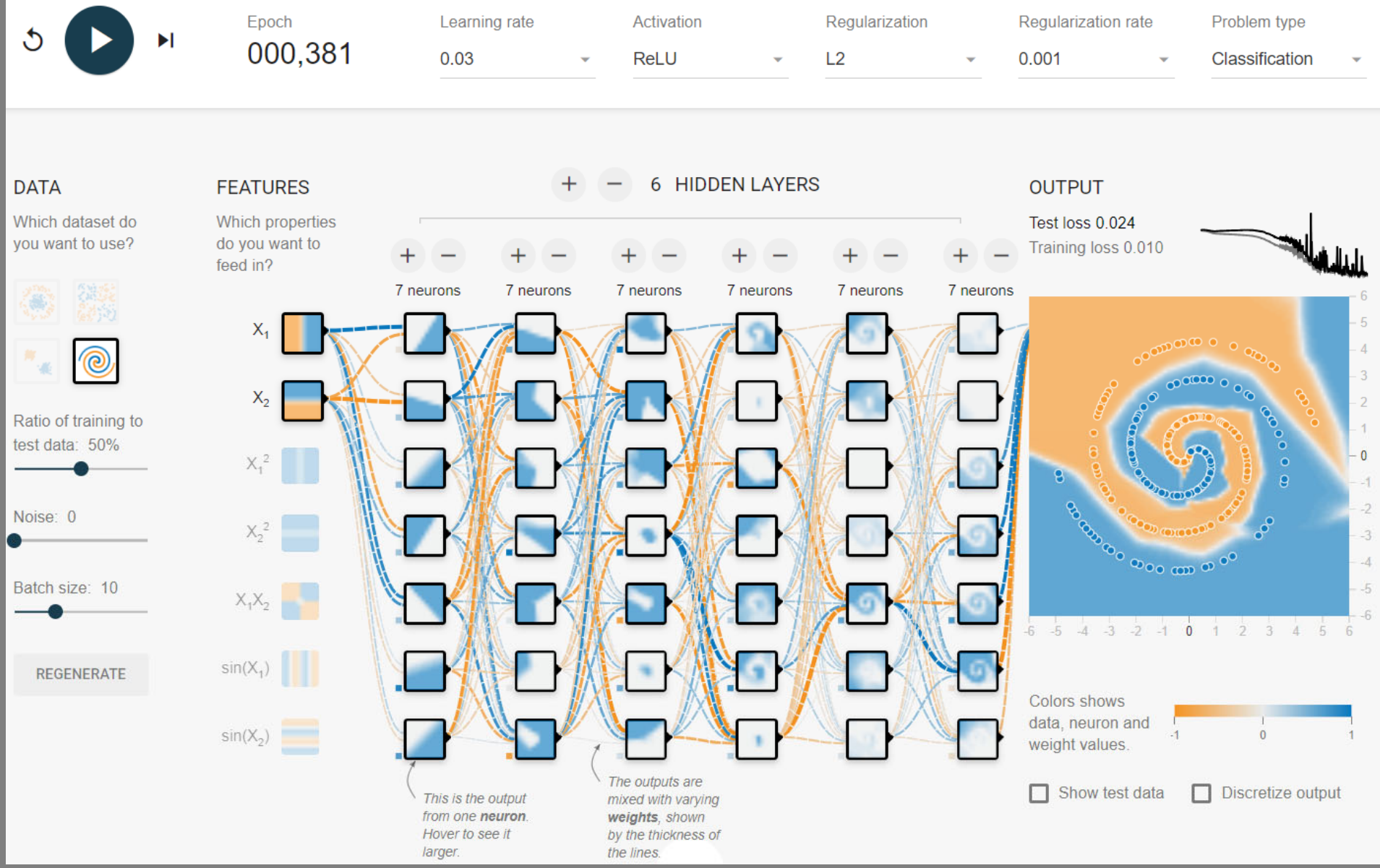
Training loss 0.005

Colors shows data, neuron and weight values. -1 0 1

Show test data | Discretize output

Deep Learning with Regularization

SPIRAL DATASET



~The End.~

Tensor Flow Codes

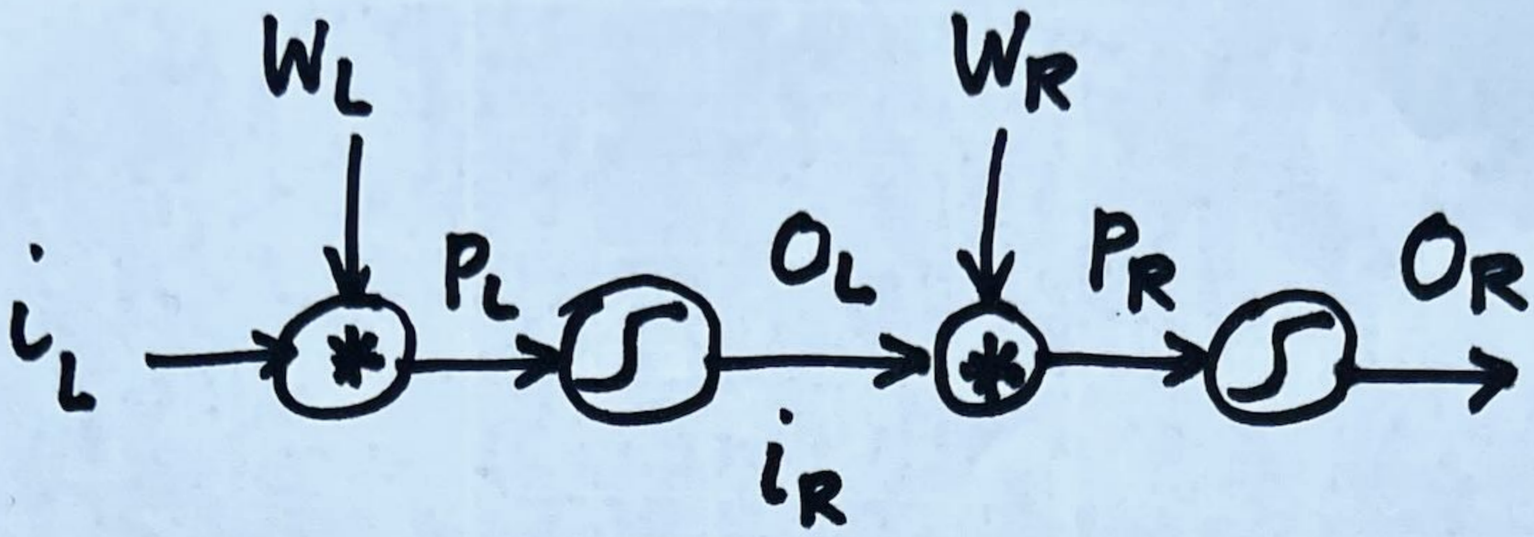
https://github.com/tensorflow/playground/blob/master/nn.ts

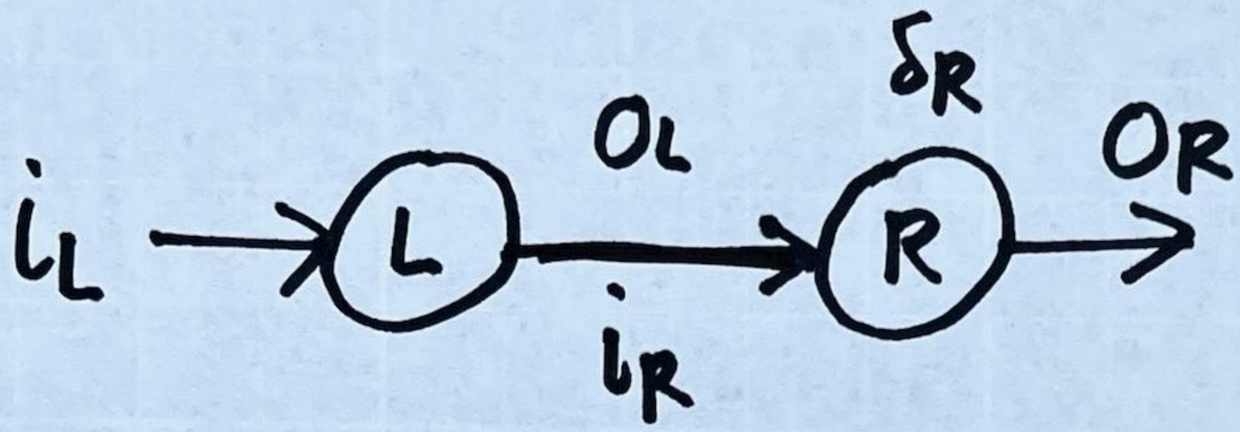
```
/** Built-in activation functions */
export class Activations {
  public static TANH: ActivationFunction = {
    output: x => (<any>Math).tanh(x),
    der: x => {
      let output = Activations.TANH.output(x);
      return 1 - output * output;
    }
  };
  public static RELU: ActivationFunction = {
    output: x => Math.max(0, x),
    der: x => x <= 0 ? 0 : 1
  };
  public static SIGMOID: ActivationFunction = {
    output: x => 1 / (1 + Math.exp(-x)),
    der: x => {
      let output = Activations.SIGMOID.output(x);
      return output * (1 - output);
    }
  };
  public static LINEAR: ActivationFunction = {
    output: x => x,
    der: x => 1
  };
}
```

References

1. MIT OpenCourseWare
2. www.cim.mcgill.ca/~jer/courses/ai/readings/backprop.pdf
3. Neural Network and Fuzzy Logic Applications in C/C++ (Wiley Professional Computing) by Stephen Welstead
4. Introduction to Deep Learning by Sandro Skansi
5. Hands-On Machine Learning with Scikit-Learn and Tensorflow: Concepts, Tools, and Techniques to Build Intelligent Systems
6. http://www.cse.unsw.edu.au/~billw/cs9444/crossentropy.html
7. On the Pairing of the Softmax Activation and Cross-Entropy Penalty Functions and the Derivation of the Softmax Activation Function by R.A. Dunne & N.A. Campbell

Derivation of Backprop Formulas





$$y = \frac{1}{1+e^{-x}} \quad \text{Sigmoid}$$

$$\frac{dy}{dx} = \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right)$$

$$= \frac{d\left((1+e^{-x})^{-1}\right)}{dx}$$

let $u = 1+e^{-x}$

$u^n = (1+e^{-x})^{-1}$

$$= (-1) * (1+e^{-x})^{-1-1} \frac{d(1+e^{-x})}{dx}$$

$$= (-1) * (1+e^{-x})^{-2} * \left(\frac{d(1)}{dx}^{0} + \frac{d(e^{-x})}{dx}\right)$$

$$= (-1) * (1+e^{-x})^{-2} * \left(e^{-x} * \frac{d(-x)}{dx}\right)$$

$$= (-1) * (1+e^{-x})^{-2} * (e^{-x} * (-1))$$

$$= \frac{e^{-x}}{(1+e^{-x})^2} = \frac{e^{-x}}{(1+e^{-x})(1+e^{-x})}$$

$$\frac{d\,u^n}{dx} = n\,u^{n-1}\frac{du}{dx}$$

$$\frac{d\,e^u}{dx} = e^u \frac{du}{dx}$$

$$= \frac{1 + e^{-x} - 1}{(1+e^{-x})(1+e^{-x})}$$

$$= \left(\frac{1}{1+e^{-x}}\right) * \left(\frac{1+e^{-x}}{1+e^{-x}} - \frac{1}{1+e^{-x}}\right)$$

$$\boxed{\frac{dy}{dx} = y * (1 - y)}$$

Derivative of P with respect to the actual output of the network

let d - desired output

o - actual output

$$P = \left(-\frac{1}{2}\right)(d-o)^2$$

$$\frac{dP}{do}$$

let $u = (d-o)$

$$\frac{dP}{do} = \left(-\frac{1}{2}\right)(2)(d-o)^{2-1}\frac{d(d-o)}{do}$$

$$= (-1)(d-o)(-1)$$

$$\boxed{\frac{dP}{do} = (d-o)}$$

$$\frac{dU^n}{dx} = nU^{n-1}\frac{dU}{dx}$$

$$\Delta W_R \propto \frac{\partial P}{\partial W_R}$$

$$\frac{\partial P}{\partial W_R} = \frac{\partial P}{\partial O_R} \times \frac{\partial O_R}{\partial W_R}$$

$$(d - O_R) \times \frac{\partial O_R}{\partial P_R} \times \frac{\partial P_R}{\partial W_R}$$

$$O_R(1 - O_R)$$

$$\frac{\partial (W_R \times O_L)}{\partial W_R}$$

$$\frac{\partial P}{\partial W_R} = [d - O_R] \times [O_R(1 - O_R)] \times [O_L]$$

$$\frac{\partial P}{\partial W_R} = \boxed{[d - O_R] \times [O_R(1 - O_R)] \times i_R}$$

$$\Delta W_L \propto \frac{\partial P}{\partial W_L}$$

$$\frac{\partial P}{\partial W_L} = \frac{\partial P}{\partial O_R} \frac{\partial O_R}{\partial W_L}$$

$$(d - O_R) \times \frac{\partial O_R}{\partial P_R} \times \frac{\partial P_R}{\partial W_L}$$

$$O_R(1 - O_R)$$

$$\frac{\partial P_R}{\partial O_L} \times \frac{\partial O_L}{\partial W_L}$$

$$\frac{\partial (W_R \times O_L)}{\partial O_L} \times \frac{\partial O_L}{\partial P_L} \times \frac{\partial P_L}{\partial W_L}$$

$$\frac{\partial (W_L \times i_L)}{\partial W_L}$$

$$= [d - O_R] \times [O_R(1 - O_R)] \times [W_R] \times [O_L(1 - O_L)] \times [i_L]$$

$$\frac{\partial P}{\partial W_R} = [d - O_R] \times [O_R(1 - O_R)] \times [i_R]$$

$$\frac{\partial P}{\partial W_L} = [d - O_R] \times [O_R(1 - O_R)] \times [W_R] \times [O_L(1 - O_L)] \times [i_L]$$

To simplify,

Let $\delta_R = (d - O_R) \times [O_R(1 - O_R)]$

$$\delta_L = \delta_R \times W_R \times [O_L(1 - O_L)]$$

$$\frac{\partial P}{\partial W_R} = \delta_R \times i_R$$

$$\frac{\partial P}{\partial W_L} = \delta_L \times i_L$$

$$W_R = W_R + \eta \frac{\partial P}{\partial W_R}$$

$$W_L = W_L + \eta \frac{\partial P}{\partial W_L}$$

Final output layer (output neurons)

$$W_F = W_F + n\frac{\partial P}{\partial W_F}$$

For all other neurons in the other layer

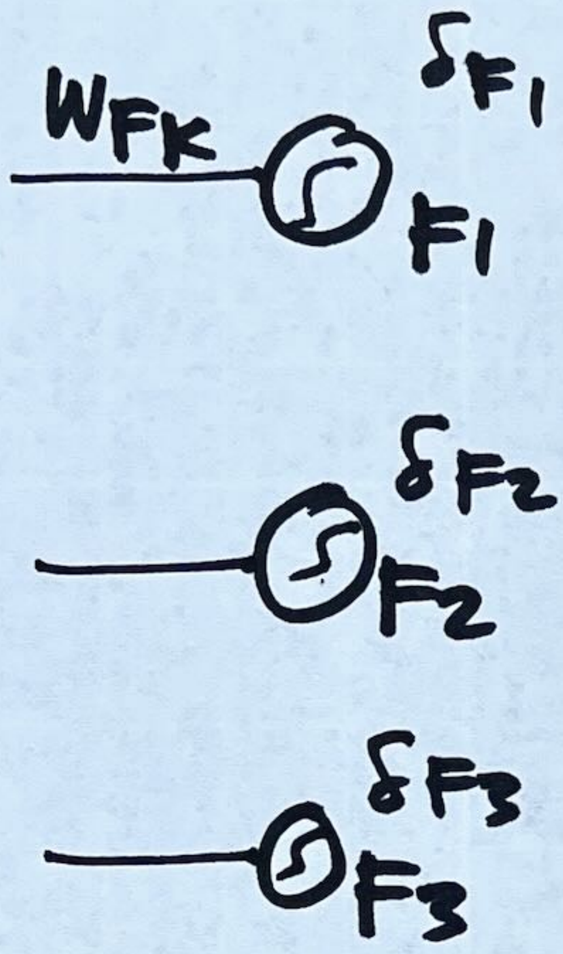
$$W_L = W_L + n\delta_L i_L$$

$$\delta_L = \delta_R \times W_R \times o_L(1-o_L)$$

Generalisation

For networks with multiple neurons par layer

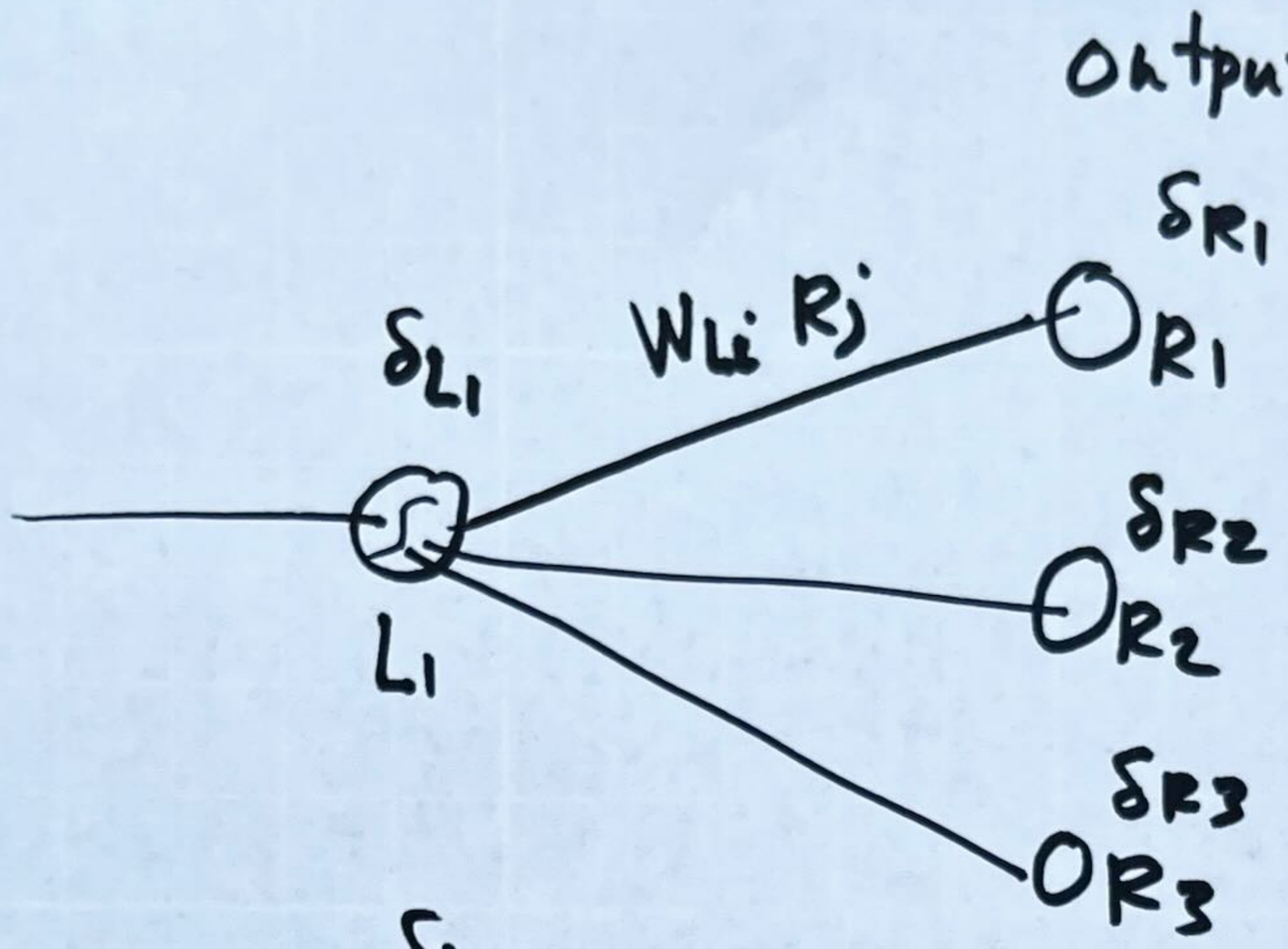
Final layer neurons :



$$W_{Fk} = W_{FK} + n \delta_{Fk} i_{Fk}$$

$$\delta_{Fk} = (d_k - o_{Fk}) \times [o_{Fk}(1 - o_{Fk})]$$

Other neurons (not in the output layer)



(right-hand side)

no. of neurons connected to Li

$$\delta_{Li} = \left[\sum_{j=1}^{m} \left(\delta_{Rj} \times W_{Li\,Rj} \right) \right] \times O_{Li}(1 - O_{Li})$$

Tutorial: Neural Networks

Multi-output Neural Network, Backpropagation Learning

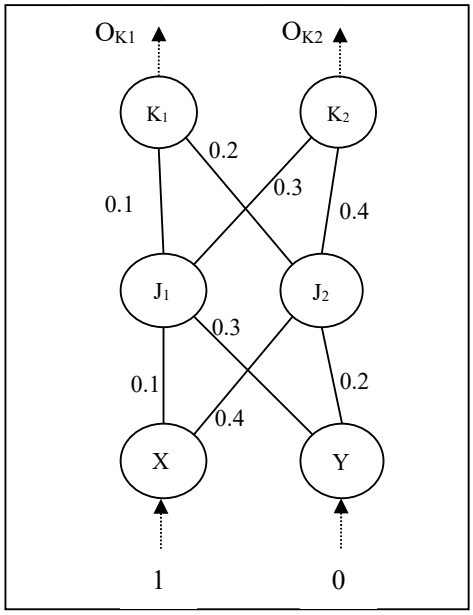


Fig.1 Initial Network Configuration

Parameters for Backpropagation learning:
Learning rate η = 2
No bias nodes are used.
The activation function used is the sigmoid function.

Training set

Pattern#	Input: **x**	Input: **y**	Target output: T_{K1}	Target output: T_{K2}
1	1	1	1	0
2	1	0	0	1
3	0	1	1	1
…	…	…	…	…

Network Evaluation

1. Given the initial network configuration in Fig. 1 and the parameters for Backpropagation learning, put one of the training patterns to be learned (training pattern #2) on the input units, and then calculate the actual outputs of the network (i.e. O_{K1} and O_{K2}).

Minimisation of Error

2. Based on the results of your calculations in Item#1, calculate the error signals at the output nodes (i.e. δ_{k1} and δ_{k2}).

3. Backpropagate δ_{k1} and δ_{k2}, then calculate the weight updates for W_{j1k1}, W_{j1k2} W_{j2k1} and W_{j2k2}

4. Calculate the error signal at the hidden nodes j1 and j2 (i.e. δ_{j1} and δ_{j2}).

5. Update the weights W_{xj1}, W_{xj2}, W_{yj1} and W_{yj2}

n.h.reyes@massey.ac.nz

TUTORIAL : Multi output Network

1.

$$net_{j1} =$$

$$O_{j1} = (0.1 + 0)$$

$$f(net_{j1}) = \frac{1}{1+e^{-0.1}} = 0.52$$

$$O_{j2} = net_{j2} = 0.4$$

$$f(net_{j2}) = \frac{1}{1+e^{-0.4}} = 0.6$$

$$O_{k1} = net_{k1} = (0.1)(0.52) + (0.2)(0.6) = 0.17$$

$$f(net_{k1}) = \frac{1}{1+e^{-0.17}} = \boxed{0.54}$$

$$O_{k2} = net_{k2} = (0.3)(0.52) + (0.4)(0.6)$$

$$= \boxed{0.4}$$

$$f(net_{k2}) = \frac{1}{1+e^{-0.4}} = \boxed{0.6}$$

2. $\delta_{k_1} = (t_{k_1} - o_{k_1})\, o_{k_1}(1 - o_{k_1})$

$= (0 - 0.54)\ 0.54\ (1 - 0.54) = \boxed{-0.13}$

$\delta_{k_2} = (1 - 0.6)\ 0.6\ (1 - 0.6) = \boxed{0.1}$

---

3 $W_{j_1 k_1} = W_{j_1 k_1} + n\, \delta_{k_1}\, o_{j_1}$

$= 0.1 + (2)(-0.13)(0.52) = \boxed{-0.04}$

$W_{j_2 k_1} = W_{j_2 k_1} + n\, \delta_{k_1}\, o_{j_2}$

$= 0.2 + (2)(-0.13)(0.6) = \boxed{0.04}$

$W_{j_1 k_2} = W_{j_1 k_2} + n\, \delta_{k_2}\, o_{j_1}$

$= 0.3 + (2)(0.1)(0.52) = \boxed{0.4}$

$W_{j_2 k_2} = W_{j_2 k_2} + n\, \delta_{k_2}\, o_{j_2}$

$= 0.4 + (2)(0.1)(0.6) = \boxed{0.52}$

4. $\delta_{j_1} = O_{j_1}(1 - O_{j_1}) * \sum_{i=1}^{m} \delta_{k_i} W_{j_1 k_i}$

$= 0.52(1-0.52) * [(-0.13)(-0.09) + (0.1)(0.4)]$

$= \boxed{0.01}$

$\delta_{j_2} = O_{j_2}(1 - O_{j_2}) * [(-0.13)(0.09) + (0.1)(-0.52)]$

$O_{j_2} \rightarrow 0.6$

$= \boxed{0.01}$

5. $W_{xj_1} = W_{xj_1} + (\eta)(\delta_{j_1})(O_x)$

$= 0.1 + (2)(0.01)(1)$

$= \boxed{0.12}$

$W_{yj_1} = W_{yj_1} + \eta \delta_{j_1} O_y$

$= \boxed{0.3}$

$W_{xj_2} = W_{xj_2} + \eta \delta_{j_2} O_x$

$= 0.4 + (2)(0.01)(1)$

$= \boxed{0.42}$

$W_{yj_2} = W_{yj_2} + \eta \delta_{j_2} O_y$

$= 0.2 + 0$

$= \boxed{0.2}$

5

159.302 CSPs

Introduction

Constraint Satisfaction Problems

n.h.reyes@massey.ac.nz

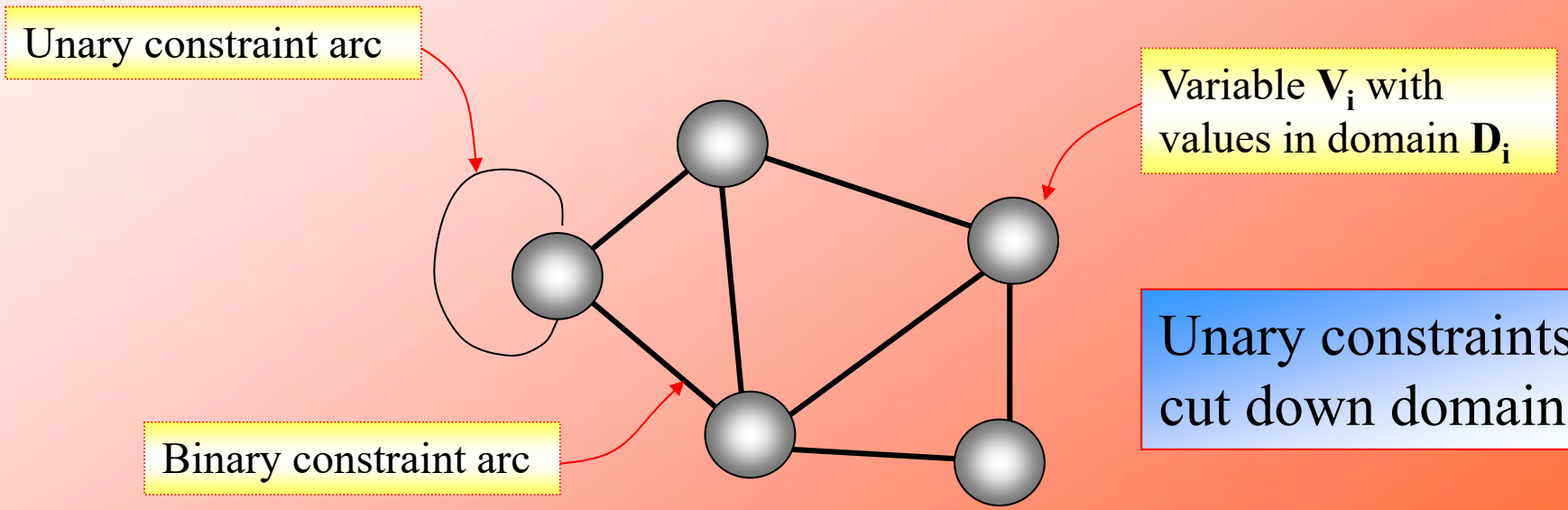
Reference of contents: ***MIT OpenCourseWare***

CSP

General class of problems: BINARY CSP

Application areas of CSPs:

- scheduling tasks, robot planning tasks, puzzles, molecular structures, sensor interpretation tasks, etc.



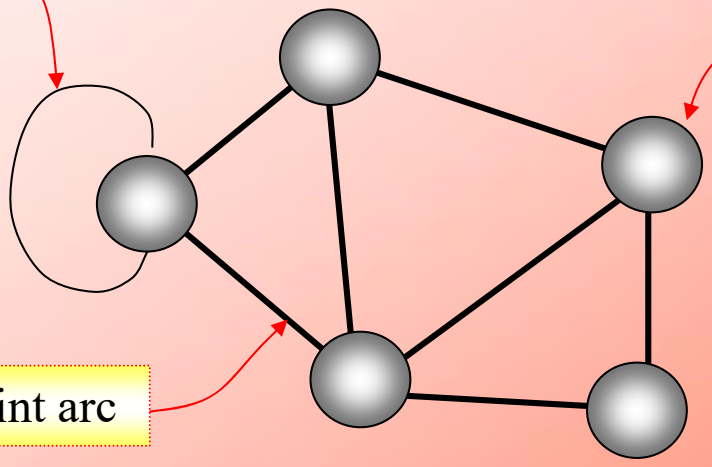
Unary constraints just cut down domains.

This diagram is called a **constraint graph**.

1. **Variables**
2. **Domain of variables**
3. **constraints**

CSP

General class of problems: BINARY CSP



Unary constraints just cut down domains.

This diagram is called a **constraint graph**.

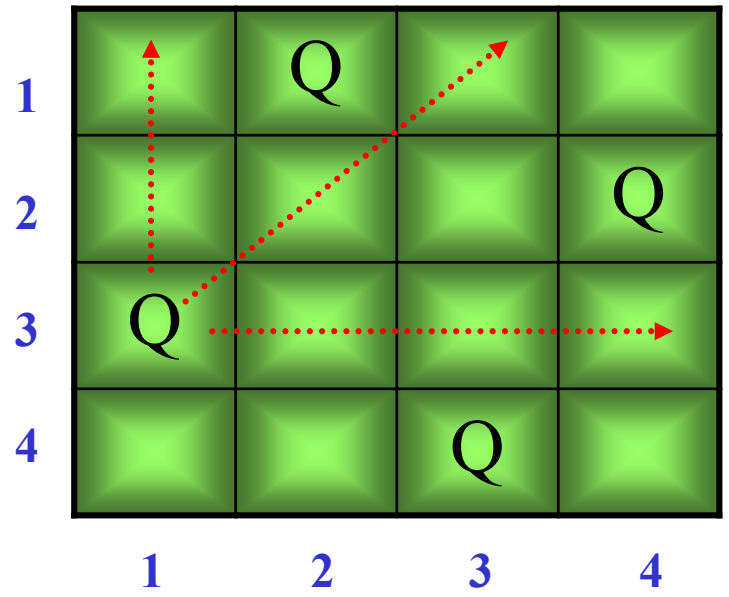
Basic problem:

- Find a $d_j \in D_i$ for each V_i s.t. all constraints are satisfied (finding consistent labeling for variables)

N-Queens as CSP

Classic "benchmark" problem

Place N queens on an N × N chessboard so that none can attack the other.



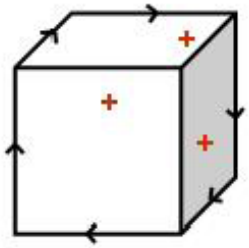
Variables are board positions in N × N chessboard

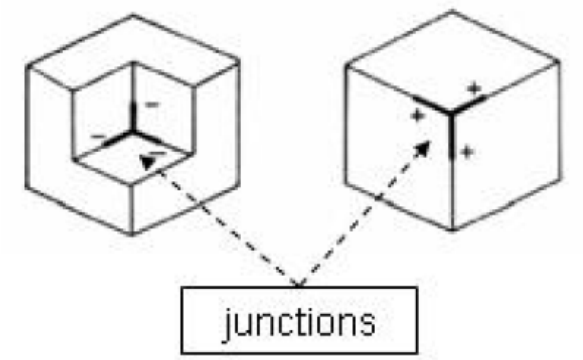
Domains Queen or blank

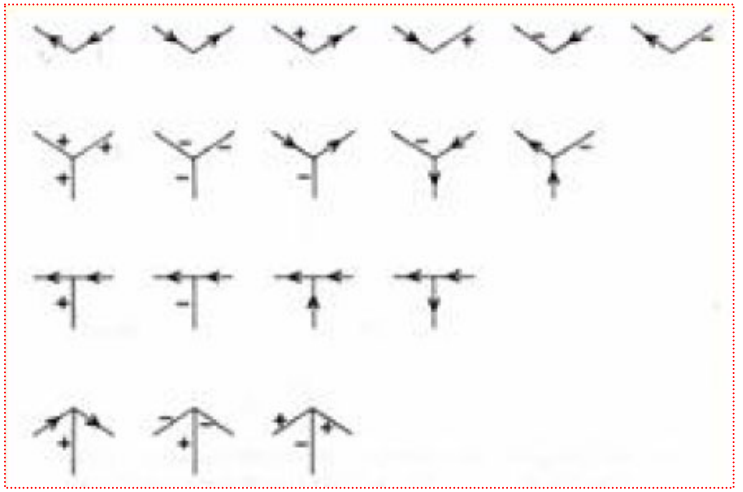
Constraints Two positions on a line (vertical, horizontal, diagonal) cannot both be Queen

Line labelings as CSP

Labeling lines in drawing as convex (+), concave (-), or boundary (>).







All legal junction labels for four junction types.

Variables are line junctions

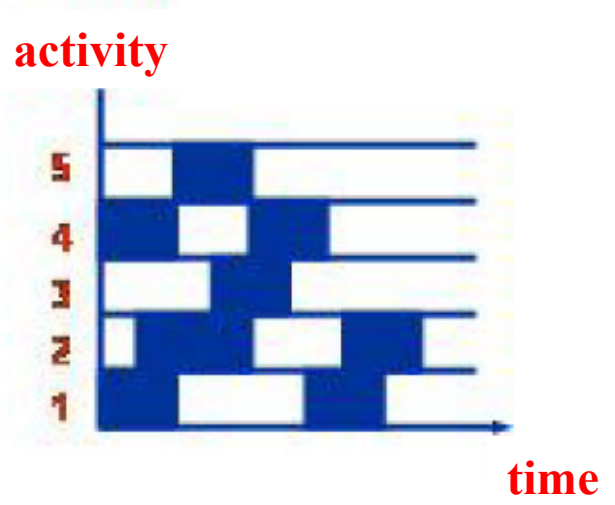
Domains are set of legal labels for that junction type

Constraints shared lines between adjacent junctions must have the same label.

CSP

Scheduling activities that share resources as CSP

Choose time for activities (e.g. observations on Hubble telescope, or semesters to take required classes).



Variables are activities

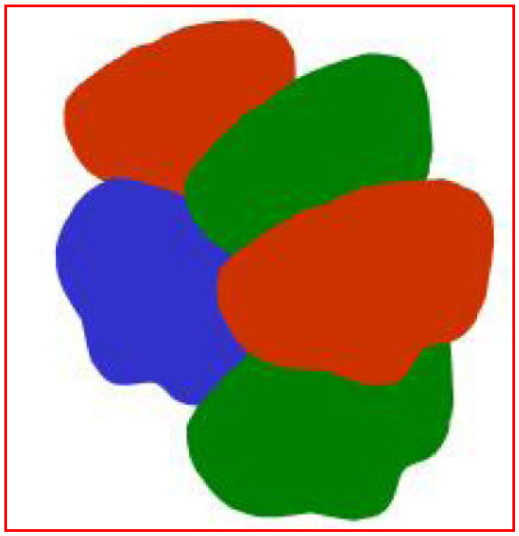
Domains are sets of start times (or "chunks" of time)

Constraints

1. Activities that use same resource cannot overlap in time.
2. Preconditions satisfied.

Graph Colouring as CSP

Pick colours for map regions, avoiding colouring adjacent regions with the same colour.



Variables are regions

Domains are colours allowed

Constraints adjacent regions must have different colours

3-SAT as CSP

Boolean Satisfiability problems - the original NP-complete problem

Find values for boolean variables A, B, C, … that satisfy the formula (in conjunctive normal form).

(A or B or !C) and (!A or C or B)

1-in-3-SAT

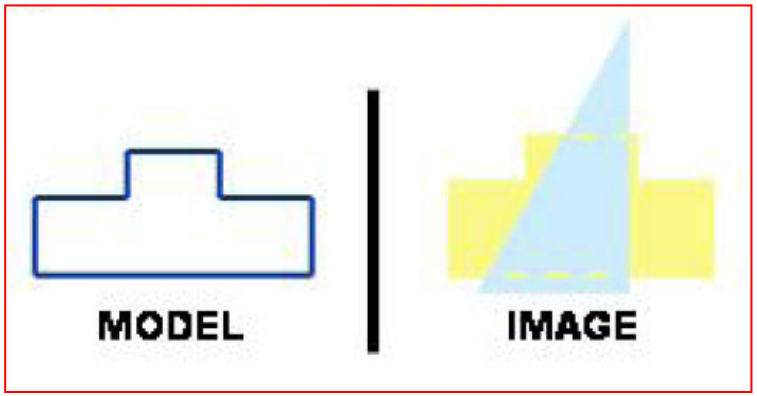
A variant of 3-SAT; determine whether there exists a truth assignment to the variables so that each clause has exactly one TRUE literal

Variables	are clauses
Domains	boolean variable assignments that make the clause true
Constraints	clauses with shared boolean variables must agree on value of variable.

CSP

Model-based recognition as CSP

Find given model in edge image, with rotation and translation allowed

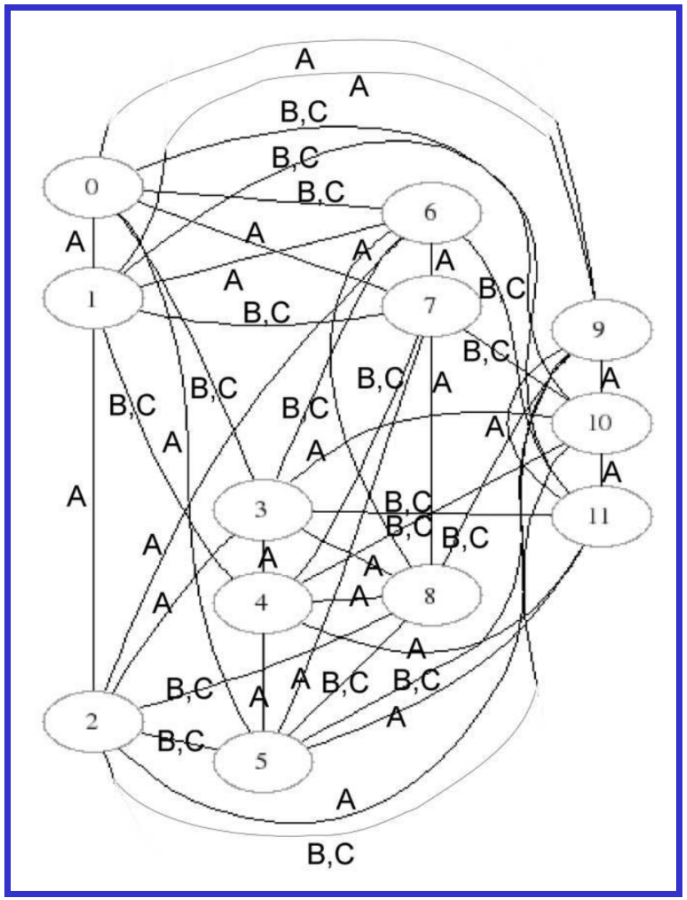


Variables are edges in model

Domains set of edges in image

Constraints angle between model & image edges must match

3D Geometric Object Recognition: Box



Constraint Graph

Variables: 12 line segments in the model

Domain: current image segments

Constraints:

- A = line segments are connected
- B = line segments are parallel
- C = line segments are of similar length

Fig. 1. After segment detection

Fig. 2. Object recognition results

Kasprzak et al, A constraint satisfaction framework with Bayesian inference for model-based object recognition

https://www.gecode.org/

generic

constraint

development

environment

Gecode

Gecode is an open source C++ toolkit for developing constraint-based systems and applications. Gecode provides a constraint solver with state-of-the-art performance while being modular and extensible. Gecode is:

Good News / Bad News

Good News very general & interesting class problems

Bad News includes NP-Hard (intractable) problems

So, good behaviour is a function of domain and not the formulation as CSP.

CSP

Example

Given **32 papers** (159101, 159201, …, 159334, 159302) & **8 semesters** (Fall 1, Winter 1, …, Winter 4). Find a legal schedule.

Assume that we are given a list of all papers available for the curriculum, together with their pre-requisites requirements, semester offered and time offered.

CSP

5

Example

Given **32 papers** (159101, 159201, …, 159334, 159302) &
8 semesters (Fall 1, Winter 1, …, Winter 4).
Find a legal schedule.

Assume that we are given a list of all papers available for the curriculum, together with their pre-requisites requirements, semester offered and time offered.

Constraints

- Pre-requisities
- Papers offered on limited semesters
- Limited number of papers per semester
- Avoid time conflicts

CSP

Example

Given **32 papers** (159101, 159201, …, 159302) & **8 semesters** (Fall 1, Winter 1, …, Winter 4). Find a legal schedule.

Constraints

- Pre-requisities
- Papers offered on limited semesters
- Limited number of papers per semester
- Avoid time conflicts

Note: CSPs are not typically for expressing (soft) **preferences** (e.g. minimise difficulty, balance subject areas, etc.)

Example

Choice of Variables & Values

Constraints

1. Pre-requisities*
2. Papers offered on limited semesters-filter
3. Limited number of papers per semester ✓
4. Avoid time conflicts-filter

Variables

A. Semesters?

Domains

- Legal combinations of, for example, 4 papers per semester (but this would result to a **huge set of values**)

- **n – number of choices**
- **r - number of elements per group**
- **No repeats**

n	32
r	4

Formula:

Permutations	863,040	if the paper order matters	**Permutations = (n!)/(n-r)!**
Combinations	35,960	if the paper order does not matter	**Combinations = (n!)/(r!(n-r)!)**

https://www.mathsisfun.com/combinatorics/combinations-permutations.html

Example

Constraints

1. Pre-requisities*
2. Papers offered on limited semesters-filter
3. Limited number of papers per semester ✓
4. Avoid time conflicts*

Choice of Variables & Values

Variables	Domains
A. Semesters?	• Legal combinations of, for example, 4 papers per semester (but this would result to a huge set of values)
B. Semester Slots?	• Papers offered during that semester

Subdivide semesters into slots e.g. 4 of them

(Fall 1, 1)(Fall 1, 2)
(Fall 1, 3)(Fall 1, 4)

8 semesters * 4 slots = 32 variables

Example

Constraints

1. Pre-requisities
2. Papers offered on limited semesters
3. Limited number of papers per semester
4. Avoid time conflicts

Choice of Variables & Values

Variables	Domains
A. Semesters?	• Legal combinations of, for example, 4 papers per semester (but this would result to a huge set of values)
B. Semester Slots?	• Papers offered during that semester
Subdivide semesters into slots (e.g. 4 of them (Fall 1, 1)(Fall 1, 2) (Fall 1, 3)(Fall 1, 4)	
C. Papers?	• Semesters or semester slots (semester slots allow expressing constraint on limited number of papers/ semester)

Example

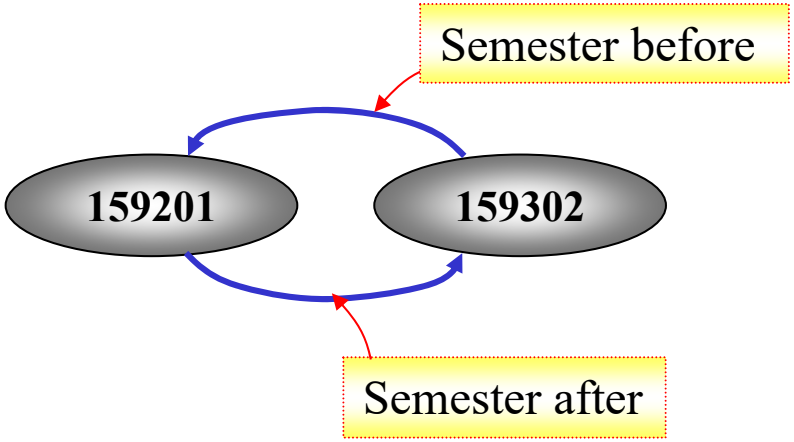
Constraints

1. Pre-requisities *
2. Papers offered on limited semesters
3. Limited number of papers per semester
4. Avoid time conflicts

Constraints

Use papers as variables and semester slots as values.

Prerequisite



- For pairs of papers that must be ordered.

- Complementary constraints

Example

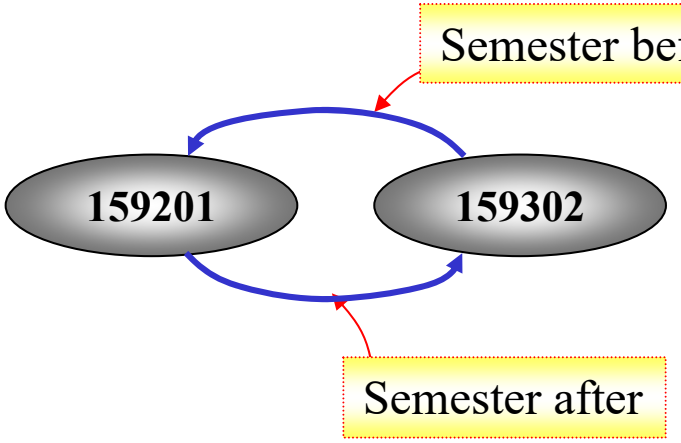
Constraints

1. Pre-requisities *
2. Papers offered on limited semesters-filter
3. Limited number of papers per semester
4. Avoid time conflicts

Constraints

Use papers as variables and semester slots as values.

Prerequisite ⇨



- For pairs of papers that must be ordered.

Papers offered only in some semesters ⇨

- Filter domain

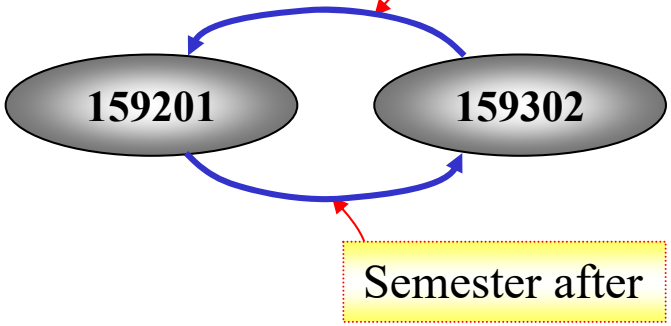
Constraints

Constraints

1. Pre-requisities*
2. Papers offered on limited semesters-filter
3. Limited number of papers per semester*
4. Avoid time conflicts

Use papers as variables and semester slots

Prerequisite ⇨

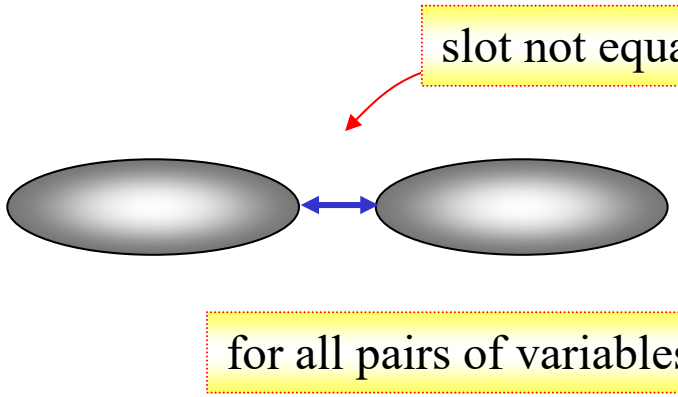


- For pairs of papers that must be ordered.

Papers offered only in some semesters ⇨

- Filter domain

Limit # papers ⇨



- Use semester-slots only once

CSP

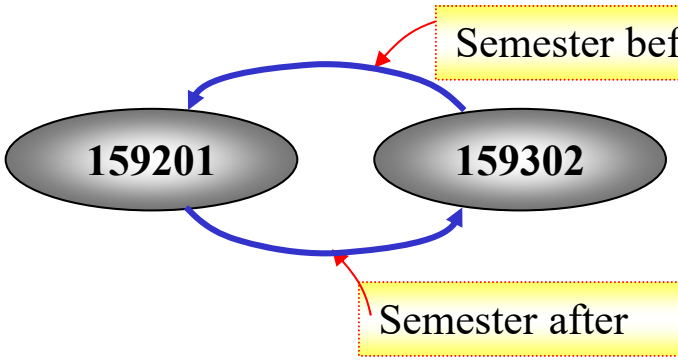
Constraints

Constraints

1. Pre-requisities*
2. Papers offered on limited semesters-filter
3. Limited number of papers per semester*
4. Avoid time conflicts*

Use papers as variables and semester slots

Prerequisite ⇨

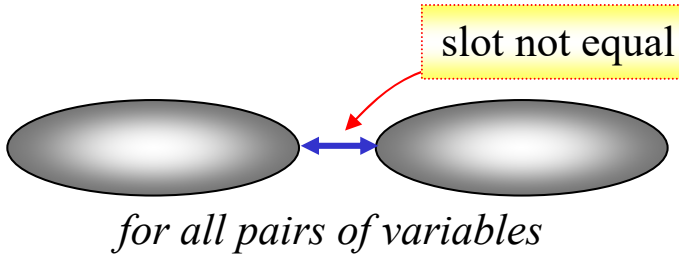


- For pairs of papers that must be ordered.

Papers offered only in some semester s ⇨

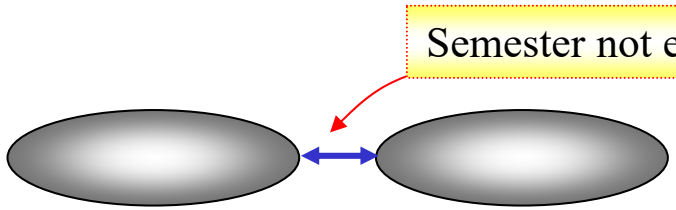
- Filter domain

Limit # papers ⇨



- Use semester-slots only once

Avoid time conflicts ⇨



- For pairs offered at same or overlapping times

5

159.302 CSP

Solving CSPs

Source of contents: ***MIT OpenCourseWare***

Solving CSPs

Approaches to solving CSPs are some combination of **constraint propagation** and **search**.

1. **Constraint propagation –** to eliminate values that could not be part of any solution

2. **Search –** to explore valid assignments

Solving CSPs

Constraint Propagation (*aka* Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$V_i \rightarrow V_j$

Directed arc (V_i, V_j) is arc **consistent** if

$\forall\, x \in D_i \; \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

For **every**

there exists **some**

Solving CSPs

Constraint Propagation (aka Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$V_i \rightarrow V_j$

Directed arc (V_i , V_j) is arc consistent if

$\forall\, x \in D_i \; \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

We can achieve consistency on arc by deleting values from D_i (domain of variable at tail of constraint arc) that fail this condition.

Solving CSPs

Constraint Propagation (aka Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$V_i \rightarrow V_j$

Directed arc (V_i , V_j) is arc consistent if

$\forall\, x \in D_i\ \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

We can achieve consistency on arc by deleting values from D_i (domain of variable at tail of constraint arc) that fail this condition.

Assume domains are of size **d** at the most, and there are **e** binary constraints.

Solving CSPs

Constraint Propagation (aka Arc Consistency)

Arc consistency eliminates values from domain of variable that can never be part of a consistent solution.

$V_i \rightarrow V_j$

Directed arc (V_i, V_j) is arc consistent if

$\forall\, x \in D_i\ \exists\, y \in D_j$ such that (x, y) is allowed by the constraint on the arc.

We can achieve consistency on arc by deleting values from $\mathbf{D_i}$ (domain of variable at tail of constraint arc) that fail this condition.

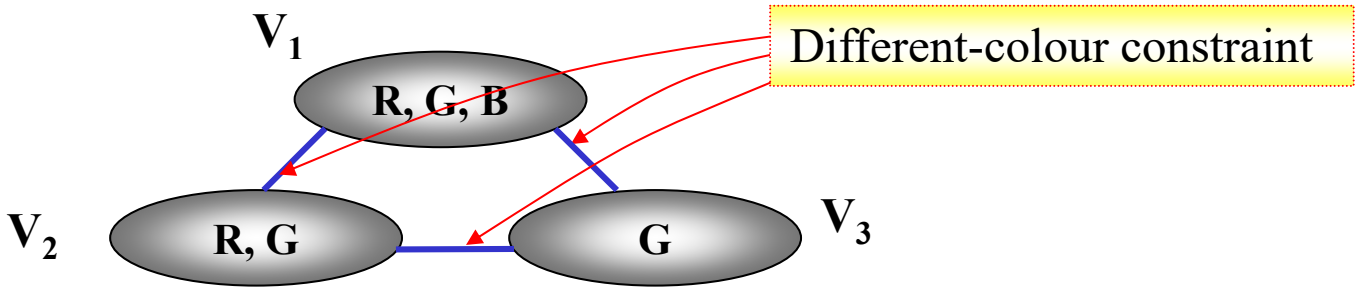
Assume domains are size at most **d** and there are **e** binary constraints.

In the **worst case**, a simple algorithm for arc consistency requires $\mathbf{O(ed^3)}$ arc tests– note that just verifying arc consistency takes $\mathbf{O(d^2)}$ for each arc.

Constraint Propagation Example

Graph Colouring

Initial domains are indicated



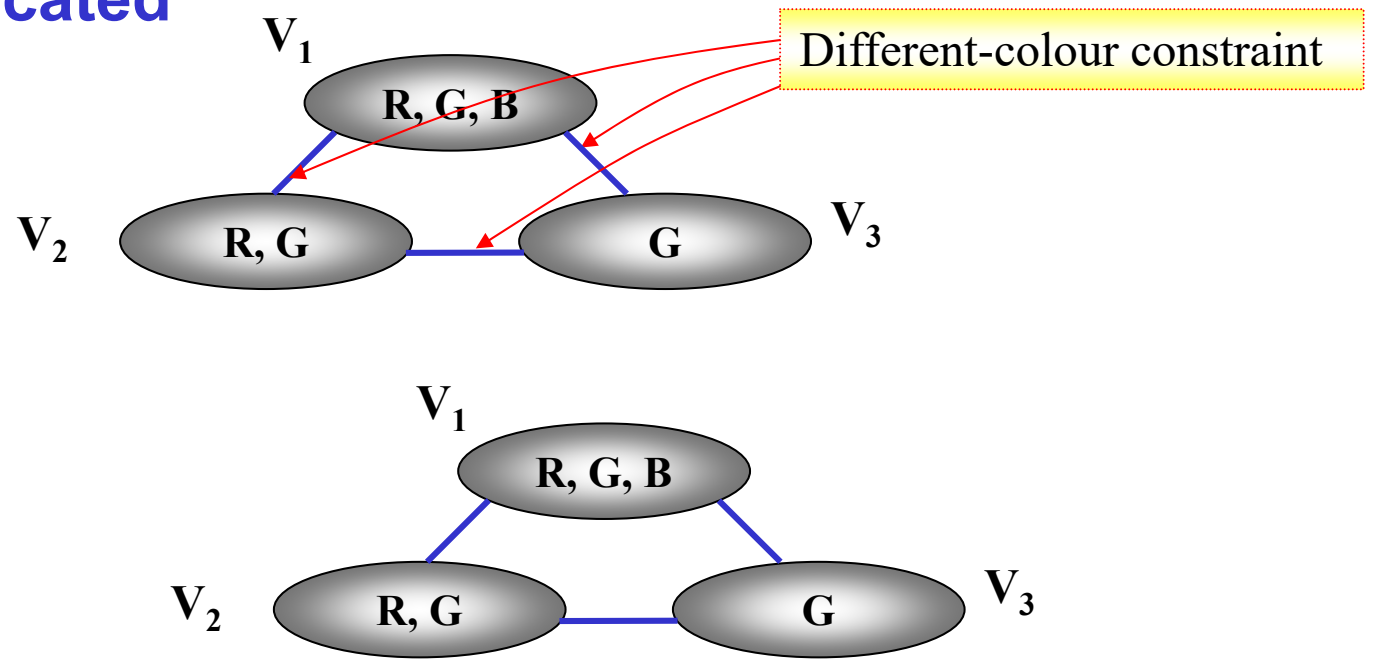
- Each variable is constrained to have values different from its neighbors

Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted



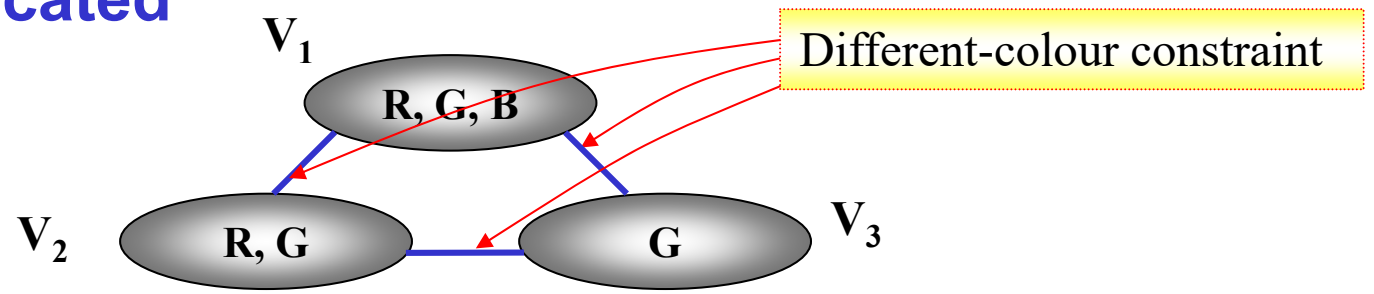
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

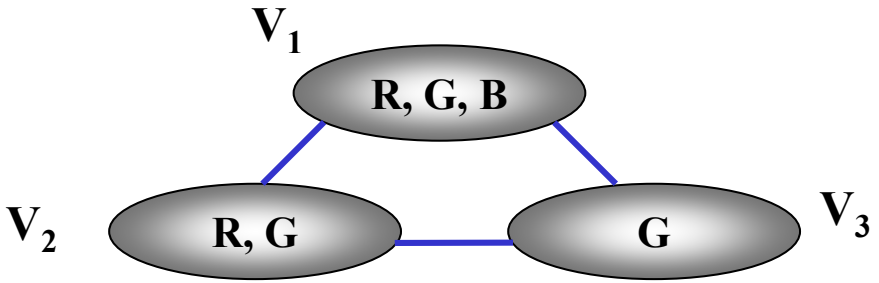
Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted
V_1-V_2	none





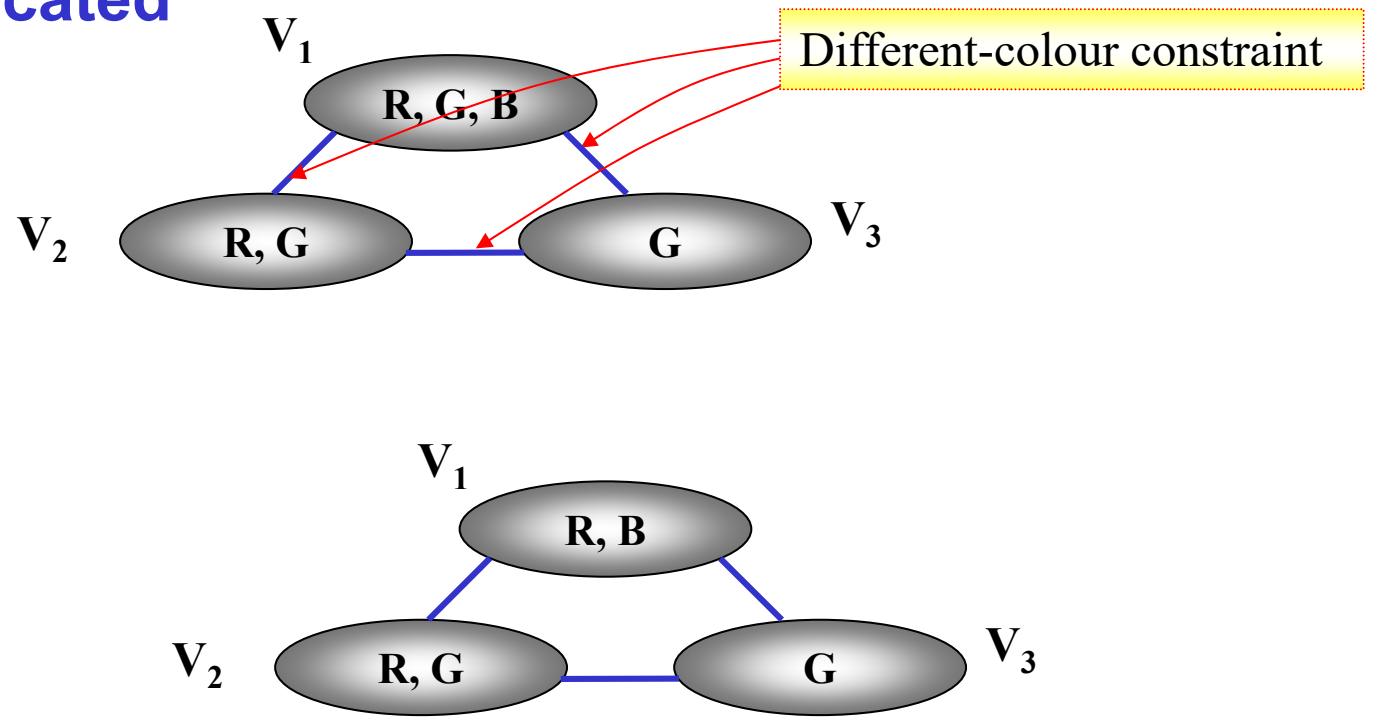
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted
V_1-V_2	none
V_1-V_3	V_1(G)



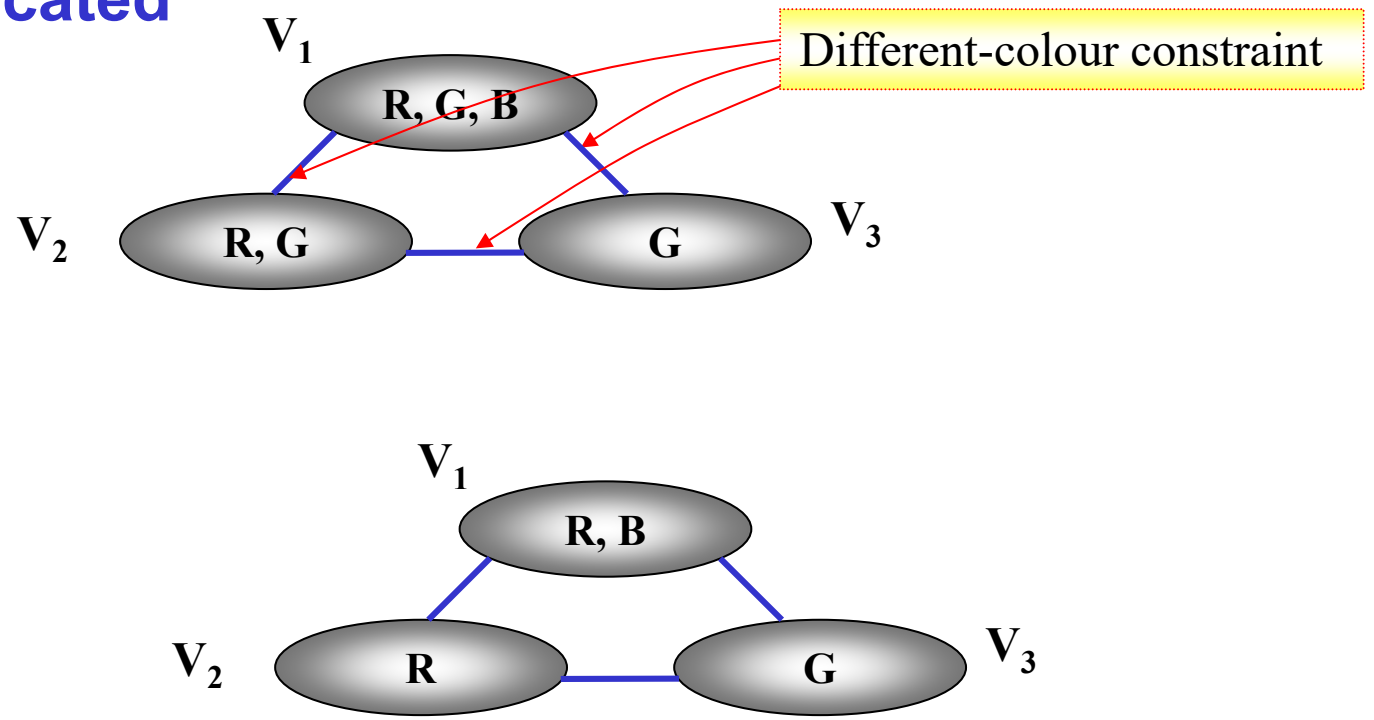
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted
V_1-V_2	none
V_1-V_3	V_1(G)
V_2-V_3	V_2(G)



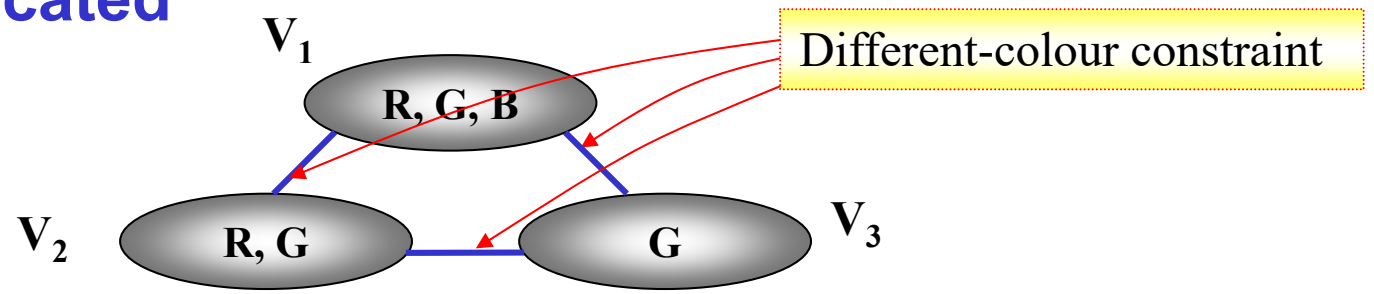
- Each undirected constraint arc is really two directed constraint arcs, the effects shown above are from examining both arcs.

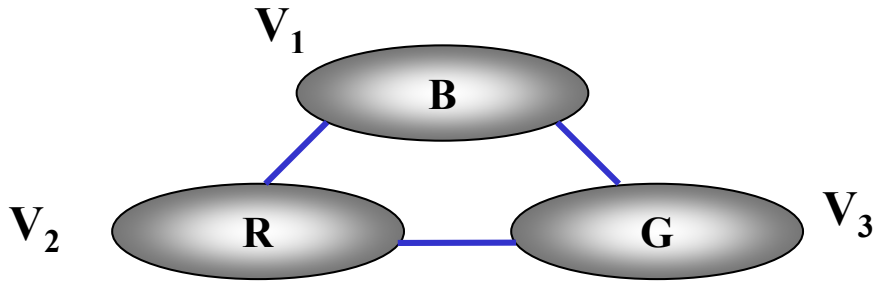
Constraint Propagation Example

Graph Colouring

Initial domains are indicated

Arc examined	Value deleted
V_1-V_2	none
V_1-V_3	V_1(G)
V_2-V_3	V_2(G)
V_1-V_2	V_1(R)
V_1-V_3	none
V_2-V_3	none

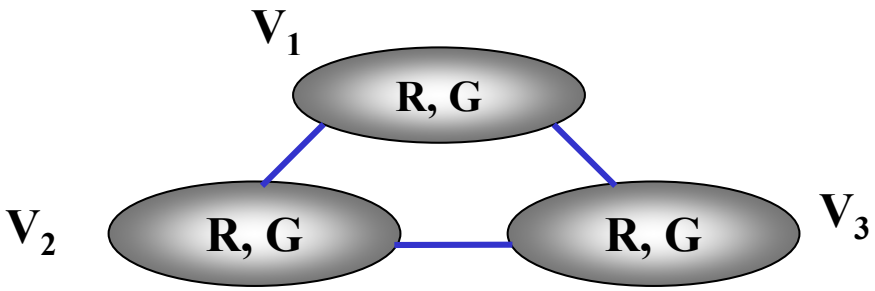




- In general we need to make one pass through any arc whose head variable has changed until no further changes are observed before we can stop.

But, arc consistency is not enough in general!

Graph Colouring

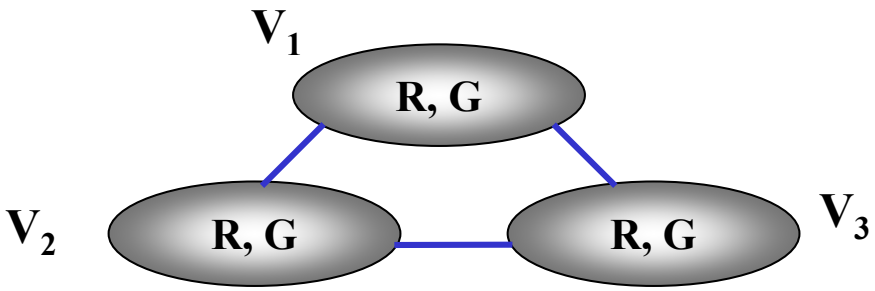


- Arc consistent but **NO SOLUTION**

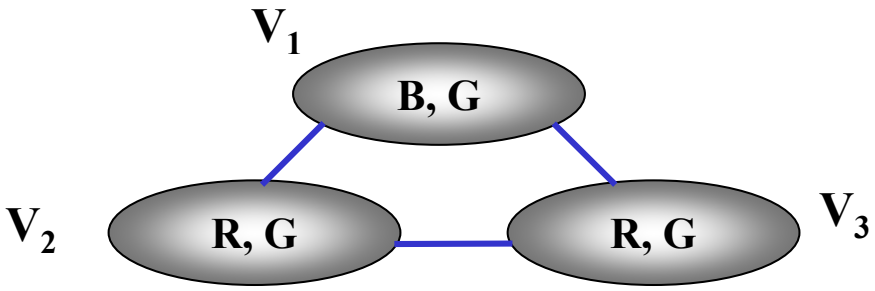
We need one colour for each variable!

But, arc consistency is not enough in general!

Graph Colouring



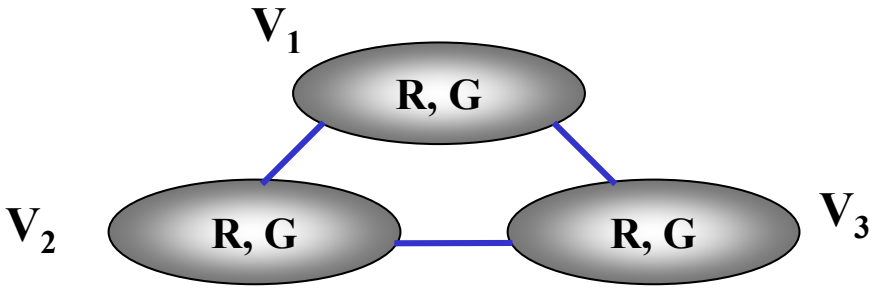
- Arc consistent but **NO SOLUTION**



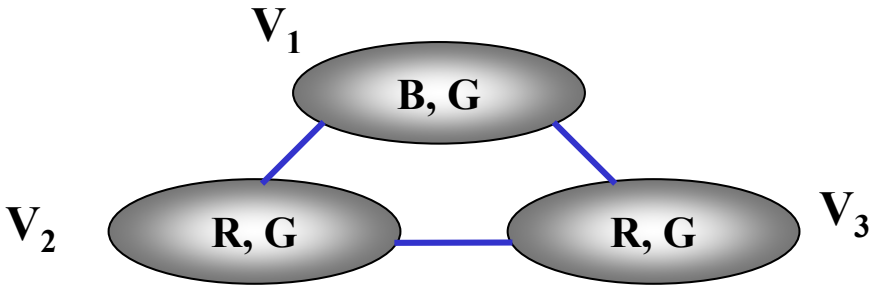
- Arc consistent but **2 SOLUTIONS:**
  - B, R, G
  - B, G, R

But, arc consistency is not enough in general!

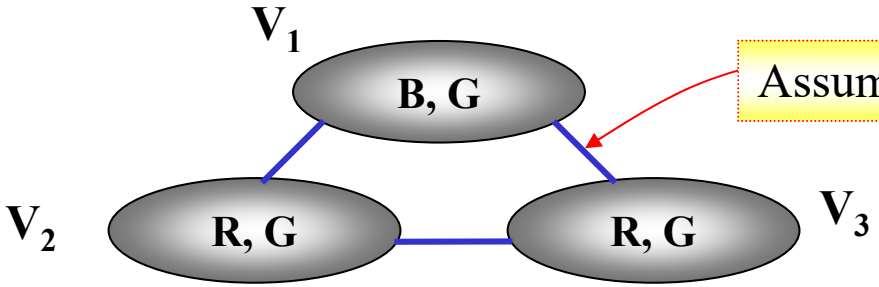
Graph Colouring



- Arc consistent but **NO SOLUTION**



- Arc consistent but **2 SOLUTIONS:**
  - B, R, G
  - B, G, R

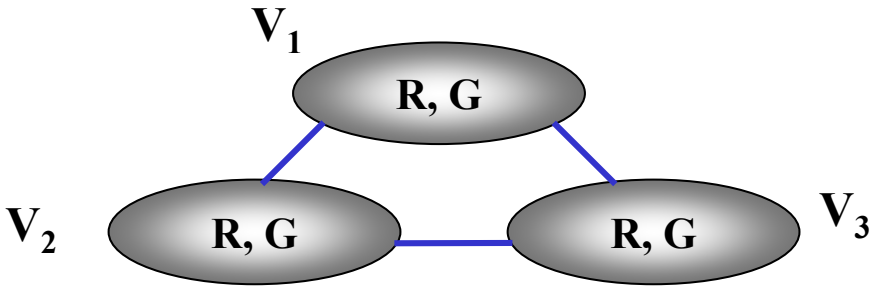


Assume B, R not allowed

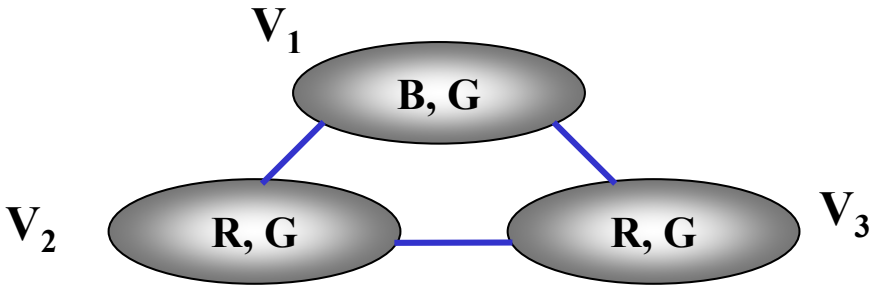
- Arc consistent, **1 SOLUTION by virtue of a special constraint**

But, arc consistency is not enough in general!

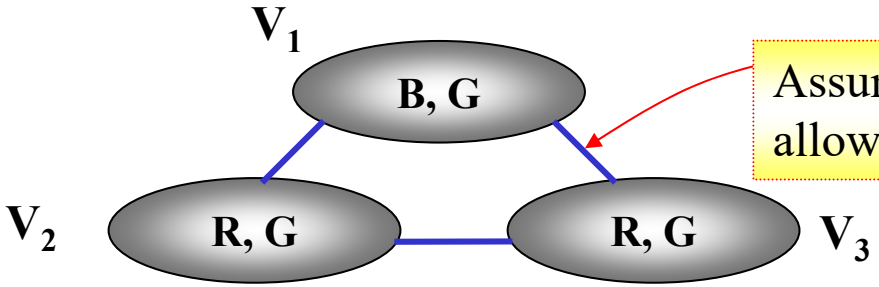
Graph Colouring



- Arc consistent but **NO SOLUTION**

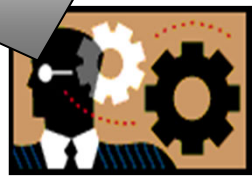


- Arc consistent but **2 SOLUTIONS:**
  - B, R, G
  - B, G, R



Assume B, R not allowed

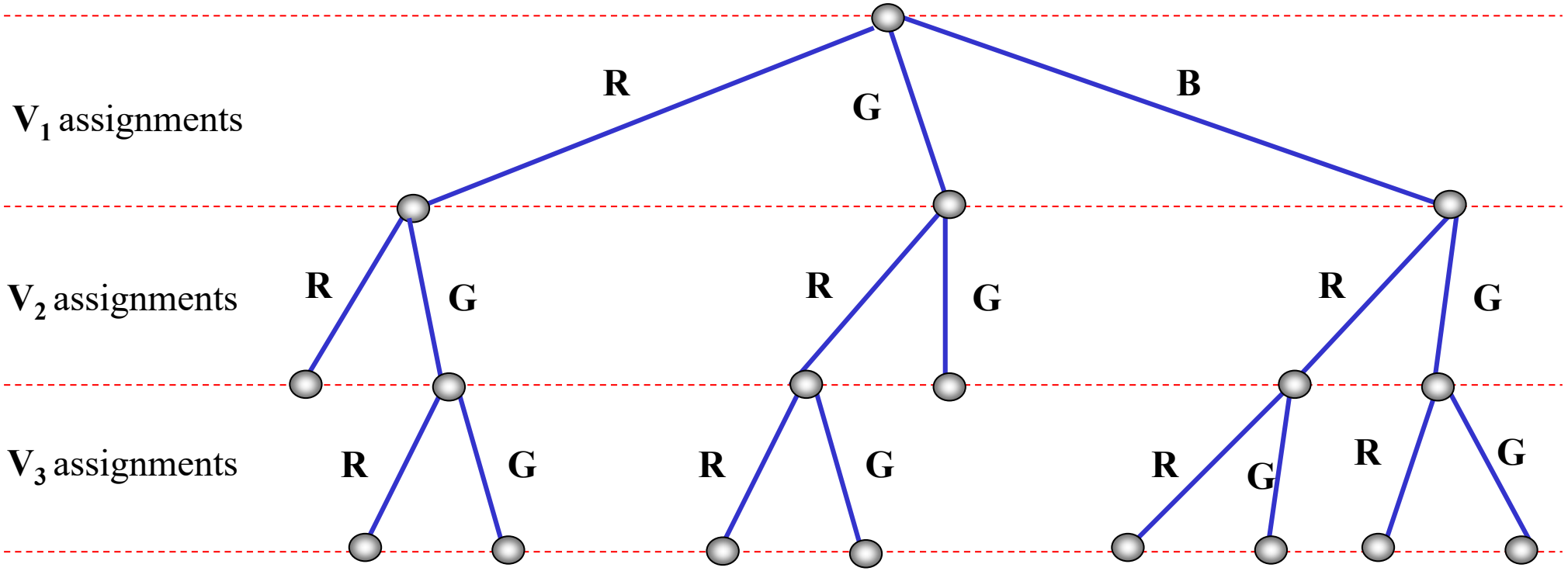
In general, if there is more than one value in the domain of any of the variables, we need to apply **Search** algorithms to find solutions (if there is any)



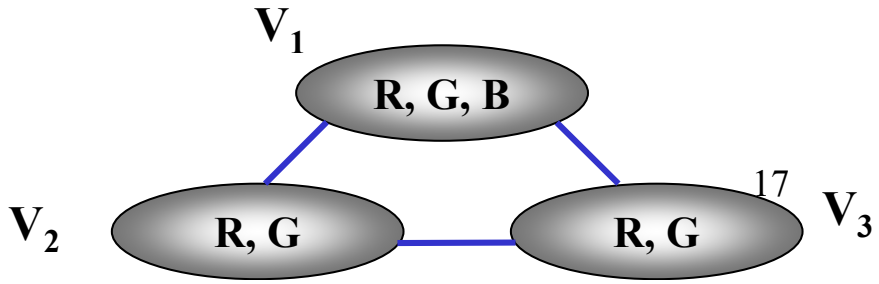
- Arc consistent, **1 SOLUTION**

Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).

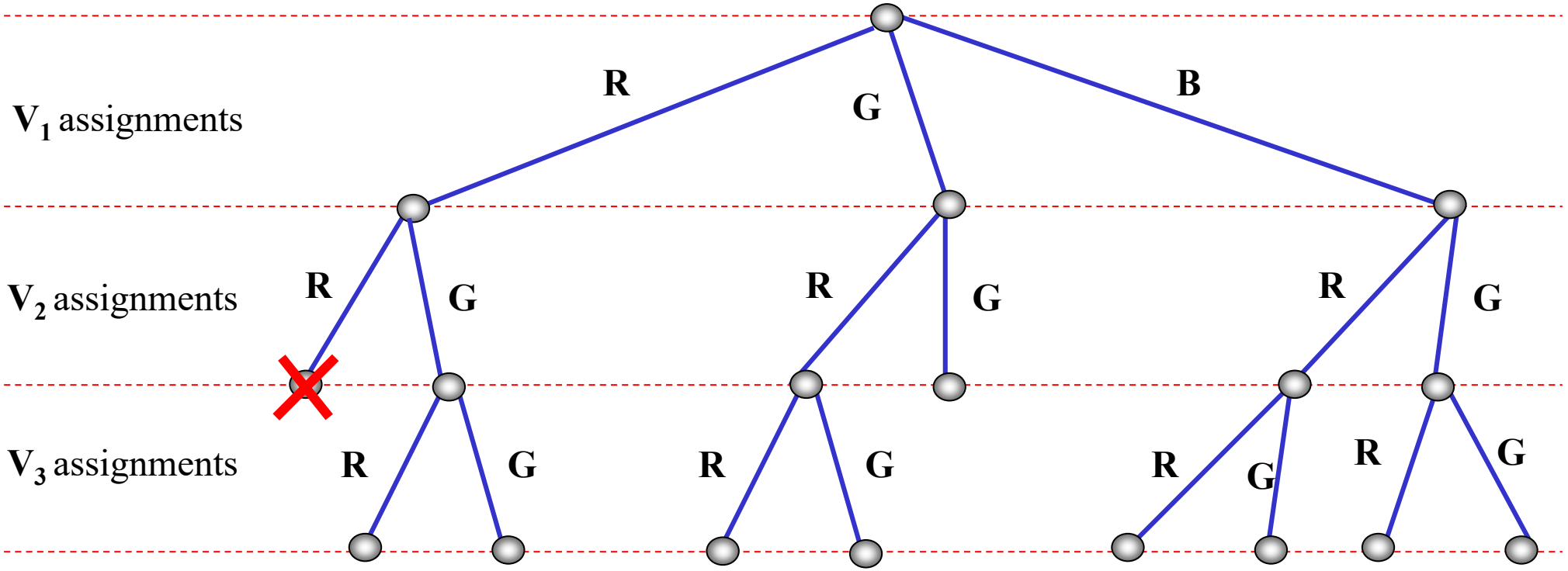


Note: Do not alphabetise the enumeration of values.



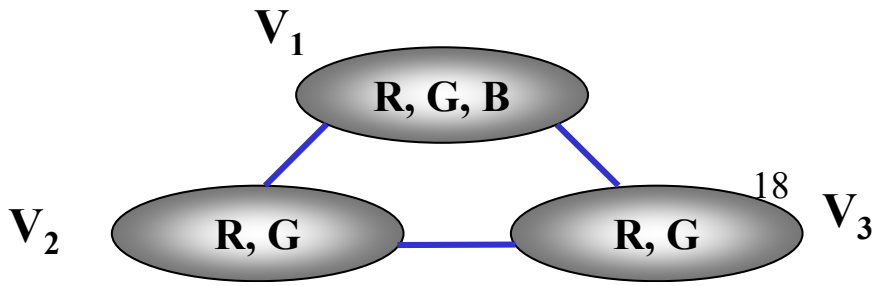
Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



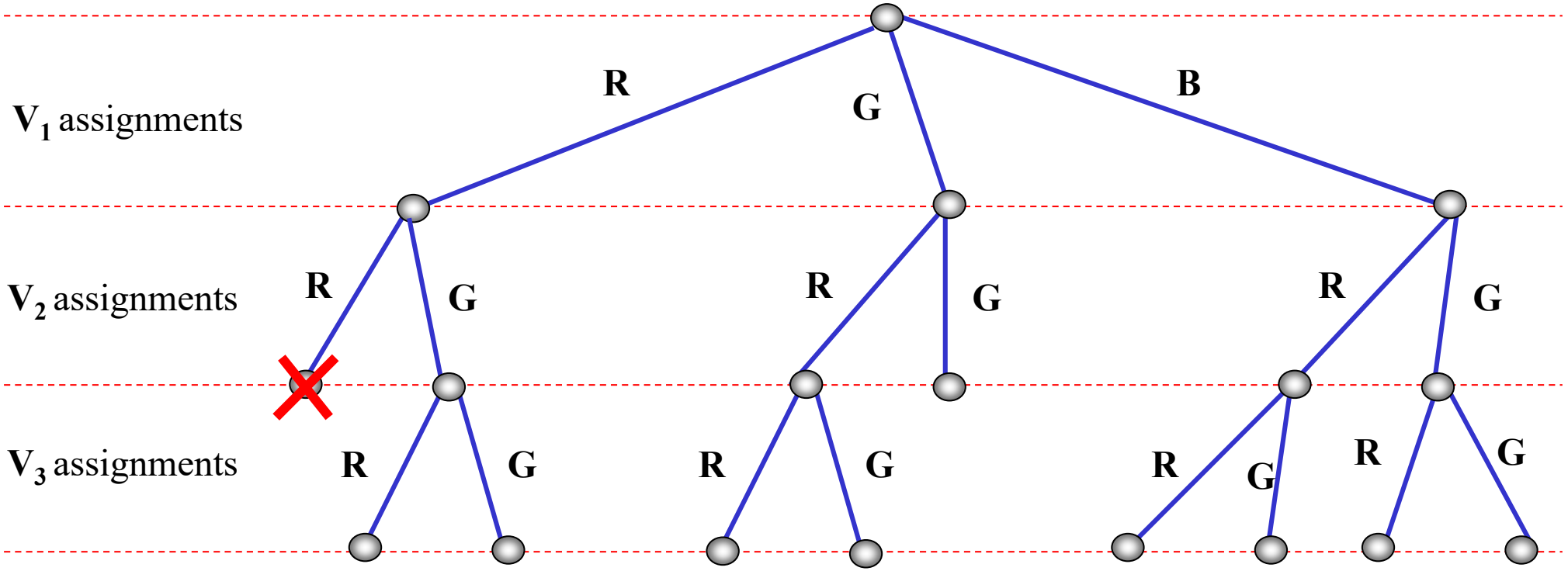
Inconsistent with V_1 = **R**

Backup at inconsistent assignment.



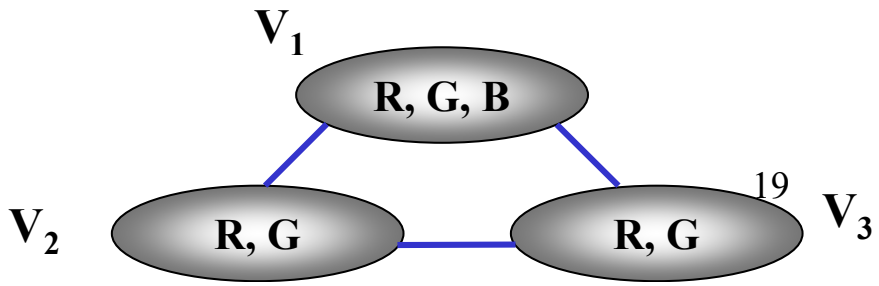
Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



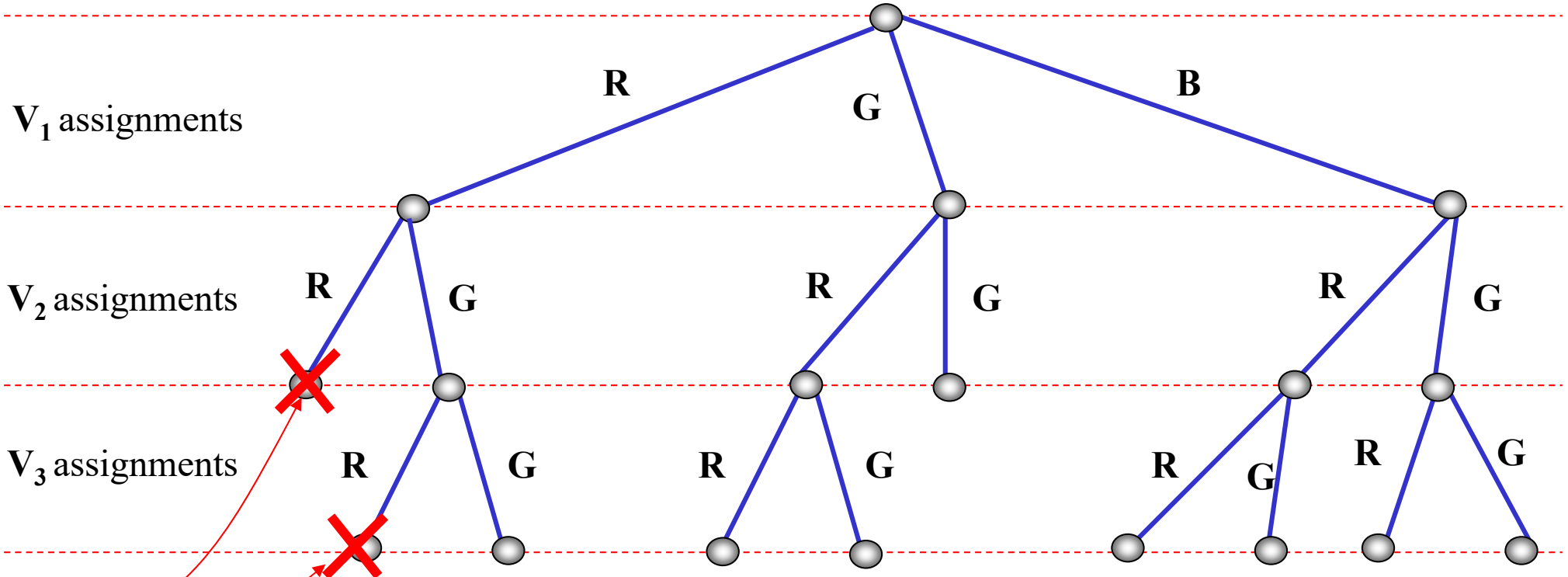
Inconsistent with V_1 = **R**

Backup at inconsistent assignment.



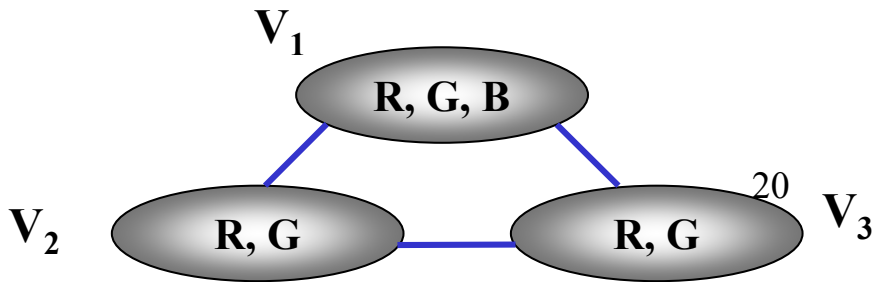
Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



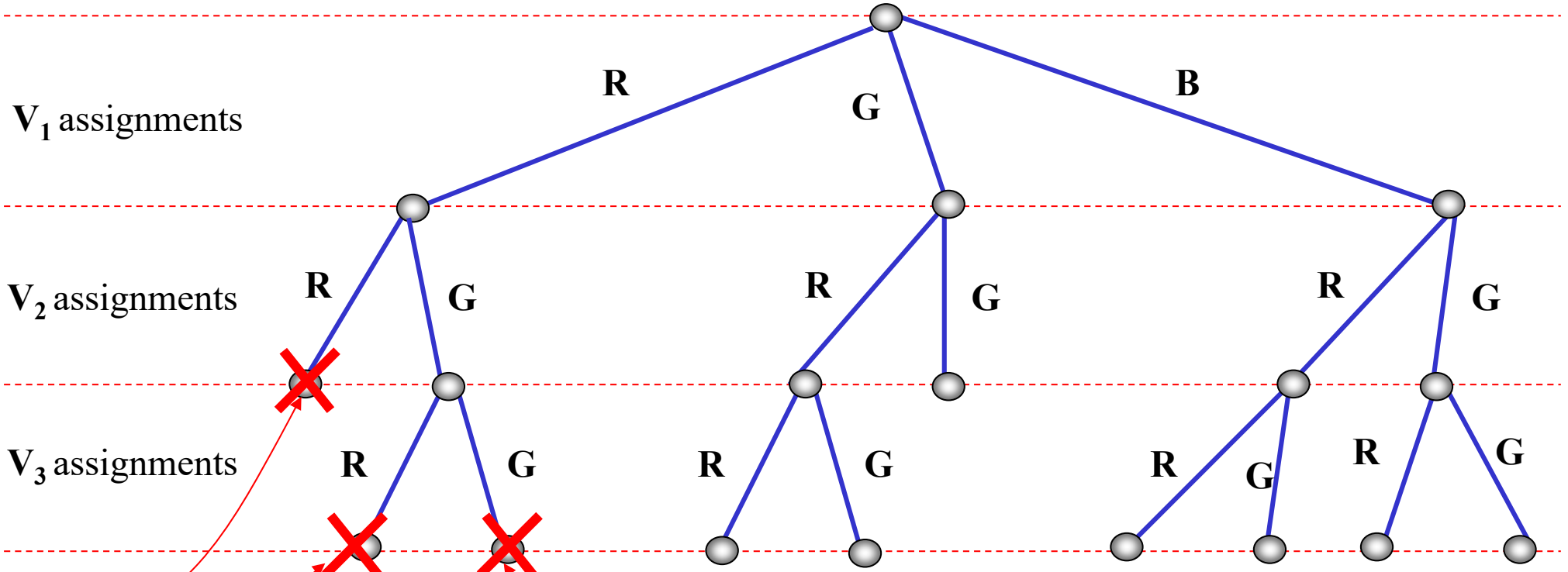
Inconsistent with V_1 = **R**

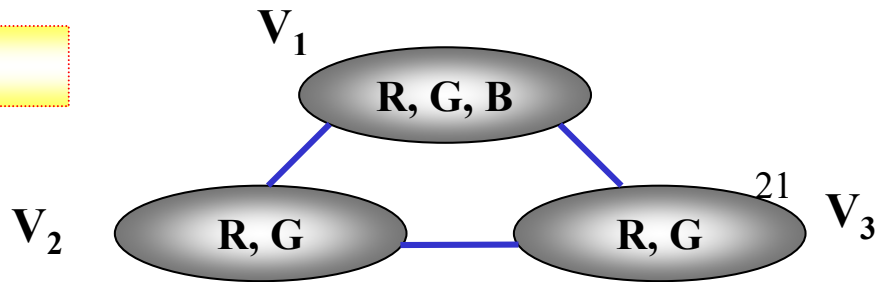
Backup at inconsistent assignment.



Searching for solutions - BackTracking (BT)

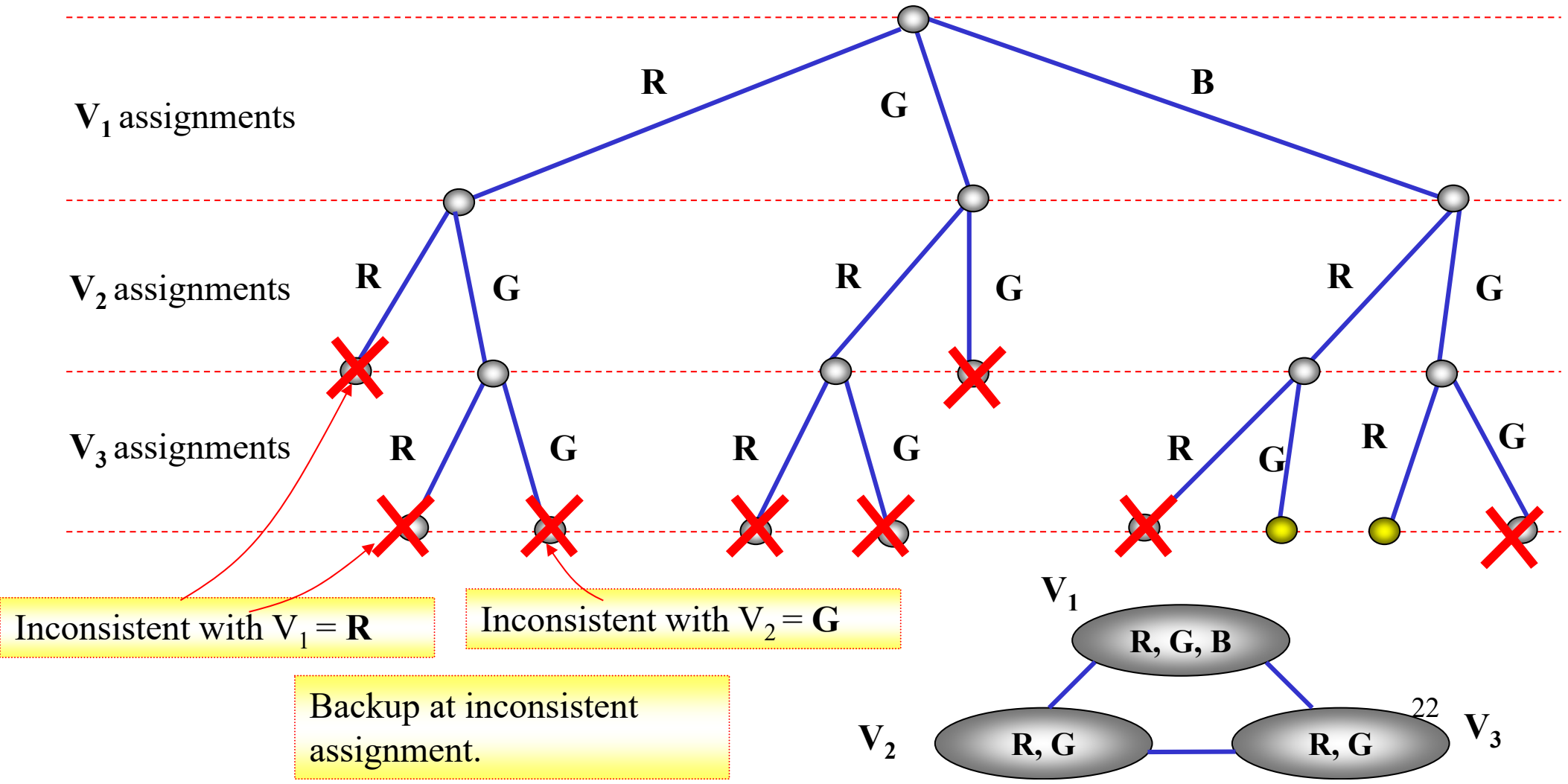
When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).





Searching for solutions - BackTracking (BT)

When we have too many values in domain (and/or constraints are weak) arc consistency doesn't do much, so we need to search. Simplest approach is pure **backtracking** (**depth-first search**).



Solving CSPs

Combine Backtracking & Constraint Propagation

How can we improve Pure BT when searching for a solution?

A node in BT tree is a partial assignment in which the domain of each variable has been set (tentatively) to singleton set.

Use constraint propagation (arc-consistency) to propagate the effect of the tentative assignment, i.e. eliminate values inconsistent with current values.

Solving CSPs

Combine Backtracking & Constraint Propagation

A node in BT tree is a partial assignment in which the domain of each variable has been set (tentatively) to singleton set.

Use constraint propagation (arc-consistency) to propagate the effect of the tentative assignment, i.e. eliminate values inconsistent with current values.

How much propagation to do?

Solving CSPs

Combine Backtracking & Constraint Propagation

A node in BT tree is a partial assignment in which the domain of each variable has been set (tentatively) to singleton set.

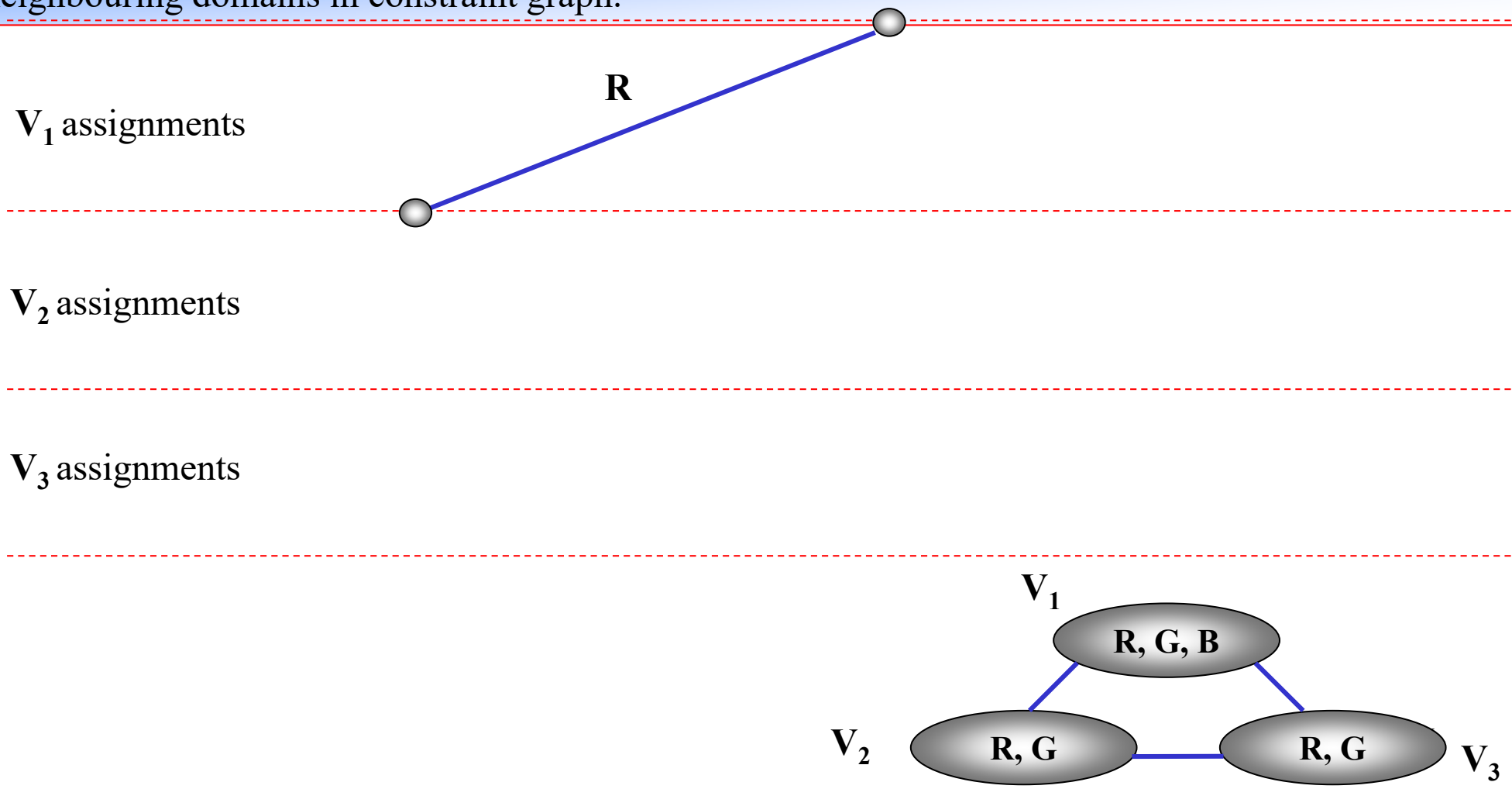
Use constraint propagation (arc-consistency) to propagate the effect of the tentative assignment, i.e. eliminate values inconsistent with current values.

Answer: Not much, just local propagation from domains with unique assignments, which is called **forward checking (FC)**. This conclusion is not necessarily obvious, but generally holds in practice.

How much propagation to do?

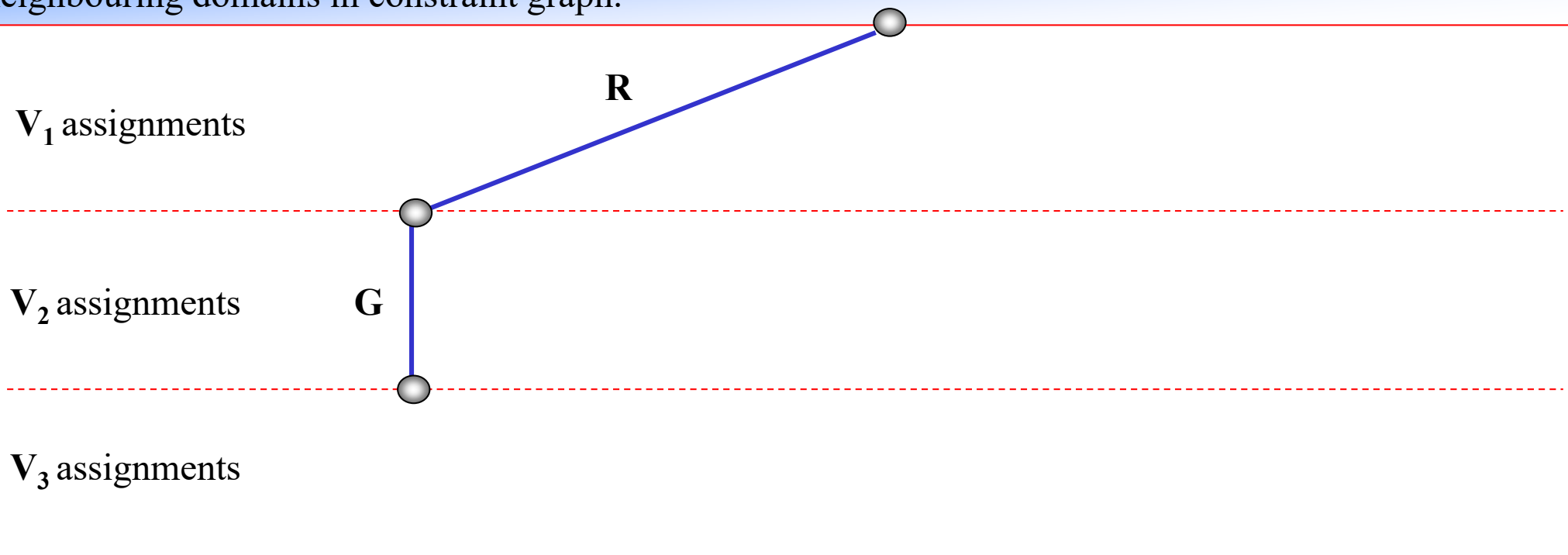
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

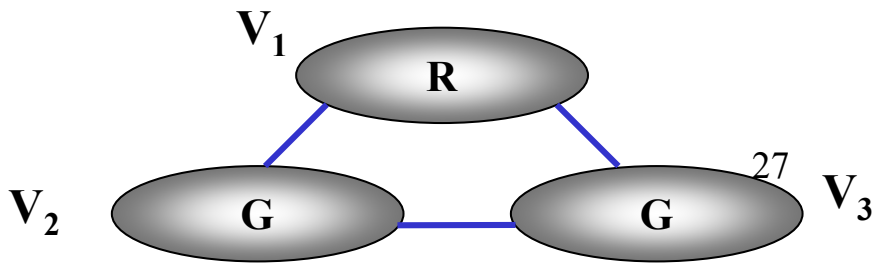


Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

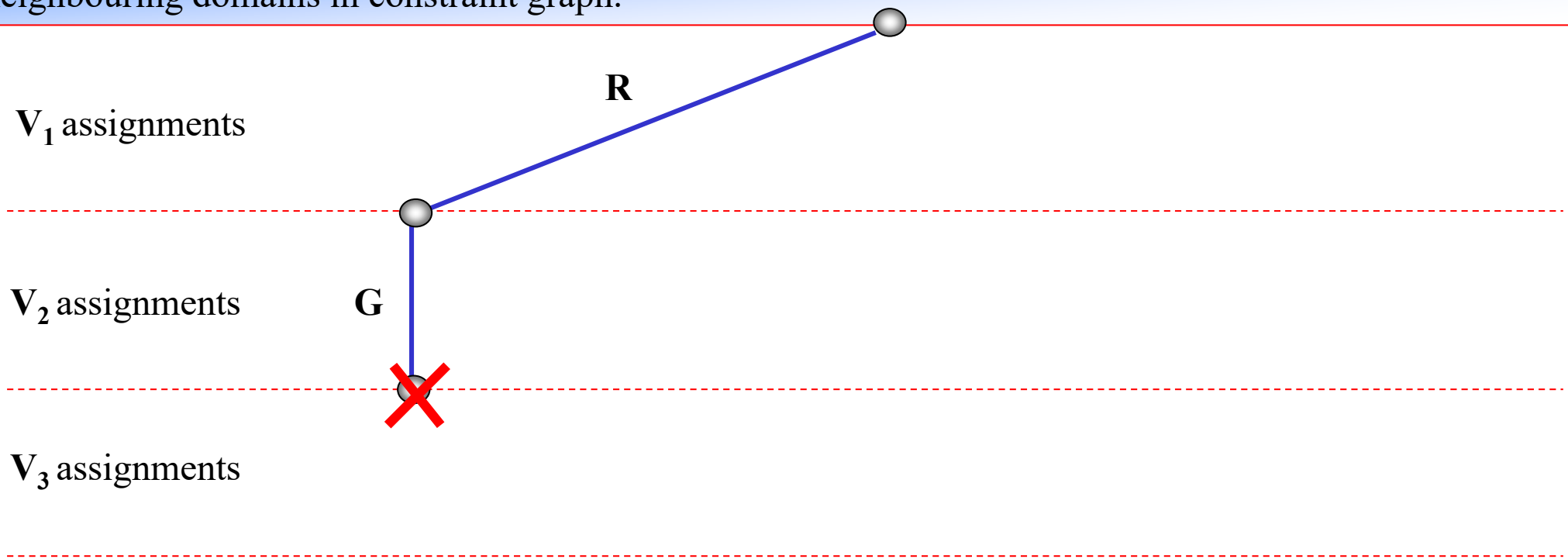


We eliminate any values that are inconsistent with the assignment.

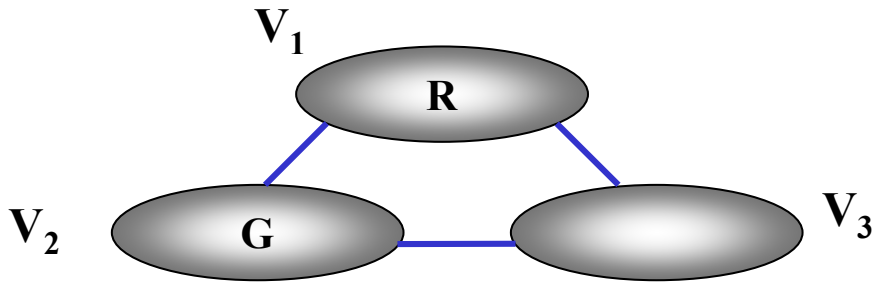


Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

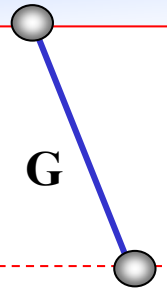


We have a conflict whenever a domain becomes empty.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

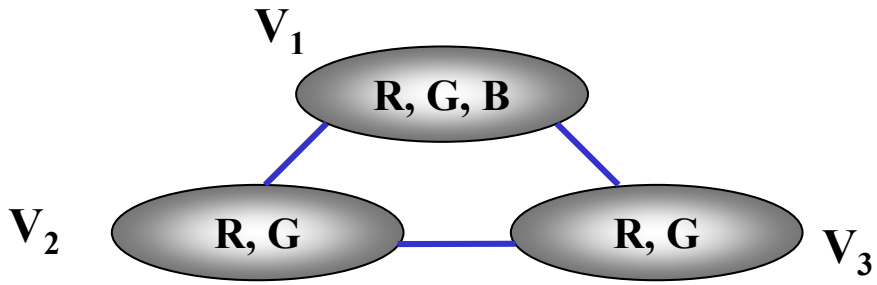


V_1 assignments

V_2 assignments

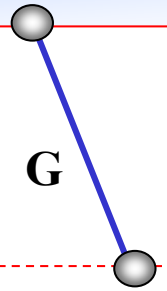
V_3 assignments

When backing up, we need to restore domain values, since deletions were done to reach consistency with tentative assignments considered during search.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

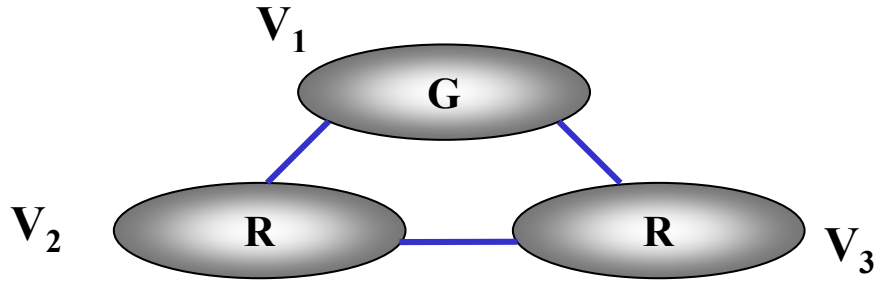


V_1 assignments

V_2 assignments

V_3 assignments

We eliminate G from V_2 and V_3.



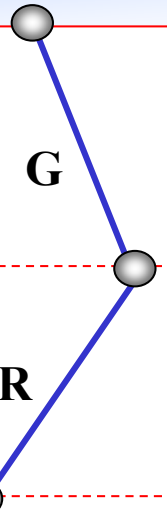
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

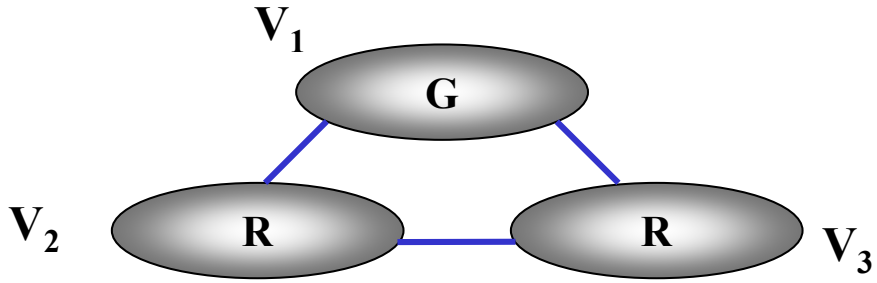
V_1 assignments

V_2 assignments

V_3 assignments

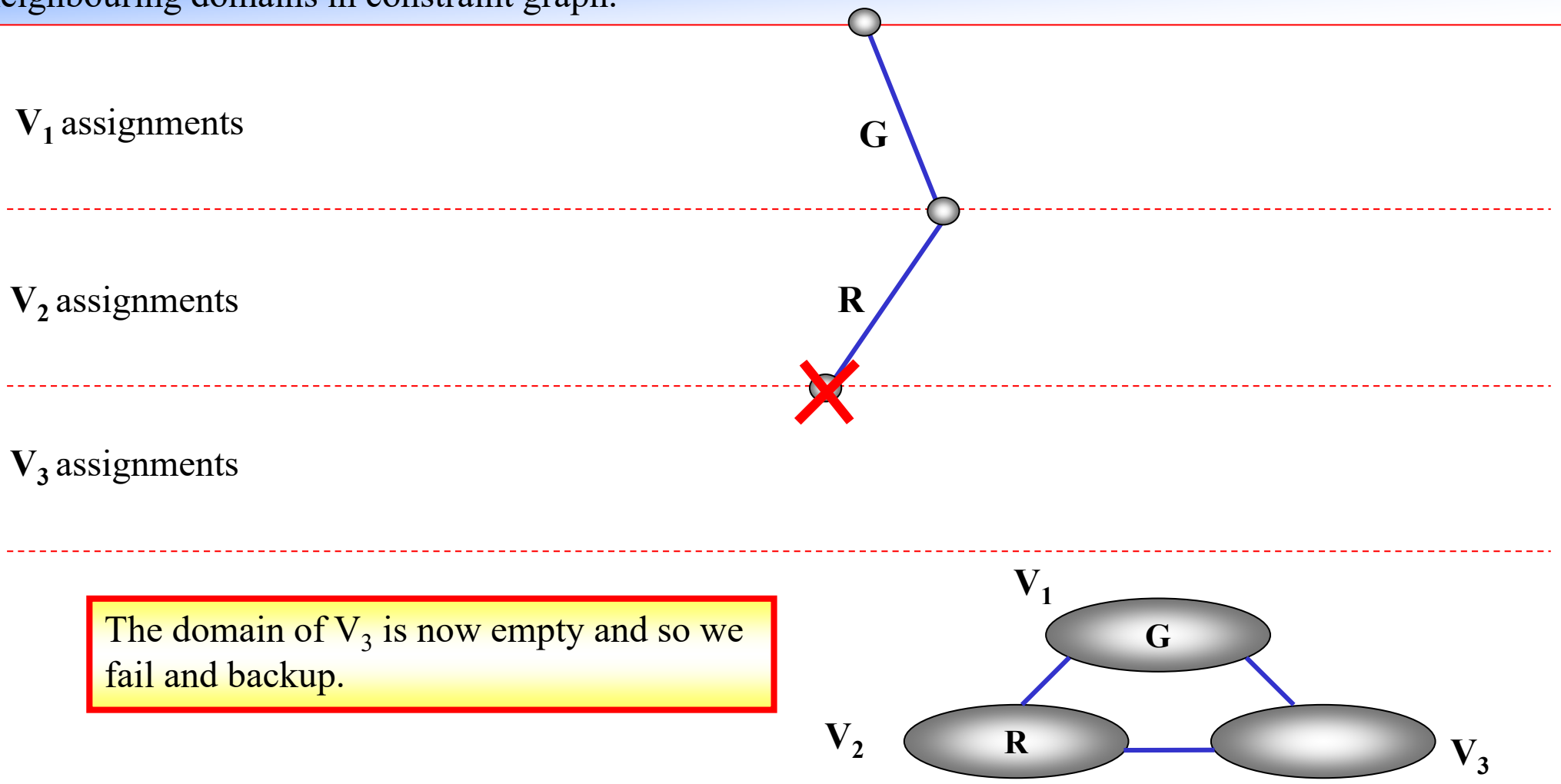


We now consider V_2 = R and propagate.



Backtracking with Forward Checking (BT-FC)

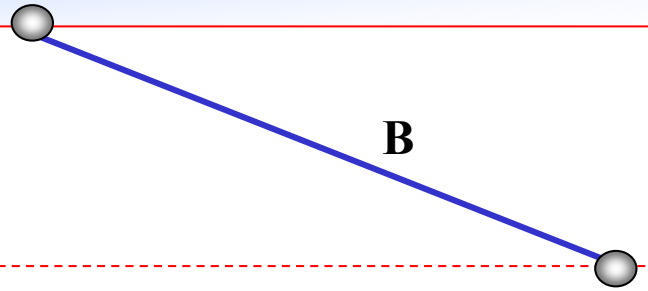
When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.



The domain of V_3 is now empty and so we fail and backup.

Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

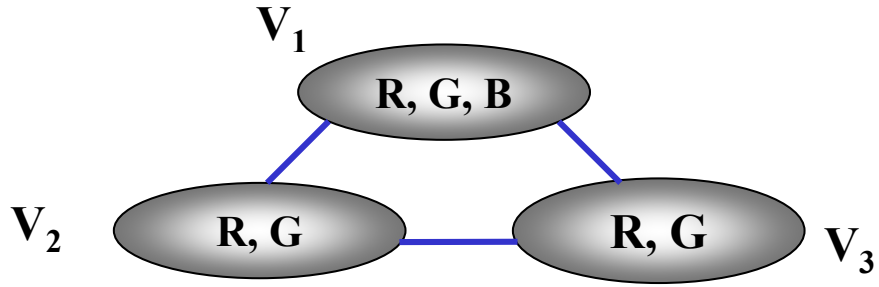


V_1 assignments

V_2 assignments

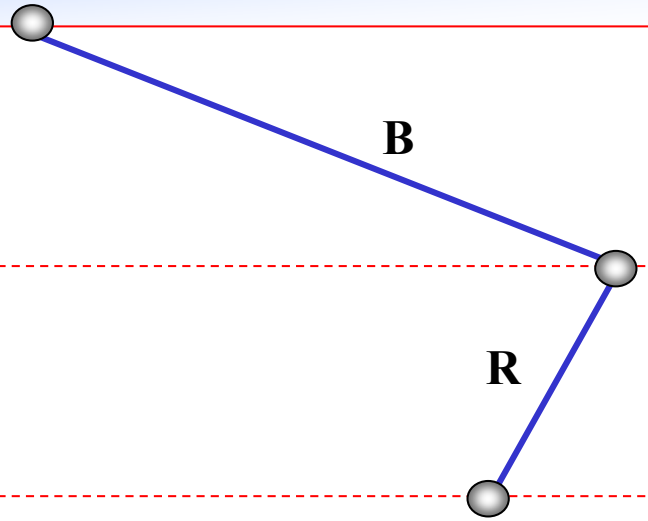
V_3 assignments

So, we move to consider V_1 = B and propagate.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

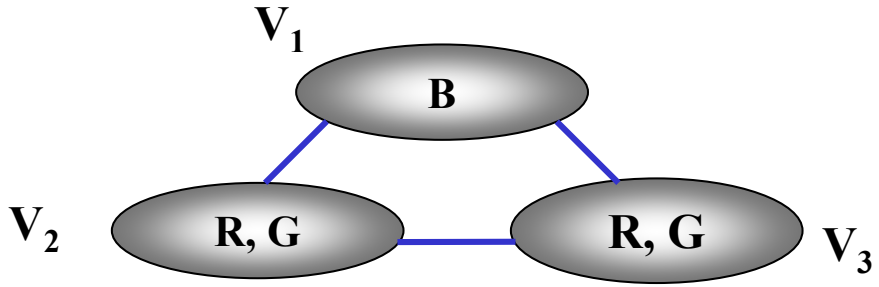


V_1 assignments

V_2 assignments

V_3 assignments

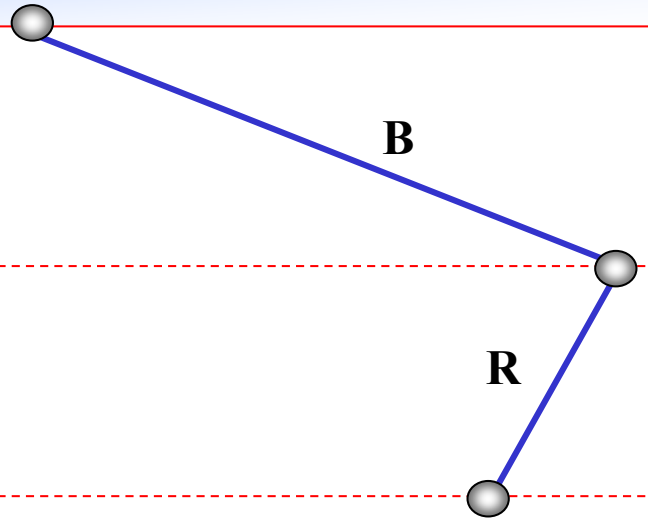
The propagation does not delete any values. We pick V_2 = **R** and propagate.



Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

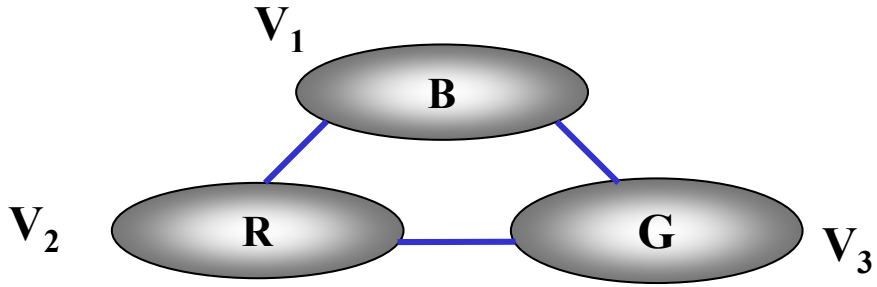
V_1 assignments



V_2 assignments

V_3 assignments

This removes the R values in the domains of V_1 and V_3.



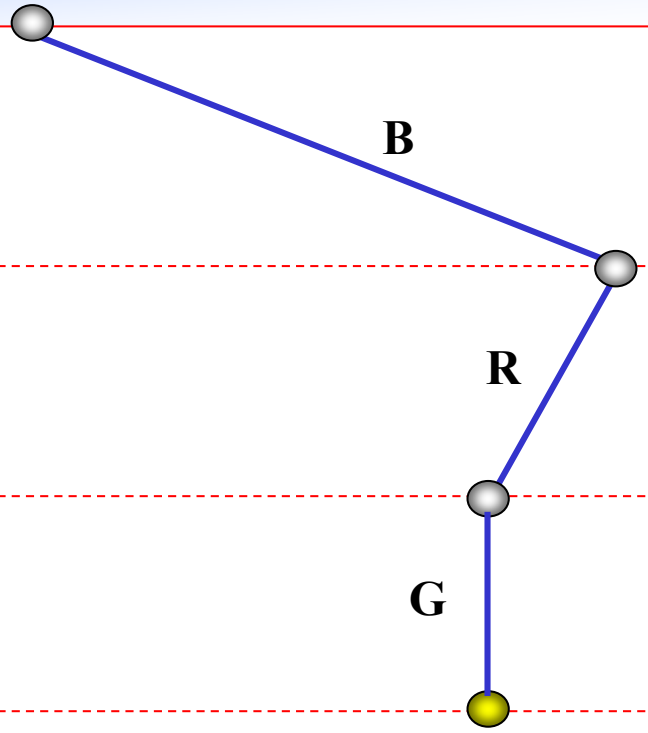
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

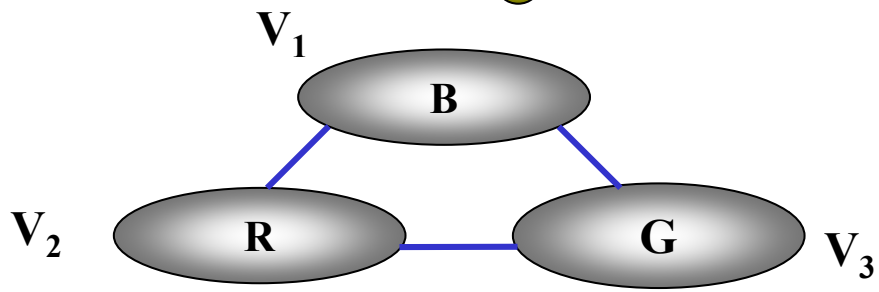
V_1 assignments

V_2 assignments

V_3 assignments



We pick $V_3 = G$ and have a consistent assignment.



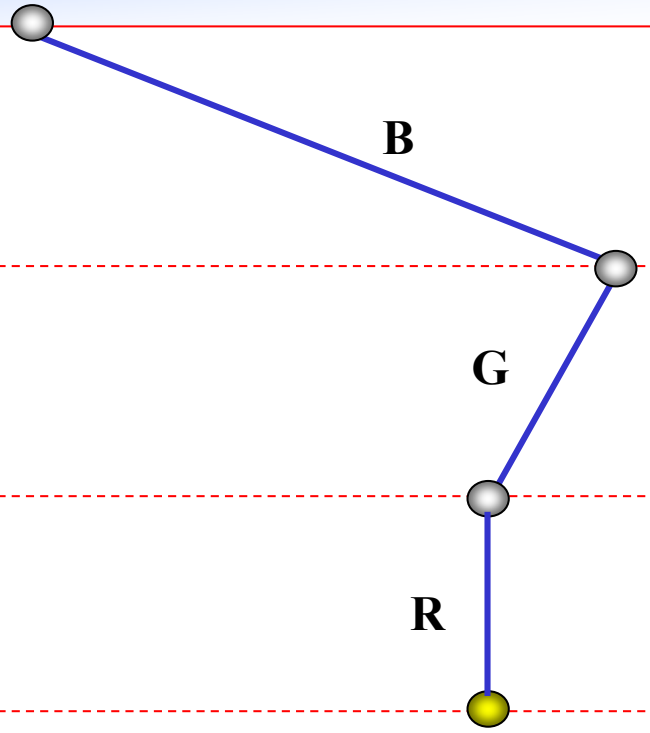
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.

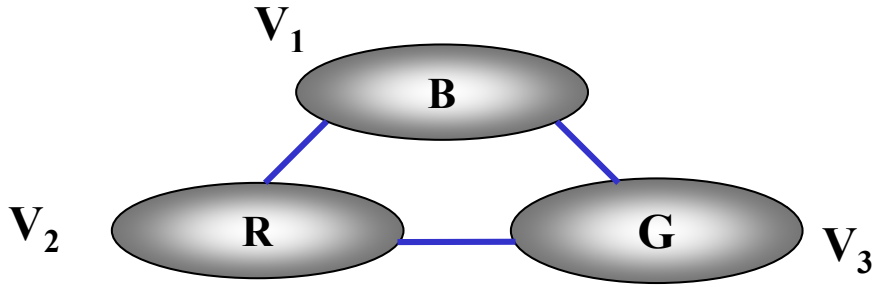
V_1 assignments

V_2 assignments

V_3 assignments

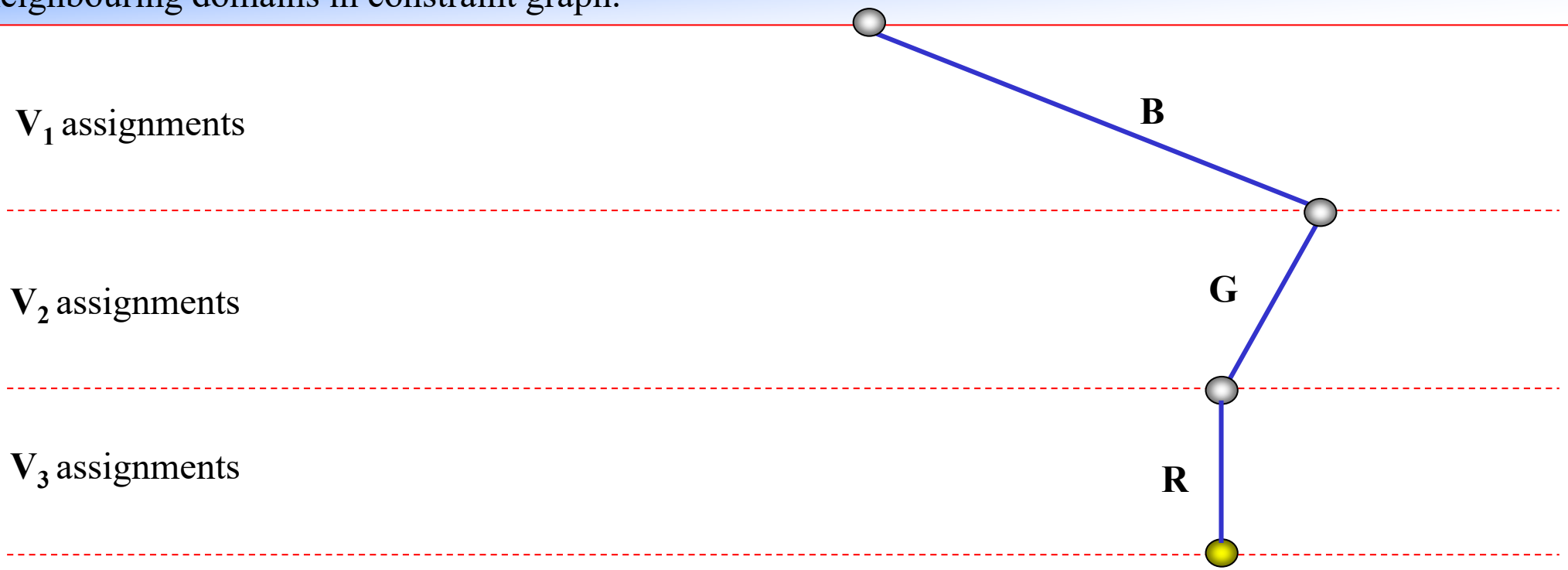


We can continue the process to find the other consistent solution.



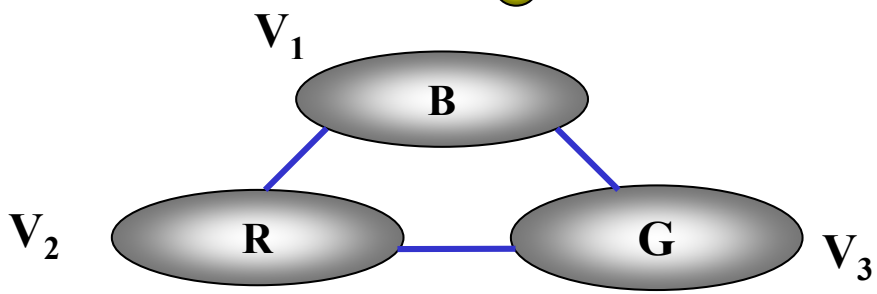
Backtracking with Forward Checking (BT-FC)

When examining an assignment $V_i = d_k$, remove any values inconsistent with that assignment from neighbouring domains in constraint graph.



No need to check previous assignments

Generally preferable to pure BT.



5

159.302 CSPs

Other Strategies

Constraint Satisfaction Problems

n.h.reyes@massey.ac.nz

Reference of contents: ***MIT OpenCourseWare***

Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed **ordering** of **variables** & **values**, (e.g. random order or place variables with constraints first.)

You can usually do better by choosing an order dynamically as the search proceeds.

Ordering of variables can *have a substantial effect on the cost of finding the answer*. We can re-order variables based on information available during a search.

Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed ordering of variables & values, e.g. random order or place variables with constraints first.

You can usually do better by choosing an order dynamically as the search proceeds.

- **Most constrained variable**
  when doing forward-checking, pick variable with fewest legal values to assign next (minimise branching factor)

Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed ordering of variables & values, e.g. random order or place variables with constraints first.

You can usually do better by choosing an order dynamically as the search proceeds.

- **Most constrained variable**
  when doing forward-checking, pick variable with fewest legal values to assign next (minimise branching factor)

- **Least constraining value**
  choose value that rules out the fewest values from neighboring domains

Solving CSPs

BT with Dynamic Ordering

Traditional backtracking uses fixed ordering of variables & values, e.g. random order or place variables with constraints first.

You can usually do better by choosing an order dynamically as the search proceeds.

- **Most constrained variable**
  when doing forward-checking, pick variable with fewest legal values to assign next (minimise branching factor)

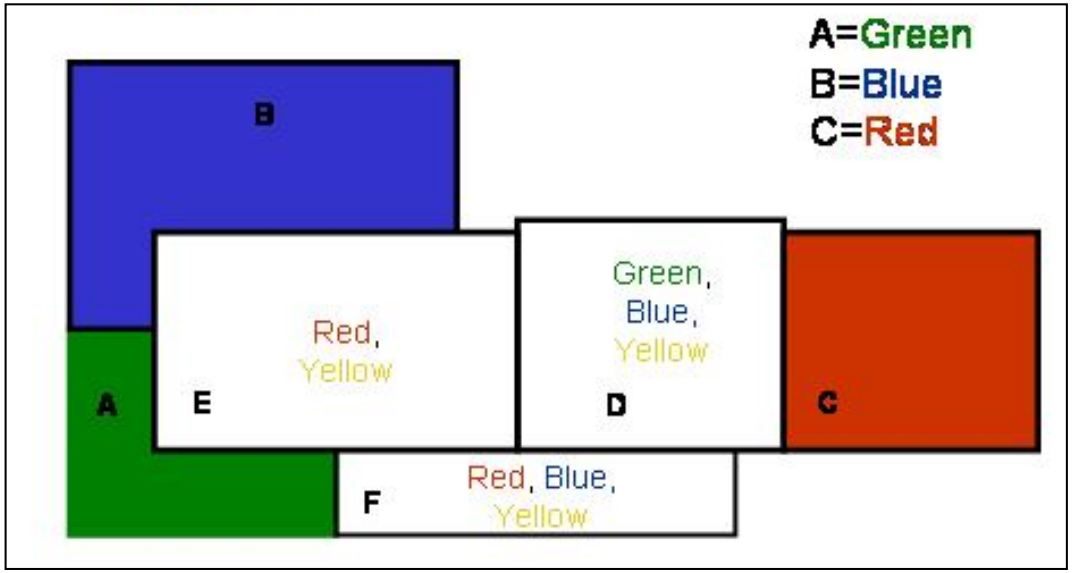
- **Least constraining value**
  choose value that rules out the fewest values from neighboring domains

e.g. This combination improves feasible N-Queens performance from about n=30 with just FC to about n=1000 with FC & ordering

Solving CSPs

Dynamic Variable and Value Ordering

Colours: R, G, B, Y



Which country should we colour next?

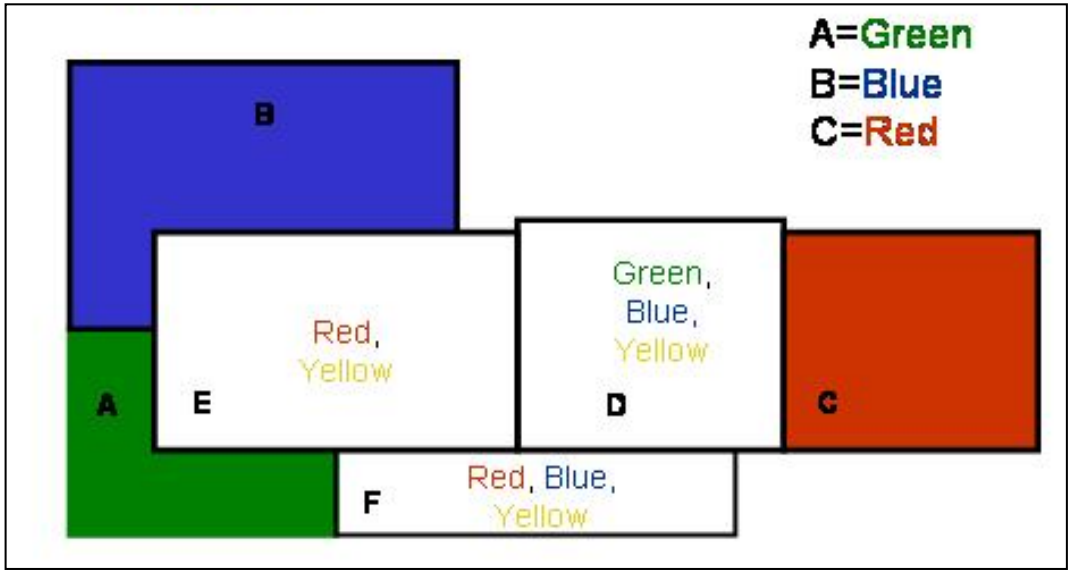
What colour should we pick for it?

The 4-Colour Map-Colouring Problem illustrates a simple situation for variable and value ordering.

Solving CSPs

Dynamic Variable and Value Ordering

Colours: R, G, B, Y



The 4-Colour Map-Colouring Problem illustrates a simple situation for variable and value ordering.

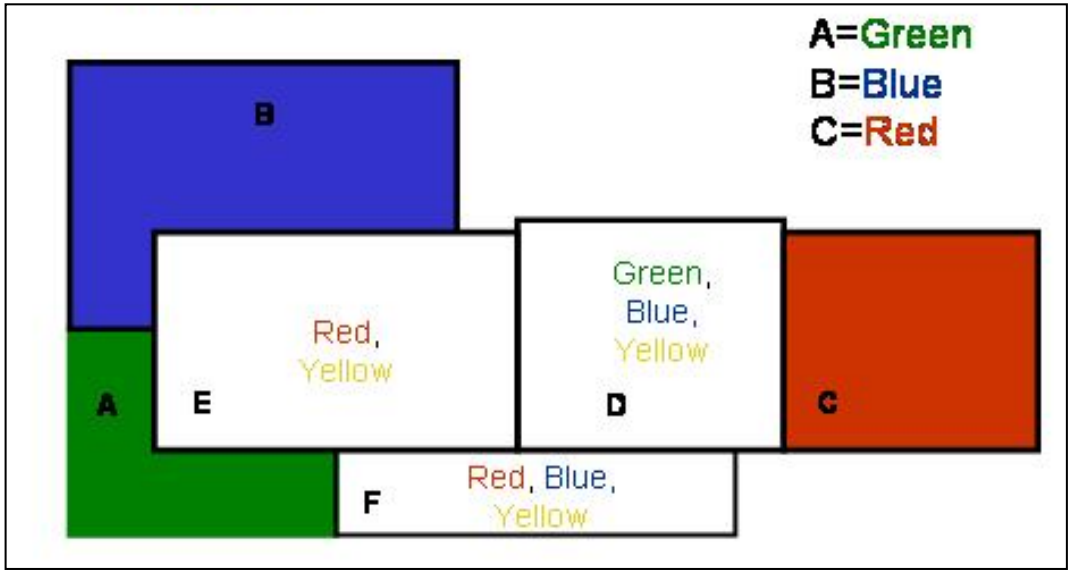
Which country should we colour next? ⇒ **E** is most constrained variable (smallest domain)

What colour should we pick for it?

Solving CSPs

Dynamic Variable and Value Ordering

Colours: R, G, B, Y



The 4-Colour Map-Colouring Problem illustrates a simple situation for variable and value ordering.

Which country should we colour next? → **E** is most constrained variable (smallest domain)

What colour should we pick for it? → **Red –** least constraining value (eliminates fewest values from neighboring domains)

Solving CSPs

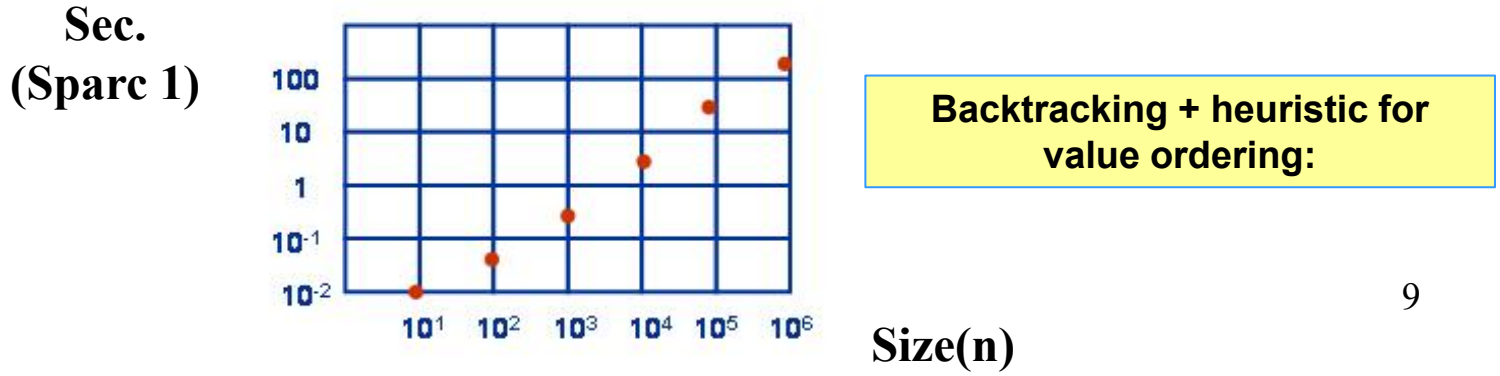
Incremental Repair (Min-Conflict Heuristic)

1. Initialise a candidate solution using "local greedy" heuristic – get solution "near" correct one.

2. Select a variable in conflict and assign it a value that minimises the number of conflicts (break ties randomly).

- Can use this heuristic as **part of systematic backtracker** that uses heuristics to do **value ordering** or in a local hill-climber (without backup).

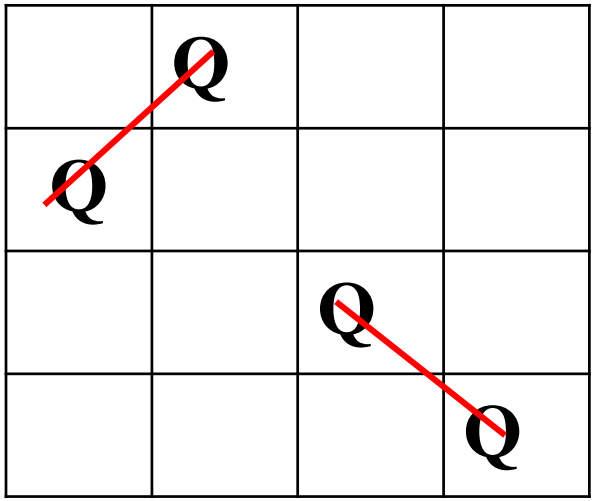
Performance on N-Queens (with ***good*** initial guess)



Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

1. Initialise a candidate solution using "greedy" heuristic – get solution "near" correct one.

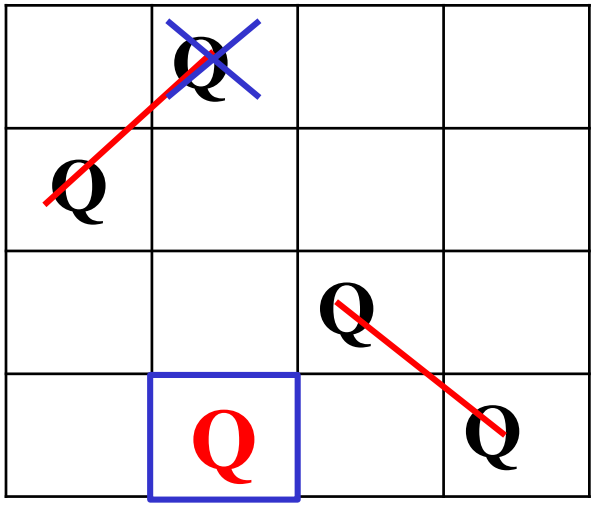


number of variables in conflict=4

Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

2. Select a variable in conflict and assign it a value that minimises the number of conflicts (break ties randomly).

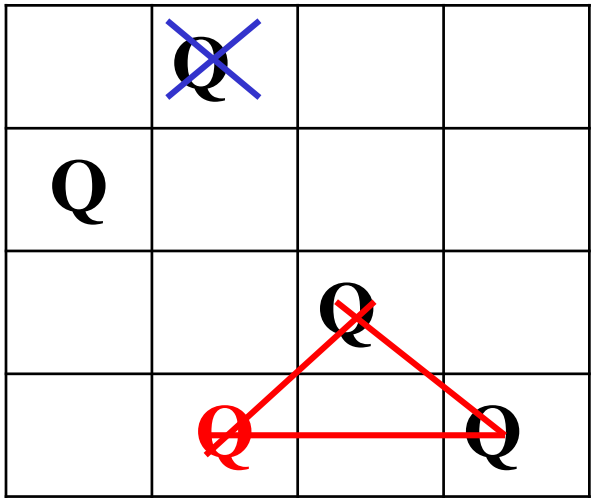


number of variables in conflict=3

Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

2. Select a variable in conflict and assign it a value that minimises the number of conflicts (break ties randomly).

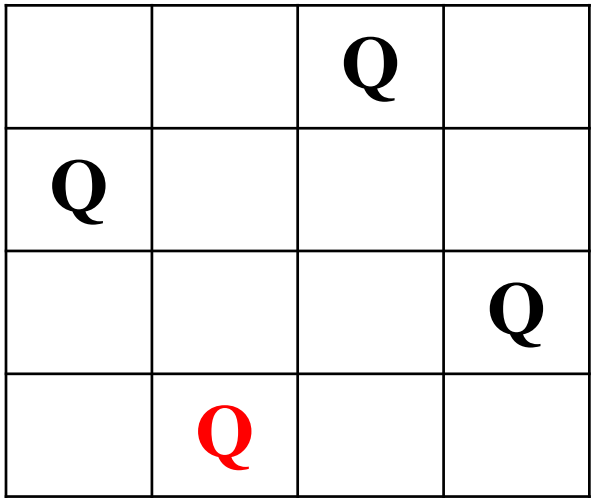


number of variables in conflict=3

Solving CSPs

Incremental Repair (Min-Conflict Heuristic)

After many trials, a solution may be found. Otherwise, re-initialise the system with another random state.



Conflicts=0

Solving CSPs

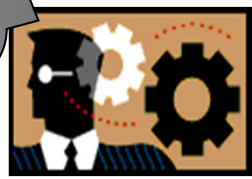
Min-Conflict Heuristic

The pure hill climber (without backtracking) can get stuck in local minima. Can add random moves to attempt getting out of minima – generally quite effective. Can also use weights on violated constraints & increase weight every cycle if it remains violated.

GSAT

- Restart the search with a new random initial state.
- Randomised hill-climber used to solve SAT problems. One of the most effective methods ever found for this problem.

GSAT can solve SAT problems of mind-boggling complexity. It has set a new standard for classifying SAT problems as "*hard*", because almost any random problem is "easy" for GSAT.



Solving CSPs

Min-Conflict Heuristic

GSAT

Example

Clauses

$A \vee \neg B \vee C$

$\neg A \vee C \vee D$

$B \vee D \vee \neg E$

$\neg C \vee \neg D \vee \neg E$

$\neg A \vee \neg C \vee E$

AND

Clauses are combined together using an AND logical connective.

Solving CSPs

GSAT as Heuristic Search

State Space: Space of all full assignments to variables

Initial State: a random full assignment

Goal State: a satisfying assignment

Actions: flip value of one variable in current assignment

Heuristic: the number of satisfied clauses (constraints); we want to maximise this score. Alternatively, minimise the number of unsatisfied clauses (constraints).

Solving CSPs

Algorithm: GSAT(F)

- **For i=1 to MaxTries**
  - Select a complete random assignment **A**
  - *Score* = number of satisfied clauses
  - **For i=1 to MaxFlips**
    - If (**A** satisfies all clauses in **F**) {
      return **A**
      }
    - Else {
      Flip a variable that maximises the *Score*
      }
    - Flip a randomly chosen variable if no variable flip increases the *Score*

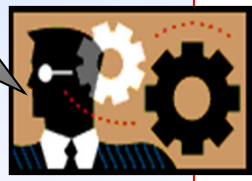
MaxTries and **MaxFlips** are user-defined. These guard against **local minima** in the search.

Solving CSPs

Algorithm: WALKSAT(F)

- **For i=1 to MaxTries**
  - Select a complete random assignment **A**
  - *Score* = number of satisfied clauses
  - **For i=1 to MaxFlips**
    - If (**A** satisfies all clauses in **F**) {
      return **A**
      }
    - Else {
      - **With probability p** //**GSAT**
        - Flip a variable that maximises the *Score*
        - Flip a randomly chosen variable if no variable flip increases the Score
      - **With probability (1-p)** //**Random Walk**
        - Pick a random unsatisfied clause C
        - Flip a randomly chosen variable in C
      }

It turns out that adding **more randomness** is a more effective strategy!

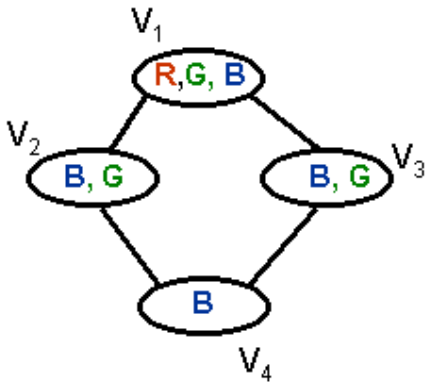


https://www.cs.rochester.edu/u/kautz/walksat/

Tutorial #9: Constraint Satisfaction Problem

Simple Map Colouring Problem

Let's look at a simple map coloring problem and explore the behavior of constraint propagation, backtracking and forward checking in this context. Here is the relevant problem specification:



1. The type of graph shown above is known as a [ ] graph for the CSP.
2. Write the values in each of the indicated variable domains **after** any changes required to achieve arc consistency for just that arc. Then, assume that the following arcs are done sequentially, with the effects on the domains propagating. Enter domains as a sequence of letters, for example, `R B`, do not include parenthesis, do not spell out colour names, leave spaces between the letters. If there are no values left in a domain, enter `none`.

V_1 - V_2: D_1= [ ] D_2= [ ]
V_1 - V_3: D_1= [ ] D_3= [ ]
V_2 - V_4: D_2= [ ] D_4= [ ]
V_3 - V_4: D_3= [ ] D_4= [ ]
V_1 - V_2: D_1= [ ] D_2= [ ]
V_1 - V_3: D_1= [ ] D_3= [ ]

3. Assume we use pure backtracking to search for a solution to this problem. We use a fixed variable ordering in the search (V_1, V_2, V_3 and V_4) and the values are considered in the order shown in the picture above. Then, show the order in which assignments are considered by **Backtracking** (this is analogous to the order in which nodes are expanded by depth-first search). Show the assignment even if it is immediately found to be inconsistent upon testing. Assume that the search continues even after finding a valid solution. Enter each assignment (one assignment in each of the spaces below) as a number from 1 to 4 (indicating the variable) followed by a letter, drawn from `R, G, B`, leave a space between them, e.g. `1 R`:
   a. [ ]
   b. [ ]
   c. [ ]
   d. [ ]
   e. [ ]

N.H.Reyes
Source: MIT OpenCourseware

f.
g.
h.
i.
j.

4. Repeat assuming we use **Backtracking with Forward Checking** to search for a solution to this problem. We use the same variable ordering and value ordering as before. Show the order in which assignments are considered by BT-FC. Whenever propagating after an assignment causes a domain to become empty, that causes backup in the search. Assume that the search continues even after finding a valid solution. Enter each assignment (one assignment in each of the spaces below) as before:

a.
b.
c.
d.
e.
f.
g.
h.
i.
j.

CSP Overview

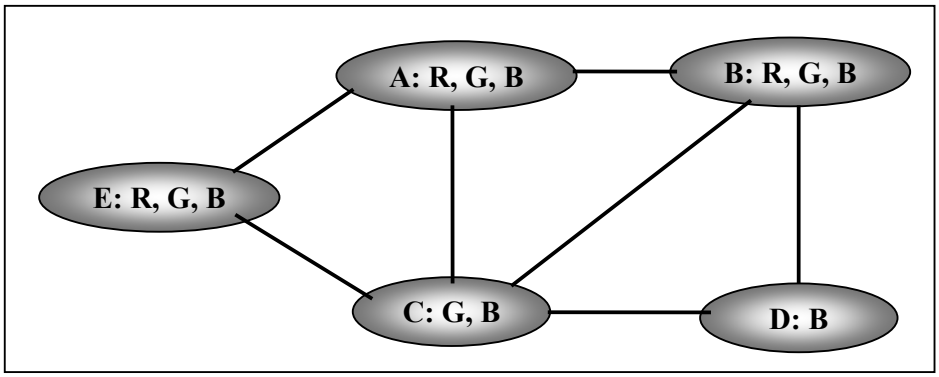
Indicate whether the following statements are true or false.

1. Constraint propagation without search is sufficient to find a satisfying set of assignments in most problems.
2. If constraint propagation leaves all variables with non-empty domains, there is at least one solution.
3. If constraint propagation leaves some variable with an empty domain, there is no solution.
4. If constraint propagation leaves all variables with singleton domains, there is a unique solution.
5. Constraint propagation takes on the order of ed^2 arc tests for one pass through all the arcs in a constraint graph with e edges and d values per domain.
6. Forward checking during backtracking requires verifying that the newly assigned value for a variable is consistent with all previously assigned values of variables.

7. Backtracking search with full constraint propagation (until arc-consistency is reached) generally results in an answer using fewer arc-tests than search with forward-checking only.
8. Backtracking with Forward Checking (BT-FC) never explores more possible variable values than pure backtracking.
9. BT-FC never tests more constraint arcs than pure backtracking.

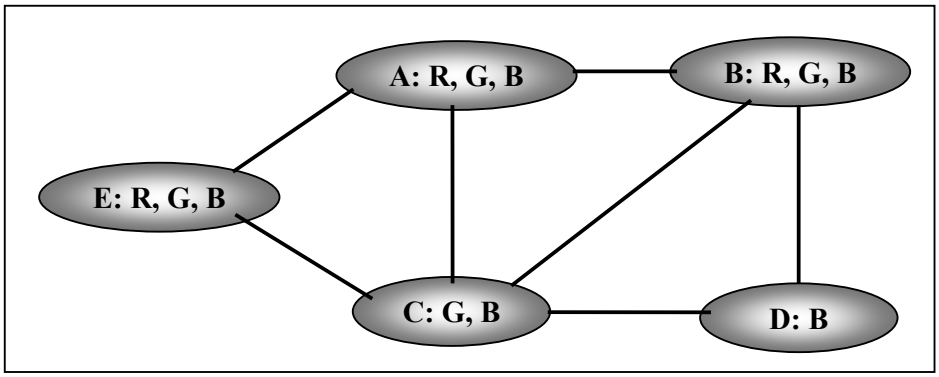
CSP Problems

Consider a map colouring problem consisting of five regions A, B, C, D and E. Your task is the standard map colouring problem. You are to assign each region a colour such that no two adjacent regions have the same colour. The legal colours are a subset of R, G and B, standing for Red, Green and Blue, respectively. The set of regions are depicted in the following constraint graph as ovals. The legal colours for each region is specified within its corresponding oval, and each pair of adjacent regions is denoted by a line between the two ovals corresponding to the two regions.



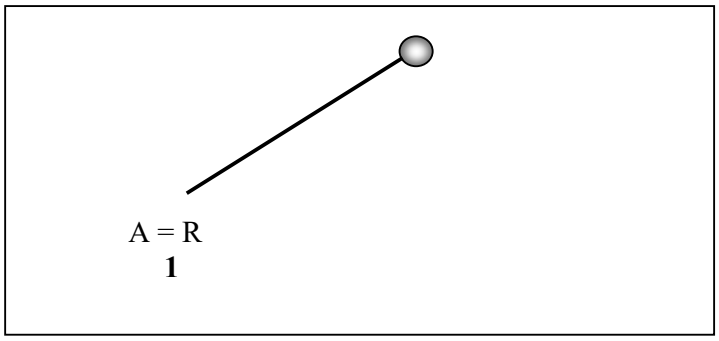
Part A. Pruned Domains

First, consider what domain elements can be eliminated using constraint propagation. Show your results on the copy of the constraint graph shown in the box below. On the graph, cross out each domain element that is eliminated by constraint propagation. For example, to start you off, we have eliminated **B**(lue) from region **B**, which is eliminated using constraint **B-D**.



Part B. Backtrack Search with Forward Checking

In this part you will use two CSP search methods to search for the first consistent solution to the map colouring problem described above. The two search methods you will use are **backtrack search** and **backtrack search with forward checking.** Use the same constraint graph.
Your task is to draw the search tree resulting from the application of each method, stopping when you reach the first solution, or the end of the search tree if no solution exists. In your tree you will label each node (except the root) with the variable/value assignment made at that point in the search (e.g. A = R). In addition, indicate the order in which assignments are made by writing a number under each assignment in your search tree. To get you going, we have started your first search tree below by indicating the root, and the first child expanded (labelled with assignment A=R and step number 1).



B.1. Simple Backtrack Search

Draw the search tree for simple backtrack search (no forward checking) in the box below. In your search, variables, denoting regions are to be ordered **alphabetically**, and colours are to be ordered **R**, then **G**, and finally **B**. Note that the full constraint propagation from Part A is **not** to be performed here.

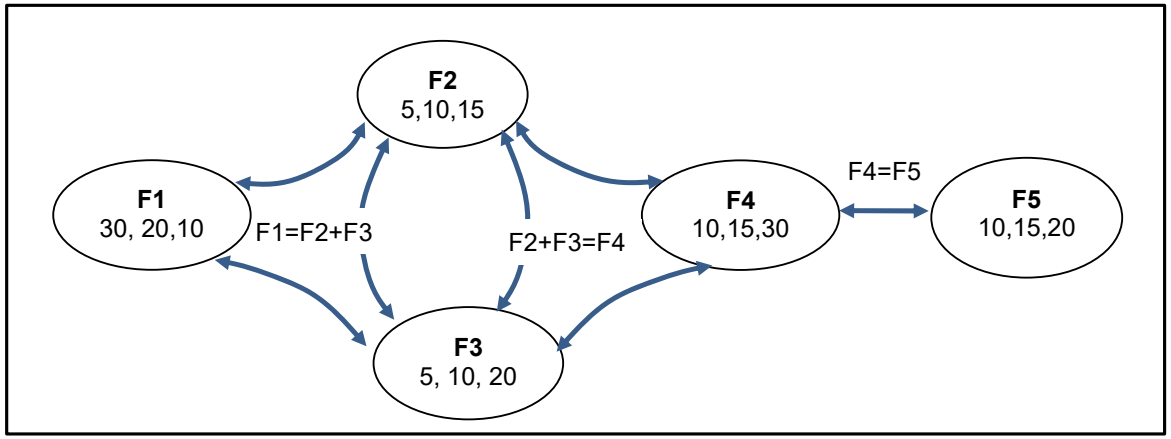
B.2. Backtrack Search with Forward Checking

Draw the search tree for backtrack search with forward checking in the box below. In your search, variables, denoting regions are to be ordered **alphabetically**, and colours are to be ordered **R**, then **G**, and finally **B**. Note that the full constraint propagation from Part A is **not** to be performed here.

B.3. Backtrack Search with Dynamic Variable Ordering

Draw the search tree for backtrack search with dynamic variable ordering in the box below. Do not perform forward checking or full-constraint propagation. Select the variable to assign at each step based on the most constrained variable heuristic. Select the value of the variable to be assigned based on the least constrained value heuristic. We have not specified the assignment for the first step in this tree.

Sensor Interpretation Problem (Maximum Flow)



CONSTRAINT GRAPH

Given a constraint graph representing a network capacity problem, solve for maximum flow in the network.

Note that there are two constraints among three variables. These are indicated by the arcs that come together.

In this constraint graph, the variables represent the integer values of flow through each node. The domains are the possible values of flow through each node. These are represented by the integer values in each node.

A device for measuring flow was used to obtain an estimate of flow that passes through each node; however, you are not sure how to interpret the readings correctly. The possible values are indicated by the domain of each variable.

Your task is to apply the **Backtracking with Forward-Checking** (BT-FC) algorithm, to solve for the maximum flow in the network.

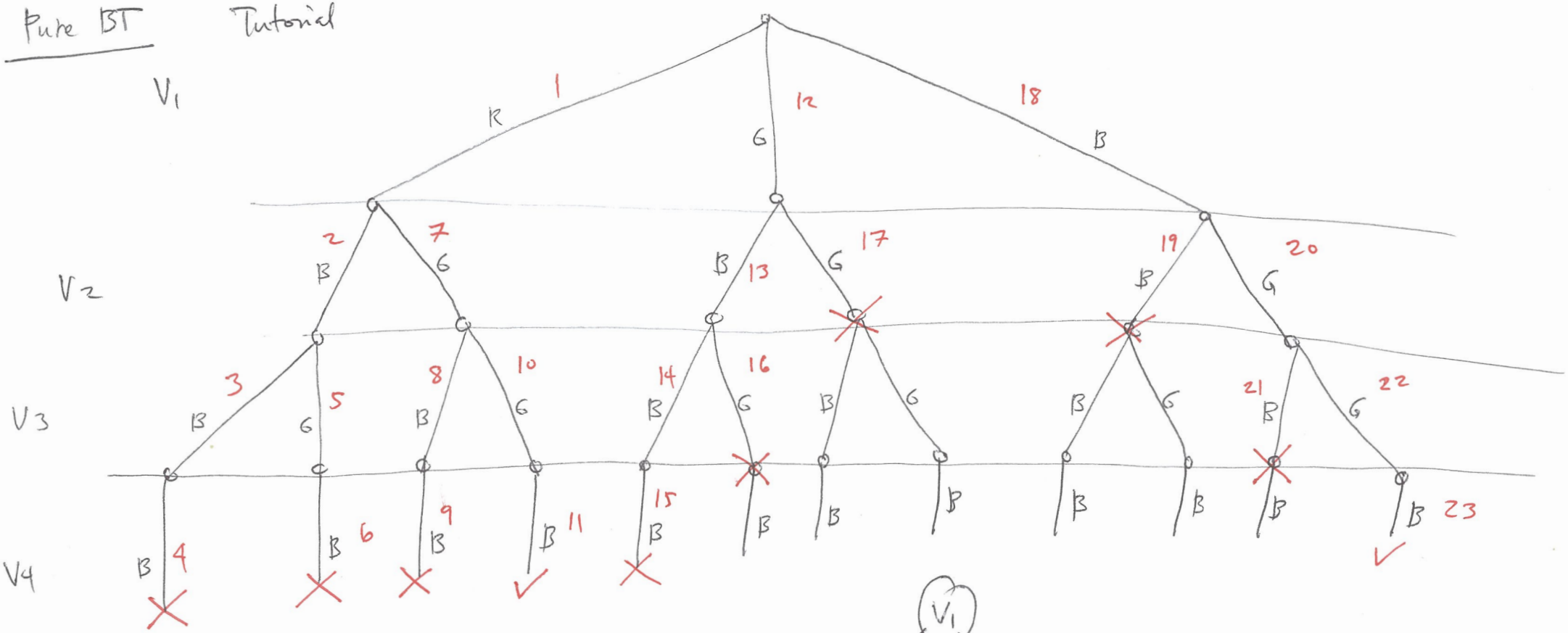
BI

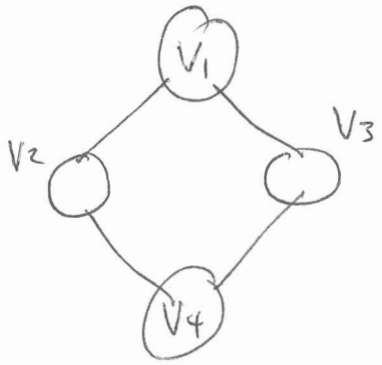
Tutorial #9

Part 1

1) 1R
2) 2B
3) 3B
4) 4B
5) 3G
6) 4B
7) 2G
8) 3B
9) 4B
10) 3G
11) 4B
12) 1G
13) 2B
14) 3B
15) 4B
16) 3G
17) 2G
18) 1B
19) 2B
20) 2G
21) 3B
22) 3G
23) 4B

Pure BT Tutorial





V_1 = R, G, B

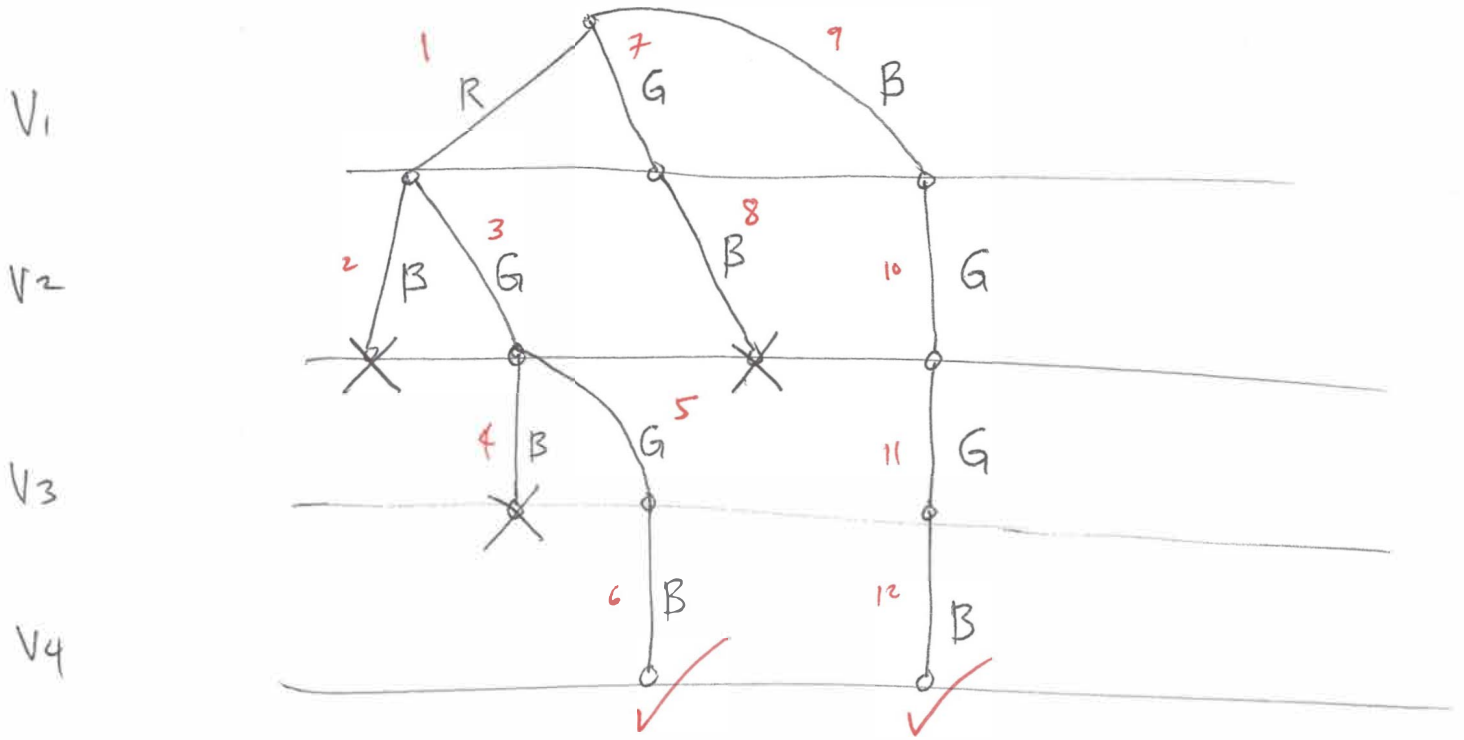
V_2 = B, G

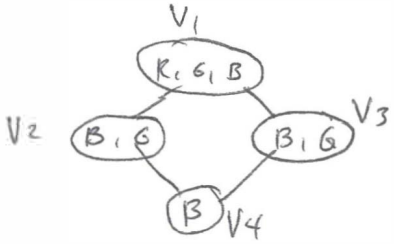
V_3 = B, G

V_4 = B

Tutorial

BT-FC





BT-FC

1. $V_1 = \textcircled{R}\ G\ B$
   $V_2 = B\ G$
   $V_3 = B\ G$
   $V_4 = B$

2. $V_1 = \textcircled{R}\ G\ B$
   $V_2 = \textcircled{B}\ G$
   $V_3 = B\ G$
   $V_4 = \cancel{B}$ empty
   no soln.

3. $V_1 = \textcircled{R}\ G\ B$
   $V_2 = \textcircled{G}$
   $V_3 = B, G$
   $V_4 = B$

4. $V_1 = \textcircled{R}\ G\ B$
   $V_2 = \textcircled{G}$
   $V_3 = \textcircled{B}\ G$
   $V_4 = \cancel{B}$ empty
   no soln.

5. $V_1 = \textcircled{R}\ G\ B$
   $V_2 = \textcircled{G}$
   $V_3 = \textcircled{G}$
   $V_4 = B$

6. $V_1 = \textcircled{R}\ G\ B$
   $V_2 = \textcircled{G}$
   $V_3 = \textcircled{G}$
   $V_4 = \textcircled{B}$
   ✓ 1 soln. found

7. $V_1 = \textcircled{G}\ B$
   $V_2 = B\ \cancel{G}$
   $V_3 = B\ \cancel{G}$
   $V_4 = B$

8. $V_1 = \textcircled{G}\ B$
   $V_2 = \textcircled{B}$
   $V_3 = B$
   $V_4 = \cancel{B}$ empty
   no soln.

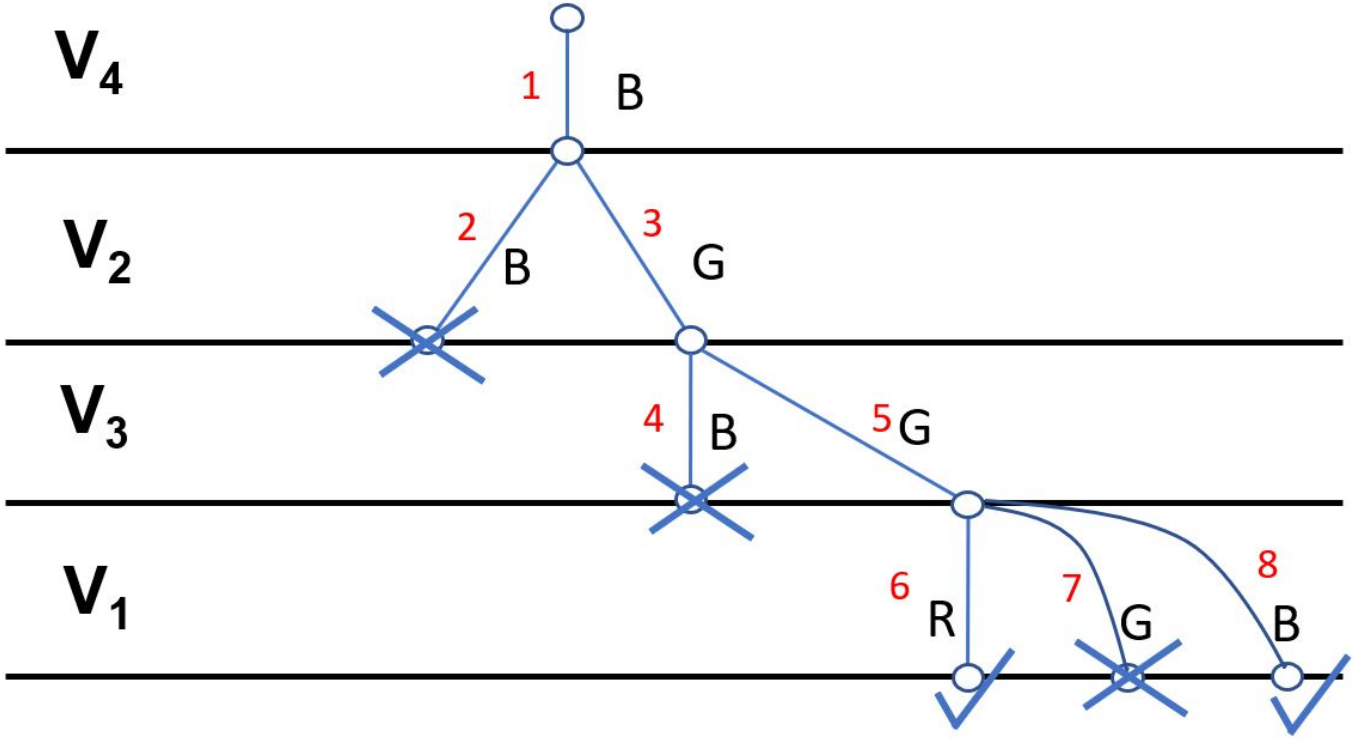
9. $V_1 = \textcircled{B}$
   $V_2 = \cancel{B}, G$
   $V_3 = \cancel{B}, G$
   $V_4 = B$

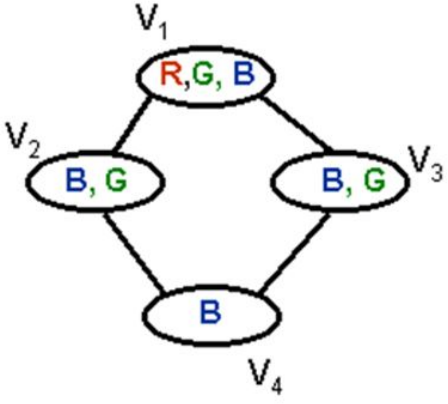
10. $V_1 = \textcircled{B}$
    $V_2 = \textcircled{G}$
    $V_3 = G$
    $V_4 = B$

11. $V_1 = \textcircled{B}$
    $V_2 = \textcircled{G}$
    $V_3 = \textcircled{G}$
    $V_4 = B$

12. $V_1 = \textcircled{B}$
    $V_2 = \textcircled{G}$
    $V_3 = \textcircled{G}$
    $V_4 = \textcircled{B}$
    ✓ 2nd solution found

Tutorial: Backtracking with Dynamic Variable Ordering





BT – Dynamic Variable Ordering

0. MCV ordering V_4, V_2, V_3, V_1

1. $V_4 = B$

2. $V_2 = B$ $V_2 = G$

$V_1 = R, G, B$ $V_1 = R, G, B$

rules out 1 rules out 1 value

$V_2 = B$, conflicts with $V_4 = B$

3. $V_2 = G$

4. V3 = B V3 = G

V1 = R, G, B V1 = R, G, B

Rules out 1 Rules out 1

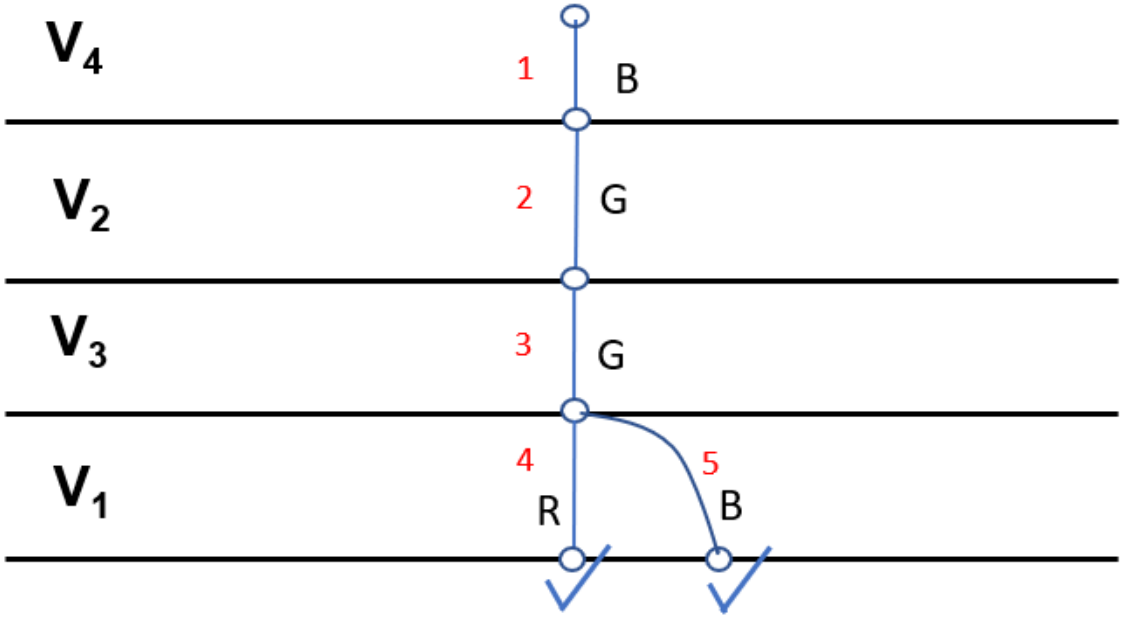
5. V3 = G

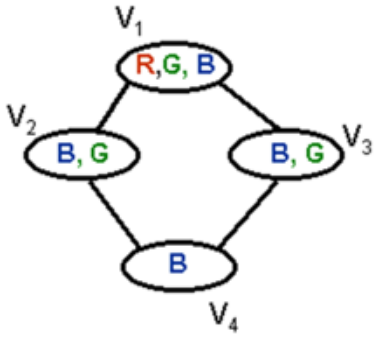
6. V1 = R, G, B

7. V1 = G, B

8. V1 = B

Tutorial: Backtracking with Forward Checking and Dynamic Variable Ordering





BT-FC Dynamic Variable Ordering

0. MCV Ordering : V_4, V_2, V_3, V_1

1. $V_4 = \{B\}$

   $V_2 = \cancel{B}, G$

   $V_3 = \cancel{B}, G$

   $V_1 = R, G, \cancel{B}$

2. $V_4 = \{B\}$

   $V_2 = \{G\}$

   $V_3 = G$

   $V_1 = R, \cancel{G}, B$

3. $V_4 = \{B\}$

   $V_2 = \{G\}$

   $V_3 = \{G\}$

   $V_1 = R, B$

4. $V_4 = \{B\}$

   $V_2 = \{G\}$

   $V_3 = \{G\}$

   $V_1 = \{R\}, B$

1st solution found!

5. $V_4 = \{B\}$

   $V_2 = \{G\}$

   $V_3 = \{G\}$

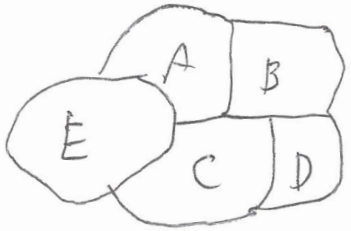
   $V_1 = \{B\}$

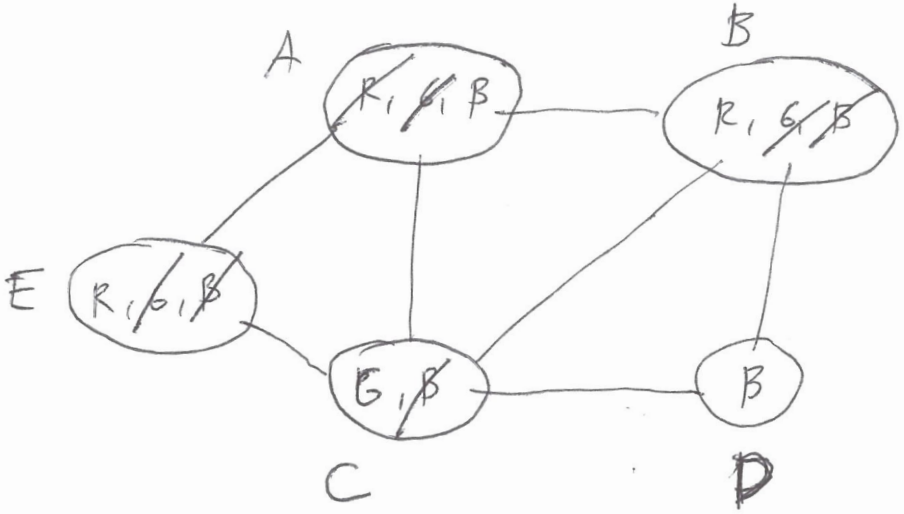
2nd solution found!

CSP Overview

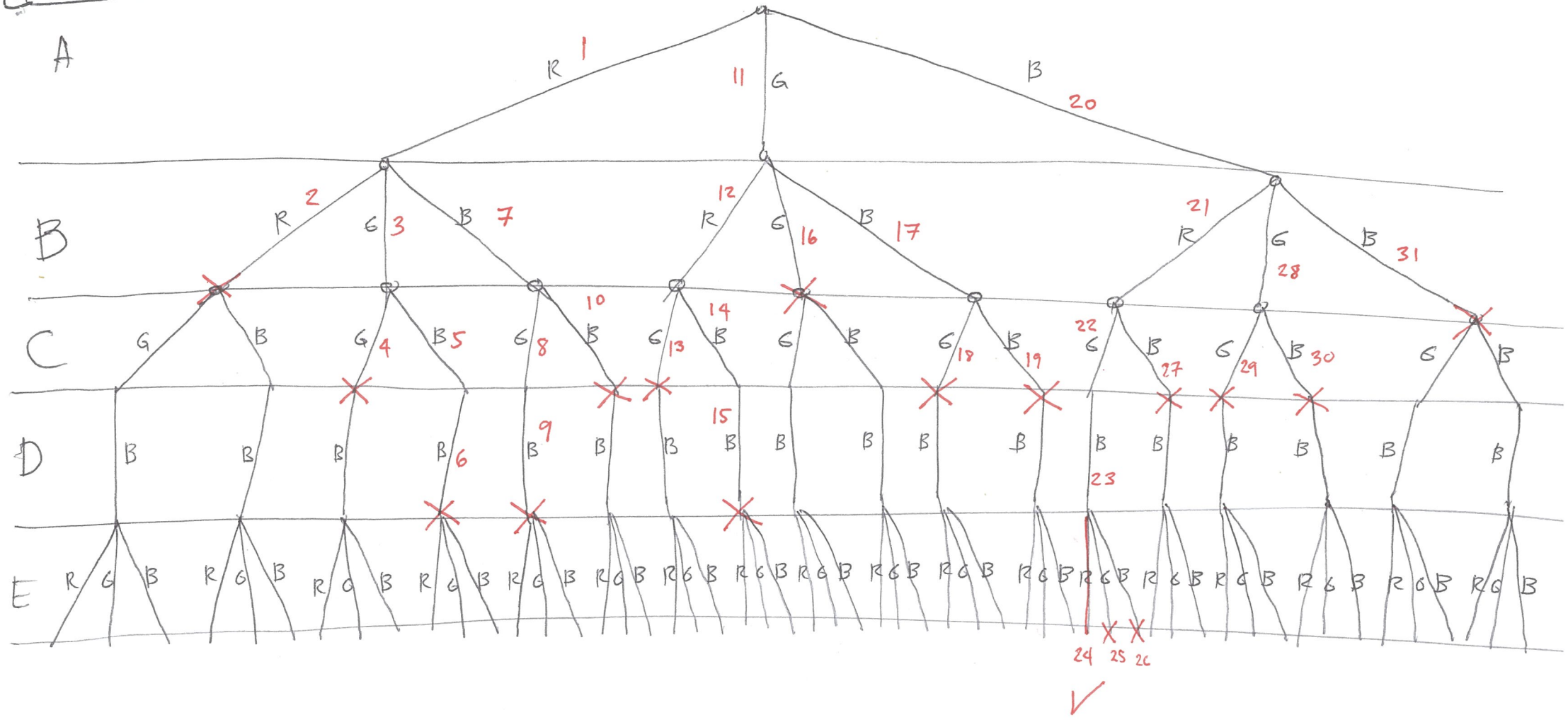
1. F
2. F
3. T
4. T
5. T
6. F
7. F
8. T
9. +

Constraint Propagation



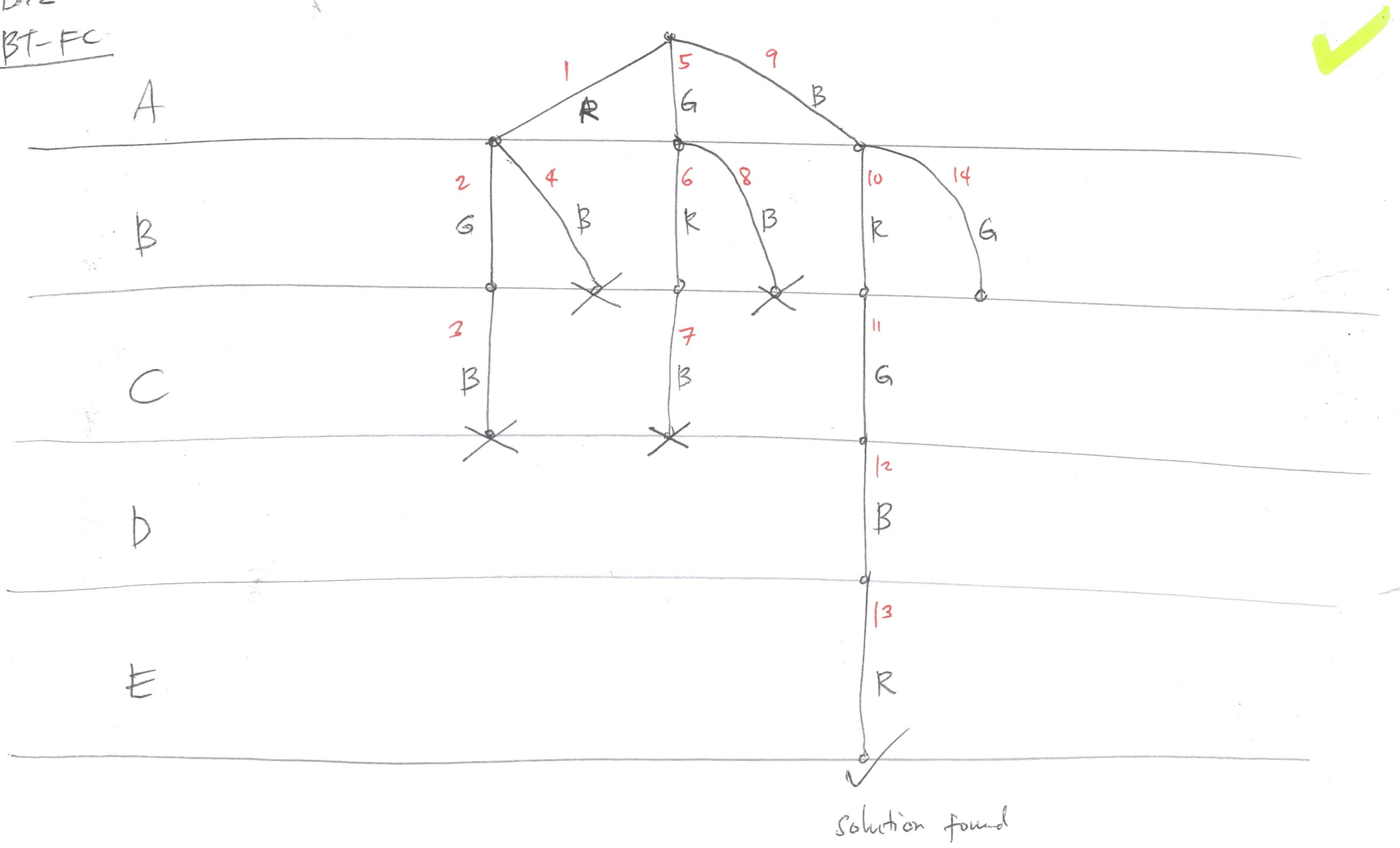


B.1 Pure BT


A
B
C
D
E
R
G
B
1
2
3
7
11
12
16
17
20
21
28
31
10
14
4
5
8
13
18
19
22
27
29
30
6
9
15
23
24
25
26

B.2

BT-FC


A
B
C
b
E
1
R
5
G
9
B
2
G
4
B
6
R
8
B
10
R
14
G
3
B
7
B
11
G
12
B
13
R
solution found

BT-FC

1. → A = R, G, B
   • B = ~~R~~, G, B
   • C = G, B
   D = B
   • E = ~~R~~, G, B

2. A = R, G, B
   → B = G, B
   • C = ~~G~~, B
   • D = B
   E = G, B

   A = R, G, B

3. B = G, B
   → C = B
   • D = ~~B~~ empty
   • E = G, ~~B~~

   ×

Backtrack to B

4. A = R, G, B
   → B = G, B
   • C = G, ~~B~~
   • D = ~~B~~ empty
   E = G, B

   ×

5. Backtrack to A
   → A = G, B
   • B = R, ~~G~~, B
   • C = ~~G~~, B
   D = B
   • E = R, ~~G~~, B

6. A = G, B
   → B = R, B
   • C = B
   • D = B
   E = R, B

7. A = G, B
   B = R, B
   → C = B
   • D = ~~B~~ empty
   • E = R, ~~B~~

   ×

8. Backtrack to B
   A = G, B
   → B = B
   • C = ~~B~~ } empty
   • D = ~~B~~ }
   E = R, B

   ×

9. Backtrack to A
   → A = B
   • B = R, G, ~~B~~
   • C = G, ~~B~~
   D = B
   • E = R, G, ~~B~~

10. A = B
    → B = R, G
    • C = G, ~~B~~
    • D = B
    E = R, G

    A = B

11. B = R, G
    → C = G
    • D = B
    • E = R, ~~G~~

    A = B

12. B = R, G
    C = G
    → D = B
    E = R

13. A = B
    B = R, G
    C = G
    D = B
    → E = R

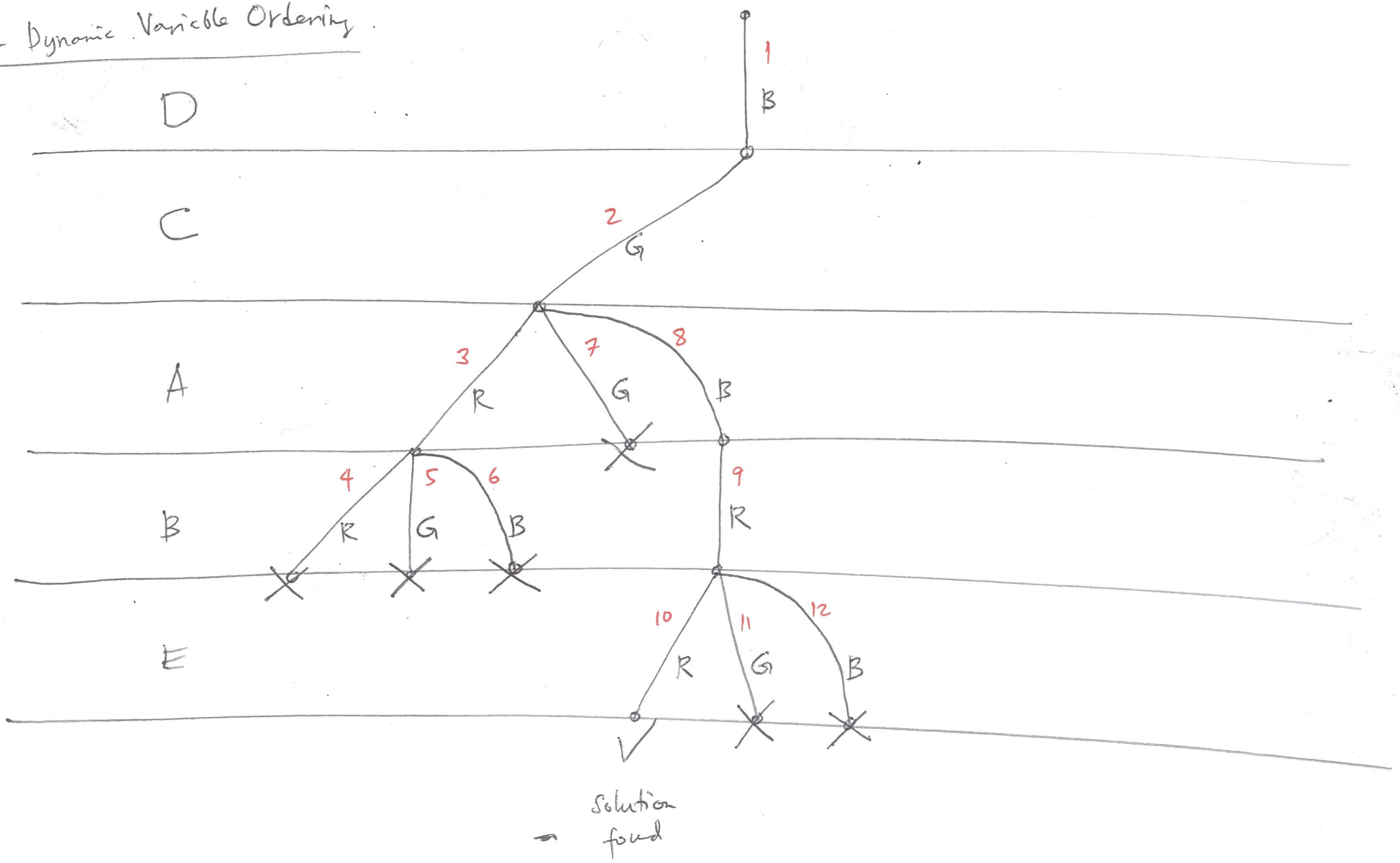
(optional)

14. Backtrack to B
    A = B
    → B = G
    • C = ~~G~~ empty
    • D = B
    E = R, G

    ×

B.3.

BT - Dynamic Variable Ordering



BT – Dynamic Variable Ordering

0. Most Constrained Variable:

D, C, A, B, E

1. D = (B)

2.

C = G	C = B
A = R, ~~G~~, B	A = R, G, ~~B~~
B = R, ~~G~~, B	B = R, G, ~~B~~
E = R, ~~G~~, B	E = R, G, ~~B~~
rules out 3	rules out 3

(pick the value that rules out the least no. of elements from neighbouring domains)

we have a tie, so pick the one that comes first from the left

C = (G)

3.

A = (R)	A = G	A = B
B = ~~R~~, G, B	B = R, ~~G~~, B	B = R, G, ~~B~~
E = ~~R~~, G, B	E = R, ~~G~~, B	E = R, G, ~~B~~
rules out 2	2	2

4.

B = (R)	B = G	B = B
× constraint violation		

5. B = (G) × constraint violation

6. B = (B) × constraint violation

7. Backtrack to A

A = (G) × constraint violation

8. A = B

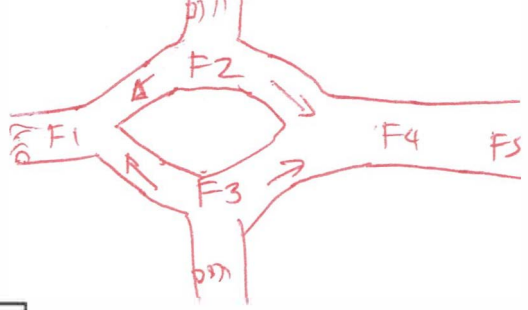
9. B = R

10. E = R

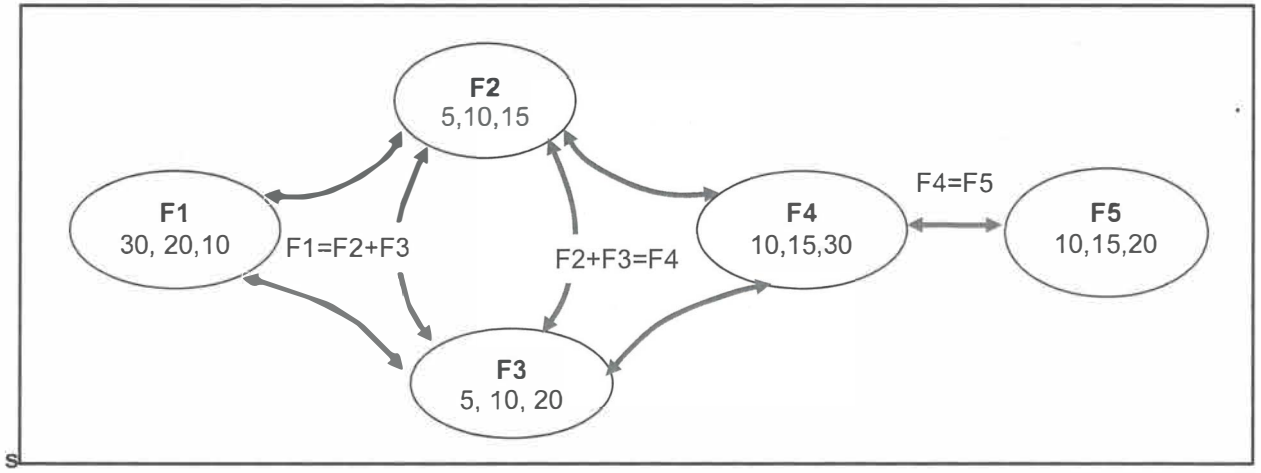
11. ~~E~~ = G ×

12. E = B

Tutorial: Constraint Satisfaction Problem (ANSWERS)



Sensor Interpretation Problem (Maximum Flow)



CONSTRAINT GRAPH

Given a constraint graph representing a network capacity problem, solve for maximum flow in the network.

Note that there are two constraints among three variables. These are indicated by the arcs that come together.

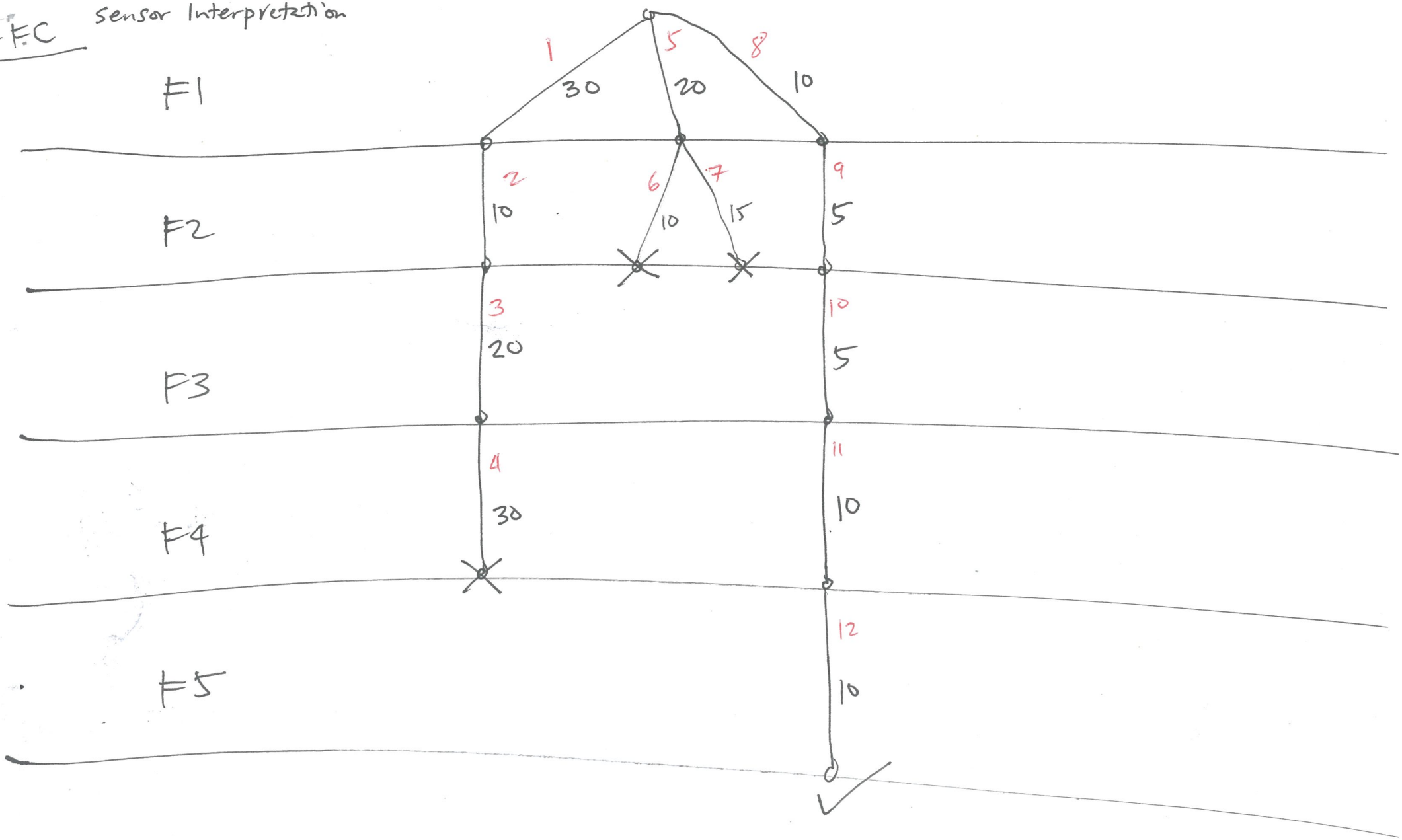
In this constraint graph, the variables represent the integer values of flow through each node. The domains are the possible values of flow through each node. These are represented by the integer values in each node.

A device for measuring flow was used to obtain an estimate of flow that passes through each node; however, you are not sure how to interpret the readings correctly. The possible values are indicated by the domain of each variable.

Your task is to apply the **Backtracking with Forward Checking** (BT-FC) algorithm, to solve for the maximum flow in the network.

BT-EC

Sensor Interpretation



BT + FC (Sensor Interpretation)

1. → F1 = (30), 20, 10
   F2 = ~~5~~, (10), ~~15~~
   F3 = ~~5~~, ~~10~~, (20)
   F4 = 10, 15, 30
   F5 = 10, 15, 20

F1 = F2 + F3

2. F1 = (30), 20, 10
   → F2 = (10)
   F3 = 20
   F4 = ~~10~~, ~~15~~, 30
   F5 = 10, 15, 20

F1 = F2 + F3
F2 + F3 = F4

propagate the effects of assigning a value to F2. This affects variables F3 & F4

3. F1 = (30), 20, 10
   F2 = (10)
   → F3 = (20)
   F4 = 30
   F5 = 10, 15, 20

F2 + F3 = F4

4. F1 = (30), 20, 10
   F2 = (10)
   F3 = (20)
   → F4 = (30)
   F5 = ~~10~~, ~~15~~, ~~20~~ empty

F2 + F3 = F4
F4 = F5

✗

5. Backtrack to F1

   → F1 = (20), 10
   F2 = ~~5~~, (10), (15)
   F3 = (5), (10), ~~20~~
   F4 = 10, 15, 30
   F5 = 10, 15, 20

F1 = F2 + F3

reset

6. F1 = (20), 10
   → F2 = (10), 15
   F3 = ~~5~~, 10
   F4 = ~~10~~, ~~15~~, ~~30~~ empty
   F5 = 10, 15, 20

F1 = F2 + F3
F2 + F3 = F4

propagate effect of assigning 10 to F2, all the way to F4 (based on the constraint given)

✗

7. Consider next value of F2

F1 = 20, 10

→ F2 = 15

F3 = 5, ~~10~~

F4 = ~~10, 15, 30~~ empty

F5 = 10, 15, 20

F1 = F2 + F3

F2 + F3 = F4

propagate effect of assigning 15 to F2. This affects F3 & F4.

8. Backtrack to F1

→ F1 = 10

F2 = 5 ~~10, 15~~

F3 = 5, ~~10, 20~~

F4 = 10, 15, 30

F5 = 10, 15, 20

F1 = F2 + F3

9.

F1 = 10

→ F2 = 5

F3 = 5

F4 = 10, ~~15~~, ~~30~~

F5 = 10, 15, 20

F1 = F2 + F3

F2 + F3 = F4

10.

F1 = 10

F2 = 5

→ F3 = 5

F4 = 10

F5 = 10, 15, 20

F1 = F2 + F3

F2 + F3 = F4

11.

F1 = 10

F2 = 5

F3 = 5

→ F4 = 10

F5 = 10, ~~15~~, ~~20~~

F2 + F3 = F4

F4 = F5

12.

F1 = 10

F2 = 5

F3 = 5

F4 = 10

→ F5 = 10

5

159.302 Games

Approaches to building two player games

n.h.reyes@massey.ac.nz

Reference: ***MIT OpenCourseWare***

Games

Board Games & Search

- **Move generation**
- **Static evaluation**
- **Min-Max**
- **Alpha-Beta**
- **Practical Matters**

1949 Shannon paper

1951 Turing paper

1958 Bernstein paper

55-60 Simon-Newell program (α-β McCarthy?)

66-67 MacHack 6 (MIT AI)

70's NW Chess 4.5

80's Cray Blitz

90's Belle, Hitech, Deep Thought, Deep Blue

Games

Game Tree Search

Initial State: initial board position and player

Operators: one for each legal move

Goal States: winning board positions

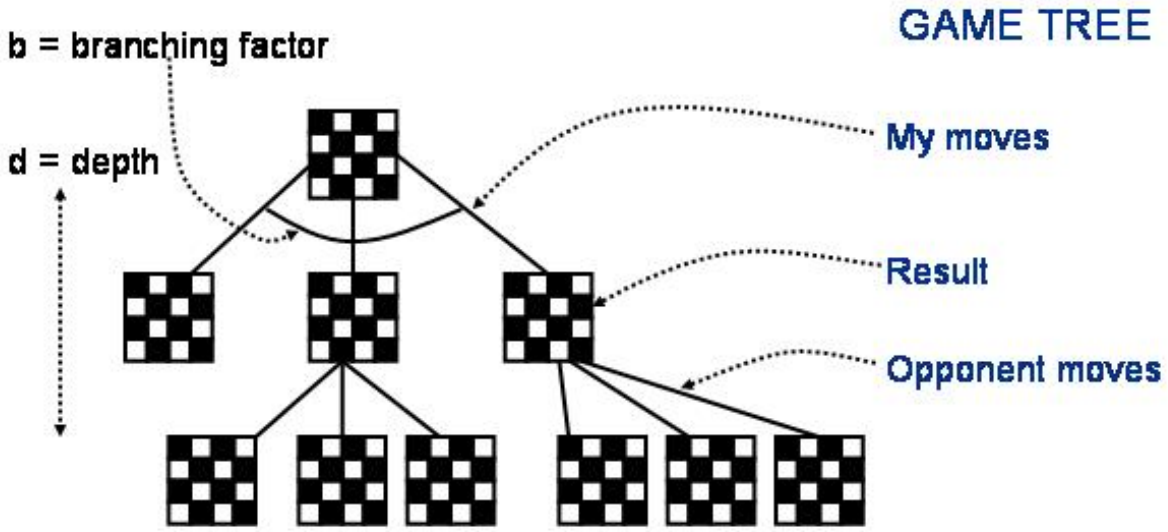
Scoring Function: assigns numeric value to states

Game tree: encodes all possible games

- We are not looking for a path, only the **next move** to make (that hopefully leads to a winning position)
- Our best move depends on what the other player does.

Games

Move Generation



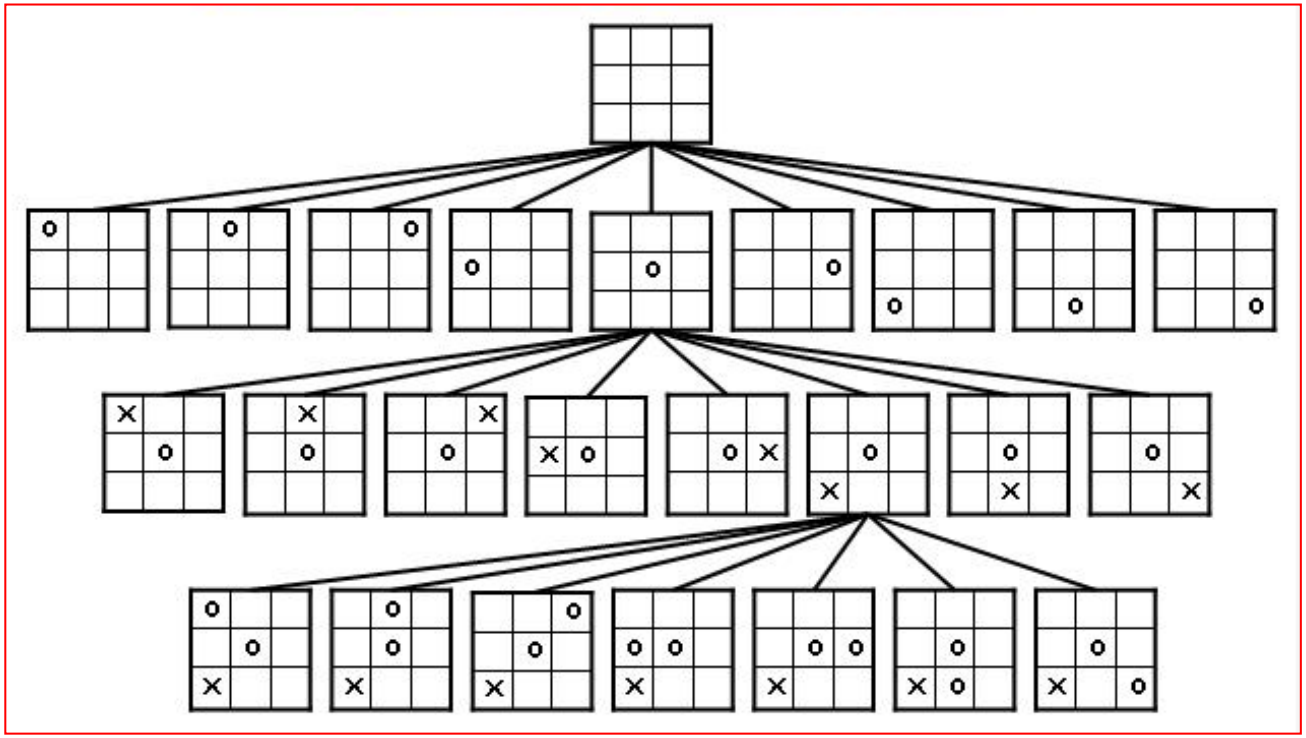
Chess (according to Google DeepMind)

$b \approx 35$

$d \approx 80$ 35^{80} is big!

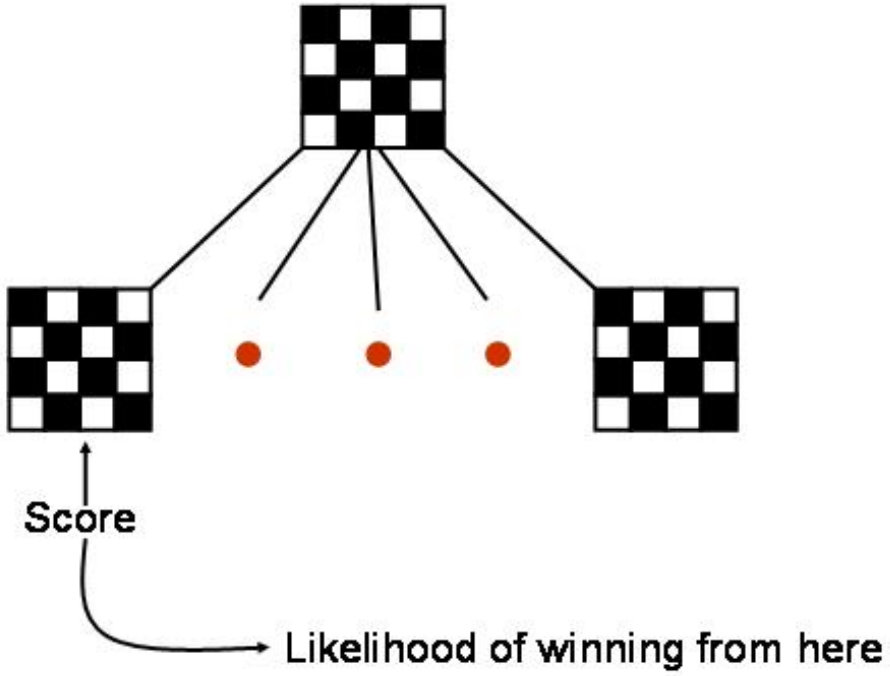
[1]Google DeepMind, 5 New Street Square, London EC4A 3TW, UK. 2Google, 1600 Amphitheatre Parkway, Mountain View, California 94043, USA.

Partial Game Tree for Tic-Tac-Toe



Even for this trivial game, the search tree is quite big.

Scoring Function



Assigns a numerical value to a board position.

Scoring Function: Static Evaluation

A linear function in which some set of coefficients is used to weight a number of "***features***" of the board position.

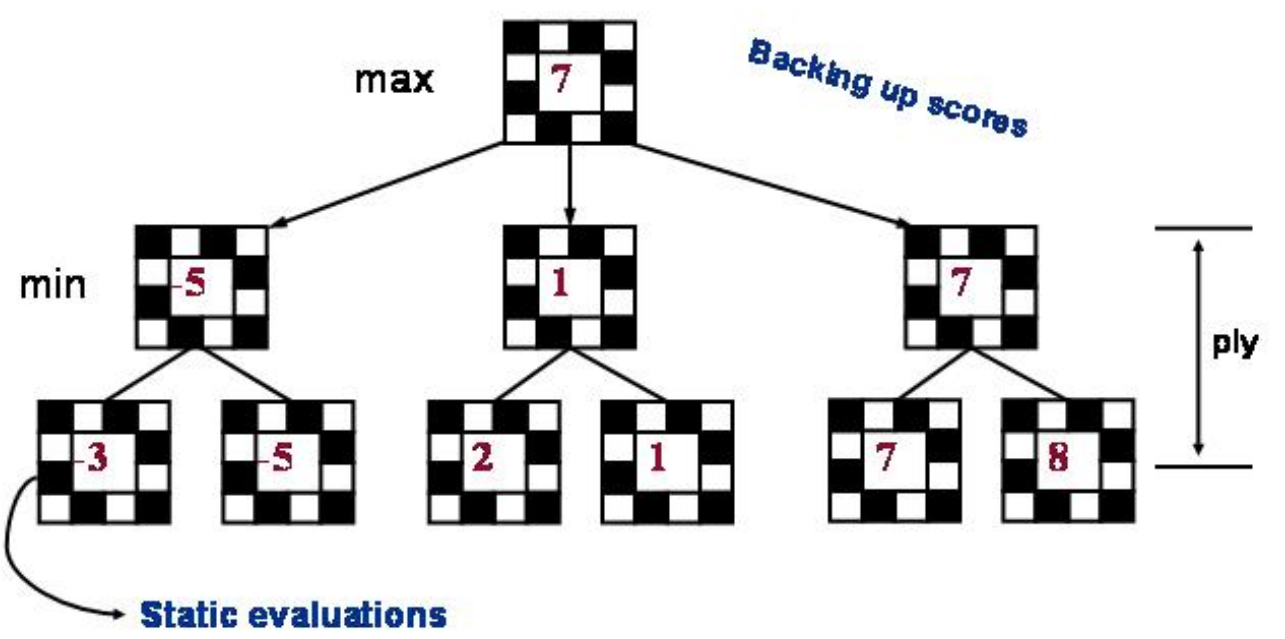
$$S = c_1 \times \text{material} + c_2 \times \text{pawn structure} + c_3 \times \text{mobility} + c_4 \times \text{king safety} + c_5 \times \text{center control} + \ldots$$

P	1
K	3
B	3.5
R	5
Q	9

Too weak to predict ultimate success.

Games

Limited look ahead + Scoring



The tree is generated on the fly.

The static evaluation is performed only on the leaf nodes of the tree.

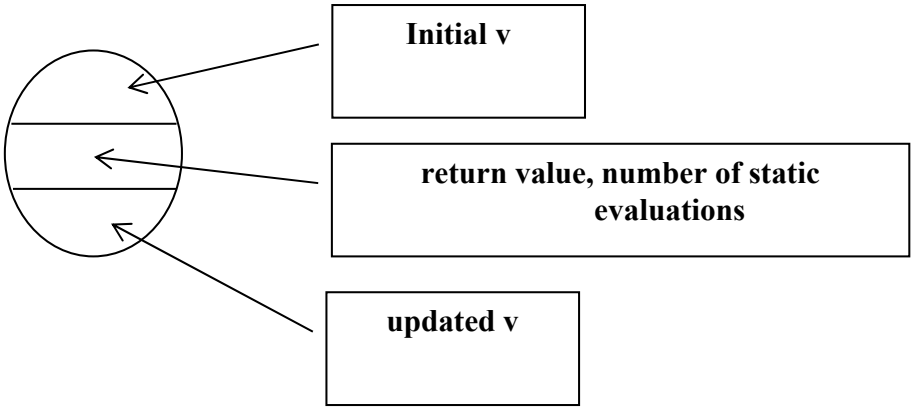
The Min-MaX Algorithm

Min-MaX Algorithm

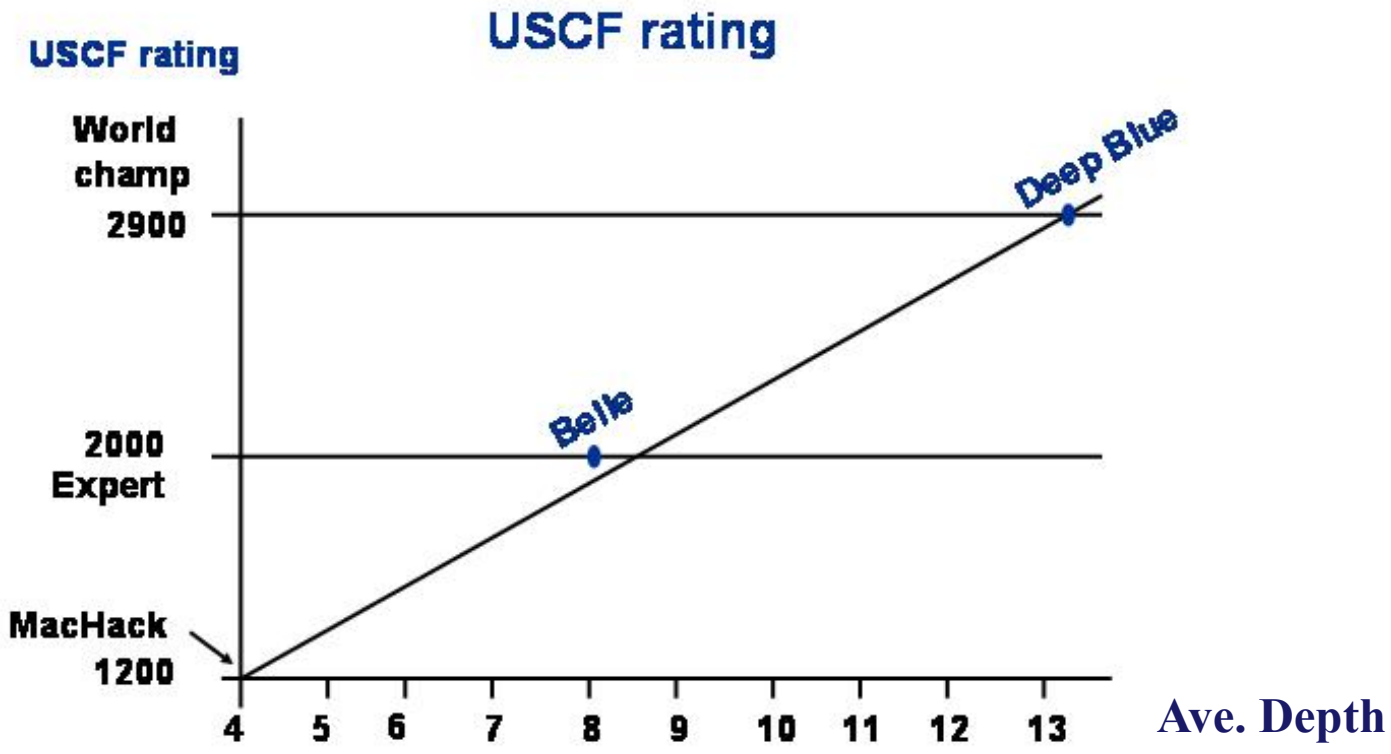
Initial call is ***MAX·VALUE****(state, MAX·DEPTH)*

- *function* ***MAX·VALUE****(state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = -∞*
  - *For each s in SUCCESSORS(state) do*
    v = MAX(v, ***MIN·VALUE****(s, depth – 1))*
  - *end*
  - *return v*

- *function* ***MIN·VALUE*** *(state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = ∞*
  - *For each s in SUCCESSORS(state) do*
    v = MIN(v, ***MAX·VALUE****(s, depth – 1))*
  - *end*
  - *return v*


Initial v
return value, number of static evaluations
updated v

USCF Rating



Somehow, it seems as if brute-force search is all that matters.

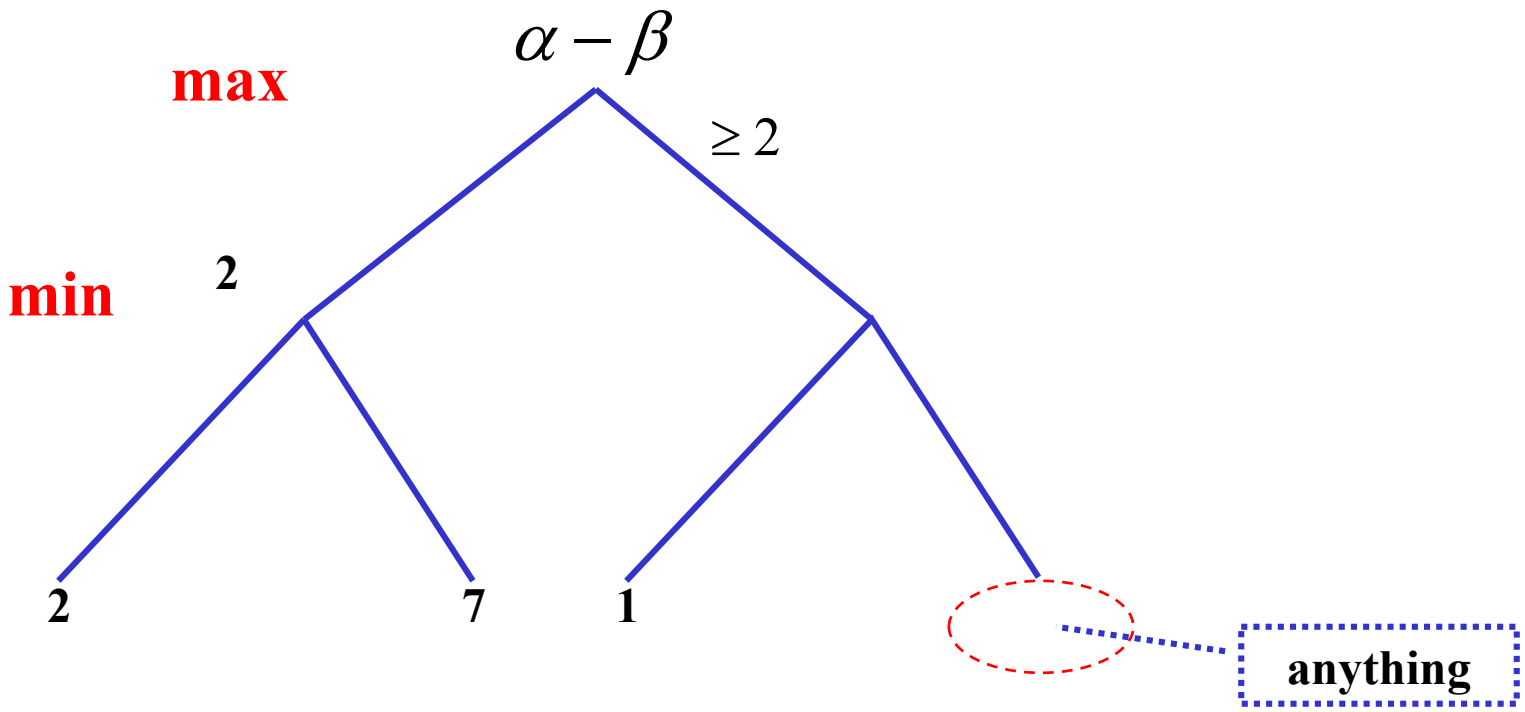
Games

Deep Blue

32 SP2 processors
each with 8 dedicated chess processors
= 256 CP

50-100 billion moves in 3 min
13-30 ply search

Alpha-Beta Pruning



α – is the lower bound on score

β – is the upper bound on score

Alpha-Beta Pruning

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***MAX·VALUE****(state, -∞, ∞, MAX·DEPTH)*

function ***MAX·VALUE****(state, α, β, depth)*

- *if (depth == 0) then return EVAL(state)*
- *For each* ***s*** *in SUCCESSORS(state) do*
  α *= MAX(α,* ***MIN·VALUE****(s, α, β, depth-1))*
  If(α ≥ β) Then return ***α*** *//cut-off*

end

return α

Maximises α, returns α

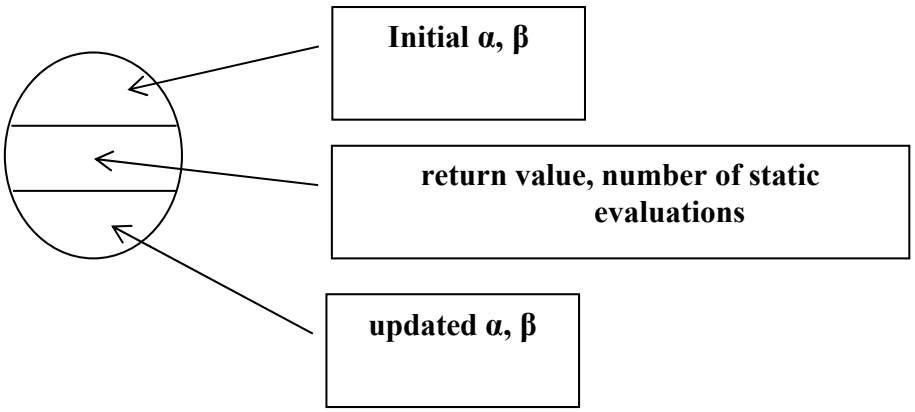
function ***MIN·VALUE****(state, α, β, depth)*

- *if (depth == 0) then return EVAL(state)*
- *For each* ***s*** *in SUCCESSORS(state) do*
  β *= MIN(β,* ***MAX·VALUE****(s, α, β, depth-1))*
  If(β ≤ α) Then return ***β*** *//cut-off*

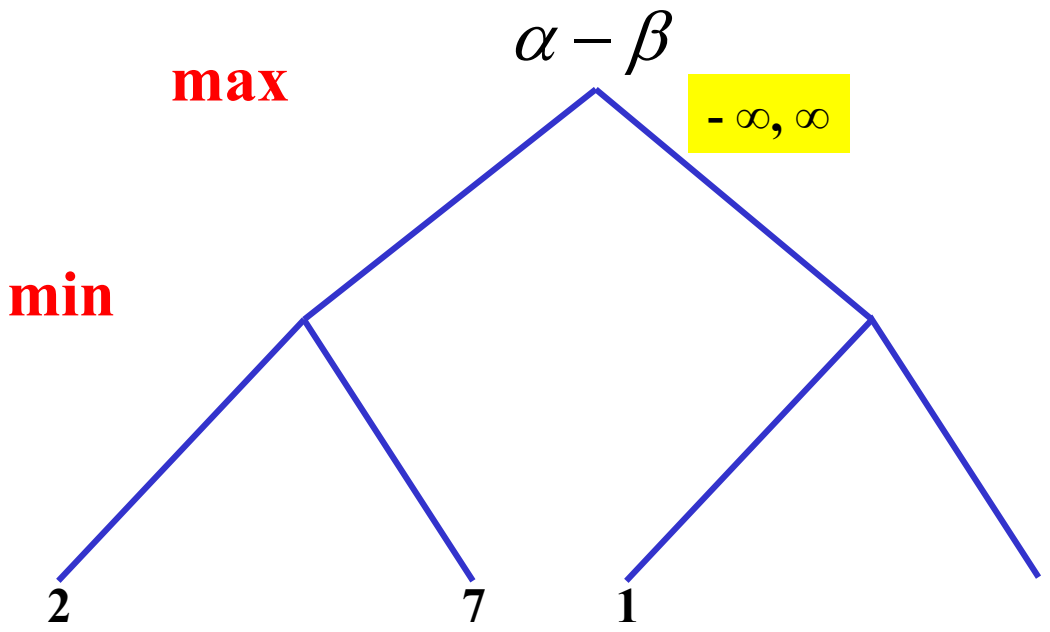
end

return ***β***

Minimises β, returns β


Initial α, β
return value, number of static evaluations
updated α, β

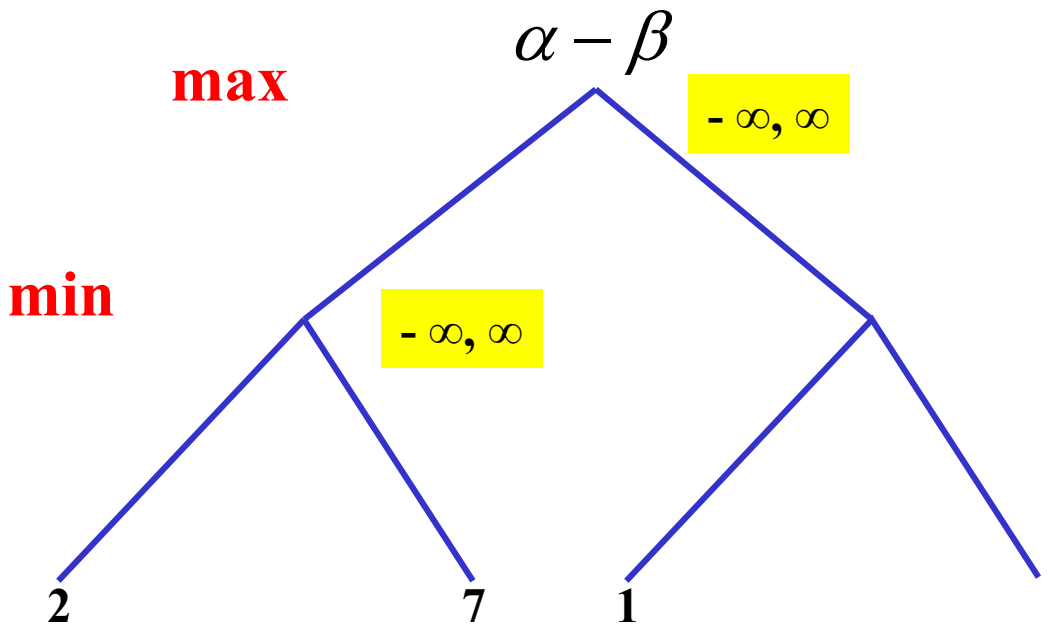
Alpha-Beta Pruning in action



We start with an initial call to ***MAX·VALUE.***

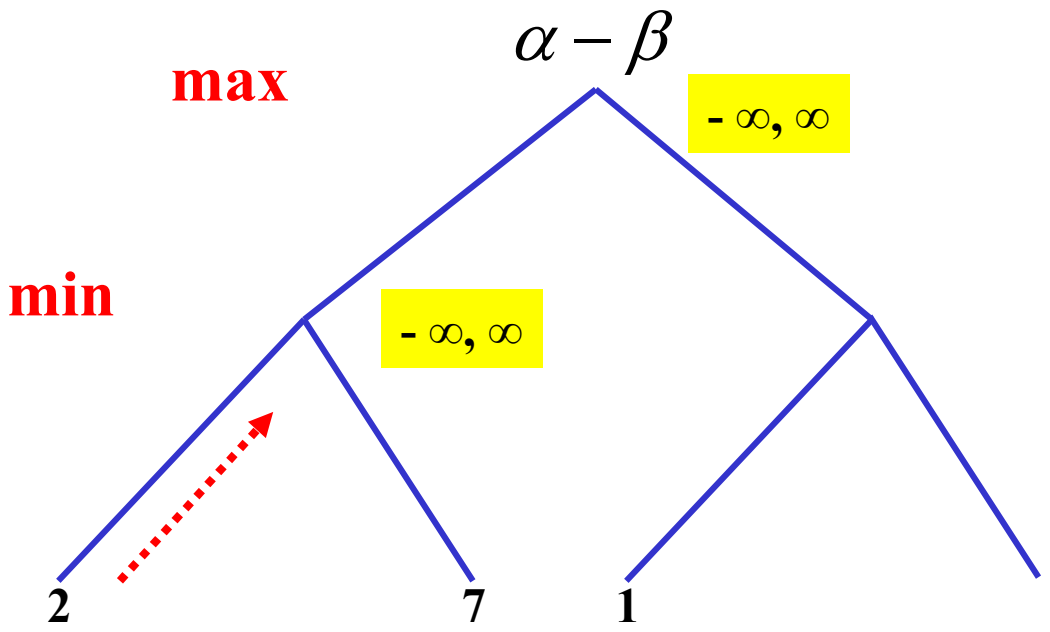
MAX·VALUE*(state, -∞, ∞, MAX·DEPTH)*

Alpha-Beta Pruning in action



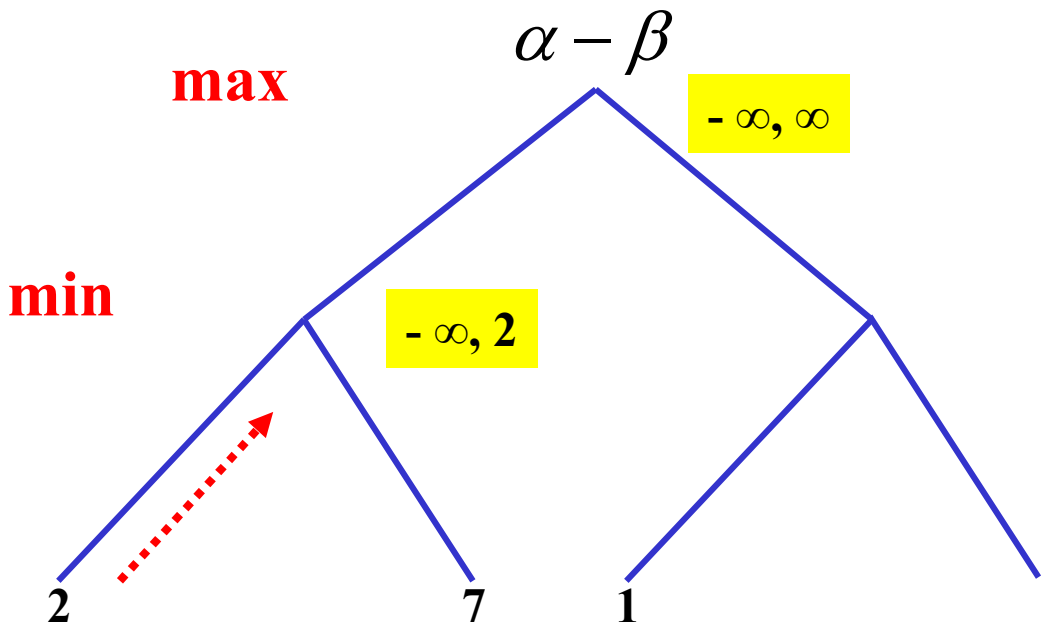
MAX·VALUE *now calls* ***MIN·VALUE*** *on the left successor with the same values of alpha and beta.*
MIN·VALUE *now calls* ***MAX·VALUE*** *on its leftmost succesor.*

Alpha-Beta Pruning in action



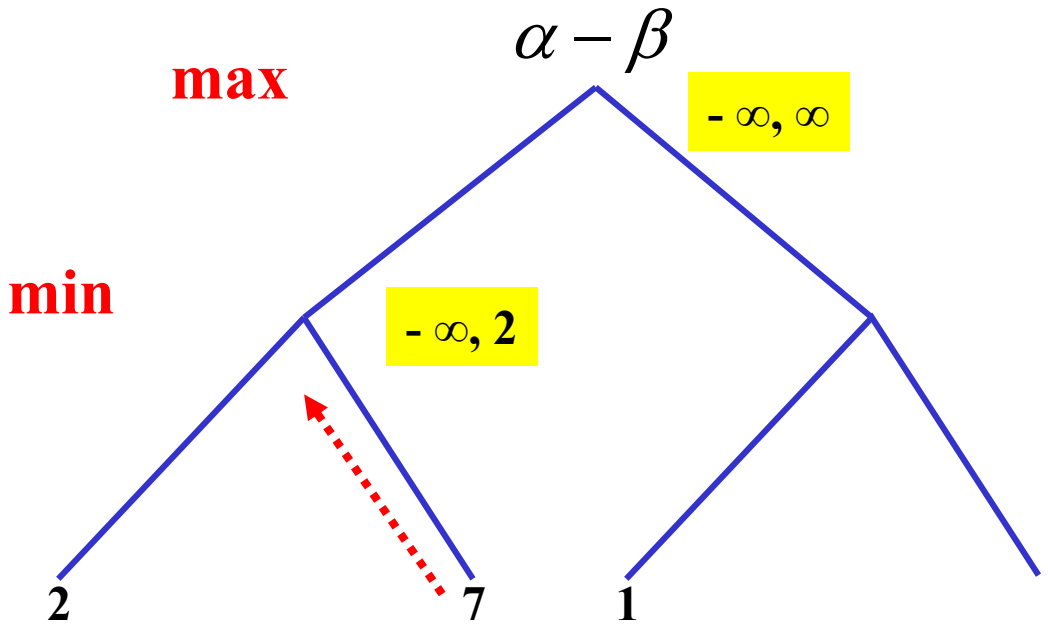
MAX·VALUE *is at the leftmost leaf, whose leaf value is 2 and so it returns that.*

Alpha-Beta Pruning in action



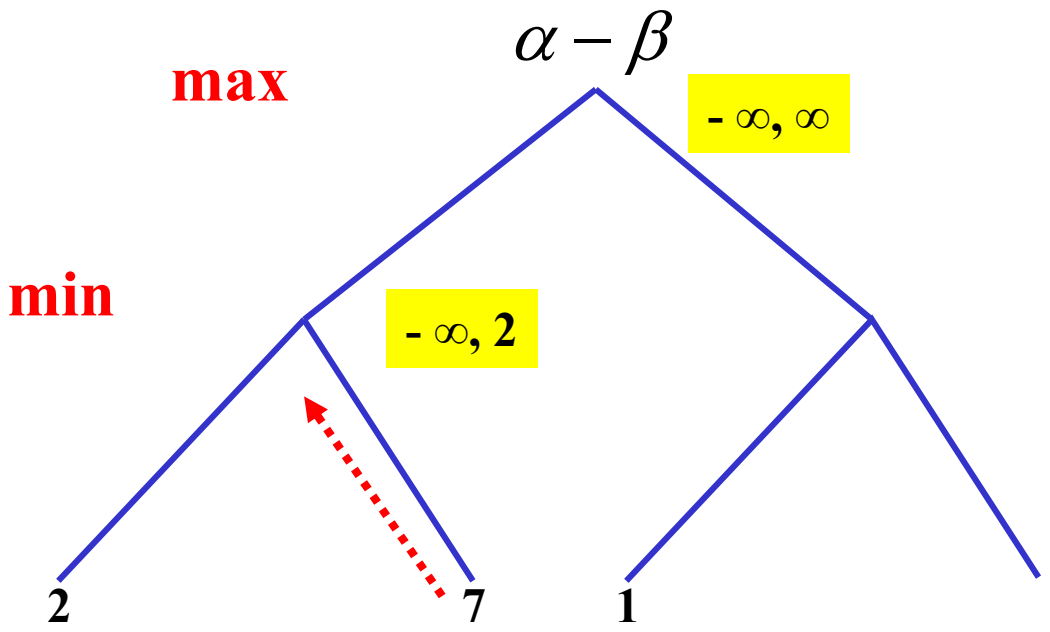
This first value, since it is less than ∞, becomes the new value of ***β*** *in* ***MIN·VALUE.***

Alpha-Beta Pruning in action



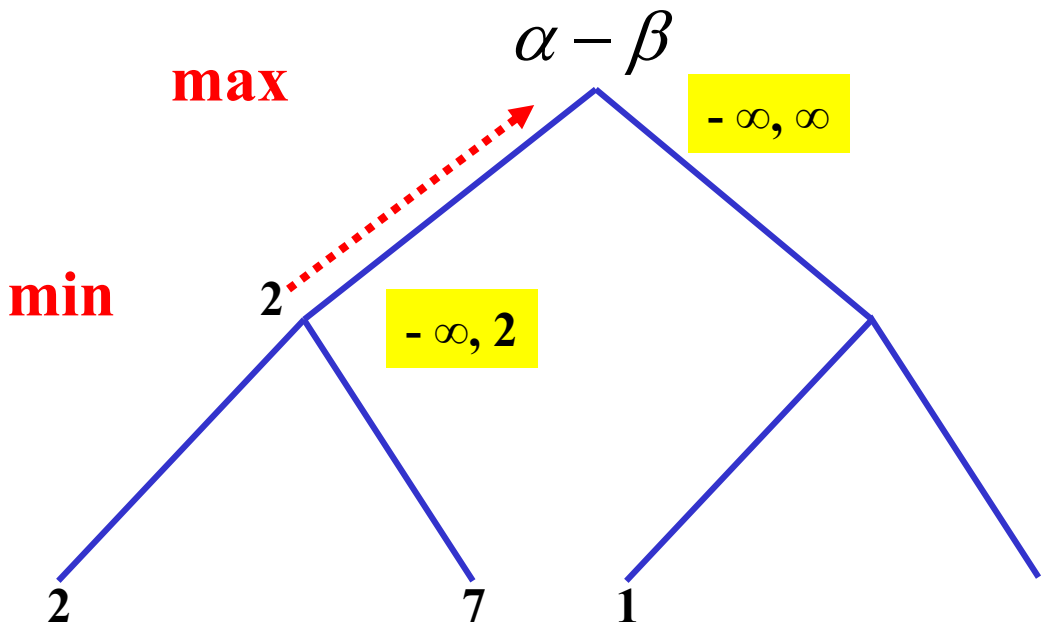
*So now we call **MAX·VALUE** with the next successor, which is also a leaf whose value is 7.*

Alpha-Beta Pruning in action



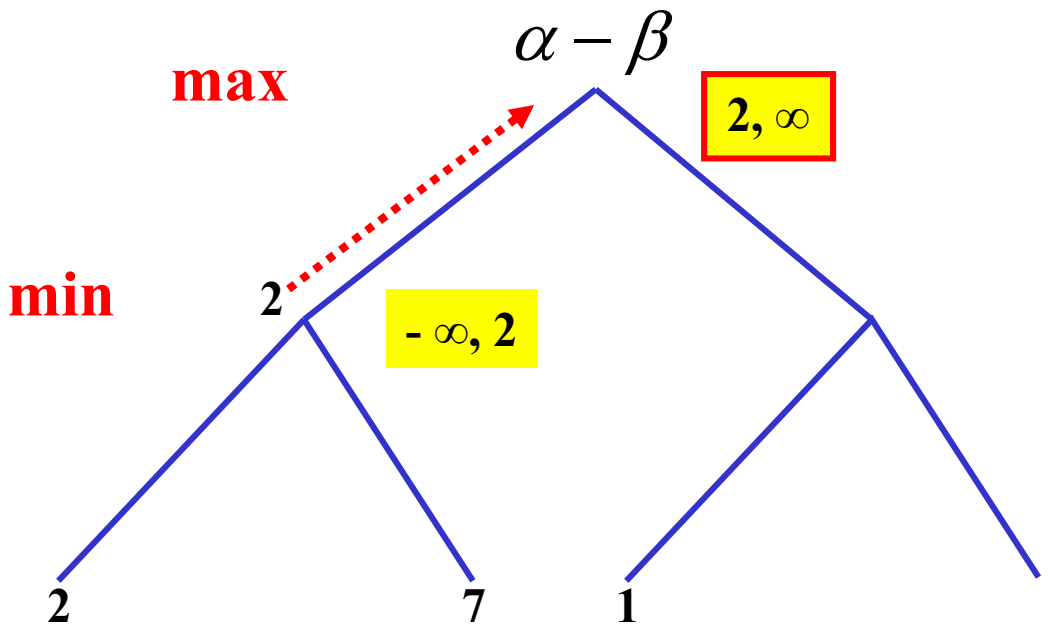
7 is not less than 2 and so the final value of β *is 2 for this node.*

Alpha-Beta Pruning in action



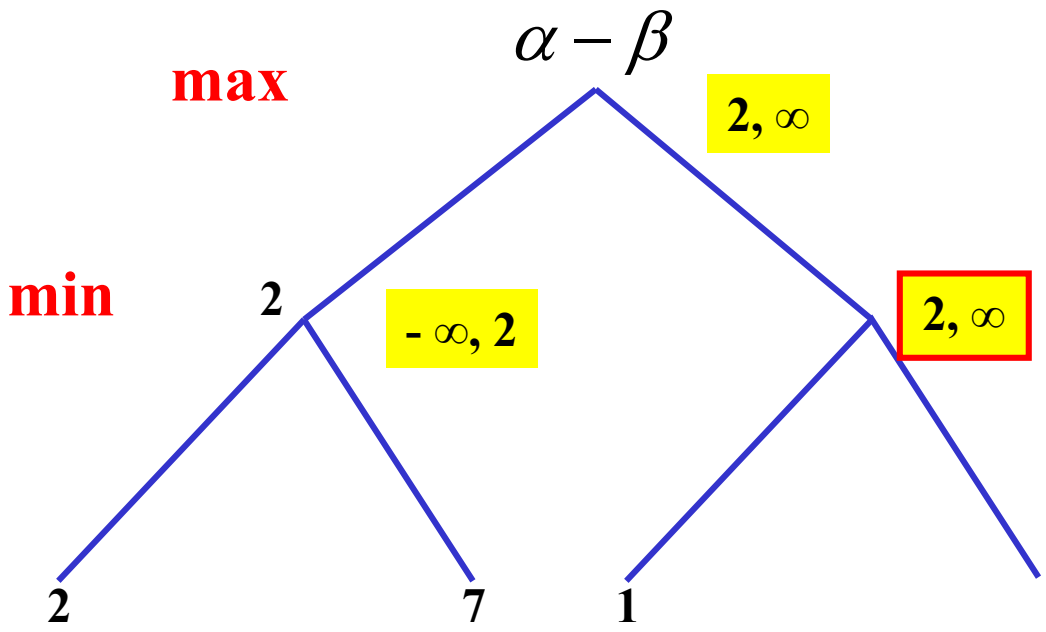
MIN·VALUE now returns 2 to its caller.

Alpha-Beta Pruning in action



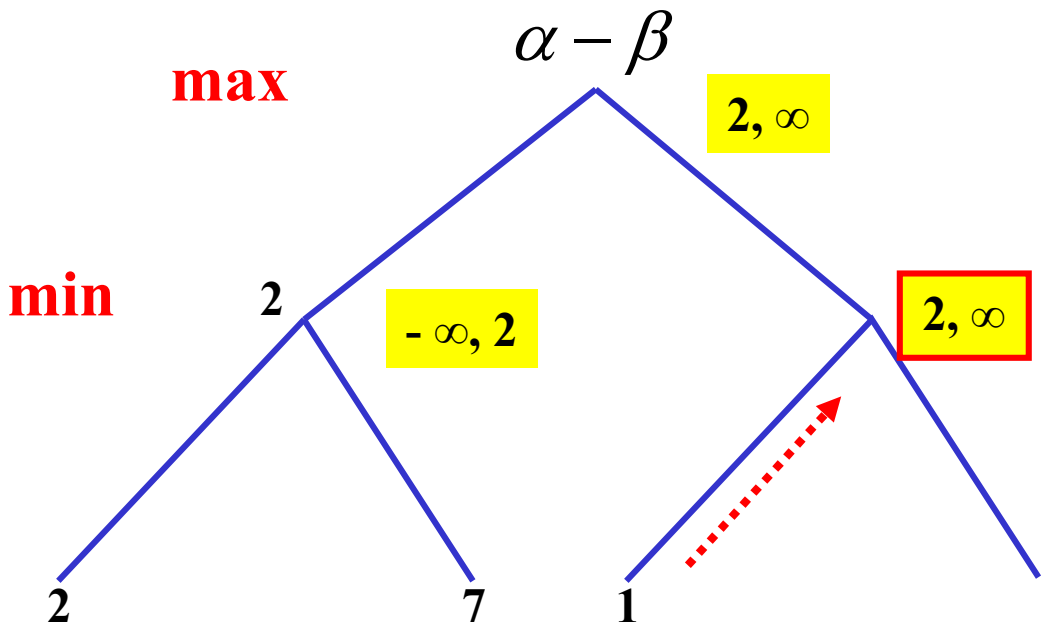
*The calling **MAX·VALUE** now sets **α** to 2, since it is bigger than -∞. Note that the range of [alpha-beta] says that the score will be greater or equal to 2 (and less than ∞).*

Alpha-Beta Pruning in action



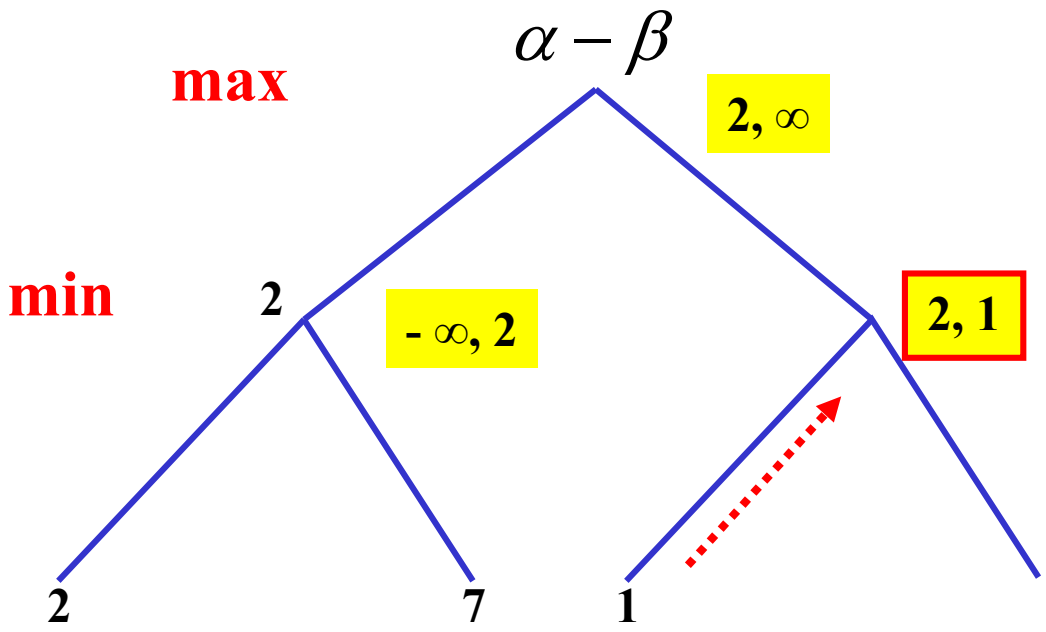
MAX·VALUE now calls MIN·VALUE with an updated range of [alpha-beta].

Alpha-Beta Pruning in action



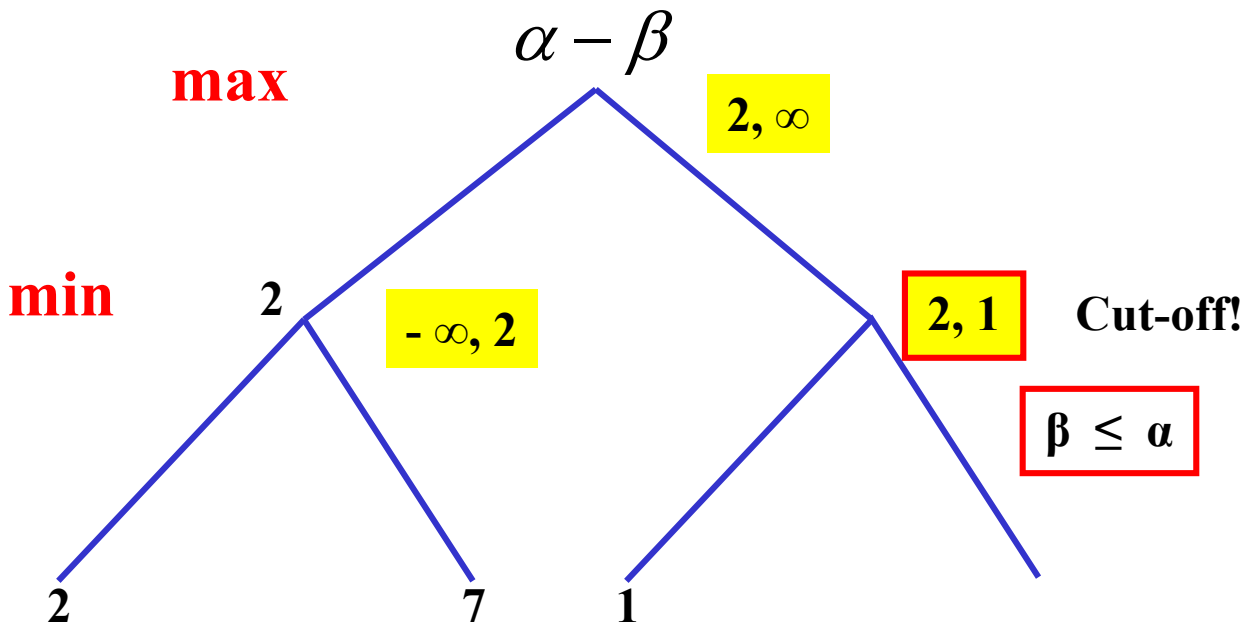
MIN·VALUE calls MAX·VALUE on the left leaf and it returns a value of 1.

Alpha-Beta Pruning in action



*This is used to update beta in **MIN·VALUE**, since it is less than ∞. Note that at this point, we have a range where **α**=2 is greater than **β**=1.*

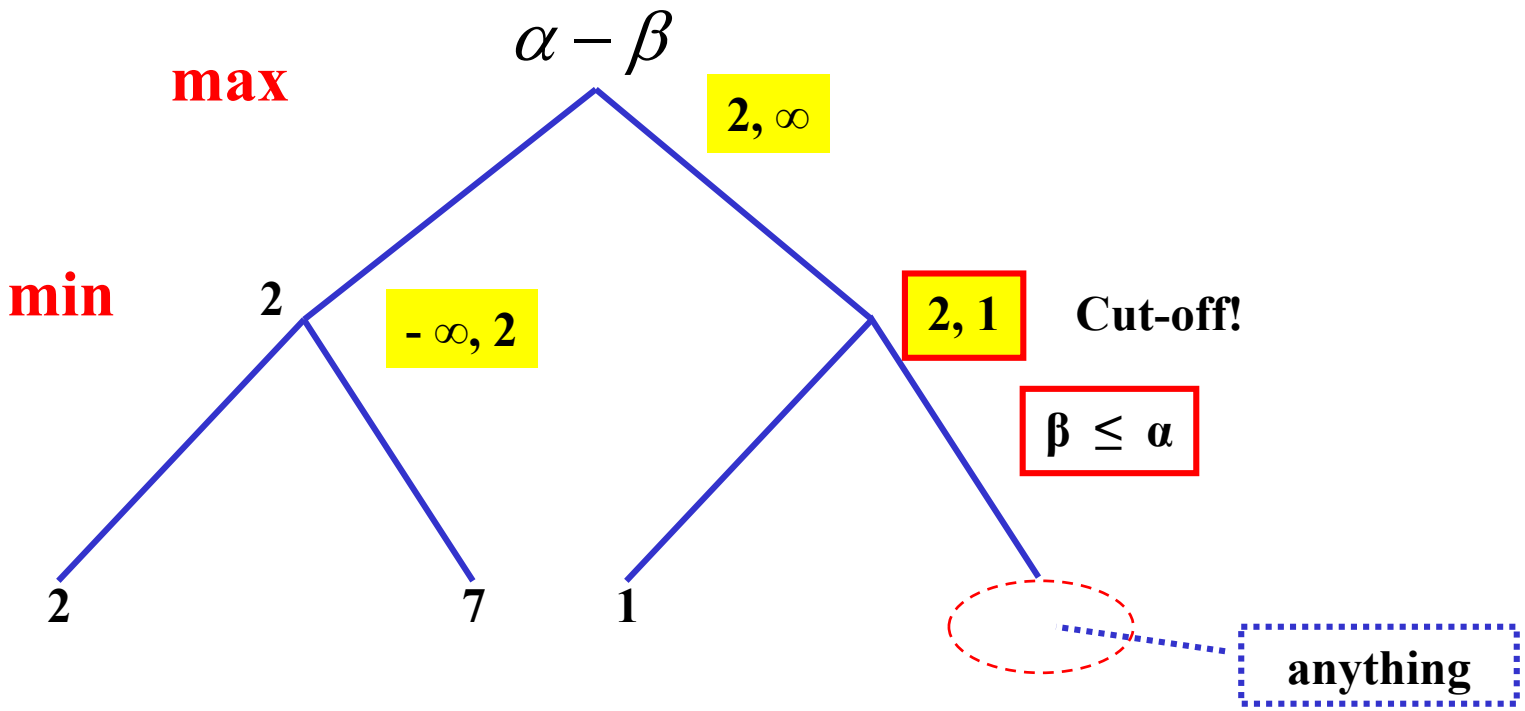
Alpha-Beta Pruning in action



*This is used to update beta in **MIN·VALUE**, since it is less than ∞. Note that at this point, we have a range where **α**=2 is greater than **β**=1.*

*This situation signals a cut-off in **MIN·VALUE** and it returns beta(=1), without looking at the right leaf.*

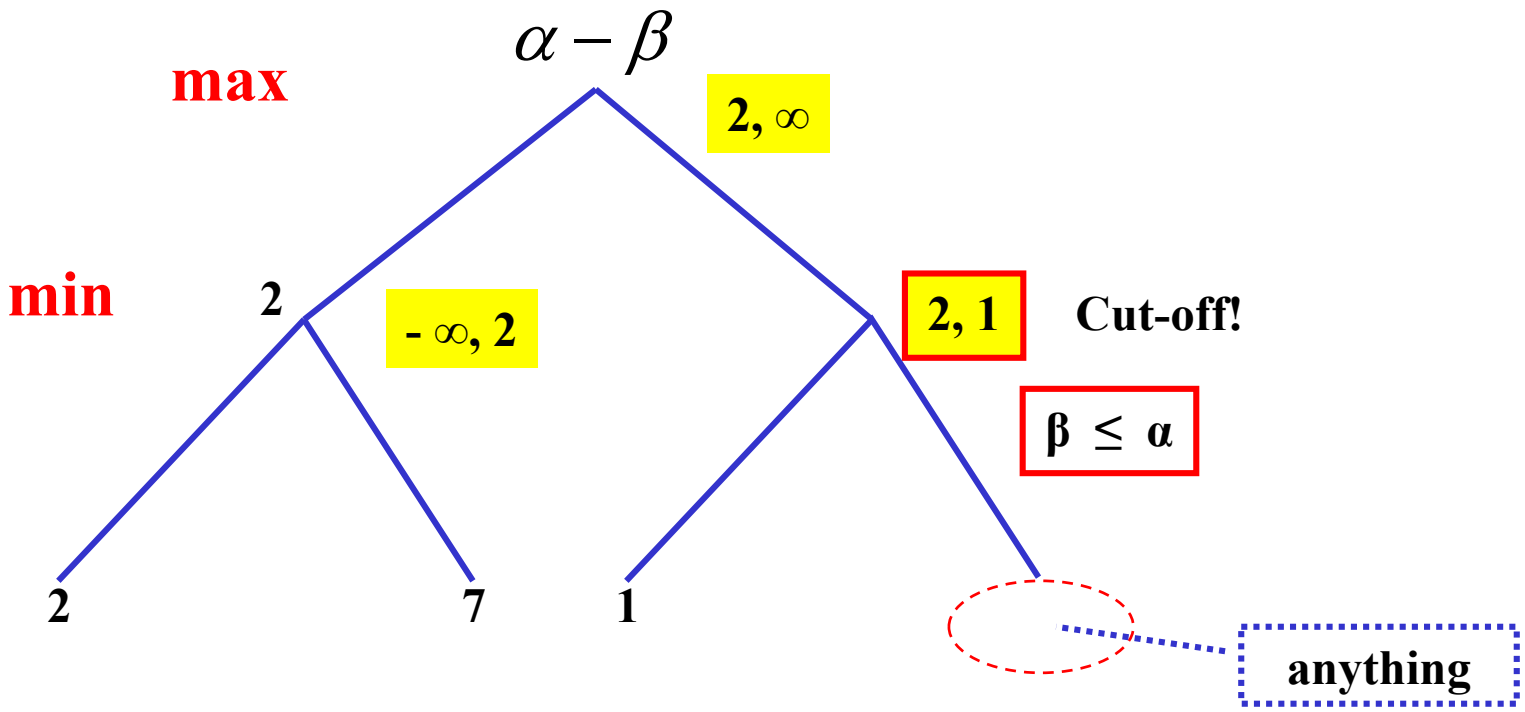
Alpha-Beta Pruning in action



*This situation signals a cut-off in **MIN·VALUE** and it returns beta(=1), <u>without looking at the right leaf</u>.*

*So, basically we had already found a move that guaranteed us a score ≥ **2** so that when we got into a situation where the score was guaranteed to be ≤ **1**, we could stop.*

Alpha-Beta Pruning in action



*So, a total of **3** static evaluations were needed instead of the **4** we would have needed under pure **Min·Max.***

α-β (NegaMax form) *Alpha-Beta Pruning in a more compact form*

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH)*

*function **ALPHA·BETA**(state, α, β, depth)*

- *if (depth == 0) then return EVAL(state)*
- *For each **s** in SUCCESSORS(state) do*

  ***α** = MAX(α, -**ALPHA·BETA**(s, -β, -α, depth-1))*

  *If(α ≥ β) Then return **α** //cut-off*

end

return α

Basically, this exploits the idea that minimizing is the same as maximising the negatives of the scores.

α-β (NegaMax form) *Alpha-Beta Pruning in a more compact form*

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH)*

function ***ALPHA·BETA****(state, α, β, depth,* ***MAXPLY****)*

Modified version

- *if ((depth == 0) and (**MAXPLY is odd**))*
  then return ***-EVAL(state)***

  *else if ((depth == 0) and (**MAXPLY is even**)) return* ***EVAL(state)***
- *For each* ***s*** *in SUCCESSORS(state) do*
  α *= MAX(α, -****ALPHA·BETA****(s, -β, -α, depth-1,* ***MAXPLY****))*
  If(α ≥ β) Then return ***α*** *//cut-off*

  end

return α

Basically, this exploits the idea that minimizing is the same as maximising the negatives of the scores.

α-β (NegaMax form) *Alpha-Beta Pruning in a more compact form*

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH, 1)*

Modified version v2

```
function ALPHA·BETA(state, α, β, depth, colour)
    • if ((depth == 0) then  return (colour * EVAL(state))
    For each s in SUCCESSORS(state) do
        α = MAX(α, -ALPHA·BETA(s, -β, -α, depth-1, -colour))
        If(α ≥ β) Then return α  //cut-off
      end
    return α
```

Basically, this exploits the idea that minimizing is the same as maximising the negatives of the scores.

Key points about α-β

1. Guaranteed same value as **Max-Min**.

2. In a perfectly ordered tree, expected work is $O(b^{d/2})$ vs. $O(b^d)$ for **Max-Min**, so can search twice as deep with the same effort!

3. With good move ordering, the actual running time is close to optimistic estimate.

Game Program

1. Move generator (heuristically ordered moves)	50%
2. Static evaluation	40%
3. Search control	10%

In practice,

- **Openings**
- **End games**

} Played by looking up moves in a **Database**

[all in place by late 60's]

Move Generator

1. Legal moves

2. Ordered by
   - most valuable victim
   - least valuable agressor

3. Killer heuristic

Static Evaluation

Initially	Very complex
70's	Very simple (material)
1997 (IBM Deep Blue)	• Deep searches: moderately complex (hardware) • PC programs: elaborate, hand-tuned

Practical matters

* It is not wise to fix the max. depth of the search.

Variable branching

Iterative Deepening

- Order best move from last search first
- use previous backed up value to initialise [α, β]
- keep track of repeated positions (transposition tables)

Horizon Effect

- quiescence
- pushing the inevitable over search horizon

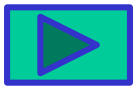
Parallelisation

- hard to do effectively

http://chessprogramming.wikispaces.com/Transposition+Table

Other Games

Backgammon



- Involves **randomness** – dice rolls
- machine-learning based player was able to draw the world champion

Bridge

- Involves **hidden information** – other player's cards, and **communication** during bidding
- Computer players play well but do not bid well

Go

World champion – AlphaGo (2016)

Note, Chess: (b≈35, d≈80)

- No new elements but huge branching factor (b≈250, d≈150); exhaustive search is infeasible
- difficult to evaluate board positions and moves

Other Games

AlphaGo

- Humans make a move based on whether a move "feels right"

- precise search tree-based computation

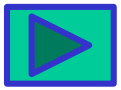
- uses Monte Carlo Tree Search + 2 Deep Neural Networks + reinforcement learning

2 Deep Neural Networks

1. **Value network** = evaluate board positions;
   reduces depth of search
2. **Policy network** = sample actions (select moves)
   = predict the outcome at state s (prunes subtree); reduces breadth of search

Other Games

AlphaGoZero



- trains solely from self-play RL, without any supervision, without human data

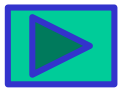
- input features: only black and white stones

- uses Monte Carlo Tree Search without rollouts + 1 Deep Neural Network + new reinforcement learning that incorporates lookahead search inside the training loop

Only 1 Deep Neural Network with the following outputs

1. **Value** = probability of winning at a board position;
2. **Policy** = move probabilities (probability of selecting each move)
   = predict the outcome at state s (prunes subtree); reduces breadth of search

AlphaZero



Games

Observations

- Initially, computers excel in well-defined activities where rules are clear
  - chess
  - mathematics
- Now, the focus is more on less defined activities (vision, language and common sense) and problems that cannot be solved by brute force.
- Success comes after a long period of gradual refinement.

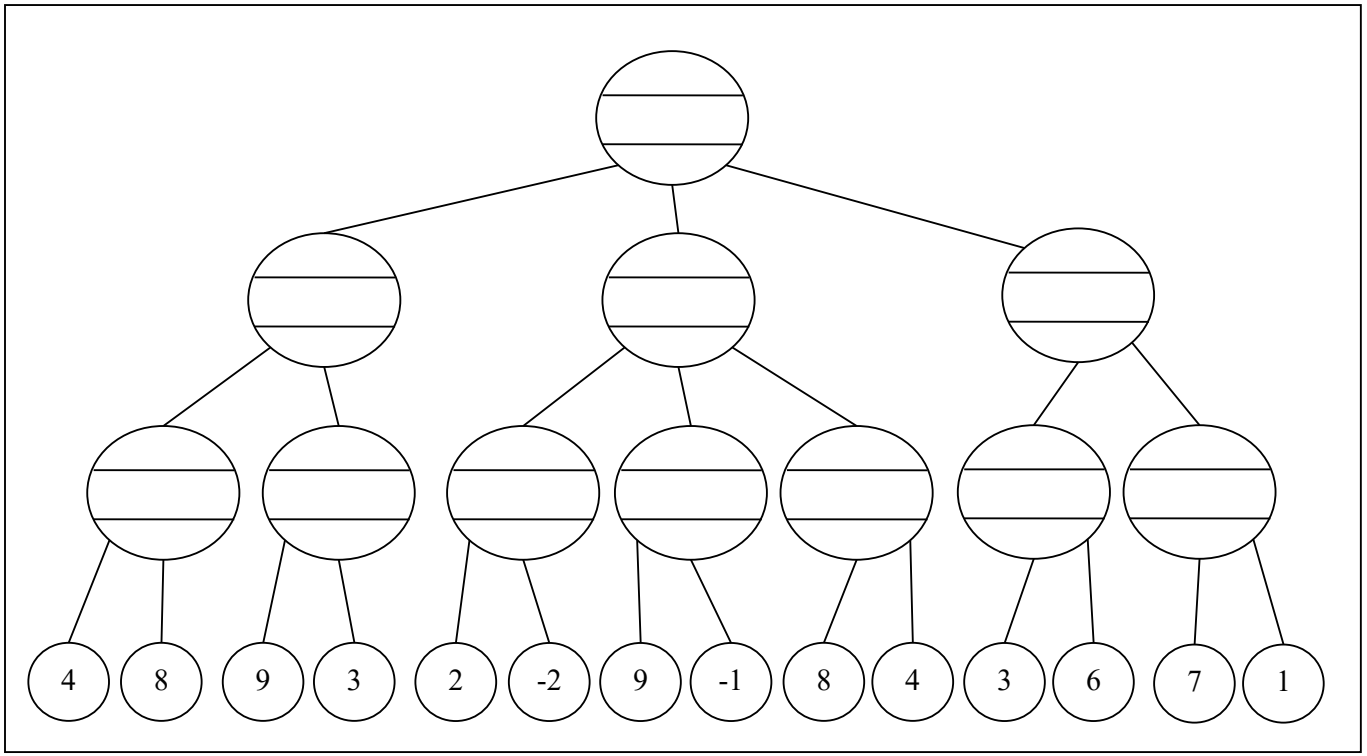
For more details on building game programs, visit:

https://www.ics.uci.edu/~eppstein/180a/links.html

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the MinMax algorithm. Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

MinMax Algorithm

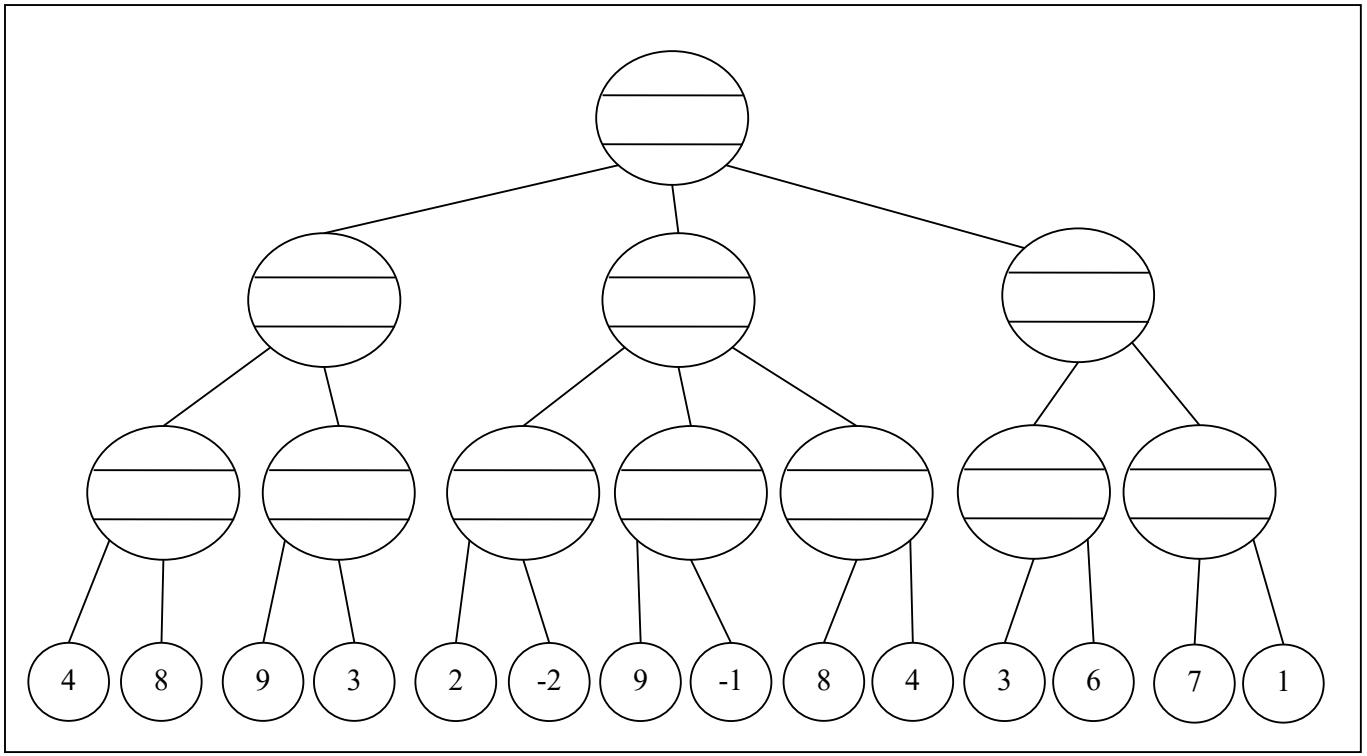


n.h.reyes@massey.ac.nz
Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the Alpha-Beta algorithm (consider nodes from left to right). Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

Alpha-Beta Pruning (left to right)

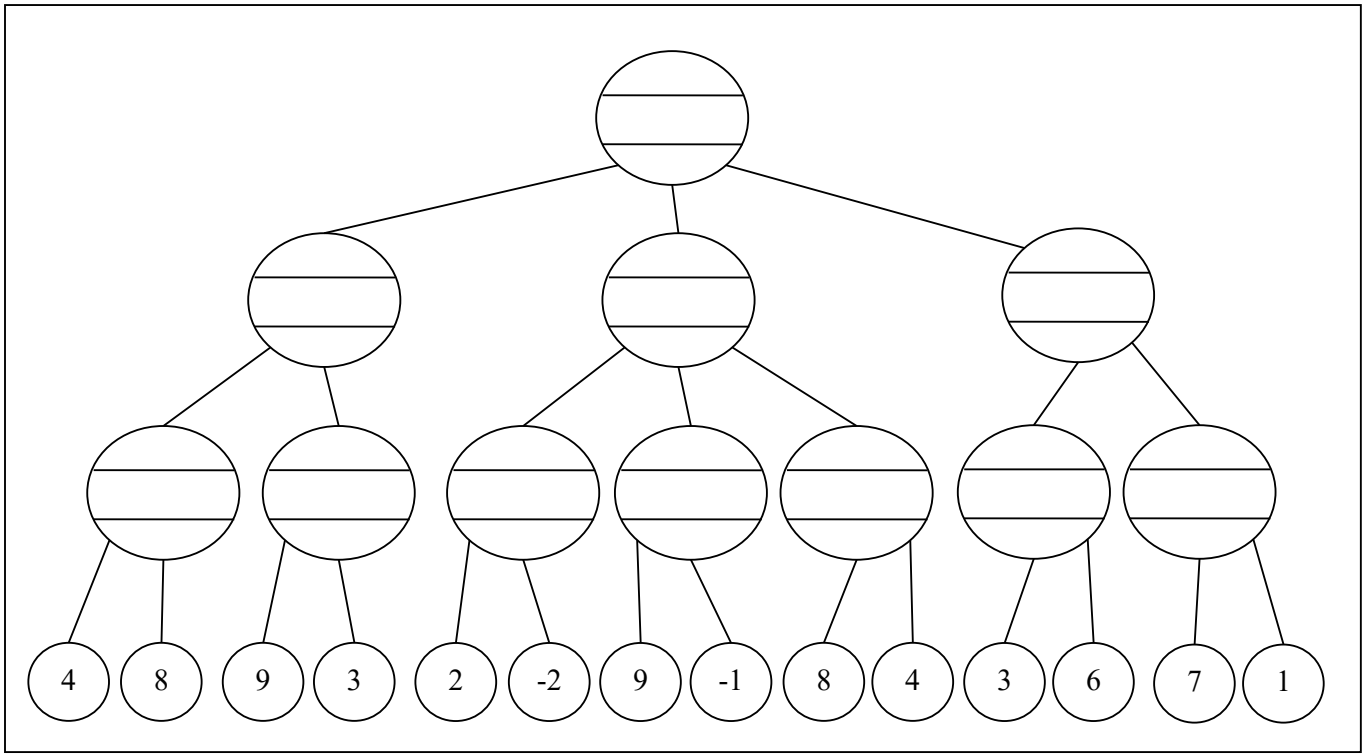


n.h.reyes@massey.ac.nz
Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the Alpha-Beta algorithm (consider nodes from right to left). Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

Alpha-Beta Pruning (right to left)



n.h.reyes@massey.ac.nz
Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Min-MaX Algorithm

- *function MAX·VALUE(state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = -∞*
  - *For each s in SUCCESSORS(state) do v = MAX(v, MIN·VALUE(s, depth – 1))*

  end

  return v

- *function MIN·VALUE (state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = ∞*
  - *For each s in SUCCESSORS(state) do v = MIN(v, MAX·VALUE(s, depth – 1))*

  end

  return v

α-β (NegaMax form)

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH)*

*function **ALPHA·BETA**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each **s** in SUCCESSORS(state) do*
  ***α** = MAX(α, -**ALPHA·BETA**(s, -β, -α, depth-1))*
  *If(α ≥ β) Then return **α** //cut-off*

end

return α

Alpha-Beta Pruning

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is *MAX·VALUE(state, -∞, ∞, MAX·DEPTH)*

*function **MAX·VALUE**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each s in SUCCESSORS(state) do*
  α = MAX(α, MIN·VALUE(s, α, β, depth-1))
  If(α ≥ β) Then return α //cut-off

end

return α

*function **MIN·VALUE**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each s in SUCCESSORS(state) do*
  β = MIN(β, MAX·VALUE(s, α, β, depth-1))
  If(β ≤ α) Then return β //cut-off

end

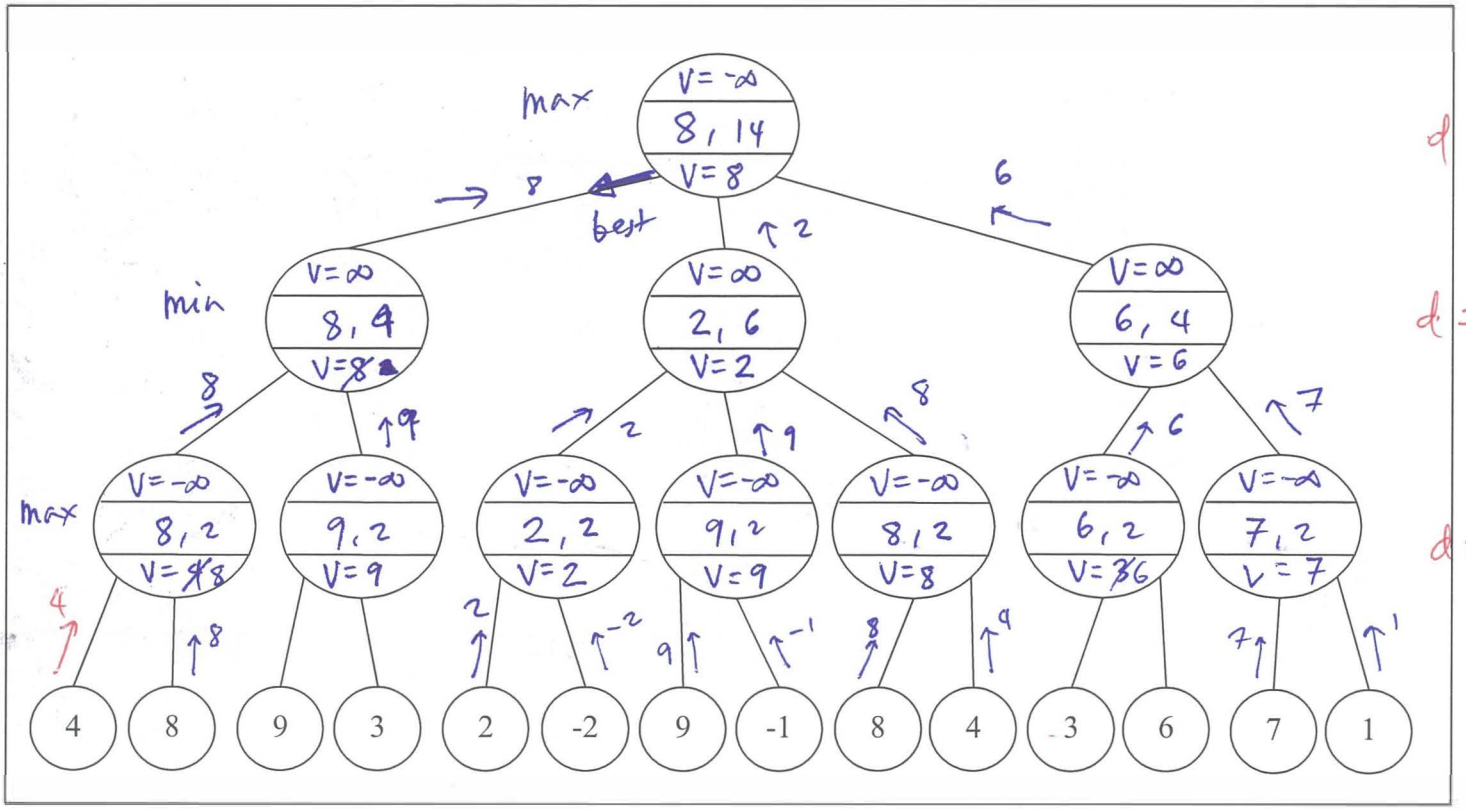
return β

n.h.reyes@massey.ac.nz
Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the MinMax algorithm. Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

MinMax Algorithm

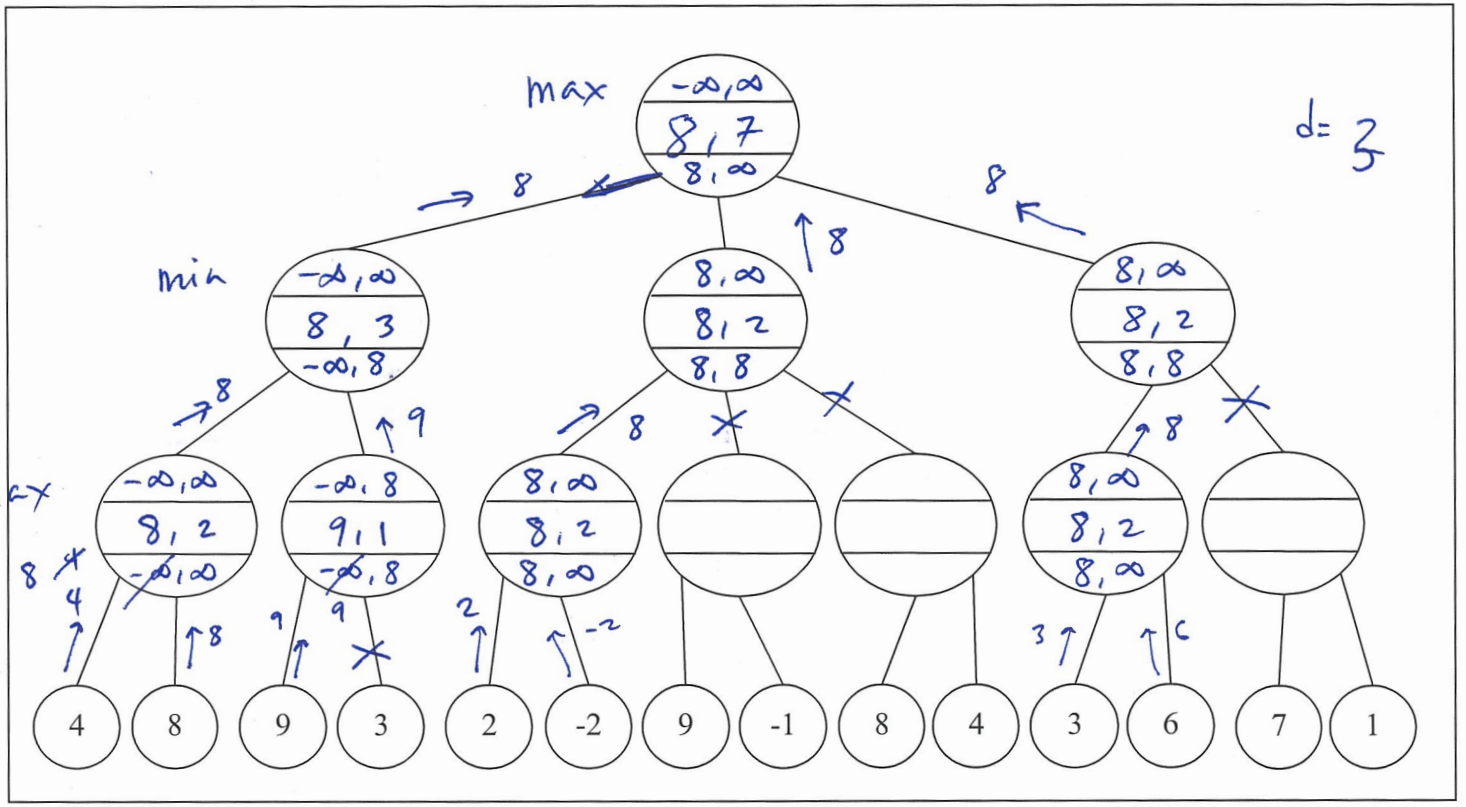


Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the Alpha-Beta algorithm (consider nodes from left to right). Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

Alpha-Beta Pruning (left to right)

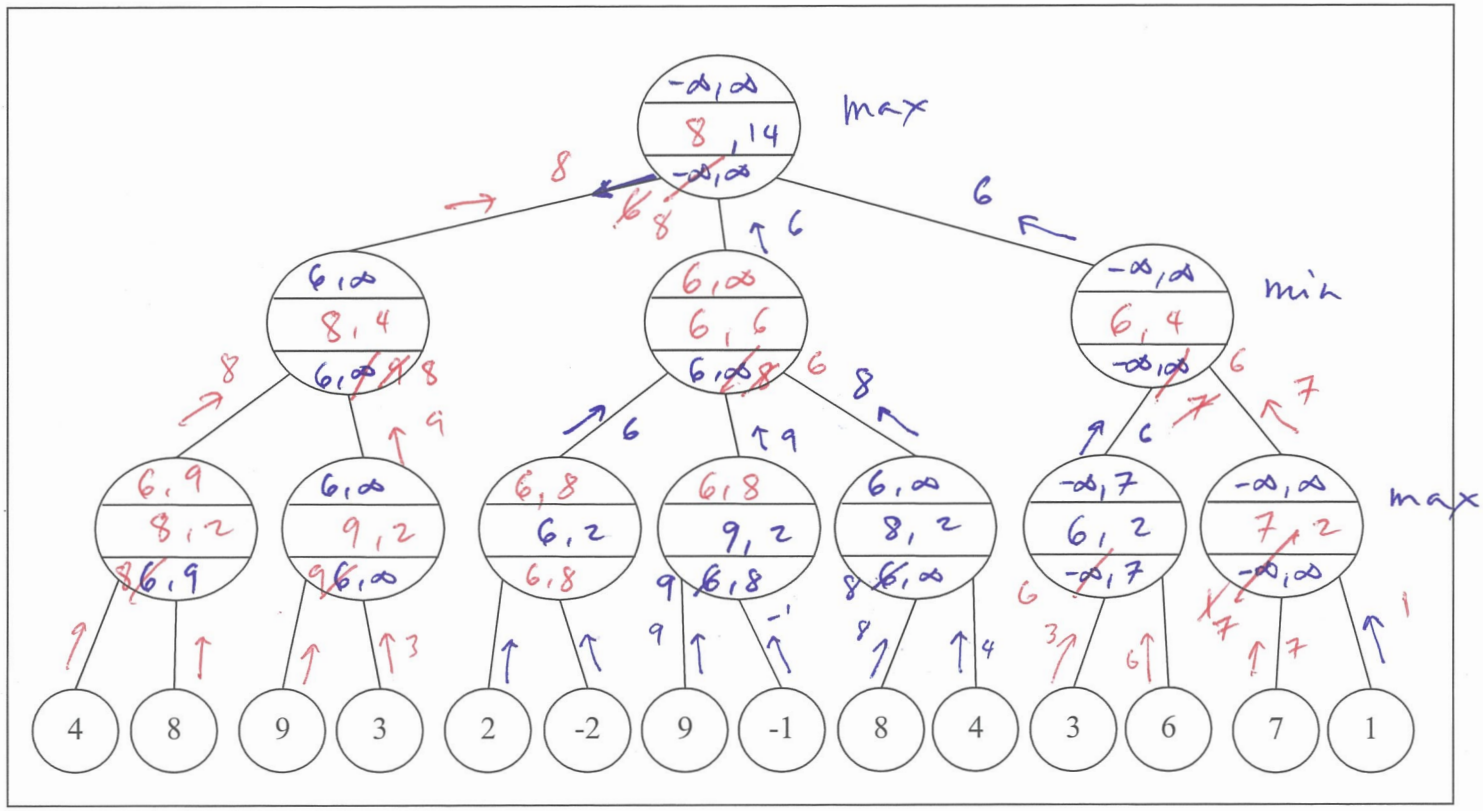


Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Refer to the given game tree and simulate the Alpha-Beta algorithm (consider nodes from right to left). Show your complete answer by filling-in each of the three sections of the interior of the nodes with the required information.

Alpha-Beta Pruning (right to left)



Source: MIT OpenCourseWare

159302 Artificial Intelligence
Tutorial #8: GAMES: Alpha-Beta Pruning

Min-MaX Algorithm

- *function MAX·VALUE(state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = -∞*
  - *For each s in SUCCESSORS(state) do v = MAX(v, MIN·VALUE(s, depth – 1))*

end
return v

- *function MIN·VALUE (state, depth)*
  - *if (depth == 0) then return EVAL(state)*
  - *v = ∞*
  - *For each s in SUCCESSORS(state) do v = MIN(v, MAX·VALUE(s, depth – 1))*

end
return v

α-β (NegaMax form)

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is ***ALPHA·BETA****(state, -∞, ∞, MAX·DEPTH)*

*function **ALPHA·BETA**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each **s** in SUCCESSORS(state) do*
  ***α** = MAX(α, -**ALPHA·BETA**(s, -β, -α, depth-1))*
  *If(α ≥ β) Then return **α** //cut-off*

end
return α

Alpha-Beta Pruning

α – is the best score for MAX; **β** – is the best score for MIN
Initial call is *MAX·VALUE(state, -∞, ∞, MAX·DEPTH)*

*function **MAX·VALUE**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each s in SUCCESSORS(state) do*
  α = MAX(α, MIN·VALUE(s, α, β, depth-1))
  If(α ≥ β) Then return α //cut-off

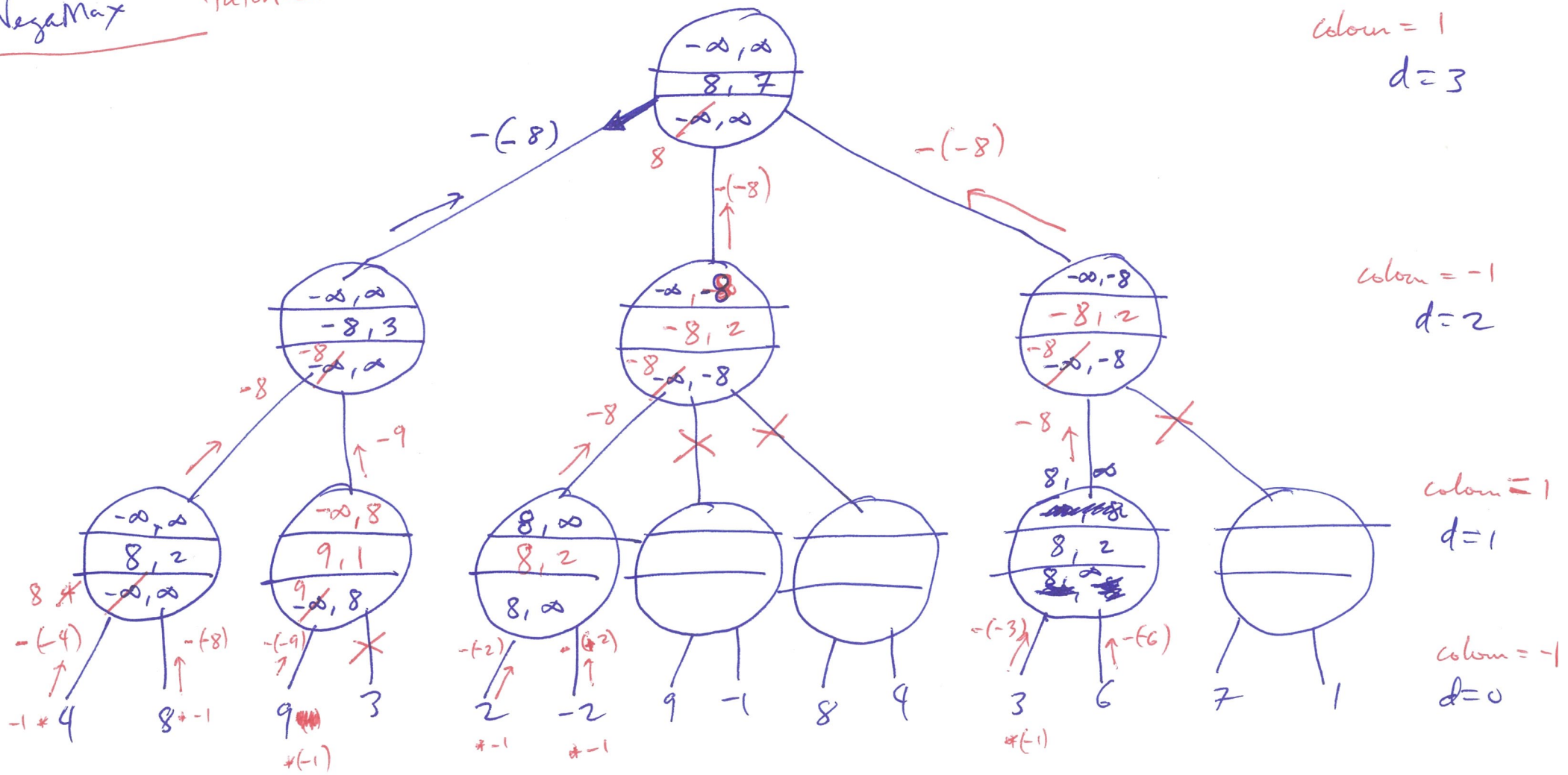
end
return α

*function **MIN·VALUE**(state, α, β, depth)*
- *if (depth == 0) then return EVAL(state)*
- *For each s in SUCCESSORS(state) do*
  β = MIN(β, MAX·VALUE(s, α, β, depth-1))
  If(β ≤ α) Then return β //cut-off

end
return β

Source: MIT OpenCourseWare

NegaMax tutorial



159.302 LECTURE 12

Propositional Logic Syntax

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Propositional Logic

- **What is a logic?**

  A formal language

Propositional Logic

- **What is a logic?**

  A formal language

  - **Syntax** – what expressions are legal

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other syntactic expressions (which will tell us something new)

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other syntactic expressions (which will tell us something new)

Why proofs?

- Two kinds of inferences an agent might want to make:

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other sytactic expressions (which will tell us something new)

Why proofs?

- Two kinds of inferences an agent might want to make:
  - **Multiple percepts** => conclusions about the world

Propositional Logic

What is a logic?

A formal language

- **Syntax** – what expressions are legal
- **Semantics** – what legal expressions mean
- **Proof System** – a way of manipulating syntactic expressions to get other sytactic expressions (which will tell us something new)

Why proofs?

- Two kinds of inferences an agent might want to make:
  - **Multiple percepts** => conclusions about the world
  - **Current state & operator** => properties of next state

Propositional Logic

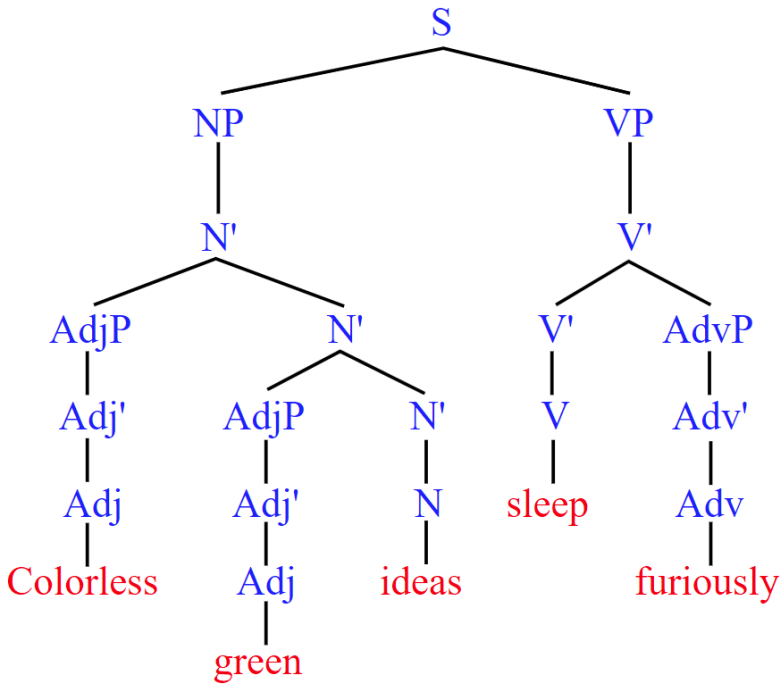
Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.



Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences
- Propositional variables are sentences: **P, Q, R, Z**

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences
- Propositional variables are sentences: **P, Q, R, Z**
- If **φ** and **ψ** are sentences, then so are

$$(\phi), \neg\phi, \phi \vee \psi, \phi \wedge \psi, \phi \rightarrow \psi, \phi \leftrightarrow \psi$$

Propositional Logic

Syntax: what you're allowed to write

- for(thing t=fizz; t == fuzz; t++){…}
- Colorless green ideas sleep furiously.

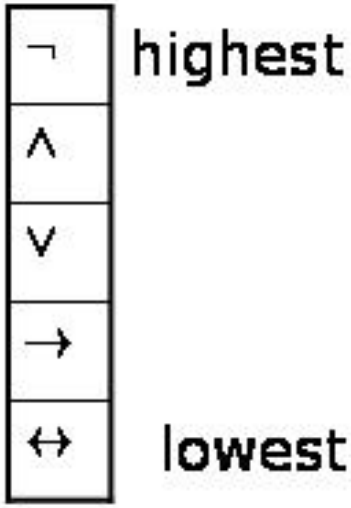
Sentences (wffs: well-formed formulas)

- **true** and **false** are sentences
- Propositional variables are sentences: **P, Q, R, Z**
- If **φ** and **ψ** are sentences, then so are

$$(\phi), \neg\phi, \phi \vee \psi, \phi \wedge \psi, \phi \rightarrow \psi, \phi \leftrightarrow \psi$$

- Nothing else is a sentence

Precedence



$A \vee B \wedge C$	$A \vee (B \wedge C)$
$A \wedge B \rightarrow C \vee D$	$(A \wedge B) \rightarrow (C \vee D)$
$A \rightarrow B \vee C \leftrightarrow D$	$(A \rightarrow (B \vee C)) \leftrightarrow D$

- Precedence rules enable "shorthand" form of sentences, but formally only the fully parenthesized form is legal.
- Syntactically ambiguous forms allowed in shorthand only when semantically equivalent: $A \wedge B \wedge C$ is equivalent to $(A \wedge B) \wedge C$ and $A \wedge (B \wedge C)$

159.302 LECTURE 2

Propositional Logic Semantics

Source: MIT OpenCourseWare

Propositional Logic

Semantics

- Meaning of a sentence is truth value {**t**, **f**}

Propositional Logic

Semantics

- Meaning of a sentence is truth value {**t**, **f**}
- **Interpretation** is an assignment of truth values to the propositional variables

holds(ϕ , i) [Sentence ϕ is **t** in interpretation i]

Propositional Logic

Semantics

- Meaning of a sentence is truth value {**t**, **f**}
- **Interpretation** is an assignment of truth values to the propositional variables

holds(ϕ, i) [Sentence ϕ is **t** in interpretation i]

fails(ϕ, i) [Sentence ϕ is **f** in interpretation i]

Propositional Logic

Semantic Rules

holds(true, i) for all i

Propositional Logic

Semantic Rules

holds(true, i)	for all i
fails(false, i)	for all i

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

holds($\phi \wedge \psi$, i) iff holds(ϕ,i) and holds(ψ, i) **(conjunction)**

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

holds($\phi \wedge \psi$, i) iff holds(ϕ,i) and holds(ψ, i) **(conjunction)**

holds($\phi \vee \psi$, i) iff holds(ϕ,i) or holds(ψ, i) **(disjunction)**

Propositional Logic

Semantic Rules

holds(true, i) for all i

fails(false, i) for all i

holds($\neg\phi$, i) if and only if fails(ϕ, i) **(negation)**

holds($\phi \wedge \psi$, i) iff holds(ϕ,i) and holds(ψ, i) **(conjunction)**

holds($\phi \vee \psi$, i) iff holds(ϕ,i) or holds(ψ, i) **(disjunction)**

holds(P, i) iff i(P) = **t**

fails(P, i) iff i(P) = **f**

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent $\rightarrow$ consequent)**

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent $\rightarrow$ consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent → consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$Q \rightarrow P$	$P \leftrightarrow Q$
f	f	t	f	f	t	t	t
f	t	t	f	t	t	f	f
t	f	f	f	t	f	t	f
t	t	f	t	t	t	t	t

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent → consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$Q \rightarrow P$	$P \leftrightarrow Q$
f	f	t	f	f	t	t	t
f	t	t	f	t	t	f	f
t	f	f	f	t	f	t	f
t	t	f	t	t	t	t	t

Note that implication is not “causality”, if P is **f** then $P \rightarrow Q$ is **t**

Implication doesn't mean "is related" in any kind of real-world way

Propositional Logic

Some important shorthand

$\phi \rightarrow \psi \equiv \neg\phi \vee \psi \quad (conditional, implication)$ **(antecedent → consequent)**

$\phi \leftrightarrow \psi \equiv (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi) \quad (biconditional, equivalence)$

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$Q \rightarrow P$	$P \leftrightarrow Q$
f	f	t	f	f	t	t	t
f	t	t	f	t	t	f	f
t	f	f	f	t	f	t	f
t	t	f	t	t	t	t	t

Note that implication is not “causality”, if P is **f** then $P \rightarrow Q$ is **t**

Implication doesn't mean "is related" in any kind of real-world way

Propositional Logic

Terminology

A sentence is **valid** iff its truth value is **t** in all interpretations

Valid sentences : $\underline{true}$, $\neg\underline{false}$, $P \vee \neg P$

A sentence is **satisfiable** iff its truth value is **t** in at least one interpretation

Satisfiable sentences: $P, \underline{true}, \neg P$

A sentence is **unsatisfiable** iff its truth value is **f** in all interpretations

Unsatisfiable sentences : $P \wedge \neg P$, $\underline{false}$, $\neg\underline{true}$

All are finitely decidable.

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$
$(s \rightarrow f) \rightarrow (\neg s \rightarrow \neg f)$	satisfiable, not valid	$s = f, f = t$ $(s \rightarrow f) = t, (\neg s \rightarrow \neg f) = f$

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$
$(s \rightarrow f) \rightarrow (\neg s \rightarrow \neg f)$	satisfiable, not valid	$s = f, f = t$ $(s \rightarrow f) = t, (\neg s \rightarrow \neg f) = f$
contrapositive $(s \rightarrow f) \rightarrow (\neg f \rightarrow \neg s)$	valid	

Examples

Sentence	Valid?	Interpretation that make sentence's truth value = **f**
$smoke \rightarrow smoke$ $\neg smoke \vee smoke$	valid	
$smoke \rightarrow fire$	satisfiable, not valid	$smoke = t, fire = f$
$(s \rightarrow f) \rightarrow (\neg s \rightarrow \neg f)$	satisfiable, not valid	$s = f, f = t$ $(s \rightarrow f) = t, (\neg s \rightarrow \neg f) = f$
contrapositive $(s \rightarrow f) \rightarrow (\neg f \rightarrow \neg s)$	valid	
$b \vee d \vee (b \rightarrow d)$ $b \vee d \vee \neg b \vee d$	valid	

Satisfiability

- Related to constraint satisfaction

- Given a sentence S, try to find an interpretation i such that ***holds(S,i)***

- Analogous to finding an assignment of values to variables such that the constraints hold

Satisfiability

- Related to constraint satisfaction
- Given a sentence S, try to find an interpretation i such that ***holds(S,i)***
- Analogous to finding an assignment of values to variables such that the constraints hold
- Brute force method: enumerate all interpretations and check

Satisfiability

- Related to constraint satisfaction
- Given a sentence S, try to find an interpretation i such that ***holds(S,i)***
- Analogous to finding an assignment of values to variables such that the constraints hold
- Brute force method: enumerate all interpretations and check
- Better methods:

Heuristic search
Constraint propagation
Stochastic search

Satisfiability Problems

- Scheduling nurses to work in a hospital
  - propositional variables represent for example, that Pat is working on Tuesday at 2pm (= P).
  - constraints on the schedule are represented using logical expressions over the variables
- Finding bugs in software
  - propositional variables represent state of the program
  - use logic to describe how the program works and to assert there is a bug
  - if the sentence is satisfiable, you've found the bug!

159.302 LECTURE 3

Propositional Logic Proof

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Flowers will be pollinated?

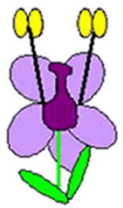
- Imagine we knew that:

  - If today is Sunny, then the bees will be Foraging $(S \rightarrow F)$
  - If the bees are Foraging, the flowers will be Pollinated $(F \rightarrow P)$
  - Today is sunny (S)

Knowledge Base

Knowledge Base (KB)/Axioms/Premise/Givens/Facts

Should we conclude that the flowers will be pollinated? (P)



Checking Interpretations

S	F	P
t	t	t
t	t	f
t	f	t
t	f	f
f	t	t
f	t	f
f	f	t
f	f	f

8 possible interpretations

start by figuring out what set of interpretations make our original sentences true.

We can see that there are **8 possible interpretations**!

If **P** is true in a set of interpretations that make all the original sentences true, then we can conclude that **P** follows from our original sentences.

Checking Interpretations

interpretations *Knowledge base*

S	F	P	S→F	F→P	S
t	t	t	t	t	t
t	t	f	t	f	t
t	f	t	f	t	t
t	f	f	f	t	t
f	t	t	t	t	f
f	t	f	t	f	f
f	f	t	t	t	f
f	f	f	t	t	f

What set of interpretations make the KB true?

All 3 sentences (Knowledge Base) are true when S=t, F=t, and P=t.

What about the sentence P? Is it ok to conclude that it is also true?

Checking Interpretations

interpretations *sentences* *New sentence*

S	F	P	S→F	F→P	S	P
t	t	t	t	t	t	t
t	t	f	t	f	t	f
t	f	t	f	t	t	t
t	f	f	f	t	t	f
f	t	t	t	t	f	t
f	t	f	t	f	f	f
f	f	t	t	t	f	t
f	f	f	t	t	f	f

P is true in that interpretation!

If Knowledge Base is true, in the same set of interpretations that make P true (P is a new sentence we want to test), then it must be ok to conclude that P from the KB.

Adding a variable

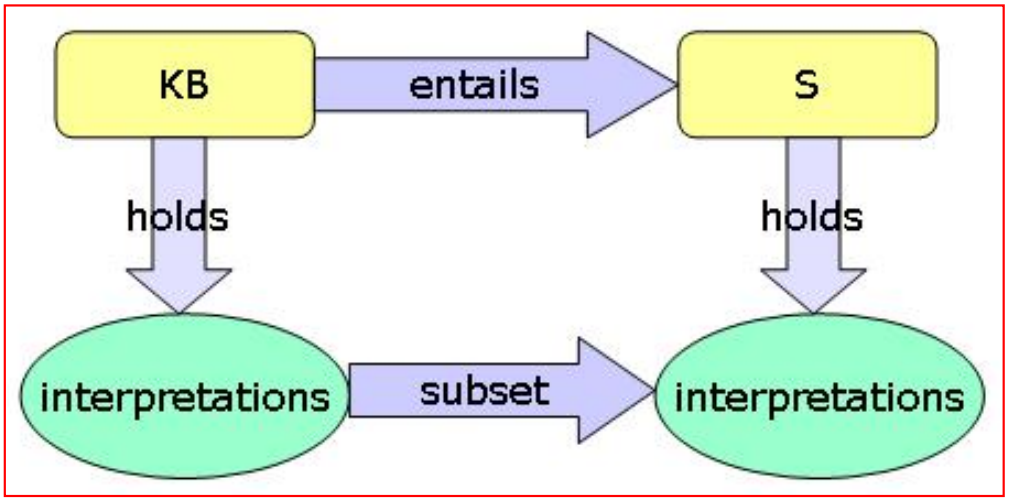
- If Dandelions are blooming (D)

D	S	F	P	S → F	F → P	S	P
t	t	t	t	t	t	t	t
t	t	t	f	t	f	t	f
t	t	f	t	f	t	t	t
t	t	f	f	f	t	t	f
t	f	t	t	t	t	f	t
t	f	t	f	t	f	f	f
t	f	f	t	t	t	f	t
t	f	f	f	t	t	f	f
f	t	t	t	t	t	t	t
f	t	t	f	t	f	t	f
…	…	…	…				

If we add another variable (D), we can see that P is still true in two possible interpretations!

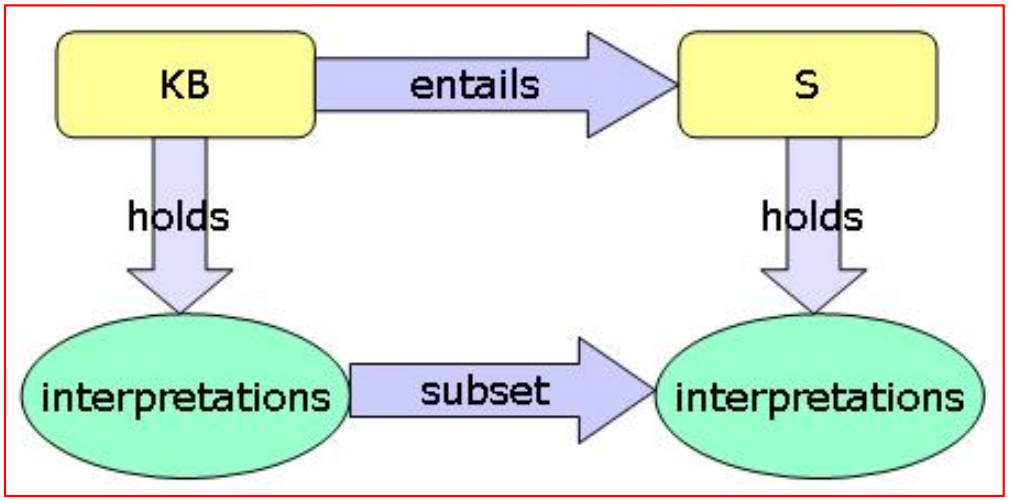
Entailment

•A knowledge base (**KB**) **entails** a sentence **S** iff every interpretation that makes **KB** true also makes **S** true



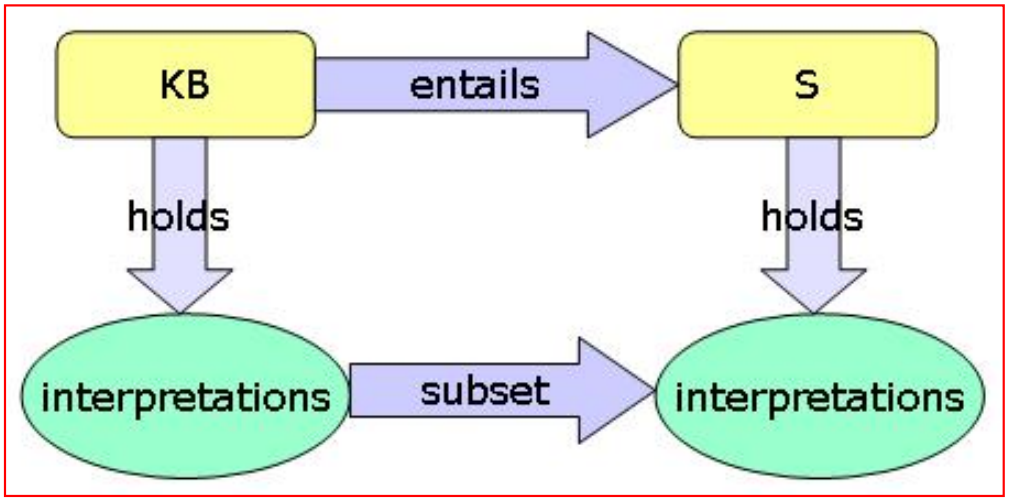
Computing Entailment

- enumerate all interpretations
- select those in which all elements of **KB** are true
- check to see if **S** is true in all of those interpretations



Computing Entailment

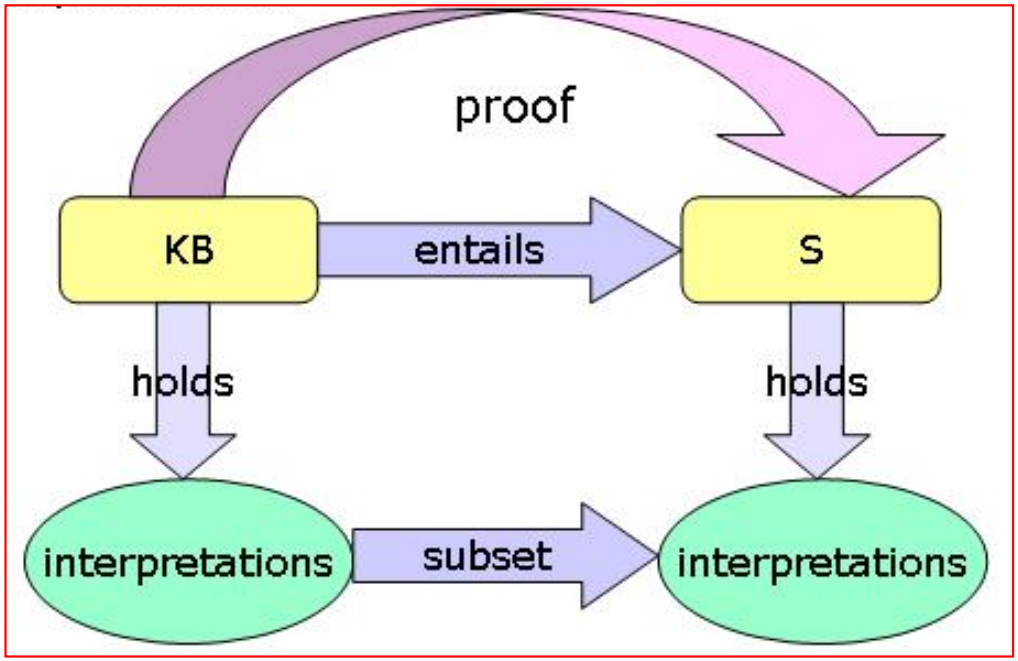
- enumerate all interpretations
- select those in which all elements of **KB** are true
- check to see if **S** is true in all of those interpretations



- This is brute force approach
- Way too many interpretations, in general!!
- e.g. (Given 100 propositional variables, there are 2^100 interpretations to check)

Entailment and Proof

• A **proof** is a way to test whether a **KB** entails a sentence, without enumerating all possible interpretations



Proof

- A **proof** is a sequence of sentences
- First ones are premises (**KB**)
- Then, you can write down on the next line the result of applying an inference rule to previous lines
- When **S** is on a line, you have **proved S** from **KB**
- If inference rules are **sound**, then any **S** you can prove from **KB** is entailed by **KB**
- If inference rules are **complete**, then any **S** that is entailed by **KB** can be proven from **KB**

Natural Deduction

- Some **inference rules**:

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Modus ponens

Affirming method

(first written by Aristotle)

Natural Deduction

- Some inference rules:

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Modus ponens

$$\frac{\alpha \rightarrow \beta \quad \neg\beta}{\neg\alpha}$$

Modus tolens

Denying method

Natural Deduction

- Some inference rules:

$$\frac{\alpha \to \beta \quad \alpha}{\beta}$$

Modus ponens

$$\frac{\alpha \to \beta \quad \neg\beta}{\neg\alpha}$$

Modus tolens

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta}$$

And-Introduction

Natural Deduction

- Some inference rules:

$$\frac{\alpha \to \beta \quad \alpha}{\beta}$$

Modus ponens

$$\frac{\alpha \to \beta \quad \neg\beta}{\neg\alpha}$$

Modus tolens

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta}$$

And-Introduction

$$\frac{\alpha \wedge \beta}{\alpha}$$

And-Elimination

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	**1 And-Elim**

$$\frac{\alpha \wedge \beta}{\alpha}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens
6	Q	1 And-Elim

$$\frac{\alpha \wedge \beta}{\alpha}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens
6	Q	1 And-Elim
7	$Q \wedge R$	5,6 And-Intro

$$\frac{\alpha \quad \beta}{\alpha \wedge \beta}$$

Natural Deduction Example

Prove S

Step	Formula	Derivation
1	$P \wedge Q$	Given
2	$P \rightarrow R$	Given
3	$(Q \wedge R) \rightarrow S$	Given
4	P	1 And-Elim
5	R	4,2 Modus-Ponens
6	Q	1 And-Elim
7	$Q \wedge R$	5,6 And-Intro
8	S	7,3 Modus-Ponens

$$\frac{\alpha \rightarrow \beta \quad \alpha}{\beta}$$

Proof Systems

• There are many natural deduction systems; they are typically "**proof checkers**", **sound** but not complete

Proof Systems

• There are many natural deduction systems; they are typically "**proof checkers**", **sound** but not complete

• **Natural deduction** uses lots of inference rules which introduces a large branching factor in the search for a proof.

Proof Systems

• There are many natural deduction systems; they are typically "**proof checkers**", **sound** but not complete

• **Natural deduction** uses lots of inference rules which introduces a **large branching factor** in the search for a proof.

• In general, you need to do "**proof by cases**" which introduces even more branching.

Prove R

Step	Formula
1	$P \vee Q$
2	$Q \rightarrow R$
3	$P \rightarrow R$

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

- Single inference rule is a sound and complete proof system

- Requires all sentences to be converted to conjunctive normal form

159.302 LECTURE 4

Propositional Logic
Conjunctive Normal Form (CNF)

Source: MIT OpenCourseWare

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and \neg C)$ are literals

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and \neg C)$ are literals, each of which is a variable or the negation of a variable.

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and \neg C)$ are literals, each of which is a variable or the negation of a variable.

- Each clause is a requirement that must be satisfied and can be satisfied in multiple ways

Conjunctive Normal Form

CNF Formulas:

$$(A \vee B \vee \neg C) \wedge (B \vee D) \wedge (\neg A) \wedge (B \vee C)$$

$(A \vee B \vee \neg C)$ is a clause, which is a disjunction of literals

$A, B, and \neg C)$ are literals, each of which is a variable or the negation of a variable.

- Each clause is a requirement that must be satisfied and can be satisfied in multiple ways

- Every sentence in propositional logic can be written in CNF

Converting to CNF

1. Eliminate arrows using definitions

Converting to CNF

1. Eliminate arrows using definitions
2. Drive in negations using De Morgan's Laws

$$\neg(\phi \vee \varphi) \equiv \neg\phi \wedge \neg\varphi)$$

$$\neg(\phi \wedge \varphi) \equiv \neg\phi \vee \neg\varphi)$$

Converting to CNF

1. Eliminate arrows using definitions

2. Drive in negations using De Morgan's Laws

$$\neg(\phi \vee \varphi) \equiv \neg\phi \wedge \neg\varphi)$$

$$\neg(\phi \wedge \varphi) \equiv \neg\phi \vee \neg\varphi)$$

3. Distribute **or** over **and**

$$(A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

Converting to CNF

1. Eliminate arrows using definitions

2. Drive in negations using De Morgan's Laws

$$\neg(\phi \vee \varphi) \equiv \neg\phi \wedge \neg\varphi)$$

$$\neg(\phi \wedge \varphi) \equiv \neg\phi \vee \neg\varphi)$$

3. Distribute **or** over **and**

$$(A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$$

4. Every sentence can be converted to CNF, but it may **grow exponentially in size**

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

1. Eliminate arrows using definitions

$$\neg(A \vee B) \vee (\neg C \vee D)$$

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

1. Eliminate arrows using definitions

$$\neg(A \vee B) \vee (\neg C \vee D)$$

2. Drive in negations using De Morgan's Laws

$$(\neg A \wedge \neg B) \vee (\neg C \vee D)$$

CNF Conversion Example

$$(A \vee B) \rightarrow (C \rightarrow D)$$

1. Eliminate arrows using definitions

$$\neg(A \vee B) \vee (\neg C \vee D)$$

2. Drive in negations using De Morgan's Laws

$$(\neg A \wedge \neg B) \vee (\neg C \vee D)$$

3. Distribute **or** over **and**

$$(\neg A \vee \neg C \vee D) \wedge (\neg B \vee \neg C \vee D)$$

159.302 LECTURE 5

Propositional Resolution

Source: MIT OpenCourseWare

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise
- Negate the desired conclusion (converted to CNF)

This is proof by contradiction!

- Assert the sentence that you're trying to prove is false
- then try to derive a contradiction.

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise
- Negate the desired conclusion (converted to CNF)
- Apply resolution rule until either
  - Derive false (a contradiction)
  - Can’t apply anymore

Propositional Resolution

Resolution rule:

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

Resolution refutation:

- Convert all sentences to CNF, then write each clause down as a premise
- Negate the desired conclusion (converted to CNF)
- Apply resolution rule until either
  - Derive false (a contradiction)
  - Can't apply anymore

Resolution refutation is sound and complete

- If we derive a contradiction, then the conclusion follows from the axioms
- If we can't apply anymore, then the conclusion cannot be proven from the axioms

Propositional Resolution

Prove R

Step	Formula
1	$P \vee Q$
2	$P \rightarrow R$
3	$Q \rightarrow R$

Step	Formula	Derivation
1	$P \vee Q$	Given
2	$\neg P \vee R$	Given
3	$\neg Q \vee R$	Given
4	$\neg R$	**Negated conclusion**
5	$Q \vee R$	**1,2**
6	$\neg P$	**2,4**
7	$\neg Q$	**3,4**
8	R	**5,7**
9	▪	**4,8**

Proof steps

Small box indicates empty clause, meaning false

Propositional Resolution

Prove R

Step	Formula
1	$P \vee Q$
2	$P \rightarrow R$
3	$Q \rightarrow R$

$$\frac{\begin{array}{c} false \vee R \\ \neg R \vee false \end{array}}{false \vee false}$$

Step	Formula	Derivation
1	$P \vee Q$	Given
2	$\neg P \vee R$	Given
3	$\neg Q \vee R$	Given
4	$\neg R$	**Negated conclusion**
5	$Q \vee R$	**1,2**
6	$\neg P$	**2,4**
7	$\neg Q$	**3,4**
8	R	**5,7**
9	▪	**4,8**

Note: disjunction is commutative.

Propositional Resolution

Prove R

Step	Formula
1	$P \vee Q$
2	$P \rightarrow R$
3	$Q \rightarrow R$

Step	Formula	Derivation
1	$P \vee Q$	Given
2	$\neg P \vee R$	Given
3	$\neg Q \vee R$	Given
4	$\neg R$	**Negated conclusion**
5	$Q \vee R$	1,2
6	$\neg P$	2,4
7	$\neg Q$	3,4
8	R	5,7
9	▪	4,8

unnecessary

$$\frac{false \vee R \qquad \neg R \vee false}{false \vee false}$$

The Power of False

Prove Z

Step	Formula
1	P
2	$\neg P$

Step	Formula	Derivation
1	P	Given
2	$\neg P$	Given
3	$\neg Z$	**Negated conclusion**
4	▪	1,2

Note that $(P \wedge \neg P) \rightarrow Z$ is valid

• Any conclusion follows from a contradiction – and so strict logic systems are very brittle.

Example Problem

Prove R

Step	Formula
1	$(P \rightarrow Q) \rightarrow Q$
2	$(P \rightarrow P) \rightarrow R$
3	$(R \rightarrow S) \rightarrow \neg(S \rightarrow Q)$

Convert to CNF

- ¬(¬ P ∨ Q) ∨ Q
- (P ∧ ¬ Q) ∨ Q
- (P ∨ Q) ∧ (¬ Q ∨ Q)
- (P ∨ Q)

- ¬(¬ P ∨ P) ∨ R
- (P ∧ ¬ P) ∨ R
- (P ∨ R) ∧ (¬ P ∨ R)

- ¬(¬ R ∨ S) ∨ ¬ (¬ S ∨ Q)
- (R ∧ ¬ S) ∨ (S ∧ ¬ Q)
- (R ∨ S) ∧ (¬ S ∨ S) ∧ (R ∨ ¬ Q) ∧ (¬ S ∨ ¬ Q)
- (R ∨ S) ∧ (R ∨ ¬ Q) ∧ (¬ S ∨ ¬ Q)

Example Problem

Prove R

Step	Formula
1	$(P \rightarrow Q) \rightarrow Q$
2	$(P \rightarrow P) \rightarrow R$
3	$(R \rightarrow S) \rightarrow \neg(S \rightarrow Q)$

Step		
1	$P \vee Q$	
2	$P \vee R$	
3	$\neg P \vee R$	
4	$R \vee S$	
5	$R \vee \neg Q$	
6	$\neg S \vee \neg Q$	
7	$\neg R$	**Neg**

Example Problem

Prove R

Step	Formula
1	$(P \to Q) \to Q$
2	$(P \to P) \to R$
3	$(R \to S) \to \neg(S \to Q)$

Step		
1	$P \vee Q$	
2	$P \vee R$	
3	$\neg P \vee R$	
4	$R \vee S$	
5	$R \vee \neg Q$	
6	$\neg S \vee \neg Q$	
7	$\neg R$	Neg
8	S	4,7
9	$\neg Q$	6,8
10	P	1,9
11	R	3,10
12	■	7,11

Proof Strategies

Unit Preference: prefer a resolution step involving a unit clause (clause with one literal).

- Produces a shorter clause – which is good since we are trying to produce a zero-length clause, that is, a contradiction.

Set of support: choose a resolution involving the negated goal or any clause derived from the negated goal.

- We're trying to produce a contradiction that follows from the negated goal, so these are "relevant" clauses
- If a contradiction exists, one can find one using the set-of-support strategy.

References

MIT Opencourseware

Logic for Computer Science and Artificial Intelligence

- http://onlinelibrary.wiley.com/book/10.1002/9781118604182

159302 – Artificial Intelligence
Tutorial - Propositional Logic

The following notations are used in the questions that follow:

- **^** (caret) for and,
- **v** (lower case v) for or,
- **~** for not,
- **->** (hyphen, >) for implication, and
- **<->** (<, hyphen, >) for biconditionals.
- Propositional variables are upper case letters (and possibly numbers).
- The symbols true and false are written out in lower case.

I. Precedence

Give a fully parenthesized version of each of these expressions. Each of your answers needs to follow the definitions legal sentences in propositional logic. Preserve the original order of the variable names in your parenthesized expression.

1. P v ~ Q -> R
2. P ^ Q v R
3. P -> Q <-> R -> S
4. ~ R v S -> Q ^ R

II. Translating Sentences

If we let the propositional variables S, Z, R, and H have the following intended meanings:

- S = the sun shines
- Z = we go to the zoo
- R = it rains
- H = we stay at home

Give, for each of the following English sentences, a propositional logic sentence with the same meaning:

1. If the sun shines then we go to the zoo.
2. It rains or the sun shines.
3. If we do not go to the zoo then we stay at home.
4. If we do not go to the zoo then it rains.
5. If we do not stay at home then the sun shines.
6. If the sun shines or we do not stay at home then we go to the zoo.
7. If the sun does not shine then we do not go to the zoo.

III. Simplifications

Use the semantics rules to simplify the following formulas as much as possible.

1. P v true
2. P ^ true
3. P v false
4. P ^ false
5. (P ^ false) v Q
6. (P ^ Q) v true
7. P ^ (Q v true)
8. P ^ (Q v false)

Source: MITOpenCourseware

IV. CNF

For each of the sentences below, derive the conjunctive normal form and enter it as a sequence of parenthesized disjunctions separated by and's.

1. (A ^ ~B) -> C
2. (A -> B) -> C
3. A -> (B -> C)
4. ((A ^ B) v C) -> D
5. ((A ^ B) v C) -> (D ^ E)

V. Propositional Resolution Refutation

Given the following propositional clauses (axioms), derive a contradiction using the resolution rule:

Prove ~P.

1 ~P v Q v S
2 ~S
3 ~Q v X
4 ~X

VI. Resolution Theorem Proving

Adams, Brown and Clark

There are three suspects for a murder: Adams, Brown, and Clark. Adams says "I didn't do it. The victim was an old acquaintance of Brown's. But Clark hated him." Brown states "I didn't do it. I didn't know the guy. Besides I was out of town all week." Clark says "I didn't do it. I saw both Adams and Brown in town around the victim that day; one of them must have done it." We know that exactly one of the suspects is guilty. Assume that the two innocent men are telling the truth, but that the guilty man might not be.

Let the propositional variables have the following definitions:
A = Adams is innocent
B = Brown is innocent
C = Clark is innocent
X = Brown knew the victim
Y = Brown was in town
Z = Adams was out of town
W = Clark liked the victim
P = Brown was with the victim
Q = Adams was with the victim
F = Brown was a friend of the victim
S = Clark knew the victim

The key to the solution is to make the suspect's statements conditional on their innocence. The following rules/axioms are sufficient.

1. If Adams was innocent, then Brown was the victim's friend and Clark did not like the victim, just as Adams said.
2. If Brown was innocent, he wasn't in town and he doesn't know the victim.

3. If Clark was innocent, then Adams was with the victim and Brown was with the victim.
4. General Knowledge:
   If Brown was with the victim, then Brown was in town.
   If Brown was a friend of the victim, the Brown knew the victim.
   If Clark liked the victim, then Clark knew the victim.
5. Now we assert that only one of A, B, and C is the guilty person:
   A v B
   A v C
   B v C

159302 – Artificial Intelligence
Tutorial #10 - Propositional Logic

The following notations are used in the questions that follow:

- **^** (caret) for and,
- **v** (lower case v) for or,
- **~** for not,
- **->** (hyphen, >) for implication, and
- **<->** (<, hyphen, >) for biconditionals.
- Propositional variables are upper case letters (and possibly numbers).
- The symbols true and false are written out in lower case.

¬ ∧ ∨ → ↔

I. Precedence

Give a fully parenthesized version of each of these expressions. Each of your answers needs to follow the definitions legal sentences in propositional logic. Preserve the original order of the variable names in your parenthesized expression.

1. P v ~ Q -> R
2. P ^ Q v R
3. P -> Q <-> R -> S
4. ~ R v S -> Q ^ R

II. Translating Sentences

If we let the propositional variables S, Z, R, and H have the following intended meanings:

- S = the sun shines
- Z = we go to the zoo
- R = it rains
- H = we stay at home

Give, for each of the following English sentences, a propositional logic sentence with the same meaning:

1. If the sun shines then we go to the zoo.
2. It rains or the sun shines.
3. If we do not go to the zoo then we stay at home.
4. If we do not go to the zoo then it rains.
5. If we do not stay at home then the sun shines.
6. If the sun shines or we do not stay at home then we go to the zoo.
7. If the sun does not shine then we do not go to the zoo.

III. Simplifications

Use the semantics rules to simplify the following formulas as much as possible.

1. P v true — true
2. P ^ true — P
3. P v false — P
4. P ^ false — false
5. (P ^ false) v Q — Q
6. (P ^ Q) v true — true
7. P ^ (Q v true) — P
8. P ^ (Q v false) — P ^ Q

II

1. $S \rightarrow Z$
2. $(R \vee S)$
3. $\neg Z \rightarrow H$
4. $\neg Z \rightarrow R$
5. $\neg H \rightarrow S$
6. $(S \vee \neg H) \rightarrow Z$
7. $\neg S \rightarrow \neg Z$

IV CNF

1. $(A \wedge \neg B) \rightarrow C \equiv \neg(A \wedge \neg B) \vee C$

$\equiv (\neg A \vee B) \vee C$

$\equiv (\neg A \vee B \vee C)$ ✓

2. $(A \rightarrow B) \rightarrow C \equiv (\neg A \vee B) \rightarrow C$

$\equiv \neg(\neg A \vee B) \vee C$

$\equiv (A \wedge \neg B) \vee C$

$\equiv (A \vee C) \wedge (\neg B \vee C)$ ✓

3. $A \rightarrow (B \rightarrow C) \equiv \neg A \vee (B \rightarrow C)$

$\equiv \neg A \vee (\neg B \vee C)$

$= (\neg A \vee \neg B \vee C)$ ✓

IV CNF

4. $((A \wedge B) \vee C) \rightarrow D \equiv \neg((A \wedge B) \vee C) \vee D$

$\equiv ((\neg A \vee \neg B) \wedge \neg C) \vee D$

another

$\equiv (\neg A \vee \neg B \vee D) \wedge (\neg C \vee D)$

5. $((A \wedge B) \vee C) \rightarrow (D \wedge E) \equiv$

$\equiv \neg((A \wedge B) \vee C) \vee (D \wedge E)$

$\equiv ((\neg A \vee \neg B) \wedge \neg C) \vee (D \wedge E)$

$\equiv (\neg A \vee \neg B \vee D) \wedge (\neg C \vee D)$

$\wedge (\neg A \vee \neg B \vee E) \wedge (\neg C \vee E)$ ✓

$\equiv$

五. Propositional Resolution Refutation

1. $\neg P \vee Q \vee S$
2. $\neg S$
3. $\neg Q \vee X$
4. $\neg X$
5. P —— negated conclusion

$$\frac{\alpha \vee \beta \qquad \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

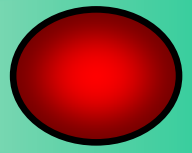
SOLUTION #1

6.	$Q \vee S$	1, 5
7.	$\neg P \vee Q$	1, 2
8.	$\neg Q$	3, 4
9.	$\neg P$	7, 8
10	■	5, 9

SOLUTION #2

6.	$\neg P \vee Q$	(1, 2)
7.	$\neg Q$	(3, 4)
8.	$\neg P$	(6, 7)
9.	■	(5, 8)

159.302 Tutorial



Propositional Logic

Tutorial Problem:

- **Use propositional resolution to solve a crime problem.**

Problem Definition

Solving a crime

- There are three suspects for a murder: Adams, Brown, and Clark.

Adams says "I didn't do it. The victim was an old acquaintance of Brown's. But Clark hated him."

Brown states "I didn't do it. I didn't know the guy. Besides I was out of town all week."

Clark says "I didn't do it. I saw both Adams and Brown in town around the victim that day; one of them must have done it."

- We know that exactly one of the suspects is guilty.
- Assume that the two innocent men are telling the truth, but that the guilty man might not be.

Knowledge Base

- The key to the solution is to make the suspect's statements conditional on their innocence. The following rules/axioms are sufficient.

If Adams was innocent, **then** Brown was the victim's friend and Clark did not like the victim, just as Adams said.

If Brown was innocent, **then** he wasn't in town and he doesn't know the victim.

If Clark was innocent, **then** Adams was with the victim and Brown was with the victim.

- **GENERAL KNOWLEDGE**

If Brown was with the victim, **then** Brown was in town.
If Brown was a friend of the victim, **then** Brown knew the victim.
If Clark liked the victim, **then** Clark knew the victim.

- **ASSERTION**

Now we assert that only one of A, B, and C is the guilty person:
A v B
A v C
B v C

One of the suspects is guilty

one of the suspects is guilty

EXPRESSION 1 $(\sim A \wedge B \wedge C) \vee (A \wedge \sim B \wedge C) \vee (A \wedge B \wedge \sim C)$

EXPRESSION 2 $(A \vee B) \wedge (A \vee C) \wedge (B \vee C)$

	A	B	C	EXPRESSION 1	EXPRESSION 2	
1	0	0	0	FALSE	FALSE	all are guilty
2	0	0	1	FALSE	FALSE	
3	0	1	0	FALSE	FALSE	
4	0	1	1	TRUE	TRUE	
5	1	0	0	FALSE	FALSE	
6	1	0	1	TRUE	TRUE	
7	1	1	0	TRUE	TRUE	
8	1	1	1	FALSE	TRUE	all are innocent

case 8 is assumed to be discarded in the crime case, as two are assumed to be innocent, not all of them.

Variables

A = Adams is innocent
B = Brown is innocent
C = Clark is innocent
X = Brown knew the victim
Y = Brown was in town
Z = Adams was out of town
W = Clark liked the victim
P = Brown was with the victim
Q = Adams was with the victim
F = Brown was a friend of the victim
S = Clark knew the victim

Axioms

A = Adams is innocent
B = Brown is innocent
C = Clark is innocent
X = Brown knew the victim
Y = Brown was in town
Z = Adams was out of town
W = Clark liked the victim
P = Brown was with the victim
Q = Adams was with the victim
F = Brown was a friend of the victim
S = Clark knew the victim

Translating the sentences, we have:

If Adams was innocent, **then** Brown was the victim's friend and Clark did not like the victim, just as Adams said.	$A \rightarrow (F \wedge \sim W)$
If Brown was innocent, **then** he wasn't in town and he doesn't know the victim.	$B \rightarrow (\sim Y \wedge \sim X)$
If Clark was innocent, **then** Adams was with the victim and Brown was with the victim.	$C \rightarrow (Q \wedge P)$
If Brown was with the victim, **then** Brown was in town.	$P \rightarrow Y$
If Brown was a friend of the victim, **then** Brown knew the victim.	$F \rightarrow X$
If Clark liked the victim, **then** Clark knew the victim.	$W \rightarrow S$
A v B A v C B v C	$A \vee B$ $A \vee C$ $B \vee C$

Next, we shall expand the sentences…

Axioms

CNF of the sentences:

	Eliminating arrows	Distributing OR over AND
$A \rightarrow (F \wedge \sim W)$	$\sim A \vee (F \wedge \sim W)$	$(\sim A \vee F) \wedge (\sim A \vee \sim W)$
$B \rightarrow (\sim Y \wedge \sim X)$	$\sim B \vee (\sim Y \wedge \sim X)$	$(\sim B \vee \sim Y) \wedge (\sim B \vee \sim X)$
$C \rightarrow (Q \wedge P)$	$\sim C \vee (Q \wedge P)$	$(\sim C \vee Q) \wedge (\sim C \vee P)$
$P \rightarrow Y$	$\sim P \vee Y$	$\sim P \vee Y$
$F \rightarrow X$	$\sim F \vee X$	$\sim F \vee X$
$W \rightarrow S$	$\sim W \vee S$	$\sim W \vee S$
$A \vee B$		$A \vee B$
$A \vee C$		$A \vee C$
$B \vee C$		$B \vee C$

Axioms

Splitting the sentences, we have:

(~A ∨ F) ∧ (~A ∨ ~W)	(~A ∨ F)
	(~A ∨ ~W)
(~B ∨ ~Y) ∧ (~B ∨ ~X)	(~B ∨ ~Y)
	(~B ∨ ~X)
(~C ∨ Q) ∧ (~C ∨ P)	(~C ∨ Q)
	(~C ∨ P)
~P ∨ Y	~P ∨ Y
~F ∨ X	~F ∨ X
~W ∨ S	~W ∨ S
A ∨ B	A ∨ B
A ∨ C	A ∨ C
B ∨ C	B ∨ C

Axioms

Pick one of the suspects, then attempt to prove that he is not innocent (e.g. Brown)

- (~A v F)
- (~A v ~W)
- (~B v ~Y)
- (~B v ~X)
- (~C v Q)
- (~C v P)
- ~P v Y
- ~F v X
- ~W v S
- A v B
- A v C
- B v C
- **B**

Brown is not innocent ≡ ~B

First, we negate the conclusion, then we add it to our axioms…

A = Adams is innocent
B = Brown is innocent
C = Clark is innocent
X = Brown knew the victim
Y = Brown was in town
Z = Adams was out of town
W = Clark liked the victim
P = Brown was with the victim
Q = Adams was with the victim
F = Brown was a friend of the victim
S = Clark knew the victim

Axioms

Pick one of the suspects, then attempt to prove that he is not innocent (e.g. Brown)

1	(~A v F)
2	(~A v ~W)
3	(~B v ~Y)
4	(~B v ~X)
5	(~C v Q)
6	(~C v P)
7	~P v Y
8	~F v X
9	~W v S
10	A v B
11	A v C
12	B v C
13	B

Proof Steps:

14	~X	13,4

Axioms

Pick one of the suspects, then attempt to prove that he is not innocent (e.g. Brown)

1	(~A v F)
2	(~A v ~W)
3	(~B v ~Y)
4	(~B v ~X)
5	(~C v Q)
6	(~C v P)
7	~P v Y
8	~F v X
9	~W v S
10	A v B
11	A v C
12	B v C
13	B

Proof Steps:

14	~X	13,4
15	~F	14,8
16	~A	15,1
17	C	16,11
18	P	17,6
19	Y	18,7
20	~B	19,3
21	■	20,13

Therefore, Brown is guilty!

You can try to prove that Adam or Clark is guilty, but the proof system will not lead to a contradiction.